INFORMATIQUE
TECHNIQUE

Expert

Le Machine Learning avec Python

De la théorie à la pratique

Préface de **Patrick Albert**
Cofondateur d'ILOG et du HUB France IA

En téléchargement

code source

+ QUIZ

Version en ligne **OFFERTE !** pendant 1 an

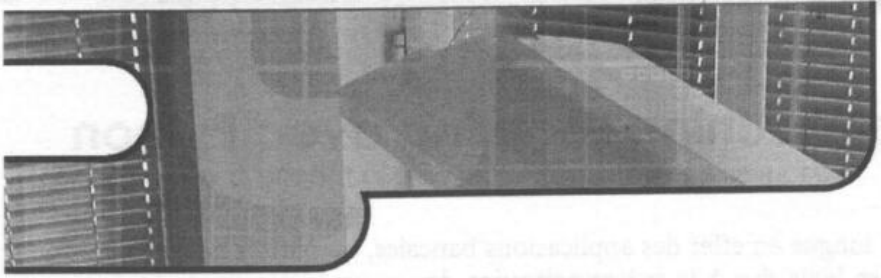
Madjid KHICHANE
PhD en Intelligence Artificielle

eni

Expert IT

Le Machine Learning avec Python

De la théorie à la pratique



Préface

Le nombre de livres sur l'Intelligence artificielle tend à croître exponentiellement ; il faut donc faire preuve d'une certaine audace pour en proposer un nouveau. Mais surtout il faut faire œuvre utile, et savoir de quoi l'on parle : maîtriser la théorie et s'être confronté à la pratique, à la mise en œuvre d'applications utiles et utilisées. L'auteur de ce livre Madjid Khichane est de ceux-là ; il allie une connaissance intime des « principes » acquise pendant trois longues années de recherche, lors de son doctorat, à une confrontation à la réalité lors un grand nombre d'expériences concrètes. Pour l'avoir encadré de près durant son doctorat, je peux assurer que clarté de pensée et sens pratique sont au rendez-vous ! Et c'est bien l'alchimie qui se produit dans ce livre consacré à l'initiation des novices, pour les guider vers l'acquisition d'une compréhension large du domaine, condition préalable à une inévitable spécialisation dans tel ou tel type d'algorithmes. Un algorithme d'intelligence artificielle est un monde en soi ; sans recul, et sans accompagnement, on s'y perd à coup sûr.

Pour développer et même simplement utiliser correctement une méthode d'intelligence artificielle, il est en effet nécessaire d'allier plusieurs types de connaissances, conceptuelles et concrètes, tant liées à la programmation qu'à l'utilisation des modèles et calculs statistiques, simples à première vue, mais constellés de pièges pour le naïf.

La liste est longue en effet des applications bancales, ou parfois néfastes, minées par des biais dus à la méconnaissance des caractéristiques d'une base d'exemples réellement utilisable. Alors que le domaine atteint une première étape de maturité, il est nécessaire de conjoindre vue d'ensemble et pratique sérieuse de la programmation, qui seule permet de paramétrer ou le plus souvent d'adapter tel ou tel algorithme. À l'heure du déploiement massif d'applications très concrètes, le temps n'est plus au bricolage.

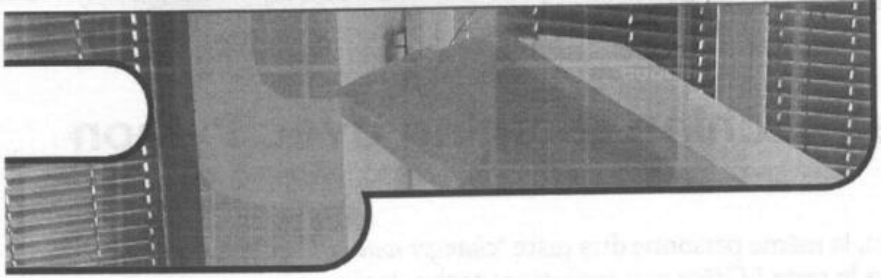
Ce livre, donc, comble un vide et relève un défi, celui d'accueillir un débutant, de guider son autoformation à l'intelligence artificielle, à ses concepts, à ses méthodes et à ses algorithmes. En particulier, la mise en œuvre d'exemples est nécessaire pour comprendre réellement certains concepts difficiles, et seule la pratique de la programmation permet de dépasser le stade trompeur de l'intuition, de « l'impression d'avoir compris ». Cela n'est en effet qu'en allant jusqu'au bout de l'implémentation d'un cas concret, en Python ou en R, suffisamment « nettoyé » – préparé – pour être praticable dans le cadre simplifié d'un exercice, que l'on découvrira les pièges bien réels posés par la mise en œuvre. L'informatique n'est pas la Mathématique ! En informatique, la théorie et la pratique sont en partie liées par les lignes de codes. Ainsi préparé, on pourra parcourir avec profit les « grands algorithmes » issus de près de 70 ans de recherche et d'applications en intelligence artificielle.

Une place de choix bien sûr est laissée au Machine Learning et à son dernier avatar le Deep Learning qui a fait sortir la Data Science des laboratoires où depuis des décennies les chercheurs s'épuisaient sans arriver à la rendre vraiment utilisable. On peut maintenant déployer ces méthodes qui ont propulsé le domaine à la face du monde et qui sont le puissant moteur du succès des Google, Amazon ou Facebook. Le livre donne les clefs pour en acquérir le « feeling », apprécier les théories auxquelles elles sont adossées, pour se « faire la main » en programmant, et pour apprendre à créer ou sélectionner, puis « nettoyer » soigneusement les masses de données nécessaires à l'apprentissage automatique. Car il faut être très vigilant, les erreurs de biais rendent, sans que l'on s'en rende compte, les modèles inutilisables ou pires, dangereux. Enfin, il faudra savoir *interpréter* les résultats, un problème parfois plus difficile qu'il n'y paraît.

© Editions ENI - All rights reserved

En synthèse, ce livre propose une sorte de « tout en un » qui ouvre un boulevard à celles et ceux qui veulent démarrer ou élargir leurs connaissances en intelligence artificielle, un « tout en un » qui regroupe et présente de façon homogène les connaissances nécessaires et fait parcourir les chemins, sentiers et boulevards qu'il faut vraiment arpenter afin de devenir un digne praticien du domaine !

Patrick Albert –
Cofondateur d'ILOG ; cofondateur du HUB France IA



Introduction générale

L'intelligence artificielle est un sujet aussi vieux que l'informatique, mais qui rencontre un succès fulgurant ces dernières années.

Aujourd'hui, l'intelligence artificielle interprète des radiographies mieux que les médecins, elle traduit des textes en différentes langues, elle reconnaît les personnes et les objets sur les images, elle joue et gagne à des jeux complexes comme le Go ou le jeu d'échecs, elle conduit des voitures, elle compose des mélodies, elle peint des tableaux, etc. Cette intelligence artificielle est de plus en plus présente dans notre vie de tous les jours.

Les technologies telles que le traitement automatique du langage naturel, la reconnaissance vocale, la reconnaissance d'objets dans les images, l'analyse des sentiments exprimés via un simple message ou exprimés sur un visage dans une photo sont toutes des technologies qui tendent à humaniser l'interface homme-machine, voire à doter la machine d'un caractère plus au moins semblable au caractère humain.

Lors de l'apparition d'Internet et des moteurs de recherche, dont Google principalement, les utilisateurs ont dû adapter leur manière de s'exprimer pour retrouver les informations qui les intéressent. À l'époque, les utilisateurs saisissaient des requêtes telles que : "*comment changer une roue de voiture Peugeot 306* ?"

Aujourd'hui, la même personne dira juste "*changer roue 306*" et le moteur de recherche fera le reste ! Grâce aux évolutions technologiques fulgurantes de ces dernières années, l'utilisateur est invité à utiliser son propre langage naturel pour interagir avec la machine, c'est-à-dire que c'est la machine qui s'adapte à l'homme et non pas l'inverse.

Le résultat de la démocratisation de l'informatique et l'adoption massive d'Internet et des plateformes de Big Data ont permis l'émergence de cas d'usage de l'intelligence artificielle dans quasiment tous les domaines.

Vu leur nombre, l'énumération de toutes les applications concrètes de l'intelligence artificielle est impossible. Cependant, nous pouvons lister quelques cas d'usage très répandus :

- La maintenance prédictive. Elle consiste à prédire avec une probabilité la date à laquelle un composant d'une machine va tomber en panne ! Par exemple, le centre de réparation des moteurs d'avions d'une compagnie aérienne sera capable de prédire la date à laquelle un avion tombera en panne. Cela va permettre d'anticiper les réparations et d'éviter les retards de vols. Dans ce cas, la maintenance prédictive est moins coûteuse que la maintenance corrective car il ne sera plus nécessaire de procéder à un changement périodique et systématique des pièces des moteurs d'avions. La maintenance prédictive peut être appliquée partout où des machines sont mises en œuvre pour réaliser des tâches (dans les usines, les centrales nucléaires, les machines à café, les ordinateurs, etc.).
- La détection de fraudes. Il peut s'agir de détecter des transactions bancaires frauduleuses, des situations fiscales irrégulières, des intrusions aux systèmes d'information ou encore de fraudes à l'assurance.
- Les moteurs de recommandation. Sur quasiment toutes les plateformes de partage multimédia et tous les sites marchands, nous retrouvons des systèmes de recommandation proposant du contenu ciblé en fonction des profils des visiteurs. Le but est d'accroître et d'optimiser les ventes. Par exemple :
  - L'Up-Selling et le Cross-Selling pour augmenter les marges et le nombre de produits vendus.
  - La recommandation de contenu, comme sur les plateformes YouTube et Netflix, en proposant des vidéos susceptibles d'intéresser les visiteurs.

© Editions ENI - All rights reserved

- L'optimisation combinatoire avec les algorithmes d'intelligence artificielle. Les algorithmes tels que les algorithmes génétiques ou les algorithmes d'optimisation par colonie de fourmis sont souvent utilisés pour optimiser des tâches impossibles à réaliser avec les algorithmes classiques basés sur des calculs de type force brute. Parmi les exemples les plus courants, nous pouvons citer :
  - L'optimisation des tournées de véhicules. Par exemple, une entreprise qui fait intervenir tous les jours des agents sur le terrain pour effectuer des livraisons et/ou des réparations aimerait minimiser les distances parcourues par ses agents.
  - L'optimisation des plannings des infirmiers et des médecins dans les hôpitaux.
  - L'optimisation des chaînes de fabrication dans les usines.
  - L'optimisation du placement des assets. Par exemple, les entreprises de télécommunication cherchent à utiliser un minimum d'antennes tout en couvrant un maximum d'espace géographique et en minimisant les interférences entre toutes ces antennes.
- Les services cognitifs. L'arrivée des plateformes cloud et Big Data a été une aubaine pour le développement de multiples services cognitifs utilisables dans différents domaines et secteurs d'activité. Parmi les services cognitifs les plus répandus :
  - La vision par ordinateur telle que la détection de formes dans les images, la détection et la reconnaissance des visages.
  - La reconversion de la voix vers le texte, la synthèse et l'identification vocale.
  - Le traitement automatique du langage naturel qui est utilisé pour comprendre et générer du texte.
- Les assistants virtuels. Les chatbots profitent grandement du développement des services cognitifs. Ces assistants virtuels bouleversent de façon significative la gestion de la relation client.

Les cas d'usage de l'intelligence artificielle ne cessent d'augmenter et il n'y a quasiment pas un jour qui passe sans que de nouvelles applications voient le jour, surtout avec le nombre croissant d'objets connectés.

L'une des questions des plus intéressantes qu'on peut se poser est de savoir quel serait le rôle que jouera l'intelligence artificielle dans l'économie de demain. Cette question est primordiale à la fois pour les sociétés privées et pour les gouvernements. Les entreprises privées voient en l'intelligence artificielle une technologie d'avenir et un levier qui va leur permettre une forte croissance économique. Ces entreprises adhèrent de plus en plus à l'idée d'« employé augmenté ». Un employé augmenté est un employé assisté par des algorithmes et des machines « intelligentes » pour réaliser ses tâches de tous les jours. Ces tâches peuvent être répétitives et sans valeur ajoutée pour l'être humain, voire dangereuses dans certains cas. Par conséquent, le travail de l'employé sera plus précis, plus sécurisé, plus rapide et plus rentable pour l'entreprise. Cette rentabilité découle directement de l'augmentation de la satisfaction client et de la croissance des ventes. De leur côté, les gouvernements, particulièrement les gouvernements européens, voient l'intelligence artificielle comme un vrai dilemme. D'un côté, ils doivent investir dans cette discipline pour rester dans la course économique mondiale et ne pas être dépendants des multinationales américaines et asiatiques ; de l'autre côté, ils doivent se préparer à faire face aux risques de suppression massive d'emplois et aux menaces qui découlent des usages de l'intelligence artificielle. Nous constatons donc que l'intelligence artificielle représente un fort potentiel de croissance économique pour les entreprises tout en étant un vrai challenge économique et social pour les gouvernements.

Au sein de la société, l'opinion publique est partagée entre enthousiasme et angoisse vis-à-vis des applications associées à l'intelligence artificielle. De l'enthousiasme, comme dans le secteur médical où des algorithmes prédisent suffisamment à l'avance des maladies afin qu'elles soient traitées à temps. Mais également de l'angoisse, par rapport aux impacts potentiellement négatifs liés d'un côté à la perspective de la perte massive d'emplois et de l'autre aux abus des usages, comme l'utilisation des données privées à des fins non éthiques et malicieuses. Effectivement, lorsque nous regardons de plus près les usages faits de l'intelligence artificielle, il n'est pas rare de trouver des cas où les données privées des citoyens sont utilisées de manière tout à fait illégale. Un des exemples les plus frappants et relativement récent est celui de la société Cambridge Analytica qui est soupçonnée d'avoir récolté les données de millions d'utilisateurs de Facebook à leur insu, et d'avoir utilisé ces données pour manipuler l'opinion publique.

© Editions ENI - All rights reserved

En dehors de ces usages malicieux, la préoccupation sociale principale qui découle de l'intelligence artificielle est la place que celle-ci va occuper dans le futur du marché de travail. D'un côté, beaucoup d'experts ont déjà tiré la sonnette d'alarme relative à la suppression en masse des emplois qui guette le développement sans limites de cette intelligence artificielle. De l'autre côté, plusieurs études concluent qu'effectivement l'intelligence artificielle va supprimer des emplois, mais qu'elle va en même temps en créer beaucoup d'autres, voire plus que ce qu'elle va supprimer.

D'après plusieurs études, notre dépendance à l'intelligence artificielle dans un avenir relativement proche sera comparable à notre dépendance à l'énergie. Cette dépendance, si elle se révèle vraie, ne sera évidemment pas sans conséquences sur les orientations et les enjeux politiques des différents gouvernements.

Aujourd'hui, l'intelligence artificielle est une discipline fortement dominée par l'Amérique du Nord avec les multinationales GAFAM, à savoir Google, Amazon, Facebook, Apple et Microsoft. Il faut cependant noter que ces cinq géants de l'informatique sont de plus en plus concurrencés par leurs rivaux asiatiques, connus sous le nom des BATX, à savoir : Baidu, Alibaba, Tencent et Xiaomi.

Dans ce paysage high-tech, l'Europe souhaite se lancer pour occuper une place importante et ainsi éviter de devenir fortement dépendante. En effet, en plus des enjeux économiques et sociaux de l'intelligence artificielle, celle-ci commence à être exploitée dans des domaines politiquement sensibles comme l'armée, l'espionnage et la manipulation de l'opinion publique à des fins politiques.

Entre 2015 et 2018, le secteur privé français avait déjà investi près d'un milliard d'euros dans le lancement de nouvelles start-up dédiées exclusivement au développement et au déploiement de l'intelligence artificielle. Cet investissement représente à peu près quatre fois les investissements sur l'ensemble des secteurs du numérique réunis. Avec tous ces efforts financiers, la France et l'Europe espèrent rattraper leur retard et occuper les devants de la scène du marché de l'intelligence artificielle, aux côtés de leurs concurrents américains et asiatiques.

Le Machine Learning avec Python

De la théorie à la pratique

La nature a toujours été une source d'inspiration inépuisable pour nous. En effet, nous avons depuis toujours observé notre environnement pour essayer de le comprendre et d'en tirer profit. Il en va de même en intelligence artificielle. Les scientifiques, particulièrement les chercheurs en informatique, ont réussi à construire de nouveaux algorithmes essentiellement basés sur notre compréhension de certains phénomènes naturels. Après avoir essayé d'imiter la pensée et le comportement humain, notamment avec les réseaux de neurones, les chercheurs se sont tournés vers les espèces qui vivent en société pour s'en inspirer. Notre compréhension de l'organisation des sociétés telles que les colonies de fourmis ou notre compréhension du modèle qui régit le déplacement d'un groupe d'oiseaux a élargi notre conception de l'intelligence artificielle. Ainsi, grâce à l'imitation virtuelle de ces espèces qui vivent en groupe, nous sommes passés d'une intelligence artificielle individuelle, focalisée sur le fonctionnement du cerveau humain, à une intelligence artificielle collective basée sur le comportement d'un groupe d'individus.

Qu'elles soient d'origine physique ou biologique, toutes ces innovations algorithmiques ont permis, chacune à sa façon, de résoudre des types de problèmes particuliers. Ces programmes informatiques basés sur une intelligence collective sont communément appelés des systèmes multiagents, ou encore des systèmes d'intelligence distribuée ou d'intelligence en essaim. L'avantage principal de ces algorithmes réside dans leur robustesse face à la survenue de pannes. En effet, l'absence de maillon faible dans ces algorithmes les rend tolérants à la défaillance.

Effectivement, dans ces systèmes multiagents, c'est l'union qui fait la force. Par exemple, dans une fourmilière, le fait qu'une fourmi meure ou arrête de travailler ne remet aucunement en question le fonctionnement global de la fourmilière.

Il existe plusieurs processus biologiques et/ou physiques à l'origine d'un grand nombre d'algorithmes. Ainsi, les chercheurs en IA sont partis du cerveau humain pour élaborer les réseaux de neurones, de la théorie d'évolution de Darwin pour élaborer les algorithmes génétiques, des fourmis réelles pour élaborer les algorithmes d'optimisation par colonie de fourmis et de la métallurgie pour concevoir les algorithmes du recuit simulé, etc.

© Editions ENI - All rights reserved

Introduction générale

Dans la nature et dans les entreprises, les issues de certains événements dépendent de phénomènes aléatoires. Étudier de tels phénomènes n'est donc évidemment pas une tâche facile. Le Machine Learning, qui est un sous-domaine de l'intelligence artificielle, propose un cadre théorique et algorithmique pour modéliser ce type de phénomènes.

Le Machine Learning propose d'étudier des systèmes pour lesquels un ensemble d'entrées et de sorties est connu, mais le traitement effectué pour transformer une entrée en une sortie est inconnu. L'objectif est de comprendre ce traitement.

Plus précisément, l'objectif du Machine Learning est d'étudier les entrées et les sorties d'un système pour deviner ou approcher le traitement effectué par ce système. Une fois que ce traitement est deviné ou approché, alors pour les prochaines entrées, le résultat à la sortie sera anticipé avant même qu'il se produise.

Le Deep Learning, ou l'apprentissage profond en français, est une méthode d'apprentissage dérivée directement du Machine Learning. En réalité, le Deep Learning est un sous-domaine du Machine Learning.

Nous savons que les performances d'un algorithme de Machine Learning dépendent essentiellement, mais pas seulement, de la quantité et de la qualité des données sur lesquelles cet algorithme s'est entraîné. Le problème est que la labélisation de quantités importantes de données par des humains est malheureusement difficile à réaliser, voire impossible. Justement, le Deep Learning permet d'automatiser cette étape d'étiquetage de données et permet ainsi d'avoir des quantités de données d'entraînement suffisantes pour obtenir des modèles efficaces.

Ces dernières années, le Deep Learning s'est montré particulièrement efficace dans le domaine de la reconnaissance d'objets dans les images.

En intelligence artificielle, plus les données d'apprentissage sont volumineuses, plus les algorithmes deviennent intelligents. Malheureusement, l'expérience nous a montré que la présence massive de ces données dans les bases de données d'une entreprise peut mener à des usages non respectueux de la vie privée des consommateurs, comme par exemple le scandale accusant Facebook de détourner les données privées de ses membres pour influencer les résultats des élections.

Pour contrer tout type d'intrusion dans la vie privée des consommateurs, la Commission européenne a mis en place la GDPR (*General Data Protection Regulation*) ou en français, RGPD (Règlement Général sur la Protection des Données). Son objectif principal est de permettre aux citoyens de l'UE de maîtriser leurs données.

Le RGPD s'applique à toute organisation qui stocke et traite des données relatives aux citoyens de l'Union européenne. Les données concernées par cette réglementation sont toutes les données privées qui peuvent être utilisées pour identifier un consommateur. Ces données peuvent être :

1. Les données pouvant servir à l'identification directe des individus, telles que le numéro de téléphone mobile ou le numéro de téléphone fixe, le nom et le prénom, l'adresse e-mail, etc.

2. Les données pouvant servir à l'identification indirecte des individus, telles que l'adresse postale, même sans le nom ou l'adresse IP, etc.

3. Les informations sensibles qui concernent la santé, l'origine raciale ou ethnique, la religion, les convictions et opinions politiques, l'appartenance à un syndicat, etc.

Pour être conforme au RGPD, il faut respecter les principes suivants :

- Principe de responsabilité : prendre toutes les mesures pour garantir la conformité au RGPD.
- Principe de licéité : les données privées ne peuvent être récoltées et traitées que dans un but bien déterminé.
- Principe d'information : toutes les personnes concernées par une récolte de données doivent être informées et consentantes.
- Principes de minimisation et limitation : seules les données nécessaires au traitement défini peuvent être utilisées.
- Sécurité par défaut : toutes les mesures de sécurité doivent être mises en œuvre afin de garantir la confidentialité, l'intégrité et la disponibilité des données privées.
- Privacy By Design : le respect de la vie privée des consommateurs doit être pris en compte dès la conception d'une application quelconque.

© Editions ENI - All rights reserved

En plus de ces principes, le RGPD exige des entreprises qu'elles mettent à disposition des consommateurs un moyen simple à utiliser pour :

- Récupérer, modifier et supprimer toutes leurs données personnelles à tout moment.
- Importer toutes leurs données personnelles dans un format standard afin de garantir l'interopérabilité entre les services.

Une entreprise peut nommer un DPO (*Data Protection Officer*) qui sera l'employé en charge de l'application du RGPD et d'assurer le lien entre son entreprise et les autorités compétentes, comme la CNIL (la Commission nationale de l'informatique et des libertés). Depuis le 25 mai 2018, une organisation qui n'est pas conforme au RGPD risque une amende de 20 millions de dollars ou une amende correspondant à 4 % de son chiffre d'affaires mondial ; c'est la plus élevée de ces deux amendes qui sera réclamée en cas de litige. Les limitations imposées par le RGPD sur les méthodes de récolte et d'exploitation des données des consommateurs ont le mérite d'un côté de pousser les entreprises à réaliser des études introspectives sur leur patrimoine applicatif, et de l'autre côté de mettre en avant les algorithmes de l'intelligence artificielle explicables. Pour cela, les algorithmes tels que le Deep Learning devront évoluer pour que leurs raisonnements soient compris par un être humain, ce qui n'est pas forcément le cas aujourd'hui.

Pour rester conforme au RGPD et développer des projets d'intelligence artificielle utilisant des données personnelles, il suffit a minima d'opérer un processus d'anonymisation ou de pseudonymisation de ces données. Ces processus sont différents et il ne faudrait donc pas les confondre.

L'anonymisation des données consiste à appliquer sur un ensemble de données une méthode irréversible de sorte qu'il serait impossible d'identifier les personnes sur lesquelles portent ces données. Il existe plusieurs méthodes d'anonymisation de données, comme la méthode du hachage qui permet de modifier les données, ou la méthode de confidentialité différentielle qui permet d'insérer un caractère aléatoire dans le processus d'accès aux données.

Théoriquement, l'anonymisation des données représente le niveau maximal de protection. Cependant, il est très difficile d'anonymiser parfaitement un ensemble de données et cette difficulté croît avec la taille des données à anonymiser. D'ailleurs, nous avons assisté à plusieurs reprises à la réidentification des individus à partir de données censées être anonymisées, mais qui finalement ne l'étaient pas ! Par exemple, lors de la compétition organisée par Netflix entre 2006 et 2009, dont le but était l'amélioration de son système de recommandation, une base de données contenant des données anonymisées d'abonnés de Netflix a été rendue publique. Des chercheurs ont réussi à identifier une partie de ces abonnés en procédant à un recoupement des données de cette compétition avec les notes attribuées par les membres d'IMDb (*Internet Movie Database*).

L'anonymisation est donc très efficace mais difficile à atteindre. Aussi, il faut noter qu'une fois que les données sont parfaitement anonymisées, elles ne sont plus concernées par le RGPD.

Contrairement à l'anonymisation des données, la pseudonymisation des données consiste à appliquer sur un ensemble de données un processus réversible dont l'objet est de remplacer certaines données par d'autres données de sorte que les enregistrements ne permettent plus l'identification de personnes. En revanche, une clé sera utilisée afin de pouvoir identifier les personnes à partir des nouvelles données. Cette clé doit être sauvegardée dans un endroit sûr avec une gestion des accès robuste.

La pseudonymisation est une alternative à l'anonymisation qui est applicable surtout dans les expérimentations de Data Science où l'identité des individus n'est pas requise.

D'un point de vue juridique, la discrimination et le respect du RGPD font partie des sujets les plus importants lors du développement d'un projet en intelligence artificielle.

Pour être conforme au RGPD, il suffit d'appliquer les principes définis dans le règlement lui-même. En revanche, la discrimination est un sujet plus difficile à cerner car elle peut s'inviter sournoisement dans nos données et nos algorithmes. La discrimination, même involontaire, peut coûter très cher et nuire à l'image de marque d'une entreprise et mener à des poursuites judiciaires.

© Editions ENI - All rights reserved

La difficulté posée par la discrimination réside dans le fait qu'elle peut impacter un projet d'intelligence artificielle, particulièrement un projet de Data Science, à différents niveaux. En effet, la discrimination peut prendre place au sein d'un projet lors de la préparation des données et/ou lors de la construction du modèle.

Avant de se lancer dans la construction d'un modèle prédictif, il faut d'abord vérifier si l'objectif de notre modèle est bien conforme à toute réglementation en vigueur, et aussi étudier l'impact des résultats de ce modèle sur les personnes concernées. Par exemple, une compagnie d'assurance qui souhaite faire une étude pour prédire l'état de santé de ses assurés devrait se poser la question de savoir si elle a le droit de réaliser une telle étude du point de vue juridique et aussi si les résultats d'un tel modèle seront respectueux des valeurs morales et éthiques. Une fois que l'objectif d'un projet est validé, il se posera la question de la récolte des données. Dans la plupart des projets en Machine Learning, les données sont issues de plusieurs ressources différentes. Cependant, malgré la multiplicité des sources des données utilisées pour entraîner un modèle, il faut être très vigilant à ce que ces données soient représentatives de la population visée par l'étude. Par exemple, une société qui souhaite proposer un service de recherche d'emploi et prédire quel emploi serait le mieux adapté à quel profil ne devrait pas se baser uniquement sur les données récoltées à travers les smartphones ! Elle risque de ne pas inclure les personnes plus âgées qui n'utilisent pas forcément ce type de technologie, et par conséquent ne pas leur proposer suffisamment d'emplois par rapport aux personnes moins âgées.

Les principales sources de biais lorsqu'on applique un algorithme sur un jeu de données pour entraîner un modèle sont les suivantes :

- Le choix des données d'entraînement, car la pertinence d'un modèle et d'autant plus élevée que le degré de diversité de ces données est grand.
- Le paramétrage des algorithmes. En effet, certains paramètres des algorithmes en Machine Learning influencent l'importance accordée aux variables utilisées lors d'une étude. Ainsi, certaines variables vont avoir des poids plus élevés que d'autres variables. Par exemple, si lors d'une étude la variable « Sexe » est plus considérée que le reste des variables, alors le risque de discrimination en fonction du sexe sera élevé.

- L'utilisation des modules tiers. Lorsque nous développons un projet en Intelligence artificielle, nous faisons souvent recours à des modules développés par d'autres sociétés. Si ces modules sont sujets à des problèmes de discrimination, alors notre solution le sera aussi.

Afin d'éviter au maximum la discrimination dans les projets d'intelligence artificielle, il faudrait veiller à ce que les données utilisées soient les plus diverses et variées possible.

Les responsabilités liées aux dysfonctionnements des programmes en intelligence artificielle sont extrêmement importantes à prendre en compte, car elles peuvent être lourdes de conséquences !

Par exemple, en 2016, Microsoft a mis en ligne un chatbot baptisé Tay, dont le but était de simuler des discussions humaines et participer sur Twitter à tous types de conversations sociales. Au bout de 24 heures d'échanges intenses sur la toile, Microsoft a dû débrancher Tay et relayer via ses canaux de communication officiels l'énoncé suivant :

« Malheureusement, dans les 24 premières heures de sa mise en ligne, nous avons constaté un effort coordonné de certains utilisateurs pour maltraiter Tay et son habileté à communiquer afin qu'elle réponde de manière inappropriée. »

Concrètement, Tay a eu des propos que nous pouvons sans ambiguïté qualifier d'injures racistes et sexistes.

Alors, qui est le responsable de ses propos ? Microsoft ? Le propriétaire du réseau social Twitter ? Les internautes ?

Cette question est fondamentale lorsque nous abordons les sujets de la responsabilité et de la justice en intelligence artificielle.

Le problème principal de la responsabilité en intelligence artificielle découle directement de l'évolution des programmes qui lui sont associés. En effet, de plus en plus, les programmes en intelligence artificielle évoluent de manière autonome et les développeurs à l'origine de ces programmes ne sont pas directement impliqués dans ce type d'évolution ! Certains programmes en intelligence artificielle peuvent devenir tellement autonomes que leur développement ne résultent plus des programmes initialement prévus et écrits par les développeurs.

© Editions ENI - All rights reserved

De plus, le Code pénal n'est applicable qu'à des personnes physiques ou des personnes morales ; or, une intelligence artificielle n'est pas une entité physique et elle est encore moins une entité morale. Pour pallier ce vide juridique, les textes de loi évoluent constamment dans le but de déterminer les responsabilités délictuelles et limiter les erreurs liées aux multiples usages de l'intelligence artificielle.

Plusieurs sujets ont ainsi été abordés au Parlement européen. Une commission européenne a par exemple proposé la création d'un statut juridique pour les personnes électroniques. Cette proposition a reçu plusieurs critiques de la part des experts de l'IA et de la robotique, puisqu'elle peut facilement aboutir à une situation où les éditeurs de logiciels ont une responsabilité moindre.

Les pratiques en matière de responsabilité civile vont sans doute être transformées, nous entendons d'ailleurs déjà parler de justice prédictive et de juriste augmenté. L'intelligence artificielle s'est déjà invitée dans les tribunaux, puisqu'elle est déjà utilisée pour prédire, par exemple, la probabilité de récidive d'un criminel. Certains avocats l'utilisent quant à eux pour préparer leurs arguments de défense et leurs textes de loi.

Un autre problème lié directement à l'assurance et aux questions de responsabilité pénale en intelligence artificielle est celui des performances nettement supérieures de la machine dans certains domaines. Par exemple, une équipe composée de scientifiques a créé un algorithme capable de distinguer les lésions de la peau des simples grains de beauté, avec un taux de réussite de 95 % sur des cas jugés compliqués, contre un taux de réussite de 87 % pour une équipe composée de 58 médecins spécialistes. Un médecin qui n'a pas su détecter une lésion de la peau suffisamment à temps risque de coûter cher à son assureur si le patient en question porte plainte. Objectivement parlant, un assureur a plus intérêt à assurer notre programme qu'un médecin !

En synthèse, même si certaines réglementations existent, les responsabilités associées aux erreurs et aux dysfonctionnements des programmes d'intelligence artificielle ne sont pas complètement définies et beaucoup de travail reste à accomplir du point de vue juridique.

La détection de tumeurs cancéreuses, la détection de fraudes, l'assistance à l'éducation, la maintenance prédictive et bien d'autres exemples sont autant de cas d'usage où l'intelligence artificielle s'est avérée bénéfique pour l'être humain. Cependant, la vitesse vertigineuse avec laquelle elle évolue et son implication, qui est de plus en plus importante dans notre vie de tous les jours, sont des sujets anxiogènes pour certains. Plusieurs experts semblent d'ailleurs être pessimistes et alarmistes sur le devenir de cette technologie. En effet, dans bien des situations, l'intelligence artificielle s'est montrée vulnérable. Dans certains cas, la fragilité de l'IA résulte des approximations liées directement à ses algorithmes ; dans d'autres cas, ces vulnérabilités sont liées à des usages malicieux. La plupart des expériences de l'intelligence artificielle avec des impacts négatifs sont liées à des faits discriminatoires.

Plusieurs organisations et sites internet ont été mis en place pour recenser les projets et les expériences utilisateurs liés à des impacts négatifs de l'usage de l'IA. Par exemple, l'organisation « Algorithmic Justice League », accessible à l'adresse www.ajlunited.org, reporte plusieurs expériences d'algorithmes d'intelligence artificielle à caractère discriminatoire d'ordre ethnique. Une des expériences relatées dans ce site est celle où un algorithme de reconnaissance faciale parvient sans problème à détecter les visages de couleur blanche mais pas ceux de couleur noire. Ce type de biais à caractère racial est probablement lié aux données qui ont été utilisées lors de l'entraînement de l'algorithme qui n'était visiblement pas suffisamment diversifiées.

Un autre exemple d'organisation est le site « NOTFLAWLESS.AI » accessible avec le lien www.notflawless.ai. Sur ce site, les visiteurs peuvent partager leurs propres expériences négatives avec l'intelligence artificielle. Cela permet deux choses :

1. Dénoncer et faire connaître les projets de l'IA qui peuvent être néfastes.

2. Permettre aux développeurs de prendre conscience des erreurs dans leurs développements afin d'y remédier.

© Editions ENI - All rights reserved

Une troisième organisation intéressante est « ALGORITHM WATCH », accessible à l'adresse https://algorithmwatch.org. Outre les histoires négatives des usages de l'IA qui y sont rapportées, ce site définit la norme ADM (*Algorithmic Decision Making Manifesto*), qui consiste en une liste des points qui doivent être considérés dans tout projet d'intelligence artificielle afin d'éviter les erreurs et les dérives de l'IA.

Beaucoup d'autres organisations dénonçant les mauvais usages de l'IA existent. Bien sûr, au-delà des échanges sur les mauvaises expériences, leur rôle est aussi de faire évoluer l'opinion publique pour une prise de conscience des impacts et des conséquences de ces mauvaises pratiques. Cette prise de conscience de l'opinion publique pourra à son tour faire pression sur les autorités et faire ainsi évoluer les lois régissant les pratiques de l'intelligence artificielle.

Malgré les innombrables utilisations nobles de l'intelligence artificielle, il reste difficile de distinguer les progrès et les menaces sous-jacents. Les menaces de l'IA peuvent être préméditées ou non intentionnelles. Nous pouvons distinguer trois types de dérives de l'IA, à savoir :

1. Les dérives impactant directement la société. L'impact sur la société qui fait le plus parler de lui est sans doute lié à la capacité de l'intelligence artificielle à remplacer l'Homme dans son travail. Et ces menaces ne se situent pas uniquement au niveau des emplois où les tâches sont répétitives. Bien au contraire, aujourd'hui, les évolutions technologiques, en matière d'algorithmes et de puissance de calcul, combinées avec la disponibilité de grandes quantités de données, ont donné naissance à des logiciels capables d'effectuer des tâches qui étaient encore il y a peu de temps, réservées uniquement à l'humain. Par exemple, comme cité plus haut, les logiciels de détection de tumeurs cancéreuses, les voitures autonomes, la détection de fraudes, etc. Bien que l'IA menace beaucoup d'emplois, elle contribue à en créer d'autres. Toute la question est de connaître le rapport entre le nombre d'emplois qui seront créés et ceux qui seront supprimés.

2. Les dérives liées aux dysfonctionnements des algorithmes. Les dérives de l'IA peuvent être liées à des dysfonctionnements qui peuvent induire des préjudices plus ou moins importants en fonction des cas.

Les exemples en la matière ne manquent pas et pour n'en citer que quelques-uns :

- Les enceintes domestiques qui écoutent des conversations à l'insu de leurs propriétaires.
- Les accidents causés par les voitures autonomes.
- Les drones autonomes militaires qui, en cas de bug, peuvent avoir des conséquences dramatiques et irréversibles.

3. Les dérives liées aux utilisations malveillantes de l'IA. Certains chercheurs pensent que dans quelques années l'intelligence artificielle pourra être détournée pour réaliser des scénarios dignes des films catastrophes hollywoodiens ! Ainsi, ils se sont penchés sur trois types de scénarios, à savoir :

- Des scénarios dans le monde numérique : imaginez un malware virulent qui change de comportement en fonction des machines infectées.
- Des scénarios dans le monde politique : les « fake news » et l'usurpation de l'identité vocale peuvent faire des ravages très importants. Imaginez un robot qui serait capable d'imiter votre voix et participer à une conférence en ligne en prônant des idées complètement différentes des vôtres ! Ou encore, une personne qui consulte naïvement des sites contenant des « fakes news » et qui sera poussée à mener des actions sur Internet en croyant en la validité des informations qu'elle a lues ! En combinant des générateurs de données, un algorithme peut créer des pages web avec des histoires imaginées et plonger ainsi les lecteurs dans un monde complètement faux et fou ! Évidemment, de telles pratiques peuvent influencer l'opinion publique sur n'importe quel sujet !
- Des scénarios dans le monde physique : nous pouvons imaginer un robot ménager détourné de ses fonctions pour atteindre et tenter de nuire physiquement à une personne !

D'après le rapport *"The Malicious Use of Artificial Intelligence : Forecasting, Prevention, and Mitigation"*, tous ces scénarios seront tout à fait réalisables d'ici peu de temps.

© Editions ENI - All rights reserved

Heureusement, ces scénarios catastrophiques ne font pas l'unanimité au sein de la communauté scientifique. En effet, pour plusieurs experts, les progrès qui peuvent nous mener vers de tels scénarios ne seront probablement jamais atteints.

Tous les algorithmes de l'IA, particulièrement ceux du Machine Learning, sont susceptibles de tomber dans le piège de la discrimination, de commettre des erreurs et d'avoir des dysfonctionnements, ou encore être la cible de pirates et d'être détournés à des fins malicieuses.

Au-delà des réglementations mises en vigueur pour prévenir les abus liés à l'usage de l'IA, il existe des bonnes pratiques qui permettent d'éviter la plupart de ces pièges. Cependant, il n'existe malheureusement pas de recette magique pour éviter tout type de risque et empêcher tout type de dérive.

Une première bonne pratique est l'application minutieuse des réglementations comme le RGPD ou toute autre loi relative à la manipulation des données privées. Le respect des réglementations permet déjà d'éviter un très grand nombre de problèmes liés à l'utilisation de l'intelligence artificielle.

Une deuxième bonne pratique est de toujours donner le dernier mot à l'être humain, et cela pour au moins les trois raisons suivantes :

1. L'être humain doit être impliqué impérativement dans le processus de décision, car pour l'instant il est le seul et unique responsable de l'impact négatif qui peut suivre une décision.

2. Même si les performances de la machine dépassent celles de l'humain dans certains domaines, plusieurs situations restent évidentes pour l'humain, mais très difficiles et compliquées pour les algorithmes de l'intelligence artificielle. Dans la réalité, ce sont souvent ces cas complexes qui génèrent les impacts négatifs les plus importants.

3. Les algorithmes de l'intelligence artificielle doivent être utilisés en tant que conseillers de l'être humain afin d'augmenter les capacités de l'agent humain et non pas l'inverse.

Une troisième bonne pratique est de mettre en place des outils de supervision et d'administration de l'intelligence artificielle afin de permettre à l'humain d'avoir le pouvoir de modifier le comportement des algorithmes à tout moment, et surtout de pouvoir analyser les résultats de façon claire et sans ambiguïté.

Il reste encore beaucoup de travail à faire afin d'avoir le contrôle total sur les algorithmes de l'IA. En effet, même si la mise en place de ces bonnes pratiques est indispensable pour limiter les risques et les dérives de la machine, dans la pratique elle s'avère insuffisante. Le problème est que pour que l'humain puisse superviser la machine, il doit savoir à tout moment quelles sont les données utilisées, comment elles sont traitées et quels sont les résultats. Mais vu la diversité et la multiplicité des sources de données et la complexité de certains algorithmes de l'IA, cette supervision peut demander des capacités largement au-delà des possibilités humaines.

En résumé, même si les bonnes pratiques ont bien été appliquées, il faut toujours prendre du recul et être le plus vigilant possible vis-à-vis de l'utilisation de l'intelligence artificielle.

Il est intéressant de distinguer l'intelligence artificielle faible de l'intelligence artificielle forte. L'intelligence artificielle faible désigne les algorithmes qui n'ont aucun moyen de résoudre des problèmes autres que ceux pour lesquels ils ont été initialement programmés. En revanche, l'intelligence artificielle forte est celle où les algorithmes sont capables d'extrapoler ce qu'ils ont appris pour résoudre un problème spécifique, afin de réaliser de nouvelles tâches qui n'ont pas été prévues par leurs programmes initiaux. En d'autres termes, l'intelligence artificielle forte désigne les algorithmes plus au moins dotés d'une conscience.

Il faut noter qu'à ce jour les programmes d'intelligence artificielle connus correspondent à la forme d'intelligence artificielle faible.

Aujourd'hui, la place médiatique de l'intelligence artificielle va au-delà des laboratoires et des entreprises pour atteindre le débat public. Cela permet d'avoir des dialogues intéressants et de soulever beaucoup de questions sur le devenir de cette discipline et de ses impacts sur nos sociétés.

© Editions ENI - All rights reserved

Bien sûr, nous ne sommes qu'au début du déploiement généralisé des technologies et c'est aux décennies à venir de nous dire si l'intelligence artificielle pourra un jour atteindre sa forme la plus puissante, et relativement la plus dangereuse, à savoir l'intelligence artificielle forte.

Les éléments à télécharger sont disponibles à l'adresse suivante :
http://www.editions-eni.fr
Saisissez la référence de l'ouvrage **EIMLPYT** dans la zone de recherche et validez. Cliquez sur le titre du livre puis sur le bouton de téléchargement.

Préface

Introduction générale

Avant-propos

1. Pourquoi ce livre ? 19
2. À qui s'adresse ce livre ? 21
3. Comment est organisé ce livre ? 23
4. Comment lire ce livre ? 24
5. Quels sont les prérequis pour la lecture de ce livre ? 25
6. Qui est l'auteur ? 25
7. Remerciements 27

Partie 1 : La Data Science - Concepts généraux

Chapitre 1-1
La Data Science

1. Objectif du chapitre 29
2. L'objectif recherché en Machine Learning 30
3. Une expérimentation Machine Learning 34
   3.1 Types de données 42
   3.2 Préparation des données 44
4. Cycle de vie d'un projet Data Science 47
5. Les algorithmes du Machine Learning 50
6. Le problème de surapprentissage 52

7. Les paramètres et les hyperparamètres 53
8. Validation croisée 55
9. Données d'entraînement, de validation et de test 59
10. Métriques de performance 60
10.1 Métriques pour les problèmes de régression 62
10.2 Métriques pour la classification 65
10.2.1Matrice de confusion binaire 65
10.2.2Matrice de confusion générale 67
10.2.3Exemple de matrice de confusion 68
10.2.4La courbe ROC 70
10.3 Métriques pour le clustering 71
11. Conclusion 71

Partie 2 : Outils techniques de la Data Science - Python, NumPy, Pandas et Jupyter

Chapitre 2-2
Le langage Python

1. Objectif du chapitre 73
2. Python en deux mots 74
3. Installer l'interpréteur Python 74
4. Les bases de la programmation Python 77
4.1 Hello world avec Python 77
4.1.1 La fonction print 77
4.1.2 La fonction input 81
4.2 Les structures de données 81
4.2.1 Les variables numériques 82
4.2.2 Les chaînes de caractères 87
4.2.3 Le type booléen 89
4.2.4 Les listes 95
4.2.5 Les tuples 98

4.2.6 Les dictionnaires 99
4.2.7 Les ensembles 101
4.2.8 Liste vs tuple vs dictionnaire vs ensemble 105
4.3 Structurer un code Python 106
4.3.1 L'indentation et les blocs de code 106
4.3.2 Écrire une instruction sur plusieurs lignes 107
4.3.3 Écrire plusieurs instructions sur une ligne 109
4.3.4 Les commentaires en Python 109
4.4 Les instructions conditionnelles 109
4.4.1 Les conditions avec la structure if 110
4.4.2 Les conditions avec la structure if-else 111
4.4.3 Les conditions avec la structure if-elif-else 112
4.5 Les boucles 114
4.5.1 La boucle for 114
4.5.2 La boucle for et la fonction zip 119
4.5.3 La boucle while 125
4.5.4 Contrôler les boucles avec break 126
4.5.5 Contrôler les boucles avec continue 127
4.6 Les fonctions 128
4.6.1 Définir et utiliser une fonction sans paramètre 129
4.6.2 Les fonctions avec paramètres 131
4.6.3 Les valeurs par défaut des paramètres 133
4.6.4 Renvoi de résultats 136
4.6.5 La portée des variables 137
4.6.6 Passage d'arguments à une fonction 140
4.6.7 Les fonctions récursives 143
4.7 Les listes en compréhension 148
4.7.1 Les origines des listes en compréhension 148
4.7.2 Construire une liste avec les listes en compréhension 149
4.7.3 Application de fonction avec une liste en compréhension 150
4.7.4 Utiliser if-else avec les listes en compréhension 151
4.7.5 Filtrer avec les listes en compréhension 152

4.8 Les expressions régulières 152
4.8.1 Regex sans caractères spéciaux 155
4.8.2 Regex avec caractères spéciaux 157
4.8.3 Regex avec les multiplicateurs 159
4.8.4 Regex avec un nombre d'occurrences limité 162
4.8.5 Regex avec groupage des résultats 163
4.8.6 Taille des motifs 164
4.8.7 Aller plus loin avec les expressions régulières 169
4.9 Gestion des exceptions 170
4.9.1 La levée des exceptions 170
4.9.2 Utiliser le bloc try-except 172
4.9.3 Gérer plusieurs exceptions 173
4.9.4 Utiliser la clause finally 175
4.9.5 Utiliser la structure try-except-finally-else 176
4.9.6 Lever une exception avec raise 178
5. Conclusion 180

Chapitre 2-2
La bibliothèque NumPy

1. Objectif du chapitre 181
2. NumPy en deux mots 182
3. Les tableaux NumPy 182
3.1 Création de tableaux NumPy 182
3.1.1 Créer un tableau à une dimension 183
3.1.2 Créer un tableau à plusieurs dimensions 183
3.2 Les dimensions d'un tableau NumPy 185
3.3 Le type et la taille d'un tableau NumPy 187
3.4 Fonction d'initialisation de tableaux NumPy 189
4. Accéder aux données d'un tableau NumPy 191
4.1 Accès aux données d'un tableau à une dimension 191
4.2 Accès aux données d'un tableau à deux dimensions 193

4.3 Accès aux données d'un tableau à trois dimensions 195
5. Modifier les données d'un tableau NumPy 196
6. Copier un tableau NumPy dans un autre tableau NumPy 197
7. Algèbre linéaire avec NumPy 199
7.1 Opérations mathématiques de base avec NumPy 199
7.2 Opérations sur les matrices avec NumPy 201
8. Réorganiser des tableaux NumPy 202
8.1 Restructurer un tableau NumPy 202
8.2 Superposer des tableaux NumPy 204
9. Statistiques descriptives avec NumPy 205
10. Lire des données NumPy à partir d'un fichier 207
11. Les masques booléens avec NumPy 208
11.1 Créer et utiliser un masque booléen 208
11.2 Un masque avec plusieurs conditions 210
11.3 Les fonctions numpy.any et numpy.all 211
12. Tableaux NumPy versus listes Python 213
12.1 Comparaison des tailles en mémoire 214
12.2 Comparaison des temps de calcul 215
12.2.1Temps de calcul sur une liste Python 216
12.2.2Temps de calcul sur un tableau NumPy 217
13. Conclusion 218

Chapitre 2-3
La bibliothèque Pandas

1. Objectif du chapitre 219
2. C'est quoi, Pandas ? 220
3. Installation de Pandas 221

4. DataFrame Pandas 222
4.1 Création d'un DataFrame à partir d'un dictionnaire 223
4.2 Création d'un DataFrame à partir d'un tableau NumPy 225
4.3 Chargement des données à partir de fichiers 226
4.3.1 Lecture des données d'un fichier CSV 227
4.3.2 Lecture d'un fichier texte 228
5. Accès aux données d'un DataFrame 229
5.1 Lire les lignes d'un DataFrame 230
5.1.1 Lire une ligne d'un DataFrame 230
5.1.2 Lire plusieurs lignes d'un DataFrame 230
5.1.3 Parcourir les lignes d'un DataFrame 231
5.1.4 Filtrer les lignes avec une condition 232
5.1.5 Filtrer les lignes avec plusieurs conditions 232
5.1.6 Filtrage avec des critères textuels 233
5.1.7 Réinitialiser les index 234
5.1.8 Filtrer avec les valeurs uniques 236
5.1.9 Filtrer avec une expression régulière 236
5.2 Accéder aux variables d'un DataFrame 237
5.2.1 Liste des variables d'un DataFrame 237
5.2.2 Accès aux valeurs d'une colonne 238
5.2.3 Accès à plusieurs colonnes 238
5.3 Lire une cellule spécifique avec les index 239
6. Modifier un DataFrame 239
6.1 Modifier les valeurs dans un DataFrame 239
6.2 Modifier la structure d'un DataFrame 240
6.2.1 Ajouter une variable à un DataFrame 240
6.2.2 Réordonner les variables d'un DataFrame 243
6.2.3 Supprimer une variable d'un DataFrame 244
6.2.4 Utiliser la méthode melt pour diminuer le nombre de variables 245
6.3 Appliquer une fonction sur une variable avec la méthode apply 247
6.4 Modification avec conditions 249

6.5 Ajouter des lignes dans un DataFrame 250
7. Tri sur les données d'un DataFrame 251
7.1 Tri avec un seul critère 251
7.2 Tri avec plusieurs critères 253
8. Sauvegarder les données d'un DataFrame 254
9. Faire des statistiques sur un DataFrame 255
9.1 Faire un résumé direct 255
9.2 Faire un résumé par agrégation 256
9.3 Agrégation avec plusieurs paramètres 258
10. Lecture des fichiers de grande taille 259
11. Conclusion 261

Chapitre 2-4
Travailler avec Jupyter

1. Objectif du chapitre 263
2. Installation de l'environnement Anaconda et Jupyter 264
3. Travailler avec Jupyter 270
3.1 Les documents dans Jupyter 272
3.1.1 Créer un dossier 273
3.1.2 Renommer un dossier 274
3.1.3 Déplacer un dossier 275
3.1.4 Charger des documents 276
3.1.5 Supprimer des éléments 277
3.1.6 Navigation dans l'arborescence des dossiers 278
3.1.7 Créer un notebook 279
3.2 Utiliser un notebook Jupyter 280
3.2.1 Renommer un notebook 280
3.2.2 Les cellules Jupyter 281
3.2.3 Les fonctionnalités d'un notebook 285

3.3 Utiliser les widgets Jupyter 291
3.3.1 Le widget FloatSlider 291
3.3.2 Associer une fonction à un slider 292
3.3.3 Le widget interact 294
3.3.4 Le widget Image 296
3.3.5 Le widget DatePicker 296
4. Conclusion 297

Partie 3 : Les statistiques

Chapitre 3-1
Statistiques

1. Objectif du chapitre 299
2. Les statistiques descriptives 300
2.1 Paramètres de position 300
2.1.1 La moyenne 300
2.1.2 Le mode 301
2.1.3 La médiane 301
2.1.4 Les quartiles 304
2.2 Paramètres de dispersion 304
2.2.1 La variance 305
2.2.2 Calcul de la variance avec la formule de Koenig 305
2.2.3 L'écart-type 305
2.2.4 L'écart interquartile 306
3. Les lois de probabilité 306
4. La loi normale 308
5. L'échantillonnage 312
5.1 Principe de l'échantillonnage 312
5.2 Résultats sur la distribution des moyennes 313
5.3 Résultats sur la distribution des proportions 322
5.4 Théorème central limite 326

6. Les statistiques inférentielles 327
6.1 Estimation ponctuelle 328
6.2 Estimation de la moyenne par intervalle de confiance 332
6.3 Estimation d'une proportion par intervalle de confiance 336
6.4 Test d'hypothèse 340
6.4.1 Tests paramétriques 341
6.4.2 Tests non paramétriques 341
6.4.3 Construire un test d'hypothèse 342
6.5 Types de tests d'hypothèse 345
6.5.1 Test de conformité 345
6.5.2 Test d'adéquation 346
6.5.3 Tests d'homogénéité 347
6.5.4 Test d'indépendance de variables 348
6.6 Exemple numérique de test de conformité d'une moyenne 349
6.7 Le paradoxe de Simpson 352
7. Conclusion 354

Partie 4 : Les grands algorithmes du Machine Learning

Chapitre 4-1
La régression linéaire et polynomiale

1. Objectif du chapitre 355
2. La régression linéaire simple 356
2.1 La régression linéaire simple de point de vue géométrique 357
2.2 La régression linéaire simple de point de vue analytique 358
2.2.1 La méthode des moindres carrés 358
2.2.2 Quelques considérations statistiques sur les données 360
3. La régression linéaire multiple 361
3.1 La méthode des moindres carrés pour la régression multiple 362
3.2 La méthode de la descente de gradient 363

3.3 Exemple de régression linéaire multiple 364
3.3.1 Définition du jeu de données utilisées 364
3.3.2 Régression linéaire multiple avec Scikit-learn 365
3.3.3 Importer les modules Scikit-learn 366
3.3.4 Lecture des données dans un DataFrame 367
3.3.5 Normalisation des données 368
3.3.6 Construction d'un modèle linéaire 371
3.3.7 Évaluation d'un modèle linéaire 373
3.3.8 Évaluer le futur comportement d'un modèle 377
3.3.9 Cross-validation avec KFold 380
4. La régression polynomiale 388
4.1 Exemple de régression polynomiale 389
4.1.1 Construction d'un modèle polynomial 389
4.1.2 Le coefficient de détermination R^2 396
4.1.3 R^2 et les valeurs extrêmes 398
4.1.4 Modèle polynomial et surapprentissage 398
5. Aller plus loin avec les modèles de régression 404
5.1 La régularisation Lasso 404
5.2 La régularisation Ridge 405
6. Conclusion 405

Chapitre 4-2
La régression logistique

1. Objectif du chapitre 407
2. La régression logistique 408
3. Prédire les survivants du Titanic 412
3.1 Définition du jeu de données Titanic 412
3.2 Réalisation du modèle de régression logistique 413
3.2.1 Chargement des modules Scikit-learn 413
3.2.2 Lecture des données 414
3.2.3 Traitement des valeurs manquantes 415

3.2.4 Transformation de variables 416
3.2.5 Sélection des variables 418
3.2.6 Traitement des variables catégorielles 420
3.2.7 Entraînement du modèle logistique 421
3.2.8 Le seuil de décision 422
4. L'algorithme One-vs-All 426
5. Conclusion 426

Chapitre 4-3
Arbres de décision et Random Forest

1. Objectif du chapitre 427
1.1 Construction d'un arbre de décision 428
1.2 Prédire la classe d'appartenance avec un arbre de décision 431
1.3 Considérations théoriques sur les arbres de décision 432
1.3.1 Choix de la variable de segmentation 433
1.3.2 Profondeur d'un arbre de décision 434
2. Problème de surapprentissage avec un arbre de décision 439
3. Random Forest 439
4. Exemple de Random Forest avec Scikit-learn 440
5. Conclusion 446

Chapitre 4-4
L'algorithme k-means

1. Objectif du chapitre 447
2. k-means du point de vue géométrique 448
3. k-means du point de vue algorithmique 455
4. Application de k-means avec Scikit-learn 457
5. L'algorithme k-means et les valeurs extrêmes 464

6. Choisir le k de k-means 470
6.1 Déterminer k avec la méthode Elbow 473
6.2 Déterminer k avec le coefficient de silhouette 479
7. Les limites de k-means 485
8. Avantages et inconvénients de l'algorithme k-means 490
9. Quelques versions de l'algorithme k-means 491
10. Conclusion 491

Chapitre 4-5
Support Vector Machine

1. Objectif du chapitre 493
2. Le SVM du point de vue géométrique 494
3. Le SVM du point de vue analytique 498
4. Données non linéairement séparables 503
4.1 Le Kernel Trick 505
4.2 La condition de Mercer 507
4.3 Exemple de fonction noyau 507
5. Détecter les fraudes de cartes de crédit 508
5.1 Les données des transactions de cartes de crédit 508
5.2 Application de l'algorithme SVM pour la détection des transactions bancaires frauduleuses 509
5.2.1 Application de l'algorithme SVM sur les données creditcard.csv 509
5.2.2 Application du SVM sur un sous-ensemble de creditcard.csv 516
5.2.3 Application du SVM sur des données normalisées 518
5.3 Les paramètres de l'algorithme SVM 523
5.3.1 Le paramètre Kernel pour la variation de la fonction noyau 524
5.3.2 Le paramètre C 525
5.3.3 Le paramètre Gamma 529

5.3.4 Le paramètre C versus le paramètre Gamma 531
5.3.5 Tuning des hyperparamètres d'un SVM avec GridSearchCV 531
6. Conclusion 535

Chapitre 4-6
Analyse en composantes principales

1. Objectif du chapitre 537
2. Pourquoi l'ACP ? 538
3. L'ACP du point de vue géométrique 540
4. L'ACP du point de vue analytique 542
5. Indicateurs de la qualité de la représentation des données 545
5.1 Indicateurs liés aux individus 545
5.1.1 Score des individus 546
5.1.2 Qualité de la représentation des individus 546
5.1.3 Contribution des individus 546
5.2 Indicateurs liés aux variables 547
5.2.1 Le cercle des corrélations 547
5.2.2 Qualité de la représentation des variables 548
5.2.3 Contribution des variables 549
6. Exemple d'ACP avec Python 549
6.1 Déterminer le nombre de facteurs pertinents 554
6.2 Interprétation des résultats sur les individus 559
6.2.1 Représentation des individus 560
6.2.2 Calcul de la qualité de la représentation des individus 564
6.2.3 Calcul de la contribution des individus 565
6.3 Interprétation des résultats sur les variables 566
6.3.1 Tracer un cercle des corrélations 566
6.3.2 Calcul de la qualité de la représentation des variables 569
6.3.3 Calcul des contributions des variables 570
6.4 Représentation de nouveaux individus 571

7. Conclusion 573

Chapitre 4-7
Les réseaux de neurones

1. Objectif du chapitre 575
2. Modélisation d'un neurone artificiel 576
   2.1 Le neurone biologique 577
   2.2 Le neurone artificiel 578
3. Architecture d'un réseau de neurones 580
4. L'algorithme de rétropropagation 583
5. Exemple d'un réseau de neurones avec Scikit-learn 594
6. Conclusion 601

Partie 5 : Le Deep Learning et le traitement automatique du langage

Chapitre 5-1
Le Deep Learning avec TensorFlow

1. Objectif du chapitre 603
2. Le Deep Learning : notions générales 604
   2.1 Réseau de neurones avec plusieurs couches d'entrée 607
   2.2 Réseau de neurones avec plusieurs couches de sortie 608
   2.3 Réseau de neurones avec des branchements conditionnels 609
   2.4 Réseau de neurones avec de la récurrence RNN 610
   2.5 Réseau de neurones avec couches de convolution CNN 611
   2.6 Éviter le surapprentissage avec les couches Dropout 613
   2.7 Le Transfer Learning 615

3. Introduction à TensorFlow 619
3.1 Installer TensorFlow 619
3.1.1 Créer un environnement virtuel 620
3.1.2 Installer des bibliothèques dans un environnement virtuel avec Anaconda 624
3.1.3 Installer la bibliothèque TensorFlow 626
3.1.4 Tester TensorFlow 627
3.2 Opérations élémentaires avec les tensors 628
3.2.1 Travailler avec les tensors 629
3.2.2 Les tensors variables 633
3.2.3 Initialiser les tensors 633
3.2.4 Opérations algébriques avec les tensors 634
4. Les réseaux de neurones avec Sequential API 635
4.1 Charger les données 636
4.2 Définir un MLP avec Sequential API 641
4.3 Accéder aux éléments d'un réseau de neurones 643
4.4 Initialisation des poids et des biais d'un réseau de neurones 645
4.5 Compiler un réseau de neurones 647
4.6 Entraîner un réseau de neurones 648
4.7 Analyser les résultats de l'entraînement d'un réseau de neurones 650
4.8 Évaluer un réseau de neurones 652
4.9 Prédire avec un réseau de neurones pour la classification 652
5. Utiliser Functional API 654
5.1 Un modèle Functional API avec plusieurs couches d'entrée 655
5.2 Un modèle Functional API avec plusieurs couches de sortie 658
6. Opérations avancées sur les réseaux de neurones 661
6.1 Monitorer un réseau de neurones 661
6.1.1 Contrôler les critères d'arrêt avec les callbacks 661
6.1.2 TensorBoard 664
6.1.3 Sauvegarder un réseau de neurones 668
6.1.4 Charger et utiliser un réseau de neurones 668
6.2 Réseaux de neurones de convolution 670

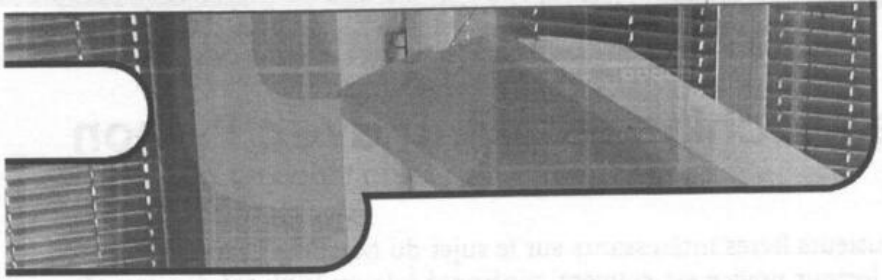
6.3 Réutiliser un réseau de neurones 674
6.4 Le Transfer Learning 676
6.4.1 Chargement des données locales 678
6.4.2 Chargement du modèle VGG16 679
6.4.3 Extraction des features 679
6.4.4 Étendre un modèle 680
6.4.5 Chargement des données de test pour le Transfer Learning 681
7. Aller plus loin avec le Deep Learning et TensorFlow 684
8. Conclusion 685

Chapitre 5-2
Le traitement automatique du langage

1. Objectif du chapitre 687
2. NLP : concepts généraux 688
2.1 Le nettoyage des données textuelles 690
2.1.1 Suppression des stopwords 690
2.1.2 Appliquer le Stemming sur un texte 692
2.1.3 Appliquer la Lemmatization sur un texte 692
2.1.4 Stemming versus Lemmatization 692
2.2 Vectorisation des données textuelles 693
2.2.1 La vectorisation par comptage d'occurrences des mots . 694
2.2.2 La vectorisation avec TF-IDF 696
2.2.3 La vectorisation avec N-Gram 698
2.2.4 Feature Engineering sur des documents 699
3. Exemple complet pour la détection des spams 700
3.1 Installation de la NLTK 701
3.2 Modèle de détection de spams 702
4. Conclusion 709

Annexe
La programmation orientée objet avec Python

1. Programmation orientée objet avec Python 711
   1.1 Pourquoi la programmation orientée objet? 711
   1.2 Classes et objets 713
      1.2.1 Définir une classe 713
      1.2.2 La fonction __init__ 714
      1.2.3 Instanciation d'un objet 716
      1.2.4 Les attributs d'un objet 717
      1.2.5 Les méthodes d'objet 719
      1.2.6 Les attributs de classe 721
      1.2.7 Les méthodes de classe 722
      1.2.8 Les méthodes statiques 724
      1.2.9 Sécuriser les attributs 725
   1.3 L'héritage 729
      1.3.1 L'héritage simple 729
      1.3.2 L'héritage multiple 733
   1.4 Les classes abstraites 735
   1.5 Les interfaces 737
   1.6 Les méthodes spéciales 741
      1.6.1 Afficher un objet avec la fonction print() 741
      1.6.2 Personnaliser les accès aux attributs d'une classe 744
      1.6.3 Vérifier la validité d'un attribut 745
      1.6.4 Comparer deux objets 747
      1.6.5 Rendre les objets callable 748
2. Les modules 749
   2.1 Importer des modules 750
   2.2 Le module principal 753
3. Pour aller plus loin avec Python 756



Avant-propos

1. Pourquoi ce livre ?

Ce livre est une simple tentative de réponse aux questions et aux constatations telles que :

- Je suis en reconversion professionnelle, comment m'initier au domaine du Machine Learning ?
- Je débute ma thèse doctorale dans le domaine de l'apprentissage automatique et je souhaite pratiquer les algorithmes du Machine Learning via des exemples concrets.
- Je souhaite bien démarrer en Machine Learning, mais je ne sais pas par où commencer ! Quelles sont alors les compétences nécessaires pour devenir Data Scientist ?
- Quels sont les livres qui pourront m'aider pour apprendre le Machine Learning ?

Évidemment, il est très difficile d'apporter une réponse qui soit tout à fait satisfaisante à ces questions. Néanmoins, ce livre a été rédigé dans l'optique d'apporter éventuellement un début de réponse à ces interrogations pour ainsi aider un débutant en Machine Learning à acquérir quelques compétences fondamentales et pratiques pour bien commencer à travailler dans la Data Science.

Il existe plusieurs livres intéressants sur le sujet du Machine Learning. En revanche, le lecteur novice est souvent confronté à la multiplicité des compétences requises et préalables pour une compréhension profonde des concepts qui y sont abordés.

En effet, pour acquérir une compréhension approfondie et pouvoir développer des applications dans le domaine de la Data Science, il est nécessaire d'avoir des compétences sur les cinq axes suivants :

1. Maîtriser un langage de programmation tel que le langage R ou le langage Python.

2. Avoir des connaissances avancées dans le domaine des statistiques, notamment dans les statistiques descriptives et les statistiques inférentielles.

3. Comprendre le mécanisme de fonctionnement des algorithmes tels que les algorithmes de régression linéaire, les algorithmes de Random Forest, les algorithmes comme k-means, les algorithmes de type SVM et les algorithmes de type réseaux de neurones, etc.

4. Comprendre et avoir une expérience dans la mise en pratique des algorithmes du Machine Learning dans des domaines tels que la reconnaissance d'objets à partir d'images ou le traitement automatique du langage, etc.

5. Savoir interpréter et restituer les résultats des algorithmes du Machine Learning afin de les expliquer pour les interlocuteurs métier.

Par conséquent, le lecteur novice est souvent amené à trouver des réponses aux questions telle que : par où commencer ? Quels livres lire ? Quelle formation en ligne serait intéressante pour lui ? Quel(s) langage(s) adopter ? Quelles connaissances acquérir ? Etc.

Bien sûr, comme pour tous les domaines de la science, il est toujours intéressant et préférable de lire plusieurs livres pour se former à la Data Science. Cependant, pour le débutant qui est perdu entre tous les livres disponibles sur les statistiques, sur les langages de programmation et sur le Machine Learning, cela peut être intéressant de lire un ouvrage qui concentre les connaissances nécessaires. C'est justement l'objectif de ce livre.

© Editions ENI - All rights reserved

Un ouvrage dédié au langage Python permettra au lecteur de maîtriser ce langage dans son ensemble et pouvoir développer tout type d'applications. Mais si l'unique but du lecteur est de pratiquer les algorithmes du Machine Learning avec Python, alors cela peut être inutile pour lui d'étudier des frameworks de développement d'applications web ou du développement de jeux vidéo.

Également, lorsque nous sommes débutants en Data Science, le fait d'entamer ce domaine par la lecture d'ouvrages sur lès statistiques et d'autres livres sur le Machine Learning peut parfois être difficile, car chaque auteur écrit avec un style et un format différent, ce qui peut induire au lecteur non-initié des difficultés inutiles.

Cet ouvrage a été rédigé dans le but de permettre au lecteur débutant d'acquérir les connaissances suivantes en un seul livre :

1. Comprendre et pratiquer le langage Python.
2. S'initier aux statistiques descriptives et inférentielles.
3. Aborder les notions fondamentales et avancées des grands algorithmes du Machine Learning.
4. S'initier au Deep Learning et au traitement automatique du langage naturel.

Le souhait et le seul but de l'auteur à travers ce livre sont qu'à la suite de sa lecture, vous puissiez sans difficulté lire d'autres livres pour aller plus loin dans votre quête de connaissances.

2. À qui s'adresse ce livre ?

Ce livre s'adresse à toute personne souhaitant acquérir des connaissances approfondies dans le domaine du Machine Learning, mais également à toute personne désireuse de s'initier au langage Python et aux statistiques descriptives et inférentielles.

Parmi les profils de lecteurs qui peuvent trouver un intérêt dans ce livre, nous pouvons citer :

- Les étudiants dans le domaine de l'intelligence artificielle et en particulier les étudiants dans le domaine de la Data Science. Ces lecteurs trouveront dans ce livre des explications approfondies relatives aux notions du Machine Learning. Ils pourront également s'appuyer sur les chapitres 2, 3, 4 et 5 afin d'acquérir des connaissances approfondies dans l'utilisation des outils techniques nécessaires à la pratique de la Data Science. Ces outils sont le langage Python avec ses bibliothèques NumPy et Pandas ainsi qu'une initiation à l'utilisation de l'environnement Jupyter. Outre ces connaissances techniques, des notions avancées sur les statistiques seront abordées et plusieurs cas pratiques seront expliqués tout au long de ce livre. À la suite de sa lecture, des connaissances avancées seront acquises et qui sont relatives aux algorithmes du Machines Learning tels que la régression linéaire simple et multiple, la régression logistique, les arbres de décision et l'algorithme Random Forest, l'algorithme k-means, l'algorithme Support Vector Machine (SVM), la méthode d'analyse en composantes principales, les réseaux de neurones, le Deep Learning et le traitement automatique du langage.
- Les personnes en reconversion professionnelle trouveront dans ce livre un point d'entrée à la Data Science où sont concentrées les connaissances nécessaires qui vont permettre de développer aisément ses propres modèles.
- Les Data Scientists pourront trouver dans ce livre une occasion de revoir les concepts fondamentaux à la base des algorithmes du Machine Learning.
- Les formateurs et enseignants peuvent trouver dans ce livre des ressources utilisables comme supports de cours et d'exercices et de travaux pratiques.

© Editions ENI - All rights reserved

3. Comment est organisé ce livre ?

Pour permettre d'acquérir progressivement les connaissances relatives aux concepts abordés, ce livre est organisé en cinq grandes parties comme suit :

1. La première partie (La Data Science - Concepts généraux) qui correspond au premier chapitre aborde les concepts les plus élémentaires de la Data Science. Ce chapitre permet d'avoir une vue globale de ce domaine et de comprendre les objectifs visés avec les algorithmes du Machine Learning. Ainsi, la lecture de cette première partie vise trois objectifs principaux. Le premier objectif est la compréhension des notions de base de la Data Science. Le deuxième objectif est d'acquérir la terminologie ou le jargon utilisé pour désigner certains concepts du Machine Learning. Le troisième objectif est de comprendre les métriques de performance qui permettent de mesurer l'efficacité des modèles.

2. La deuxième partie (Outils techniques de la Data Science - Python, NumPy, Pandas et Jupyter) est constituée des chapitres 2, 3, 4, et 5. Cette partie vise l'acquisition de connaissances purement techniques nécessaires, entre autres, pour la pratique de la Data Science. En plus d'apprendre à programmer avec le langage Python, ces chapitres permettent aussi d'aborder les deux fameuses bibliothèques NumPy et Pandas. Également, cette partie permet au lecteur de découvrir l'environnement Jupyter, qui est l'un des environnements les plus utilisés par les Data Scientists.

3. La troisième partie (Les statistiques), qui est constituée de l'unique chapitre 6, permet d'aborder des notions élémentaires et avancées sur les statistiques descriptives et les statistiques inférentielles.

4. La quatrième partie (Les grands algorithmes du Machine Learning) permet d'étudier de manière avancée des algorithmes du Machine Learning. Elle occupe la plus grande partie de ce livre et elle est constituée des sept chapitres allant du chapitre 7 au chapitre 13. Chacun de ces chapitres aborde un des algorithmes majeurs du Machine Learning. Ainsi, le lecteur abordera les algorithmes de régression linéaire et de régression polynomiale, les algorithmes de régression logistique, les arbres de décision et Random Forest, les algorithmes de type k-means, les algorithmes Support Vector Machine (SVM), la procédure d'analyse en composantes principales (ACP) et les réseaux de neurones.

Chacun de ces algorithmes sera abordé en trois phases dont la complexité est progressive : la première phase consiste à comprendre l'algorithme étudié d'un point de vue purement intuitif, la deuxième permet de comprendre l'algorithme étudié d'un point de vue formel et la troisième phase permet de mettre en pratique l'algorithme étudié à travers un exemple concret. Cette dernière phase permet également de découvrir différentes fonctionnalités de la bibliothèque Scikit-learn.

5. La cinquième partie (Le Deep Learning et le traitement automatique du langage), constituée des deux derniers chapitres 14 et 15, permet d'aborder les sujets du Deep Learning et du traitement automatique du langage naturel. Le Deep Learning sera abordé via des exemples concrets en utilisant la bibliothèque TensorFlow 2. Également, des notions élémentaires sur le sujet du traitement automatique du langage naturel seront abordées et un exemple complet sera développé en utilisant la bibliothèque NLTK.

4. Comment lire ce livre ?

La lecture de ce livre peut bien sûr être adaptée en fonction du profil du lecteur. Pour le profil novice à la programmation et à la Data Science, l'auteur recommande de lire tous les chapitres dans l'ordre.

De manière générale, chaque chapitre peut être lu indépendamment des autres chapitres. Par exemple, un enseignant qui souhaite préparer un cours sur les algorithmes SVM ou un cours sur l'ACP peut lire directement le chapitre 11 ou le chapitre 12, car l'auteur a fait des efforts particuliers afin de rendre la lecture individuelle des chapitres possible.

Dans tous les chapitres, l'auteur a essayé d'intégrer des exemples concrets afin de rendre palpables certains concepts théoriques qui sont réputés difficiles à comprendre sans la pratique.

© Editions ENI - All rights reserved

5. Quels sont les prérequis pour la lecture de ce livre ?

Ce livre a été pensé dans l'optique d'être lu par un novice en programmation et en Data Science. Théoriquement, un niveau Terminal en mathématiques est suffisant afin de comprendre la plupart des notions abordées dans ce livre. Cependant, la compréhension des sections où sont abordées des formulations mathématiques de certains algorithmes nécessite d'avoir un niveau en mathématiques plus ou moins équivalent à un niveau Bac+3.

6. Qui est l'auteur ?

Après avoir obtenu le diplôme d'ingénieur en informatique à l'université Mouloud Mammeri à Tizi-Ouzou/Algérie et le diplôme de Master 2 en intelligence artificielle à l'université Paris V (René Descartes), Madjid Khichane a poursuivi ses études doctorales en Intelligence artificielle en collaboration avec la société IBM et l'université Claude Bernard (Lyon I). Cette thèse doctorale a donné naissance à des innovations algorithmiques dans le domaine de l'apprentissage par renforcement qui sont publiées dans des conférences internationales de premier niveau.

Parmi ces publications :

Conférence	CPAIOR - 7th International Conference on Artificial Intelligence and Operations Research Techniques in Constraint Programming for Combinatorial Optimization Problems
Titre	Strong Combination of Ant Colony Optimization with Constraints Programming Optimization
Auteurs	Madjid Khichane, Patrick Albert, Christine Solnon
Lieux/lien	Italie| http://cpaior2010.ing.unibo.it/

Conférence	Lion 3 Learnig and Intelligent OptimizatioN
Titre	An ACO-based Reactive Framework for Ant Colony Optimization: First Experiments on Constraint Satisfaction Problems
Auteurs	Madjid Khichane, Patrick Albert, Christine Solnon
Lien	Italie| http://intelligent-optimization.org

Conférence	ANTs 2008 - Sixth International Conference on Ant Colony Optimization and Swarm Intelligence
Titre	Integration of ACO in a Constraint Programming Language
Auteurs	Madjid Khichane, Patrick Albert, Christine Solnon
Lien	Bruxelles| http://iridia.ulb.ac.be/ants2008/

Madjid Khichane est également l'auteur des deux livres suivants :

1. Data Science avec Microsoft Azure - Maîtrisez le Machine Learning sur Cortana Intelligence Suite - 14 mars 2018.

2. Microsoft Bot Framework - Maîtrisez le développement de chatbots avec les services cognitifs d'Azure - 10 juillet 2019.

© Editions ENI - All rights reserved

7. Remerciements

La rédaction de cet ouvrage a été une aventure aussi excitante qu'enrichissante. Mais sans doute n'y serais-je pas parvenu sans l'aide de mes proches.

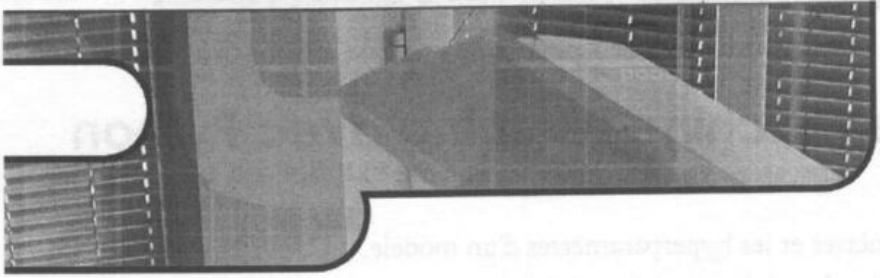
Je remercie mon épouse et mon papa pour leurs encouragements, leurs relectures et leurs corrections. Un grand merci pour mes filles Séléna et Alicia pour leur soutien, chacune à sa façon. Je remercie ma mère et mon père, mes frères et ma sœur pour leurs encouragements incessants tout au long de mon parcours.

Merci à Patrick Albert d'avoir accepté de préfacer ce livre, mais surtout un grand merci à lui et à Christine Solnon pour m'avoir ouvert la porte du monde de l'intelligence artificielle lorsqu'en 2007 ils ont accepté ma candidature pour ce stage sur l'optimisation combinatoire avec les colonies de fourmis.

Pour ne risquer d'oublier personne, je remercie tous ceux qui ont contribué à ce livre soit par leurs remarques et relectures, soit par nos échanges sur le sujet de l'intelligence artificielle.

Je tiens aussi à remercier l'équipe des Éditions ENI pour leur patience, leur écoute attentive, leur compréhension, leur soutien et leurs encouragements durant toute la durée de la rédaction de ce livre. Un grand merci à Manon Raffin, Julie Potier, Juliette Py et Jérôme Gabillaud ainsi qu'au relecteur pour la qualité de son travail.

Je remercie le lecteur, à qui je souhaite une bonne lecture et duquel j'espère avoir des retours pour des améliorations dans de potentielles futures éditions.



Partie 1
La Data Science - Concepts généraux

Chapitre 1-1
La Data Science

1. Objectif du chapitre

Le Machine Learning propose un cadre théorique et algorithmique pour modéliser des phénomènes ou des systèmes afin de comprendre leur fonctionnement pour ainsi anticiper leur comportement. L'objectif de ce premier chapitre est d'introduire les concepts généraux du Machine Learning et de la Data Science.

Après la lecture de ce chapitre, même si certains concepts ne sont pas clairs, sachez que des éclaircissements seront apportés dans les chapitres suivants. Le lecteur est d'ailleurs invité à revenir à ce chapitre pour faire le lien entre ces concepts et les cas concrets abordés dans les autres chapitres. En effet, ce premier chapitre servira de base aux suivants, de sorte qu'après sa lecture, vous pourrez directement aborder n'importe quel autre chapitre du livre.

À la fin de ce chapitre, le lecteur aura abordé :

- les étapes fondamentales d'une expérimentation Machine Learning,
- les types de données manipulées en Machine Learning,
- les problèmes liés aux données fréquemment rencontrées,
- les grandes parties d'un projet en Data Science,
- la classification des algorithmes en Machine Learning,
- la notion de surapprentissage,

- les paramètres et les hyperparamètres d'un modèle,
- les données de validation et la validation croisée,
- les métriques de performances des modèles de régression, de classification et de clustering.

2. L'objectif recherché en Machine Learning

Les phénomènes étudiés en Machine Learning peuvent être considérés comme des systèmes qui, en fonction du contexte de leur environnement, présentent un comportement particulier en vue de certains événements.

La figure 1-1 ci-dessous montre le schéma général d'un système qui prend en entrée un ensemble de valeurs, $X = (x_1, x_2, \ldots, x_m)$, puis il réalise un traitement spécifique en utilisant un ensemble de paramètres, $F(x_1, x_2, \ldots, x_m|P)$, et enfin, ce système donne un résultat y en sortie. De tels systèmes avec des entrées, un traitement et des sorties, sont rencontrés partout dans la vie de tous les jours.

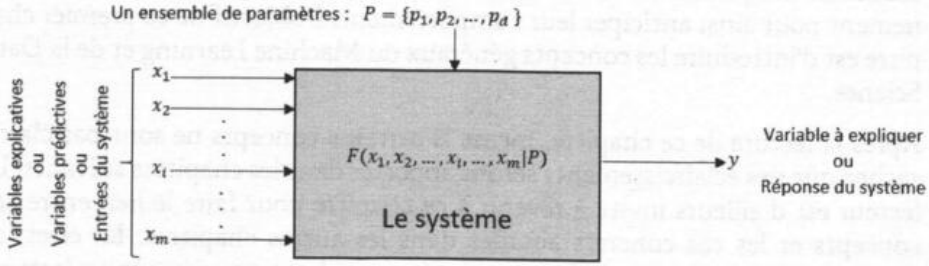


Figure 1-1 : un système avec entrées, traitement et sorties

Généralement, les problèmes concernés par la thématique de Machine Learning sont justement ces systèmes pour lesquels un ensemble d'entrées et de sorties est connu, mais où le traitement effectué pour transformer une entrée en une sortie n'est pas connu. Le but du Machine Learning est idéalement de comprendre ce traitement, ou du moins de trouver le traitement le plus proche possible du vrai traitement effectué au sein du système.

© Editions ENI - All rights reserved

Bien sûr, l'hypothèse du Machine Learning est que ce traitement existe et que le fait d'avoir le résultat y à partir de l'entrée $X = (x_1, x_2, \ldots, x_m)$ n'est pas le fruit du pur hasard.

En d'autres termes, l'objectif du Machine Learning est d'étudier les entrées $X = (x_1, x_2, \ldots, x_m)$ et les sorties y d'un système pour deviner ou approcher le traitement $F(x_1, x_2, \ldots, x_m|P)$. Une fois que ce traitement est deviné ou approché, pour les prochaines entrées du système, le résultat à la sortie sera anticipé avant même qu'il se produise. C'est en cette dernière proposition que le caractère prédictif en Machine Learning trouve sa justification.

Exemple de modélisation

Sans plus tarder, cette section propose de montrer à travers un exemple la démarche globale de l'approche Machine Learning. Cette approche sera affinée au fur et à mesure que vous avancerez dans ce chapitre.

Supposons qu'un hôpital dispose d'un jeu de données composé de 1000 observations sur des patients différents. Supposons aussi que chacune de ses observations renseigne les informations suivantes pour un patient donné :

- `Fumeur` : lorsque cette information vaut `Oui`, elle indique que le patient est un fumeur ou un ex-fumeur. Cette information vaut `Non` si le patient n'a jamais fumé.
- `ECG` : cette information vaut `Oui` si le patient a eu un électrocardiogramme anormal au cours des deux dernières années. Elle vaut `Non` sinon.
- `Age` : cette information indique l'âge du patient.
- `Pression` : cette information vaut `Oui` si le patient a enregistré au cours des deux dernières années une pression artérielle anormale. Elle vaut `Non` sinon.
- `IMC` : cette information indique l'indice de masse corporelle du patient.
- `Cardiaque` : cette information vaut `Oui` si le patient est atteint d'une maladie cardiaque, elle vaut `Non` si le patient est sain.

Au final, cet hôpital dispose d'un tableau de 1000 lignes sous la forme suivante :

	Fumeur	ECG	Age	Pression	IMC	Cardiaque
$Patient_1$	Oui	Non	45	Oui	22	Non
$Patient_2$	Non	Oui	75	Oui	29	Oui
$Patient_i$	Oui	Oui	18	Non	23	Non
$Patient_{1000}$	Non	Non	73	Non	23	Non

Figure 1-2 : exemple de tableau de données traitées en Machine Learning

Ce tableau de 1000 observations montre entre autres que certains patients sont atteints d'une maladie cardiaque et d'autres non. L'une des questions qu'on peut se poser par rapport à ces observations est la suivante :

Peut-on distinguer les patients atteints d'une maladie cardiaque des patients sains en se basant sur les informations fournies par les cinq variables `Fumeur`, `ECG`, `Age`, `Pression` et `IMC` ?

Voilà une question faite pour être traitée par un algorithme de Machine Learning !

Pour cet exemple, l'objectif d'un algorithme de Machine Learning est de construire un modèle capable de prendre en entrée les mesures sur les cinq variables `Fumeur`, `ECG`, `Age`, `Pression` et `IMC` d'un patient et de prédire si ce patient est atteint d'une maladie cardiaque ou pas.

Dans ce cas, les cinq variables `Fumeur`, `ECG`, `Age`, `Pression`, `IMC` sont communément appelées les variables prédictives et la variable `Cardiaque` est appelée la variable cible ou la variable à prédire.

Pour parvenir à construire un tel modèle prédictif en Machine Learning, un Data Scientist commence par l'analyse et la préparation des 1000 observations précédentes, puis il applique un algorithme de Machine Learning sur une partie de ces 1000 observations pour construire un modèle ; à la fin, il évalue la pertinence de ce modèle sur l'autre partie de ces observations. Si les résultats sont satisfaisants, ce modèle sera déployé en production pour effectuer de futures prédictions.

© Editions ENI - All rights reserved

Supposons que parmi les 1000 patients, 800 patients sont sains et 200 patients sont atteints d'une maladie cardiaque. Pour construire un modèle prédictif de la maladie cardiaque, le Data Scientist divise les 1000 observations en deux groupes. Le premier groupe va contenir par exemple 750 observations, parmi lesquelles 150 patients malades et 600 patients sains. Le deuxième groupe va contenir 50 patients malades et 200 patients sains. Ensuite, le Data Scientist déroule un algorithme de Machine Learning sur les observations du premier groupe pour apprendre un modèle. Ce modèle va être utilisé pour faire des prédictions sur les 250 observations du deuxième groupe afin d'évaluer sa pertinence.

En résumé, le premier groupe de 750 observations sera utilisé pour entraîner un modèle. Le deuxième groupe de 250 observations sera utilisé pour évaluer le pouvoir prédictif de ce modèle.

En effet, une fois qu'un modèle a été entraîné sur les patients du premier groupe, ce modèle peut être utilisé pour prédire pour chaque patient du deuxième groupe s'il est malade ou pas. Comme la réponse à cette dernière question est connue pour chacun des 250 patients de ce deuxième groupe, on peut mesurer la force de prédiction de notre modèle.

Dans cette démarche, le Data Scientist a utilisé les données disponibles à l'hôpital, les 1000 observations, pour construire et évaluer un modèle. En fonction des résultats de l'évaluation de ce modèle sur les observations du deuxième groupe, l'équipe médicale prend la décision d'utiliser ce modèle pour effectuer les prédictions sur les futurs patients.

Les 1000 observations de cet exemple forment ce qui est généralement appelé la base de données d'apprentissage. Le premier groupe des 750 patients est communément appelé les données d'entraînement et le deuxième groupe des 250 patients forme les données de test.

Dans cet exemple, dans un souci pédagogique, la démarche proposée est beaucoup plus simplifiée par rapport à une démarche réelle. En effet, dans la réalité, les choses sont un peu plus compliquées. Justement, la section suivante présente la démarche de construction d'un modèle en Machine Learning avec plus de détails.

3. Une expérimentation Machine Learning

Comme toute expérimentation scientifique, réussir une expérimentation en Machine Learning nécessite d'appliquer une méthodologie avec beaucoup de rigueur. Comme évoqué à la section précédente, l'objectif du Machine Learning est de répondre à un besoin métier en partant des données à la recherche du modèle prédictif le plus robuste possible. Pour y parvenir, il est nécessaire de se poser plusieurs questions, définir un ensemble d'étapes à suivre, fixer l'objectif de chaque étape, faire plusieurs tests avec plusieurs hypothèses différentes, et surtout, ne pas hésiter à se reposer des questions quitte à tout remettre en cause à n'importe quel moment de l'expérimentation. La figure 1-3 suivante propose une succession d'étapes à suivre pour réaliser une expérimentation en Machine Learning.

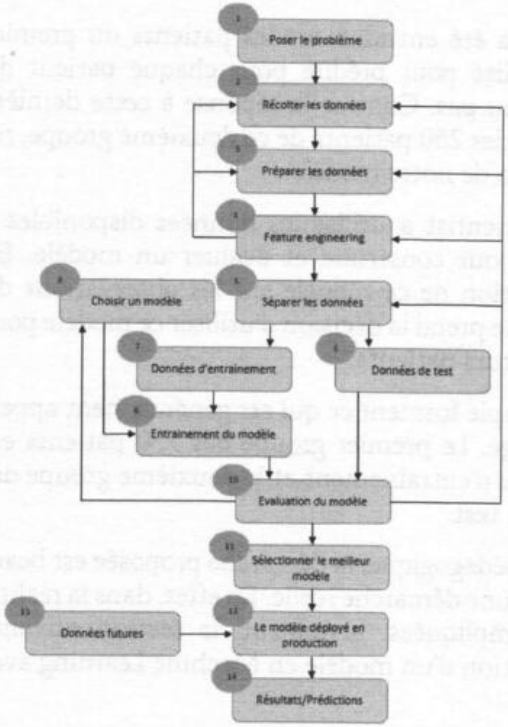


Figure 1-3 : les étapes d'une expérimentation Machine Learning

© Editions ENI - All rights reserved

Ces étapes peuvent être définies comme suit :

1. Poser le problème : c'est le début de toute l'histoire. Le besoin peut venir des équipes opérationnelles et/ou métier qui constatent une optimisation possible des processus de production ou autres processus. Le besoin peut également être mis en avant par les équipes Data Science qui démontrent l'intérêt d'imbriquer .de l'intelligence artificielle dans certains processus métier. Cette démonstration est généralement concrétisée par des expérimentations et/ou des POC (*Proof Of Concept*) réalisés sur des données fictives ou réelles, cela dépend des budgets alloués.

   Une fois qu'un besoin est défini, l'équipe Data Science intervient pour discuter de la valeur ajoutée de l'intelligence artificielle pour répondre à ce besoin et, le cas échéant, discuter des méthodes et des solutions à mettre en œuvre pour parvenir aux résultats escomptés.

   Ainsi, l'équipe Data Science encadre et formalise le problème et revient vers les équipes métier avec la proposition d'une solution, un planning de réalisation, une équipe avec diverses compétences, une estimation du coût du projet, etc.

   Cette première étape est cruciale pour un projet en Data Science car, au-delà du besoin métier qui doit être bien cerné et bien formalisé, plusieurs questions doivent être posées pour lesquelles les réponses ne seront pas forcément établies de façon claire avant d'aller plus loin dans le projet. Parmi ces questions, on peut citer : quelles sont les sources de données les plus pertinentes à utiliser dans l'étude ? Est-ce que les données dont on dispose sont suffisantes, en taille et en qualité, pour aboutir au résultat attendu ? Comment déployer la solution en production ? Quelles sont les compétences nécessaires pour réaliser le projet de bout en bout ? Cette dernière question est légitime, car un projet en Data Science ne se résume pas seulement au développement d'un modèle prédictif. Il est nécessaire de mettre en place tout un environnement dans lequel le modèle prédictif joue certainement le rôle central, mais il ne sera pas la seule composante de cet environnement. La section suivante donne plus de détails sur les différentes composantes d'une solution complète relative à un projet en Data Science.

Une autre question, la plus importante de toutes celles à se poser lors de cette première étape, est d'étudier l'impact de la solution à mettre en œuvre sur le métier et les équipes de l'entreprise. En effet, les solutions de type Machine Learning, et de façon plus générale les solutions qui relèvent du domaine de l'intelligence artificielle, impliquent des changements parfois majeurs dans le mode de fonctionnement de l'entreprise et des changements dans les habitudes de travail des salariés. Pour cette dernière raison en particulier, des questions d'ordre éthique et social doivent être posées et étudiées de façon approfondie lors de cette étape avant de faire un Go/No-Go sur un projet.

2. Récolter les données : lors de cette étape, l'équipe en charge du projet part en quête des données. Il faut commencer par la sélection des différentes sources des données et charger ces données dans un Dataset. C'est ce Dataset, une fois chargé, qui va constituer les données d'apprentissage vues à la section précédente. Cette deuxième étape peut être parsemée de plusieurs difficultés, mais celles souvent rencontrées sont la diversité des données et la diversité des compétences techniques qu'il est nécessaire de posséder dans une équipe afin de récupérer ces données.

3. Préparer les données : les données récoltées à l'étape précédente sont dans un état brut, c'est-à-dire qu'elles sont telles qu'elles étaient dans les sources de données d'origine. Dans cet état, généralement, ces données ne sont pas adaptées à une expérimentation en Machine Learning. En effet, le plus souvent, les données brutes présentent des anomalies. Par exemple, certaines variables peuvent avoir des valeurs non renseignées pour certaines observations ; certaines observations peuvent être dupliquées ; les valeurs de certaines variables peuvent être erronées, etc. Justement, cette étape de préparation des données sert à traiter ces anomalies, c'est pourquoi cette étape et aussi appelée l'étape de nettoyage des données. La partie préparation des données est l'une des étapes les plus importantes dans une expérience Machine Learning ; la section Préparation des données y est donc consacrée entièrement.

© Editions ENI - All rights reserved

4. Feature engineering : à ce stade, les données ont été récupérées et nettoyées de toutes les anomalies détectées. On peut donc désormais leur appliquer les algorithmes du Machine Learning pour construire des modèles prédictifs. Cependant, les informations véhiculées par les données brutes peuvent ne pas être suffisantes pour qu'un algorithme, aussi efficace soit-il, aboutisse à un modèle prédictif puissant. En effet, le Data Scientist ajoute généralement de nouvelles variables aux variables brutes. Ces nouvelles variables combinées avec les anciennes variables permettent de ressortir des informations qui étaient cachées jusqu'alors. Cette tâche qui consiste à ajouter de nouvelles variables pour enrichir l'ensemble des données et rendre les algorithmes plus puissants peut se révéler difficile. Certains la considèrent comme une tâche artistique ! Le choix de ces nouvelles variables dépend essentiellement de deux facteurs : l'expérience du Data Scientist et l'expertise du domaine du problème en cours de traitement.

   Il faut noter que les trois dernières étapes, à savoir la récolte des données, la préparation des données et le Feature engineering, sont celles qui prennent le plus de temps dans une expérimentation Machine Learning. Généralement, ces trois étapes s'accaparent 60 % à 80 %, voire même dans certains cas 90 % du temps total d'un projet.

5. Séparer les données : il s'agit lors de cette étape de séparer réellement les données en deux groupes. Un groupe sert à entraîner un modèle, puis, une fois que ce modèle est prêt, le deuxième groupe de données est utilisé pour tester l'efficacité de ce modèle.

Comme abordé à la section précédente, les données d'apprentissage sont constituées d'un ensemble d'observations pour lesquelles les résultats sont connus. Si on veut évaluer un modèle prédictif, il suffit de suivre les quelques étapes suivantes :

- Diviser les données en deux groupes, soit $groupe_1$ et $groupe_2$. Pour rappel, les deux groupes disposent d'un ensemble d'observations avec les résultats associés à ces mêmes observations.
- Entraîner un modèle en utilisant *groupe*1.
- Évaluer le modèle entraîné à l'étape précédente sur les données de $groupe_2$. Lors de cette étape, le modèle entraîné est tout simplement utilisé pour prédire les résultats des observations de $groupe_2$. Et comme les résultats de $groupe_2$ sont connus, l'évaluation du modèle consiste à comparer ses prédictions sur les observations de $groupe_2$ avec les résultats connus de ce même groupe.

Cette étape peut paraître très simple à réaliser, mais dans les faits, c'est une tâche qui demande beaucoup de réflexion ainsi que plusieurs essais. Par exemple, il faudrait réfléchir à la taille de la portion des données d'apprentissage à utiliser pour l'entraînement et penser à une stratégie de répartition des observations entre les deux groupes.

L'objectif de séparer les données d'apprentissage est donc d'entraîner puis d'évaluer un modèle. Il existe une méthode très répandue qui s'appelle la validation croisée et qui consiste à séparer l'ensemble des données d'apprentissage en plusieurs groupes (plus que deux groupes). Parmi tous ces groupes, un seul sera mis de côté pour le test ultime. Tous les autres groupes seront, un par un, utilisés comme groupe de test et les autres tous ensemble comme un seul groupe d'entraînement. La section Validation croisée abordera plus en détail le concept de la validation croisée.

© Editions ENI - All rights reserved

Également, la procédure de séparation des données en deux groupes $groupe_1$ et $groupe_2$ peut être complétée avec la séparation de $groupe_1$ en deux sous-groupes $groupe_{1,1}$ et $groupe_{1,2}$. Le groupe $groupe_{1,1}$ sera utilisé comme données d'entraînement et $groupe_{1,2}$ sera utilisé comme données de validation. Dans les prochaines sections Validation croisée et Données d'entraînement, de validation et de test, nous reviendrons sur l'intérêt dans une expérimentation en Machine Learning des trois groupes $groupe_{1,1}$ pour l'entraînement, $groupe_{1,2}$ pour la validation et $groupe_2$ pour le test.

6. Données de tests : à cette étape, il n'y a pas d'actions à réaliser à proprement parler, si ce n'est celle de marquer la variable à prédire pour qu'à l'étape 10 cette variable soit utilisée pour le Scoring et l'évaluation du modèle.
7. Données d'entraînement : à cette étape, les données sont prêtes à être utilisées par un algorithme pour construire un modèle prédictif. Par contre, dans certains cas, certaines opérations peuvent être nécessaires avant le lancement d'un algorithme.

   Par exemple, si la variable à prédire présente un déséquilibre, alors des données artificielles peuvent être ajoutées lors de cette étape. La notion de variable déséquilibrée sera traitée à la section Préparation des données de ce chapitre.

   Également, si on souhaite mettre en place la validation croisée, c'est à cette étape qu'il faut séparer les données en plusieurs groupes. De plus, si nous ne souhaitons pas utiliser la validation croisée, mais uniquement un groupe de données de validation, alors c'est à cette étape que nous devons sélectionner les données de validation à mettre de côté. Pour rappel, la validation croisée et les données de validation seront expliquées plus tard dans ce chapitre.

8. Choisir un modèle : cela revient à choisir l'algorithme à utiliser pour construire le modèle. Comme nous l'évoquerons dans la section Les algorithmes du Machine Learning, le choix de l'algorithme se fera en fonction de l'objectif de l'expérimentation en cours. Dans la réalité, plusieurs algorithmes seront testés et comparés avant de sélectionner le meilleur d'entre eux. La section Métriques de performance aborde ces métriques qui permettent justement de comparer les modèles en fonction de leurs résultats.
9. Entraînement du modèle : c'est à cette étape qu'un algorithme de Machine Learning est appliqué sur les données d'entraînement. Vous remarquez qu'avant d'arriver à cette étape, plusieurs étapes ont été nécessaires.
10. Évaluation du modèle : cette étape est composée de deux tâches. La première consiste à utiliser le modèle entraîné à l'étape 9 pour effectuer des prédictions sur les données de test. Ainsi, pour chaque observation des données de test, une prédiction de la variable à prédire sera réalisée. La deuxième tâche consiste à comparer les valeurs prédites avec les vraies valeurs observées. Traditionnellement, la première tâche s'appelle le Scoring du modèle et la deuxième se nomme l'évaluation du modèle.

    Les étapes 8, 9 et 10 seront répétées autant que possible, mais bien sûr en utilisant des algorithmes différents, ou tout simplement en changeant les paramètres d'un même algorithme. En effet, comme vous allez le découvrir dans les chapitres suivants, les résultats d'un algorithme peuvent significativement être différents en fonction de ses paramètres.

    Outre le choix de l'algorithme et de ses paramètres, plusieurs autres facteurs peuvent influencer significativement la recherche du meilleur modèle. En réalité, quasiment toutes les étapes précédentes influencent le résultat de l'algorithme choisi. Cependant, les étapes de préparation de données et de Feature engineering sont généralement les étapes les plus influentes.

© Editions ENI - All rights reserved

Lors de la préparation des données, plusieurs décisions importantes seront prises. Par exemple, si certaines observations présentent des valeurs manquantes pour une variable numérique donnée, alors soit ces observations seront écartées de l'expérimentation, soit ces valeurs manquantes seront remplacées chacune par une autre valeur telle que la moyenne de cette même variable. Le choix de la valeur qui viendra remplacer des valeurs manquantes a bien sûr une influence sur les résultats obtenus. Toutes les opérations de préparation des données qui seront vues à la section Préparation des données auront une influence sur les résultats.

Lors de l'étape de Feature engineering, des variables supplémentaires seront ajoutées aux variables déjà existantes pour rendre les données, d'une certaine façon, plus intelligentes. Ces opérations qui consistent à ajouter ou à faire apparaître des informations utiles dans les données permettent généralement d'améliorer la qualité des modèles obtenus.

Également, lors de l'étape de la séparation des données, le choix des données qui seront utilisées pour la phase d'entraînement du modèle a bien évidemment une influence sur le résultat de l'algorithme qui sera exécuté sur ces données. Il faut noter dès maintenant qu'un modèle robuste en Machine Learning est un modèle pour lequel les performances ne changent pas de façon significative lorsque les données d'entraînement changent.

En résumé, la recherche du meilleur modèle passe par l'exécution d'un même algorithme plusieurs fois avec des paramètres différents ; le test de plusieurs algorithmes différents sur le même jeu de données ; le test de plusieurs techniques de préparation des données, notamment plusieurs techniques de remplacement des valeurs manquantes ; la recherche de la combinaison des variables les plus influentes sur les résultats ; le test de plusieurs stratégies de séparation de données, etc.

11. Sélectionner le meilleur modèle : une fois que les étapes 2 à 10 sont répétées plusieurs fois avec des paramètres différents, comme mentionné ci-dessus, le meilleur modèle est choisi. La comparaison des modèles est basée sur des métriques de performance en fonction du type du problème en cours de traitement. Ces métriques seront présentées en détail à la section Métriques de performance de ce chapitre.

12. Le modèle déployé en production : le modèle sélectionné à l'étape précédente est déployé en production de sorte qu'il puisse être exploité par des applications tierces afin d'effectuer des prédictions sur des données fraîches qui ne sont pas connues de la base d'apprentissage.
13. Données futures : cette étape concerne l'envoi des données nouvellement observées en vue d'effectuer des prédictions sur la variable à prédire. Il peut s'agir lors de cette étape de simples interrogations du modèle déployé précédemment pour obtenir des prédictions ou il peut s'agir d'un environnement, plus au moins complexe, à mettre en place afin de récolter et de traiter les données futures, puis d'interroger le modèle déployé précédemment.
14. Résultats/Prédictions : cette étape consiste à renvoyer les résultats des prédictions à une application tierce ou de déposer les résultats dans une base de données ou encore d'afficher ces résultats dans un tableau de bord.

Les étapes présentées ci-dessus ont décrit les tâches nécessaires pour partir d'un jeu de données et aboutir à un modèle prédictif. Vous avez sans doute remarqué que le concept de la donnée est au centre de la discussion dans toutes ces étapes. En effet, comme vous allez le découvrir au fur et à mesure que vous avancerez dans ce livre, les modèles issus des algorithmes du Machine Learning sont dérivés immédiatement des données. La section suivante présente les types de données manipulés par les algorithmes du Machine Learning.

3.1 Types de données

En Machine Learning, les données utilisées sont des échantillons de population. Ces échantillons sont composés de plusieurs observations. Chaque observation est un ensemble de valeurs associées à un ensemble de variables. Les méthodes qui seront appliquées pour réaliser une étude sur ces variables dépendent de la nature de ces mêmes variables.

© Editions ENI - All rights reserved

Le plus souvent, un même échantillon de données est défini par différents types de variables. Il existe deux catégories de variables : les variables quantitatives et les variables qualitatives définies comme suit :

- Variables quantitatives : elles peuvent prendre des valeurs numériques ordonnées et sur lesquelles les opérations mathématiques telles que l'addition et la multiplication ont un sens. De façon générale, les variables quantitatives servent à mesurer des quantités. Il existe deux types de variables quantitatives :
  - Les variables quantitatives continues : une variable quantitative est continue si elle peut prendre des valeurs réelles définies dans un intervalle. En d'autres termes, l'ensemble des valeurs possibles pour une variable quantitative continue est un ensemble de valeurs réelles non dénombrable. Par exemple, la variable qui permet de mesurer la température ambiante est une variable quantitative continue puisqu'elle peut prendre un nombre de valeurs réelles infini et non dénombrable.
  - Les variables quantitatives discrètes : une variable quantitative est discrète si elle ne peut prendre que des valeurs numériques définies dans un ensemble fini ou définies dans un ensemble infini, mais dénombrable.
- Variables qualitatives : les variables qualitatives portent des informations exprimées généralement sous forme littérale et non pas numérique, car elles ne correspondent pas à des grandeurs mathématiques. Il existe deux types de variables qualitatives :
  - Les variables nominales : une variable nominale peut prendre des valeurs définies par un ensemble de noms. Par exemple, la variable qui renseigne le sport préféré d'une population. Cette variable peut prendre des valeurs comme `Handball`, `Football`, etc.
  - Les variables ordinales : une variable ordinale peut être vue comme un mix entre les variables quantitatives discrètes et les variables nominales. En effet, une variable ordinale peut prendre ses valeurs dans un ensemble de noms sur lequel une relation d'ordre peut être définie. Par exemple, la variable qui renseigne l'avis des clients sur un produit peut être définie par l'ensemble des valeurs {`Satisfait`, `Moyen`, `Pas satisfait`} avec la relation d'ordre `Satisfait > Moyen > Pas satisfait`.

Les variables qualitatives sont souvent codifiées par des valeurs numériques. Par exemple, la variable qui modélise l'avis des clients peut prendre les valeurs {1,2,3} pour désigner respectivement {`Satisfait, Moyen, Pas satisfait`}. L'erreur à ne pas commettre est d'appliquer les opérations arithmétiques, comme l'addition et la multiplication, sur ces valeurs. En effet, dans notre exemple, 1+1=2, ce qui correspond à `Satisfait + Satisfait = Moyen` ; cela n'a pas de sens.

3.2 Préparation des données

La préparation des données est une étape importante de toute expérimentation en Machine Learning. Elle peut prendre la plus grande partie du temps d'un projet en Data Science. Concrètement, cette étape consiste à analyser et détecter d'éventuelles anomalies dans les données récoltées et d'apporter une correction à ces anomalies lorsque c'est possible.

Le plus souvent, les données préexistent au projet de Data Science. Ces données n'ont pas été créées à la base pour être utilisées par un programme de Machine Learning. Les observations qui présentent des anomalies ne devraient pas être systématiquement ignorées, car elles peuvent être porteuses d'informations précieuses. La suite de cette section aborde quelques anomalies les plus fréquemment rencontrées sur les données :

- Valeurs manquantes : il arrive souvent que pour un sous-ensemble d'observations, certaines variables ne soient pas renseignées. C'est ce qu'on appelle les valeurs manquantes. Ces valeurs manquantes peuvent être remplacées par des valeurs synthétiques supposées être proches des valeurs qui auraient dû être présentes.

  Plusieurs techniques de remplacement des valeurs manquantes existent. Par exemple, une valeur manquante d'une variable numérique peut être remplacée par la moyenne, par le maximum, par le minimum ou par la médiane de cette même variable.

  Dans l'exemple de la figure 1-2, la variable `Age` peut ne pas être renseignée pour certains patients. Dans ce cas, on peut par exemple remplacer ces valeurs manquantes par la moyenne de l'âge des patients pour qui cette variable est renseignée.

© Editions ENI - All rights reserved

Choisir de remplacer les valeurs numériques manquantes d'une variable par la moyenne ou la médiane de cette variable est un peu arbitraire et peut induire un biais dans la construction des modèles prédictifs, notamment lorsque le nombre de ces valeurs manquantes est important.

Si une variable numérique présente des valeurs manquantes, on peut exécuter une régression linéaire sur cette variable en fonction des autres variables pour remplacer les valeurs manquantes. Cette dernière technique est peut-être compliquée pour le remplacement des valeurs manquantes, mais peut s'avérer très performante car les nouvelles valeurs ne seront pas choisies de façon aléatoire comme pour les techniques de remplacement par la moyenne ou la médiane par exemple.

Un point important à prendre en compte lorsqu'on remplace les valeurs manquantes est que souvent, dans les jeux de données récoltés, les valeurs manquantes sont signalées par une valeur spéciale. En effet, les valeurs manquantes ne sont pas forcément représentées par des cases vides dans les tableaux. Dans notre exemple sur les patients, la variable `IMC` peut contenir des valeurs égales à zéro ou des valeurs négatives pour signaler les patients pour lesquels l'indice de masse corporelle n'est pas renseigné. L'erreur à ne pas faire est donc de considérer ces valeurs spéciales comme des valeurs normales.

- Détection des valeurs aberrantes : les valeurs aberrantes présentes dans un jeu de données peuvent être des valeurs normales, mais extrêmes, ou tout simplement des valeurs erronées. Un moyen très simple de détecter ces valeurs extrêmes est d'utiliser la boîte à moustache sur chaque variable numérique.

  C'est au Data Scientist de choisir quelle décision prendre face à des valeurs aberrantes. Ces valeurs aberrantes peuvent bien sûr influencer le comportement des algorithmes du Machine Learning en introduisant un biais dans leurs raisonnements. Généralement, la prise de la décision d'éliminer ou de garder ces valeurs aberrantes dépend de la fréquence de ces valeurs aberrantes et de la taille du jeu de données en cours d'analyse.

  Par exemple, dans le tableau de données de la figure 1-2, si certaines valeurs de la variable `Age` sont égales ou supérieures à 130, alors il faudrait d'abord vérifier, si cela est possible, si ces valeurs sont correctes ou pas.

Puis, si ces valeurs ne sont pas des erreurs de renseignement et en fonction de leur nombre, le Data Scientist peut décider d'inclure ou pas les patients en question dans l'étude.

- Normalisation des données : les différentes variables dans les jeux de données sont généralement exprimées dans des échelles différentes. Ces différences d'échelles introduisent naturellement un biais lorsqu'une comparaison est effectuée entre des quantités; ou des mesures, dans lesquelles plusieurs variables sont impliquées.

  Dans l'exemple sur les données de patients, supposons qu'un patient a `IMC=16` et `Age=110`, donc c'est un patient anorexique avec un âge très avancé, et supposons qu'un deuxième patient a `IMC=41` et `Age=10`, il s'agit donc d'un patient avec une obésité morbide et un âge très jeune. Si l'on compare ces deux patients avec la formule : *IMC+0.25 Age* on obtient 43.5 pour les deux patients ! Cela signifie qu'un algorithme qui utilise cette formule pour comparer les patients entre eux considérera que nos deux patients sont égaux alors qu'en réalité ces deux patients présentent des caractéristiques complètement opposées. Il est donc important de procéder à la normalisation des variables afin qu'elles soient représentées dans des échelles comparables avant d'effectuer ce type de comparaison.

- Gestion des variables déséquilibrées : une variable est dite déséquilibrée lorsqu'elle peut prendre des modalités sous-représentées par rapport à d'autres modalités.

  Dans l'exemple de la figure 1-2, supposons que le nombre de patients qui présentent une maladie cardiaque soit de 30, alors que le nombre de patients sains est de 970 sur le total des 1000 patients. Dans ce cas-là, la modalité `Oui` pour la variable `Cardiaque` est sous-représentée.

  Les données qui présentent ce problème de la sous-représentation de certaines modalités met un frein aux algorithmes pour apprendre la structure générale des observations qui donne comme résultat cette modalité sous-représentée.

  Des techniques existent pour pallier ce problème de déséquilibre de variables. Elles consistent généralement à générer des observations artificielles à partir des observations existantes. Ces techniques diffèrent par la manière avec laquelle ces nouvelles observations sont générées.

© Editions ENI - All rights reserved

Outre celles citées ci-dessus, d'autres opérations de préparation de données existent et qui sont aussi couramment rencontrées. Par exemple :

- Le formatage des données qui consiste généralement en un changement du type de variables.
- L'agrégation des modalités qui ont le même sens ; par exemple, pour la variable `Cardiaque`, de l'exemple sur les patients, qui peut prendre la valeur `Yes` pour certains patients pour dire qu'ils sont malades et prendre la valeur `Oui` pour d'autres patients pour dire la même chose ; dans ce cas, il serait intéressant de ne considérer que la valeur `Yes` ou la valeur `Oui` pour tous les patients malades.
- L'élimination des observations dupliquées, qui sont complètement identiques.

4. Cycle de vie d'un projet Data Science

Les sections précédentes ont montré qu'une expérimentation en Machine Learning consiste à construire un modèle prédictif en se basant sur un ensemble de données d'apprentissage. Une fois qu'un modèle est construit, il doit être déployé en production pour que d'autres applications puissent l'interroger. Ce déploiement en production est souvent l'objectif final d'une équipe de Data Scientists intervenant sur un projet.

Cependant, il existe des projets de Data Science pour lesquels le déploiement du modèle en production n'est pas une fin en soi. Ce déploiement de modèle en production ne représente qu'une partie d'un projet plus grand que le projet de réalisation d'une expérimentation Machine Learning.

En effet, de plus en plus, les entreprises souhaitent s'équiper de solutions End-to-End, c'est-à-dire des solutions qui prennent en compte toutes les composantes d'un projet où les données sont capturées et suivies en temps pseudo-réel jusqu'à leur transformation en action dite intelligente.

La figure 1-4 ci-après montre à quoi peut ressembler une solution End-to-End. La première chose à remarquer sur cette figure est que le modèle prédictif déployé en production ne représente effectivement qu'une des composantes du projet. C'est l'ensemble de tous ces composants qui forme la solution complète, et l'équipe Data Science peut intervenir sur tous ces composants.

Ces solutions End-to-End de Data Science peuvent être composées des éléments suivants (voir sur la figure 1-4) :

- Sources de données : il s'agit de tout type d'objet capable d'envoyer des données sur un serveur distant.
- Concentrateurs d'événements : lorsque les sources de données sont multiples, afin d'éviter les problèmes liés à l'ingestion de gros volumes de données, des concentrateurs d'événements peuvent être mis en place pour réceptionner les données en temps pseudo-réel.
- Traitement des flux de données : des agrégations sur les flux de données qui arrivent en continu au niveau des concentrateurs d'événements peuvent être nécessaires afin d'avoir des données brutes harmonisées.
- Cluster Big Data : généralement, le stockage des données qui arrivent en continu nécessite de grands volumes de stockage et de traitement.
- Le modèle déployé en production : c'est le modèle qui résulte d'une expérimentation Machine Learning.
- Base de données : souvent, une base de données relationnelle classique est utilisée pour le stockage des résultats des prédictions sur les données fraîches.
- Visualisation/Tableau de bord : pour exploiter les résultats de prédiction d'un modèle sur les données fraîches, il serait intéressant d'avoir une vue simple de ces résultats sur une interface ergonomique.
- ETL/Orchestrateur d'événements : ce composant sert à exécuter toutes les transformations nécessaires sur les données pour les rendre compréhensibles par le modèle prédictif, il interroge le modèle prédictif sur ces données et, enfin, il envoie les résultats de prédiction à tous les outils qui souhaitent en bénéficier tels que des tableaux de bord.

© Editions ENI - All rights reserved

Remarque

Pour rappel, il est à noter que toutes les données traitées par tous les composants de la figure 1-4 concernent les données que nous avons appelées les données futures dans les sections précédentes, et donc il ne s'agit pas des données d'apprentissage. Les données d'apprentissage sont les données déjà présentes dans l'entreprise et qui sont utilisées pour la construction du modèle lorsque l'expérimentation Machine Learning a eu lieu.

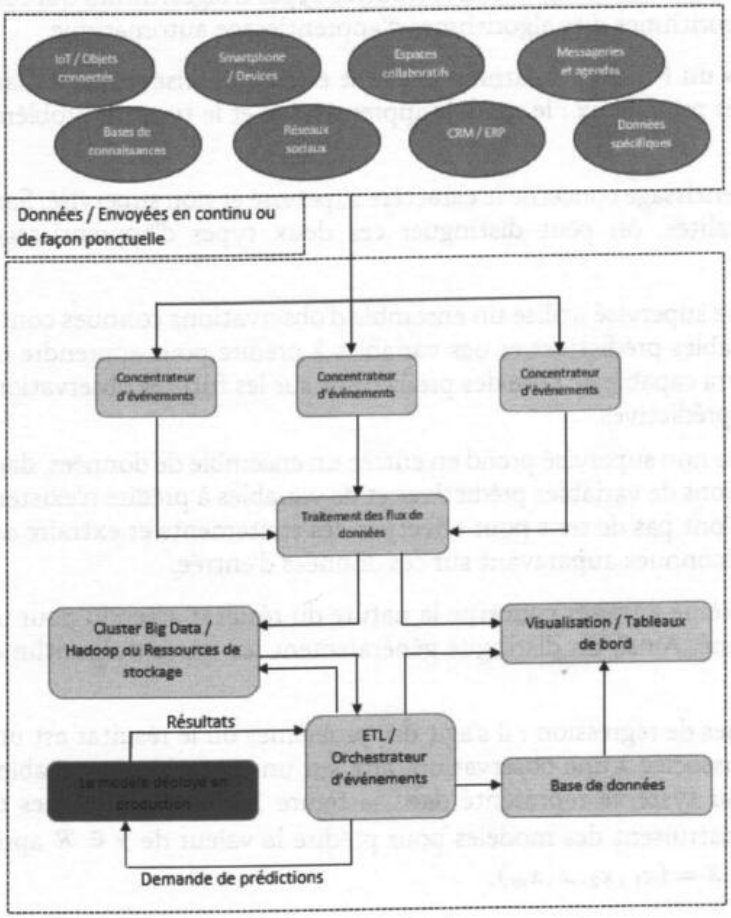


Figure 1-4 : les composants d'une solution End-to-End

5. Les algorithmes du Machine Learning

Au début de ce chapitre, les algorithmes de Machine Learning ont été introduits comme étant des algorithmes qui cherchent à comprendre le fonctionnement des systèmes qui donnent des résultats après avoir effectué un traitement sur leurs entrées. Ces algorithmes produisent donc des modèles prédictifs. Bien que ce type d'algorithme caractérise de façon générale le domaine du Machine Learning, il existe d'autres types d'algorithmes qui font partie de ces algorithmes dits algorithmes d'apprentissage automatique.

Les algorithmes du Machine Learning peuvent être catégorisés en se basant sur deux critères principaux : le mode d'apprentissage et le type du problème traité.

Le mode d'apprentissage concerne le caractère supervisé et non supervisé. Sans perte en généralités, on peut distinguer ces deux types d'apprentissages comme suit :

- Un algorithme supervisé utilise un ensemble d'observations connues contenant des variables prédictives et des variables à prédire pour apprendre un modèle qui sera capable de faire des prédictions sur les futures observations des variables prédictives.
- Un algorithme non supervisé prend en entrée un ensemble de données, dans lequel les notions de variables prédictives et de variables à prédire n'existent pas ou elles n'ont pas de sens pour effectuer des traitements et extraire des structures méconnues auparavant sur ces données d'entrée.

Le type de problème à traiter concerne la nature du résultat attendu pour un algorithme donné. Ainsi, on distingue généralement les types d'algorithmes suivants :

- Les algorithmes de régression : il s'agit des problèmes où le résultat est une valeur réelle associée à une observation faite sur un ensemble de variables. Par rapport au système représenté dans la figure 1-1, les algorithmes de régression construisent des modèles pour prédire la valeur de $y \in \mathcal{R}$ après avoir observé $X = (x_1, x_2, \ldots, x_m)$.

© Editions ENI - All rights reserved

- Les algorithmes de classification : ces algorithmes tentent d'affecter une étiquette ou une classe à une observation faite sur un ensemble de variables. L'ensemble des valeurs possibles pour cette étiquette est dénombrable et défini au préalable.
- Les algorithmes de clustering : ces algorithmes tentent de trouver des regroupements des observations, de telle sorte que les observations qui se ressemblent soient mises dans les mêmes groupes et celles qui divergent soient mises dans des groupes différents. Ces algorithmes sont très sensibles à la façon de calculer les distances entre les observations.
- Les algorithmes d'optimisation combinatoire : ce sont les algorithmes qui tentent de trouver l'optimum d'une fonction en utilisant des heuristiques et/ou des métaheuristiques. Il existe plusieurs métaheuristiques basées sur l'apprentissage automatique et qui permettent de guider un algorithme pour se diriger vers les endroits les plus intéressants de l'espace de recherche des solutions.

Loin d'être exhaustif, le tableau ci-après donne des exemples d'algorithmes et leurs caractéristiques associées :

Apprentissage	Famille d'algorithmes	Algorithme
Supervisé	Régression	Régression linéaire simple et polynomiale
	Régression + Classification	Régression logistique
		Random Forest
		Réseaux de neurones
		Support Vector Machine (SVM)
Non supervisé	Clustering	K-Means
	Optimisation	Algorithmes génétiques
		Optimisation par Colonies de fourmis

Figure 1-5 : exemples d'algorithmes d'apprentissage automatique

Remarque

De façon générale, les algorithmes d'optimisation utilisant des techniques issues de l'intelligence artificielle ne sont pas mentionnés comme des algorithmes faisant partie du domaine de Machine Learning. En effet, le fonctionnement de ces algorithmes d'optimisation, comme ceux cités dans le tableau ci-dessus, n'est pas basé sur un ensemble de données d'apprentissage. Cependant ils utilisent l'apprentissage automatique pour se diriger dans l'espace de recherche de façon intelligente. C'est cette dernière caractéristique qui permet de les mentionner dès qu'on parle d'algorithmes d'apprentissage automatique.

6. Le problème de surapprentissage

Le problème du surapprentissage peut être rencontré par tous les algorithmes du Machine Learning. On dit d'un modèle qu'il souffre du problème de surapprentissage lorsque ses performances sont très satisfaisantes sur les données d'entraînement, mais que ses capacités de prédiction sur les données de test et sur les données de production sont très faibles. La figure ci-dessous illustre ce problème :

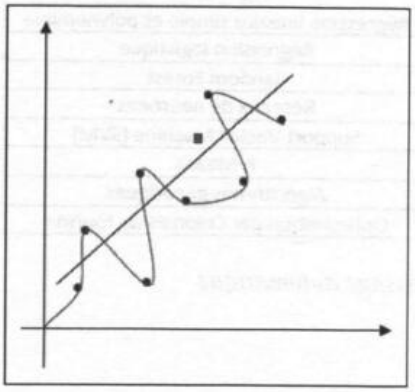


Figure 1-6 : le problème du surapprentissage

© Editions ENI - All rights reserved

Sur cette figure, les points représentent les données d'entraînement et le carré un exemple de donnée de test. Bien que le modèle représenté par la fonction en courbes soit capable de prédire avec exactitude toutes les données d'entraînement, c'est bien le modèle représenté par la droite qui est meilleur puisqu'il possède une meilleure capacité de prédiction sur les données de test. Le modèle représenté par la fonction en courbes présente un risque élevé de s'éloigner de façon importante des données de test alors que le modèle représenté par la droite a plus de chances d'être plus proche de ces données de test. Ici, le modèle représenté par la fonction en courbes a surappris les données d'entraînement.

Généralement, on admet qu'un modèle est en situation de surapprentissage lorsque ses performances sont très satisfaisantes sur les données d'entraînement, mais médiocres sur les données de test.

7. Les paramètres et les hyperparamètres

Il est important de distinguer les paramètres d'un modèle de ses hyperparamètres. Sans perte de généralités, les paramètres d'un modèle sont les variables qui le définissent alors que les hyperparamètres d'un modèle sont des variables utilisées pour trouver des valeurs intéressantes pour les paramètres de ce même modèle !

En d'autres termes, les paramètres d'un modèle sont les variables pour lesquelles un algorithme de Machine Learning tente de fixer des valeurs. Les valeurs de ces paramètres, une fois fixées, seront utilisées directement dans le calcul des prédictions réalisées avec ce modèle. Les hyperparamètres sont les variables utilisées par un algorithme de Machine Learning afin de fixer les valeurs des paramètres d'un modèle. Les valeurs des hyperparamètres ne sont donc pas utilisées lors du calcul des prédictions avec un modèle. Nous pouvons dire que les hyperparamètres sont eux-mêmes des paramètres qui définissent la manière avec laquelle un algorithme devrait fonctionner lors de la recherche des valeurs intéressantes pour les paramètres d'un modèle.

Par exemple, imaginons un algorithme $\mathcal{A}$ qui utilise deux hyperparamètres n_1 et n_2 afin de fixer les valeurs des deux facteurs a et a_0 du modèle linéaire simple $F(x_i) = ax_i + a_0$ avec $a, \in \mathbb{R}, a_0 \in \mathbb{R}$ (le chapitre La régression linéaire et polynomiale est entièrement dédié aux modèles de régression linéaire).

Comme nous allons le voir en détail lorsque nous aborderons les algorithmes du Machine Learning, dans la formule $F(x_i) = ax_i + a_0$, la variable x_i est associée aux données de la base de données d'apprentissage et les deux facteurs a et a_0 sont les paramètres de ce modèle pour lesquels un algorithme de Machine Learning va tenter de trouver les valeurs optimales en utilisant les données d'apprentissage, chose que nous n'allons pas faire dans cet exemple.

Au début de cet algorithme imaginaire $\mathcal{A}$, les deux hyperparamètres sont fixés de manière aléatoire comme suit : n_1=3 et n_2=4. L'algorithme $\mathcal{A}$ affecte au paramètre a la moyenne de n_1 nombres générés de manière aléatoire. C'est-à-dire que l'algorithme $\mathcal{A}$ affecte au paramètre a la moyenne de trois nombres générés aléatoirement, soit par exemple les nombres 10, 20 et 30. Ce qui nous donne une valeur de $a = \frac{10+20+30}{n_1} = \frac{10+20+30}{3} = 20$. De la même manière que pour fixer la valeur de a, l'algorithme $\mathcal{A}$ affecte au paramètre a_0 la moyenne de n_2 nombres générés de manière aléatoire, soit les nombres 35, 5, 42 et 18. Ce qui nous donne une valeur de $a_0 = \frac{35+5+42+18}{n_2} = \frac{35+5+42+18}{4} = 25$.

L'exécution de l'algorithme avec les valeurs de n_1=3 et n_2=4 aboutit donc au modèle $F(x_i) = 20x_i + 25$.

Avec ce modèle, pour la prédiction de la valeur de F pour une valeur quelconque de x_i, les valeurs des facteurs a=20 et a_0=25 sont utilisées de manière directe, alors que lors d'une telle prédiction les valeurs des hyperparamètres n_1=3 et n_2=4 ne sont pas utilisées, c'est ce qui distingue les paramètres des hyperparamètres. En revanche, ce sont bien les valeurs n_1=3 et n_2=4 qui ont permis l'obtention des valeurs a=20 et a_0=25 et si on exécute l'algorithme $\mathcal{A}$ avec des valeurs différentes pour n_1 et n_2, alors les deux paramètres a et a_0 vont avoir des valeurs différentes, avec une probabilité élevée puisque leurs valeurs dans cet exemple sont fixées suivant une procédure aléatoire.

Dans cet exemple, l'algorithme utilise les hyperparamètres n_1 et n_2 afin de trouver les valeurs des paramètres a et a_0.

© Editions ENI - All rights reserved

Remarque

Afin de nous focaliser sur les notions d'hyperparamètres et de paramètres, dans cet exemple nous avons intentionnellement omis l'utilisation de données d'apprentissage pour fixer les paramètres du modèle F, car ces deux concepts d'hyperparamètres et de paramètres n'ont pas de lien direct avec les données d'apprentissage !

En résumé, les hyperparamètres sont des paramètres exploités par un algorithme afin de fixer les valeurs des paramètres d'un modèle.

8. Validation croisée

Jusqu'à présent, les sections précédentes ont montré que la construction d'un modèle prédictif est basée sur la séparation en deux groupes de l'ensemble des observations d'apprentissage : un groupe pour l'entraînement du modèle et un deuxième pour le test des performances de ce modèle. Une fois qu'un modèle est construit, l'objectif est de le déployer en production pour qu'il puisse être interrogé sur des observations futures. Le problème posé par une telle démarche de construction de modèle est que nous n'avons pas une idée précise sur les erreurs de prédiction attendues en production, ou, du moins, nous n'avons qu'une idée très approximative de ces erreurs. De plus, comme l'a montré la section Le problème de surapprentissage, un modèle déployé en production peut être affecté par le problème de surapprentissage.

En effet, une fois qu'un modèle est construit, même s'il présente des performances intéressantes, il est très important de se poser les deux questions suivantes :

1. Les performances du modèle sur l'ensemble des données de test ne seraient-elles pas le fruit du surapprentissage dû à une forte ressemblance entre ces données de test et les données d'entraînement ?
2. Quel est le taux d'erreur de ce modèle auquel nous pouvons nous attendre en production sur les futures observations ?

La validation croisée tente justement de répondre à ces deux questions. Elle permet d'un côté d'estimer le taux d'erreur du modèle en production sur les futures observations, et de l'autre côté de mettre en évidence si le modèle est en situation de surapprentissage sur les données d'entraînement.

Plusieurs méthodes permettent de mettre en place une validation croisée sur un modèle. Sans être exhaustive, la liste ci-dessous donne les méthodes de validation croisées les plus répandues :

- La méthode Hold Out : cette méthode consiste à séparer les données d'apprentissage en deux groupes, comme nous l'avons fait dans les sections précédentes. En revanche, jusqu'à présent, le premier groupe est utilisé uniquement pour entraîner le modèle et le deuxième groupe pour mesurer les performances de ce modèle. Avec cette méthode, les performances du modèle seront également mesurées sur l'ensemble des données d'entraînement.

  Ainsi, les mesures de performance sur les données d'entraînement vont être comparées avec les mesures de performance sur les données de test. Si les performances du modèle changent drastiquement entre les données d'entraînement et les données de test, alors il y a un risque de surapprentissage élevé, et de plus le modèle aura un comportement non contrôlé en production.

  En revanche, si les performances sur les données d'entraînement et sur les données d'apprentissage sont comparables, alors nous pouvons supposer que le modèle va se comporter de façon similaire sur les observations futures et sur les observations d'apprentissage.

  Cette dernière hypothèse doit être avancée avec beaucoup de précautions, car la méthode de sélection des observations dans les deux groupes, le groupe d'entraînement et le groupe de test, peut avoir une forte influence sur la comparaison des performances sur ces deux groupes.

  Pour mettre en évidence l'influence de la méthode de sélection sur la comparaison des performances entre les données d'entraînement et les données de test, il suffit de prendre un sous-ensemble d'observations du groupe d'entraînement et de les affecter au groupe de test, puis de recalculer le modèle et de comparer ses performances sur les deux groupes.

© Editions ENI - All rights reserved

De façon plus générale, si le fait de transférer des observations d'un groupe à un autre change de façon significative la comparaison des performances du modèle sur les deux groupes, alors la méthode Hold Out n'est pas suffisante pour avoir une idée sur le taux d'erreur du modèle en production.

- La méthode k-fold : l'idée de base de cette méthode est de diviser l'ensemble des données d'apprentissage en k partitions de même taille.

  Par exemple, pour une base d'apprentissage avec 1000 observations et k=5, on obtient 5 partitions avec 200 observations chacune. Puis, tour à tour, ces cinq partitions vont être utilisées comme groupe de test et le reste comme groupe d'entraînement.

  Pour cet exemple avec cinq partitions, le modèle va être entraîné et testé cinq fois. Supposons que les cinq partitions sont nommées : $partition_1$, $partition_2$, $partition_3$, $partition_4$ et $partition_5$.

  À la première itération, $partition_1$ est sélectionnée comme groupe de test et les quatre partitions $partition_2$, $partition_3$, $partition_4$ et $partition_5$ forment le groupe d'entraînement. Une fois que le modèle est entraîné sur le groupe d'entraînement, il est testé sur la partition $partition_1$. Les mesures de performances sur cette dernière partition seront conservées.

  À la deuxième itération, la partition $partition_2$ est utilisée comme groupe de test et les quatre partitions $partition_1$, $partition_3$, $partition_4$ et $partition_5$ forment le groupe d'entraînement. Le modèle est entraîné sur le groupe d'entraînement et testé sur la partition $partition_2$. Ce processus est rejoué en sélectionnant à chaque fois une partition différente pour le groupe de test.

  À la fin, une fois que chacune des partitions a joué le rôle de groupe de test, la mesure de performance globale est la moyenne des mesures de performance des cinq itérations.

  Avec cette méthode, la mesure de performance globale obtenue à la fin de toutes les itérations est plus stable et plus robuste par rapport à la méthode Hold Out, car chaque observation a été utilisée pour l'entraînement et le test du modèle. Par conséquent, le taux des erreurs de prédiction sur les futures observations peut être estimé par le taux d'erreur moyen des taux d'erreurs obtenus après chaque itération.

- La méthode Leave One : cette méthode ressemble à la méthode k-fold. La seule différence est qu'avec la méthode Leave One, le nombre de partitions est égal au nombre d'observations, c'est-à-dire que chaque observation constitue une partition à elle seule. Sinon, le reste de la démarche de cette méthode est semblable à la démarche de la méthode k-fold.

  Cette méthode peut s'avérer plus intéressante que les deux méthodes précédentes pour l'évaluation du taux d'erreur du modèle, mais elle peut également s'avérer très coûteuse en temps de calcul puisque le modèle sera entraîné un nombre de fois égal au nombre d'observations.

- La méthode Bootstrap : de façon générale, sur un ensemble d'apprentissage à N observations, cette méthode fonctionne en plusieurs itérations comme suit : à chaque itération, elle construit un ensemble d'apprentissage en sélectionnant N observations avec remise à partir de l'ensemble d'apprentissage d'origine. Puis l'entraînement du modèle est effectué sur la totalité des observations de cet ensemble construit et le test est effectué sur l'ensemble d'apprentissage d'origine. Ce processus est réitéré un certain nombre de fois. Le taux d'erreur du modèle est la moyenne des taux d'erreurs à chaque itération.

Remarque

Quelle que soit la méthode de validation croisée choisie, il ne faut pas oublier que l'objectif est d'une part de ne pas tomber dans le piège de surapprentissage, et d'autre part d'avoir une meilleure estimation du taux d'erreur du modèle qui sera déployé en production.

Remarque

Également, quelle que soit la méthode de validation croisée utilisée, il serait intéressant de garder un sous-ensemble d'observations de côté, qui ne seront jamais utilisées lors de l'entraînement du modèle, pour réaliser un test de modèle. Ce test servira comme une simulation de tests sur des données futures et permettra d'affirmer ou d'infirmer les conclusions suite à la validation croisée.

© Editions ENI - All rights reserved

9. Données d'entraînement, de validation et de test

La section précédente a montré qu'il est important de garder de côté un sous-ensemble des données d'apprentissage pour uniquement l'évaluation d'un modèle et les données de ce sous-ensemble ne doivent en aucun cas être utilisées lors de l'entraînement du modèle. En effet, le fait d'utiliser un sous-ensemble pour uniquement évaluer un modèle est la seule manière de simuler un environnement de production. Nous avons vu que cet ensemble de données est appelé l'ensemble des données de test.

En somme, l'ensemble des données que nous avons à notre disposition doit toujours être divisé en deux groupes de données : $groupe_1$ les données d'entraînement et $groupe_2$ les données de test. Donc, les données de $groupe_2$ ne doivent en aucun cas être utilisées dans une procédure d'entraînement d'un modèle. Les techniques de validation croisée vues dans la section précédente peuvent toutes être appliquées en utilisant uniquement les données d'entraînement, soit les données de $groupe_1$. Dans ce cas-là, les données choisies parmi les données de $groupe_1$ pour évaluer un modèle seront appelées les données de validation.

De façon plus générale, lors de la formation d'un modèle, il est très intéressant de diviser les données de $groupe_1$ en deux sous-groupes de données : les données d'entraînement et les données de validation, soient les deux sous-groupes $groupe_{1,1}$ pour les données d'entraînement et $groupe_{1,2}$ pour les données de validation.

De cette façon, nous pouvons lancer un algorithme avec des valeurs particulières pour les hyperparamètres de cet algorithme et entraîner un modèle avec les données de $groupe_{1,1}$. Ce modèle sera évalué sur les données de validation de $groupe_{1,2}$. En fonction des résultats de l'évaluation d'un modèle sur ces données de validation, nous pouvons réfléchir à de nouvelles valeurs pour les hyperparamètres de l'algorithme utilisé. Puis nous pouvons relancer le même algorithme que le précédent, mais en utilisant cette fois les nouvelles valeurs pour ses hyperparamètres, pour entraîner un modèle avec les données de $groupe_{1,1}$ et évaluer ce modèle sur les données de validation du groupe $groupe_{1,2}$.

Ainsi, nous pouvons répéter autant de fois que l'on souhaite, avec des valeurs différentes des hyperparamètres à chaque fois, l'exécution d'un même algorithme pour l'entraînement d'un modèle avec les données de $groupe_{1,1}$ et l'évaluation de ce modèle avec les données de validation de $groupe_{1,2}$.

Avec une telle démarche, bien que les données de validation, c'est-à-dire les données de $groupe_{1,2}$, n'ont pas été utilisées pour l'entraînement d'un modèle, elles auraient tout de même servi pour l'ajustement des hyperparamètres de l'algorithme utilisé ! De ce fait, le modèle obtenu peut être en situation de surapprentissage sur les données de $groupe_{1,2}$! En effet, même si les données de $groupe_{1,2}$ n'ont pas été vues par le modèle lors de son entraînement, elles auraient influencé nos choix des valeurs des hyperparamètres ! C'est bel et bien pour cette raison qu'il est important d'avoir le groupe de données de test $groupe_2$ pour l'évaluation d'un modèle, car non seulement ces données de test n'ont pas été vues par le modèle lors de son entraînement, mais en plus elles n'auraient pas servi pour l'ajustement des hyperparamètres !

En synthèse, les données d'entraînement servent pour l'ajustement des paramètres d'un modèle, les données de validation peuvent servir pour l'ajustement des hyperparamètres et enfin les données de test servent pour simuler les données de production.

10. Métriques de performance

Pour conserver un cadre formel cohérent tout au long de ce livre, les notations suivantes seront adoptées :

- $X = \{X_1, X_2, \ldots, X_i, \ldots, X_n\}$: l'ensemble des n observations de la base d'apprentissage. Chaque observation X_i est composée des valeurs prises par m variable explicatives. Ainsi, $X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,m})$ est l'observation i avec $x_{i,1}$ la valeur de la première variable explicative, $x_{i,2}$ la valeur de la deuxième variable explicative, et ainsi de suite.
- $Y = \{y_1, y_2, \ldots, y_i, \ldots, y_n\}$: l'ensemble des résultats obtenus sur les n observations de la base d'apprentissage, avec y_i le résultat de l'observation $X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,m})$.

© Editions ENI - All rights reserved

Par exemple y_1 est le résultat de l'observation $X_1 = (x_{1,1}, x_{1,2}, \ldots, x_{1,m})$.

- $Y^{futur} = \{y_{n+1}, y_{n+2}, \ldots\}$: l'ensemble des futurs résultats associés aux futures observations $X^{futur} = \{X_{n+1}, X_{n+2}, \ldots\}$. Par exemple, y_{n+1} est le résultat qui va être obtenu sur l'observation $X_{n+1} = (x_{n+1,1}, x_{n+1,2}, \ldots, x_{n+1,m})$.

Pour rappel, l'objectif des algorithmes du Machine Learning est de construire des modèles prédictifs en utilisant la base d'apprentissage, capables de prédire y_{n+1} lorsque X_{n+1} sera observé, puis prédire y_{n+2} lorsque X_{n+2} sera observé, et ainsi de suite.

En résumé, la base d'apprentissage peut être représentée comme suit :

$$X = \begin{pmatrix} x_{1,1}, x_{1,2} & \cdots & x_{1,m} \\ \vdots \quad \vdots & \ddots & \vdots \\ x_{n,1}, x_{n,2} & \cdots & x_{n,m} \end{pmatrix}, Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

Lorsque le nombre de variables explicatives m est égale à 1, pour simplifier la notation, l'indice des variables est omis et la base d'apprentissage est représentée par : $X = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$

- $\hat{Y} = \{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_i, \ldots, \hat{y}_n\}$: l'ensemble des prédictions obtenues avec un modèle sur les résultats de la base d'apprentissage. Ainsi, $\hat{y}_i$ est la valeur prédite par un modèle pour une observation $X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,m})$ alors que, pour rappel, la vraie valeur est y_i.
- Donc, pour dire qu'un modèle a fait une prédiction correcte pour l'observation $X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,m})$, il faudrait que $\hat{y}_i = y_i$.

Les mesures de performance d'un modèle en Machine Learning sont essentiellement basées sur la comparaison des vrais résultats Y et les prédictions $\hat{Y}$ sur les observations de la base d'apprentissage.

Cette comparaison entre les vrais résultats et les prédictions dépend du type du modèle choisi pour décrire la relation entre X et Y. Généralement :

- Pour les modèles de régression, les indicateurs utilisés sont :
  - L'erreur absolue moyenne MAE (*Mean Absolute Error*).
  - L'erreur absolue relative RAE (*Relative Absolute Error*).
  - La racine carrée de la moyenne du carré des erreurs RMSE (*Root Mean Squared Error*).
  - Le carré des erreurs relatives RSE (*Relative Squared Error*).
  - Le coefficient de détermination R^2.
- Pour les modèles de classification :
  - La matrice de confusion. À partir de cette matrice de confusion, plusieurs indicateurs de performance peuvent être calculés. Sans être exhaustifs, parmi ces indicateurs les plus utilisés se trouvent l'indicateur de sensibilité, appelé aussi le Recall, l'indicateur de précision, l'indicateur F1 Score.
  - La courbe ROC (*Receiver Operating Characteristic*).
- Pour les modèles de clustering : pour les modèles de clustering, les mesures de performances sont essentiellement basées sur des mesures indiquant le degré de rapprochement entre les éléments regroupés ensemble et le degré d'éloignement des éléments répartis dans des groupes différents.

La suite de cette section donne tous les détails de calcul de tous les indicateurs cités ci-dessus.

10.1 Métriques pour les problèmes de régression

Mesurer la performance d'un modèle de régression revient à comparer les observations connues de la variable à prédire Y avec les prédictions du modèle pour cette même variable Y.

Soit F la fonction correspondant à notre modèle de régression. Pour toute observation $X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,m})$ on obtient la prédiction :

$F(X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,m})|P) = \hat{y}_i$ (voir figure 1-1)

© Editions ENI - All rights reserved

Pour mesurer la précision de cette prédiction, il suffit de calculer l'écart entre $\hat{y}_i$ et la vraie valeur y_i qui se trouve dans la base de données de test. Cet écart s'appelle le résidu ou l'erreur de prédiction et il est noté généralement par : $\varepsilon = \hat{y}_i - y_i$

Ainsi, on peut définir :

– L'erreur absolue moyenne $\mathbf{MAE} = \frac{1}{n} \sum_{i=1}^{n} | \hat{y}_i - y_i |$

Cette quantité MAE mesure la moyenne des erreurs en valeur absolue sur les n observations qui forment la base de données d'apprentissage.

Cette mesure est intéressante pour comparer les performances de deux modèles pour le même problème.

En effet, supposons que deux algorithmes de régression ont été exécutés sur le même jeu de données pour résoudre le même problème. Alors, en se basant sur l'indicateur MAE, c'est l'algorithme qui obtient le plus petit MAE qui sera considéré comme l'algorithme le plus performant des deux.

L'inconvénient du MAE est qu'il n'est pas adapté pour comparer les performances de deux algorithmes exécutés sur des jeux de données différents.

Par exemple, considérons un premier groupe de cinq étudiants avec leurs notes moyennes résumées dans la première ligne du tableau de la figure 1-7 ci-après et un deuxième groupe également de cinq étudiants avec leurs notes moyennes résumées dans la première ligne du tableau de la figure 1-8 ci-après. Considérons maintenant les résultats de prédiction de ces moyennes par deux algorithmes, l'Algorithme 1 et l'Algorithme 2, résumées également dans les deuxièmes lignes de ces deux tableaux.

Moyenne Etudiant	14	15	17	14	12
Prédiction Algorithme 1	16	17	15	12	10

Figure 1-7 : résultats de prédiction de l'Algorithme 1

Moyenne Etudiant	5	2	4	1	3
Prédiction Algorithme 2	7	0	2	3	1

Figure 1-8 : résultats de prédiction de l'Algorithme 2

L'indicateur MAE pour les deux algorithmes est égal à 2. Si on se réfère uniquement à cet indicateur, alors les deux algorithmes ont les mêmes performances. Pourtant, l'Algorithme 2 prédit souvent le double ou le triple de la vraie moyenne alors que les prédictions de l'Algorithme 1 ne s'écartent des vraies moyennes qu'avec de faibles pourcentages.

- L'erreur relative absolue $RAE = \frac{1}{n}\sum_{i=1}^{n}\frac{|\hat{y}_i - y_i|}{|\bar{Y} - y_i|}$ avec $\overline{Y}$ la moyenne des vraies valeurs.

  Cette quantité donne le taux de déviation des prédictions par rapport aux vraies valeurs. Plus cette valeur est proche de zéro, meilleur est le modèle.

  Si on reprend l'exemple précédent des deux figures 1-7 et 1-8, le RAE pour l'Algorithme 1 est égal à 0,14, alors que pour l'Algorithme 2 il est égal à 0,91.

  Avec cet indicateur, l'Algorithme 1 est nettement meilleur puisqu'il regarde non seulement les écarts entre les prédictions et les valeurs observées, mais en plus il s'intéresse au taux de déviation entre les prédictions et la réalité.

  L'erreur relative est intéressante pour comparer les performances des modèles, mais malheureusement cette mesure présente certaines limites, à savoir :

  - Elle ne peut pas être utilisée avec des valeurs nulles.
  - Lorsque les écarts entre les valeurs prédites et les valeurs observées deviennent très importants, le taux de déviation peut dépasser la valeur 1.

- La racine carrée de la moyenne du carré des erreurs $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$

  Le RMSE met au carré les différences de prédiction pour ainsi mettre un accent encore plus fort sur les écarts entre les prédictions et les valeurs observées.

© Editions ENI - All rights reserved

- Le carré des erreurs relatives $RSE = \frac{\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}{\sum_{i=1}^{n}(\bar{Y} - y_i)^2}$

  Cette quantité donne le taux de déviation au carré des prédictions par rapport aux vraies valeurs.

- Le coefficient de détermination $R^2 = 1 - \frac{\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}{\sum_{i=1}^{n}(\bar{Y} - y_i)^2}$

Le coefficient de détermination permet de mesurer à quel point un modèle parvient à expliquer la variance associée à la variable à interpréter à partir des variables explicatives. Nous allons détailler ce coefficient de détermination au chapitre La régression linéaire et polynomiale.

10.2 Métriques pour la classification

Les algorithmes de classification supervisés utilisent une base de données d'apprentissage dans laquelle les classes sont connues pour ajuster les paramètres de leurs modèles. Puis ces modèles vont servir à prédire les classes des futures observations. Tout comme pour les modèles de régression, les modèles de classification doivent eux aussi être évalués et validés avant d'être utilisés en production pour prédire les classes de ces futures observations. La validation des modèles de classification se fait généralement en utilisant la matrice de confusion. La suite de cette section explique de manière théorique la matrice de confusion sur un problème de classification binaire, puis une généralisation de cette matrice de confusion sera donnée, et enfin un exemple de matrice de confusion binaire sera expliqué.

10.2.1 Matrice de confusion binaire

Afin de mieux comprendre l'intérêt d'utiliser les matrices de confusion, cette section commence par expliquer ce qu'est une matrice de confusion pour un problème de classification binaire.

Supposons que nous possédons une base de données de m observations et que chaque observation appartient soit à la classe "+", la classe des positifs, soit à la classe "−", la classe des négatifs. Comme nous l'avons vu précédemment dans ce chapitre, un algorithme de Machine Learning utilise un groupe d'observations pour apprendre les paramètres d'un modèle et un deuxième groupe d'observations de test pour évaluer les performances de ce modèle.

Pour un problème de classification, les résultats de l'évaluation d'un modèle peuvent être résumés comme dans le tableau suivant :

		Prédictions		
		"+"	"−"	
Observations	"+"	VP	FN	$VP + FN$
	"−"	FP	VN	$FP + VN$
		$VP + FP$	$FN + VN$	m_{test}

Figure 1-9 : matrice de confusion binaire

Avec m_{test} le nombre d'observations dans le groupe de données de test.

Ce tableau correspond à la forme générale de la matrice de confusion pour un problème de classification binaire. Les éléments de cette matrice sont définis comme suit :

- *VP* : le nombre des vrais positifs. Ce nombre correspond aux individus observés de la classe "+" et qui sont effectivement prédits de la classe "+" C'est le nombre d'individus positifs correctement prédits.
- *FP* : le nombre des faux positifs. Ce nombre correspond aux individus observés de la classe "+", mais prédits de la classe "−". C'est le nombre d'individus négatifs, mais faussement prédits positifs.
- *FN* : le nombre des faux négatifs. Comme pour *FP*, ce nombre correspond aux individus observés de la classe "+", mais prédits de la classe "−". C'est le nombre d'individus positifs, mais faussement prédits négatifs.
- *VN* : le nombre des vrais négatifs. Ce nombre correspond aux individus observés de la classe "−" et qui sont effectivement prédits de la classe "−". C'est le nombre d'individus négatifs correctement prédits.

En se basant sur ces définitions, le taux d'erreur du modèle peut être défini comme suit : $\frac{FP+FN}{m_{test}}$. Évidemment, plus cette quantité est grande, moins on fait confiance à ce modèle.

© Editions ENI - All rights reserved

Également, à partir de la matrice de confusion, on peut mesurer deux quantités que l'on retrouve dans quasiment toutes les études en Data Science lorsqu'il s'agit de modéliser un problème de classification :

- Le Recall, qui est égal à $\frac{VP}{VP+FN}$. Cette quantité correspond au taux des individus positifs détectés par le modèle. En d'autres termes, cette quantité sera égale à 1 si tous les individus qui sont réellement positifs sont classés positifs par le modèle.

Remarque

Attention, le Recall peut être égal à 1 même si certains individus qui sont réellement négatifs ont été classés positifs par un modèle, et également la précision peut être égale à 1 même si certains individus qui sont réellement positifs ont été prédits négatifs !

Cette section s'est focalisée sur la matrice de confusion pour un modèle binaire. La section suivante montre la forme générale de la matrice de confusion pour un modèle multiclasse.

10.2.2 Matrice de confusion générale

Comme pour la définition de la matrice de confusion binaire, nous allons supposer que nous disposons d'un ensemble de m_{test} individus pour évaluer un modèle de classification. En revanche, cette fois, nous supposons que chaque observation peut appartenir à une classe parmi n classes disponibles avec $n \geq 2$. Donc, le modèle classe chaque observation dans une de ces n classes, puis une comparaison avec les vraies classes de ces observations est réalisée. Les résultats de cette comparaison peuvent être résumés comme suit :

		Prédictions			
		C_1	C_i	C_n	
Observations	C_1	Q_1^1	Q_1^i	Q_1^n	$Q_1^1+Q_1^i+Q_1^n$
	C_i	Q_i^1	Q_i^i	Q_i^n	$Q_i^1+Q_i^i+Q_i^n$
	C_n	Q_n^1	Q_n^i	Q_n^n	$Q_n^1+Q_n^i+Q_n^n$
		$Q_1^1+Q_i^1+Q_n^1$	$Q_1^i+Q_i^i+Q_n^i$	$Q_1^n+Q_i^n+Q_n^n$	m_{test}

Figure 1-10 : matrice de confusion générale

Dans ce tableau, un élément Q_j^i correspond à la quantité d'observations qui sont de la classe C_j sachant qu'elles ont été prédites par un modèle de la classe C_i.

À partir de ce tableau, nous pouvons calculer le Recall de la classe C_1 qui est égal à $\frac{Q_1^1}{Q_1^1+Q_1^i+Q_1^n}$. La précision, quant à elle, est égale à $\frac{Q_1^1}{Q_1^1+Q_i^1+Q_n^1}$..

Chacune des classe C_i peut être utilisée pour construire une matrice de confusion binaire associée à cette casse C_i. En effet, il suffit de construire une classe $C_{\neq i}$ à partir de toutes les autres classes C_j avec j ≠ i en faisant l'addition des prédictions associées aux classes C_j comme dans le tableau suivant qui représente la matrice de confusion binaire associée à la classe C_1 :

		Prédictions	
		C_1	$C_{\neq 1}$
Observations	C_1	Q_1^1	$Q_1^i+Q_1^n$
	$C_{\neq 1}$	$Q_i^1+Q_n^1$	$Q_i^i+Q_n^i+Q_i^n+Q_n^n$

Figure 1-11 : construction de classe de confusion binaire à partir de la matrice de confusion générale

10.2.3 Exemple de matrice de confusion

Cette section donne quelques exemples de matrices de confusion. Nous supposons que trois modèles de classification binaire sont testés sur un ensemble de 200 observations. Les trois tableaux suivants donnent les matrices de confusion des résultats.

		Prédictions		
		" + "	" − "	
Observations	" + "	80	30	110
	" − "	20	70	90
		100	100	200

Figure 1-12 : matrice de confusion du modèle 1

© Editions ENI - All rights reserved

		Prédictions		
		" + "	" – "	
Observations	" + "	90	20	110
	" – "	0	90	90
		90	110	200

Figure 1-13 : matrice de confusion du modèle 2

		Prédictions		
		" + "	" – "	
Observations	" + "	110	0	110
	" – "	80	10	90
		190	10	200

Figure 1-14 : matrice de confusion du modèle 3

La première matrice de confusion a un Recall d'environ 72,72 % et une précision de 80 %.

La deuxième matrice de confusion a un Recall d'environ 81,81 % et une précision de 100 %.

La troisième matrice de confusion a un Recall de 100 % et une précision d'environ 57,89 %.

Ces résultats montrent que le troisième modèle parvient à détecter tous les cas positifs. En revanche, c'est ce modèle qui est le moins précis puisque, sur tous les positifs qu'il a prédits, il y a 100 %-57,89 % = 42,11 % de cas qui sont réellement négatifs.

Un modèle qui a une valeur élevée de précision mais une faible valeur pour le Recall peut être préféré à un modèle qui a une valeur plus élevée pour le Recall et une valeur moins importante pour la précision, et vice versa. L'importance donnée au Recall et à la précision dépend fortement du problème traité. Généralement, un modèle qui a plus de précision sera préféré si le fait de considérer un individu positif alors qu'il est négatif entraîne des conséquences graves. Dans d'autres cas, un modèle avec plus de Recall sera préféré si c'est le fait de ne pas détecter un cas positif qui entraîne des conséquences graves.

Remarque

Une matrice de confusion n'est pas forcément symétrique et dans la réalité l'est rarement. Également, un modèle de classification parfait se traduit par une matrice de confusion diagonale. Dans ce dernier cas, il est important de vérifier si la perfection d'un tel modèle n'est pas due au surapprentissage.

10.2.4 La courbe ROC

Certains algorithmes de classification se basent sur un seuil fixe pour classer les observations. La courbe ROC donne les résultats obtenus en fonction de ce seuil de classification. La courbe ROC a une allure comme dans la figure suivante 1-15 ci-dessous.

Soit une matrice de confusion associée à un modèle de classification binaire, donc avec la classe des positifs et la classe des négatifs. Pour une courbe ROC et en fonction d'une valeur pour le seuil de classification, l'axe des abscisses représente l'évolution du taux des faux négatifs et l'axe des ordonnées représente l'évolution du taux des vrais positifs.

Dans la figure 1-15 suivante, la droite qui coupe en deux le plan des abscisses et des ordonnées symbolise un modèle qui fait des prédictions de façon aléatoire pure. En revanche, le graphique en courbe représente un modèle qui donne des résultats meilleurs qu'un modèle aléatoire puisque, pour le même taux de faux négatifs, le modèle en courbe donne des résultats avec un taux de vrais positifs supérieur.

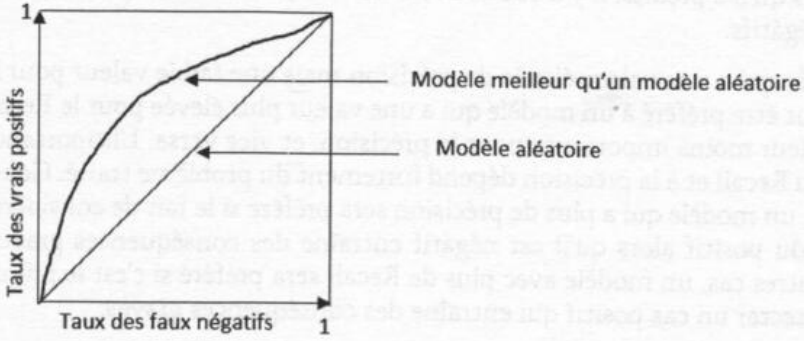


Figure 1-15 : exemple de courbe ROC

© Editions ENI - All rights reserved

10.3 Métriques pour le clustering

Les algorithmes de clustering répartissent un ensemble d'individus dans des paquets ou clusters. L'objectif de cette répartition est de rassembler les individus qui se ressemblent dans le même paquet et ceux qui sont différents dans des paquets différents.

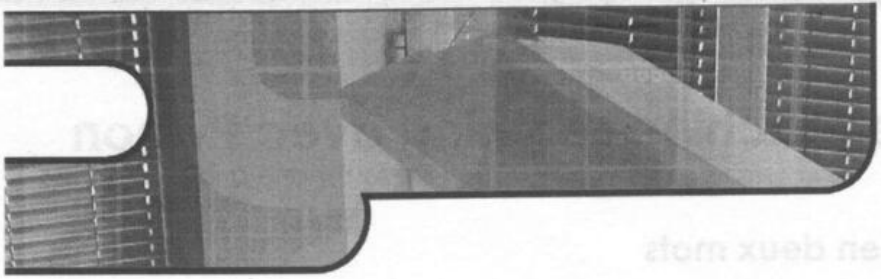
Dans le cas de ces algorithmes de clustering, des indicateurs de performance sont calculés pour quantifier le degré de rapprochement des individus au sein d'un même paquet, intracluster, et également quantifier le degré d'éloignement des individus appartenant à des paquets différents, interclusters.

Généralement, l'algorithme préféré sera celui qui construit des paquets en minimisant les distances entre les individus du même paquet et en maximisant les distances entre les individus des paquets différents.

Le chapitre L'algorithme k-means explique les mesures de performance de l'algorithme k-means, qui est l'un des algorithmes de clustering les plus connus.

11. Conclusion

Ce chapitre a permis d'introduire les notions de base de la Data Science. Il a commencé par donner une compréhension intuitive du Machine Learning, puis les autres sections ont été consacrées à l'introduction de la démarche à suivre lors d'une expérimentation Machine Learning et aussi à l'introduction des concepts de préparation des données, de surapprentissage, de données de validation et de la validation croisée et des métriques de performance. Le chapitre suivant va permettre au lecteur novice ou débutant en langage Python d'apprendre ou d'approfondir ses connaissances dans ce langage qui est le langage de choix lorsqu'on travaille dans le domaine du Machine Learning.



Partie 2
Outils techniques de la Data Science - Python, NumPy, Pandas et Jupyter

Chapitre 2-1
Le langage Python

1. Objectif du chapitre

L'objectif de ce chapitre est de donner au lecteur novice en langage Python les connaissances suffisantes pour bien démarrer avec ce langage et pour comprendre tous les exemples pratiques qui seront abordés dans ce livre.

Nous allons nous concentrer uniquement sur les aspects pratiques du langage Python sans trop nous attarder sur le côté théorique. L'idée est de vous permettre de vous familiariser avec ce langage afin que vous puissiez, sans difficulté, exprimer vos idées dans un code source écrit avec Python. Ainsi, la stratégie adoptée lors de la rédaction de ce chapitre est de s'attaquer directement à l'essentiel du langage Python.
À la fin de ce chapitre, le lecteur aura abordé :

- la structure d'un programme Python,
- les variables et les structures de données,
- les structures conditionnelles,
- les itérations avec les boucles,
- les fonctions,
- les listes en compréhension,
- les expressions régulières,
- la gestion des exceptions.

2. Python en deux mots

Le langage Python a été créé par Guido van Rossum en 1989. C'est un langage multiplateforme distribué sous une licence libre. Il permet plusieurs paradigmes de programmation tels que la programmation fonctionnelle, la programmation impérative ou la programmation orientée objet, et il propose une syntaxe très simple qui fait de ce langage un choix judicieux pour une personne qui souhaite s'initier aux notions fondamentales de la programmation.

L'un des points forts du langage Python réside dans la richesse des bibliothèques développées spécifiquement pour Python et qui permettent de gagner en productivité dans quasiment tous les domaines et tous les contextes métier ! L'un des exemples de ces bibliothèques est la bibliothèque Scikit-learn que nous allons utiliser par la suite dans ce livre et qui est dédiée au domaine du Machine Learning. Deux autres exemples sont la bibliothèque NumPy, dédiée au calcul numérique, et la bibliothèque Pandas, dédiée quant à elle à la manipulation de données. Ces deux dernières seront également abordées en détail dans un chapitre dédié à chacune d'elles.

3. Installer l'interpréteur Python

Python est un langage de programmation interprété, c'est-à-dire que le code source Python ne sera transformé en langage machine, qui est le seul langage compréhensible par un processeur, qu'au moment de son exécution. Ainsi, lorsqu'un code source Python est lancé pour être exécuté par un ordinateur, c'est l'interpréteur Python qui intervient afin de transformer en langage machine ligne par ligne ce code source. Chaque ligne du code source est transformée au moment de son exécution. Ce mécanisme de transformation du code source ligne par ligne peut engendrer une perte de performance, puisque les instructions qui seront appelées à être exécutées à maintes reprises seront interprétées plusieurs fois. Notons que ce problème n'existe pas avec les langages compilés, puisqu'avec ces langages, comme le C ou le C++, le code source est entièrement transformé en langage machine avant son exécution.

© Editions ENI - All rights reserved

Avant de commencer à programmer avec le langage Python, il est nécessaire d'installer l'interpréteur Python sur son ordinateur. Pour installer cet interpréteur, rien de plus simple, suivez les instructions suivantes.

- Rendez-vous sur le site officiel de Python : https://www.python.org/downloads/.
- Cliquez sur le menu **Downloads** pour accéder à la page des téléchargements et choisissez la version de Python correspondant à votre système d'exploitation (dans ce livre, c'est la plateforme Windows qui est utilisée), puis lancez le téléchargement du programme d'installation.

Remarque

Dans cet ouvrage, la version de Python utilisée est la version ***3.9.2*** *qui est la dernière version de Python en date.*

- Une fois que le programme d'installation est téléchargé, exécutez-le.
- Sur la première fenêtre du programme d'installation, cochez la case **Add Python 3.9 to PATH**. Cela va permettre à votre système d'exploitation de savoir où trouver l'interpréteur Python une fois l'installation terminée. Sur cette même page, cliquez sur **Install Now**.
- Votre système d'exploitation peut vous demander d'accepter ou non l'installation de Python. Si c'est le cas, répondez oui sans crainte, car avec l'interpréteur Python, la seule chose qui pourrait vous faire peur est son nom !
- Sur la dernière fenêtre d'installation, cliquez sur le bouton **Disable path length limit**. Également, votre système d'exploitation peut vous demander de valider cette dernière opération.
- Enfin, cliquez sur le bouton **Close**.

Une fois l'installation terminée, pour vérifier que l'interpréteur Python est correctement installé, suivez les quelques étapes ci-après.

- Lancez la console de commande de votre système d'exploitation.
- Puis lancez l'interpréteur Python avec la commande suivante :

```
python
```

Si tout va bien, vous devriez obtenir un résultat similaire à celui de la figure ci-dessous :

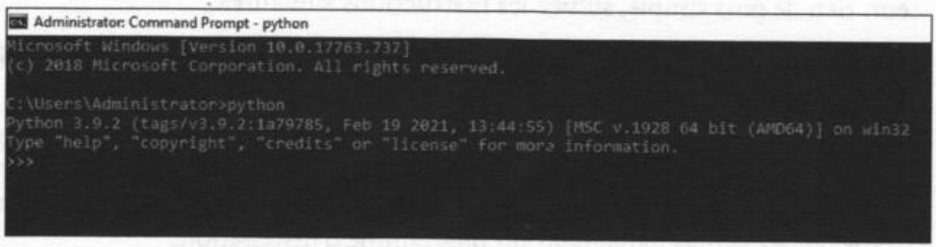


Figure 2-1 : Lancement de l'interpréteur Python dans une console sous Windows

- Dans l'interpréteur Python, saisissez la commande ou l'instruction ci-dessous, et appuyez sur la touche [Entrée] pour l'exécuter :

```
print('Hello world dans la console python')
```

Vous devriez obtenir le même résultat que celui de la figure ci-après :

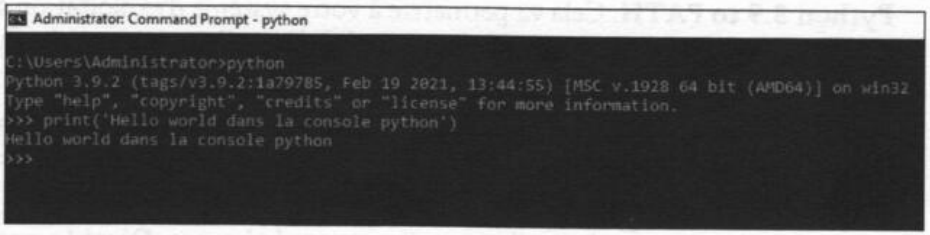


Figure 2-2 : Exécution d'une commande Python

Remarque

Si vous rencontrez des problèmes lors du lancement de la commande Python, essayez de résoudre ce problème en désinstallant Python et en réexécutant le programme d'installation que vous avez téléchargé précédemment.

- Pour quitter l'interpréteur Python, saisissez la commande `quit()` et appuyez sur la touche [Entrée].

© Editions ENI - All rights reserved

Le code source Python peut être rédigé avec n'importe quel éditeur de texte. Cependant, il existe plusieurs environnements de développement qui proposent des versions gratuites tels que PyCharm et Spyder qui sont dédiés au langage Python, ou encore Visual Studio Code qui est également bien adapté pour Python. Ces environnements permettent d'être plus productifs lorsque nous développons des programmes avec le langage Python. Bien sûr, l'utilisation de l'un de ces environnements est recommandée. Cependant, pour aborder un nouveau langage de programmation, le mieux est d'utiliser un simple éditeur de texte.

■Remarque

Dans ce chapitre, et jusqu'au chapitre La bibliothèque Pandas, nous allons utiliser Notepad++ comme éditeur de code source. Puis nous découvrirons en détail l'environnement Jupyter au chapitre Travailler avec Jupyter, puisque nous allons utiliser cet environnement à partir du chapitre Statistiques, lorsque nous commencerons réellement à aborder le mode du Machine Learning.

4. Les bases de la programmation Python

4.1 Hello world avec Python

Comme de coutume dans les livres portant sur les langages de programmation, nous allons commencer notre apprentissage du langage Python en réalisant le fameux programme qui affiche *Hello world* à l'écran.

Cette section va aborder deux fonctions de base du langage Python : `print` et `input`.

4.1.1 La fonction print

Pour réaliser votre premier script Python, suivez les étapes ci-dessous.

- Créez dans un dossier de votre choix un nouveau fichier et donnez-lui le nom *hello_world_01* et l'extension *.py*. Vous pouvez également utiliser le fichier *hello_world_01.py* situé dans le dossier *Code\Bases de ce chapitre*.

- Ouvrez ce nouveau fichier, puis saisissez la ligne du code Python ci-dessous et enregistrez les modifications apportées à ce fichier :

```
print("Hello world")
```

Et voilà ! Avec Python, vous n'avez besoin de rien de plus pour réaliser votre tout premier programme ! Vous pouvez d'ores et déjà exécuter ce premier script Python !

- Lancez votre invite de commandes, puis positionnez-vous dans le dossier où vous avez créé le fichier *hello_world_01.py*. Dans cette invite de commandes, saisissez la commandes ci-dessous et appuyez sur la touche [Entrée] :

```
python hello_world_01.py
```

Vous devriez voir le message "`Hello world`" s'afficher à votre écran.

La fonction `print` utilisée dans cet exemple est une fonction native Python, c'est-à-dire qu'il s'agit d'une fonction déjà définie dans Python. Dans la suite de ce chapitre, nous allons voir comment créer vos propres fonctions.

Cette fonction `print` affiche à l'écran la valeur de la chaîne de caractères qu'elle reçoit en argument. En réalité, Python commence par l'évaluation de l'argument de la fonction `print` avant d'appeler cette fonction `print` à proprement parler. En effet, vous pouvez appeler la fonction `print` en lui transmettant en paramètre une expression plutôt qu'une chaîne de caractères comme dans l'exemple suivant.

- Dans le même dossier que l'exemple précédent, créez un deuxième fichier et donnez-lui le nom *hello_world_02* et l'extension *.py*. Vous pouvez utiliser le fichier *hello_world_02.py* situé dans le dossier *Code\Bases* de ce chapitre.
- Ouvrez ce nouveau fichier, puis saisissez la ligne du code Python ci-dessous et enregistrez les modifications apportées à ce fichier :

```
print(5+4+3+2+1)
```

- Dans votre invite de commandes, saisissez la commande ci-dessous et appuyez sur la touche [Entrée] :

```
python hello_world_02.py
```

© Editions ENI - All rights reserved

Vous devriez voir la valeur 15 s'afficher à l'écran. Dans ce dernier exemple, Python a d'abord évalué la valeur de l'expression passée en argument à la fonction `print`, puis il a transformé le résultat de cette expression en une chaîne de caractères qui est affichée par la fonction `print`.

Les arguments de la fonction `print` peuvent être formatés de plusieurs façons. Le programme suivant donne quelques exemples.

- Dans le même dossier que l'exemple précédent, créez un troisième fichier et donnez-lui le nom *format* et l'extension *.py*. Vous pouvez également utiliser le fichier *format.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
a = 10
b = 300_000_000

print("a =",a, "b =",  b, "b/a =", b/a)
print("a = {} b = {} b/a = {}".format(a,b,b/a))
print(f"a = {a} b = {b} b/a = {b/a}")
print(f"a = {a} b = {b:,} b/a = {b/a:,}")
print("a = %d"%a, "b = %d"%b, "b/a = %f"%(b/a))
```

Ce programme commence par la création des deux variables `a=10` et `b=300000000`. Remarquez l'utilisation du caractère '_' dans la valeur de la variable `b`. Ce caractère est autorisé avec Python et son but est de simplifier la lecture des grands nombres.

Le programme continue avec cinq appels à la fonction `print`. Chaque appel de `print` affiche les mêmes valeurs `a`, `b` et `(b/a)`, mais de façon différente à chaque appel.

- Exécutez ce programme en saisissant dans l'invite de commandes l'instruction ci-dessous :

```
python format.py
```

Vous devriez obtenir le même résultat que celui ci-dessous :

```
a = 10 b = 300000000 b/a = 30000000.0
a = 10 b = 300000000 b/a = 30000000.0
a = 10 b = 300000000 b/a = 30000000.0
a = 10 b = 300,000,000 b/a = 30,000,000.0
a = 10 b = 300000000 b/a = 30000000.000000
```

Le premier appel à la fonction `print` permet d'afficher les arguments reçus en paramètres de façon linéaire dans l'ordre de leur apparition.

Le deuxième appel à la fonction `print` montre comment construire une chaîne de caractères en réservant dans cette même chaîne de caractères des zones d'affichage où seront affichés les arguments transmis à la fonction `format`. Une zone d'affichage est définie par une accolade ouvrante suivie d'une accolade fermante, l'ordre d'occupation de ces zones réservées correspondant à l'ordre d'apparition des variables et/ou des expressions dans les arguments de la fonction `format`.

Le troisième appel à la fonction `print` montre comment utiliser la forme contractée de la fonction `format` utilisée dans le deuxième appel à la fonction `print`. Cette fois, la chaîne de caractères transmise à la fonction `print` commence par la lettre `f`. Cette forme contractée de la fonction `format` offre plus de flexibilité, puisque les variables et/ou expressions à afficher seront immédiatement placées à l'endroit où elles seront affichées dans le résultat final.

Le quatrième appel à la fonction `print` montre un exemple de formatage afin de personnaliser le rendu de l'affichage des résultats. Ici, la variable `b` et l'expression `(b/a)` sont affichées en utilisant le caractère virgule pour faciliter la lecture des grands nombres.

Le cinquième appel à la fonction `print` montre comment utiliser le caractère `'%'` afin d'indiquer le type de la variable ou de l'expression à afficher et comment concaténer une chaîne de caractères avec une variable à afficher.

Il existe plusieurs façons d'utiliser la fonction `format` afin de construire une chaîne de caractères suivant un gabarit spécifique, mais nous n'allons pas nous attarder sur ce sujet.

Remarque

Dans l'annexe A, lorsque nous aborderons la programmation orientée objet, nous verrons comment personnaliser le comportement de la fonction `print` afin de répondre à des besoins d'affichage spécifiques.

© Editions ENI - All rights reserved

4.1.2 La fonction input

La fonction `input`, qui est native au langage Python, permet à l'utilisateur d'un programme de saisir des données à l'aide de son clavier. Voici un exemple de programme qui affiche le nombre de caractères présents dans une chaîne de caractères saisie par l'utilisateur.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant puis exécutez-le. Vous pouvez également utiliser le fichier *input.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
str_user = input("Veuillez saisir une chaîne de caractères :")

print(f"La chaîne de caractères que vous avez saisie contient
      {len(str_user)} caractères.")
```

La première instruction de ce programme appelle la fonction `input` pour inviter l'utilisateur à saisir une chaîne de caractères à l'aide du clavier. Cette chaîne de caractères est stockée dans la variable `str_user`. La dernière instruction permet d'afficher le nombre de caractères qui composent la chaîne de caractères `str_user` à l'aide de la fonction `len`.

- À l'exécution de ce programme, saisissez une chaîne de caractères lorsque vous serez invité à le faire, puis appuyez sur la touche [Entrée] afin de valider votre saisie. Ce programme donne le résultat suivant après avoir saisi la phrase `Encore Hello world` :

```
Veuillez saisir une chaîne de caractères :Encore Hello world
La chaîne de caractères que vous avez saisie contient 18
caractères.
```

4.2 Les structures de données

Les structures de données permettent de définir la nature des données traitées par un programme. Il existe des types de données natifs à Python, c'est-à-dire des types de données prêts à être utilisés dès l'installation de Python, et des types de données définis par l'utilisateur.

Voici la définition de quelques types de base et natifs à Python :

- Le type de données numérique associé aux nombres entiers et aux nombres réels pour la réalisation des calculs arithmétiques.
- Le type de données alphanumérique associé aux chaînes de caractères pour la manipulation des données non numériques.
- Le type booléen pour la manipulation des variables dont les données ne peuvent prendre que deux valeurs possibles.
- Les listes pour regrouper les données dans une structure similaire à celle d'un tableau.
- Les tuples pour le regroupement de données dans des structures immuables.
- Les dictionnaires pour stocker les données dans des tables de hachage à l'aide de clés.
- Les ensembles pour stocker des données non ordonnées et sans duplication.

Les sous-sections suivantes abordent chaque type de données ci-dessus, et dans l'annexe A, nous allons voir comment définir de nouveaux types de données lorsque nous aborderons la programmation orientée objet.

4.2.1 Les variables numériques

Les variables numériques permettent la manipulation de données quantitatives auxquelles nous pouvons associer des valeurs entières ou des valeurs réelles ou encore des nombres complexes.

D'un point de vue purement conceptuel, les variables peuvent être vues comme des emplacements ou des cases dans la mémoire vive de l'ordinateur dans lesquels nous pouvons écrire des données. Dans le cas de variables numériques, ces données sont des quantités numériques. Le premier programme suivant montre comment manipuler des données entières, tandis que les deux programmes suivants montrent comment manipuler des données réelles.

Les nombres entiers

Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez-le. Vous pouvez également utiliser le fichier *nombres_entiers.py* situé dans le dossier *Code\Bases* de ce chapitre.

© Editions ENI - All rights reserved

```
variable_1 = 5
print(f"Le type de variable_1 est : {type(variable_1)}")

variable_2 = 15
somme=variable_1 + variable_2

print(f"La somme de variable_1 et variable_2 est égale à :
{somme}")

somme=somme+10
print(f"La nouvelle valeur de la variable somme est égale à
:{somme}")

somme+=20
print(f"La dernière valeur de la variable somme est égale à
:{somme}")
```

La première instruction crée et affecte la variable `variable_1` avec le nombre entier 5.

La deuxième instruction affiche le type de la variable `variable_1` en appliquant la fonction `type` sur la variable `variable_1`.

La troisième instruction crée et affecte la variable `variable_2` avec le nombre entier 15.

La quatrième instruction réalise l'opération d'addition entre les deux variables `variable_1` et `variable_2`, puis affecte le résultat à la nouvelle variable `somme`. L'instruction suivante affiche le résultat de cette addition.

L'instruction suivante ajoute la valeur 10 à la valeur déjà enregistrée dans la variable `somme`. Même si l'écriture `somme=somme+10` semble mathématiquement incorrecte, elle est tout à fait logique en informatique !

En effet, cette écriture ne signifie pas que la valeur de la variable `somme` est égale à la valeur de la variable `somme` plus 10. Elle veut plutôt dire que la valeur actuellement enregistrée dans la case mémoire désignée par la variable `somme` doit être additionnée avec la valeur 10, puis affecter le résultat de cette addition à la case mémoire désignée par la variable `somme`. Ce qui aura comme conséquence que l'ancienne valeur de la case mémoire désignée par la variable `somme` sera écrasée par la nouvelle valeur !

L'avant-dernière instruction ajoute la valeur 20 à la valeur déjà présente dans la variable `somme` en utilisant la forme contractée de l'opération d'addition dont la syntaxe s'écrit avec les deux symboles `+=`. Cette instruction `somme+=20` est équivalente à l'instruction `somme=somme+20`.

La dernière instruction affiche la nouvelle valeur de la variable `somme`.

Toutes les opérations arithmétiques classiques telles que la soustraction, la multiplication et la division s'emploient de la même manière que l'opération d'addition montrée dans cet exemple.

Le programme précédent affiche le résultat suivant :

```
Le type de variable_1 est : <class 'int'>
La somme de variable_1 et variable_2 est égale à : 20
La nouvelle valeur de la variable somme est égale à :30
La dernière valeur de la variable somme est égale à :50
```

La première ligne montre que le type de la variable `variable_1` est `<class 'int'>`. La valeur `int` signifie que `variable_1` est destinée à stocker des nombres entiers. Ce type de variable ne peut stocker qu'une seule valeur à la fois, c'est-à-dire que nous ne pouvons pas stocker deux nombres entiers en même temps dans cette variable `variable_1`. En revanche, nous pouvons changer la valeur de la variable `variable_1` en lui affectant tout simplement une nouvelle valeur comme nous l'avons fait dans l'exemple précédent avec la variable `somme` lorsque nous lui avons par exemple ajouté la valeur 10, alors qu'elle contenait déjà la somme des deux variables `variable_1` et `variable_2`. Dans la suite de ce chapitre, nous allons découvrir le sens du mot `class` présent dans le type de la variable `variable_1` et nous allons également découvrir d'autres types de données qui permettent de stocker plusieurs valeurs à la fois.

Les nombres flottants

Les nombres flottants correspondent aux nombres réels. Suivez les étapes ci-dessous pour développer un petit programme dans lequel nous allons manipuler les nombres flottants.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez-le. Vous pouvez également utiliser le fichier *nombres_réels-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

© Editions ENI - All rights reserved

```
x = 5.0
print(f"Le type de x est : {type(x)}")

y=x/2
print(f"la valeur de y est: {y}")

n = 10
z=float(n)
print(f"n est de type :{type(n)} et z est de type {type(z)}")
```

Ce programme commence par la déclaration et l'initialisation de la variable x avec le nombre réel 5.0. Python reconnaît que la variable x est une variable réelle à partir du moment où nous lui avons affecté une valeur réelle ! En effet, avec le langage Python, le typage des variables est réalisé de manière dynamique au moment de l'exécution ou de l'interprétation des instructions.

La deuxième instruction affiche le type de la variable x, puis l'instruction suivante calcule le résultat de la division de la valeur de la variable x par 2 et affecte le résultat à une nouvelle variable y et le résultat est affiché.

Ensuite, la variable n sera affectée avec la valeur 2. Donc la variable n sera de type entier. L'avant-dernière instruction transforme le contenu de la variable n en un type flottant en utilisant la fonction `float()` et affecte le résultat à la nouvelle variable z. La dernière instruction affiche tout simplement les types des deux variables n et z.

Ce code donne le résultat suivant :

```
Le type de x est : <class 'float'>
la valeur de y est: 2.5
n est de type :<class 'int'> et z est de type <class 'float'>
```

Ces résultats montrent que la variable x est bien de type `float`, ce qui correspond au type des nombres flottants. Remarquez que la variable n est de type `int`. En effet, la variable n a été créée en tant que nombre entier et le fait d'appliquer la fonction `float()` sur cette variable ne modifie pas son type, car l'instruction `float(n)` ne va pas transformer le contenu de la case mémoire désignée par la variable n, mais elle va plutôt créer une copie de la variable n et c'est cette copie qui sera transformée en flottant et stockée dans la nouvelle variable z.

L'exemple suivant va nous permettre de montrer comment inclure le module `math` et utiliser certaines de ses fonctionnalités. Un module est tout simplement une partie du code qui regroupe un certain nombre de fonctionnalités dédiées à la réalisation d'un type d'opérations. Par exemple, ce module `math` regroupe plusieurs fonctions dédiées aux opérations mathématiques !

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *nombres_réels-2.py* situé dans le dossier Code\Bases de ce chapitre.

```
import math

pi=math.pi

print(f"round({pi}) = {round(pi)}")
print(f"round({pi},2) = {round(pi,2)}")
print(f"trunc({pi}) = {math.trunc(pi)}")
print(f"ceil({pi}) = {math.ceil(pi)}")
print(f"floor({pi}) = {math.floor(pi)}")

print(f"la racine carrée de 25 est égale à : {math.sqrt(25)}")
print(f"La valeur de e est : {math.e} et ln(e) =
{math.log(math.e)}")
```

La première instruction permet d'importer le module `math` afin de rendre ses fonctionnalités disponibles et de pouvoir les utiliser dans notre programme.

La deuxième instruction affecte le fameux chiffre magique π à la variable `pi`.

Les cinq instructions suivantes introduisent quelques fonctionnalités du module `math` qui permettent de reformater les données. Ces fonctions sont les suivantes :

- La fonction `round` permet d'arrondir un nombre réel vers le nombre le plus proche. Par exemple, `round(5.334)` sera arrondi vers 5. Avec cette fonction, nous pouvons spécifier le nombre de décimales à utiliser pour l'opération de transformation. Par exemple, `round(5.334,2)` sera arrondi vers 5.33 et `round(5.336,2)` sera arrondi vers 5.34.
- La fonction `math.trunc` permet tout simplement de tronquer un nombre en coupant les chiffres après la décimale.
- La fonction `math.ceil` permet d'arrondir vers le nombre le plus haut.

© Editions ENI - All rights reserved

– La fonction `math.floor` permet quant à elle d'arrondir vers le nombre le plus petit.

L'avant-dernière instruction calcule la racine carrée du chiffre 25 en utilisant la fonction `math.sqrt`, et enfin, la dernière instruction affiche la valeur du chiffre e correspondant à la base des logarithmes naturels et affiche également le résultat de l'application de la fonction logarithmique `math.log` sur la valeur de e.

L'exécution de ce code donne le résultat suivant :

```
round(3.141592653589793) = 3
round(3.141592653589793,2) = 3.14
trunc(3.141592653589793) = 3
ceil(3.141592653589793) = 4
floor(3.141592653589793) = 3
la racine carrée de 25 est égale à : 5.0
La valeur de e est : 2.718281828459045 et ln(e) = 1.0
```

4.2.2 Les chaînes de caractères

Les chaînes de caractères correspondent aux données non numériques qui, par définition, ne représentent pas des quantités mathématiques.

Une chaîne de caractères est en réalité une structure de données formée par l'assemblage de plusieurs caractères placés les uns à côté des autres pour former un tableau ou une liste de caractères. Nous allons découvrir les listes un peu plus loin dans ce chapitre.

Par exemple, la chaîne de caractères `ch='Alicia'` est une liste composée des caractères `['A','l','i','c','i','a']`. Chaque caractère occupe une position fixe dans la liste associée à une chaîne de caractères. Ainsi, dans la chaîne de caractères `ch`, le caractère `'A'` occupe la position avec l'indice 0, la lettre `'l'` occupe la position avec l'indice 1, et ainsi de suite. La chaîne de caractères `ch` est de taille 6 puisqu'elle contient six caractères, et le dernier caractère 'a' occupe la position avec l'indice 5.

En langage Python, les chaînes de caractères sont de type `str` comme le montre le code de l'exemple suivant.

Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce programme. Vous pouvez également utiliser le fichier *str.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
str_1 = 'De nouveau Hello'
print(f"Le type de str_1 est : {type(str_1)}")
print("1- ", str_1)
print("2- ", str_1[:10])
print("3- ", str_1[11:])
print("4- ", str_1[3:10])
print("5- ", str_1[::2])

str_2 = str_1 + ' word'
print(f"str_2 ={str_2}")
```

- Ce code commence par l'affectation de la chaîne de caractères `De nouveau Hello` à la variable `str_1`. La deuxième instruction affiche le type de la variable `str_1`. Chacune des cinq instructions suivantes affiche tout ou partie de la chaîne de caractères `str_1` comme suit :
- `print("1- ", str_1)` : affiche tout le contenu de la variable `str_1`.
- `print("2- ", str_1[:10])` : affiche les caractères contenus dans `str_1` et qui sont positionnés entre les indices 0 et 9. La notation `[:10]` fait référence à une liste contenant les valeurs de 0 jusqu'à 9 inclus. Les éléments de cette liste jouent ici le rôle d'indices pour accéder aux caractères de la variable `str_1`.
- `print("3- ", str_1[11:])` : affiche les caractères de `str_1` en commençant par le caractère situé à l'indice 11 et en allant jusqu'à la fin.
- `print("4- ", str_1[3:10])` : affiche les caractères de `str_1` en commençant par le caractère situé à l'indice 3 et en allant jusqu'au caractère situé à l'indice 9.
- `print("5- ", str_1[::2])` : affiche tous les caractères de `str_1` en allant de la première position, la position 0, jusqu'à la fin en faisant des sauts de deux caractères à chaque fois.

L'avant-dernière instruction concatène la chaîne de caractères `str_1` et le mot `world` et affecte le résultat à la nouvelle chaîne de caractères `str_2`. La dernière instruction affiche tout simplement `str_2`.

© Editions ENI - All rights reserved

Ce code affiche le résultat suivant :

```
Le type de str_1 est : <class 'str'>
1-  De nouveau Hello
2-  De nouveau
3-  Hello
4-  nouveau
5-  D ova el
str_2 =De nouveau Hello world
```

Constatez que le type de la variable `str_1` est `str` et constatez également l'effet de l'opération d'addition sur les deux chaînes de caractères `str_1` et `str_2` ! Bien entendu, cette opération d'addition n'est pas la même que l'opération d'addition arithmétique que nous avons utilisée précédemment pour réaliser l'addition de deux nombres. Ici, il s'agit de la concaténation de chaînes de caractères et le fait de pouvoir réaliser cette concaténation avec le symbole d'addition n'est qu'une possibilité syntaxique offerte par le langage Python afin de faciliter et d'alléger l'écriture du code.

4.2.3 Le type booléen

Les variables booléennes sont des variables au même titre que les variables de type entier ou de type réel, à la différence que les variables booléennes ne peuvent prendre que les deux valeurs possibles `True` et `False`.

Le plus souvent, le type booléen est utilisé pour évaluer la valeur logique d'une expression. Les deux programmes suivants donnent des exemples d'utilisation de variables et d'expressions booléennes.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *bool-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
a, b, c = True, False, True

print(f"a={a}, b={b}, c={c}")
print(f"Le type de la variable \'a\' est  : {type(a)}")
print(f"a = b : {a==b}")
print(f"a = c : {a==c}")

d = a+c
print(f"d = (a + c) = {d}")
```

```
print(f"Le type de la variable \'d\' est : {type(d)}")
```

La première instruction initialise trois variables booléennes `a`, `b` et `c` avec respectivement les trois valeurs logiques `True`, `False` et `True`.

La deuxième instruction affiche les valeurs des trois variables `a`, `b` et `c` et l'instruction suivante affiche le type de la variable `a`. La quatrième instruction affiche le résultat de la comparaison logique entre la variable `a` et la variable `b` et la cinquième instruction affiche le résultat de la comparaison entre la variable `a` et la variable `c`. Le symbole `==` est l'opérateur de comparaison logique d'égalité entre deux expressions. Ainsi, `a==b` sera évalué à `True` si la valeur de la variable `a` est égale à la valeur de la variable `b`, sinon cette comparaison sera évaluée à `False`. Le même type de comparaison est réalisé avec `a==c`.

La variable `d` est initialisée avec la somme des deux variables booléennes `a` et `c`.

Lorsqu'une variable booléenne est utilisée avec un opérateur arithmétique, la valeur `True` est considérée comme étant égale à 1 et la valeur `False` est considérée comme étant égale à 0.

L'avant-dernière instruction affiche la valeur de la variable `d` et la dernière instruction affiche le type de cette variable `d` afin de vérifier le type attribué à la somme des deux variables booléennes `a` et `c`.

L'exécution de ce code donne le résultat suivant :

```
a=True, b=False, c=True
Le type de la variable 'a' est  : <class 'bool'>
a = b : False
a = c : True
d = (a + c) = 2
Le type de la variable 'd' est : <class 'int'>
```

On voit que `bool` est le type associé aux variables booléennes. Le résultat de la comparaison entre `a` et `b` est `False` puisque la variable `a` est égale à `True`, alors que la variable `b` est égale à `False`. La comparaison entre `a` et `c` donne le résultat `True` puisque ces deux variables ont la même valeur `True`.

Observez la valeur de la variable `d` qui est égale à 2. En effet, comme nous l'avons mentionné précédemment, la valeur booléenne `True` est évaluée à la

© Editions ENI - All rights reserved

valeur 1 dans l'expression a+c. Cette expression est équivalente à 1+1 puisque a=True et b=True ! D'ailleurs, la dernière ligne du résultat affiche que le type de la variable d est int, c'est-à-dire que la somme des deux variables booléennes a et c aboutit à la création d'une variable de type entier !

Dans l'exemple précédent, nous avons comparé et additionné des variables booléennes qui ont été créées et initialisées avec les valeurs logiques True et False. En réalité, toutes les variables numériques ou alphanumériques peuvent être utilisées en tant que variables booléennes en utilisant l'opérateur de conversion bool !

Par exemple, lorsqu'une variable entière de type int est utilisée dans une expression logique, avec l'opérateur bool elle sera évaluée à False si sa valeur est égale à 0, sinon elle sera évaluée à True pour toute valeur entière autre que 0. Idem pour une chaîne de caractères qui sera évaluée à False si elle est vide et évaluée à True à partir du moment où elle contient au moins un caractère.

L'exemple du code suivant illustre l'utilisation combinée des variables booléennes avec des variables numériques et des chaînes de caractères, et ce code montre également comment utiliser les opérateurs logiques or, and et not.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *bool-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
a,b,pi = True, 5, 3.14
print(f"a={a}, b={b}, pi={pi}")

print(f"a = 0  : {a==0}")
print(f"a = 1  : {a==1}")
print(f"a = bool(50) : {a==bool(50)}")
print(f"a = bool(pi) : {a==bool(pi)}")
print(f"a > False  : {a>False}")

print(f"(b = 5) ou (b = 6) : {b==5 or b==6}")
print(f"(b = 6) ou (b = 5) : {b==6 or b==5}")
print(f"not(a) = : {not a}")
print(f"(a = True) and  (b = 5) : {(a == True and b == 5)}")

ch1="Un dernier Hello world"
ch2=""
```

```
print(f"bool(ch1) = {bool(ch1)}")
print(f"bool(ch2) = {bool(ch2)}")
```

La première instruction initialise trois variables booléennes `a`, `b` et `pi` avec respectivement les trois valeurs `True`, 5 et 3.14.

La deuxième instruction affiche les valeurs des trois variables `a`, `b` et `pi`.

Le reste des instructions est expliqué ci-après :

- `print(f"a = 0 : {a==0}")` : affiche le résultat du test de l'égalité entre la variable booléenne `a` et la valeur 0.
- `print(f"a = 1 : {a==1}")` : affiche le résultat du test de l'égalité entre la variable booléenne `a` et la valeur 1.
- `print(f"a = bool(50) : {a==bool(50)}")` : affiche le résultat du test de l'égalité entre la variable booléenne `a` et la valeur booléenne correspondant à 50. Ici, le résultat attendu avec cette instruction est le même que le résultat qui devrait être obtenu avec l'instruction précédente ! L'idée est de voir que la comparaison d'une variable booléenne avec la valeur 1 est équivalente à la comparaison de cette variable booléenne avec n'importe quelle autre valeur différente de 0 en utilisant l'opérateur `bool`.
- `print(f"a = bool(pi) : {a==bool(pi)}")` : affiche le résultat du test de l'égalité entre la variable booléenne `a` et la variable réelle `pi` après sa conversion en booléen.
- `print(f"a>False :{a>False}")` : affiche le résultat de la comparaison avec l'opérateur strictement supérieur > entre la variable booléenne `a` et la valeur `False`. Lors d'une comparaison avec l'un des opérateurs >, ≥,< ou ≤ entre deux valeurs booléennes, la valeur `True` est considérée supérieure à la valeur `False`.

  Les autres opérateurs de comparaison binaire <, ≤, ≥ et != s'utilisent de la même manière que l'opérateur > de cette instruction. Par exemple, pour comparer si `b` est inférieur ou égal à `pi`, nous aurions écrit `b<=pi`, ou encore pour comparer si `b` est différent de 6, nous aurions écrit `a!=6`.
- `print(f"(b = 5) ou (b = 6) : {b==5 or b==6}")` : affiche le résultat des deux tests d'égalité entre la variable `b` avec respectivement les valeurs 5 et 6. Ces deux tests sont reliés avec l'opérateur logique `or` qui correspond à l'opérateur en logique mathématique *OU*.

© Editions ENI - All rights reserved

Remarque

Lors de l'évaluation de l'expression `(b==5 or b==6)`, si la valeur de la variable `b` réussit le premier test `b==5`, c'est-à-dire si `b` est égal à 5, alors le deuxième test `b==6` ne sera pas effectué puisqu'avec l'opérateur logique OU, dès qu'un terme est vrai, c'est toute l'expression qui sera évaluée à vraie. De ce fait, pour des raisons de performance, lorsque l'opérateur `or` est utilisé dans une expression, il est fortement recommandé de positionner à gauche les termes qui ont le plus de chances d'être évalués à vrai afin de minimiser le nombre de tests à exécuter ; cela est d'autant plus vrai si l'expression est évaluée plusieurs fois dans le programme.

- `print(f"(b = 6) ou (b = 5) : {b==6 or b==5}")` : cette instruction affiche le résultat de la même expression que celle utilisée avec l'instruction précédente, sauf qu'ici le programme va d'abord comparer `b` avec la valeur 6 puis, si ce test échoue, c'est-à-dire si la variable `b` est différente de 6, alors le deuxième test `b==5` sera réalisé. Ces deux dernières instructions vont certainement afficher les mêmes résultats, mais comme mentionné juste avant, elles ne vont pas exécuter le même nombre de tests ! Dans notre exemple, nous savons que la variable `b` vaut 5, donc l'instruction précédente ne va exécuter que le test `b==5` alors que cette dernière instruction va certainement réaliser les deux tests, le test `b==6` et le test `b==5`.
- `print(f"not(a) = : {not a}")` : affiche le résultat de la négation de la variable `a`.
- `print(f"(a = True) and (b = 5) : {(a == True and b == 5)}")` : cette instruction affiche le résultat de l'évaluation de l'expression composée par les deux tests `a==True` et `b==5`. Ces deux tests sont liés par l'opérateur logique `and`. Contrairement à l'opérateur logique `or`, l'opérateur logique `and` s'arrête dès qu'un des tests effectués échoue !

  Également, pour des raisons de performance, lorsque l'opérateur logique `and` est utilisé, il est plus judicieux de placer à gauche les termes qui ont le plus de chances d'échouer afin de minimiser le nombre de tests réalisés.
- `ch1="Un dernier Hello world"` et `ch2=""` : la première instruction affecte la chaîne de caractères `"Un dernier Hello world"` à la variable `ch1` et la deuxième instruction affecte une chaîne de caractères vide à la variable `ch2`.

- Les deux dernières instructions convertissent les deux chaînes de caractères ch1 et ch2 vers le type booléen et affichent les résultats.

Le résultat de l'exécution de ce programme est le suivant :

```
a=True, b=5, pi=3.14
a = 0  : False
a = 1  : True
a = bool(50) : True
a = bool(pi) : True
a > False  : True
(b = 5) ou (b = 6) : True
(b = 6) ou (b = 5) : True
not(a) = : False
(a = True) and  (b = 5) : True
bool(ch1) = True
bool(ch2) = False
```

Remarquez que le test a==0 a retourné la valeur False puisque a est égal à True alors que les tests a==1 et a==bool(50) et a==bool(pi) ont tous les trois retourné la valeur True.

La comparaison entre la valeur de la variable a avec la valeur False a retourné True puisque True est supérieur à False !

Les deux expressions avec l'opérateur logique or ont retourné la valeur True, puisque dans ces deux expressions b=5 permet l'évaluation globale de chacune de ces deux expressions à True mais, pour rappel, dans la première expression il n'y avait que le premier test b==5 qui a été évalué alors que dans la seconde expression les deux tests ont bien été évalués pour l'aboutissement à la valeur True globale.

La négation de la variable booléenne a a retourné False, puisque a=True. Le test avec l'opérateur and a réussi, car les deux termes qui le composent sont vrais. Enfin, comme attendu, la conversion de la chaîne de caractères ch1 vers le type booléen a retourné la valeur True et la conversion de la chaîne de caractères vide ch2 vers le type booléen a retourné la valeur False.

© Editions ENI - All rights reserved

4.2.4 Les listes

Nous avons vu que les variables numériques et alphanumériques sont des structures de données qui permettent de stocker une seule valeur à la fois. Les listes sont des structures de données qui permettent justement de stocker toute une série de valeurs en même temps. Avec le langage Python, les valeurs stockées dans une liste peuvent être de types différents. Ainsi, dans une même liste, nous pouvons par exemple stocker des nombres entiers, des valeurs booléennes et des chaînes de caractères !

Les éléments d'une liste sont ordonnés suivant leur position dans cette liste. Ces positions sont définies par un index dont la première valeur est 0 et qui correspond à la première position dans une liste. De ce fait, l'élément situé à la première position d'une liste sera accessible avec l'indice 0. L'élément situé à la deuxième position d'une liste sera accessible avec l'indice 1, et ainsi de suite. Avec Python, on peut également utiliser des indices négatifs afin d'accéder aux éléments d'une liste en commençant le comptage en partant de l'élément situé à la dernière position. Ainsi, l'indice -1 désigne l'élément situé à la fin d'une liste, l'indice -2 désigne la position de l'avant-dernier élément, etc.

Par exemple, soit `L=[5,6,7,8,9]` une liste contenant cinq éléments. La notation `L[0]` désigne l'élément situé à la première position de la liste `L` et qui est égale à 5 ; la notation `L[1]` désigne l'élément situé à la deuxième position de la liste `L` et qui est égal à 6, etc. ; la notation `L[-1]` désigne le dernier élément de la liste `L` et qui est égal à 9.

Pour déclarer une liste, il suffit d'indiquer les éléments qui la composent en séparant chaque élément par une virgule et en encadrant le tout entre crochets comme le montre l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant puis exécutez ce code. Vous pouvez également utiliser le fichier *list.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list_1 = [1,28,True,"Python",64,False,"Langage
C",12,"Pascal",10]
print("list_1 = ",list_1)
print(f"Le nombre d'éléments dans la liste list_1 est :
      {len(list_1)}")

item0 = list_1[0]
```

```
item1 = list_1[-1]

print(f'item0 = {item0}')
print(f'item1 = {item1}')

list_1[1] = "json"
list_1[6] = "C++"
print("list_1 = ",list_1)

list_2 = list_1 +[1,2,3]
print("list_2 = ",list_2)
```

Ce code commence par la déclaration de la liste `list_1` avec dix éléments qui sont un mélange entre des nombres entiers, des valeurs booléennes et des chaînes de caractères.

Le premier appel à la fonction `print` permet d'afficher le contenu de la liste `list_1` et le deuxième `print` permet d'afficher la taille de cette liste en utilisant la fonction `len`.

Les deux instructions `item0 = list_1[0]` et `item1 = list_1[-1]` permettent d'affecter la première et la dernière valeur de la liste `list_1` aux variables respectives `item0` et `item1`. Puis les valeurs des deux variables `item0` et `item1` sont affichées.

L'instruction `list_1[1] = "json"` permet de modifier le contenu de la liste `list_1` en affectant la chaîne de caractères `"json"` à la valeur située à la deuxième position dans la liste `list_1`. De la même manière, l'instruction `list_1[6] = "C++"` modifie le contenu de la liste `list_1` en affectant la chaîne de caractères `"C++"` à la valeur située à la septième position dans la liste `list_1`. La liste `list_1` est de nouveau affichée afin de visualiser l'effet de ces deux dernières modifications.

L'instruction `list_2 = list_1 +[1,2,3]` permet de montrer comment utiliser l'opérateur + entre deux listes. Cet opérateur permet de concaténer deux listes en créant une nouvelle liste composée des éléments de la première liste située à gauche de cet opérateur suivis des éléments de la deuxième liste située à droite. La dernière instruction affiche tout simplement le contenu de la liste `list_2`.

© Editions ENI - All rights reserved

Le résultat du programme précédent est le suivant :

```
list_1 =  [1, 28, True, 'Python', 64, False, 'Langage C', 12,
           'Pascal', 10]

Le nombre d'éléments dans la liste list_1 est : 10

item0 = 1
item1 = 10

list_1 =  [1, 'json', True, 'Python', 64, False, 'C++', 12,
           'Pascal', 10]

list_2 =  [1, 'json', True, 'Python', 64, False, 'C++', 12,
           'Pascal', 10, 1, 2, 3]
```

On voit que le nombre d'éléments dans la liste `list_1` est égal à 10 et que la valeur de l'élément situé à la position 0 est 1, ce qui correspond au premier élément de la liste, et la valeur de l'élément situé à la position -1 est égale à 10, ce qui correspond au dernier élément de cette liste.

Le dernier affichage de la liste `list_1` montre que la valeur 28 qui était située à la deuxième position avec l'indice 1 a été modifiée et remplacée par la chaîne de caractères `'json'` et également la chaîne de caractères `'Langage C'` qui était située à la position 7 avec l'indice 6 a été modifiée et remplacée par la chaîne de caractères `'C++'`.

La dernière ligne de ces résultats affiche le résultat de l'application de l'opérateur + sur les deux listes `list_1` et `[1,2,3]`. Vous remarquez l'ajout des éléments de la liste `[1,2,3]` à la fin de la liste `list_1`.

Remarque

Avec les listes, nous pouvons utiliser la notation `[indice_début : indice_fin]` afin d'accéder à toutes les valeurs situées entre l'indice `indice_début` et l'indice `(indice_fin-1)`. Cette notation avancée qui permet d'indexer les valeurs d'une liste est commune avec la notation qui permet d'accéder aux valeurs de tableaux NumPy. C'est pour cette raison que l'accès aux valeurs d'une liste avec les intervalles d'index sera abordé au chapitre suivant lorsque nous étudierons en détail la bibliothèque NumPy.

Si une instruction tente d'accéder à une liste avec un indice qui n'existe pas, alors une erreur est levée et l'exécution du programme s'arrête. Par exemple, dans notre programme précédent, si on ajoute à la fin du fichier *list.py* l'instruction `print(list_1[11])` et si on exécute à nouveau ce programme, alors cette exécution s'arrête brutalement après avoir indiqué le message `IndexError: list index out of range`. En effet, la liste `list_1` ne possède que dix éléments alors que cette instruction tente d'accéder à un supposé onzième élément qui n'existe pas !

4.2.5 Les tuples

Un tuple est une structure de données proche de la notion de liste, sauf qu'un tuple n'est pas modifiable. Tout comme pour la liste, les éléments d'un tuple sont ordonnés suivant leur position. Le code suivant montre comment créer et utiliser les tuples.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *tuple.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
tuple_ = (1,2,True,"Fortran")

print(f"tuple_ = {tuple_}")
print(f"Le type de tuple_ est {type(tuple_)}")

t1=(1,2,3)
a,_,b = t1
print(f"a={a} et b={b}")

t2=(1,2,3,4,5,6,7,8,9,10)
(v1,*r,v2) =t2
print(f"v1 = {v1}, r={r}, v2={v2}")
```

Ce code déclare et initialise le tuple `tuple_` avec les quatre valeurs `1`, `2`, `True` et `"Fortran"`, puis le contenu de `tuple_` et son type sont affichés.

L'instruction `t1=(1,2,3)` déclare et initialise le tuple `t1` avec les trois valeurs 1, 2 et 3. L'instruction suivante `a,_,b = t1` montre comment récupérer dans les deux variables `a` et `b` respectivement la première et la dernière valeur du tuple `t1` en ignorant la deuxième valeur. Puis les valeurs des variables `a` et `b` sont affichées.

© Editions ENI - All rights reserved

Le tuple `t2` est créé et initialisé avec dix valeurs. L'instruction `(v1,*r,v2) =t2` montre comment dézipper les éléments du tuple `t2` en récupérant le premier élément dans la variable `v1`, le dernier élément dans la variable `v2` et toutes les valeurs situées entre la première et la dernière valeur seront affectées dans un nouveau tuple `r`. Enfin, les valeurs des variables `v1`, `r` et `v2` sont affichées.

Ce code affiche le résultat suivant :

```
tuple_ = (1, 2, True, 'Fortran')
Le type de tuple_ est <class 'tuple'>
a=1 et b=3
v1 = 1, r=[2, 3, 4, 5, 6, 7, 8, 9], v2=10
```

On voit que le type de la variable `tuple_` est `tuple`. La variable `a` est affectée avec la valeur `1` qui correspond au premier élément du tuple `t1` et la variable `b` est affectée avec la valeur `3` qui correspond au dernier élément du tuple `t1`.

La variable `v1` récupère le contenu du premier élément du tuple `t2` et qui est égal à `1`, la variable `r` est un tuple contenant les huit éléments `[2,3,4,5,6,7,8,9]`, et enfin, la variable `v2` contient la valeur du dernier élément du tuple `t2`.

Les tuples sont très utiles notamment pour empaqueter les résultats de retour d'une fonction. Les fonctions seront abordées dans la suite de ce chapitre.

4.2.6 Les dictionnaires

Un dictionnaire permet de stocker un ensemble de valeurs tout comme les listes et les tuples mais, à la différence de ces derniers, un dictionnaire permet d'indexer ses éléments en utilisant des clés. Chaque élément d'un dictionnaire est référencé avec une clé unique.

Également, nous avons vu que les éléments d'une liste sont délimités par des crochets et les éléments d'un tuple sont délimités par des parenthèses. Les éléments d'un dictionnaire sont délimités par des accolades comme le montre l'exemple de code suivant où nous allons voir comment créer des dictionnaires et comment les modifier.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *dict.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
dict1 = dict({"clé1": 1, "clé2": 2, "Clé3": "Hello"})
print('dict1 = ', dict1)
print(f"Le type de dict1 est {type(dict1)}")

dict2 = {}
dict2["clé1"] = 1
dict2["clé2"] = 2
dict2["clé3"] = 3
dict2["clé4"] = 4
print('dict2 = ', dict2)

dict2["clé2"] = "deux"
print('dict2 = ', dict2)
```

Ce code commence par la création et l'initialisation du dictionnaire `dict1` avec les couples clés/valeurs : `"clé1": 1, "clé2": 2, "Clé3": "Hello"`, puis affiche le contenu de ce dictionnaire ainsi que son type.

Le dictionnaire `dict2` a été créé sans aucun objet au début, puis les couples clé/valeur "clé1": 1, `"clé2": 2`, `"clé3": 3` et `"clé4": 4` ont été ajoutés un par un. L'avant-dernière instruction montre comment accéder à un élément d'un dictionnaire et comment modifier son contenu. Ainsi, la chaîne de caractères `"deux"` a remplacé la valeur qui était insérée précédemment dans le dictionnaire `dict2` avec la clé `clé2`. La dernière instruction affiche le nouveau contenu du dictionnaire `dict2`.

Ce code affiche les résultats suivants :

```
dict1 =  {'clé1': 1, 'clé2': 2, 'Clé3': 'Hello'}
Le type de dict1 est <class 'dict'>
dict2 =  {'clé1': 1, 'clé2': 2, 'clé3': 3, 'clé4': 4}
dict2 =  {'clé1': 1, 'clé2': 'deux', 'clé3': 3, 'clé4': 4}
```

On voit que dans Python les dictionnaires ont le type `dict`. Le dernier affichage de `dict2` montre que la valeur associée à la clé `clé2` a bien été modifiée.

© Editions ENI - All rights reserved

4.2.7 Les ensembles

Un ensemble est une structure de données qui permet de stocker une collection de données non ordonnées et sans duplication. Les éléments appartenant à un ensemble ne sont pas modifiables ; en revanche, l'ensemble lui-même est modifiable.

Les ensembles sont un moyen très intéressant pour la réalisation des opérations mathématiques telles que l'intersection, l'union, la différence et la différence symétrique sur des ensembles.

Le code de l'exemple suivant montre comment créer et modifier des ensembles.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *set-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
set_1 = {True,False,1,2,2,'SET'}
print(f"Le type de set_1 est {type(set_1)}")
print(f"set_1 = {set_1}")


set_2 = {(1,2),(1,2),(1,2,3)}
print(f"set_2 = {set_2}")

set_3 = set([6,7])
print(f"set_3 = {set_3}")

set_3.update(set_1, {8,9})
print(f"set_3 = {set_3}")

set_3.discard(8)
set_3.remove(9)
print(f"set_3 = {set_3}")
```

Hormis les instructions d'affichage avec la fonction `print`, les instructions de ce code sont expliquées comme suit :

- `set_1 = {True,False,1,2,2,'SET'}` : cette instruction crée et initialise l'ensemble `set_1` avec les six valeurs `True, False, 1, 2, 2, 'SET'`. Cet ensemble contient des éléments de types différents, à savoir les types booléen, entier et chaîne de caractères. Remarquez l'ajout de deux occurrences du chiffre 2.
- `set_2 = {(1,2),(1,2),(1,2,3)}` : cette instruction crée et initialise l'ensemble `set_2` avec deux occurrences du tuple `(1,2)` et une occurrence du tuple `(1,2,3)`.
- `set_3 = set([6,7])` : cette instruction permet de créer l'ensemble `set_3` à partir de la liste `[6,7]` en utilisant la fonction `set()`.

Remarque

Si vous tentez d'ajouter une liste à un ensemble sans utiliser la fonction `set()`, par exemple si nous avons écrit `set_3={[6,7]}` à la place de `set_3=set([6,7])`, alors le programme lève une erreur d'exécution, puisque, lors de l'ajout d'une liste, c'est une référence à cette liste qui sera ajoutée et non pas une copie des éléments qui la composent. Comme une variable associée à une liste est modifiable, une erreur est levée, car un ensemble ne peut pas accepter des objets modifiables. Dans les deux premières instructions, lorsque nous avons ajouté des éléments sans l'utilisation de la fonction `set`, ce sont en réalité des copies non modifiables de ces éléments qui ont été ajoutées. En d'autres termes, nous ne pouvons pas ajouter des variables dans un ensemble.

- `set_3.update(set_1, {8,9})` : cette instruction permet d'ajouter à l'ensemble `set_3` tous les éléments de l'ensemble `set_1` et d'ajouter aussi les éléments de l'ensemble `{8,9}`.
- `set_3.discard(8)` : cette instruction permet de supprimer l'élément `8` de l'ensemble `set_3`.
- `set_3.remove(9)` : cette instruction permet de supprimer l'élément `9` de l'ensemble `set_3`. La différence entre les fonctions `remove()` et `discard()` est qu'avec la fonction `remove`, si l'élément indiqué entre parenthèses n'existe pas dans l'ensemble en question, alors une erreur d'exécution est levée.

© Editions ENI - All rights reserved

Ce programme affiche le résultat suivant :

```
Le type de set_1 est <class 'set'>
set_1 = {False, True, 2, 'SET'}
set_2 = {(1, 2), (1, 2, 3)}
set_3 = {6, 7}
set_3 = {False, True, 2, 'SET', 6, 7, 8, 9}
set_3 = {False, True, 2, 'SET', 6, 7}
```

Les points les plus importants à retenir de ces résultats sont les suivants :

- Le type des ensembles dans le langage Python est désigné avec le mot-clé `set`.
- L'ensemble `set_1` ne contient qu'une seule occurrence du chiffre 2, alors que deux occurrences ont bien été ajoutées ! Cela est dû au fait que les éléments d'un ensemble ne sont jamais dupliqués, contrairement aux autres structures de données que nous avons vues précédemment telles que les listes et les tuples.
- Le chiffre 1 n'apparaît pas dans l'ensemble `set_1`. En effet, comme nous l'avons évoqué précédemment, la valeur booléenne `True` est considérée comme équivalente à la valeur 1 ! Dans cet exemple, c'est la valeur `True` qui apparaît lors de l'affichage de l'ensemble `set_1`, car c'est la valeur `True` qui a été ajoutée en premier avant le chiffre 1 dans l'instruction `set_1 = {True,False,1,2,2,'SET'}`. Si vous changez l'ordre d'ajout de ces deux valeurs en mettant d'abord le chiffre 1, par exemple si vous remplacez cette instruction avec l'instruction `set_1 = {False,1,True,2,2,'SET'}`, alors, lors de l'affichage de l'ensemble `set_1`, c'est le chiffre 1 qui apparaîtra au lieu de la valeur `True`.
- Une seule occurrence du tuple `(1,2)` apparaît dans l'ensemble `set_2` bien que deux occurrences `(1,2)` aient été ajoutées !
- Grâce à l'utilisation de la fonction `set()`, les deux éléments 6 et 7 ont été extraits de la liste `[6,7]` et ont été ajoutés dans l'ensemble `set_3`.
- Le deuxième affichage de l'ensemble `set_3` montre que tous les éléments de l'ensemble `set_1` et les éléments de l'ensemble `{8,9}` ont bien été ajoutés à l'ensemble `set_3`.
- Le dernier affichage de l'ensemble `set_3` montre que les deux éléments 8 et 9 ont été supprimés de l'ensemble `set_3`.

L'exemple de code suivant montre comment réaliser les opérations mathématiques de base sur des ensembles.

Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *set-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
A={1,2,3,4,5,6}
B={4,5,6,7,8,9}

print(f"A = {A}")
print(f"B = {B}")

union = A|B
print(f"A | B = {union}")

inter = A&B
print(f"A & B = {inter}")

diff_A_B = A - B
print(f"A - B = {diff_A_B}")

sym_diff = A^B
print(f"A ^ B = {sym_diff}")
```

Ce code crée deux ensembles `A` et `B` puis réalise les opérations suivantes :

- `A|B` : cette opération correspond à l'opération de l'union entre les deux ensembles `A` et `B`. Pour réaliser cette opération, nous pouvons également utiliser la notation `A.union(B)`.
- `A&B` : cette opération correspond à l'opération de l'intersection entre les deux ensembles `A` et `B`. Pour réaliser cette opération, nous pouvons également utiliser la notation `A.intersection(B)`.
- `A-B` : cette opération correspond à l'opération de la différence entre l'ensemble `A` et l'ensemble `B`. Cette opération retourne les éléments présents dans l'ensemble `A`, mais pas dans l'ensemble `B`. Attention, par définition, `A-B` est bien différent de `B-A`. Pour réaliser cette opération, nous pouvons également utiliser la notation `A.difference(B)`.

© Editions ENI - All rights reserved

– `A^B` : cette opération correspond à l'opération de la différence symétrique entre les deux ensembles `A` et `B`. Pour rappel, la différence symétrique entre deux ensembles `A` et `B` est un ensemble composé des éléments qui appartiennent soit à l'ensemble `A`, soit à l'ensemble `B`, mais pas les deux. Pour réaliser cette opération, nous pouvons également utiliser la notation `A.symmetric_difference(B)`.

Ce dernier programme affiche les résultats ci-dessous :

```
A = {1, 2, 3, 4, 5, 6}
B = {4, 5, 6, 7, 8, 9}
A | B = {1, 2, 3, 4, 5, 6, 7, 8, 9}
A & B = {4, 5, 6}
A - B = {1, 2, 3}
A ^ B = {1, 2, 3, 7, 8, 9}
```

4.2.8 Liste vs tuple vs dictionnaire vs ensemble

Les listes, les tuples, les dictionnaires et les ensembles sont les types ou les structures de données en langage Python qui permettent de stocker et d'organiser les données. Connaître les différences entre ces quatre structures de données peut vous aider à choisir une structure plutôt qu'une autre en fonction des besoins. En effet, ces structures présentent des différences importantes qui ont un impact direct sur la complexité et les performances d'un programme.

Le tableau ci-dessous donne les points communs et les différences entre ces quatre structures de données :

Critère/structure	list	tuple	dict	set
Types de données	Hétérogènes	Hétérogènes	Hétérogènes	Hétérogènes
Représentation	[v1,v2,...]	(v1,v2,...)	{(clé1/v1), (clé2/v2),... }	{v1,v2,...}
Ordre sur les données	oui	oui	oui (depuis la version 3.7)	non
Duplication des éléments	oui	oui	Non pour les clés mais oui pour les valeurs.	non
Modification	oui	non	oui	Oui sur l'ensemble mais pas sur les éléments.

Figure 2-3 : Comparaison entre les listes, les tuples, les dictionnaires et les ensembles

4.3 Structurer un code Python

4.3.1 L'indentation et les blocs de code

Un bloc de code est composé d'une ou plusieurs instructions qui forment une unité de code. Les instructions d'un bloc de code seront toutes exécutées les unes après les autres de façon séquentielle à moins qu'une de ces instructions provoque explicitement la fin de l'exécution de ce bloc ou l'arrêt du programme principal. Un programme Python est organisé sous forme d'un bloc de code principal contenant des blocs imbriqués.

Le texte ci-après montre un exemple d'organisation du code sous forme de blocs :

```
Instruction 1 du bloc 1
Instruction 2 du bloc 1:
   Instruction 1 du bloc 2
Instruction 3 du bloc 1
Instruction 4 du bloc 1:
   Instruction 1 du bloc 3
   Instruction 2 du bloc 3:
      Instruction 1 du bloc 4
      Instruction 2 du bloc 4
   Instruction 3 du bloc 3
Instruction 5 du bloc 1
```

On distingue un bloc d'un sous-bloc en utilisant un décalage vers la droite avec quatre espaces ou une tabulation. Par exemple, dans le texte précédent, l'unique instruction du bloc 2 est en retrait de quatre espaces vers la droite par rapport aux instructions du bloc 1, et les deux instructions du bloc 4 sont en retrait de quatre espaces vers la droite par rapport aux instructions du bloc 3, et ce même bloc 3 est en retrait de quatre espaces vers la droite par rapport aux instructions du bloc 1. Dans ce texte, le bloc 4 est un sous-bloc du bloc 3 et le bloc 3 est lui-même un sous-bloc du bloc 1 ; ces trois blocs forment donc des blocs imbriqués.

L'exécution d'un programme commence toujours par la première instruction du bloc du code principal, c'est-à-dire du bloc 1 dans notre exemple. L'exécution se poursuit en exécutant une à une de manière séquentielle les instructions du bloc principal et des instructions des blocs imbriqués en fonction de leur apparition dans le fichier source.

© Editions ENI - All rights reserved

Chacune des instructions d'un bloc donné peut provoquer la fin de l'exécution du bloc auquel cette instruction appartient. Le passage de l'exécution d'un bloc à un autre est réalisé suivant certaines conditions décidées par le programmeur. Dans l'exemple précédent, l'instruction du bloc 1 située juste avant le début du bloc 2, c'est-à-dire l'instruction 2 du bloc 1, décide si les instructions du bloc 2 seront exécutées ou pas. De la même manière, l'instruction 4 du bloc 1 a la tâche de décider si les instructions du bloc 3 seront exécutées ou pas et l'instruction `Instruction 2` du bloc 3 décide de l'exécution ou pas des instructions du bloc 4. En résumé, c'est la dernière instruction avant le début d'un nouveau bloc qui décide de l'exécution de ce nouveau bloc. En Python, le caractère '`:`' est placé devant toutes les instructions situées avant le début d'un nouveau bloc.

La décision d'exécuter ou pas un bloc de code est généralement réalisée avec une instruction conditionnelle. Les instructions conditionnelles seront abordées dans la suite de ce chapitre.

Remarque

En Python, comme mentionné précédemment, la délimitation des blocs de code peut se réaliser en utilisant quatre espaces ou le caractère de tabulation. Cependant, il est recommandé d'utiliser l'indentation avec les quatre espaces pour des raisons purement pratiques et de compatibilité afin de vous éviter des erreurs de compilation de certains outils qui ne prennent pas en charge l'indentation avec le caractère de tabulation.

4.3.2 Écrire une instruction sur plusieurs lignes

En Python, une instruction peut occuper plusieurs lignes. Par exemple, l'instruction suivante est tout à fait valide :

```
somme = 1+2+3 \
+4+5+6 \
+7
```

Le caractère \, utilisé comme dans ce dernier exemple, est appelé le caractère de continuation. Il permet d'indiquer à l'interpréteur Python que la définition de l'instruction en cours se poursuit sur la ligne suivante. Attention à ne pas laisser d'espaces sur la même ligne après le caractère \.

En plus du caractère de continuation, on peut également exprimer une instruction sur plusieurs lignes en encadrant cette instruction entre crochets `[]`, entre parenthèses `()` ou encore entre accolades `{}` comme dans l'exemple du code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *multi_lignes.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
s1 = 1+2+3 \
+4+5+6 \
+7
print(f"s1={s1}")

s2 = [1+2+3
+4+5+6
+7]
print(f"s2={s2}")

s3 = (1+2+3
+4+5+6
+7)
print(f"s3={s3}")

s4 = {1+2+3
+4+5+6
+7}
print(f"s4={s4}")
```

Dans ce code, les variables `s1`, `s2`, `s3` et `s4` sont calculées avec des instructions définies sur plusieurs lignes en utilisant le caractère de continuation pour `s1`, les crochets `[]` pour `s2`, les parenthèses `()` pour `s3` et les accolades `{}` pour `s4`. Ce programme affiche le résultat suivant :

```
s1=28
s2=[28]
s3=28
s4={28}
```

© Editions ENI - All rights reserved

4.3.3 Écrire plusieurs instructions sur une ligne

Dans l'exemple précédent, nous avons vu comment écrire une seule instruction sur plusieurs lignes, mais il est également possible en langage Python d'écrire plusieurs instructions sur la même ligne, séparées par des points-virgules, comme le montre l'exemple du code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *ligne_multi_instructions.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
a = 1+2+3; b=5; c=a*b; print(f"c={c}")
```

Ce code est défini par quatre instructions séparées avec des points-virgules et il donne le résultat suivant :

```
c=30
```

4.3.4 Les commentaires en Python

En langage Python, pour commenter une ligne du code, il suffit de la faire précéder par le caractère #. Pour commenter plusieurs lignes de code, il est possible de procéder, au choix, en utilisant le caractère # au début de chaque instruction à commenter, ou bien en encadrant tout le code à mettre en commentaire entre des triples guillemets simples `'''` ou doubles `"""`.

4.4 Les instructions conditionnelles

La notion d'instruction conditionnelle est primordiale, car c'est avec ce type d'instruction que nous contrôlons le flux d'exécution d'un programme. Une instruction conditionnelle implique l'évaluation d'une expression booléenne. Si l'évaluation de cette expression booléenne aboutit à la valeur `True`, alors on dit que le test de l'instruction conditionnelle a réussi ; sinon, si l'évaluation de cette expression booléenne aboutit à la valeur `False`, on dit que le test de l'instruction conditionnelle a échoué.

Si le test d'une instruction conditionnelle réussit, le bloc du code qui la succède immédiatement sera exécuté.

4.4.1 Les conditions avec la structure if

En Python, nous pouvons exprimer une condition avec le mot-clé `if` comme dans le code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *if.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
user_input = input("Veuillez saisir un nombre entier :")
n = int(user_input)
if (n % 2) == 0:
    print(f"{n} est un nombre pair")

print("fin du programme!")
```

Ce code demande à l'utilisateur de saisir un nombre entier quelconque, puis l'opération modulo `%` sera réalisée entre le nombre saisi et la valeur 2. Nous savons que si le résultat du modulo, c'est-à-dire le reste de la division, entre le nombre saisi et le chiffre 2 est égal à 0, alors le nombre saisi est pair sinon, si le résultat du modulo est 1, alors le nombre saisi est impair !

Grâce à la condition `(n % 2) == 0` utilisée avec le mot-clé `if` afin de créer une instruction conditionnelle, ce programme affiche le nombre saisi suivi du message "`est un nombre pair`" lorsque le test `if (n % 2) == 0` réussit.

Si nous exécutons ce programme en lui fournissant le nombre 10 par exemple, on obtient le résultat suivant :

```
Veuillez saisir un nombre entier :10
10 est un nombre pair
fin du programme!
```

Et si nous exécutons ce programme avec la valeur 11, le résultat suivant est affiché :

```
Veuillez saisir un nombre entier :11
fin du programme!
```

© Editions ENI - All rights reserved

- Exécutez ce programme et testez-le avec plusieurs valeurs entières différentes afin de bien comprendre le fonctionnement d'une instruction conditionnelle.

Remarque

Si vous testez ce programme en lui fournissant une chaîne de caractères en entrée, alors une erreur se produit à l'exécution et le programme s'arrête brutalement, car une exception est levée lors de la transformation vers un nombre entier avec la fonction `int` de la chaîne de caractères saisie. Nous aborderons la gestion des exceptions plus loin dans ce chapitre.

4.4.2 Les conditions avec la structure if-else

Dans l'exemple précédent, nous affichons un message lorsque le nombre saisi est pair, mais nous n'avons pas dit explicitement à notre programme où enchaîner l'exécution dans le cas où le nombre saisi est impair ! Dans ce cas, lorsque le test de l'instruction conditionnelle échoue, l'interpréteur Python enchaîne l'exécution en continuant à traiter les instructions situées au même niveau du bloc du code de l'instruction conditionnelle `if`, et le sous-bloc défini après le caractère : de cette instruction conditionnelle `if` sera tout simplement ignoré.

Si nous souhaitons que l'interpréteur Python exécute un bloc de code spécifique lorsque le test de l'instruction conditionnelle échoue, on peut utiliser le mot-clé `else` conjointement avec le mot-clé `if` comme dans l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *if-else.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
user_input = input("Veuillez saisir un nombre entier :")
n = int(user_input)
if (n % 2) == 0:
   print(f"{n} est un nombre pair")
else :
   print(f"{n} est un nombre impair")

print("fin du programme!")
```

La seule différence entre ce code et le code précédent est l'utilisation du mot-clé `else` afin d'indiquer au programme notre souhait d'exécuter le bloc du code situé après le caractère : devant le `else` dans la mesure où le test `(n % 2) == 0` échoue. Cette fois-ci, si nous exécutons ce programme en lui fournissant la valeur `11`, on obtient l'affichage suivant :

```
Veuillez saisir un nombre entier :11
11 est un nombre impair
fin du programme!
```

4.4.3 Les conditions avec la structure if-elif-else

Avec la structure conditionnelle `if-else`, nous indiquons une condition avec le mot-clé `if` et deux blocs de code : un premier bloc de code qui sera exécuté si la condition `if` réussit et un deuxième bloc de code qui sera exécuté si cette condition échoue.

La structure conditionnelle `if-elif-else` est plus expressive que la structure conditionnelle `if-else`, car avec `if-elif-else` nous pouvons réaliser plusieurs tests logiques et associer chacun de ces tests à un bloc de code spécifique, comme le montre l'exemple du code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *if-elif-else.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
user_input = input("Veuillez saisir un nombre entier :")
n = int(user_input)

if (n % 2)==0:
   print(f"{n} est un nombre pair")
   print(f"{n}**2 = {n**2}")

elif (n%3)==0:
   print(f"{n} est un multiple de 3")
   print(f"{n}**3 = {n**3}")

elif (n%5)==0:
   print(f"{n} est un multiple de 5")
   print(f"{n}**5 = {n**5}")

else :
```

© Editions ENI - All rights reserved

```
    print(f"{n} est un nombre impair, mais il n'est pas un
multiple de 3" \
" ni un multiple de 5. ")

print("fin du programme!")
```

La logique de ce nouvel exemple de code est proche de la logique de l'exemple précédent. Cette fois, grâce à l'utilisation du mot-clé `elif`, nous avons ajouté les deux tests `(n%3)==0` pour tester si oui ou non le nombre saisi est un multiple de 3 et le test `(n%5)==0` pour tester si oui ou non le nombre saisi est un multiple de 5. Nous avons également utilisé `else` afin d'indiquer un bloc de code à exécuter dans le cas où aucun des tests réalisés avec `if` et les deux `elif` ne se sera réalisé.

Remarque

L'utilisation du mot-clé `elif` n'implique pas forcément la présence de la clause `else`. En effet, nous pouvons écrire une structure conditionnelle en utilisant uniquement les mots-clés `if` et `elif`.

Les blocs de code associés aux mots-clés `if` et `elif` sont définis avec deux instructions et non pas avec une seule instruction comme dans les exemples précédents. Cette fois, si le nombre saisi est pair, un message sera affiché pour indiquer que ce nombre est pair et le résultat correspondant à ce nombre à la puissance 2 sera également affiché. Dans le cas où le nombre saisi serait un multiple de 3, alors le premier `elif` affiche un message pour indiquer que ce nombre est bien un multiple de 3 et affiche le résultat de ce nombre puissance 3. Si le nombre saisi est un multiple de 5, alors le deuxième `elif` permet d'afficher un message indiquant que ce nombre est bien un multiple de 5 et le résultat correspondant à ce nombre puissance 5 sera affiché. Enfin, dans tous les autres cas, la clause `else` affiche un message indiquant que le nombre saisi est un nombre impair et qu'il n'est un multiple ni de 3 ni de 5.

L'exécution de ce code suivie de la saisie du nombre 35 donne le résultat suivant :

```
Veuillez saisir un nombre entier :35
35 est un multiple de 5
35**5 = 52521875
fin du programme!
```

Testez ce programme avec plusieurs valeurs différentes afin de voir concrètement le fonctionnement de la structure conditionnelle `if-elif-else`.

Un point très important à prendre en considération lorsque vous utilisez la structure `if-elif-else` est d'être bien conscient que les tests successifs avec la clause `if` et les clauses `elif` peuvent s'emboîter et devenir une source d'anomalies difficilement décelables ! Par exemple, si nous exécutons le code de l'exemple précédent avec la saisie du nombre 15, nous pouvons par inadvertance penser que c'est le bloc associé au test `(n%5)==0` qui sera exécuté alors que c'est bien le bloc associé au test `(n%3)==0` qui sera réellement exécuté, puisque c'est ce test qui sera rencontré en premier par l'interpréteur Python. En résumé, il faut veiller à ce que la succession des tests utilisés avec `if-elif-else` corresponde bien à la logique que vous souhaitez implémenter.

4.5 Les boucles

Les boucles permettent d'implémenter des traitements répétitifs. Un traitement est considéré comme une partie d'un programme composée d'une ou de plusieurs instructions. C'est lorsqu'un traitement doit être exécuté plusieurs fois et de manière successive que les boucles entrent en jeu.

Avec le langage Python, les boucles peuvent être implémentées en utilisant l'un des deux mots-clés `for` et `while`. Cette section montre comment construire des boucles en utilisant ces deux mots-clés et explique comment choisir entre la boucle `for` et la boucle `while`.

4.5.1 La boucle for

Nous utilisons une boucle `for` pour répéter un traitement un nombre de fois connu à l'avance, ou du moins, connu juste avant l'exécution de cette boucle.

Par exemple, analysons le programme suivant :

```
print("itération 0")
print("itération 1")
print("itération 2")
print("itération 3")
print("itération 4")
print("itération 5")
```

© Editions ENI - All rights reserved

```
print("itération 6")
print("itération 7")
print("itération 8")
print("itération 9")
print("fin du programme!")
```

En plus de la dernière instruction qui affiche un message indiquant la fin du programme, ce code est composé de dix instructions qui affichent un message différent chacun. Ces dix instructions sont formées avec la même fonction `print`. Si nous considérons que chacun des appels à cette fonction `print` est un traitement, alors les dix exécutions de ce traitement peuvent être automatisées et remplacées par une boucle. De plus, si nous observons les dix messages affichés, on voit qu'ils sont tous composés du message `itération` suivi d'un nombre allant de 0 à 9. Nous pouvons donc en conclure que nous pouvons réécrire ce programme à l'aide d'une boucle qui fera exécuter la fonction `print` un nombre de fois égal à 10 et afficher à chaque fois le message `itération` suivi d'un numéro correspondant au rang de l'exécution en cours. Connaître à l'avance le nombre de fois que le traitement de cette boucle sera exécuté justifie le choix de l'utilisation de la boucle `for` comme dans l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
for i in range(10):
    print(f"itération {i}")
print("fin du programme!")
```

La première chose que nous pouvons observer est que les onze instructions du code précédent ont été remplacées par les trois instructions de ce dernier programme. L'exécution de ce programme donne le résultat suivant :

```
itération 0
itération 1
itération 2
itération 3
itération 4
itération 5
itération 6
itération 7
itération 8
```

```
itération 9
fin du programme!
```

La fonction `range(n)` retourne un intervalle composé des valeurs allant de `0` jusqu'à `(n-1)`. L'écriture `for i in range(10)` signifie que `i` est une variable qui va prendre tour à tour les valeurs de l'intervalle `[0,10[`, et à chaque fois que la variable `i` prend une valeur différente, le sous-bloc de code associé à la boucle `for` sera exécuté. Dans notre exemple, le sous-bloc de code associé à la boucle `for` est composé par l'unique instruction `print(f"itération {i}")`. Cette dernière instruction sera donc exécutée dix fois avec à chaque fois une valeur différente de la variable `i` comme le montre le résultat ci-dessus.

Chaque exécution d'un bloc de code associé à une boucle est appelée une itération de la boucle. Dans notre exemple, il y a donc eu dix itérations de la boucle `for` avec une valeur différente de la variable `i` à chaque itération.

À chaque fois qu'un traitement doit être exécuté de manière successive un nombre de fois connu à l'avance, la boucle `for` peut être utilisée. De plus, la variable de la boucle, comme la variable `i` dans notre exemple, permet de connaître le rang ou l'ordre de chacune des exécutions du bloc de code associé à une boucle. Cette variable associée à la boucle `for` est couramment appelée la variable d'itération, puisqu'elle permet de connaître l'ordre de l'itération en cours. La variable de la boucle est également appelée la variable compteur, puisqu'elle permet justement de compter les itérations de la boucle.

Plus haut dans ce chapitre, nous avons abordé les structures de données de type `list`. Ainsi, nous avons vu comment créer et comment modifier des listes. Le code suivant permet d'accéder aux valeurs d'une liste une par une à l'aide d'une boucle `for`.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
for value in [10,11,15,18]:
    print(f"value = {value}")
print("fin du programme!")
```

© Editions ENI - All rights reserved

Dans l'exemple précédent, l'intervalle des valeurs possibles de la variable d'itération a été construit avec la fonction `range()`. Dans ce dernier code, la variable d'itération `value` prend tour à tour les valeurs de la liste `[10,11,15,18]` comme le montre le résultat suivant :

```
value = 10
value = 11
value = 15
value = 18
fin du programme!
```

Ce dernier code montre comment accéder aux valeurs d'une liste une à une à l'aide d'une boucle `for`. Cependant, souvent lorsque nous parcourons les valeurs d'une liste, nous aimerions connaître également l'ordre ou la position de chacune de ses valeurs. Pour cela, l'exemple du code ci-dessous montre comment parcourir les valeurs d'une liste tout en ayant les indices des valeurs dans cette liste :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
for i, value in enumerate([10,11,15,18]) :
    print(f"à l'itération {i} value = {value}")
print("fin du programme!")
```

Ce dernier code ressemble au code précédent, sauf que cette fois nous avons utilisé la fonction `enumerate` sur la liste `[10,11,15,18]` pour former la variable d'itération de la boucle `for`. Plus précisément, ici nous avons deux variables d'itération `i` et `value` qui correspondent respectivement à l'indice d'une position de la liste `[10,11,15,18]` et à la valeur située à cette position. À la première exécution de cette boucle `for`, la variable `i` prend la valeur `0` et la variable `value` prend la valeur `10`. À la deuxième itération, la variable `i` prend la valeur `1` et la variable `value` prend la valeur `11`, et ainsi de suite comme le montre le résultat de l'exécution de ce code ci-dessous :

```
à l'itération 0 value = 10
à l'itération 1 value = 11
à l'itération 2 value = 15
à l'itération 3 value = 18
fin du programme!
```

En réalité, dans ce dernier exemple, la fonction `enumerate` retourne des tuples `(v1, v2)` dont les valeurs `v1` et `v2` sont réparties à chaque itération entre la variable `i` pour `v1` et la variable `value` pour `v2`.

Dans tous les exemples précédents, le bloc de code associé à la boucle `for` était composé d'une seule instruction. L'exemple suivant montre un cas un peu plus complexe de l'utilisation de la boucle `for` avec un sous-bloc de code composé de plusieurs instructions.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
for n in [1,2,3,4,5] :
    if (n % 2)==0:
        print(f"{n} est un nombre pair")
        print(f"{n}**2 = {n**2}")
    elif (n%3)==0:
        print(f"{n} est un multiple de 3")
        print(f"{n}**3 = {n**3}")
    elif (n%5)==0:
        print(f"{n} est un multiple de 5")
        print(f"{n}**5 = {n**5}")
    else :
        print(f"{n} est un nombre impair, mais il n'est pas un
multiple de 3" \
              " ni un multiple de 5. ")
print("fin du programme!")
```

Dans ce nouvel exemple de code, nous avons repris le programme réalisé précédemment lorsque nous avons abordé la structure conditionnelle `if-elif-else`. Nous avons remplacé la partie du code où l'utilisateur était invité à saisir une valeur au clavier par une boucle `for`. Ainsi, la variable d'itération `n` prend successivement les valeurs de la liste `[1,2,3,4,5]`, puis les conditions associées aux expressions logiques `(n % 2)==0`, `(n % 3)==0` et `(n % 5)==0` sont appliquées tour à tour et un message est affiché en fonction de la condition qui est réalisée. Sinon, si aucune de ces conditions n'est réalisée, le bloc du code associé à la clause `else` est exécuté. Ce dernier code donne le résultat ci-dessous :

```
1 est un nombre impair, mais il n'est pas un multiple de 3 ni un
multiple de 5.
```

© Editions ENI - All rights reserved

```
2 est un nombre pair
2**2 = 4
3 est un multiple de 3
3**3 = 27
4 est un nombre pair
4**2 = 16
5 est un multiple de 5
5**5 = 3125
fin du programme!
```

4.5.2 La boucle for et la fonction zip

Nous avons vu que la boucle `for` est utilisée afin d'exécuter un bloc de code un nombre de fois connu à l'avance, comme par exemple parcourir l'ensemble des éléments d'une structure de données telle qu'une liste. Souvent, lorsque nous développons des programmes complexes, nous avons besoin de parcourir plusieurs listes en parallèle, c'est-à-dire exécuter un bloc de code un certain nombre d'itérations, et à chaque itération, rendre accessibles les valeurs de plusieurs listes suivant l'ordre respectif de ces valeurs dans leur liste.

Par exemple, si nous disposons des deux listes `l1=[1,2,3]` et `l2=[5,6,7]`, nous souhaitons réaliser un traitement dans une boucle en considérant à la première itération de cette boucle les deux valeurs du tuple `(1,5)`, à la deuxième itération les valeurs du tuple `(2,6)`, et enfin, les deux valeurs `(3,7)`. Pour réaliser ce type d'accès, que nous pouvons qualifier d'accès parallèle aux valeurs d'un ensemble de listes, on utilise la formidable fonction `zip`.

Avant d'utiliser la fonction `zip` conjointement avec la boucle `for`, nous allons d'abord examiner son comportement via quelques exemples simples.

Dans le premier exemple ci-dessous, nous allons appliquer la fonction `zip` sur deux listes et afficher le résultat.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *zip-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list1 = [1,2,3,4,5]
list2 = ['a','b','c','d','e']
list1_list2 = list(zip(list1,list2))
print(f"list1={list1}")
print(f"list2={list2}")
```

```
print(f"list1_list2={list1_list2}")
```

Ce code crée deux listes `list1` et `list2`, puis la fonction `zip` est appelée en lui fournissant en paramètres ces deux listes. Le résultat de l'exécution de la fonction `zip` est transformé en liste avec la fonction `list` et la liste résultante est stockée dans la variable `list1_list2`. Ce programme se termine par l'affichage des trois variables `liste1`, `liste2` et `list1_list2` comme suit :

```
list1=[1, 2, 3, 4, 5]
list2=['a', 'b', 'c', 'd', 'e']
list1_list2=[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e')]
```

On voit que le résultat de l'application de la fonction `list` sur le résultat de la fonction `zip` appliquée sur les deux listes `list1` et `list2` est une liste formée de couples dont les premiers éléments, c'est-à-dire les valeurs à gauche de ces couples, sont les valeurs de la liste `list1` et les valeurs à droite sont les valeurs de la liste `list2`.

En résumé, ces résultats montrent que la fonction `zip` permet de combiner deux listes pour former une seule structure de données composée de tuples formés à partir des valeurs des listes fournies en paramètres en respectant l'ordre des valeurs des listes d'origine.

Remarque

Dans cet exemple, nous avons appliqué la fonction `zip` sur deux listes, mais nous pouvons l'appliquer sur un nombre supérieur de listes.

Dans cet exemple, nous avons appliqué la fonction `zip` sur les deux listes `list1` et `list2` qui avaient un nombre égal d'éléments. En effet, dans chacune de ces deux listes, il y avait cinq valeurs `[1, 2, 3, 4, 5]` pour `list1` et `['a', 'b', 'c', 'd', 'e']` pour `list2`. Nous pouvons appliquer la fonction `zip` sur des listes de tailles différentes. Le résultat sera calculé en se basant uniquement sur la liste de plus petite taille, comme le montre l'exemple de code ci-dessous :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *zip-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

© Editions ENI - All rights reserved

```
list1 = [1,2]
list2 = ['a','b','c','d','e']
list1_list2 = list(zip(list1,list2))
print(f"list1={list1}")
print(f"list2={list2}")
print(f"list1_list2={list1_list2}")
```

Dans ce dernier code, `list1` a deux valeurs et `list2` a cinq valeurs et le résultat de l'application de la fonction `zip` sur ces deux listes est le suivant :

```
list1=[1, 2]
list2=['a', 'b', 'c', 'd', 'e']
list1_list2=[(1, 'a'), (2, 'b')]
```

Ces résultats montrent que les valeurs 'c', 'd' et 'e' de la liste `list2` ont été tout simplement ignorées par la fonction `zip`, puisque dans la liste `list1_list2`nous ne retrouvons que les deux couples `(1, 'a')` et `(2, 'b')` qui sont constitués des deux premières valeurs de la liste `list1` et des deux premières valeurs de la liste `list2`.

Il existe des cas où nous avons besoin d'appliquer cette fonction `zip` sur des listes de taille différente, mais où nous souhaitons également que les valeurs de la liste qui a la plus grande taille soient incluses dans les résultats. Évidemment, dans ces cas-là, les dernières valeurs de la liste de plus grande taille n'auront pas de valeurs correspondantes dans les listes de plus petite taille. La solution est d'affecter des valeurs par défaut à la place des valeurs manquantes dans les listes de plus petite taille, comme le montre l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *zip-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
from itertools import zip_longest

list1 = [1,2]
list2 = ['a','b','c','d','e']
list1_list2 = list(zip_longest(list1,list2, fillvalue='X'))

print(f"list1={list1}")
print(f"list2={list2}")
print(f"list1_list2={list1_list2}")
```

Ce code commence par l'instruction `from itertools import zip_longest` qui permet d'importer la fonction `zip_longest` à partir du module `itertools`.

Comme nous l'avons indiqué précédemment, un module peut être vu comme un regroupement de fonctionnalités, développées afin de répondre à des besoins spécifiques. Nous allons découvrir bien d'autres modules au fur et à mesure de l'avancement de ce livre.

Le module `itertools` n'est pas chargé par défaut par l'interpréteur Python, et c'est bien pour cette raison que nous avons utilisé cette première instruction qui permet de charger la fonction `zip_longest` pour ainsi la rendre disponible à notre programme.

Le reste de ce programme ressemble au programme précédent, mais cette fois, au lieu d'utiliser la fonction `zip`, nous avons utilisé la fonction `zip_longest()` en lui fournissant les deux arguments `list1` et `list2` qui correspondent aux deux listes à partir desquelles les tuples vont être construits et un troisième argument `fillvalue='X'` qui indique que la valeur `X` sera utilisée pour compléter les tuples si des valeurs sont manquantes. Ce programme affiche les résultats suivants :

```
list1=[1, 2]
list2=['a', 'b', 'c', 'd', 'e']
list1_list2=[(1, 'a'), (2, 'b'), ('X', 'c'), ('X', 'd'),
('X', 'e')]
```

Vous remarquez que cette fois la liste `list1_list2` contient cinq valeurs et non pas que deux valeurs comme dans l'exemple précédent et qu'à partir du troisième tuple de cette liste, l'élément situé à gauche correspond à la valeur `X` qui a été indiquée avec l'argument `fillvalue`.

Avant de montrer comment combiner l'utilisation de la fonction `zip` avec la boucle `for`, il est intéressant de savoir que cette fonction `zip` peut être utilisée pour réaliser la fonction inverse de ce que nous avons vu lors des exemples précédents. C'est-à-dire que nous pouvons par exemple fournir à la fonction `zip` une liste de couples et obtenir en retour deux tuples séparés dont le premier tuple contiendrait toutes les valeurs situées à gauche des couples d'origine et le deuxième tuple contiendrait les valeurs situées à droite des couples d'origine comme dans l'exemple de code suivant.

© Editions ENI - All rights reserved

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *zip-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list1 = [1,2,3,4,5]
list2 = ['a','b','c','d','e']
list1_list2 = list(zip(list1,list2))
print(f"list1={list1}")
print(f"list2={list2}")
print(f"list1_list2={list1_list2}")

tuple1,tuple2 = zip(*list1_list2)
print(f"tuple1={tuple1}")
print(f"tuple2={tuple2}")
```

Ce code crée la liste `list1_list2` en appliquant la fonction `zip` sur les deux listes `list1` et `list2`. Puis, avec l'instruction `tuple1,tuple2 = zip(*list1_list2)`, les deux tuples `tuple1` et `tuple2` sont créés à partir des couples qui seraient stockés dans la liste `list1_list2`. Le résultat de l'exécution de ce code est le suivant :

```
list1=[1, 2, 3, 4, 5]
list2=['a', 'b', 'c', 'd', 'e']
list1_list2=[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e')]
tuple1=(1, 2, 3, 4, 5)
tuple2=('a', 'b', 'c', 'd', 'e')
```

On voit que `tuple1` contient toutes les valeurs situées à gauche des couples de la liste `list1_list2` alors que `tuple2` contient toutes les valeurs situées à droite de cette même liste.

La fonction `zip` est souvent utilisée avec la boucle `for` dans le but de parcourir plusieurs listes en parallèle. Dans l'exemple de code suivant, la fonction `zip` est utilisée pour parcourir les deux listes `list1` et `list2` en parallèle.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-zip-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list1 = [1,2,3,4,5]
list2 = [6,7,8,9,10]
for v1,v2 in zip(list1, list2):
    print(f"{v1} + {v2} = {v1+v2}")
```

Ce programme combine la puissance de la fonction `zip` avec la boucle `for` afin de parcourir deux à deux les valeurs des deux listes `list1` et `list2`. Dans cet exemple, le bloc de code associé à la boucle `for` se résume à l'unique appel de la fonction `print` qui permet d'afficher le résultat de l'expression `v1+v2`, avec `v1` une des valeurs de la liste `list1` et `v2` la valeur qui a la même position que `v1`, mais dans la liste `list2`. Le résultat de ce programme est le suivant :

```
1 + 6 = 7
2 + 7 = 9
3 + 8 = 11
4 + 9 = 13
5 + 10 = 15
```

On voit que les éléments des deux listes `list1` et `list2` ont été additionnés deux à deux suivant leur position dans leurs listes respectives.

Il est encore plus intéressant de parcourir les valeurs de plusieurs listes en parallèle et d'avoir le rang de chaque itération comme dans l'exemple du code suivant :

```
list1 = [1,2,3,4,5]
list2 = [6,7,8,9,10]
for i, (v1,v2) in enumerate(zip(list1, list2)):
   print(f"{i}: {v1} + {v2} = {v1+v2}")
```

Ce code ressemble beaucoup au code précédent, sauf qu'ici nous avons ajouté la variable `i` qui permet de connaître le rang de l'itération en cours. Cette variable `i` est renseignée par la fonction `enumerate` qui est elle-même appliquée sur le résultat de la fonction `zip`. Le résultat de ce programme est le suivant :

```
0: 1 + 6 = 7
1: 2 + 7 = 9
2: 3 + 8 = 11
3: 4 + 9 = 13
4: 5 + 10 = 15
```

On voit que la variable `i` indique bien l'ordre de l'itération en cours !

© Editions ENI - All rights reserved

4.5.3 La boucle while

La boucle `for` est souvent utilisée lorsque le nombre d'itérations à effectuer est connu à l'avance. La boucle `while` sert à réaliser des itérations dont le nombre n'est pas connu à l'avance. La boucle `while` est utilisée avec une ou plusieurs conditions. Ces conditions servent justement pour décider de l'arrêt ou de la continuité de l'exécution des itérations de la boucle `while`. L'exemple suivant illustre l'utilisation de la boucle `while`.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *while.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n = int(input("Veuillez saisir un chiffre entre 5 et 10 ? "))
while n in range(5,11):
    print(f"{n}**2 = {n**2}")
    n = int(input("Veuillez saisir un chiffre entre 5 et 10 ? "))
print("fin du programme!")
```

Ce programme commence par inviter l'utilisateur à saisir un chiffre entre 5 et 10 et stocke dans la variable `n` le résultat de la transformation de la chaîne de caractères saisie par l'utilisateur en un nombre entier en utilisant la fonction `int()`. La condition de la boucle `while` contrôle si la valeur de cette variable `n` est bien dans l'intervalle `[5,10]`. Si la valeur de la variable `n` est dans cet intervalle, alors le corps de la boucle `while` est exécuté, sinon le programme passe à l'instruction située après le bloc du code de cette boucle `while` et qui, dans notre exemple, correspond à la dernière instruction qui affiche le message `fin du programme !`. Dans la mesure où le chiffre saisi est dans l'intervalle `[5,10]`, le carré de ce chiffre sera calculé et affiché, puis de nouveau l'utilisateur sera invité à saisir un nouveau chiffre.

Le résultat suivant montre un exemple d'exécution de ce programme :

```
Veuillez saisir un chiffre entre 5 et 10 ? 5
5**2 = 25
Veuillez saisir un chiffre entre 5 et 10 ? 6
6**2 = 36
Veuillez saisir un chiffre entre 5 et 10 ? 5
5**2 = 25
Veuillez saisir un chiffre entre 5 et 10 ? 6
6**2 = 36
Veuillez saisir un chiffre entre 5 et 10 ? 7
```

```
7**2 = 49
Veuillez saisir un chiffre entre 5 et 10 ? 7
7**2 = 49
Veuillez saisir un chiffre entre 5 et 10 ? 8
8**2 = 64
Veuillez saisir un chiffre entre 5 et 10 ? 9
9**2 = 81
Veuillez saisir un chiffre entre 5 et 10 ? 10
10**2 = 100
Veuillez saisir un chiffre entre 5 et 10 ? 11
fin du programme!
```

On voit que tant que la valeur saisie par l'utilisateur est dans l'intervalle `[5,10]`, le bloc de la boucle `while` sera exécuté. De ce fait, le nombre d'exécutions de cette boucle ne peut pas être connu à l'avance et c'est dans cette dernière constatation que l'utilisation de la boucle `while` est justifiée par rapport à l'utilisation de la boucle `for`.

4.5.4 Contrôler les boucles avec break

Souvent, lorsque nous développons une boucle, avec `for` ou `while`, nous avons besoin d'arrêter le déroulement de cette boucle de façon prématurée, car un événement particulier s'est produit.

Dans le code suivant, nous avons utilisé la boucle `for` afin de contrôler si le nombre saisi par l'utilisateur est un nombre premier ou pas.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-break.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n = int(input("Veuillez saisir un nombre > 1 ? "))
prime=True
if n < 2:
   prime = False
else:
   for i in range(2,n):
       if (n%i)==0:
           prime=False
           break
if prime :
   print(f"Le nombre {n} est un nombre premier")
else :
```

© Editions ENI - All rights reserved

```
    print(f"Le nombre {n} n'est pas un nombre composé")
print("fin du programme!")
```

Pour contrôler si le nombre saisi par l'utilisateur est premier ou pas, nous avons divisé ce nombre dans une boucle `for` sur les nombres inférieurs à lui et supérieurs à 1, mais dès qu'un diviseur est trouvé, la variable `prime` sera affectée à `False` pour mémoriser le fait qu'un diviseur a été trouvé et cette boucle `for` s'arrête en utilisant la clause `break`. Dans ce programme, si le nombre `n` est strictement inférieur à 2, il sera considéré comme étant un nombre composé, donc pas un nombre premier.

Dans cet exemple, nous avons utilisé la boucle `for`, car nous savons que le nombre maximal d'itérations est fixé par le nombre `(n-1)`. En revanche, nous savons que si un diviseur du nombre `n` existe, alors il ne sera pas nécessaire d'exécuter les itérations restantes de cette boucle, puisqu'un seul diviseur supérieur à 1 et inférieur à `n` suffit pour déclarer qu'un nombre n'est pas premier, d'où la justification d'arrêter le déroulement des itérations avec `break`.

4.5.5 Contrôler les boucles avec continue

Avec la clause `continue`, on peut décider d'arrêter l'exécution d'une itération d'une boucle, mais pas les prochaines itérations ! En effet, dans certaines conditions, on aimerait interrompre l'exécution de l'itération en cours pour continuer à partir de la prochaine itération.

Le code suivant permet d'afficher le carré des chiffres pairs entre 1 et une valeur saisie au clavier.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *for-continue.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n = int(input("Veuillez saisir un nombre > 1 ? "))

for i in range(1, n+1):
    if (i%2)!=0 :
        continue
    print(f"{i}**2 = {i**2}")

print("fin du programme!")
```

La boucle `for` de ce code parcourt les valeurs de 1 jusqu'à `n` et à chaque fois qu'une de ces valeurs est impaire, l'instruction `print` située au sein du bloc de code de cette boucle `for` sera ignorée et une nouvelle itération sera considérée.

L'exécution de ce code avec la valeur 10 saisie au clavier donne le résultat ci-dessous :

```
Veuillez saisir un nombre > 1 ? 10
2**2 = 4
4**2 = 16
6**2 = 36
8**2 = 64
10**2 = 100
fin du programme!
```

On voit que les valeurs 3, 5, 7 et 9 ont été ignorées par le programme ! En effet, pour ces valeurs, l'instruction `continue` permet de repartir du début de la boucle `for` en ignorant tout ce qui se trouve après cette clause `continue`.

Remarque

Bien que les clauses `break` et `continue` peuvent s'avérer intéressantes et justifiées, il faut les éviter autant que possible, car elles peuvent introduire de la complexité qui nuit à la bonne lisibilité des programmes. En résumé, `break` et `continue` sont utiles, mais à utiliser avec modération.

4.6 Les fonctions

Nous avons vu que les boucles permettent d'automatiser un traitement dont lequel un bloc de code doit être exécuté plusieurs fois de façon successive. En revanche, comment faire lorsqu'un bloc de code doit être exécuté plusieurs fois, mais pas forcément de manière successive ?

Lorsque les instructions d'un bloc de code apparaissent plusieurs fois dans des endroits différents de notre programme, nous pouvons tout simplement factoriser ces instructions dans une seule et même partie de ce programme. Cette partie du programme où les instructions de plusieurs blocs de code ont été factorisées est appelée une procédure ou un sous-programme.

© Editions ENI - All rights reserved

Cette procédure sera appelée autant de fois que nécessaire à partir d'endroits différents d'un programme. Si l'exécution d'une procédure se termine en retournant une valeur, alors cette procédure sera considérée comme étant une fonction.

Remarque

Contrairement à certains langages de programmation qui distinguent explicitement le concept de procédure de celui de fonction, comme le langage Pascal, avec le langage Python, ces deux concepts se définissent avec la même syntaxe et les procédures sont considérées comme des fonctions.

Nous avons déjà rencontré et utilisé le concept de fonction depuis le début de ce chapitre. Par exemple, nous avons utilisé la fonction `print` à chaque fois que nous avons affiché les résultats de nos programmes précédents, ou encore nous avons utilisé la fonction `input` pour inviter l'utilisateur à saisir des valeurs à l'aide du clavier. Les fonctions telles que `print` et `input` sont prédéfinies nativement dans le langage Python et c'est bien pour cette raison que pour les utiliser il a suffi de les appeler en utilisant leur nom. Cette section montre justement comment définir vos propres fonctions.

4.6.1 Définir et utiliser une fonction sans paramètre

Le code de l'exemple suivant montre comment définir et appeler une fonction sans paramètre.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def print_random_list():

    rd_list = []
    for i in range(5):
        rd_list.append(rd.randint(1,10))

    print(f"rd_list={rd_list}")

rd.seed(1)
print_random_list()
```

Cet exemple commence par importer le module `random`. Ce module permet d'utiliser un générateur de nombres aléatoires.

Le mot-clé `def` permet de déclarer et de définir la fonction `print_random_list`. Remarquez que le nom de cette fonction est suffixé avec les deux parenthèses `()` et que rien n'a été mentionné entre ces parenthèses. Cela signifie que la fonction `print_random_list` n'attend pas de paramètres pour s'exécuter. Les deux points à la fin de la déclaration de cette fonction indiquent que le bloc du code suivant correspond à sa définition.

Le corps de la fonction `print_random_list` commence par la définition de la liste vide `rd_list`. Puis, dans une boucle `for` de cinq itérations, les éléments de cette liste seront générés de manière aléatoire en utilisant la fonction `randint` du module `random`. Chaque élément de la liste `rd_list` sera aléatoire, mais compris dans l'intervalle `[1,10]` qui est défini par les deux arguments de cette fonction `randint`. La dernière instruction du bloc de code de la fonction `print_random_list` permet d'afficher la liste `rd_list`.

Après la définition de la fonction `print_random_list` et de retour au programme principal, l'instruction `rd.seed(1)` permet d'initialiser le générateur de nombres aléatoires afin que, d'une exécution à une autre, les nombres générés soient les mêmes. La dernière instruction de ce programme appelle tout simplement la fonction `print_random_list` en évoquant son nom suivi des parenthèses vides `()`.

L'exécution de ce programme affiche le résultat suivant :

```
rd_list=[3, 10, 2, 5, 2]
```

Remarque

Comme mentionné précédemment, si vous exécutez ce programme plusieurs fois, vous obtenez les mêmes valeurs de la liste `rd_list`, puisque le générateur de nombres aléatoires est initialisé avec la même valeur 1. Si vous supprimez l'instruction `rd.seed(1)` de votre programme ou si vous utilisez un autre nombre que 1 lors de l'appel de cette fonction, par exemple si vous l'appelez avec `rd.seed(2)`, alors il y a de très fortes chances que vous obteniez des valeurs différentes de la liste `rd_list` !

© Editions ENI - All rights reserved

Remarque

Pour générer une liste de cinq nombres aléatoires compris entre 1 et 10, comme dans notre exemple, nous aurions pu utiliser directement la fonction `rd.choices(range(1,11),k=5)` *sans avoir à écrire le code de la fonction* `print_random_list` *! Cependant, le code de notre exemple nous permet d'introduire le concept de fonction de manière pédagogique.*

Lorsque nous développons un programme Python, il est très important de distinguer le programme principal des sous-programmes correspondant aux fonctions qui, elles, sont définies au sein de ce programme principal. Par exemple, dans le code précédent, les trois instructions :

```
import random as rd
...
rd.seed(1)
print_random_list()
```

sont définies au sein du programme principal. Le bloc du code défini à la suite des deux points : situé juste après le nom de la fonction `print_random_list` est appelé le corps de cette fonction et considéré comme un sous-programme du programme principal.

4.6.2 Les fonctions avec paramètres

La fonction `print_random_list` que nous avions définie précédemment n'attend pas d'arguments puisque, lors de sa définition, nous avons laissé les deux parenthèses `()` vides. À chaque fois que cette fonction sera appelée, elle affichera une liste de cinq valeurs aléatoires comprises entre 1 et 10. Justement, la limitation de cette fonction réside dans le fait que, d'une exécution à une autre, le nombre d'éléments dans la liste `rd_list` sera toujours de cinq et chaque élément de cette liste sera toujours compris entre 1 et 10. Cela peut être suffisant pour les besoins d'un programme. Cependant, lorsque nous développons une fonction, il est intéressant de la rendre la plus générique possible afin d'élargir les possibilités de son utilisation ! Par exemple, nous pouvons réécrire la fonction `print_random_list` comme dans le programme suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def print_random_list(n, min_val, max_val):

    rd_list = []
    for i in range(n):
        rd_list.append(rd.randint(min_val, max_val))

    print(f"rd_list={rd_list}")

rd.seed(1)
print_random_list(5, 1, 10)
print_random_list(10, 1, 100)
```

Les différences entre ce nouveau programme et le programme précédent sont les suivantes :

- Lors de la déclaration de la fonction `print_random_list`, les parenthèses ne sont plus vides ! Cette fois, nous avons déclaré explicitement que cette fonction attend les trois valeurs `n`, `min_val` et `max_val` pour être exécutée.
- Dans le bloc de code de la fonction `print_random_list`, nous avons utilisé la valeur du paramètre `n` pour fixer le nombre d'itérations de la boucle `for`. Également, nous avons utilisé les valeurs des paramètres `min_val` et `max_val` afin de fixer respectivement les bornes inférieure et supérieure de l'intervalle de définition des valeurs de la liste `rd_list`.
- Nous avons aussi appelé à deux reprises la fonction `print_random_list`. Le premier appel a été effectué avec les arguments 5, 1 et 10 pour reproduire le même comportement de cette fonction que celui de l'exemple précédent et générer une liste de cinq nombres aléatoires compris entre 1 et 10. Le deuxième appel a été effectué avec les arguments 10, 1 et 100 permet de générer une liste de dix nombres aléatoires compris entre 1 et 100.

L'exécution de ce programme affiche le résultat suivant :

```
rd_list=[3, 10, 2, 5, 2]
rd_list=[64, 98, 58, 61, 84, 49, 27, 13, 63, 4]
```

La différence entre la définition de la fonction `print_random_list` de ce dernier programme est clairement plus générique que sa définition dans le programme précédent !

© Editions ENI - All rights reserved

4.6.3 Les valeurs par défaut des paramètres

Lors de la définition de la fonction `print_random_list` précédente, nous avons utilisé des paramètres que l'on appelle des paramètres positionnels. Ils sont positionnels car l'ordre de la transmission des arguments `n`, `min_val` et `max_val` lors de l'appel de la fonction `print_random_list` est important ! Ainsi, dans l'appel `print_random_list(10,1,100)`, la valeur 10 qui correspond à la valeur de l'argument situé à la première position sera associée avec le paramètre `n`, la valeur `1` qui correspond à la valeur de l'argument situé à la deuxième position sera associée avec le paramètre `min_val` et la valeur `100` qui correspond à la valeur de l'argument situé à la troisième position sera associée avec le paramètre `max_val`.

Pour nous débarrasser de la contrainte où l'ordre de la passation des arguments d'une fonction est important, nous pouvons utiliser des paramètres par mot-clé lors de la définition d'une fonction, comme dans le code de l'exemple suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def print_random_list(n = 5, min_val =1, max_val=100):
    rd_list = []
    for i in range(n):
        rd_list.append(rd.randint(min_val, max_val))
    print(f"rd_list={rd_list}")

rd.seed(1)
print_random_list(10, 100, 200)
print_random_list()
print_random_list(min_val = -100, max_val=100, n=4)
print_random_list(min_val = 50, max_val=55)
print_random_list(n=6)
```

Dans ce nouveau code, le corps de la fonction `print_random_list` est identique par rapport au corps de cette même fonction de l'exemple précédent. En revanche, cette fois nous avons précisé des valeurs par défaut pour les paramètres `n`, `min_val` et `max_val` et qui sont respectivement les valeurs `5`, `1` et `100`.

Affecter des valeurs par défaut aux paramètres d'une fonction a deux impacts sur l'utilisation de cette fonction : le premier impact est que lors de l'appel à cette fonction, un paramètre qui a une valeur par défaut devient facultatif. Le deuxième impact est que l'ordre du passage des arguments à cette fonction peut ne pas correspondre à l'ordre dans lequel les paramètres ont été définis. Ces deux impacts sont illustrés par les cinq appels de la fonction `print_random_list` à la fin du code de ce dernier exemple et qui ont la signification suivante :

- Dans l'appel `print_random_list(10, 100, 200)`, le paramètre `n` sera égal à 10, le paramètre `min_val` sera égal à 100 et le paramètre `max_val` sera égal à 200.
- Dans l'appel `print_random_list()`, les trois paramètres `n`, `min_val` et `max_val` vont prendre leurs valeurs par défaut ! C'est-à-dire que le paramètre `n` sera égal à 5, le paramètre `min_val` sera égal à 1 et le paramètre `max_val` sera égal à 100.
- Dans l'appel `print_random_list(min_val = -100, max_val=100, n=4)`, le paramètre `n` sera égal à 4, le paramètre `min_val` sera égal à -100 et le paramètre `max_val` sera égal à 100.
- Dans l'appel `print_random_list(min_val = 50, max_val=55)`, le paramètre `n` sera égal à sa valeur par défaut est qui est égal à 5, le paramètre `min_val` sera égal à 50 et le paramètre `max_val` sera égal à 55.
- Dans l'appel `print_random_list(n=6)`, le paramètre `n` sera égal à 6 et les paramètres `min_val` et `max_val` seront égaux à leur valeur par défaut qui sont respectivement 1 et 100.

Le code de cet exemple affiche les résultats suivants :

```
rd_list=[117, 172, 197, 108, 132, 115, 163, 197, 157, 160]
rd_list=[84, 49, 27, 13, 63]
rd_list=[-93, -1, 10, 55]
rd_list=[50, 55, 53, 52, 55]
rd_list=[30, 76, 14, 41, 4, 3]
```

En règle générale, dès qu'un paramètre est associé à une valeur par défaut, ce paramètre devient un paramètre par mot-clé et donc un paramètre facultatif !

© Editions ENI - All rights reserved

Si l'appel à une fonction ne contient pas de valeur pour un argument correspondant à un paramètre par mot-clé, alors c'est la valeur par défaut de ce paramètre qui sera utilisée.

Comme nous l'avons vu, une instruction qui permet de définir une fonction commence par le mot-clé `def` suivi du nom de la fonction à définir, puis le nom de cette fonction est suivi de la liste des paramètres de cette fonction qui sont indiqués entre parenthèses et le caractère : se positionne à la fin de cette instruction pour indiquer le début du corps de cette fonction. Dans une telle instruction, le nom de la fonction ainsi que les paramètres indiqués entre parenthèses sont appelés la signature de cette fonction.

Dans la signature d'une fonction, nous pouvons utiliser des paramètres positionnels en même temps que des paramètres par mot-clé. Dans ce cas-là, il faut toujours commencer par la déclaration des paramètres positionnels suivis par les paramètres par mot-clé comme dans l'exemple suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def print_random_list(n, min_val =1, max_val=100):
    rd_list = []
    for i in range(n):
        rd_list.append(rd.randint(min_val, max_val))
    print(f"rd_list={rd_list}")

rd.seed(1)
print_random_list(5)
print_random_list(5, min_val= 90)
```

Dans cet exemple, le paramètre n est un paramètre positionnel, puisqu'il n'a pas de valeur par défaut, alors que les deux autres paramètres `min_val` et `max_val` sont des paramètres par mot-clé. De ce fait, lors de l'appel à la fonction `print_random_list`, le premier argument est obligatoire et il correspondra toujours au paramètre n, alors que les arguments `min_val` et `max_val` sont facultatifs, comme le montrent les deux dernières instructions de notre exemple.

Ce programme affiche le résultat suivant :

```
rd_list=[18, 73, 98, 9, 33]
rd_list=[91, 97, 97, 97, 100]
```

Le premier appel `print_random_list(5)` mentionne l'argument obligatoire correspondant au paramètre positionnel n, mais ne mentionne pas les arguments correspondant aux deux autres paramètres `min_val` et `max_val` qui sont des paramètres par mot-clé et donc facultatifs. Également, le deuxième appel `print_random_list(5, min_val= 90)` mentionne l'argument obligatoire correspondant au paramètre n, mais uniquement l'argument correspondant au paramètre `min_val` qui est mentionné.

Si, lors de l'appel à une fonction définie avec des paramètres positionnels et des paramètres par mot-clé, nous commençons par mentionner les arguments correspondant à un des paramètres par mot-clé, alors l'interpréteur Python nous signale une erreur. Par exemple, une erreur est affichée si vous remplacez la dernière instruction du dernier exemple par l'instruction suivante :

```
print_random_list(min_val= 90, 5)
```

En résumé, lors de la déclaration d'une fonction, il faut toujours commencer par la déclaration des paramètres positionnels suivis de la déclaration des paramètres par mot-clé, et lors de l'appel à une telle fonction, il faut toujours commencer par les arguments correspondant aux paramètres positionnels en respectant l'ordre de leur déclaration suivis par les arguments correspondant aux paramètres par mot-clé pour lesquels l'ordre n'est pas important.

4.6.4 Renvoi de résultats

Une fonction peut renvoyer un résultat à la suite de son exécution, comme le montre l'exemple suivant :

Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def get_random_list(n, min_val =1, max_val=100):
    rd_list = []
    for i in range(n):
```

© Editions ENI - All rights reserved

```
        rd_list.append(rd.randint(min_val, max_val))

    return rd_list

rd.seed(1)

my_list = print_random_list(10, 1,100)
print(f"rd_list={my_list}")
```

Ce nouveau code reprend l'idée de l'exemple précédent, mais cette fois nous avons supprimé l'instruction d'affichage de la liste `rd_list` qui était définie dans le bloc de code de la fonction `print_random_list` du programme précédent et nous avons ajouté dans ce bloc de code une instruction particulière qui est l'instruction `return rd_list`. Cette dernière instruction permet de retourner la liste `rd_list` à chaque fois que la fonction `get_random_list` est appelée. Dans le programme principal, la liste `my_list` reçoit le résultat de l'exécution de la fonction `get_random_list`, puis la liste `my_list` est affichée avec la dernière instruction. Ce programme affiche le résultat suivant :

```
rd_list=[18, 73, 98, 9, 33, 16, 64, 98, 58, 61]
```

Dans cet exemple, la fonction `get_random_list` a renvoyé une liste, mais de façon générale, nous pouvons renvoyer n'importe quelle structure de données à l'aide de l'instruction `return`.

4.6.5 La portée des variables

Les variables définies au sein d'un bloc de code d'une fonction sont appelées des variables locales contrairement aux variables définies au niveau du programme principal qui, elles, sont appelées des variables globales. Pour mieux comprendre les notions de variables locales et de variables globales, examinons le programme suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *local-var.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n=10

def my_function():
    n=2
```

```
    print(f"Dans la fonction my_function n= {n}")

print(f"Dans le programme principal n= {n}")
my_function()
print(f"Dans le programme principal n= {n}")
```

Ce programme commence par la déclaration d'une variable globale n=10 qui est située au niveau du programme principal. La fonction my_function définit elle aussi une variable n=2. Cette dernière variable est locale puisqu'elle est définie au sein d'un sous-programme my_function !

L'exécution de ce programme affiche le résultat suivant :

```
Dans le programme principal n= 10
Dans la fonction my_function n= 2
Dans le programme principal n= 10
```

La première ligne de ces résultats montre la valeur de la variable n avant l'appel à la fonction my_function.

La deuxième ligne correspond au résultat de print de la fonction my_function. Le résultat de cette première ligne montre que la variable n est égale à 2 et c'est sans surprise, puisque nous lui avons explicitement affecté la valeur 2 en début du bloc de code de la fonction my_function. En revanche, la troisième ligne, qui correspond au résultat de la fonction print située au sein du programme principal, montre que la variable n est égale à 10 ! En effet, l'affectation n=2 dans la fonction my_function n'a aucun effet sur la variable n du programme principal ! En réalité, lors de l'exécution de l'instruction n=2 de la fonction my_function, il y avait simultanément dans la mémoire de l'ordinateur deux variables distinctes et qui portent le même nom n. En d'autres termes, la variable n de la fonction my_function est une variable locale à cette fonction et elle occupe des bits mémoire différents des bits mémoires occupés par la variable globale n du programme principal.

Plus encore, si vous remplacez l'instruction n=2 par n=n+2 dans la fonction my_function, vous remarquerez que le programme signale une erreur à l'exécution car, bien que la variable n existe dans le programme principal, lors de l'évaluation de l'expression n+2 au sein de cette fonction my_function, l'interpréteur Python ne connaît pas encore la variable n et donc il ne pourra pas réaliser cette opération d'addition.

© Editions ENI - All rights reserved

En synthèse, une fois dans une fonction, l'interpréteur Python ne reconnaît plus les variables définies au sein du programme principal.

Dans la pratique, nous avons souvent besoin que des fonctions utilisent des variables globales. Pour cela, il faut utiliser le mot-clé `global`, comme le montre l'exemple du code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *global-var.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n=10

def my_function():
    global n
    n=2
    print(f"Dans la fonction my_function n= {n}")


print(f"Dans le programme principal n= {n}")
my_function()

print(f"Dans le programme principal n= {n}")
```

Ce programme est quasiment le même que le programme précédent, mais cette fois nous avons utilisé l'instruction `global n` au début de la fonction `my_function`. En mentionnant au sein de la fonction `my_function` que la variable n est globale, au moment de l'exécution de l'instruction n=2, l'interpréteur Python ne va pas créer une deuxième variable, mais il va plutôt utiliser la variable n déjà définie au sein du programme principal. Dans ce cas, toute modification de la variable n au sein de la fonction `my_function` persistera à la fin de l'exécution de cette fonction. Ainsi, l'exécution de ce nouveau code affiche le résultat suivant :

```
Dans le programme principal n= 10
Dans la fonction my_function n= 2
Dans le programme principal n= 2
```

On voit bien que cette fois la variable n qui était égale à 10 dans le programme principal avant l'appel de la fonction `my_function` a été modifiée après l'exécution de cette fonction.

4.6.6 Passage d'arguments à une fonction

Avec le langage Python, lorsqu'une fonction est appelée, ses paramètres sont affectés avec les références mémoire des arguments utilisés lors de cet appel.

Attention, pour ceux qui ont l'habitude de travailler avec des langages tels que C, C++ ou Java, avec le langage Python on ne peut pas passer des arguments par valeurs. Tous les arguments seront passés par référence. De plus, toute modification dans une fonction d'une variable de type immuable, tel que le type `int` ou le type `str`, ne sera pas persistante après l'exécution de cette fonction. En effet, lorsque nous modifions la valeur d'un objet de type immuable, c'est un autre objet avec une autre référence mémoire qui sera réellement créé. Pour mieux comprendre le mécanisme du passage des arguments à une fonction, examinons le code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-5.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
def change_param(int_param, str_param, list_param):

    int_param=10
    str_param = "Un trou de ver n'est pas forcément transparent"
    list_param.append(10)

    print(f"Dans change_param
:\n\tint_param={int_param}\n\tstr_param={str_param}\n\tlist_param
={list_param}\n")


my_int=1
my_str="Un trou noir c'est troublant"
my_list=[1,2,3]

print(f"Dans le programme principal:
\n\tmy_int={my_int}\n\tmy_str={my_str}\n\tmy_list={my_list}\n")

change_param(my_int, my_str, my_list)

print(f"Dans le programme principal:
\n\tmy_int={my_int},\n\tmy_str={my_str}\n\tmy_list={my_list}\n")
```

© Editions ENI - All rights reserved

Dans le programme principal, nous avons déclaré les trois variables suivantes :

- La variable `my_int` de type `int` qui est donc une variable immuable.
- La variable `my_str` de type `str` qui est donc une variable immuable.
- La variable `my_list` de type `list` qui est donc une variable modifiable.

Puis nous avons affiché les valeurs de ces trois variables. Ensuite, nous avons appelé la fonction `change_param` en lui fournissant ces trois variables en arguments. De retour au programme principal, nous avons de nouveau affiché les valeurs de ces trois variables.

La fonction `change_param` modifie les valeurs de ses paramètres puis affiche ces nouvelles valeurs.

L'exécution de ce nouveau programme affiche le résultat suivant :

```
Dans le programme principal:
       my_int=1
       my_str=Un trou noir c'est troublant
       my_list=[1, 2, 3]

Dans change_param :
       int_param=10
       str_param=Un trou de ver n'est pas forcément transparent
       list_param=[1, 2, 3, 10]

Dans le programme principal:
       my_int=1,
       my_str=Un trou noir c'est troublant
       my_list=[1, 2, 3, 10]
```

Ces résultats montrent qu'il n'y a que la variable `my_list` pour laquelle la modification dans la fonction `change_param` a persisté après l'exécution de cette fonction. Cela correspond au résultat attendu, puisqu'il n'y a que cette variable qui est d'un type modifiable. Les modifications sur les deux autres variables, `my_int` et `my_str`, n'ont pas persisté après l'exécution de la fonction `change_param`, car elles sont de type immuable.

Dans la fonction `change_param`, lorsque l'affectation `int_param=10` a été réalisée, l'interpréteur Python a réellement créé un autre objet différent de l'objet dont la référence a été reçue en paramètre. Idem lors de l'affectation de la variable `str_pram` dans la fonction `change_param`.

Si nous voulons persister dans le programme appelant des modifications d'une fonction sur un objet de type immuable, il est obligatoire de retourner la valeur de cet objet avec l'instruction `return` dans la fonction qui réalise la modification. Dans ce cas, de retour au programme appelant, il suffit de recevoir la valeur retournée par cette fonction dans l'objet pour lequel nous souhaitons persister ces modifications, comme dans l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saïsissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-6.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
def change_param(int_param, str_param, list_param):

    int_param=10
    str_param = "Un trou de ver n'est pas forcément transparent"
    list_param.append(10)

    print(f"Dans change_param :
\n\tint_param={int_param}\n\tstr_param={str_param}\n\tlist_param=
{list_param}\n")

    return (int_param, str_param)


my_int=1
my_str="Un trou noir c'est troublant"
my_list=[1,2,3]

print(f"Dans le programme principal
:\n\tmy_int={my_int}\n\tmy_str={my_str}\n\tmy_list={my_list}\n")


my_int,my_str=change_param(my_int, my_str, my_list)

print(f"Dans le programme principal
:\n\tmy_int={my_int},\n\tmy_str={my_str}\n\tmy_list={my_list}\n")
```

© Editions ENI - All rights reserved

Ce programme est similaire au programme précédent, mais cette fois nous avons ajouté l'instruction `return (int_param, str_param)` à la fin de la fonction `change_param` et nous avons reçu le résultat de cette fonction dans les variables `my_int` et `my_str` avec l'instruction :

```
my_int,my_str=change_param(my_int, my_str, my_list)
```

Le résultat de l'exécution de ce programme est le suivant :

```
Dans le programme principal:
      my_int=1
      my_str=Un trou noir c'est troublant
      my_list=[1, 2, 3]

Dans change_param :
      int_param=10
      str_param=Un trou de ver n'est pas forcément transparent
      list_param=[1, 2, 3, 10]

Dans le programme principal:
      my_int=10,
      my_str=Un trou de ver n'est pas forcément transparent
      my_list=[1, 2, 3, 10]
```

Cette fois, toutes les modifications au sein de la fonction `change_param` ont persisté dans le programme principal.

Remarque

Dans la mesure du possible, éviter les effets de bord des fonctions est une bonne pratique, car cela peut rendre l'analyse d'un code plus complexe.

4.6.7 Les fonctions récursives

Nous avons vu qu'une fonction est tout simplement un sous-programme qui peut être appelé de n'importe quel endroit d'un programme à partir du moment où cette fonction a été définie.

Une fonction peut également appeler d'autres fonctions, voire s'appeler elle-même ! Lorsqu'une fonction fait des appels à elle-même, elle est dite récursive. L'exemple du code suivant montre une fonction récursive qui permet de calculer la factorielle d'un nombre entier reçu en paramètre.

Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *rec_function.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
def factorial(n):
    if n<2:
        return 1
    else:
        return n*factorial(n-1)

print(f"4!={factorial(4)}")
```

La fonction `factorial` est une fonction récursive puisque nous trouvons un appel à elle-même au sein de son propre bloc de code. Cette fonction retourne la valeur 1 si la valeur de son paramètre est inférieure à 2 ; sinon, si la valeur de son paramètre est supérieure à 2, elle calcule l'expressionn`*factorial(n-1)`. Dans cette dernière expression, l'appel récursif à la fonction `factorial` a été réalisé en utilisant un argument égal à la valeur du paramètre n moins 1 !

Afin de mieux comprendre la notion de fonction récursive, nous allons dérouler notre programme étape par étape pour calculer `4!`. Sachant que `4! = 4*3*2*1 =24`, voici les étapes du déroulement de notre programme :

1. La première étape correspond au premier appel à la fonction `factorial` avec la valeur 4 au niveau de la dernière ligne du programme principal. À cet instant, l'interpréteur Python alloue un espace mémoire pour l'exécution de la fonction `factorial` en initialisant son unique paramètre n avec la valeur 4.
2. À cette deuxième étape, l'interpréteur Python exécute le bloc du code de la fonction `factorial` ; il s'agit de la première exécution de ce bloc de code. À ce stade, la condition `if 4<2` échoue et c'est donc le bloc associé à la clause `else` qui est exécuté et qui est sous la forme :

```
return 4 * factorial(3)
```

© Editions ENI - All rights reserved

À ce stade, l'interpréteur Python doit d'abord exécuter `factorial(3)` avant de finaliser l'exécution de cette dernière instruction. De ce fait, il sauvegarde le contexte de l'exécution de la fonction `factorial(4)` et il appelle une fonction `factorial` en lui fournissant l'argument 3. En effet, pour l'interpréteur Python, l'appel récursif à la fonction `factorial` sera traité comme tout autre appel de fonction non récursif !

Donc, on se rappellera qu'à l'étape 2, il y a bien une fonction qui attend le résultat de `factorial(3)` avant de calculer et retourner `4 * factorial(3)`.

3. À cette troisième étape, l'interpréteur Python procède à l'exécution de la fonction `factorial(3)`. Il alloue un espace mémoire pour l'exécution de la fonction `factorial` avec le paramètre n initialisé avec la valeur 3.

4. À cette quatrième étape, l'interpréteur Python exécute le bloc du code de la fonction `factorial` ; il s'agit de la deuxième exécution de ce bloc de code. À ce stade, la condition `if 3<2` échoue, et c'est donc le bloc associé à la clause `else` qui est exécuté et qui est sous la forme :

```
return 3 * factorial(2)
```

À ce stade, l'interpréteur Python doit d'abord exécuter `factorial(2)` avant de finaliser l'exécution de cette dernière instruction. De ce fait, il sauvegarde le contexte de l'exécution de la fonction `factorial(3)` et il appelle une fonction `factorial` en lui fournissant l'argument 2. Comme lors de l'appel récursif précédent, pour l'interpréteur Python, l'appel récursif `factorial(2)` sera traité comme tout autre appel de fonction non récursif !

Donc, on se rappellera qu'à l'étape 4, il y a bien une fonction qui attend le résultat de `factorial(2)` avant de calculer et retourner la valeur de `3 * factorial(2)`.

5. À cette cinquième étape, l'interpréteur Python procède à l'exécution de la fonction `factorial(2)`. Il alloue un espace mémoire pour l'exécution de cette fonction `factorial` avec le paramètre n initialisé avec la valeur 2.

6. À cette sixième étape, l'interpréteur Python exécute le bloc du code de la fonction `factorial` ; il s'agit de la troisième exécution de ce bloc de code. Encore à ce stade, la condition `if 2<2` échoue et c'est donc le bloc associé à la clause `else` qui est exécutée et qui est sous la forme :

   ```
   return 2 * factorial(1)
   ```

   À ce stade, l'interpréteur Python doit d'abord exécuter `factorial(1)` avant de finaliser l'exécution de cette dernière instruction. De ce fait, il sauvegarde le contexte de l'exécution de la fonction `factorial(2)` et il appelle une fonction `factorial` en lui fournissant l'argument `1`. Comme lors de l'appel récursif précédent, pour l'interpréteur Python, l'appel récursif `factorial(1)` sera traité comme tout autre appel de fonction non récursif !

   Donc, on se rappellera qu'à l'étape 6, il y a une fonction qui attend le résultat de `factorial(1)` avant de calculer et retourner la valeur de `2 * factorial(1)`.

7. À cette septième étape, l'interpréteur Python procède à l'exécution de la fonction `factorial(1)`. Il alloue un espace mémoire pour l'exécution de cette fonction `factorial` avec le paramètre `n` initialisé avec la valeur `1`.

8. À cette huitième étape, l'interpréteur Python exécute le bloc du code de la fonction `factorial`, il s'agit de la quatrième exécution de ce bloc de code. Cette fois, la condition `if 1<2` réussit et c'est le bloc de la clause `if` qui est exécuté. Donc l'instruction `return 1` est exécutée afin de retourner la valeur `1` à la fonction appelante au niveau de l'étape 6 précédente, et ainsi l'exécution de l'appel `factorial(1)` prend fin. Cette fois, il n'y a pas eu d'appel récursif !

© Editions ENI - All rights reserved

9. De retour à l'étape 6, l'instruction qui était en attente et qui pour rappel est l'instruction suivante :

   ```
   return 2 * factorial(1)
   ```

   se transforme en :

   `return 2 * 1` (car `factorial(1)` a retourné 1)

   Puis l'expression `(2*1)` est évaluée et la valeur résultante de 2 est retournée à la fonction appelante de l'étape 4.

10. De retour à l'étape 4, l'instruction qui était en attente et qui, pour rappel, est l'instruction suivante :

    ```
    return 3 * factorial(2)
    ```

    se transforme en :

    `return 3 * 2` (car `factorial(2)` a retourné 2)

    Puis l'expression `(3*2)` est évaluée et la valeur résultante de 6 est retournée à la fonction appelante de l'étape 2.

11. De retour à l'étape 2, l'instruction qui était en attente et qui, pour rappel, est l'instruction suivante :

    ```
    return 4 * factorial(3)
    ```

    se transforme en :

    `return 4 * 6 (car factorial(3)` a retourné 6)

    Puis l'expression `(4*6)` est évaluée et la valeur résultante de 24 est retournée à la fonction appelante située au niveau de l'argument de la fonction `print` du programme principal. Cette fonction `print` devrait donc afficher la valeur `24`.

Comme attendu, l'exécution de ce programme donne le résultat suivant :

```
4!=24
```

Lors du développement de fonctions récursives, il faut faire très attention aux critères d'arrêt des appels récursifs. En effet, il faut bien s'assurer que la ou les conditions d'arrêt sont correctement positionnées dans le corps d'une fonction

récursive, au risque de saturer la mémoire de l'ordinateur à cause des appels récursifs infinis ! Dans notre exemple, c'est la condition `if n<2` conjuguée avec les appels récursifs `factorial(n-1)` où nous avons décrémenté la valeur de l'argument `n` qui nous garantissent que nos appels récursifs prennent fin à un moment donné.

Bien sûr, dans notre exemple, si vous tentez d'exécuter la fonction `factorial` avec un nombre inapproprié, comme par exemple un nombre négatif, le programme retourne une valeur erronée. Pour que ce programme fonctionne correctement, quelle que soit la valeur transmise à la fonction `factorial`, il est nécessaire d'ajouter des contrôles sur la valeur du paramètre `n`. Nous avons omis ces contrôles dans l'exemple pour des raisons de clarté afin de nous focaliser sur l'aspect récursif de cette fonction `factorial`.

4.7 Les listes en compréhension

La notion de listes en compréhension (*List Comprehension* en anglais) est une syntaxe qui permet de définir des listes de manière compacte et élégante.

4.7.1 Les origines des listes en compréhension

En réalité, la syntaxe d'une *List Comprehension* est inspirée de celle utilisée en mathématiques pour définir des ensembles. Par exemple, en mathématiques, nous pouvons définir un ensemble comme suit $A = \{\mathcal{F}(x); x \in B\}$.

Avec cette notation, on définit un ensemble A dont les éléments sont le résultat de l'application de la fonction $\mathcal{F}$ sur les éléments de l'ensemble B. En d'autres termes, l'ensemble A est défini à partir de l'ensemble B.

Cette idée consiste à définir une structure de données à partir d'une autre structure qui est utilisée en Python avec les listes en compréhension. En revanche, contrairement aux mathématiques où les éléments d'un ensemble ne sont ni ordonnés ni redondants, les listes en compréhension définissent de simples listes Python dont les éléments sont ordonnés et peuvent être redondants !

© Editions ENI - All rights reserved

4.7.2 Construire une liste avec les listes en compréhension

Concrètement, avec les listes en compréhension, nous pouvons définir des listes à partir d'autres objets tels que les listes comme dans l'exemple suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *list_comprehension-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list_1 = [1,2,3,4,5,6,7,8,9,10]
print(f"list_1={list_1}")

list_2 =[]
for x in list_1:
   list_2.append(x*2)

print(f"list_2={list_2}")
```

Ce code définit la liste `list_1` avec les valeurs de 1 à 10. Puis la liste `list_2` a été définie avec une boucle `for` et affichée avec la fonction `print`.

Avec la boucle `for`, nous avons tout simplement parcouru un par un les éléments de la liste `list_1` et à chaque fois nous avons ajouté le double de ces éléments dans la liste `list_2`. L'exécution de ce programme affiche le résultat suivant :

```
list_1=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
list_2=[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Avec la syntaxe des listes en compréhension, nous pouvons réécrire le programme précédent comme suit :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *list_comprehension-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list_1 = [1,2,3,4,5,6,7,8,9,10]
print(f"list_1={list_1}")

list_2 = [x*2 for x in list_1]

print(f"list_2={list_2}")
```

Les deux premières lignes ainsi que la dernière ligne de ce nouveau programme sont identiques aux deux premières lignes et la dernière ligne du programme précédent ; en revanche, cette fois, en utilisant une liste en compréhension, nous avons remplacé les trois lignes du code du programme précédent où nous avons défini la liste `list_2` avec une seule ligne qui est :

```
list_2 = [x*2 for x in list_1]
```

Cette écriture veut tout simplement dire que `list_2` est composée d'une liste de valeurs dont chacune est le double d'une des valeurs de la liste `list_1`. L'exécution de ce nouveau code donne un résultat identique aux résultats du programme précédent.

4.7.3 Application de fonction avec une liste en compréhension

Dans l'exemple précédent, nous avons multiplié par deux les valeurs de `list_1` afin de calculer les valeurs de `list_2`. En pratique, nous pouvons appliquer n'importe quelle fonction sur les valeurs de `list_1` ! Examinons le code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *list_comprehension-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list_1 = [1,2,3,4,5,6,7,8,9,10]
print(f"list_1={list_1}")


def Fibonacci(n):
    if (n == 0):
        return 0
    elif n==1:
        return 1
    else:
        return Fibonacci(n-1)+Fibonacci(n-2)

list_2 = [Fibonacci(x) for x in list_1]
print(f"list_2={list_2}")
```

© Editions ENI - All rights reserved

Dans ce code, nous avons défini la liste `list_1` comme dans les exemples précédents, puis nous avons défini la fonction `Fibonacci`. Cette fonction calcule la somme des termes entre l'indice 0 et l'indice n de la fameuse suite de Fibonacci. Vous remarquez que cette fonction est récursive !

L'avant-dernière instruction construit la nouvelle liste `list_2` qui est une liste en compréhension en appliquant la fonction `Fibonacci` sur les éléments de la liste `list_1`.

Le résultat de ce nouveau code est le suivant :

```
list_1=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
list_2=[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

Ce dernier code a montré comment, grâce aux listes en compréhension, nous pouvons appliquer une fonction quelconque pour définir facilement en une seule ligne de code de nouvelles listes de façon intuitive !

4.7.4 Utiliser if-else avec les listes en compréhension

Lors de la définition d'une liste en compréhension, nous pouvons également utiliser la structure conditionnelle `if-else` afin de réaliser un traitement différent en fonction de certaines conditions. L'exemple suivant montre comment calculer le double ou le triple de chacun des éléments de la liste `list_1` selon qu'un élément soit pair ou impair.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *list_comprehension-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list_1 = [1,2,3,4,5,6,7,8,9,10]
print(f"list_1={list_1}")

list_2 = [x*2 if (x%2)==0 else x*3 for x in list_1]
print(f"list_2={list_2}")
```

Ce code définit la liste `list_2` en utilisant une liste en compréhension. La structure conditionnelle `if-else` de cette liste en compréhension permet de contrôler le caractère pair ou impair sur les éléments de la liste `list_1`. Ainsi, si un élément de la liste `list_1` est pair, alors il sera multiplié par 2 et ajouté à la liste `list_2` ; sinon, si cet élément est impair, alors il sera multiplié par 3 et il sera également ajouté à la liste `list_2`.

Le résultat de ce programme est le suivant :

```
list_1=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
list_2=[3, 4, 9, 8, 15, 12, 21, 16, 27, 20]
```

4.7.5 Filtrer avec les listes en compréhension

Si nous voulions utiliser une condition pour filtrer les valeurs de la liste parcourue dans une liste en compréhension, nous pouvons utiliser la structure conditionnelle `if` comme dans l'exemple suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *list_comprehension-5.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
list_1 = [1,2,3,4,5,6,7,8,9,10]
print(f"list_1={list_1}")

list_2 = [x for x in list_1 if (x%2)==0]
print(f"list_2={list_2}")
```

Ce programme permet tout simplement de copier les nombres pairs de la liste `list_1` dans la liste `list_2` en utilisant une liste en compréhension. La condition `if` de ce programme permet de ne considérer que les valeurs de `x` qui sont paires. Le résultat de ce programme est le suivant :

```
list_1=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
list_2=[2, 4, 6, 8, 10]
```

On voit que la liste `list_2` ne contient que les nombres pairs de la liste `list_1` !

4.8 Les expressions régulières

Les expressions régulières, appelées aussi regex, sont un moyen très puissant pour la recherche de motifs dans des chaînes de caractères. Elles sont utiles dans bien des situations, notamment lorsque nous développons des modèles de traitement du langage naturel où la matière première est le texte !

Le module `re` fourni nativement avec l'installation de Python permet justement de manipuler les regex.

© Editions ENI - All rights reserved

Dans cette section, nous allons découvrir les regex à travers quelques exemples concrets.

Une regex est une suite de caractères qui désigne un motif ou un modèle de chaînes de caractères, c'est-à-dire qu'une regex correspond en réalité à un ensemble de chaînes de caractères ! On dit également qu'une regex valide un ensemble de chaînes de caractères. L'ensemble de chaînes de caractères que valide une regex peut potentiellement être infini. L'objectif de l'utilisation d'une regex est de rechercher l'existence d'un motif dans une chaîne de caractères. Voyons tout de suite comment exprimer des regex avec Python.

Une regex s'exprime elle-même sous forme d'une chaîne de caractères ! La chaîne de caractères d'une regex peut contenir les deux types de caractères suivants :

1. Les caractères normaux : par exemple, les lettres de l'alphabet et les chiffres sont des caractères normaux.
2. Les métacaractères ou les caractères spéciaux : par exemple, les caractères {}, ^, $, *, +, (), [] et ? sont tous des caractères spéciaux.

Tandis que les caractères normaux permettent d'indiquer les caractères faisant partie d'une regex, les caractères spéciaux permettent de construire la forme de la regex elle-même.

Voici quelques regex intéressantes et pratiques (si vous avez des difficultés à comprendre ces exemples, ne vous inquiétez pas, les choses seront beaucoup plus claires lorsque nous réaliserons des exemples de code juste après) :

- `regex="exemple"`: cette regex permet tout simplement de rechercher le mot `"exemple"` dans une chaîne de caractères. Cette regex ne contient que des caractères normaux ! Par exemple, soit la phrase suivante :

  "Le problème du voyageur de commerce est un **exemple** de problèmes NP-Difficiles, mais aussi, c'est un **exemple** de problèmes NP-Complets."

  L'objectif de l'application de notre regex `regex= "exemple"` sur cette phrase est de rechercher s'il existe des occurrences du mot `"exemple"` dans cette phrase. Ici, cette application va détecter la présence des deux occurrences du mot "exemple" ainsi que leur emplacement dans la phrase !

- `"regex = "^abc"` : cette regex permet de vérifier si une chaîne de caractères donnée commence par les trois lettres `abc`. Dans cet exemple, les lettres 'a', 'b' et 'c' sont des caractères normaux alors que le caractère '^' est un caractère spécial qui indique notre souhait de rechercher la suite de caractères 'abc' "abc" située en début de chaîne de caractères. Par exemple, voyons ce qui se passe si nous appliquons cette regex sur la phrase suivante :

  "**abc** sont les trois premières lettres de l'alphabet français."

  Cette phrase commence bien par les trois lettres "abc". Donc l'application de notre regex= "^abc" sur cette phrase va détecter et retourner la sous-chaîne de caractères "abc" en nous indiquant qu'elle a bien été trouvée en début de cette phrase. En revanche, dans la phrase suivante :

  "Les trois lettres abc font partie de l'alphabet français."

  Cette fois, l'application de notre regex `regex="^abc"` ne va rien détecter malgré le fait que la sous-chaîne de caractères "abc" fasse partie de cette phrase, car l'utilisation du caractère spécial "^" indique que le motif "abc" doit être présent au début de la chaîne de caractères.

- `regex="abc$"` : cette regex permet de vérifier si une chaîne de caractères se termine par les trois lettres "abc". Dans cet exemple, c'est le caractère `"$"`, qui est un caractère spécial, qui indique notre souhait de rechercher la suite de caractères "abc" situés en fin de chaîne de caractères.

- `regex="ab\.cd"` : cette regex permet de rechercher des sous-chaînes de caractères qui commencent par les deux lettres `"ab"`, suivies d'un point et qui se terminent par les deux lettres `"cd"`. Ici, le caractère backslash '\' est utilisé afin d'indiquer que le caractère `"."` doit être considéré en tant que caractère simple et non pas un caractère spécial, comme dans l'exemple qui suit.

- `regex="ab.cd"` : cette regex permet de rechercher des sous-chaînes de caractères qui commencent par les deux lettres `"ab"`, suivies de n'importe quel autre caractère et qui se terminent par les deux lettres 'cd'. Ici, le caractère `"."` est un caractère spécial qui désigne la possibilité de le remplacer par n'importe quel autre caractère ! Par exemple, les mots `abxcd`, `abXcd` ou `ab4cd` ou encore `ab%cd` sont tous des mots valides à l'égard de cette regex.

© Editions ENI - All rights reserved

- `regex="[AB]"` : cette regex permet de rechercher des sous-chaînes de caractères qui contiennent soit la lettre capitale 'A', soit la lettre capitale 'B'. Dans cette regex, les deux caractères '[' et ']' sont des caractères spéciaux et les motifs qui sont indiqués entre ces deux crochets sont exclusifs, ici soit 'A', soit 'B'.
- `regex="[A-Z]"` : cette regex permet de rechercher des sous-chaînes de caractères qui contiennent une lettre de l'alphabet en capitale. Le caractère '-' permet de désigner un intervalle de valeurs, ici toutes les lettres de A à Z.
- `regex="[0-9]"` : cette regex permet de rechercher des sous-chaînes de caractères qui contiennent un chiffre entre 0 et 9.
- `regex="[A-Za-z0-9]"` : cette regex permet de rechercher des sous-chaînes de caractères qui contiennent un caractère alphanumérique.
- `regex="abc*"` : cette regex permet de rechercher des sous-chaînes de caractères qui commencent par les deux lettres 'ab' et qui se terminent par zéro ou plus d'occurrences de la lettre 'c'.
- `regex="abc+"` : cette regex permet de rechercher des sous-chaînes de caractères qui commencent par les deux lettres 'ab' et qui se terminent par une ou plusieurs occurrences de la lettre 'c'.
- `regex="(abc)+"` : cette regex permet de rechercher des sous-chaînes de caractères qui contiennent le mot 'abc' une ou plusieurs fois. La différence avec la regex précédente est l'utilisation des parenthèses qui sont des caractères spéciaux et qui permettent de regrouper des motifs.
- `regex="(abc){2,5}"` : cette regex permet de rechercher des sous-chaînes de caractères qui contiennent un nombre d'occurrences entre 2 et 5 du mot `"abc"`.

Pour mieux comprendre les regex et leur utilisation, nous allons réaliser les quelques exemples de code en utilisant le module `re`.

4.8.1 Regex sans caractères spéciaux

Dans ce premier exemple, nous allons rechercher des sous-chaînes de caractères correspondant à une regex sans caractères spéciaux.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *regex-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import re

txt = "Le problème du voyageur de commerce est un exemple de
problèmes NP-Difficiles, mais aussi, .c'est un exemple de
problèmes NP-Complets"

regex ="exemple"
res = re.search(regex, txt)

if res :
   print(f"La première occurrence du motif '{regex}' a été
détectée entre l'indice {res.start()} et l'indice {res.end()-1}")
else :
   print(f"Le motif '{regex}' est absent dans '{txt}'")
```

Ce code commence par l'import de la bibliothèque `re` qui est un module fourni avec Python dédié à l'exploitation des expressions régulières. La variable `txt` contient l'exemple de chaîne de caractères que nous avons évoquée plus haut et sur laquelle nous allons appliquer une expression régulière. Dans cette chaîne de caractères, le mot `exemple` apparaît deux fois.

La variable `regex` est une simple chaîne de caractères à laquelle nous avons affecté la valeur `"exemple"`.

Dans la variable `res`, nous avons enregistré le résultat de l'exécution de la fonction `search` du module `re`. Cette fonction prend les deux arguments `regex` et txt. La valeur du premier argument sera utilisée comme étant une expression régulière, alors que la valeur du deuxième argument sera utilisée comme étant la chaîne de caractères dans laquelle la recherche sera effectuée. En d'autres termes, la fonction `re.search` va appliquer la `regex` regex sur la chaîne de caractères `txt`.

L'application de l'expression régulière `regex` sur `txt` se réalise en parcourant les caractères de `txt` un par un en partant de la gauche vers la droite à la recherche des occurrences correspondant au motif décrit par `regex`.

La suite du code est une simple structure conditionnelle qui affiche un message selon qu'un résultat ait été trouvé ou pas.

© Editions ENI - All rights reserved

Dans la mesure où un résultat a été trouvé, c'est-à-dire que le test `if res` réussit, un message est affiché indiquant qu'effectivement un résultat a été trouvé et la position de ce résultat dans la chaîne `txt`. Cette position correspond aux indices dans `txt` des caractères constituant le résultat. Ainsi, `res.start()` retourne l'indice du premier caractère de la sous-chaîne de caractères trouvée et `res.end()` retourne l'indice du caractère situé immédiatement après le dernier indice du résultat ; c'est pour cette raison que dans le code nous avons affiché la valeur de `res.end()-1`.

L'exécution de ce programme affiche le résultat suivant :

```
La première occurrence du motif 'exemple' a été détectée entre
l'indice 43 et l'indice 49
```

Remarquez que ce programme n'a trouvé que la première occurrence du mot "exemple". En effet, la fonction `re.search()` ne s'intéresse qu'à la première occurrence correspondant à la regex utilisée. Pour rechercher toutes les occurrences correspondant à une regex, il faut utiliser la fonction `re.findall()` ou encore la fonction `re.finditer()` que nous allons découvrir dans la suite de cette section.

4.8.2 Regex avec caractères spéciaux

L'expression régulière utilisée dans l'exemple précédent ne contenait que des caractères normaux. L'exemple qui suit montre quatre autres exemples de regex où des caractères spéciaux sont utilisés.

▶ Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *regex-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import re

def print_res(res, regex, source):
    if res :
        print(f"{regex} - '{source}' : True")
    else :
        print(f"{regex} - '{source}' : False")

txt_1 = "pi = 3.14"
txt_2 = "Le nombre pi=3.14 est Irrationnel."
```

```
regex_1 ="^pi"
a1 = re.search(regex_1, txt_1)
a2 = re.search(regex_1, txt_2)
print_res(a1, regex_1, txt_1)
print_res(a2, regex_1, txt_2)

regex_2 = "nnel\.$"
b1 = re.search(regex_2, txt_1)
b2 = re.search(regex_2, txt_2)
print_res(b1, regex_2, txt_1)
print_res(b2, regex_2, txt_2)

regex_3 = "[A-Z]"
c1 = re.search(regex_3, txt_1)
c2 = re.search(regex_3, txt_2)
print_res(c1, regex_3, txt_1)
print_res(c2, regex_3, txt_2)

regex_4 = "[0-9]"
d1 = re.search(regex_4, txt_1)
d2 = re.search(regex_4, txt_2)
print_res(d1, regex_4, txt_1)
print_res(d2, regex_4, txt_2)
```

Comme dans l'exemple précédent, ce code commence par l'import du module `re`. La fonction `print_res` sert uniquement pour l'affichage des résultats. Le paramètre `res` correspond au résultat de l'application de l'expression régulière `regex` sur la chaîne de caractères `source`. Cette fonction affiche `True` lorsque le résultat `res` contient des éléments, sinon elle affiche `False` lorsqu'aucun résultat n'a été trouvé.

Les deux variables `txt_1` et `txt_2` sont deux exemples de chaînes de caractères. Le reste de ce programme est composé de quatre blocs distincts. Dans chaque bloc, une expression régulière différente est appliquée sur les deux chaînes de caractères `txt_1` et `txt_2` et les résultats sont affichés avec la fonction `print_res`. Les regex de ces quatre blocs sont les suivantes :

- `regex_1 ="^pi"` pour rechercher une sous-chaîne de caractères commençant par `"pi"`.
- `regex_2 = "nnel\.$"` pour rechercher une sous-chaîne de caractères se terminant par `"nnel."`.

© Editions ENI - All rights reserved

- `regex_3 = "[A-Z]"` pour rechercher une sous-chaîne de caractères contenant une lettre en capitale de A à Z.
- `regex_4 = "[0-9]"` pour rechercher une sous-chaîne de caractères contenant un chiffre de 0 à 9.

Chacune de ces quatre expressions régulières est appliquée sur `txt_1` puis sur `txt_2` et le résultat est affiché à l'aide de la fonction `print_res`.

L'exécution de ce code affiche le résultat suivant :

```
^pi - 'pi = 3.14' : True
^pi - 'Le nombre pi=3.14 est Irrationnel.' : False
nnel\.$ - 'pi = 3.14' : False
nnel\.$ - 'Le nombre pi=3.14 est Irrationnel.' : True
[A-Z] - 'pi = 3.14' : False
[A-Z] - 'Le nombre pi=3.14 est Irrationnel.' : True
[0-9] - 'pi = 3.14' : True
[0-9] - 'Le nombre pi=3.14 est Irrationnel.' : True
```

On voit que :

- `txt_1` commence bien par `pi`, mais pas `txt_2`.
- `txt_1` ne se termine pas par `nnel.`, contrairement à `txt_2`.
- `txt_1` ne contient aucune lettre capitale, contrairement à `txt_2`, puisqu'elle contient la lettre 'I' au début du mot `Irrationnel`.
- Les deux chaînes de caractères `txt_1` et `txt_2` contiennent un chiffre.

4.8.3 Regex avec les multiplicateurs

La répétition d'un motif est un concept couramment utilisé avec les regex. Il s'agit de rechercher si un motif donné se répète plusieurs fois de façon successive dans une chaîne de caractères. Pour exprimer la répétition d'un motif, on peut utiliser les trois caractères '?', '+' et '*'. Le caractère spécial '?' exprime une répétition de zéro ou une seule fois d'un motif. Le caractère spécial '+' exprime une répétition d'au moins une seule fois d'un motif, alors que le caractère spécial '*' exprime une répétition de zéro ou plusieurs fois d'un motif. Le code du programme suivant montre des exemples où ces trois caractères spéciaux sont utilisés.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *regex-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import re


txt = "YabbbYaaYbbY"
print(f"txt={txt}")

regex_1 = re.compile("a*b+")
res_1= regex_1.findall(txt)
print(f"res_1={res_1}")

regex_2 = re.compile("Ya?b")
res_2= regex_2.findall(txt)
print(f"res_2={res_2}")
```

Après avoir importé le module `re`, ce code définit et affiche la chaîne de caractères `txt="YabbbYaaYbbY"`.

La variable `regex_1` est associée à l'expression régulière `"a*b+"` via la fonction `re.compile`. Cette fonction `re.compile` est très utile et recommandée lorsqu'une expression régulière est utilisée un grand nombre de fois dans un même programme ; ce n'est pas le cas dans notre exemple, mais le but est de montrer comment l'utiliser.

Cette expression régulière `"a*b+"` admet tous les mots qui commencent par zéro ou plus d'occurrences de la lettre 'a' suivies d'au moins une occurrence de la lettre 'b'.

La variable `res_1` stocke le résultat de la méthode `regex_1.findall` qui est appliquée sur la chaîne de caractères `txt`. Cette méthode, contrairement à la méthode `re.search` utilisée dans les exemples précédents, permet de rechercher toutes les occurrences correspondant à l'expression régulière utilisée. Pour rappel, la méthode `re.search` ne recherche que la première occurrence correspondant à une expression régulière.

La méthode `findall` peut être utilisée directement avec le module `re`, c'est-à-dire que nous pouvons écrire `re.findall(...)` tout comme nous l'avons fait avec la méthode `re.search(...)` précédemment. Cependant, comme dans cet exemple, nous avons compilé une expression régulière avec la

© Editions ENI - All rights reserved

méthode `compile()` dans l'objet `regex_1`, cela nous donne la possibilité d'utiliser directement cet objet `regex_1` afin d'évoquer les méthodes telles que `findall` ou `search` et bien d'autres méthodes.

Comme la méthode `findall` recherche toutes les occurrences correspondant à l'expression régulière utilisée, le résultat de son exécution est une liste contenant toutes les sous-chaînes de caractères trouvées. De ce fait, dans notre code, la variable `res_1` sera une liste.

La variable `regex_2` est associée à l'expression régulière "`Ya?b`" via la fonction `re.compile`. Cette expression régulière admet tous les mots qui commencent par la lettre 'Y' en capitale suivie optionnellement par la lettre 'a', puisque devant la lettre 'a' nous avons utilisé le caractère spécial '?', et qui se terminent obligatoirement par une occurrence de la lettre 'b'.

La variable `res_2` stocke le résultat de la méthode `regex_2.findall` qui est appliquée sur la chaîne de caractères `txt`.

L'exécution de ce code donne le résultat suivant :

```
txt=YabbbYaaYbbY
res_1=['abbb', 'bb']
res_2=['Yab', 'Yb']
```

On voit qu'il n'y a que deux occurrences qui sont extraites de `txt` et qui valident l'expression régulière `regex_1="a*b+"`. La première occurrence 'abbb' est extraite, car elle commence par une lettre 'a' suivie de trois lettres 'b', ce qui correspond à notre `regex_1`. La deuxième occurrence 'bb' est également extraite, car elle est valide même si elle ne commence pas par des lettres 'a', puisque dans le motif '`a*b+`' la lettre 'a' est suivie du caractère spécial '*' qui indique zéro ou plus d'occurrences de la lettre 'a'. Remarquez que la sous-chaîne de caractères '`aa`' de `txt` n'a pas été extraite, car dans le motif '`a*b+`', la lettre 'b' est suivie du caractère spécial '+' et non pas *, ce qui indique que la lettre 'b' doit absolument avoir au moins une occurrence, ce qui n'est pas le cas dans la sous-chaîne de caractères '`aa`' qui est suivie de la lettre 'Y' dans `txt` et non pas d'une lettre 'b'.

On voit également que `res_2` contient deux occurrences correspondant à l'expression régulière `regex_2="Ya?b"`. Ces deux occurrences sont les deux seules sous-chaînes de caractères de `txt` qui commencent par 'Y' et qui se terminent par 'b' et qui contiennent zéro ou une occurrence de la lettre 'a'.

Comme nous allons le voir plus loin dans cette section, le caractère spécial '?' a d'autres usages, outre le fait de désigner un motif optionnel.

4.8.4 Regex avec un nombre d'occurrences limité

Dans l'exemple précédent, nous avons utilisé les caractères spéciaux + et * afin d'exprimer des expressions régulières avec répétition de motifs. Cependant, avec le caractère spécial +, le nombre d'occurrences d'un motif peut aller de 1 à l'infini, et avec le caractère *, le nombre d'occurrences d'un motif peut aller de 0 à l'infini ; nous ne pouvons donc pas fixer le nombre d'occurrences d'un motif à une quantité précise, autre que 0 ou 1.

L'exemple de code suivant montre comment contrôler le nombre d'occurrences d'un motif en fixant le nombre minimal et maximal d'occurrences souhaitées :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *regex-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import re


txt = "abaabbaaabbb"
print(f"txt={txt}")

regex_1 = re.compile("a{3}b{3}")
res_1= regex_1.findall(txt)
print(f"res_1={res_1}")


regex_2 = re.compile("a{2,3}b{2,3}")
res_2= regex_2.findall(txt)
print(f"res_2={res_2}")
```

Ce code définit et affiche la chaîne de caractères `txt="abaabbaaabbb"`, puis utilise deux blocs de code pour tester les deux expressions régulières suivantes sur `txt` :

- `a{3}b{3}` : cette expression régulière correspond aux mots qui commencent avec exactement trois lettres 'a' et qui se terminent par exactement trois lettres 'b'.

© Editions ENI - All rights reserved

- `a{2,3}b{2,3}` : cette expression régulière correspond aux mots qui commencent par au moins deux occurrences de la lettre 'a' et au plus trois occurrences de la lettre 'a' et qui se terminent par au moins deux occurrences de la lettre 'b' et au plus trois occurrences de la lettre 'b'.

Chacune de ces deux expressions régulières a été appliquée sur la chaîne de caractères `txt`. Le résultat est le suivant :

```
txt=abaabbaaabbb
res_1=['aaabbb']
res_2=['aabb', 'aaabbb']
```

On voit que dans `txt`, il n'y a qu'une seule sous-chaîne de caractères qui commence par exactement trois lettres 'a' et qui se termine par exactement trois lettres 'b'. En revanche, la méthode `findall` a trouvé deux occurrences pour l'expression régulière `'a{2,3}b{2,3}'`.

4.8.5 Regex avec groupage des résultats

Souvent, lorsque nous recherchons des motifs avec des expressions régulières, nous avons besoin d'identifier des parties spécifiques au sein des résultats trouvés. Pour cela, nous devons spécifier des zones distinctes au sein même de l'expression régulière, comme le montre le nouvel exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *regex-5.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import re

txt = "x=48.85 et y=2.35"

regex = re.compile("([0-9]+)\.([0-9]+)")
res = regex.findall(txt)

print(txt)
for n1,n2 in res:
    print(f"n1={n1}; n2={n2}")
```

Dans cet exemple, nous souhaitons capturer la partie entière et la partie décimale des nombres réels présents dans la chaîne de caractères `txt`.

L'expression régulière `"([0-9]+)\.([0-9]+)"` correspond à toutes les chaînes de caractères qui commencent par une succession de chiffres entre 0 et 9, qui représentent la partie entière d'un nombre, suivis par le caractère '.' et qui se terminent par une succession de chiffres entre 0 et 9 qui représentent la partie décimale d'un nombre. Dans cette expression régulière, des parenthèses '()' sont utilisées afin de regrouper la première suite de chiffres entre 0 et 9 et également pour regrouper la deuxième suite de chiffres entre 0 et 9. Ces deux regroupements vont nous permettre d'accéder séparément à la partie entière et la partie décimale des nombres trouvés dans les résultats retournés par la méthode `findall`.

Dans cet exemple, la méthode `findall` va nous retourner une liste dont chaque élément est un couple ! Le premier élément de chacun de ces couples va correspondre au premier regroupement à gauche avec les parenthèses dans l'expression régulière `([0-9]+)\.([0-9]+)` et le deuxième élément de chacun de ces couples va correspondre au deuxième regroupement à droite. Justement, la boucle à la fin de ce code permet d'accéder aux éléments de ces couples pour ainsi afficher la partie entière séparément de la partie décimale des nombres qui seront trouvés. Le résultat de ce code est le suivant :

```
x=48.85 et y=2.35
n1=48; n2=85
n1=2; n2=35
```

Cet exemple a montré comment identifier distinctement des parties dans les résultats retournés après une recherche avec une expression régulière en utilisant des regroupements avec des parenthèses.

4.8.6 Taille des motifs

Par défaut, lorsque nous recherchons des motifs avec des expressions régulières, la taille des sous-chaînes de caractères extraites est maximale. En effet, l'algorithme d'extraction des sous-chaînes de caractères cherche à étendre le plus possible une expression régulière. Cette notion d'extension des expressions régulières est très importante à comprendre, car elle peut être source d'ambiguïtés et d'erreurs !

© Editions ENI - All rights reserved

L'exemple de code suivant illustre bien le mécanisme d'extension des expressions régulières :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *regex-6.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import re

def apply_regex(regex, txt):
    print(f"\nApplication de la regex \"{regex}\" sur \"{txt}\"")
    regex = re.compile(regex)
    res = regex.finditer(txt)
    for e in res:
        print(f"e.group() = {e.group()}, e.start()={e.start()},
              e.end()={e.end()}")


txt = "abcabcabcabcabc"
regex_1 = "(abc){2,5}"
apply_regex(regex_1, txt)
regex_2 = "(abc){2,5}?"
apply_regex(regex_2, txt)
```

Dans ce nouvel exemple, la fonction `apply_regex` permet tout simplement d'appliquer l'expression régulière `regex` reçue en paramètre sur la chaîne de caractères `txt` qui est également reçue en paramètre. Dans cette fonction, l'application de l'expression régulière sur `txt` est réalisée avec la méthode `finditer` et non pas avec les méthodes `findall` ou `search` vues précédemment. Nous avons vu que la différence entre `findall` et `search` réside dans le fait que `findall` cherche toutes les occurrences valides vis-à-vis de l'expression régulière utilisée, alors que la méthode `search` se contente de retourner uniquement la première occurrence. Le problème avec la méthode `search` est qu'elle retourne la liste des sous-chaînes de caractères extraites de la chaîne de caractères sur laquelle l'expression régulière est appliquée. De ce fait, nous ne pouvons pas utiliser les méthodes `start()` et `end()` pour accéder aux informations de localisation des sous-chaînes extraites ! Si nous avons besoin de ces informations, il est intéressant d'utiliser la méthode `finditer` plutôt que la méthode `findall` comme dans notre fonction `apply_regex`.

La méthode `finditer` retourne une liste d'objets de type `re.Match`. Les objets de ce type nous permettent de localiser les positions des sous-chaînes de caractères trouvées.

Ce code définit la chaîne de caractères `txt="abcabcabcabcabc"`, qui est une succession de cinq occurrences du mot "abc", puis ce programme appelle la fonction `apply_regex` pour appliquer sur `txt` les deux expressions régulières suivantes :

- `(abc){2,5}` : cette expression régulière correspond à tous les mots composés de deux à cinq successions du mot "abc".
- `(abc){2,5}?` : comme la regex précédente, cette expression régulière correspond à tous les mots composés de deux jusqu'à cinq successions du mot "abc", sauf que cette fois nous avons ajouté le caractère spécial '?' à la fin de la définition de cette regex.

Voyons concrètement l'effet du caractère '?' sur les résultats. L'exécution de ce programme affiche les résultats suivants :

```
Application de la regex "(abc){2,5}" sur "abcabcabcabcabc"
e.group() = abcabcabcabcabc, e.start()=0, e.end()=15

Application de la regex "(abc){2,5}?" sur "abcabcabcabcabc"
e.group() = abcabc, e.start()=0, e.end()=6
e.group() = abcabc, e.start()=6, e.end()=12
```

On voit que l'application de l'expression régulière "`(abc){2,5}`" sur txt a retourné tout le contenu de `txt` ! En effet, lorsque l'algorithme de recherche applique cette expression régulière sur `txt`, il cherche le mot le plus long qui valide cette expression régulière. De ce fait, lorsque l'algorithme a rencontré les deux premières occurrences de "abc", c'est-à-dire lorsque l'algorithme a rencontré la sous-chaîne de caractères "abcabc", même si cette sous-chaîne de caractères valide l'expression régulière "`(abc){2,5}`", cet algorithme ne s'arrête pas, mais il continue plutôt la recherche afin de voir s'il ne peut pas trouver une sous-chaîne de caractères plus longue et qui valide toujours cette regex. Dans notre exemple, les cinq occurrences du mot "abc" valident notre expression régulière et l'algorithme retourne donc toute la chaîne `txt`.

Dans le deuxième résultat, on voit que l'algorithme a trouvé deux occurrences qui valident la regex "`(abc){2,5}?`". Grâce à l'utilisation du caractère spécial

© Editions ENI - All rights reserved

'?', nous pouvons dire à l'algorithme de considérer les mots de taille la plus petite possible ; c'est-à-dire qu'il extrait un mot dès que celui-ci valide la regex. Dans notre exemple, on voit que l'algorithme a considéré le mot constitué des deux premières occurrences de "abc" situées entre l'indice 0 et l'indice 5 de `txt`, puis il a considéré un deuxième mot constitué des troisième et quatrième occurrences situées entre l'indice 6 et l'indice 11 de `txt`. En revanche, la dernière occurrence '"abc" située en fin de `txt` n'a pas été utilisée puisque, si elle est considérée toute seule, elle ne valide pas la regex `(abc){2,5}`.

Dans ce dernier exemple, nous aurions pu tout simplement utiliser "`(abc){2}`" à la place de la regex "`(abc){2,5}?`" et obtenir les mêmes résultats, mais le but était bien d'attirer votre attention sur le mécanisme d'extension des expressions régulières.

Maintenant que nous avons étudié l'extension des expressions régulières à travers un exemple pédagogique, considérons un exemple plus concret où l'utilisation du caractère '?' pour limiter la taille des sous-chaînes de caractères extraites est cruciale.

Dans ce nouvel exemple, nous allons tenter d'extraire toutes les sous-chaînes de caractères entourées par des crochets. Pour cela, nous allons utiliser le code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *regex-7.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import re

txt = "abc...[AAA], [1980],[Descartes] et [Covid-2 : 2020]...xyz"
print(f"txt={txt}")

regex = re.compile("\[.*\]")
res = regex.findall(txt)
print(f"res={res}")
```

Dans ce programme, nous avons utilisé l'expression régulière "`\[.*\]`" pour l'extraction des sous-chaînes de caractères de `txt` qui sont entourées avec des crochets `[]`. Dans cette expression régulière, les deux crochets `[]` sont utilisés en tant que caractères simples, puisqu'ils ont été précédés par le caractère d'échappement '\'.

Naïvement, nous pouvons penser que cette expression régulière "\[.*\]" est suffisante pour la détection de toutes les phrases de txt entourées par des crochets, c'est-à-dire les quatre chaînes de caractères "AAA", "1980", "Descartes" et "Covid-2 : 2020". Voyons comment ce programme va se comporter. L'exécution de ce programme affiche le résultat suivant :

```
txt=abc...[AAA], [1980],[Descartes] et [Covid-2 : 2020]...xyz
res=['[AAA], [1980],[Descartes] et [Covid-2 : 2020]']
```

On voit que la fonction findall n'a trouvé qu'une seule sous-chaîne de caractères correspondant à notre expression régulière au lieu d'en trouver quatre ! Ce programme a détecté que la sous-chaîne de caractères "[AAA], [1980],[Descartes] et [Covid-2 : 2020]" est l'unique sous-chaîne de caractères entourée par des crochets de txt alors qu'en réalité il y a bien les quatre sous-chaînes de caractères. Alors pourquoi ces quatre sous-chaînes de caractères n'ont pas été détectées ?

Ce résultat s'explique tout simplement par le fait que l'algorithme de recherche des sous-chaînes correspondant à l'expression régulière "\[.*\]" ne s'arrête pas au niveau du premier crochet fermant ']' rencontré après le mot "[AAA", et cela à cause du principe d'extension des expressions régulières évoqué précédemment ! En effet, le caractère ']' correspond également au motif '.' qui fait partie de notre expression régulière, l'algorithme va donc continuer à rechercher un mot plus long que [AAA], et cela jusqu'à ce qu'il arrive au dernier crochet fermant.

Pour détecter les quatre sous-chaînes de caractères entourées par des crochets de notre exemple, il suffit de remplacer dans ce dernier code l'expression régulière "\[.*\]" par l'expression régulière "\[.*?\]". Dans cette nouvelle expression régulière, nous avons utilisé le caractère spécial '?' afin d'indiquer à l'algorithme de rechercher des sous-chaînes de caractères de taille la plus petite possible. L'exécution de ce dernier code après le remplacement de l'expression régulière "\[.*\]" par l'expression régulière "\[.*?\]" affiche le résultat suivant :

```
txt=abc...[AAA], [1980],[Descartes] et [Covid-2 : 2020]...xyz
res=['[AAA]', '[1980]', '[Descartes]', '[Covid-2 : 2020]']
```

Cette fois, on voit bien que res est une liste de quatre éléments !

© Editions ENI - All rights reserved

4.8.7 Aller plus loin avec les expressions régulières

Nous pouvons exprimer des expressions régulières complexes en utilisant le caractère d'échappement combiné avec certaines lettres. Par exemple :

- L'expression régulière `"\w"` est un motif qui correspond à un caractère alphanumérique équivalent à l'expression régulière `"[a-zA-Z0-9\_]"`.
- L'expression régulière `"\W"` est un motif qui correspond à tout caractère différent des caractères alphanumériques.
- L'expression régulière `"\s"` est un motif qui correspond aux espaces présents dans un document.
- L'expression régulière `"\d"` est un motif qui correspond à un caractère numérique équivalent à l'expression régulière `"[0-9]"`.

Il existe plusieurs fonctions du module `re`, outre celles que nous avons découvertes dans cette section, qui sont d'une grande utilité lorsque nous manipulons des données textuelles. Nous n'allons citer que deux d'entre elles que nous pensons incontournables :

- La fonction `re.sub(pattern, repl, txt)` permet de rechercher et de remplacer toutes les occurrences de la regex `pattern` par `repl` dans la chaîne de caractères `txt`.
- La fonction `re.split(pattern, txt)` permet de fragmenter la chaîne de caractères `txt` en utilisant les occurrences de la regex `pattern` comme séparateur.

Nous pouvons dire bien plus de choses sur les expressions régulières, mais nous allons nous arrêter là, car couvrir complètement le sujet des expressions régulières nécessite un ouvrage dédié. Cependant, les notions que nous avons abordées dans cette section sont largement suffisantes pour exprimer une large gamme de motifs !

4.9 Gestion des exceptions

4.9.1 La levée des exceptions

Jusqu'à présent, dans les exemples de code que nous avons abordés dans ce chapitre, nous ne nous sommes pas préoccupés de la gestion des cas particuliers qui peuvent causer des dysfonctionnements de nos programmes. Cela a été fait volontairement afin de nous concentrer sur les concepts abordés dans les différentes sections.

Quel que soit le langage de programmation utilisé, lorsque certains événements particuliers ne sont pas gérés par un programme, ces événements peuvent provoquer l'arrêt brutal de ce programme. Pour donner un exemple, considérons le code du programme suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Exception-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
a=int(input("Veuillez saisir un nombre : "))
print(f"10/{a}={10/a}")

print("Je suis la dernière instruction de ce programme")
```

Ce programme transforme la saisie de l'utilisateur en un nombre entier, puis il affiche le résultat de la division de 10 sur ce nombre entier.

Si vous exécutez ce programme en saisissant un nombre entier, alors ce programme affichera tout simplement le résultat de la division de 10 sur le nombre entier que vous auriez saisi. Cependant, lorsque nous développons ce type de programme, nous n'avons pas la certitude que l'utilisateur va réellement saisir un nombre entier ! À la place, il pourra par erreur saisir une chaîne de caractères qui contient des caractères autres que des digits. Dans ce dernier cas, notre programme ne pourra pas transformer cette chaîne de caractères en un nombre entier avec la fonction `int()` ! En effet, si vous exécutez ce programme en saisissant par exemple `"1a1"`, ce programme affiche le résultat suivant :

```
Veuillez saisir un nombre : 1a1
Traceback (most recent call last):
  File "Exception-1.py", line 1, in <module>
```

© Editions ENI - All rights reserved

```
    a=int(input("Veuillez saisir un nombre "))
ValueError: invalid literal for int() with base 10: 'la1'
```

Comme ce programme ne sait pas comment transformer `"la1"` en un nombre entier avec la fonction `int`, il nous le fait savoir via l'affichage de ce message d'erreur.

Avec cette dernière exécution, le problème est que non seulement l'instruction qui transforme la saisie de l'utilisateur en un nombre entier ne s'est pas correctement exécutée, mais en plus, toutes les instructions situées après cette instruction qui a échoué ne seront pas exécutées, car le programme s'est arrêté de manière brutale !

Ainsi, dans notre exemple, le dernier appel à la fonction `print` ne s'est pas exécuté, car il y a eu une erreur fatale qui a provoqué l'arrêt de tout le programme.

En programmation, on appelle ce type d'erreur les exceptions. Dans notre exemple, on dira qu'il y a eu une exception qui a provoqué l'arrêt du programme.

Pour éviter qu'un programme s'arrête complètement de manière brutale lorsqu'une erreur est survenue, il est nécessaire d'indiquer à ce programme quoi faire si une exception survient.

Plus encore, nous pouvons dire à notre programme comment se comporter en fonction du type d'exception qui a été rencontrée. En effet, chaque exception a un type. Dans notre exemple, le message d'erreur affiché indique que l'erreur qui a provoqué l'arrêt de notre programme est de type `ValueError`. De plus, dans ce message d'erreur, il est même indiqué que l'erreur a été provoquée par la valeur `"la1"` qui est un argument invalide pour la fonction `int()`.

▶ Réexécutez ce programme en saisissant cette fois la valeur 0 au clavier.

Cette fois, c'est le message d'erreur suivant qui est affiché :

```
Veuillez saisir un nombre : 0
Traceback (most recent call last):
  File "Exception-1.py", line 2, in <module>
    print(f"10/{a}={10/a}")
ZeroDivisionError: division by zero
```

Dans ce nouveau message d'erreur, on voit que l'exception qui a provoqué l'arrêt du programme a été causée par l'instruction `print(f"10/{a}={10/a}")` et non par la première instruction qui fait appel à la fonction `int()` comme lors du premier message d'erreur.

Également, dans ce nouveau message d'erreur, on voit que l'exception est de type `ZeroDivisionError` qui indique qu'une tentative de division par zéro a été détectée.

4.9.2 Utiliser le bloc try-except

Avec le langage Python, on utilise la structure `try-except` afin d'indiquer à un programme comment se comporter lorsqu'il rencontre des exceptions, comme le montre l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Exception-2.py* situé dans le dossier Code\Bases de ce chapitre.

```
try:
    a=int(input("Veuillez saisir un nombre :        "))
    print(f"10/{a}={10/a}")
except:
    print("Une exception a été capturée.")

print("Je suis la dernière instruction de ce programme")
```

Ce code est similaire au code précédent, mais cette fois nous avons entouré les deux premières instructions du code précédent avec la structure `try-except`. Le bloc de code contenu dans la clause `try` sera exécuté, et si une exception est rencontrée, alors l'exécution de ce bloc `try` sera interrompue pour continuer au niveau du bloc de code de la clause `except`. Si lors de l'exécution du bloc `try` aucune erreur n'est rencontrée, alors l'exécution du bloc `except` sera ignorée et le programme continuera avec les instructions situées après le bloc `except`.

Si vous exécutez ce programme en saisissant une valeur autre qu'un nombre entier, alors ce programme va tout simplement afficher le message `"Une exception a été capturée."`. En effet, grâce à la clause `except`, notre programme sait cette fois comment agir lorsqu'une exception survient lors de l'exécution des instructions du bloc `try`. Également, si vous saisissez la valeur

© Editions ENI - All rights reserved

0, ce programme affichera le même message d'erreur que lorsque vous avez saisi une valeur non entière.

L'exécution de ce programme avec la saisie de `"1a1"` affiche le résultat suivant :

```
Veuillez saisir un nombre : 1a1
Une exception a été capturée.
Je suis la dernière instruction de ce programme
```

Remarquez que cette fois la dernière instruction `print` de notre programme s'est exécutée bien qu'une exception ait été lancée. En effet, cette fois, même si une exception a été lancée pour indiquer qu'une valeur incorrecte a été saisie au clavier, le fait d'avoir prévu un bloc de code `except` a permis d'attraper cette exception et de permettre au programme de continuer son exécution !

4.9.3 Gérer plusieurs exceptions

Le problème avec le programme de notre dernier exemple, bien qu'il fonctionne quelle que soit la valeur saisie lors de son exécution, est qu'il ne distingue pas l'exception qui sera lancée si nous saisissons une valeur alphanumérique au lieu d'un nombre entier, d'une exception qui sera lancée si nous saisissons la valeur 0 ou un nombre réel !

La distinction entre les différentes origines d'erreurs est d'une importance capitale pour le développement de programmes robustes.

Pour distinguer les différents types d'exceptions, nous pouvons utiliser autant de clauses `except` que nécessaire. L'exemple de code suivant montre comment distinguer une exception levée par la saisie d'une valeur alphanumérique au lieu d'un nombre entier et une exception levée à cause d'une tentative de division par zéro.

Remarque

Avec la fonction `int()`, les nombres réels sont considérés comme des valeurs alphanumériques. Ainsi, le même type d'exception sera levé avec la saisie de la valeur `"1a1"` ou la saisie d'un nombre réel tel que `"3.14"`.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Exception-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
try:
    a=int(input("Veuillez saisir un nombre : "))
    print(f"10/{a}={10/a}")
except ValueError:
    print("Vous avez saisi une valeur non numérique !")
except ZeroDivisionError :
    print("La division par zéro est impossible !")

print("Je suis la dernière instruction de ce programme")
```

Dans ce nouveau programme, nous avons utilisé deux clauses `except` pour indiquer au programme comment se comporter en fonction du type de l'exception levée. Ainsi, ce programme affiche le message "`Vous avez saisi une valeur non numérique !`" lorsqu'une exception est levée à cause de la saisie d'une valeur alphanumérique et il affiche le message "`La division par zéro est impossible !`" lorsqu'une exception est levée à cause d'une tentative de division par zéro.

De cette façon, si notre programme lève une exception, nous connaissons la cause précise de cette exception, contrairement au programme de l'exemple précédent où la levée d'une exception a été gérée avec une seule clause `except`.

L'exécution de ce programme avec la saisie de "`1a1`" affiche le résultat suivant :

```
Veuillez saisir un nombre : 1a1
Vous avez saisi une valeur non numérique !
Je suis la dernière instruction de ce programme
```

On voit que le message associé à l'exception `ValueError` a été affiché et on voit aussi que le programme a continué son exécution après le bloc `try-except`, puisqu'il exécute également la dernière instruction `print` du programme qui est située en dehors de ce bloc `try-except`.

L'exécution de ce programme avec la saisie de la valeur 0 affiche le résultat suivant :

```
Veuillez saisir un nombre : 0
La division par zéro est impossible !
Je suis la dernière instruction de ce programme
```

© Editions ENI - All rights reserved

On voit que le message associé à l'exception `ZeroDivisionError` a été affiché et, comme lors de l'exécution précédente, le programme a continué son exécution après le bloc `try-except`.

4.9.4 Utiliser la clause finally

Souvent, lorsque nous développons des blocs de code pour la gestion des exceptions, nous avons besoin de réaliser certaines tâches, qu'une exception ait été levée ou non. Par exemple, si un fichier a été ouvert en lecture, nous aimerions fermer la connexion vers ce fichier, qu'une exception ait été levée ou non.

Pour s'assurer qu'une tâche s'exécute indépendamment si une exception est levée ou pas, il suffit d'utiliser la clause `finally` comme dans le programme suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Exception-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
try:
   a=int(input("Veuillez saisir un nombre : "))
   print(f"10/{a}={10/a}")
except ValueError:
   print("Vous avez saisi une valeur non numérique !")
except ZeroDivisionError :
   print("La division par zéro est impossible !")
finally :
   print("Je suis finally !")

print("Je suis la dernière instruction de ce programme")
```

Ce nouveau programme est similaire au programme de l'exemple précédent, mais cette fois nous avons ajouté un bloc de code associé à la nouvelle clause `finally`. L'instruction `print` de ce nouveau bloc `finally` sera exécutée dans les deux cas. Elle sera exécutée si le programme fonctionne normalement sans la levée d'exception et elle sera également exécutée si une exception quelconque est levée.

Ce programme affiche le résultat suivant après la saisie de la valeur 10 :

```
Veuillez saisir un nombre : 10
10/10=1.0
Je suis finally !
Je suis la dernière instruction de ce programme
```

On voit que le programme a fonctionné normalement avec le calcul de `10/10` et la clause `finally` a été exécutée, puisque le message "`Je suis finally !`" est affiché.

Également, ce programme affiche le message suivant après la saisie de "`1a1`" :

```
Veuillez saisir un nombre : 1a1
Vous avez saisi une valeur non numérique !
Je suis finally !
Je suis la dernière instruction de ce programme
```

On voit que le message "`Je suis finally`" a été affiché.

Ces deux dernières exécutions de notre programme montrent que la clause `finally` s'exécute indépendamment du fait qu'il y ait une levée d'exception ou pas.

4.9.5 Utiliser la structure try-except-finally-else

Avec la gestion des exceptions, il est fortement recommandé de n'insérer dans un bloc `try` que des instructions susceptibles de lever des exceptions.

Par exemple, si vous souhaitez insérer des instructions qui ne sont pas susceptibles de lever des exceptions juste après une instruction potentiellement source d'exceptions, alors il est considéré comme une bonne pratique d'ajouter ces instructions dans un bloc `else` au lieu de les ajouter à la fin du bloc `try` comme dans l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Exception-5.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
try:
    a=int(input("Veuillez saisir un nombre : "))
    print(f"10/{a}={10/a}")
except ValueError:
    print("Vous avez saisi une valeur non numérique !")
```

© Editions ENI - All rights reserved

```
except ZeroDivisionError :
    print("La division par zéro est impossible !")
else :
    print("Je suis else !")
finally :
    print("Je suis finally !")

print("Je suis la dernière instruction de ce programme")
```

Dans ce nouveau programme, nous affichons le message "`Je suis else !`" dans un bloc de code associé à la clause `else`. Cette clause `else` ne sera exécutée que si aucune exception n'est levée !

Ce programme affiche le résultat suivant après la saisie de la valeur 10 :

```
Veuillez saisir un nombre : 10
10/10=1.0
Je suis else !
Je suis finally !
Je suis la dernière instruction de ce programme
```

On voit que la clause `else` a bien été exécutée, puisqu'il n'y a pas eu de levée d'exception.

En réexécutant ce programme avec la saisie de la valeur 0, le résultat suivant est affiché :

```
Veuillez saisir un nombre : 0
La division par zéro est impossible !
Je suis finally !
Je suis la dernière instruction de ce programme
```

En revanche, comme lors de cette dernière exécution il y a eu la levée de l'exception `ZeroDivisionError`, le bloc de code associé à la clause `else` n'a pas été exécuté !

L'utilisation de la clause `else` est bien sûr recommandée, mais pas obligatoire. On peut ajouter les instructions du bloc `else` à la fin du bloc `try`, mais une telle pratique peut rendre la lisibilité d'un programme plus complexe. En effet, le fait d'adopter l'utilisation des blocs `else` avec la structure `try-except` permet d'analyser les codes sources de manière plus aisée, puisqu'à ce moment-là nous savons que dans le bloc `try` il n'y a que des instructions qui peuvent potentiellement lever des exceptions.

4.9.6 Lever une exception avec raise

Dans les exemples précédents, nous avons vu que la gestion des exceptions se réalise en utilisant des clauses `except` placées après un bloc `try`. Dans certains cas, nous ne souhaitons pas gérer une exception dès sa levée. Au contraire, nous souhaitons gérer ces exceptions dans des endroits précis de notre code. Dans d'autres cas, nous souhaitons lever des exceptions de manière explicite.

Lorsqu'une exception est levée dans une fonction, cette exception peut soit être traitée à l'aide d'un bloc `except` dans cette même fonction, soit la gestion de cette exception peut être déléguée à la fonction appelante. C'est-à-dire que si dans une fonction `fonction_1` on appelle la fonction `fonction_2`, et qu'une exception est levée dans la fonction `fonction_2` alors cette exception peut être soit gérée dans `fonction_2` à l'aide d'une clause `except`, soit la fonction `fonction_2` décide de ne pas gérer cette exception et de déléguer sa gestion à la fonction `fonction_1`. Dans ce cas-là, la fonction `fonction_2` doit utiliser une instruction `raise` afin de tenir au courant la fonction `fonction_1` de cette exception comme le montre l'exemple de code suivant :

Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Exception-6.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
def fonction_2():
    try:
        a=int(input("Veuillez saisir un nombre : "))
        print(f"10/{a}={10/a}")
    except Exception as ex :
        print("Exception levée dans la fonction fonction_2")
        raise ex

def fonction_1():
    try:
        fonction_2()
    except ValueError:
        print("Vous avez saisi une valeur non numérique !")
    except ZeroDivisionError :
        print("La division par zéro est impossible !")
```

© Editions ENI - All rights reserved

```
fonction_1()
print("Je suis la dernière instruction de ce programme")
```

Dans ce programme, nous avons défini les deux fonctions `fonction_1` et `fonction_2`.

La fonction `fonction_1` appelle la fonction `fonction_2` dans un bloc `try` et gère les exceptions qui seront potentiellement levées suite à l'appel de cette fonction `fonction_2`.

La fonction `fonction_2` réalise dans un bloc `try` les deux instructions que nous avons utilisées dans nos précédents exemples et qui peuvent potentiellement lever l'une des deux exceptions `ValueError` et `ZeroDivisionError`. Cependant, si l'une de ces deux exceptions est levée dans la fonction `fonction_2`, alors en utilisant l'instruction `raise` située dans un bloc `except`, cette exception sera remontée à la fonction appelante `fonction_1` et c'est dans cette dernière fonction que l'exception sera gérée.

La notation `except Exception as ex` permet de nommer une exception avec le nom `ex` pour ainsi faire référence à cette exception. Justement, l'instruction `raise` fait référence à cette exception `ex`.

Dans cet exemple, le programme principal lance la fonction `fonction_1`, puis affiche le message "`Je suis la dernière instruction de ce programme`".

L'exécution de ce programme avec la saisie de la valeur 0 affiche le message suivant :

```
Veuillez saisir un nombre : 0
Exception levée dans la fonction fonction_2
La division par zéro est impossible !
Je suis la dernière instruction de ce programme
```

On voit que le bloc de code associé à la clause `except` de la fonction `fonction_2` a été exécuté, mais que la gestion de l'exception `ZeroDivisionError` s'est poursuivie dans le bloc de code associé à la clause `except` de la fonction `fonction_1`.

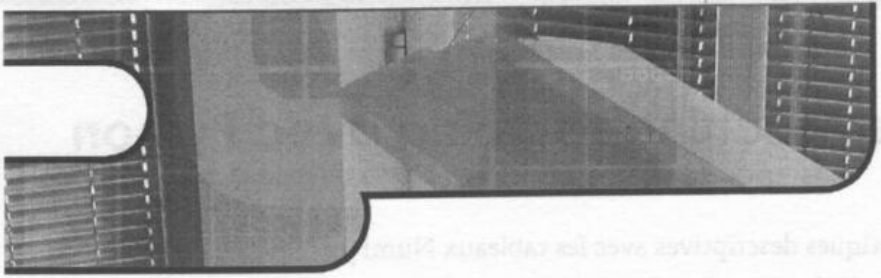
C'est en utilisant l'instruction ainsi `raise` que nous pouvons lancer explicitement des exceptions.

Dans les exemples de cette section, nous avons découvert les deux types d'exceptions `ValueError` et `ZeroDivisionError`, mais bien d'autres types d'exceptions sont prédéfinies dans le langage Python. Nous pouvons également créer nos propres exceptions en héritant de la classe `Exception`. Les notions telles que les classes et l'héritage seront l'objet de l'annexe A.

5. Conclusion

Ce chapitre nous a permis de découvrir l'essentiel du langage Python. Après avoir installé l'interpréteur Python, nous avons réalisé des exemples de code pour mettre en pratique les notions fondamentales de Python telles que les variables de base, les structures de données, la gestion des flux d'exécution d'un programme avec les structures conditionnelles et les boucles, les fonctions, les listes en compréhension, les expressions régulières et les exceptions. Bien que la lecture de ce chapitre ne soit pas suffisante pour couvrir l'intégralité du langage Python, les notions abordées le sont largement pour exprimer tout type d'algorithme !

© Editions ENI - All rights reserved



Chapitre 2-2
La bibliothèque NumPy

1. Objectif du chapitre

Dans la pratique, dès lors qu'on commence à développer un programme informatique, nous nous retrouvons à manipuler des structures de données complexes. Ces structures de données sont souvent des listes, des dictionnaires, des tuples, mais aussi des vecteurs à une ou à plusieurs dimensions, telles que les matrices.

Justement, la bibliothèque NumPy, qui est l'objet de ce chapitre, simplifie grandement, entre autres, la réalisation des opérations algébriques sur les vecteurs.

À la fin de ce chapitre, le lecteur aura abordé :

- la création, l'initialisation et la manipulation des tableaux NumPy à une et à plusieurs dimensions,
- la manipulation des informations relatives aux tableaux NumPy, telles que leurs dimensions, leurs tailles et leurs types,
- les différentes façons d'indexer un tableau NumPy afin d'accéder aux données qu'il contient,
- la modification des données de tableaux NumPy,
- la réalisation d'opérations algébriques sur les tableaux NumPy,
- la restructuration de tableaux NumPy,

- les statistiques descriptives avec les tableaux NumPy,
- la lecture de données d'un fichier et leur stockage dans un tableau NumPy,
- la réalisation d'opérations booléennes sur des tableaux NumPy afin de créer des masques,
- la comparaison entre les listes Python et les tableaux NumPy.

2. NumPy en deux mots

La bibliothèque NumPy permet de réaliser des calculs numériques de façon simple, intuitive et surtout performante sur des vecteurs à une ou à plusieurs dimensions. Les performances élevées de la bibliothèque NumPy sont dues en grande partie à la gestion efficace des tableaux de nombres. En effet, comme nous allons le découvrir tout au long de ce chapitre et particulièrement à la dernière section, NumPy gère les tableaux de nombres de manière plus concise que la manière avec laquelle Python gère les listes de nombres.

Le succès de la bibliothèque NumPy est également dû au fait que NumPy est utilisée comme la base de développement de plusieurs autres bibliothèques célèbres, telle que la bibliothèque Pandas, que nous allons aborder lors du prochain chapitre. Également, un des points forts de la bibliothèque NumPy est qu'elle est adaptée pour stocker et traiter les données matricielles associées aux images.

3. Les tableaux NumPy

3.1 Création de tableaux NumPy

Avant de se lancer dans l'utilisation de la bibliothèque NumPy, il est nécessaire de l'installer, car elle ne fait pas partie des modules natifs de Python. Pour cela, utilisez la commande suivante dans un interpréteur de commande.

```
pip install numpy
```

© Editions ENI - All rights reserved

Une fois que l'exécution de la commande précédente est terminée, un message sera affiché pour vous indiquer que NumPy a bien été installée. À ce stade, vous êtes prêt pour écrire des programmes exploitant NumPy !

3.1.1 Créer un tableau à une dimension

Pour exploiter la bibliothèque NumPy, il est nécessaire d'importer le module numpy. Traditionnellement, l'alias np est utilisé afin de référencer la bibliothèque NumPy.

Suivez les étapes ci-dessous afin de réaliser votre premier programme avec NumPy.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant puis exécutez ce code :

```
import numpy as np

vecteur = np.array([1,2,3,4])
print("vecteur = ", vecteur)
```

Ce programme commence par importer la bibliothèque NumPy avec l'alias np, puis définit la variable `vecteur` qui correspond à un nouveau tableau NumPy créé avec la fonction `numpy.array`. Ce nouveau tableau contient les valeurs 1, 2, 3 et 4. La dernière instruction permet simplement d'afficher les valeurs du tableau `vecteur`, comme on peut le voir avec le résultat ci-après :

```
vecteur =  [1 2 3 4]
```

Le tableau `vecteur` est un tableau NumPy à une dimension, c'est-à-dire qu'il est défini suivant un seul axe. La section suivante montre comment créer des tableaux NumPy à plusieurs dimensions.

3.1.2 Créer un tableau à plusieurs dimensions

Pour créer un tableau NumPy à deux dimensions, c'est-à-dire un tableau défini sur deux axes, suivez les étapes ci-après.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np
```

```
vecteur_2d = np.array([[1,2,3,4],[5,6,7,8]])
print("vecteur_2d = \n", vecteur_2d)
```

Ce programme, comme le précédent, commence par importer la bibliothèque NumPy avec l'alias np, puis définit la variable `vecteur_2d` qui correspond à un nouveau tableau NumPy créé avec la fonction `numpy.array`. Ce nouveau tableau est en réalité une matrice qui contient les valeurs `1`, `2`, `3` et `4` à la première ligne et les valeurs `5`, `6`, `7` et `8` à la deuxième ligne. La dernière instruction permet simplement d'afficher les valeurs du tableau `vecteur_2d`, comme on peut le voir avec le résultat ci-après :

```
vecteur_2d =
[[1 2 3 4]
[5 6 7 8]]
```

Dans le dernier programme, la création de la matrice `vecteur_2d` a été possible en considérant tout simplement que chaque ligne de cette matrice est elle-même un tableau à une dimension. C'est pour cette raison que la fonction `numpy.array` a reçu en paramètre deux tableaux, le tableau `[1,2,3,4]` et le tableau `[5,6,7,8]`, séparés avec une virgule, et ces deux tableaux sont eux-mêmes encadrés avec les deux crochets `[` et `]`. En suivant cette même logique, nous pouvons créer un tableau à trois dimensions comme suit :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_3d = np.array([[[1,1,1],[1,1,1]],[[2,2,2],[2,2,2]]])
print("vecteur_3d = \n", vecteur_3d)
```

Ce programme est similaire au programme précédent, sauf que cette fois la fonction `numpy.array` reçoit en paramètre deux matrices, la matrice `[[1,1,1],[1,1,1]]` et la matrice `[[2,2,2],[2,2,2]]`. Ce programme donne le résultat suivant :

```
vecteur_3d =
[[[1 1 1]
  [1 1 1]]

 [[2 2 2]
  [2 2 2]]]
```

© Editions ENI - All rights reserved

Dans la suite de ce chapitre, nous allons voir comment lire et modifier les données des tableaux à une ou à plusieurs dimensions.

3.2 Les dimensions d'un tableau NumPy

Souvent, lorsque nous manipulons des tableaux, notamment ceux à plusieurs dimensions, nous sommes amenés à réaliser des tâches en fonction du nombre de dimensions et/ou du nombre d'éléments de ces tableaux. Par exemple, nous pouvons avoir besoin d'itérer sur un bloc de code autant de fois que le nombre de dimensions d'un tableau donné, ou bien avoir besoin de connaître le nombre d'éléments stockés dans chacun des tableaux définis sur un axe donné !

Pour illustrer comment lire le nombre de dimensions d'un tableau ainsi que le nombre d'éléments stockés dans chacun des tableaux relatifs à un axe donné, nous allons réaliser les deux programmes à suivre.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur = np.array([1,2,3,4])
print("vecteur : ", vecteur)
print(f"le nombre de dimensions de vecteur = {vecteur.ndim}")
print(f"les dimensions du vecteur vecteur = {vecteur.shape}")
```

Ce programme définit le tableau `vecteur` à une dimension avec la fonction `numpy.array`, puis affiche les deux informations relatives respectivement au nombre de dimensions qui définissent ce tableau avec l'attribut `numpy.ndim` et la taille de chacune de ses dimensions avec l'attribut `numpy.shape`. Ce programme affiche les résultats suivants :

```
vecteur :  [1 2 3 4]
le nombre de dimensions de vecteur = 1 les dimensions du vecteur
vecteur = (4,)
```

Dans ces résultats, on voit que le tableau `vecteur` est défini suivant un seul axe où sont stockés quatre éléments.

Notez que, bien que le tableau `vecteur` soit défini sur une seule dimension, l'attribut `shape` a retourné un tuple `(4,)`.

Voyons maintenant ce que retournent les attributs `ndim` et `shape` sur un tableau défini sur trois axes, c'est-à-dire un tableau de dimension 3.

Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_3d = np.array([[[1,1,1,1],[1,1,1,1],[1,1,1,1]],
                       [[2,2,2,2],[2,2,2,2],[2,2,2,2]]])
print("vecteur_3d : \n", vecteur_3d)
print(f"le nombre de dimensions de vecteur_3d =
{vecteur_3d.ndim}")
print(f"les dimensions du vecteur vecteur_3d =
{vecteur_3d.shape}")
```

Ce code définit le tableau `vecteur_3d` en l'initialisant avec deux matrices, puis affiche le résultat avec les attributs `ndim` et `shape` sur ce tableau. Le résultat est le suivant :

```
vecteur_3d :
[[[1 1 1 1]
  [1 1 1 1]
  [1 1 1 1]]

 [[2 2 2 2]
  [2 2 2 2]
  [2 2 2 2]]]
le nombre de dimensions de vecteur_3d = 3
les dimensions du vecteur vecteur_3d = (2, 3, 4)
```

L'attribut `ndim` a retourné la valeur `3` pour indiquer que le vecteur `vecteur_3d` est défini suivant trois axes. L'attribut `shape` a retourné la valeur `(2,3,4)`. Ce 3-uplet peut être lu comme suit : le tableau `vecteur_3d` contient deux matrices, et chacune de ces deux matrices est définie par trois lignes, et enfin chacune des lignes des deux matrices est définie par quatre colonnes.

© Editions ENI - All rights reserved

3.3 Le type et la taille d'un tableau NumPy

En plus des attributs ndim et shape qui permettent de lire les dimensions d'un tableau NumPy, les fonctions et attributs suivants sont également utiles pour récupérer les informations relatives au type d'un tableau, au type des éléments d'un tableau, à la taille mémoire en octets occupée par chacun des éléments d'un tableau, au nombre total des éléments d'un tableau ainsi qu'au nombre total d'octets utilisés par un tableau. Le programme à suivre est un exemple d'utilisation de toutes ces fonctions.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_1 = np.array([[1,2,3,4],[5,6,7,8]], dtype='int16')
vecteur_2 = np.array([[1,2,3,4],[5,6,7,8]], dtype='int32')
vecteur_3 = np.array([[1,2,3,4],[5,6,7,8]], dtype='float')

print(f"- Le type de vecteur_1 = {type(vecteur_1)}\n"
     +f"- Le type de vecteur_2 = {type(vecteur_2)}\n"
     +f"- Le type de vecteur_3 = {type(vecteur_3)}\n")

print(f"- Le type des éléments de vecteur_1 =
         {vecteur_1.dtype}\n"
     +f"- Le type des éléments de vecteur_2 =
          {vecteur_2.dtype}\n"
     +f"- Le type des éléments de vecteur_3 =
          {vecteur_3.dtype}\n")

print(f"- Taille de chaque élément de vecteur_1 =
         {vecteur_1.itemsize} octets\n"
     +f"- Taille de chaque élément de vecteur_2 =
         {vecteur_2.itemsize} octets\n"
     +f"- Taille de chaque élément de vecteur_3 =
         {vecteur_3.itemsize} octets\n")

print(f"- Nombre d'éléments dans vecteur_1 = {vecteur_1.size}\n"
     +f"- Nombre d'éléments dans vecteur_2 = {vecteur_2.size}\n"
     +f"- Nombre d'éléments dans vecteur_3 = {vecteur_3.size}\n")

print(f"- Taille totale de vecteur_1 = {vecteur_1.nbytes}
```

```
        octets\n"
    +f"- Taille totale de vecteur_2 = {vecteur_2.nbytes}
        octets\n"
    +f"- Taille totale de vecteur_3 = {vecteur_3.nbytes}
        octets")
```

Ce programme déclare trois vecteurs : `vecteur_1` défini sur une seule dimension, `vecteur_2` défini sur deux dimensions et `vecteur_3` défini sur trois dimensions. Pour chacun de ces trois vecteurs, le programme affiche le type avec la fonction `type`, le type des éléments qu'il contient avec l'attribut `dtype`, la taille en octets de chacune des valeurs qu'il contient avec l'attribut `itemsize`, le nombre de valeurs qu'il contient avec l'attribut `size` et enfin le nombre d'octets total qu'il contient avec l'attribut `nbytes`. Les résultats obtenus sont affichés ci-dessous :

```
- Le type de vecteur_1 = <class 'numpy.ndarray'>
- Le type de vecteur_2 = <class 'numpy.ndarray'>
- Le type de vecteur_3 = <class 'numpy.ndarray'>

- Le type des éléments de vecteur_1 = int16
- Le type des éléments de vecteur_2 = int32
- Le type des éléments de vecteur_3 = float64

- Taille de chaque élément de vecteur_1 = 2 octets
- Taille de chaque élément de vecteur_2 = 4 octets
- Taille de chaque élément de vecteur_3 = 8 octets

- Nombre d'éléments dans vecteur_1 = 8
- Nombre d'éléments dans
vecteur_2 = 8
- Nombre d'éléments dans vecteur_3 = 8

- Taille totale de vecteur_1 = 16 octets
- Taille totale de vecteur_2 = 32 octets
- Taille totale de vecteur_3 = 64 octets
```

© Editions ENI - All rights reserved

3.4 Fonction d'initialisation de tableaux NumPy

Dans certains cas, on aimerait déclarer un tableau NumPy sans pour autant connaître a priori et avec précision toutes les valeurs qui y seront stockées ; dans certains autres cas, on souhaiterait réaliser des opérations sur des tableaux qui nécessitent que ces tableaux soient initialisés avec des valeurs spéciales ; ou encore, dans d'autres cas, on souhaite tout simplement créer un tableau et l'initialiser avec des valeurs particulières. Justement, NumPy offre une multitude de fonctions qui permettent de créer et d'initialiser des tableaux.

Le code suivant montre comment créer un tableau avec dix valeurs, toutes égales à 0.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np
vecteur_1 = np.zeros(10)
vecteur_2 = np.zeros(10, dtype='int32')
vecteur_3 = np.ones((3,4), dtype='int16')
print("vecteur_1 = ", vecteur_1)
print("vecteur_2 = ", vecteur_2)
print("vecteur_3 =\n ", vecteur_3)
```

Ce programme utilise à deux reprises la fonction `np.zeros` afin d'initialiser les deux tableaux `vecteur_1` et `vecteur_2` avec dix valeurs égales à zéro, d'où la valeur `10` du premier paramètre des deux appels à cette fonction. Le deuxième appel à la fonction `np.zeros` utilise un deuxième paramètre `dtype='int32'` afin de préciser que les données du tableau `vecteur_2` sont de type `'int32'` et non pas de type `'float'`, qui est le type par défaut. Également, la fonction `np.ones` a été utilisée pour créer et initialiser `vecteur_3` avec une matrice de trois lignes et quatre colonnes dont toutes les valeurs sont égales à `1`. Ce programme se termine par l'affichage des trois tableaux : `vecteur_1`, `vecteur_2` et `vecteur_3`.

Ce code affiche le résultat ci-dessous :

```
vecteur_1 =  [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
vecteur_2 =  [0 0 0 0 0 0 0 0 0 0]
vecteur_3 =
```

```
[[1 1 1 1]
 [1 1 1 1]
 [1 1 1 1]]
```

Remarquez la présence du caractère "." à côté des zéros du premier tableau qui indique que les données de `vecteur_1` sont de type `float`.

Une autre fonction d'initialisation couramment utilisée est la fonction `full` qui permet d'initialiser un tableau avec une valeur donnée en paramètre. Le programme suivant illustre un exemple d'utilisation de cette fonction `full`, mais aussi des fonctions d'initialisation `full_like` et `identity`.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np
vecteur_1 = np.full((3,4), 5)
vecteur_2 = np.full_like(vecteur_1, 10)
vecteur_3 = np.identity(3)
print("vecteur_1 =\n ", vecteur_1)
print("vecteur_2 =\n", vecteur_2)
print("vecteur_3 = \n", vecteur_3)
```

Dans ce programme, la fonction `full` permet de créer et d'initialiser `vecteur_1` avec une matrice de trois lignes et quatre colonnes dont toutes les valeurs sont égales à 5. La fonction `full_like` permet de créer et d'initialiser la matrice `vecteur_2` avec le même nombre de lignes et de colonnes que la matrice `vecteur_1` et avec toutes les valeurs égales à 10. La fonction `identity` permet d'affecter à `vecteur_3` la matrice identité d'ordre 3. Les trois dernières instructions de ce programme permettent d'afficher les trois tableaux `vecteur_1`, `vecteur_2` et `vecteur_3`. Ce programme affiche le résultat suivant :

```
vecteur_1 =
 [[5 5 5 5]
  [5 5 5 5]
  [5 5 5 5]]
vecteur_2 =
[[10 10 10 10]
 [10 10 10 10]
 [10 10 10 10]]
vecteur_3 =
[[1. 0. 0.]
```

© Editions ENI - All rights reserved

```
[0. 1. 0.]
[0. 0. 1.]]
```

Remarque

NumPy propose bien plus encore de fonctions d'initialisation, cette section n'a montré que celles qui sont couramment utilisées.

4. Accéder aux données d'un tableau NumPy

4.1 Accès aux données d'un tableau à une dimension

Une fois qu'un tableau est créé, l'accès à ses données se fait principalement via les indices de position.

Avec Python, les indices de position commencent à la valeur `0`, c'est-à-dire que le premier élément d'un tableau `T`, qui correspond à la première position, est situé à l'indice `0`, et pour y accéder il suffit de référencer ce tableau `T` avec l'indice `0` entre crochets comme suit : `T[0]`. Cette notation est similaire à la notation utilisée avec les listes Python.

Nous pouvons utiliser la notation avec le caractère : afin d'accéder aux valeurs d'un tableau NumPy ou d'une liste Python situées avec des indices définis dans un intervalle. Cette notation s'utilise comme suit :

```
[indice_début : indice_fin : pas]
```

La valeur `indice_début` indique la première position d'un tableau à partir de laquelle nous souhaitons commencer à lire les données. La valeur `indice_fin` indique jusqu'à quelle position nous souhaitons lire les données. Attention, cet indice n'est pas inclus et la dernière valeur qui sera lue est la valeur située à l'indice `(indice_fin - 1)`. La valeur pas indique le saut qui sera effectué entre `indice_début` et `indice_fin`.

Par exemple, pour accéder à toutes les valeurs d'un tableau `T` situées entre la deuxième position et la quatrième position incluse, nous pouvons utiliser la notation `T[1:4]`.

Également, si nous souhaitons accéder à toutes les valeurs d'un tableau `T` situées entre l'indice 2 et l'indice 10, mais en ignorant tous les indices impairs, alors nous pouvons utiliser la notation `T[2:10:2]`.

Une autre forme d'indexation autorisée avec NumPy, mais pas avec les listes Python, est l'indexation par une liste. Par exemple, pour accéder aux valeurs d'un tableau NumPy `T` situées avec les indices 3, 5 et 6, nous pouvons utiliser la notation `T[[3,5,6]]`, en désignant la liste des indices entre crochets.

Le programme suivant illustre comment créer et accéder aux données d'un tableau à une dimension en utilisant les notations citées ci-dessus :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_1d = np.array([1,2,3,4,5,6,7,8,9,10])
print("vecteur_1d = ",vecteur_1d)
print("vecteur_1d[1] = ", vecteur_1d[1])
print("vecteur_1d[[1,3]] = ", vecteur_1d[[1,3]])
print("vecteur_1d[[0,-1]] = ", vecteur_1d[[0,-1]])
print("vecteur_1d[2:6] = ", vecteur_1d[2:6])
print("vecteur_1d[2:] = ", vecteur_1d[2:])
print("vecteur_1d[0:10:2] = ", vecteur_1d[0:10:2])
```

Ce programme crée un tableau à une dimension `vecteur_1d`, puis affiche tous les éléments de ce tableau avec le premier appel à la fonction `print`. Le deuxième appel à la fonction `print` permet d'afficher la valeur du tableau `vecteur_1d` située à la position 1, qui correspond à la valeur 2 de ce tableau puisque la première valeur est située à la position 0. Le troisième appel de print affiche les deux valeurs du tableau `vecteur_1d` situées aux positions 1 et 3. La quatrième instruction affiche les valeurs situées aux positions 0 et -1. L'indice `-1` correspond à la dernière valeur du tableau. Le cinquième `print` affiche les valeurs situées entre l'indice 2 et l'indice 6 exclu. Le sixième `print` affiche toutes les valeurs en commençant de l'indice 2 jusqu'à la fin. Le dernier `print` affiche toutes les valeurs situées entre l'indice 0 et l'indice 10 en faisant des pas de deux.

© Editions ENI - All rights reserved

L'exécution de ce programme affiche les résultats suivants :

```
vecteur_1d =  [ 1  2  3  4  5  6  7  8  9 10]
vecteur_1d[1] =  2
vecteur_1d[[1,3]] =  [2 4]
vecteur_1d[[0,-1]] =  [ 1 10]
vecteur_1d[2:6] =  [3 4 5 6]
vecteur_1d[2:] =  [ 3  4  5  6  7  8  9 10]
vecteur_1d[0:10:2] =  [1 3 5 7 9]
```

4.2 Accès aux données d'un tableau à deux dimensions

La logique d'accès aux données d'une matrice NumPy est la même que celle qui permet d'accéder aux données d'un tableau à une dimension. Mais avec les matrices, une valeur est désignée avec deux indices : un premier indice pour indiquer la position de la valeur à laquelle on souhaite accéder par rapport au premier axe et un deuxième indice pour indiquer la position de cette même valeur par rapport au deuxième axe.

Également, nous pouvons utiliser l'indexation par intervalle et par liste que nous avons vue précédemment. Le programme suivant illustre les accès aux données d'une matrice.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_2d = np.array([[1,2,3],
                       [4,5,6],
                       [7,8,9],
                       [10,11,12],
                       [13,14,15]])

print("vecteur_2d[0,0] = ",vecteur_2d[0,0])
print("vecteur_2d[1,1] = ",vecteur_2d[2,1])
print("vecteur_2d[2,2] = ",vecteur_2d[4,2],"\n")
print("vecteur_2d[:,:] =\n ",vecteur_2d[:,:],"\n")
print("vecteur_2d[1,:] = ",vecteur_2d[1,:],"\n")
print("vecteur_2d[:,1] = ",vecteur_2d[:,1],"\n")
print("vecteur_2d[0:5:2,1] = ",vecteur_2d[0:5:2,1],"\n")
```

Ce programme crée la matrice `vecteur_2d` qui contient cinq lignes et trois colonnes. Puis, dans l'ordre, il affiche les informations suivantes :

- La valeur située à la première ligne et la première colonne `vecteur_2d[0,0]`.
- La valeur située à la troisième ligne et la deuxième colonne `vecteur_2d[2,1]`.
- La valeur située à la cinquième ligne et la troisième colonne `vecteur_2d[4,2]`.
- Toutes les valeurs de toutes les lignes et toutes les colonnes `vecteur_2d[:,:]`.
- Toutes les valeurs de toutes les colonnes, mais que de la deuxième ligne `vecteur_2d[1,:]`.
- Toutes les valeurs de toutes les lignes, mais que de la deuxième colonne `vecteur_2d[:,1]`.
- Les valeurs de la deuxième colonne situées entre l'indice 0 et l'indice 5 exclu en faisant des pas de deux.

Ce programme donne les résultats suivants :

```
vecteur_2d[0,0] =  1
vecteur_2d[1,1] =  8
vecteur_2d[2,2] =  15

vecteur_2d[ :,:] =
 [[ 1  2  3]
  [ 4  5  6]
  [ 7  8  9]
  [10 11 12]
  [13 14 15]]

vecteur_2d[ 1,:] =  [4 5 6]

vecteur_2d[ :,1] =  [ 2  5  8 11 14]

vecteur_2d[0:5:2,1] =  [ 2  8 14]
```

© Editions ENI - All rights reserved

4.3 Accès aux données d'un tableau à trois dimensions

Voyons maintenant un dernier exemple pour illustrer les accès aux données d'un tableau à trois dimensions.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_3d = np.array([[[1,0,0],[0,0,0]],
                       [[0,2,0],[0,0,3]]])
print("vecteur_3d[0,0,0] = ", vecteur_3d[0,0,0])
print("vecteur_3d[1,0,1] = ", vecteur_3d[1,0,1])
print("vecteur_3d[1,1,2] = ", vecteur_3d[1,1,2])
```

Ce programme crée un tableau `vecteur_3d` à trois dimensions. Dans les faits, ce tableau `vecteur_3d` peut être vu comme un assemblage de matrices. Dans cet exemple, `vecteur_3d` contient deux matrices situées au niveau du premier axe, c'est-à-dire que le premier axe indique la matrice à laquelle on souhaite accéder. La première matrice est `[[1,0,0],[0,0,0]]` et la deuxième matrice est `[[0,2,0],[0,0,3]]`. Le deuxième axe indique les positions des lignes au niveau des matrices. Le troisième axe définit les positions au niveau des colonnes des matrices.

Le tableau `vecteur_3d` contient volontairement des zéros partout sauf au niveau des trois positions qui sont affichées avec les trois appels à la fonction `print`. Le résultat de ce programme est le suivant :

```
vecteur_3d[0,0,0] =  1
vecteur_3d[1,0,1] =  2
vecteur_3d[1,1,2] =  3
```

Remarque

L'accès aux données de tableaux définis dans des dimensions supérieures à trois suit la même logique que les accès aux tableaux de dimensions inférieures. Chaque position indiquée entre les deux crochets correspond à un axe d'une des dimensions.

5. Modifier les données d'un tableau NumPy

La modification des données d'un tableau NumPy est aussi simple que la lecture de ses données. En effet, pour modifier une valeur ou des valeurs d'un tableau NumPy, il suffit d'indiquer les indices des positions à modifier et la valeur à affecter à ces positions. Le programme suivant donne un exemple :

▶ Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur = np.array([[1,2,3],
                    [4,5,6],
                    [7,8,9]])

vecteur[1,1] = 99
print("Après vecteur[1,1] = 99 le tableau devient:\n",vecteur,
      end="\n\n")
vecteur[0,:] = 99
print("Après vecteur[0,:] = 99 le tableau devient:\n",vecteur,
      end="\n\n")
vecteur[:,2] = -99
print("Après vecteur[:,2] = -99 le tableau devient:\n",vecteur,
      end="\n\n")
```

Ce programme crée le tableau `vecteur` à deux dimensions puis réalise les trois affectations suivantes :

- Affectation de la valeur `99` à la position située à l'index `[1,1]`, c'est-à-dire à la position située à la deuxième ligne et à la deuxième colonne.
- Affectation de la valeur `99` aux positions définies avec l'index `[0,:]`, c'est-à-dire toutes les positions de la première ligne.
- Affectation de la valeur `-99` aux positions définies avec l'index `[:,2]`, c'est-à-dire toutes les positions de la troisième colonne.

Ce programme affiche les résultats suivants :

```
Après vecteur[1,1] = 99 le tableau devient:
[[ 1  2  3]
 [ 4 99  6]
 [ 7  8  9]]
```

© Editions ENI - All rights reserved

```
Après vecteur[0,:] = 99 le tableau devient:
[[99 99 99]
[ 4 99  6]
[ 7  8  9]]

Après vecteur[:,2] = -99 le tableau devient:
[[ 99  99 -99]
[  4  99 -99]
[  7   8 -99]]
```

6. Copier un tableau NumPy dans un autre tableau NumPy

Un point important à prendre en compte lorsque nous manipulons les tableaux, et de façon générale lorsque nous manipulons des objets, est que lorsque nous affectons un tableau `T1` à un autre tableau `T2`, le résultat de cette affectation n'est pas la copie des valeurs stockées dans `T1` dans le tableau `T2`, mais seulement la copie de l'adresse où est stocké `T1` dans la variable `T2` ! En d'autres termes, après l'affectation de `T1` à `T2`, les deux tableaux pointent vers les mêmes adresses mémoire et la modification de l'un des deux tableaux affecte l'état de l'autre tableau. L'exemple de programme suivant montre ce qui se passe réellement lors de l'affectation d'un tableau dans un autre.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_1 = np.array([1,2,3])
vecteur_2 = vecteur_1
vecteur_2[2] = 99
print("vecteur_1 =", vecteur_1)
print("vecteur_2 =", vecteur_2)
```

Ce programme crée tout simplement un tableau `vecteur_1` puis affecte ce même tableau au deuxième tableau `vecteur_2`. Puis la valeur `99` est affectée à la troisième position de `vecteur_2`. Les deux dernières instructions affichent l'état des deux vecteurs `vecteur_1` et `vecteur_2`.

Le résultat est le suivant :

```
vecteur_1 = [ 1  2 99]
vecteur_2 = [ 1  2 99]
```

Ce résultat montre que le tableau `vecteur_1` a été impacté par l'affectation de la valeur `99` à la deuxième position du tableau `vecteur_2` et finalement les deux tableaux contiennent les mêmes valeurs !

C'est tout à fait normal puisque `vecteur_1` et `vecteur_2` désignent tous deux la même adresse mémoire. L'exemple du programme suivant montre comment réaliser une vraie copie d'un tableau dans un autre tableau :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_1 = np.array([1,2,3])
vecteur_2 = vecteur_1.copy()
vecteur_2[2] = 99
print("vecteur_1 =", vecteur_1)
print("vecteur_2 =", vecteur_2)
```

Ce programme est le même que le précédent sauf que cette fois l'instruction `vecteur_2 = vecteur_1` est remplacée par l'instruction `vecteur_2 = vecteur_1.copy()`. La fonction `copy` crée un nouveau tableau avec de nouvelles adresses mémoire dans lesquelles sont réellement copiées les valeurs de `tableau_1` et les nouvelles adresses mémoire seront pointées par le tableau `vecteur_2`. Le résultat de ce programme est le suivant :

```
vecteur_1 = [1 2 3]
vecteur_2 = [ 1  2 99]
```

Cette fois, l'affectation de la valeur `99` à l'une des positions du tableau `vecteur_2` n'affecte pas l'état du tableau `vecteur_1`, car ici `vecteur_1` et `vecteur_2` pointent vers des adresses mémoire différentes.

© Editions ENI - All rights reserved

7. Algèbre linéaire avec NumPy

7.1 Opérations mathématiques de base avec NumPy

NumPy est intéressant à utiliser dès que nous manipulons des tableaux numériques, car comme nous allons le voir à l'avant-dernière section de ce chapitre, l'utilisation de ces tableaux permet d'avoir des programmes plus performants en matière de rapidité d'exécution.

Un autre avantage de NumPy est que les opérations algébriques sur les espaces vectoriels peuvent être facilement exprimées. Avec les tableaux NumPy, notamment les tableaux à deux dimensions, les opérations classiques telles que la multiplication ou l'inversion de matrices deviennent très faciles à programmer.

L'exemple de programme suivant montre comment réaliser tout simplement les opérations mathématiques de base avec des tableaux à une dimension. Ces mêmes opérations peuvent être appliquées sur des tableaux à plusieurs dimensions.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vecteur_1 = np.array([1,2,3,4,5])
print("vecteur_1 = ",vecteur_1)
print("vecteur_1 + 2 = ",vecteur_1+2)
print("vecteur_1 - 2 = ",vecteur_1-2)
print("vecteur_1 * 2 = ",vecteur_1*2)
print("vecteur_1 / 2 = ",vecteur_1/2)
print("vecteur_1 ** 2 = ",vecteur_1**2)
print("Cosinus(vecteur_1) = ",np.cos(vecteur_1))

vecteur_2 = np.array([5,4,3,2,1])
print("vecteur_2 = ",vecteur_2)
print("vecteur_1 * vecteur_2 = ",vecteur_1*vecteur_2)
```

Ce programme crée le tableau `vecteur_1` à une dimension en lui affectant les valeurs 1, 2, 3, 4 et 5, puis réalise dans l'ordre les opérations suivantes :

- `vecteur_1+2` : ajout de 2 à chacune des valeurs de `vecteur_1`.
- `vecteur_1-2` : soustraction de 2 de chacune des valeurs de `vecteur_1`.
- `vecteur_1*2` : multiplication par 2 des valeurs de `vecteur_1`.
- `vecteur_1/2` : division par 2 des valeurs de `vecteur_1`.
- `vecteur_1**2` : puissance de 2 des valeurs de `vecteur_1`.
- `np.cos(vecteur_1)` : calcul du cosinus des valeurs de `vecteur_1`.

Puis le programme définit le tableau `vecteur_2` et affiche le résultat de la multiplication `vecteur_1*vecteur_2` qui correspond à la multiplication deux à deux des éléments des deux tableaux `vecteur_1` et `vecteur_2`.

Ce programme affiche les résultats suivants :

```
vecteur_1 =  [1 2 3 4 5]
vecteur_1 + 2 =  [3 4 5 6 7]
vecteur_1 - 2 =  [-1  0  1  2  3]
vecteur_1 * 2 =  [ 2  4  6  8 10]
vecteur_1 / 2 =  [0.5 1.  1.5 2.  2.5]
vecteur_1 ** 2 =  [ 1  4  9 16 25]
Cosinus(vecteur_1) =  [ 0.54030231 -0.41614684 -0.9899925
                       -0.65364362  0.28366219]
vecteur_2 =  [5 4 3 2 1]
vecteur_1 * vecteur_2 =  [5 8 9 8 5]
```

Cet exemple montre à quel point la réalisation des opérations mathématiques de base est simple et intuitive avec les tableaux NumPy.

Il faut bien noter le fait que les opérations de l'exemple précédent sont possibles avec les tableaux NumPy, mais impossibles avec les listes Python ! Par exemple, si vous essayez de diviser une liste par le nombre, l'interpréteur Python vous signalera une erreur.

Remarque

Attention, nous pouvons réaliser l'opération `l1+l2` avec `l1` et `l2` deux listes Python, mais le résultat de cette addition est simplement la concaténation des éléments de la liste `l1` aux éléments de la liste `l2`.

© Editions ENI - All rights reserved

7.2 Opérations sur les matrices avec NumPy

Dans cet exemple, nous allons réaliser la multiplication de deux matrices afin de montrer comment NumPy facilite la réalisation des opérations algébriques qui sont plus compliquées à réaliser avec nos propres programmes et en utilisant les listes Python.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

mat_1 = np.random.randint(-10,10, size=(3,4))
mat_2 = np.random.randint(-10,10, size=(4,2))

print("mat_1 :\n", mat_1,"\n")
print("mat_2 :\n", mat_2)
result = np.matmul(mat_1, mat_2)
print("mat_1 * mat_2 :\n", result)
```

Ce programme crée et initialise les deux matrices `mat_1` et `mat_2` en utilisant la fonction `numpy.random.randint`. Cette fonction génère un tableau NumPy avec des valeurs comprises dans l'intervalle indiqué par les deux valeurs correspondant aux deux premiers paramètres et avec des dimensions désignées avec le paramètre `size`. Ici, l'intervalle est `[-10, 10]` et les dimensions des deux matrices sont respectivement `(3,4)` et `(4,2)`.

La suite de ce programme consiste en l'affichage des deux matrices `mat_1` et `mat_2` et l'affichage du résultat de leur multiplication. Cette multiplication est réalisée par le simple appel de la fonction `numPy.matmul`.

Le résultat de ce programme est le suivant :

```
mat_1 :
[[-7 -4  4 -3]
[-4 -2  1  1]
[ 1  8 -8  9]]

mat_2 :
[[ 3 -5]
[-6 -4]
[ 8 -6]
```

```
[-4  1]]

mat_ * mat_2 :
[[  47   24]
 [   4   23]
 [-145   20]]
```

Cet exemple montre qu'avec NumPy la réalisation de la multiplication de deux matrices est aussi simple que l'appel d'une fonction.

De la même manière, NumPy propose de réaliser les opérations classiques de l'algèbre telles que le calcul du déterminant d'une matrice, le calcul de l'inverse d'une matrice ou encore le calcul des valeurs propres d'une matrice.

8. Réorganiser des tableaux NumPy

La réorganisation d'un tableau consiste en la modification de la structure de ce tableau. Cette modification de structure peut intervenir à plusieurs niveaux. Par exemple, un tableau à une dimension qui contient huit valeurs numériques peut être transformé en un tableau à deux dimensions qui représente une matrice avec quatre lignes et deux colonnes, ou encore transformé en un tableau à trois dimensions avec deux matrices réparties sur le premier axe, qui auront chacune deux lignes et deux colonnes. La réorganisation de tableaux peut également résulter de la concaténation verticale ou horizontale de plusieurs tableaux.

8.1 Restructurer un tableau NumPy

L'exemple du programme suivant montre comment partir d'un tableau à deux dimensions pour construire deux autres tableaux. Le premier tableau sera également défini sur deux axes, mais le second nouveau tableau sera défini sur trois axes.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

mat_1 = np.array([[1,2,3,4],[5,6,7,8]])
```

© Editions ENI - All rights reserved

```
mat_2 = mat_1.reshape(4,2)
mat_3 = mat_1.reshape(2,2,2)
print("mat_1:\n", mat_1)
print("mat_2:\n", mat_2)
print("mat_3:\n", mat_3)
```

Ce programme crée la matrice `mat_1` qui contient huit valeurs réparties sur deux lignes de quatre valeurs chacune. La matrice `mat_2` est créée en utilisant la fonction `reshape` avec comme paramètres les valeurs `4` et `2`. Ce premier appel à la fonction `reshape` va créer une nouvelle matrice avec quatre lignes et deux colonnes. Le deuxième appel à la fonction `reshape` va créer un nouveau tableau à trois dimensions dans lequel les données de la matrice `mat_1` vont être réparties sur deux matrices de deux lignes et deux colonnes chacune. Voici le résultat du programme précédent :

```
mat_1:
[[1 2 3 4]
[5 6 7 8]]
mat_2:
[[1 2]
[3 4]
[5 6]
[7 8]]
mat_3:
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Le point auquel il faut faire attention lorsque la fonction `reshape` est utilisée est que le nombre de valeurs dans le tableau à transformer doit être égal au nombre de valeurs possibles dans le nouveau tableau suivant les paramètres que nous donnons à la fonction `reshape`. Par exemple, dans notre exemple précédent, nous ne pouvons pas utiliser la fonction `reshape` comme suit :

```
mat_4 = mat_1.reshape(2,20)
```

ou bien :

```
mat_5 = mat_1.reshape(2,2)
```

car le nombre de valeurs dans la matrice `mat_1` est de 8, alors que le nombre de valeurs attendues dans les deux matrices `mat_4` et `mat_5` est respectivement de 40 et 4, ce qui ne correspond pas au nombre de valeurs dans le tableau `mat_1`. Bien sûr, dans ces deux derniers cas, Python va signaler une erreur à l'exécution.

8.2 Superposer des tableaux NumPy

On peut également créer un nouveau tableau à partir de la superposition de plusieurs autres tableaux. Cette superposition peut s'opérer de manière horizontale ou verticale. Le programme suivant illustre un exemple de superposition horizontale et de superposition verticale de tableaux.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

vec_1 = np.array([1,2,3,4,5])
vec_2 = np.array([6,7,8,9,10])

v_stack_vec = np.vstack([vec_1, vec_2])
print("Avec vstack:\n",v_stack_vec)

v_stack_big_vec = np.vstack([vec_1, vec_2,vec_1, vec_2])
print("\nEncore avec vstack:\n",v_stack_big_vec)

h_stack_vec = np.hstack([vec_1, vec_2])
print("\nAvec hstack:\n",h_stack_vec)
```

Ce programme crée deux tableaux `vec_1` et `vec_2`. Puis le tableau `v_stack_vec` est créé en appliquant la fonction `numpy.vstack` sur les deux tableaux `vec_1` et `vec_2` afin de les superposer verticalement et créer une matrice avec deux lignes. De nouveau, la fonction `numpy.vstack` et utilisée pour créer le tableau `v_stack_big_vec` en superposant verticalement les tableaux `vec_1`, `vec_2`, `vec_1` et `vec_2` et créer une matrice avec quatre lignes et cinq colonnes. Enfin, la fonction `numpy.hstack` est utilisée pour superposer horizontalement les deux tableaux `vec_1` et `vec_2` pour la création du nouveau tableau `h_stack_vec` qui sera un tableau à une dimension

© Editions ENI - All rights reserved

et qui contiendra les dix valeurs présentes dans les deux tableaux vec_1 et vec_2. Ce programme affiche les résultats ci-dessous :

```
Avec vstack:
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]

Encore avec vstack: [[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [ 1  2  3  4  5]
 [ 6  7  8  9 10]]

Avec hstack:
[ 1  2  3  4  5  6  7  8  9 10]
```

Remarque

La superposition de tableaux à plusieurs dimensions se réalise de la même manière que la superposition de tableaux à une seule dimension vue ci-dessus.

9. Statistiques descriptives avec NumPy

Le sixième chapitre est entièrement dédié aux statistiques, mais en attendant, cette section montre comment réaliser les statistiques descriptives élémentaires sur les données d'un tableau NumPy. Considérons le programme à suivre.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

mat_1 = np.random.randint(-10,10, size=(5,5))
print("mat_1 :\n", mat_1)

min_value = np.min(mat_1)
print(f"\nLa valeur minimale de toute la matrice est:
{min_value}\n")
max_value = np.max(mat_1)
print(f"La valeur maximale de toute la matrice est:
{max_value}\n")
```

```
min_value_by_col = np.min(mat_1, axis=0)
print(f"Les valeurs minimales par colonne sont:
{min_value_by_col}\n")
min_value_by_row = np.min(mat_1, axis=1)
print(f"Les valeurs minimales par ligne sont:
{min_value_by_row}\n")

mean_value_by_col = np.mean(mat_1, axis=0)
print(f"Les moyennes par colonne sont: {mean_value_by_col}\n")
std_by_col = np.std(mat_1, axis=0)
print(f"Les écarts-types par colonne sont: {std_by_col}\n")
sum_by_row = np.sum(mat_1, axis=1)
print(f"Les sommes par ligne sont: {sum_by_row}\n")
```

Ce programme définit et affiche la matrice `mat_1`. Cette matrice est initialisée avec des valeurs pseudo-aléatoires en utilisant la fonction `numpy.random.randint`. Puis les statistiques suivantes sont calculées et affichées :

- `np.min(mat_1)` : cette fonction permet de calculer la valeur minimale parmi toutes les valeurs de la matrice `mat_1`.
- `np.max(mat_1)` : cette fonction permet de calculer la valeur maximale parmi toutes les valeurs de la matrice `mat_1`.
- `np.min(mat_1, axis=0)` : cette fonction permet de calculer la valeur minimale par colonne. La valeur retournée par cette fonction est un tableau NumPy dont chaque position contient la valeur minimale de la colonne de la matrice `mat_1` située à la même position.
- `np.min(mat_1, axis=1)` : cette fonction permet de calculer la valeur minimale par ligne. La valeur retournée par cette fonction est un tableau NumPy dont chaque position contient la valeur minimale de la ligne de la matrice `mat_1` située à la même position.
- `np.mean(mat_1, axis=0)` : cette fonction permet de calculer la moyenne par colonne. La valeur retournée par cette fonction est un tableau NumPy dont chaque position contient la moyenne de la colonne de la matrice `mat_1` située à la même position.
- `np.std(mat_1, axis=0)` : cette fonction permet de calculer l'écart-type par colonne. La valeur retournée par cette fonction est un tableau NumPy dont chaque position contient l'écart-type de la colonne de la matrice `mat_1` située à la même position.

© Editions ENI - All rights reserved

– `np.sum(mat_1, axis=1)` : cette fonction permet de calculer la somme par ligne. La valeur retournée par cette fonction est un tableau NumPy dont chaque position contient la somme des valeurs de la ligne de la matrice `mat_1` située à la même position.

Ce programme affiche les résultats suivants :

```
mat_1 :
[[-3  3  1  6 -5]
 [ 2 -9  0  8  9]
 [-5  0 -9  7  0]
 [ 5  6 -5 -9 -6]
 [ 5 -9 -9 -9  7]]

La valeur minimale de toute la matrice est: -9

La valeur maximale de toute la matrice est: 9

Les valeurs minimales par colonne sont: [-5 -9 -9 -9 -6]

Les valeurs minimales par ligne sont: [-5 -9 -9 -9 -9]

Les moyennes par colonne sont: [ 0.8 -1.8 -4.4  0.6  1. ]

Les écarts-types par colonne sont: [4.11825206 6.17737808
4.2708313  7.8638413  6.09918027]

Les sommes par ligne sont: [  2  10  -7  -9 -15]
```

10. Lire des données NumPy à partir d'un fichier

Lire les données numériques stockées dans un fichier est très simple et pratique avec NumPy. Le programme suivant montre comment lire les données stockées dans un fichier *np_data.txt*.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

my_data = np.genfromtxt("../Data/np_data.txt", delimiter =',')
```

```
my_data =  my_data.astype('int32')
print(my_data)
```

Ce programme lit dans le tableau `my_data` les données du fichier *np_data.txt* avec la fonction `numpy.genfromtxt` en lui indiquant le chemin vers ce fichier et le caractère délimiteur des différentes valeurs dans ce fichier ; ici, c'est le caractère ','. Le fichier *np_data.txt* se trouve dans le dossier Data de ce chapitre. Le programme affiche les données du tableau `my_data`, puis transforme le type de ces données avec la fonction `astype`. En effet, le type par défaut des données est le type `float`, et à l'aide de la fonction `astype` nous pouvons indiquer le type souhaité pour ces données, ici le type `int32`. L'exécution de ce programme affiche les résultats suivants :

```
my_data: [[  2.   5.  36.   2.   8.]
 [ 55.   6.  97.   4. 256.]
 [245.   4.  32. 258.  65.]
 [134. 345. 652. 945. 121.]]

my_data:
[[  2   5  36   2   8]
 [ 55   6  97   4 256]
 [245   4  32 258  65]
 [134 345 652 945 121]]
```

11. Les masques booléens avec NumPy

11.1 Créer et utiliser un masque booléen

Souvent, lorsque nous manipulons des tableaux NumPy, nous avons besoin d'accéder à leurs valeurs suivant certaines conditions.

Le programme suivant montre comment accéder aux valeurs d'un tableau NumPy suivant des conditions fixées au préalable.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code.

© Editions ENI - All rights reserved

```
import numpy as np

my_data = np.genfromtxt("../Data/np_data.txt", delimiter =',')
my_data =  my_data.astype('int32')
print("my_data :\n",my_data)

bool_data = my_data > 100
print("\nLes indices booléens pour les valeurs > 100 :\n
",bool_data)

gt_100 = my_data[bool_data]
print("\nLes valeurs > 100 :\n",gt_100)
```

Ce programme lit les données stockées dans le fichier *np_data.txt* dans le tableau `my_data`. Puis il stocke dans un deuxième tableau `bool_data`, de la même taille que le tableau `my_data`, des valeurs booléennes en fonction du résultat de la comparaison avec la valeur `100` des valeurs du tableau `my_data`. Si la valeur du tableau `my_data` à une position donnée est supérieure à `100`, alors à cette même position du tableau `bool_data` sera stockée la valeur `True`, sinon c'est la valeur `False` qui sera affectée à cette position du tableau `bool_data`. Également, ce programme crée un troisième tableau `gt_100` dans lequel seront récupérées les valeurs de `my_data` suivant le masque défini avec les valeurs booléennes stockées dans le tableau `bool_data`. Ce programme affiche les résultats suivants :

```
my_data :
[[  2   5  36   2   8]
 [ 55   6  97   4 256]
 [245   4  32 258  65]
 [ 34  45 652  45 121]]

Les indices booléens pour les valeurs > 100 :
 [[False False False False False]
  [False False False False  True]
  [True False False  True False]
  [False False  True False  True]]
 Les valeurs > 100 :
[256 245 258 652 121]
```

Comme vous pouvez le constater, les valeurs de la matrice `my_data` supérieures à `100` sont toutes renseignées dans le tableau `gt_100`.

11.2 Un masque avec plusieurs conditions

On peut également combiner plusieurs conditions lors de la construction d'un masque relatif aux valeurs d'un tableau NumPy. L'exemple ci-dessous utilise une combinaison de deux conditions pour la construction de masques.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

my_data = np.genfromtxt("../Data/np_data.txt", delimiter =',')
my_data =  my_data.astype('int32')
print("my_data :\n",my_data)

in_50_100 = (my_data > 50) & (my_data < 100)
not_in_50_100 = ~((my_data > 50) & (my_data < 100))
print("Masque in_50_100  :\n ",in_50_100)
print("\nMasque not_in_50_100  :\n ",not_in_50_100)
```

Ce programme utilise, crée et affiche le tableau `my_data` à partir des données du fichier *np_data.txt* en utilisant la fonction `numpy.genfromtxt` comme dans l'exemple précédent. Puis les deux opérations suivantes sont réalisées :

- `in_50_100 = (my_data > 50) & (my_data < 100)` : cette instruction permet de construire un masque et de le stocker dans le tableau `in_50_100`. Ce masque est lui-même un tableau contenant des valeurs booléennes, `True` ou `False`. À une position donnée, ce masque va avoir la valeur `True` si la valeur considérée du tableau `my_data` est incluse dans l'intervalle `[50,100]`, sinon la valeur à cette position sera `False`.
- `not_in_50_100 = ~((my_data > 50) & (my_data < 100))` : cette instruction permet de construire un masque et de le stocker dans le tableau `not_in_50_100`. Ce masque est lui-même un tableau contenant des valeurs booléennes, `True` ou `False`. À une position donnée, ce masque va avoir la valeur `False` si la valeur considérée du tableau `my_data` est incluse dans l'intervalle `[50,100]`, sinon la valeur à cette position sera `True`. Ce masque est exactement l'inverse du premier masque `in_50_100`.

Ce programme se termine par l'affichage des deux masques `in_50_100` et `not_in_50_100`.

© Editions ENI - All rights reserved

Le résultat est le suivant :

```
my_data :
[[  2   5  36   2   8]
 [ 55   6  97   4 256]
 [245   4  32 258  65]
 [134 345 652 945 121]]

Masque in_50_100  :
[[False False False False False]
 [True False  True False False]
 [False False False False  True]
 [False False False False False]]

Masque not_in_50_100  :
 [[ True  True  True  True  True]
 [False  True False  True  True]
 [ True  True  True  True False]
 [ True  True  True  True  True]]
```

11.3 Les fonctions numpy.any et numpy.all

La bibliothèque NumPy propose plusieurs fonctions qui permettent de construire des masques adaptés en fonction des besoins. Les deux fonctions `numpy.all` et `numpy.any` font sans doute partie des fonctions les plus utilisées. Le programme suivant montre comment les utiliser pour la construction de deux masques différents.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code :

```
import numpy as np

my_data = np.genfromtxt("../Data/np_data.txt", delimiter =',')
my_data =  my_data.astype('int32')
print("my_data :\n",my_data)

any_gt_100_by_row = np.any(my_data>100, axis=1)
print("Est-ce qu'au moins une valeur de la ligne est>100:\n",
      any_gt_100_by_row)

all_gt_100_by_row = np.all(my_data>100, axis=1)
```

```
print("Est-ce que toutes les valeurs de la ligne sont>100:\n",
      all_gt_100_by_row)
```

Ce programme utilise le même tableau `my_data` que les deux derniers programmes et qui est lu à partir du fichier *np_data.txt*. Les deux instructions les plus importantes de ce programme sont les suivantes :

- `any_gt_100_by_row = np.any(my_data>100, axis=1)` : cette instruction permet de construire un masque de taille égale au nombre de lignes du tableau `my_data`, puisque l'argument `axis` est égal à `1`, et dès qu'une ligne donnée contient au moins une valeur supérieure à `100`, alors la valeur de ce masque correspondant à cette ligne sera égale à `True`, sinon cette valeur sera égale à `False`.
- `all_gt_100_by_row = np.all(my_data>100, axis=1)` : cette instruction permet de construire un masque de taille égale au nombre de lignes du tableau `my_data`, puisque l'argument `axis` est égal à `1`, et dès qu'une ligne donnée contient au moins une valeur inférieure à `100`, alors la valeur de ce masque correspondant à cette ligne sera égale à `False`, sinon cette valeur sera égale à `True`.

Le résultat de ce programme est le suivant :

```
my_data :
[[  2   5  36   2   8]
 [ 55   6  97   4 256]
 [245   4  32 258  65]
 [134 345 652 945 121]]

Est-ce qu'au moins une valeur de la ligne est > 100:
[False  True  True  True]

Est-ce que toutes les valeurs de la ligne sont > 100:
[False False False  True]
```

© Editions ENI - All rights reserved

12. Tableaux NumPy versus listes Python

Nous avons vu qu'avec la bibliothèque NumPy les opérations sur les vecteurs à plusieurs dimensions sont réalisées de manière simple et intuitive. En plus de cet avantage, les tableaux NumPy présentent un autre avantage sur les listes natives Python : celui d'être plus performants en termes du temps d'utilisation du processeur.

Les tableaux NumPy sont plus rapides à l'exécution que les listes Python pour les deux raisons suivantes :

1. La restriction sur les types de données : une liste Python permet de stocker des objets hétérogènes. Par exemple, une même liste Python peut contenir simultanément des chaînes de caractères, des nombres entiers et des nombres flottants. Avec les tableaux NumPy, ceci n'est pas possible. En effet, un tableau NumPy contient des données homogènes.

   Le fait que les données d'un tableau NumPy sont toutes de même type permet de les parcourir de manière optimisée.

   Soit la liste Python `[10, 'Hello', 3.14]`. Pour parcourir et lire tous les éléments de cette liste, Python doit d'abord réaliser un contrôle sur le type du premier élément afin de savoir sa taille en mémoire ; ici, c'est un entier. Puis lorsque Python accède au deuxième élément, il doit également d'abord contrôler le type de cet élément afin de savoir que c'est un objet de type chaîne de caractères et le lire correctement. Enfin, Python doit encore réaliser un contrôle sur le type du troisième élément pour comprendre que ce dernier élément est un nombre flottant et le lire en tant que tel. Avec NumPy, les données sont toutes de même type et donc tous ces contrôles ne sont plus nécessaires.

2. L'organisation des données en mémoire : les éléments d'une liste Python sont stockés de manière éparse dans la mémoire, c'est-à-dire que les objets d'une même liste sont éparpillés un peu partout dans la mémoire vive. Pour retrouver tous les éléments d'une liste, Python utilise un tableau dans lequel il stocke l'adresse mémoire où est stocké chaque objet. Pour parcourir tous les éléments d'une liste Python, il est donc nécessaire de parcourir le tableau où sont stockées toutes les adresses de ses éléments, puis de charger les données stockées dans les zones mémoire correspondantes. Avec les tableaux NumPy, les données sont stockées de manière contiguë dans la mémoire, c'est-à-dire que pour parcourir un tableau, il suffit de connaître le type des éléments de ce tableau, qui est le même pour tous les éléments, et de se déplacer d'un élément à un autre en faisant un saut égal au nombre d'octets nécessaires pour stocker des objets de ce type.

Les deux sections suivantes montrent à travers des exemples concrets qu'un tableau NumPy est plus rapide qu'une liste Python et qu'il occupe une taille mémoire sensiblement inférieure à la taille mémoire occupée par une liste Python.

Remarque

En plus des deux avantages mentionnés ci-dessus, les tableaux NumPy sont mieux adaptés pour des calculs SIMD, car leur utilisation permet une gestion efficace du cache.

12.1 Comparaison des tailles en mémoire

Afin de réaliser un exemple pour comparer les tailles mémoire nécessaires à un tableau NumPy et à une liste Python, suivez les étapes ci-dessous.

- Créez un fichier de type *.py* et saisissez dans ce nouveau fichier le code suivant puis exécutez-le :

```
import numpy as np
import sys

py_list = list(range(10000))
```

© Editions ENI - All rights reserved

```
numpy_list = np.array([py_list] )

print(f"La taille de py_list: {sys.getsizeof(py_list)}")
print(f"La taille de numpy_list: {sys.getsizeof(numpy_list)}")
print(f"numpy_list.nbytes: {numpy_list.nbytes}")
```

Ce code commence par l'import des deux bibliothèques numpy et sys. La bibliothèque sys est utilisée afin de calculer la taille en octets des objets en mémoire avec la fonction getsizeof. La liste py_list est créée en l'initialisant avec 10000 entiers et le tableau NumPy numpy_list a été créé et initialisé avec les éléments de la liste py_list. Puis ce code affiche tout simplement la taille mémoire occupée par la liste py_list et la taille mémoire occupée par le tableau numpy_list. La fonction getsizeof retourne la taille mémoire exacte occupée par des structures de données natives Python. Cependant, pour les structures de données autres que les données natives, cette fonction peut retourner des valeurs qui ne seront pas exactes. C'est pour cette raison que nous avons utilisé dans ce code la fonction nbytes afin d'avoir le cœur net par rapport à notre comparaison entre la taille mémoire occupée par nos deux objets py_list et numpy_list.

Le code précédent affiche les résultats suivants :

```
La taille de py_list: 80056
La taille de numpy_list: 40112
numpy_list.nbytes: 40000
```

Ces résultats montrent clairement que le tableau NumPy numpy_list occupe une taille mémoire proche de la moitié de la taille mémoire occupée par la liste py_list.

12.2 Comparaison des temps de calcul

Les deux exemples ci-après montrent la différence en termes de temps CPU nécessaire pour réaliser la somme des éléments d'une liste Python, dans le premier exemple, puis pour réaliser la même opération de somme mais avec un tableau NumPy, dans le deuxième exemple.

12.2.1 Temps de calcul sur une liste Python

Ce premier exemple permet de calculer le temps nécessaire pour le calcul de la somme de tous les éléments d'une liste Python. Pour le réaliser, suivez les étapes ci-dessous.

- Saisissez dans un nouveau fichier de type *.py* le code suivant puis exécutez-le :

```
import time
max_value = 300_000_000
list_1 =list(range(1,max_value+1))
start_time = time.time()
sum_list_1 = sum(list_1)
end_time = time.time()
print(f"Temps d'exécution : {end_time - start_time}")
print(f"La somme = : {sum_list_1:,}")
```

Ce code commence par importer la bibliothèque `time` afin de calculer le temps consommé par la fonction `sum`. Puis la liste Python `list_1` a été créée. Cette liste contient 300 millions de nombres entiers allant de 1 jusqu'à 300,000,000. La variable `start_time` permet de noter le temps juste avant l'exécution de la fonction `sum` puis la variable `end_time` permet de noter le temps juste après son exécution. Ce code se termine par l'affichage du temps consommé par la fonction `sum` ainsi que du résultat de la somme de tous les éléments de la liste `list_1`.

Remarque

Vous avez sans doute noté l'utilisation du caractère _ lors de la définition de la variable `max_value`. Cette notation est non seulement permise avec Python, mais en plus elle est recommandée lors de la manipulation de grands nombres afin de faciliter leur lecture. Remarquez également l'utilisation de la notation `{sum_list_1:,}` à la dernière instruction de ce programme afin d'indiquer notre souhait d'afficher le résultat associé à cette variable en utilisant le caractère "," afin de faciliter également la lecture des résultats.

Ce code donne le résultat suivant :

```
Temps d'exécution : 156.46044254302979
La somme = : 45,000,000,150,000,000
```

© Editions ENI - All rights reserved

Remarque

Attention, vous pouvez obtenir des résultats sensiblement différents de ceux affichés, car cela dépend de la puissance de la machine sur laquelle ce code est exécuté.

La section suivante donne le même exemple, mais en utilisant un tableau NumPy à la place d'une liste Python.

12.2.2 Temps de calcul sur un tableau NumPy

Pour réaliser le même calcul que celui de l'exemple précédent, mais en utilisant un tableau NumPy, suivez les étapes ci-après.

- Saisissez dans un nouveau fichier de type *.py* le code suivant puis exécutez-le :

```
import numpy as np
import time
max_value = 300_000_000
list_1 = np.array(np.arange(max_value+1), dtype='int64')
start_time = time.time()
sum_list_1 = np.sum(list_1)
end_time = time.time()
print(f"temps d'exécution : {end_time - start_time}")
print(f"la somme = : {sum_list_1:,}")
```

Le code ci-dessus est très similaire au code précédent. La seule différence est qu'au lieu d'utiliser une liste Python, ici la structure de données `list_1` est un tableau NumPy. L'exécution de ce nouveau code donne les résultats suivants :

```
Temps d'exécution : 0.2656247615814209 La somme = :
45,000,000,150,000,000
```

Ces résultats montrent que le temps nécessaire pour le calcul de la somme de tous les éléments d'un tableau NumPy est relativement négligeable par rapport au temps nécessaire pour réaliser la même opération avec une liste Python.

Remarque

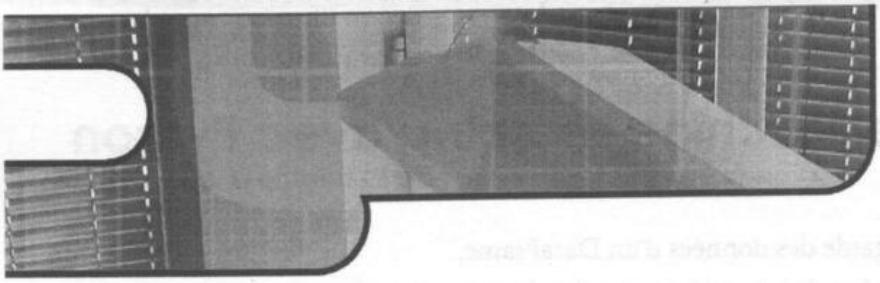
Dans les deux exemples précédents, nous n'avons calculé les temps d'exécution que de la fonction `sum`. Si vous calculez le temps nécessaire pour l'initialisation de la liste Python dans le premier exemple et le temps nécessaire pour l'initialisation du tableau NumPy dans le deuxième exemple, vous allez constater que l'initialisation de la liste Python prend un temps beaucoup plus important que le temps nécessaire pour initialiser le tableau NumPy.

Il faut noter aussi que la manière dont nous avons calculé le temps d'exécution n'est pas précise. Cependant, elle nous donne tout de même une bonne idée du temps CPU nécessaire pour l'exécution d'une portion d'un programme. Pour obtenir une idée plus précise, il vaut mieux utiliser des outils dédiés comme la bibliothèque `cprofile`. Ici, nous avons préféré l'utilisation de la bibliothèque `time` pour des raisons de simplification des codes.

13. Conclusion

Ce chapitre nous a permis de découvrir NumPy à travers un petit tour d'horizon des fonctions que cette bibliothèque propose. Nous avons essayé d'insister sur le fait que cette bibliothèque facilite grandement la manipulation des tableaux et les opérations qui leur sont associées. Nous avons également montré l'avantage en matière de temps CPU qu'on peut tirer via l'utilisation de cette bibliothèque. Le chapitre suivant va nous permettre de découvrir la bibliothèque Pandas, conçue sur la base de la bibliothèque NumPy et qui nous propose des fonctionnalités intéressantes pour la manipulation des jeux de données sous forme de DataFrame. Ce format de données en DataFrame est incontournable dans le domaine de la Data Science !

© Editions ENI - All rights reserved



Chapitre 2-3
La bibliothèque Pandas

1. Objectif du chapitre

Le traitement et l'analyse des données nécessitent d'organiser ces dernières dans un format plus au moins commun et qui correspond au format table. Également, avec la croissance incessante des sources de données, la taille des fichiers où ces données sont stockées devient de plus en plus conséquente. Avec la bibliothèque Pandas, le Data Scientist peut s'affranchir de toutes ces questions relatives au format et à la taille des données pour ne se concentrer que sur le problème à résoudre.

Ce chapitre est complètement dédié à l'exploration des possibilités que propose Pandas afin de permettre aux lecteurs de se familiariser avec les différentes opérations de base qui sont nécessaires pour toute utilisation de cette bibliothèque tant appréciée des Data Scientists et des Data Analysts.

À la fin de ce chapitre, le lecteur aura abordé :

- une introduction à la bibliothèque Pandas,
- une présentation des DataFrames Pandas,
- un chargement des données à partir de fichiers dans un DataFrame,
- la lecture et la modification des données d'un DataFrame,
- les tris et les requêtes sur les données d'un DataFrame.
- la modification de la structure d'un DataFrame,

- la sauvegarde des données d'un DataFrame,
- la réalisation des statistiques sur les données d'un DataFrame,
- un résumé des données d'un DataFrame,
- la lecture des fichiers de très grande taille avec Pandas.

2. C'est quoi, Pandas ?

Pandas est sans doute l'une des bibliothèques d'analyse de données les plus utilisées par les Data Scientists. Cette bibliothèque a été pensée dans l'objectif de rendre la manipulation des données le plus simple et le plus efficace possible, en particulier la manipulation des données sous forme de tables et de séries temporelles.

Avec Pandas, nous pouvons manipuler des données définies dans un espace à plusieurs dimensions. Les espaces définis selon une seule dimension sont adaptés pour la manipulation et l'analyse des séries temporelles. Lorsque les données sont définies dans un espace indexé selon deux dimensions, il s'agit de données organisées sous forme de table. Ces tables sont stockées dans des structures de données de type DataFrame.

Plusieurs logiciels, comme le fameux logiciel Excel, permettent de réaliser des traitements avancés sur les données. Cependant, la plupart de ces programmes présentent les deux lacunes suivantes :

1. Un manque de flexibilité. L'exploitation des données est restreinte aux usages permis par les fonctionnalités proposées par ces logiciels. Même si certains logiciels modernes proposent des possibilités d'extension de leurs fonctionnalités, ils restent tout de même limités soit par rapport à la nature de ces extensions possibles, soit par la plateforme sur laquelle ils ont été développés.
2. Taille des données traitées. La plupart des logiciels présentent des limitations quant à la taille des données qui peuvent être traitées simultanément. Souvent, lorsque ces logiciels chargent des fichiers de données de grande taille, cela provoque un ralentissement important de leur fonctionnement, voir causer l'arrêt brutal de leur exécution.

© Editions ENI - All rights reserved

Justement, avec Pandas, ces deux limitations peuvent être facilement dépassées. D'une part, Pandas propose nativement un certain nombre de fonctionnalités qui sont préprogrammées ; d'autre part, l'utilisation du langage Python nous donne la possibilité d'étendre ces fonctionnalités quasiment à tous les usages et multiplateforme. Également, avec Pandas, les données de grande taille ne présentent plus un problème puisque les données stockées dans des fichiers de grande taille peuvent facilement êtes lues.

Un autre avantage - qui n'est pas des moindres - présenté par la bibliothèque Pandas est l'organisation des données dans des DataFrames. Ces dernières structures présentent les données dans un format que tout Data Scientist aimerait avoir !

3. Installation de Pandas

Pour installer la bibliothèque Pandas, il suffit d'exécuter la commande suivante :

```
pip install pandas
```

Cette commande installe automatiquement la dernière version de la bibliothèque `pandas`.

Afin de nous assurer que le module `pandas` est correctement installé sur votre machine, nous allons tester un programme simple qui permet d'importer le module `pandas` et d'afficher la version qui est installée. Pour réaliser ce programme, suivez les étapes ci-dessous :

- Créez un nouveau fichier et donnez-lui le nom *test_pandas.py*.
- Saisissez les trois instructions ci-après dans ce fichier *test_pandas.py*.

```
import pandas as pd
print("La version pandas installée est :", pd.__version__)
print("Programme terminé")
```

- Lancez une console de commandes et exécutez le programme Python que vous avez défini dans le fichier *test_pandas.py*.

Si le module pandas est correctement installé, à l'exécution de votre programme, vous allez obtenir le résultat suivant :

```
La version pandas installée est : 1.0.1
Programme terminé
```

Remarque

Si le module pandas a bien été installé, mais que vous obtenez un message d'erreur lors de l'exécution de l'exemple ci-dessus, alors essayez de désinstaller le module pandas et d'installer une version antérieure à l'aide des deux commandes suivantes :

```
pip uninstall pandas
pip install pandas==1.0.1
```

Remarque

À ce stade, avec tous les exemples des deux derniers chapitres, vous savez créer un programme Python dans un fichier .py et vous savez exécuter ce programme pour visualiser ses résultats. Dans la suite de ce chapitre, nous n'allons donc pas indiquer à chaque exemple de code la nécessité de créer et d'exécuter nos programmes. Également, les codes Python des exemples de ce chapitre sont de très petite taille, c'est pour cette raison que nous n'avons pas fourni les codes sources associés. Pour tester ces exemples, il est donc nécessaire de les saisir par vous-même.

4. DataFrame Pandas

Un DataFrame Pandas est la structure de données utilisée pour le stockage et la manipulation de données en mémoire. En fonction des besoins, il existe plusieurs méthodes pour la création d'un DataFrame. Par exemple, un DataFrame peut être créé à partir d'un dictionnaire ou à partir d'un tableau NumPy, ou encore à partir de données stockées dans un fichier. Les deux sous-sections suivantes montrent comment créer un DataFrame à partir d'un dictionnaire, à partir d'un tableau NumPy, et aussi à partir d'un fichier.

Quelle que soit la méthode utilisée, une fois qu'un DataFrame est créé, ce dernier se comporte comme un dictionnaire Python où les clés sont associées aux colonnes et les valeurs sont des séries attachées à ces colonnes.

© Editions ENI - All rights reserved

4.1 Création d'un DataFrame à partir d'un dictionnaire

Pour créer un DataFrame à partir d'un dictionnaire, exécutez l'exemple de code suivant :

```
import pandas as pd
myDataset = pd.DataFrame({'Col1':[1,2,3],'Col2':[1.0, 2.0,
3.0],'Col3':['a','b','c']})
print(myDataset)

print("Le type des colonnes :\n", myDataset.dtypes)
```

Ce programme permet de créer un DataFrame avec les trois colonnes `Col1`, `Col2` et `Col3` et avec trois lignes de données. Ce programme devrait afficher le résultat suivant :

```
   Col1  Col2 Col3
0     1   1.0    a
1     2   2.0    b
2     3   3.0    c
Le type des colonnes :
Col1      int64
Col2    float64
Col3     object
dtype: object
```

Avec Pandas, le type chaîne de caractères `str` correspond au type `object`.

Pour changer le type d'une colonne, on peut utiliser la méthode `astype()` de Pandas. Par exemple, le code suivant permet de changer le type de la colonne `Col1` de type `int64` vers le type `float64` :

```
import pandas as pd
myDataset = pd.DataFrame({'Col1':[1,2,3],'Col2':[1.0, 2.0,
3.0],'Col3':['a','b','c']})
print(myDataset)
print("Le type des colonnes :\n", myDataset.dtypes)

myDataset['Col1'] = myDataset['Col1'].astype(float)

print("Le nouveau type des colonnes :\n", myDataset.dtypes)
```

Ce programme affiche le résultat suivant :

```
   Col1  Col2 Col3
0     1   1.0    a
1     2   2.0    b
2     3   3.0    c

Le type des colonnes :
Col1      int64
Col2    float64
Col3     object
dtype: object

Le nouveau type des colonnes :
Col1    float64
Col2    float64
Col3     object
dtype: object
```

Remarquez que les trois lignes de données de `myDataset` ont été par défaut indexées avec `0`, `1` et `2`. Dans cet exemple, les clés du dictionnaire ont été utilisées en tant que noms de colonnes. On peut indiquer que ces clés doivent être utilisées en tant qu'index comme dans l'exemple de code suivant :

```
import pandas as pd
myDataset = pd.DataFrame.from_dict({'Col1':[1,2,3],'Col2':[1.0,
2.0, 3.0],'Col3':['a','b','c']}, orient='index')
print(myDataset)
```

Dans ce nouveau code, nous avons utilisé la méthode `from_dict` pour pouvoir spécifier le paramètre `orient='index'` et ainsi considérer les clés du dictionnaire en tant qu'index et non pas en tant que noms des colonnes. Ce programme devrait afficher le résultat suivant :

```
      0  1  2
Col1  1  2  3
Col2  1  2  3
Col3  a  b  c
```

Cette fois, les valeurs `0`, `1` et `2` ont été utilisées pour nommer les colonnes du DataFrame `myDataset`.

© Editions ENI - All rights reserved

4.2 Création d'un DataFrame à partir d'un tableau NumPy

Le code suivant montre comment créer un DataFrame Pandas à partir d'un tableau NumPy :

```
import pandas as pd
import numpy as np
npArray= np.array([[1,2,3],[4,5,6],[7,8,9]])

myDataset = pd.DataFrame(npArray,
columns=['Col1','Col2','Col3'])

print(myDataset)
```

Ce programme crée le DataFrame `myDataset` à partir du tableau `npArray` et il affiche le résultat suivant :

```
   Col1  Col2  Col3
0     1     2     3
1     4     5     6
2     7     8     9
```

Lorsqu'un DataFrame est créé à partir d'un tableau NumPy, il faut faire attention au fait que toute modification de ce tableau NumPy se répercute sur le DataFrame ! Analysons le code suivant :

```
import pandas as pd
import numpy as np
npArray= np.array([[1,2,3],[4,5,6],[7,8,9]])
myDataset = pd.DataFrame(npArray,
columns=['Col1','Col2','Col3'])

print("Premier affichage de myDataset :")
print(myDataset)

npArray[0,0]=  99

print("Deuxième affichage de myDataset :")
print(myDataset)
```

Ce programme permet de créer le DataFrame `myDataset` à partir du tableau NumPy `npArray`, puis la première valeur du tableau `npArray`, qui était à `1`, a été modifiée à `99`.

Ce programme affiche le résultat suivant :

```
Premier affichage de myDataset :
   Col1  Col2  Col3
0     1     2     3
1     4     5     6
2     7     8     9

Deuxième affichage de myDataset :
   Col1  Col2  Col3
0    99     2     3
1     4     5     6
2     7     8     9
```

Ces résultats montrent que même si le tableau `npArray` a été modifié après la création du DataFrame `myDataset`, cette modification a été répercutée sur `myDataset`.

4.3 Chargement des données à partir de fichiers

Pour illustrer le chargement des données dans un DataFrame à partir d'un fichier, nous allons utiliser un exemple de jeu de données qui sera dupliqué et stocké dans deux fichiers distincts : un fichier *.csv* et un fichier *.txt*. L'objectif est de montrer comment charger les données à partir de ces deux types de fichiers qui représentent les formats de fichiers les plus fréquemment rencontrés dans la pratique.

Le jeu de données qui sera utilisé est produit par Météo France dans le but d'étudier la qualité de l'air. Parmi les mesures et les prévisions qui peuvent être mises en place avec ce jeu de données, on trouve la prévision de la concentration d'ozone pour un jour donné en se basant sur les données de la veille.

Ces données sont disponibles sur divers sites. Vous pouvez les télécharger avec le lien : http://www.math.univ-toulouse.fr/~besse/Wikistat/data/.

© Editions ENI - All rights reserved

Également, ces données ont été extraites et mises en forme par le service concerné de Météo France. Elles sont décrites par les variables suivantes :

- **JOUR** : type de jour ; férié (1) ou pas (0).
- **O3obs** : concentration d'ozone effectivement observée le lendemain à 17 h heure locale correspondant souvent au maximum de pollution observé.
- **MOCAGE** : prévision de cette pollution obtenue par un modèle déterministe de mécanique des fluides (équation de Navier et Stockes).
- **TEMPE** : température prévue par Météo France pour le lendemain 17 h.
- **RMH2O** : rapport d'humidité.
- **NO2** : concentration en dioxyde d'azote.
- **NO** : concentration en monoxyde d'azote.
- **STATION** : lieu de l'observation : Aix-en-Provence, Rambouillet, Munchhausen, Cadarache et Plan-de-Cuques.
- **VentMOD** : force du vent.
- **VentANG** : orientation du vent.

■Remarque

Les deux fichiers ozone.txt et ozone.csv, qui sont disponibles dans le dossier Data de ce chapitre, contiennent le même jeu de données défini ci-dessus.

4.3.1 Lecture des données d'un fichier CSV

Le petit programme suivant permet la lecture du contenu du fichier *ozone.csv* et stocke les résultats dans la variable `myDataset`. Testez ce programme pour visualiser les résultats qui seront équivalents à ceux de la figure 4-1 ci-après.

```
import pandas as pd
myDataset = pd.read_csv('Data\ozone.csv')
print(myDataset)
```

La première instruction `import pandas as pd` permet d'importer le module `pandas` afin que ses fonctionnalités soient visibles dans la suite du programme. Les fonctionnalités de `pandas` seront accessibles via le diminutif `pd`.

La fonction `pd.read_csv('Data\ozone.csv')` permet la lecture des données contenues dans le fichier *ozone.csv* et le résultat sera stocké dans la variable `myDataset`.

La dernière instruction `print(myDataset)` affiche à l'écran les cinq premières lignes et les cinq dernières lignes stockées dans le DataFrame `myDataset`. Également, en fin de l'affichage de ces résultats, le nombre total des lignes et des colonnes contenues dans ce DataFrame sera affiché.

```
      JOUR  O3obs  MOCAGE  TEMPE     RMH2O    NO2     NO STATION  VentMOD  VentANG
0        1     91    93.2   21.5  0.00847  1.602  0.424     Aix   9.5000 -0.64350
1        1    100   104.6   20.2  0.00881  2.121  0.531     Aix   8.0100 -0.04996
2        0     82   103.6   17.4  0.00951  1.657  0.467     Aix   9.3771 -0.12832
3        0     94    94.8   18.8  0.00855  2.350  0.701     Aix   9.4578 -0.34516
4        0    107    99.0   23.7  0.00731  1.653  0.452     Aix   7.8791 -0.41822
...    ...    ...     ...    ...      ...    ...    ...     ...      ...      ...
1036     0    116   233.6   22.1  0.01048  4.843  0.431     Pla   4.8052 -0.20964
1037     0     60   162.5   15.8  0.00789  3.166  0.411     Pla   2.7203 -0.62880
1038     0     74   198.4   16.2  0.00811  8.349  0.942     Pla   4.0311  0.12435
1039     0    121   191.7   16.4  0.00722  2.101  0.199     Pla   3.8000  0.00000
1040     1     93   221.2   16.7  0.00919  2.290  0.202     Pla   2.3022  0.04345

[1041 rows x 10 columns]
```

Figure 4-1 : Affichage du DataFrame `myDataset` avec la fonction `print`

4.3.2 Lecture d'un fichier texte

Souvent, lorsque les données d'une table sont stockées dans un fichier texte, par exemple un fichier de type *.txt*, les valeurs des variables sont séparées avec le caractère spécial `\t`, qui indique une tabulation. Le programme suivant permet de lire le contenu du fichier *ozone.txt*.

```
import pandas as pd
myDataset = pd.read_csv('Data\ozone.txt')
print(myDataset.head(3))
```

L'argument 3 utilisé lors de l'appel à la méthode `head` du DataFrame `myDataset` permet d'afficher uniquement les trois premières lignes.

À l'exécution de ce programme, vous devriez obtenir les résultats suivants :

```
  JOUR\tO3obs\tMOCAGE\tTEMPE\tRMH2O\tNO2\tNO\tSTATION\tVentMOD\tVentANG
0  1\t91\t93.2\t21.5\t0.00847\t1.602\t0.424\tAix\...
1  1\t100\t104.6\t20.2\t0.00881\t2.121\t0.531\tAi...
2  0\t82\t103.6\t17.4\t0.00951\t1.657\t0.467\tAix...
```

Figure 4-2 : Lecture d'un fichier texte dans un DataFrame Pandas

© Editions ENI - All rights reserved

Ces résultats indiquent que les données n'ont pas été correctement lues puisque dans la table `myDataset` il n'y a qu'une seule colonne, qui est affichée à la première ligne, puis chacune des autres lignes affichées correspond à une seule valeur. Cela n'est bien sûr pas le résultat souhaité. Contrairement aux fichiers de type CSV, pour obtenir les données dans le bon format lorsqu'elles sont lues à partir d'un fichier texte, il faut indiquer à la fonction `read_csv` le caractère qui est utilisé dans le fichier pour délimiter les valeurs les unes des autres ; ici, il s'agit du caractère `\t`. Pour corriger notre petit programme précédent, ajoutez le paramètre `delimiter="\t"` à l'appel de la fonction `read_csv()` comme suit :

```
myDataset = pd.read_csv('Data\ozone.txt', delimiter="\t")
```

À l'exécution de ce programme, vous devriez obtenir exactement les mêmes résultats que ceux de la figure 4-3 ci-dessous :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
0	1	91	93.2	21.5	0.00847	1.602	0.424	Aix	9.5000	-0.64350
1	1	100	104.6	20.2	0.00881	2.121	0.531	Aix	8.0100	-0.04996
2	0	82	103.6	17.4	0.00951	1.657	0.467	Aix	9.3771	-0.12832

Figure 4-3 : Lecture du fichier ozone.txt avec le délimiteur `\t`

5. Accès aux données d'un DataFrame

Avec les DataFrames de la bibliothèque Pandas, une fois que les données sont lues à partir d'un fichier ou de toute autre source de données, la manipulation de ces données peut être réalisée de façon très simple et intuitive. Cette section donne tous les éléments nécessaires pour accéder, filtrer et modifier les données d'un DataFrame.

5.1 Lire les lignes d'un DataFrame

5.1.1 Lire une ligne d'un DataFrame

Une fois que des données sont lues et stockées dans un DataFrame, pour accéder à une ligne précise de ce DataFrame il suffit d'utiliser sa propriété `iloc` en indiquant l'indice ou la clé à laquelle cette ligne est associée, comme dans l'exemple de code suivant où la ligne associée à la clé 5 a été lue et affichée.

```
import pandas as pd
myDataset = pd.read_csv('Data\ozone.csv')
print(myDataset.iloc[5])
```

Ce programme affiche le résultat suivant :

```
JOUR            0
O3obs         150
MOCAGE      114.3
TEMPE        23.6
RMH2O     0.01182
NO2         5.316
NO          1.343
STATION       Aix
VentMOD    6.3127
VentANG    0.0634
Name: 5, dtype: object
```

Figure 4-4 : Accès et affichage de la ligne d'indice 5

5.1.2 Lire plusieurs lignes d'un DataFrame

Pour accéder à un ensemble de lignes situées entre deux indices, il suffit d'utiliser la notation que nous avons l'habitude d'utiliser pour accéder aux éléments d'une liste Python après avoir indiqué la propriété `iloc` d'un DataFrame, comme dans l'exemple de code suivant :

```
import pandas as pd
myDataset = pd.read_csv('Data\ozone.csv')
print(myDataset.iloc[5:10])
```

© Editions ENI - All rights reserved

Ce programme affiche les lignes du DataFrame `myDataset` situées entre l'indice `5` et l'indice `10` exclu, comme le montre le résultat dans la figure 4-5 suivante :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
5	0	150	114.3	23.6	0.01182	5.316	1.343	Aix	6.3127	0.06341
6	0	164	127.7	26.6	0.00937	2.841	0.550	Aix	4.8042	0.04164
7	1	135	164.3	23.5	0.01087	15.733	2.911	Aix	4.8795	0.79989
8	1	121	144.1	23.3	0.01168	8.807	1.867	Aix	5.1088	0.70226
9	0	129	112.8	23.7	0.01091	18.911	4.700	Aix	5.4406	0.94200

Figure 4-5 : Accès et affichage des lignes situées dans l'intervalle `[5:10]`

5.1.3 Parcourir les lignes d'un DataFrame

L'ensemble des lignes d'un DataFrame peuvent être parcourues en utilisant une boucle. L'exemple de code suivant permet de parcourir toutes les lignes du DataFrame `myDataset` à l'aide d'une boucle `for`, et à chaque itération de cette boucle, l'indice et les valeurs d'une ligne seront affichés.

```
import pandas as pd
myDataset = pd.read_csv('Data\ozone.csv')
for index, row in myDataset.iterrows():
    print("La valeur d'indice [{}] : \n{}".format(index, row))
```

La figure ci-dessous affiche le dernier résultat de ce programme correspondant à la dernière ligne du DataFrame `myDataset` et qui est située à l'indice `1040`.

```
La valeur d'indice [1040] :
JOUR            1
O3obs          93
MOCAGE      221.2
TEMPE        16.7
RMH2O     0.00919
NO2          2.29
NO          8.202
STATION       Pla
VentMOD    2.3022
VentANG   0.04345
Name: 1040, dtype: object
```

Figure 4-6 : Affichage de la ligne de `myDataset` située à l'indice `1040`

5.1.4 Filtrer les lignes avec une condition

Nous pouvons également accéder aux lignes d'un DataFrame en utilisant des conditions, c'est-à-dire ne récupérer que certaines lignes qui respectent une ou plusieurs conditions. L'exemple de code ci-dessous permet de constituer le DataFrame `dayoff` à partir des lignes du DataFrame `myDataset` pour lesquelles le champ `JOUR` est égal à `1`. En d'autres termes, le DataFrame `dayoff` représente les données associées aux jours fériés de `myDataset`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
dayOff = myDataset.loc[myDataset["JOUR"]==1]
print(dayOff.head(5))
```

Remarquez que cette fois, pour accéder aux lignes du DataFrame `myDataset` via la condition `myDataset["JOUR"]==1`, nous avons utilisé la propriété `loc` et non pas la propriété `iloc` comme précédemment lorsque nous avions accédé aux lignes du DataFrame `myDataset` via des indices.

En petite synthèse, l'accès aux données d'un DataFrame Pandas se réalise principalement en utilisant les propriétés `loc` et `iloc`, la première propriété utilisant les indices des lignes et la deuxième utilisant des conditions sur le contenu des lignes.

Ce programme affiche le résultat suivant :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
0	1	91	93.2	21.5	0.00847	1.602	0.424	Aix	9.5000	-0.64350
1	1	100	104.6	20.2	0.00881	2.121	0.531	Aix	8.0100	-0.04996
7	1	135	164.3	21.5	0.01087	15.733	2.911	Aix	4.8795	0.79089
8	1	121	144.1	23.3	0.01168	8.887	1.867	Aix	5.1088	0.78276
13	1	83	158.9	15.4	0.01456	10.254	1.839	Aix	2.5942	0.48089

Figure 4-7 : Affichage des cinq premières lignes du DataFrame dayoff

5.1.5 Filtrer les lignes avec plusieurs conditions

La propriété `loc` permet d'indiquer plusieurs conditions pour le filtrage des données d'un DataFrame. Ainsi, l'exemple de code suivant montre comment filtrer les données du DataFrame `myDataset` avec les deux conditions `myDataset["JOUR"]==1` et `myDataset["O3obs"]>=100` pour récupérer uniquement les jours fériés, correspondant aux lignes avec le champ `JOUR` égal à `1`, et pour lesquels la valeur du champ `O3obs` est supérieure ou égale à `100`.

© Editions ENI - All rights reserved

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
result = myDataset.loc[
                        (myDataset["JOUR"]==1) &
                        (myDataset["O3obs"]>=100)
                        ]
print(result.head(5))
```

Ce programme affiche le résultat suivant :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
1	1	100	104.6	20.2	0.00881	2.121	0.531	Aix	8.0100	-0.04996
7	1	135	164.3	23.5	0.01087	15.733	2.911	Aix	4.8795	0.79989
8	1	121	144.1	23.3	0.01168	8.807	1.867	Aix	5.1088	0.70226
20	1	118	113.0	26.2	0.01064	4.615	0.995	Aix	6.2769	0.53496
21	1	130	136.3	27.2	0.00912	4.276	0.827	Aix	6.0033	0.52328

Figure 4-8 : Affichage des jours fériés pour lesquels la valeur du champ `O3obs` est supérieure ou égale à `100`

5.1.6 Filtrage avec des critères textuels

Avec la propriété `loc`, nous pouvons exprimer des conditions basées sur une recherche textuelle. Le code suivant reprend l'exemple précédent en ajoutant une condition textuelle. Ainsi, le DataFrame `result` va contenir les lignes de `myDataset` correspondant aux jours fériés dont la valeur du champ `O3obs` est supérieure ou égale à `100` et pour lesquels le champ `STATION` est égal soit à la valeur `Ram`, soit à la valeur `Cad`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
result = myDataset.loc[
                        (myDataset["JOUR"]==1) &
                        (myDataset["O3obs"]>=100) &
                        ((myDataset["STATION"] == "Ram") |
                         (myDataset["STATION"] == "Cad"))
                        ]
print(result)
```

Ce programme affiche le résultat suivant :

```
     JOUR  O3obs  MOCAGE  TEMPE     RMH2O     NO2     NO STATION  VentMOD  VentANG
422     1    103   100.5   20.7  0.00748   0.981  0.233     Cad  9.43660 -0.17038
428     1    203   182.0   25.4  0.01275   5.952  0.812     Cad  6.10740  0.69264
429     1    208   188.0   25.3  0.01184   7.850  1.130     Cad  6.79560  0.57576
446     1    106   138.0   34.1  0.01192   1.112  0.159     Cad  7.55050 -0.19999
447     1    114   157.8   32.9  0.01569   3.481  0.454     Cad  7.18470  0.60731
...   ...    ...     ...    ...      ...     ...    ...     ...      ...      ...
763     1    191   200.4   27.8  0.00856   3.482  0.367     Ram  0.30000 -1.57079
765     1    104   128.3   29.6  0.00776   2.831  0.472     Ram  3.10644 -0.99172
766     1    167   168.4   29.0  0.00648  12.671  1.801     Ram  2.60000  0.39479
804     1    110   140.7   23.9  0.00608  11.722  1.807     Ram  1.65529  1.13417
810     1    128   122.4   28.3  0.00878   6.638  1.015     Ram  4.60110  0.02174

[69 rows x 10 columns]
```

Figure 4-9 : Affichage des jours fériés pour lesquels la valeur du champ `O3obs` est supérieure ou égale à `100` et `STATION` est égal à `Cad` ou `Ram`

5.1.7 Réinitialiser les index

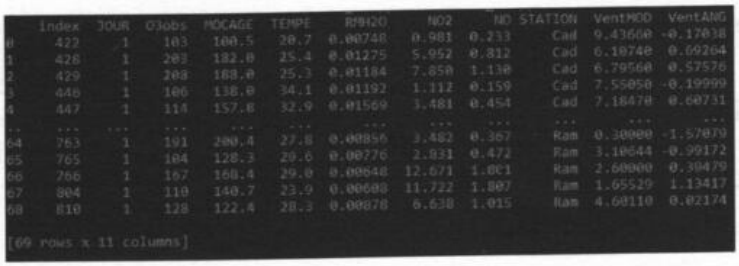
Si nous observons de plus près le résultat de la figure 4-9 ci-dessus, on constate que les index associés aux lignes du DataFrame `result` ne correspondent pas aux numéros des lignes ! En effet, les index de ce DataFrame correspondent aux numéros de ses lignes dans le DataFrame d'origine et qui est le DataFrame `myDataset`. Par exemple, la première ligne de `result` est la 422e ligne de `myDataset` et, de même, la deuxième ligne de `result` est la 428e ligne de `myDataset`. Dans la pratique, nous avons souvent besoin que les index d'un DataFrame construit à la suite d'un filtrage correspondent aux numéros des lignes de ce DataFrame et non pas aux numéros des lignes dans le DataFrame d'origine. Dans ce cas-là, on peut utiliser la méthode `reset_index` qui permet d'une part de mettre en place un nouvel index correspondant aux numéros des lignes dans un DataFrame, et d'autre part de créer un nouveau champ dans lequel seront stockés les index d'origine.

Ajoutez l'instruction suivante dans le code précédent juste avant l'appel à la fonction `print` puis exécutez ce programme.

```
result = result.reset_index()
```

© Editions ENI - All rights reserved

Ce programme affiche le résultat suivant :



	index	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
0	422	1	103	100.5	20.7	0.00748	0.981	0.233	Cad	9.43600	-0.17038
1	428	1	203	182.0	25.4	0.01275	5.952	0.812	Cad	6.10740	0.69264
2	429	1	208	188.0	25.3	0.01184	7.850	1.130	Cad	6.79560	0.57576
3	440	1	106	138.0	34.1	0.01192	1.112	0.159	Cad	7.55050	-0.19999
4	447	1	114	157.8	32.9	0.01569	3.481	0.454	Cad	7.18470	0.60731
..	...	...	...	...	...	...	...	...	...	...	...
64	763	1	191	200.4	27.8	0.00856	3.482	0.367	Ram	0.30000	-1.57079
65	765	1	104	128.3	29.6	0.00776	2.831	0.472	Ram	3.10644	-0.99172
66	766	1	167	168.4	29.0	0.00648	12.671	1.801	Ram	2.60000	0.39479
67	804	1	110	140.7	23.9	0.00608	11.722	1.807	Ram	1.65529	1.13417
68	810	1	128	122.4	28.3	0.00878	6.638	1.015	Ram	4.60110	0.02174

[69 rows x 11 columns]

Figure 4-10 : Réinitialisation des index à la suite d'un filtrage de données avec la création de la colonne `index`

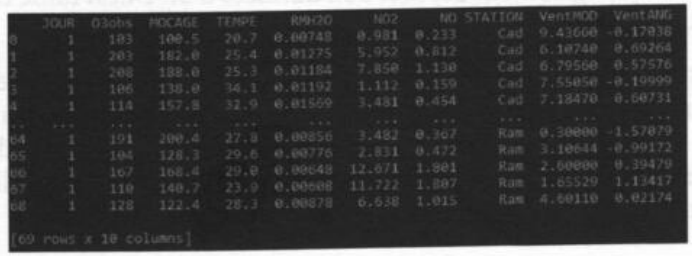
En comparant le résultat de la figure 4-9 à celui de la figure 4-10, on constate que les index dans la table `result` de la figure 4-9 correspondent bien aux numéros des lignes dans cette même table et qu'un nouveau champ a été créé avec le nom `index` pour stocker les valeurs de l'index d'origine.

Le nouveau champ `index` est utile lorsque nous souhaitons garder la trace des numéros des lignes dans leur table d'origine avant le filtrage. Cependant, si nous n'avons pas besoin de garder ces informations, alors nous pouvons utiliser l'argument `drop=True` lors de l'appel à la méthode `rest_index`.

Remplacez l'appel à la méthode `rest_index` dans le code précédent avec l'instruction suivante :

```
result = result.reset_index(drop=True)
```

Ce programme va afficher le résultat suivant :



	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
0	1	103	100.5	20.7	0.00748	0.981	0.233	Cad	9.43600	-0.17038
1	1	203	182.0	25.4	0.01275	5.952	0.812	Cad	6.10740	0.69264
2	1	208	188.0	25.3	0.01184	7.850	1.130	Cad	6.79560	0.57576
3	1	106	138.0	34.1	0.01192	1.112	0.159	Cad	7.55050	-0.19999
4	1	114	157.8	32.9	0.01569	3.481	0.454	Cad	7.18470	0.60731
..	...	...	...	...	...	...	...	...	...	...
64	1	191	200.4	27.8	0.00856	3.482	0.367	Ram	0.30000	-1.57079
65	1	104	128.3	29.6	0.00776	2.831	0.472	Ram	3.10644	-0.99172
66	1	167	168.4	29.0	0.00648	12.671	1.801	Ram	2.60000	0.39479
67	1	110	140.7	23.9	0.00608	11.722	1.807	Ram	1.65529	1.13417
68	1	128	122.4	28.3	0.00878	6.638	1.015	Ram	4.60110	0.02174

[69 rows x 10 columns]

Figure 4-11 : Réinitialisation des index à la suite d'un filtrage de données sans la création de la colonne `index`

Cette fois, les index ont été réinitialisés sans le champ `index` !

5.1.8 Filtrer avec les valeurs uniques

Lorsque nous manipulons un DataFrame, nous avons souvent besoin de connaître les différentes valeurs uniques présentes dans une colonne donnée. Par exemple, le code suivant permet d'afficher toutes les valeurs uniques de la colonne `STATION` :

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
print("Les valeurs uniques du champ STATION de myDataset:
",myDataset['STATION'].unique())
```

Le résultat est le suivant :

```
Les valeurs uniques du champ STATION de myDataset:  ['Aix' 'Als' 'Cad' 'Ram' 'Pla']
```

Figure 4-12 : Affichage des valeurs uniques de la colonne `STATION` de `myDataset`

Ce résultat montre que dans `myDataset`, l'ensemble des valeurs possibles pour la variable `STATION` est l'ensemble `{'Aix', 'Als', 'Cad', 'Ram', 'Pla'}`.

5.1.9 Filtrer avec une expression régulière

Dans les exemples précédents, nous avons vu que l'accès aux données d'un DataFrame peut être réalisé en utilisant la propriété `iloc` en indiquant des indices ou en utilisant la propriété `loc` en indiquant des filtres sous forme de conditions. Nous pouvons également accéder aux données d'un DataFrame en utilisant des expressions régulières comme filtres avec la propriété `loc` !

L'exemple de code suivant affiche les valeurs uniques du champ `STATION` des deux DataFrames `myDataset` et `result`. Le DataFrame `result` est un sous-ensemble du DataFrame `myDataset` obtenu après l'application de l'expression régulière `".a."` sur le champ `STATION` de `myDataset` en utilisant la méthode `str.contains`.

© Editions ENI - All rights reserved

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')

print("Les valeurs uniques du champ STATION de myDataset:
      ",myDataset['STATION'].unique())

result = myDataset.loc[myDataset["STATION"].str.contains(".a.",
                                                    regex=True)]

print("Les valeurs uniques du champ STATION de result:",
      result['STATION'].unique())
```

Ce programme affiche le résultat suivant :

```
Les valeurs uniques du champ STATION de myDataset: ['Aix' 'Als' 'Cad' 'Ram' 'Pla']
Les valeurs uniques du champ STATION de result: ['Cad' 'Ram']
```

Figure 4-13 : Affichage des valeurs uniques de la colonne `STATION` de `myDataset` et de `result`

Ce résultat montre que le DataFrame `result` contient uniquement les lignes de `myDataset` pour lesquelles le champ `STATION` contient une sous-chaîne de caractères correspondant à l'expression régulière `".a."`.

5.2 Accéder aux variables d'un DataFrame

5.2.1 Liste des variables d'un DataFrame

L'attribut `columns` d'un DataFrame contient la liste des noms de toutes ses colonnes. Ainsi, le programme suivant affiche la liste des noms de toutes les colonnes du DataFrame `myDataset` :

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
variables = myDataset.columns
print(variables)
```

Ce programme affiche les noms des colonnes sous forme d'index comme suit :

```
Index(['JOUR', 'O3obs', 'MOCAGE', 'TEMPE', 'RMH2O', 'NO2', 'NO',
'STATION', 'VentMOD', 'VentANG'],
      dtype='object')
```

5.2.2 Accès aux valeurs d'une colonne

Pour accéder aux valeurs d'une colonne d'un DataFrame, il suffit d'indiquer le nom de cette colonne entre crochets comme dans l'exemple de code suivant qui permet d'afficher les cinq premières valeurs de la colonne `VentMOD`.

```
import pandas as pd
myDataset = pd.read_csv('Data\ozone.csv')
VentMOD = myDataset["VentMOD"]
print(VentMOD.head(5))
```

Ce programme affiche le résultat suivant :

```
0    9.5000
1    8.0100
2    9.3771
3    9.4578
4    7.8791
Name: VentMOD, dtype: float64
```

Figure 4-14 : Affichage des cinq premières valeurs de la variable `VentMOD`

5.2.3 Accès à plusieurs colonnes

Pour accéder aux valeurs de plusieurs colonnes d'un DataFrame, il suffit d'indiquer le nom des colonnes concernées dans une liste comme dans l'exemple de code suivant qui permet d'afficher les cinq premières valeurs des deux variables `JOUR` et `VentMOD`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
data = myDataset[["JOUR","VentMOD"]]
print(data[0:5])
```

© Editions ENI - All rights reserved

Ce programme affiche le résultat suivant :

```
   JOUR  VentMOD
0     1   9.5000
1     1   8.0100
2     0   9.3771
3     0   9.4578
4     0   7.8791
```

Figure 4-15 : Affichage des cinq premières valeurs des deux variables `JOUR` et `VentMOD`

5.3 Lire une cellule spécifique avec les index

Dans les sections précédentes, nous avons vu comment lire un ensemble de lignes d'un DataFrame avec des critères de restriction relatifs aux champs à inclure dans les résultats ou relatifs aux valeurs de certaines variables. On peut également accéder à une cellule bien précise d'un DataFrame en précisant son numéro de ligne et son numéro de colonne. L'exemple de code suivant permet de lire la valeur située à la ligne numéro 8, c'est-à-dire la neuvième ligne, et la colonne numéro 2, c'est-à-dire la troisième colonne.

```
import pandas as pd
myDataset = pd.read_csv('Data\ozone.csv')
print(myDataset.iloc[8,2])
```

Ce programme affiche la valeur `144.1` (voir la figure 4-5 plus haut).

6. Modifier un DataFrame

6.1 Modifier les valeurs dans un DataFrame

Une fois qu'un DataFrame a été créé et initialisé avec des valeurs, nous pouvons modifier ses valeurs en réalisant des affectations des nouvelles valeurs directement aux valeurs qu'on souhaite modifier. Par exemple, le code suivant montre comment modifier la valeur du DataFrame `myDataset` située à la troisième ligne et la troisième colonne en lui affectant la valeur `150`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
print(myDataset.head(3))
myDataset.iloc[2,2]= 150
print("\n",myDataset.head(3))
```

Ce programme affiche la valeur située à la troisième ligne et la troisième colonne avant et après lui avoir affecté la valeur `150`, comme visible dans le résultat suivant :

```
   JOUR  O3obs  MOCAGE  TEMPE    RMH2O   NO2     NO STATION  VentMOD  VentANG
0     1     91    93.2   21.5  0.00847  1.602  0.424     Aix   9.5000 -0.64350
1     1    100   104.6   20.2  0.00881  2.121  0.531     Aix   8.0100 -0.04996
2     0     82   103.6   17.4  0.00951  1.657  0.467     Aix   9.3771 -0.12832

   JOUR  O3obs  MOCAGE  TEMPE    RMH2O   NO2     NO STATION  VentMOD  VentANG
0     1     91    93.2   21.5  0.00847  1.602  0.424     Aix   9.5000 -0.64350
1     1    100   104.6   20.2  0.00881  2.121  0.531     Aix   8.0100 -0.04996
2     0     82   150.0   17.4  0.00951  1.657  0.467     Aix   9.3771 -0.12832
```

Figure 4-16 : Modification de la valeur `myDataset[2,2]`

6.2 Modifier la structure d'un DataFrame

6.2.1 Ajouter une variable à un DataFrame

Pour ajouter une nouvelle colonne à un DataFrame, il suffit d'indiquer les valeurs de cette nouvelle colonne pour toutes les lignes déjà existantes dans ce DataFrame.

L'exemple du code suivant ajoute la nouvelle colonne `Résultat` au DataFrame `myDataset` en affectant la valeur `False` à toutes les lignes déjà existantes pour cette nouvelle variable.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
myDataset["Résultat"] = False
print(myDataset)
```

© Editions ENI - All rights reserved

Ce programme affiche le résultat suivant :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG	Résultat
0	1	91	93.2	21.5	0.00847	1.602	0.424	Aix	9.5000	-0.64350	False
1	1	100	104.6	20.2	0.00881	2.121	0.531	Aix	8.0100	-0.04996	False
2	0	82	103.6	17.4	0.00951	1.657	0.467	Aix	9.3771	-0.12832	False
3	0	94	94.8	18.8	0.00855	2.350	0.701	Aix	9.4578	-0.34516	False
4	0	107	99.0	23.7	0.00731	1.653	0.452	Aix	7.8791	-0.41822	False
...	...	...	...	...	...	...	...	...	...	...	...
1036	0	116	233.6	22.1	0.01048	4.843	0.431	Pla	4.8052	-0.20964	False
1037	0	60	162.5	15.8	0.00789	3.166	0.411	Pla	2.7203	-0.62880	False
1038	0	74	198.4	16.2	0.00811	8.349	0.942	Pla	4.8311	0.12435	False
1039	0	121	191.7	16.4	0.00722	2.101	0.199	Pla	3.8000	0.00000	False
1040	1	93	221.2	16.7	0.00919	2.290	0.202	Pla	2.3022	0.04345	False

[1041 rows x 11 columns]

Figure 4-17 : Modification de la valeur `myDataset[2,2]`

L'exemple du code suivant crée la nouvelle colonne `Pow2_O3obs` à partir du carré de la colonne `O3obs`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
myDataset["Pow2_O3obs"] = pow(myDataset["O3obs"],2)
print(myDataset.head(5))
```

Le résultat est le suivant :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG	Pow2_O3obs
0	1	91	93.2	21.5	0.00847	1.602	0.424	Aix	9.5000	-0.64350	8281
1	1	100	104.6	20.2	0.00881	2.121	0.531	Aix	8.0100	-0.04996	10000
2	0	82	103.6	17.4	0.00951	1.657	0.467	Aix	9.3771	-0.12832	6724
3	0	94	94.8	18.8	0.00855	2.350	0.701	Aix	9.4578	-0.34516	8836
4	0	107	99.0	23.7	0.00731	1.653	0.452	Aix	7.8791	-0.41822	11449

Figure 4-18 : Création de la nouvelle colonne `Pow2_O3obs`

Le fait de lever au carré une variable permet d'accentuer les différences entre ses valeurs. Par exemple, la différence en valeur absolue entre la valeur de la variable `O3obs` de la première ligne et la valeur de cette même variable à la deuxième ligne est égale à `|91-100|=9`, alors que la différence entre les deux valeurs correspondantes de la variable `Pow2_O3obs` est égale à `8281-10000|=1719`.

On peut également créer une nouvelle colonne à partir de la somme sur d'autres variables comme dans l'exemple de code suivant :

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
myDataset["Total"] = myDataset.iloc[:,5:7].sum(axis=1)
print(myDataset.head(5))
```

Ce code permet de créer la nouvelle colonne `Total` qui est égale à la somme des valeurs des variables situées entre l'index 5 inclus et l'index 7 exclu, c'est-à-dire que la colonne `Total` sera la somme des deux variables `NO2` et `NO`.

Le résultat est le suivant :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG	Total
0	1	91	93.2	21.5	0.00847	1.602	0.424	Aix	9.5000	-0.64350	2.026
1	1	100	104.6	20.2	0.00881	2.121	0.531	Aix	8.0100	-0.04996	2.652
2	0	82	103.6	17.4	0.00951	1.657	0.467	Aix	9.3771	-0.12832	2.124
3	0	94	94.8	18.8	0.00855	2.350	0.701	Aix	9.4578	-0.34516	3.051
4	0	107	99.0	23.7	0.00731	1.653	0.452	Aix	7.8791	-0.41822	2.105

Figure 4-19 : Création de la nouvelle colonne `Total` à partir de la somme des deux variables `NO2` et `NO`

Dans la pratique, lorsque nous réalisons des accès et des calculs sur des colonnes d'un DataFrame, il est fortement recommandé de ne pas utiliser les index des variables concernées comme dans l'exemple précédent. Quand cela est possible, préférez l'utilisation des noms des variables manipulées en utilisant la propriété `loc` à la place de la propriété `iloc`. Par exemple, notre dernier code serait plus propre si nous remplacions l'instruction utilisant `iloc` par une instruction avec `loc` comme suit :

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')

myDataset["Total"] = myDataset.loc[:,["NO2","NO"]].sum(axis=1)

print(myDataset.head(5))
```

Ce programme affiche exactement les mêmes résultats que le précédent, mais cette fois nous avons utilisé les noms des deux variables `NO2` et `NO` au lieu de leurs index.

© Editions ENI - All rights reserved

Remarque

L'utilisation des index de variables au lieu de leurs noms peut être une source d'erreurs, car il suffit que l'ordre des colonnes change pour qu'un programme ne se comporte pas comme attendu.

6.2.2 Réordonner les variables d'un DataFrame

Pour réordonner les colonnes d'un DataFrame, il suffit d'indiquer l'ordre souhaité entre crochets.

Dans l'exemple de code précédent, nous avions créé la nouvelle colonne `Total` à partir de la somme des deux colonnes NO2 et NO. Par défaut, cette nouvelle colonne `Total` est placée tout à droite du DataFrame, c'est-à-dire que la colonne `Total` est la dernière colonne. Bien sûr, il serait plus pratique que cette colonne `Total` soit positionnée à la suite de la colonne NO, et c'est ce que réalise le code suivant :

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
myDataset["Total"] = myDataset.iloc[:,5:7].sum(axis=1)
print(myDataset.head(5),"\n\n")

columns = list(myDataset.columns.values)

myDataset = myDataset[columns[0:7]+[columns[-1]]+columns[7:10]]

print(myDataset.head(5))
```

Ce programme ajoute la nouvelle colonne `Total` comme précédemment à partir de la somme des deux colonnes NO2 et NO, puis repositionne cette nouvelle colonne juste après la colonne d'index 6 et qui correspond à la colonne NO.

Le résultat est le suivant :

```
   JOUR  O3obs  MOCAGE  TEMPE     RMH2O    NO2     NO STATION  VentMOD  VentANG  Total
0     1     91    93.2   21.5  0.00847  1.602  0.424     Aix   9.5000 -0.64350  2.026
1     1    100   104.6   20.2  0.00881  2.121  0.531     Aix   8.0100 -0.04996  2.652
2     0     82   103.6   17.4  0.00951  1.657  0.467     Aix   9.3771 -0.12832  2.124
3     0     94    94.8   18.8  0.00855  2.350  0.701     Aix   9.4578 -0.34516  3.051
4     0    107    99.0   23.7  0.00731  1.653  0.452     Aix   7.8791 -0.41822  2.105

   JOUR  O3obs  MOCAGE  TEMPE     RMH2O    NO2     NO  Total STATION  VentMOD  VentANG
0     1     91    93.2   21.5  0.00847  1.602  0.424  2.026     Aix   9.5000 -0.64350
1     1    100   104.6   20.2  0.00881  2.121  0.531  2.652     Aix   8.0100 -0.04996
2     0     82   103.6   17.4  0.00951  1.657  0.467  2.124     Aix   9.3771 -0.12832
3     0     94    94.8   18.8  0.00855  2.350  0.701  3.051     Aix   9.4578 -0.34516
4     0    107    99.0   23.7  0.00731  1.653  0.452  2.105     Aix   7.8791 -0.41822
```

Figure 4-20 : Repositionnement de la colonne `Total`

6.2.3 Supprimer une variable d'un DataFrame

Pour supprimer une colonne, il suffit d'appeler la méthode `drop` d'un objet de type DataFrame en lui indiquant le nom de la colonne à supprimer.

L'exemple suivant reprend le code précédent pour créer la nouvelle colonne `Pow2_O3obs`, puis la méthode `drop` est appelée afin de supprimer la nouvelle colonne qui vient d'être créée.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
print("(Création de la nouvelle colonne Pow2_O3obs")
myDataset["Pow2_O3obs"] = pow(myDataset["O3obs"],2)
print(myDataset.head(3),"\n\n")
print("Suppression de la colonne Pow2_O3obs")
myDataset = myDataset.drop(columns=["Pow2_O3obs"])
print(myDataset.head(3))
```

© Editions ENI - All rights reserved

Ce code affiche le résultat suivant :

```
Création de la nouvelle colonne Pow2_O3obs
   JOUR  O3obs  MOCAGE  TEMPE     RMH2O    NO2    NO STATION  VentMOD  VentANG  Pow2_O3obs
0     1     91    93.2   21.5  0.00847  1.602  0.424     Aix   9.5000 -0.64350        8281
1     1    100   104.6   20.2  0.00881  2.121  0.531     Aix   8.0100 -0.04996       10000
2     0     82   103.6   17.4  0.00951  1.657  0.467     Aix   9.3771 -0.12832        6724


Suppression de la colonne Pow2_O3obs
   JOUR  O3obs  MOCAGE  TEMPE     RMH2O    NO2    NO STATION  VentMOD  VentANG
0     1     91    93.2   21.5  0.00847  1.602  0.424     Aix   9.5000 -0.64350
1     1    100   104.6   20.2  0.00881  2.121  0.531     Aix   8.0100 -0.04996
2     0     82   103.6   17.4  0.00951  1.657  0.467     Aix   9.3771 -0.12832
```

Figure 4-21 : Création puis suppression de la colonne `Pow2_O3obs`

Ce résultat montre que la colonne `Pow2_O3obs` n'existe plus dans le DataFrame `myDataset` après avoir appelé la méthode `drop`.

Remarquez que le paramètre de la méthode `drop` est une liste associée à l'argument `columns`. Cette liste permet de supprimer plusieurs colonnes en indiquant leurs noms séparés par des virgules.

6.2.4 Utiliser la méthode melt pour diminuer le nombre de variables

Dans certains datasets, nous disposons d'un nombre important de colonnes. Souvent, nous avons besoin de diminuer le nombre de colonnes d'un jeu de données ou bien nous avons besoin d'utiliser les noms de certaines colonnes en tant que valeurs et non pas en tant que variables.

La méthode `melt` permet de transformer un ensemble de colonnes en valeurs. Pour bien illustrer son utilisation, nous allons considérer un jeu de données autre que celui du fichier *ozone.csv* utilisé précédemment. Nous allons utiliser les données du fichier *ecoles-creches-idf.csv*. Ce fichier contient les données relatives aux adresses postales et aux coordonnées sous forme de latitude et longitude, ainsi que les données relatives à la pollution atmosphérique en matière des polluants NO2, PM10 et PM25 autour des crèches et des écoles de la région Île-de-France. Les valeurs de ces trois polluants ont été enregistrées sous forme d'une moyenne par année allant de l'année 2012 jusqu'à l'année 2017. Par exemple, dans ce fichier, les variables `PM10_2012`, `PM10_2013`, `PM10_2014`, `PM10_2015`, `PM10_2016`, `PM10_2017` enregistrent la concentration du polluant PM10 respectivement de 2012 à 2017.

Remarque

Cette base de données a été constituée par Datactivist pour le compte de l'association Respire afin d'alerter sur les niveaux d'exposition des populations fragiles à la pollution aérienne. Elle sert de base à la carte "De l'air pour nos enfants !" mise en ligne par Respire le 27 mars 2019.

L'exemple de code suivant montre comment transformer les variables PM10_2012, PM10_2013, PM10_2014, PM10_2015, PM10_2016, PM10_2017 en valeurs stockées dans les deux nouvelles colonnes PM10-Année et Valeur.

```
import pandas as pd
myDataset = pd.read_csv('Data\ecoles-creches-idf.csv')

print("myDataset avant l'application de la méthode melt")
print(myDataset)

myDataset= myDataset.melt(['ID','geometry'],['PM10_2012',
                          'PM10_2013','PM10_2014',
                          'PM10_2015','PM10_2016',
                          'PM10_2017'],
                          'PM10-Année','Valeur')

print("\nmyDataset après l'application de la méthode melt")
print(myDataset)
```

Ce code lit le contenu du fichier *ecoles-creches-idf.csv* dans le DataFrame myDataset, puis en appliquant la méthode melt sur ce DataFrame il crée un nouveau DataFrame dans lequel il récupère les deux colonnes 'ID','geometry' à partir de myDataset et il crée les deux nouvelles colonnes 'PM10-Année','Valeur'. La colonne PM10-Année va servir à enregistrer les noms des colonnes PM10_2012, PM10_2013, PM10_2014, PM10_2015, PM10_2016, PM10_2017 et la colonne Valeur va servir à enregistrer les valeurs de ces variables.

© Editions ENI - All rights reserved

Le résultat suivant sera affiché :

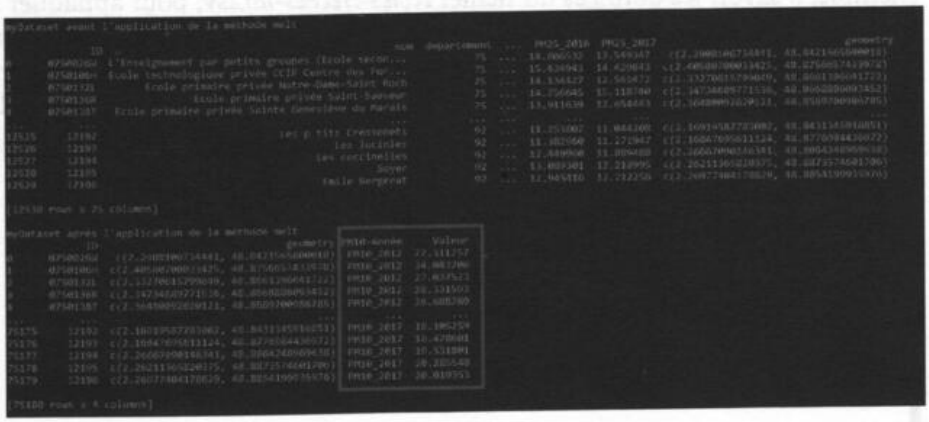


Figure 4-22 : Création des deux colonnes `PM10-Année` et `Valeur` avec `melt` à partir des variables `['PM10_2012','PM10_2013',PM10_2014', 'PM10_2015','PM10_2016', 'PM10_2017']`

Remarquez que le nombre de lignes dans le nouveau DataFrame est de 15180 alors que le nombre de lignes dans le DataFrame d'origine n'était que de 12530. Cela correspond bien au résultat attendu, puisque chaque ligne du DataFrame d'origine est fractionnée en six lignes, une ligne par année de 2012 à 2017.

En résumé, cet exemple a montré comment répartir un ensemble de colonnes dans de nouvelles lignes d'un DataFrame.

6.3 Appliquer une fonction sur une variable avec la méthode apply

Dans l'exemple précédent, nous avons appliqué une méthode prédéfinie sur l'ensemble d'un DataFrame, il s'agissait de la méthode `melt`. Il existe plusieurs méthodes prédéfinies que nous pouvons appliquer sur l'ensemble d'un DataFrame. Dans certains cas, nous souhaitons écrire notre propre fonction, puis appliquer cette fonction sur une colonne donnée.

L'exemple du code suivant reprend le même jeu de données que dans le dernier exemple, à savoir les données du fichier *ecoles-creches-idf.csv*, pour appliquer la fonction `get_latitude` sur la colonne `geometry`. Cette méthode sera appliquée grâce à la méthode `apply` et elle permet d'extraire la partie à gauche de la variable `geometry` qui correspond à la latitude d'une école donnée. Le résultat retourné par la fonction `get_latitude` sera stocké dans la nouvelle colonne `Latitude`.

```
import pandas as pd
import re
myDataset = pd.read_csv('Data\ecoles-creches-idf.csv')
myDataset=myDataset[["ID","geometry"]]

print("Avant l'application de la méthode get_latitude :")
print(myDataset.head(5))


def get_latitude(s, flag):
    res = re.split('\(|\,|\)',s)
    if flag == 0:
        res = res[1]
    else :
        res= res[2]
    return float(res)


myDataset['Latitude'] =myDataset['geometry'].apply(get_latitude,
args=(0,))

print("Après l'application de la méthode get_latitude :")
print(myDataset.head(5))
```

Remarquez l'utilisation du module `re` vu au chapitre Le langage Python pour utiliser une expression régulière et fragmenter les valeurs de la colonne `geometry` et extraire l'élément à gauche correspondant à la latitude.

© Editions ENI - All rights reserved

Ce programme affiche le résultat suivant :

```
Avant l'application de la méthode get_latitude :
        ID                                   geometry
0  0750026W  c(2.2908106734441, 48.8421565600018)
1  0750106H  c(2.40580700033425, 48.8756657433978)
2  0750132L  c(2.33270615799049, 48.8661396641722)
3  0750136R  c(2.34734689771536, 48.8668886093452)
4  0750138T  c(2.36480092620121, 48.8569700986785)

Après l'application de la méthode get_latitude :
        ID                                   geometry  Latitude
0  0750026W  c(2.2908106734441, 48.8421565600018)  2.290811
1  0750106H  c(2.40580700033425, 48.8756657433978)  2.405807
2  0750132L  c(2.33270615799049, 48.8661396641722)  2.332706
3  0750136R  c(2.34734689771536, 48.8668886093452)  2.347347
4  0750138T  c(2.36480092620121, 48.8569700986785)  2.364801
```

Figure 4-23 : Utilisation de la méthode `apply` pour appliquer la fonction `get_latitude` sur la colonne `geometry`

Remarque

Remarquez que les données de la colonne `Latitude` sont celles situées à gauche de pairs de la colonne `geometry` avec un arrondissement vers la valeur la plus proche.

6.4 Modification avec conditions

Souvent, nous avons besoin d'apporter des modifications sur un DataFrame en fonction de certaines conditions. L'exemple de code suivant montre comment associer une valeur `True` ou `False` à la nouvelle colonne `Résultat` en fonction de la différence entre les deux colonnes `O3obs` et `MOCAGE`. Si la différence entre ces deux colonnes est inférieure à 2.5, alors la colonne `Résultat` sera égale à `True`, sinon elle sera égale à `False`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')

myDataset["Résultat"] = False

columns = list(myDataset.columns.values)
```

```
myDataset = myDataset[columns[0:3]+[columns[-1]]+columns[3:10]]

dif = abs(myDataset["O3obs"]- myDataset["MOCAGE"])
myDataset.loc[dif < 2.5 , "Résultat"]= True
print(myDataset.head(5))
```

Ce programme affiche le résultat suivant :

	JOUR	O3obs	MOCAGE	Résultat	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
0	1	91	93.2	True	21.5	0.00847	1.602	0.424	Aix	9.5000	-0.64350
1	1	100	104.6	False	20.2	0.00881	2.121	0.531	Aix	8.0100	-0.04996
2	0	82	103.6	False	17.4	0.00951	1.657	0.467	Aix	9.3771	-0.12832
3	0	94	94.8	True	18.8	0.00855	2.350	0.701	Aix	9.4578	-0.34516
4	0	107	99.0	False	23.7	0.00731	1.653	0.452	Aix	7.8791	-0.41822

Figure 4-24 : Modification de la colonne `Résultat` en fonction de la différence entre `O3obs` et `MOCAGE`

6.5 Ajouter des lignes dans un DataFrame

La méthode `append` appliquée sur un objet de type DataFrame permet d'ajouter une nouvelle ligne ou plusieurs lignes dans un nouveau DataFrame qui sera retourné en résultat. L'exemple suivant montre comment ajouter une nouvelle ligne dans le DataFrame `myDataset`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
print("Avant l'ajout d'une nouvelle ligne")
print(myDataset.tail(3))

myDataset = myDataset.append({"JOUR":0, "O3obs":10, "MOCAGE":10,
"TEMPE":10, "RMH2O":10, "NO2":10, "NO":10, "STATION":"Aix",
"VentMOD":10, "VentANG":10}, ignore_index=True)

print("\nAprès l'ajout d'une nouvelle ligne")
print(myDataset.tail(3))
```

© Editions ENI - All rights reserved

Ce programme affiche le résultat suivant :

```
Avant l'ajout d'une nouvelle ligne
      JOUR  O3obs  MOCAGE  TEMPE     RMH2O    NO2     NO STATION  VentMOD  VentANG
1038     0     74   198.4   16.2  0.00811  8.349  0.942     Pla   4.0311  0.12435
1039     0    121   191.7   16.4  0.00722  2.101  0.199     Pla   3.8000  0.00000
1040     1     93   221.2   16.7  0.00919  2.290  0.202     Pla   2.3022  0.04345

Après l'ajout d'une nouvelle ligne
      JOUR  O3obs  MOCAGE  TEMPE     RMH2O     NO2      NO STATION  VentMOD   VentANG
1039     0    121   191.7   16.4   0.00722   2.101   0.199     Pla   3.8000   0.00000
1040     1     93   221.2   16.7   0.00919   2.290   0.202     Pla   2.3022   0.04345
1041     0     10    10.0   10.0  10.00000  10.000  10.000     Aix  10.0000  10.00000
```

Figure 4-25 : Ajout d'une nouvelle ligne avec la méthode append

Pour ajouter plusieurs lignes, il suffit d'utiliser la méthode append comme dans cet exemple avec une liste contenant ces nouvelles lignes à la place de l'objet de type dictionnaire. Cette liste sera une liste dont chaque élément est un dictionnaire correspondant à une nouvelle ligne.

De même, pour concaténer deux DataFrames, nous pouvons utiliser la méthode append appliquée sur l'un des deux DataFrames et le second DataFrame sera utilisé en premier argument de l'appel à cette méthode append.

7. Tri sur les données d'un DataFrame

7.1 Tri avec un seul critère

La méthode `sort_values` d'un objet de type DataFrame permet de trier le DataFrame sur lequel elle est appliquée. Par défaut, ce tri est réalisé suivant l'ordre croissant des valeurs de la colonne ou des colonnes utilisée(s) pour le tri. L'exemple suivant montre comment trier le DataFrame `myDataset` suivant l'ordre croissant des valeurs de la colonne `NO2`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
no2_sort = myDataset.sort_values('NO2')
print(no2_sort)
```

Ce programme affiche le résultat suivant :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
897	1	150	109.1	25.4	0.01358	0.258	0.080	Pla	5.38520	1.02401
991	0	104	163.3	24.1	0.00927	0.280	0.084	Pla	4.30120	0.95055
842	0	126	131.7	18.7	0.01063	0.310	0.047	Pla	5.07740	1.01011
912	1	95	129.0	23.4	0.02076	0.319	0.052	Pla	5.90080	1.55385
915	1	166	173.5	18.2	0.00655	0.328	0.001	Pla	5.77060	-1.08390
..	...	...	...	...	...	...	...	...	...	...
32	0	139	92.5	26.0	0.01426	23.822	7.519	Aix	5.36000	0.93103
707	1	96	96.1	15.2	0.00735	28.383	7.560	Ram	3.15753	0.19118
83	0	93	163.1	19.0	0.00690	30.934	5.006	Aix	3.93190	0.12751
197	0	117	154.7	20.4	0.00663	31.114	3.776	Aix	3.80789	0.23861
825	0	96	91.2	20.5	0.00510	44.396	9.429	Ram	2.54950	0.84090

Figure 4-26 : Un DataFrame trié suivant l'ordre croissant des valeurs de la variable NO2

La méthode `sort_values` admet le paramètre `ascending` qui est égal à `True` par défaut. Ce paramètre indique l'ordre ascendant ou descendant à utiliser lors du tri. Ainsi, si vous exécutez le code précédent en ajoutant l'argument `ascending=False` comme deuxième paramètre à l'appel de la méthode `sort_values` comme ci-dessous :

```
no2_sort = myDataset.sort_values('NO2', ascending=False)
```

vous obtenez le résultat suivant :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
825	0	96	91.2	20.5	0.00510	44.396	9.429	Ram	2.54950	0.84090
197	0	117	154.7	20.4	0.00663	31.114	3.776	Aix	3.80789	0.23861
83	0	93	163.1	19.0	0.00690	30.934	5.006	Aix	3.93190	0.12751
707	1	96	96.1	15.2	0.00735	28.383	7.560	Ram	3.15753	0.19118
32	0	139	92.5	26.0	0.01426	23.822	7.510	Aix	5.36000	0.93103
..	...	...	...	...	...	...	...	...	...	...
915	1	166	173.5	18.2	0.00655	0.328	0.001	Pla	5.77060	-1.08390
912	1	95	129.0	23.4	0.02076	0.319	0.052	Pla	5.90080	1.55385
842	0	126	131.7	18.7	0.01063	0.310	0.047	Pla	5.07740	1.01011
991	0	104	163.3	24.1	0.00927	0.280	0.084	Pla	4.30120	0.95055
897	1	150	109.1	25.4	0.01358	0.258	0.080	Pla	5.38520	1.02401

[1041 rows x 10 columns]

Figure 4-27 : Un DataFrame trié suivant l'ordre décroissant des valeurs de la variable NO2

© Editions ENI - All rights reserved

7.2 Tri avec plusieurs critères

Nous pouvons également trier un DataFrame suivant plusieurs colonnes. L'exemple de code suivant permet de trier les données du DataFrame `myDataset` suivant l'ordre croissant des valeurs de la colonne `O3obs` puis suivant l'ordre décroissant des valeurs de la colonne `TEMPE` et le résultat du tri est stocké dans le nouveau DataFrame `sortBy_O3obs_TEMPE`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
sortBy_O3obs_TEMPE = myDataset.sort_values(['O3obs','TEMPE'],
ascending=[1,0])
print(sortBy_O3obs_TEMPE.head(25))
```

Ce programme affiche le résultat suivant :

	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
418	0	19	48.2	20.5	0.00786	7.777	2.143	Als	3.58469	1.04473
417	0	23	65.1	18.3	0.01106	2.519	0.507	Als	2.77849	0.52807
178	1	26	187.3	21.7	0.01044	20.248	2.278	Aix	4.12311	0.39852
249	0	27	122.7	22.1	0.02082	2.742	0.444	Als	2.14009	0.91795
250	0	31	84.2	18.0	0.01849	1.264	0.312	Als	3.33766	-0.15037
405	1	33	91.3	19.7	0.01281	2.272	0.298	Als	2.28254	1.06795
370	0	35	87.8	20.3	0.00931	1.528	0.356	Als	4.60977	1.14596
359	0	36	85.1	21.6	0.00991	0.902	0.175	Als	5.34509	0.30400
785	0	38	75.3	22.5	0.00926	1.231	0.329	Ram	6.50308	0.03076
410	0	39	51.7	12.7	0.00861	6.336	1.874	Als	4.20476	1.12842
407	0	41	67.7	21.2	0.00970	4.203	1.043	Als	3.96232	0.82110
419	0	41	63.4	18.2	0.00655	1.795	0.402	Als	4.20119	0.66731
409	0	42	96.7	23.5	0.01019	2.519	0.343	Als	5.73062	0.51049
343	0	44	71.5	19.8	0.01089	1.070	0.292	Als	5.34509	0.30400
362	0	45	59.9	21.0	0.01082	2.112	0.682	Als	7.35731	1.02805
316	0	46	97.7	18.1	0.00831	1.839	0.413	Als	6.30317	0.84152
729	0	46	102.3	16.4	0.01022	3.519	0.822	Ram	2.40416	-1.27536
199	1	46	67.2	15.0	0.00978	0.708	0.264	Als	4.44072	0.62549
769	0	48	93.8	22.3	0.01061	1.635	0.363	Ram	7.47061	-0.48760
344	0	48	89.1	16.7	0.00853	1.514	0.359	Als	5.00400	-0.03998
11	0	48	146.0	14.3	0.01453	13.664	2.976	Aix	5.80348	-0.03447
706	1	48	58.4	11.9	0.00764	4.477	1.944	Ram	5.73149	-0.82242
641	0	51	83.3	19.4	0.01430	1.791	0.509	Ram	6.35138	0.58364
320	1	51	63.4	16.5	0.00747	2.396	0.863	Als	1.02956	0.50710
601	1	52	218.2	23.1	0.00952	6.148	0.482	Cad	5.86690	0.51703

Figure 4-28 : Un DataFrame trié suivant l'ordre croissant de la variable `O3obs` et l'ordre croissant de la colonne `TEMPE`

Remarquez que l'ensemble des valeurs de la colonne `O3obs` sont triées suivant l'ordre croissant. Lorsque deux lignes successives ont une même valeur pour la colonne `O3obs`, les valeurs correspondantes de la colonne `TEMPE` sont triées suivant l'ordre décroissant. C'est le cas par exemple des deux colonnes avec les identifiants `769` et `344`.

8. Sauvegarder les données d'un DataFrame

Les données d'un DataFrame peuvent être enregistrées dans un fichier, par exemple un fichier *.csv* ou un fichier texte ou encore un fichier Excel.

L'exemple suivant montre comment sauvegarder les données du DataFrame `myDataset` dans un fichier *.csv*. Ce DataFrame contient initialement les données du fichier *ozone.csv* auxquelles nous avons ajouté la colonne `Pow2_O3obs`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
myDataset["Pow2_O3obs"] = pow(myDataset["O3obs"],2)
columns = list(myDataset.columns.values)
myDataset = myDataset[columns[0:2]+[columns[-1]]+columns[2:10]]
print(myDataset.head(5))

myDataset.to_csv('Data\ozone2.csv')
```

À la fin de l'exécution de ce programme, vous devriez trouver le fichier *ozone2.csv* créé dans le dossier *Data*. Si vous ouvrez ce fichier avec un logiciel tel qu'Excel, vous devriez voir des résultats comme ceux de la figure suivante :

	A	B	C	D	E	F	G	H	I	J	K	L
		JOUR	O3obs	Pow2_O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
0	0	1	91	8281	93.2	21.5	0.00847	1.602	0.424	Aix	9.5	-0.6435
1	1	1	100	10000	104.6	20.2	0.00881	2.121	0.531	Aix	8.01	-0.04996
2	2	0	82	6724	103.6	17.4	0.00951	1.657	0.467	Aix	9.3771	-0.12832
3	3	0	94	8836	94.8	18.8	0.00855	2.35	0.701	Aix	9.4578	-0.34516
4	4	0	107	11449	99	23.7	0.00731	1.653	0.432	Aix	7.8791	-0.41822
5	5	0	150	22500	114.3	23.6	0.01182	5.316	1.343	Aix	6.3127	0.06341

Figure 4-29 : Le DataFrame `myDataset` sauvegardé avec la colonne des index

© Editions ENI - All rights reserved

Remarquez la présence de la première colonne où sont enregistrés les index des lignes du DataFrame `myDataset`. Pour enregistrer un DataFrame sans la colonne des index, nous pouvons indiquer l'argument `index=False` lors de l'appel à la méthode `to_csv`.

Remplacez l'appel à la méthode `to_csv` du code précédent avec l'appel suivant :

```
myDataset.to_csv('Data\ozone2.csv', index= False)
```

Cette fois, vous devriez obtenir un fichier CSV comme celui de la figure ci-dessous :

JOUR	O3obs	Pow2_O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	STATION	VentMOD	VentANG
1	91	8281	93.2	21.5	0.00847	1.602	0.424	Aix	9.5	-0.6435
1	100	10000	104.6	20.2	0.00881	2.121	0.531	Aix	8.01	-0.04996
0	82	6724	103.6	17.4	0.00951	1.657	0.467	Aix	9.3771	-0.12832
0	94	8836	94.8	18.8	0.00855	2.35	0.701	Aix	9.4578	-0.34516

Figure 4-30 : Le DataFrame `myDataset` sauvegardé sans la colonne des index

Remarque

Utilisez la méthode `to_excel` à la place de la méthode `to_csv` pour sauvegarder un DataFrame dans un fichier Excel.

9. Faire des statistiques sur un DataFrame

9.1 Faire un résumé direct

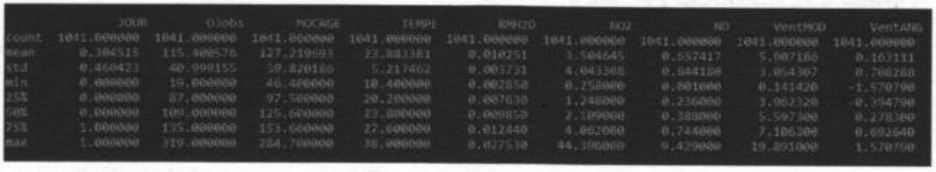
Avec Pandas, on peut résumer un DataFrame avec les statistiques de position classiques en utilisant la méthode `describe` appliquée sur un objet de type DataFrame. L'exemple du code suivant permet de résumer le jeu de données du fichier *ozone.csv*.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
stat = myDataset.describe()

print(stat)
```

Ce programme crée l'objet `stat`, qui est un DataFrame, à partir des statistiques caractéristiques de position des données du DataFrame `myDataset`. Le résultat affiché sera le suivant :



	JOUR	Obobs	NOCAGE	TEMPE	RMH2O	NO2	NO	VentMOD	VentANG
count	1041.000000	1041.000000	1041.000000	1041.000000	1041.000000	1041.000000	1041.000000	1041.000000	1041.000000
mean	0.384515	115.400576	127.219693	23.883381	0.010251	3.504645	0.557417	5.907186	0.163111
std	0.460421	40.999155	39.820186	5.217462	0.003731	4.043308	0.844180	3.054307	0.700288
min	0.000000	19.000000	46.400000	10.400000	0.002858	0.250000	0.001000	0.141420	-1.570790
25%	0.000000	87.000000	97.500000	20.200000	0.007630	1.240000	0.236000	3.962320	-0.394790
50%	0.000000	109.000000	125.600000	23.800000	0.009859	2.109000	0.388000	5.597300	0.278300
75%	1.000000	135.000000	153.600000	27.600000	0.012440	4.062000	0.744000	7.186300	0.692640
max	1.000000	319.000000	284.700000	38.000000	0.027530	44.390000	9.429000	19.891000	1.570790

Figure 4-31 : Les statistiques de position du DataFrame `myDataset`

Ainsi, pour chaque variable du DataFrame `myDataset`, les statistiques suivantes sont calculées :

- `cout` : le nombre de valeurs présentes pour une colonne.
- `mean` : la moyenne de chaque colonne.
- `std` : l'écart-type de chaque colonne.
- `min` : la valeur minimale de chaque variable.
- `25%`, `50%` et `75%` : les quartiles associés à chaque colonne.
- `max` : la valeur maximale de chaque colonne.

Remarque

Nous reviendrons sur les notions théoriques et pratiques des statistiques de position et également des statistiques inférentielles dans le chapitre Statistiques.

9.2 Faire un résumé par agrégation

L'exemple précédent nous a montré comment résumer l'ensemble des données d'un DataFrame avec les statistiques de position. Ces statistiques ont été calculées en considérant l'ensemble des valeurs de chaque colonne. Souvent, nous avons besoin de résumer nos données en les considérant par groupes, c'est-à-dire réaliser des agrégations avant d'opérer les calculs des statistiques.

© Editions ENI - All rights reserved

L'exemple de code suivant permet de calculer la moyenne des colonnes en faisant l'agrégation des données du fichier *ozone.csv* suivant la colonne `STATION` en utilisant la méthode `groupby`.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
mean= myDataset.groupby(["STATION"]).mean()
print(mean)
```

Ce programme affiche le résultat suivant :

STATION	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	VentMOD	VentANG
Aix	0.296482	119.497487	135.400000	25.766332	0.009962	6.732271	1.261814	6.794890	0.095520
Als	0.310811	103.031532	100.561261	23.440541	0.010876	2.055248	0.418748	4.030981	0.542178
Cad	0.301980	130.366337	137.567327	26.896535	0.009894	2.559297	0.386723	6.927032	0.034590
Pla	0.302885	122.230769	153.218346	21.160096	0.010253	2.029635	0.356861	7.610496	0.048896
Ram	0.309524	103.433333	111.947143	22.366190	0.010204	4.348600	0.895062	4.381315	0.063185

Figure 4-32 : Les moyennes des colonnes de `myDataset` avec l'agrégation suivant la colonne `STATION`

Contrairement aux résultats de la figure 4-31 où la moyenne `mean` était calculée sur l'ensemble des valeurs de chaque colonne, cette fois la moyenne est calculée suivant les différentes valeurs de la colonne `STATION`. Ainsi, nous pouvons constater par exemple que pour la variable `O3obs`, la moyenne est plus faible pour les valeurs `Als` et `Ram` que pour les autres valeurs.

Pour simplifier la lecture des résultats lors du calcul des statistiques de position combiné avec l'utilisation des agrégations avec la méthode `groupby`, il est plus intéressant de réaliser des tris sur les résultats selon les colonnes qui nous intéressent.

En remplaçant l'instruction qui appelle la méthode `groupby` du code précédent par l'instruction suivante, nous calculons la moyenne des colonnes avec l'agrégation selon la colonne `STATION`, comme précédemment, suivi d'un tri sur la colonne `O3obs`.

```
mean= myDataset.groupby(["STATION"]).mean().sort_values("O3obs",
ascending=False)
```

Le résultat sera le suivant :

STATION	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	VentMOD	VentANG
Als	0.310811	103.031532	100.561261	23.440541	0.010876	2.055248	0.418748	4.030981	0.542178
Ram	0.309524	103.433333	111.947143	22.366190	0.010204	4.348600	0.895062	4.381315	0.063185
Aix	0.296482	119.497487	135.400000	25.766332	0.009962	6.732271	1.261814	6.794890	0.095520
Pla	0.302885	122.230769	153.216346	21.160096	0.010253	2.029635	0.356861	7.610496	0.048896
Cad	0.301980	130.366337	137.567327	26.896535	0.009804	2.559297	0.386723	6.927832	0.034590

Figure 4-33 : Les moyennes des colonnes de `myDataset` avec l'agrégation selon la colonne `STATION` et un tri sur la colonne `O3obs`

Cette fois, les données sont triées suivant l'ordre croissant sur les valeurs de la colonne `O3obs`.

9.3 Agrégation avec plusieurs paramètres

Plus intéressant encore : résumer les données par paquets groupés selon plusieurs colonnes. L'exemple de code suivant calcule la moyenne de chaque colonne du jeu de données *ozone.csv* après avoir agrégés les données selon les deux colonnes `STATION` et `JOUR` ; cela va nous permettre de comparer les valeurs de ces moyennes en fonction des jours fériés ou non.

```
import pandas as pd

myDataset = pd.read_csv('Data\ozone.csv')
byDayOff= myDataset.groupby(["STATION","JOUR"]).mean()
print(byDayOff)
```

Ce programme affiche le résultat suivant :

STATION	JOUR	O3obs	MOCAGE	TEMPE	RMH2O	NO2	NO	VentMOD	VentANG
Aix	0	119.571429	136.391429	25.797857	0.010122	6.837729	1.282143	6.864774	0.073687
	1	119.322034	133.047458	25.691525	0.009581	6.482034	1.213576	6.829063	0.147328
Als	0	103.267974	101.909804	23.594118	0.010888	2.095902	0.418065	4.041690	0.531743
	1	102.507246	97.571014	23.100000	0.010850	1.965101	0.420261	4.007236	0.565317
Cad	0	130.000000	137.509929	26.800000	0.010079	2.594823	0.394241	6.916354	0.027821
	1	131.213115	137.700000	27.119672	0.009467	2.477180	0.369344	6.951713	0.050234
Pla	0	122.917241	154.862759	21.135172	0.010381	2.183779	0.325607	7.830073	0.000176
	1	120.650794	149.426984	21.217460	0.009959	1.674857	0.313714	7.105121	0.161028
Ram	0	103.179310	112.291034	22.362759	0.010446	4.353931	0.884745	4.399357	0.046589
	1	104.000000	111.180000	22.373846	0.009663	4.336708	0.918077	4.341068	0.100384

Figure 4-34 : Les moyennes des colonnes de `myDataset` avec l'agrégation suivant les colonnes `STATION` et `JOUR`

© Editions ENI - All rights reserved

10. Lecture des fichiers de grande taille

L'un des atouts de Pandas les plus appréciés par les Data Scientists réside dans le fait que nous pouvons lire des fichiers de très grande taille. Le problème avec les fichiers de grande taille est que nos ordinateurs n'ont pas assez de mémoire vive pour les charger entièrement.

Avec Pandas, nous pouvons lire un fichier paquet par paquet, c'est-à-dire qu'à un instant donné, il n'y aura qu'un seul paquet qui sera chargé dans la mémoire vive.

L'exemple de code suivant montre comment lire le fichier *ozone.csv* paquet par paquet :

```
import pandas as pd

for bloc in pd.read_csv('Data\ozone.csv', chunksize = 5):
    print("Nouveau bloc:")
    print(bloc)
```

Grâce à l'utilisation du paramètre `chunksize` de la méthode `read_csv`, nous pouvons indiquer la taille du paquet qui sera chargé en mémoire. Dans cet exemple, le fichier *ozone.csv* sera chargé par paquets de cinq lignes.

Ce programme affiche les paquets :

```
Nouveau bloc:
      JOUR  O3obs  MOCAGE  TEMPE    RMH2O    NO2     NO STATION  VentMOD  VentANG
1035     0     93   177.3   20.4  0.00928  5.375  0.684     Pla   5.9908 -0.58337
1036     0    116   233.6   22.1  0.01048  4.843  0.431     Pla   4.8052 -0.20964
1037     0     60   162.5   15.8  0.00789  3.166  0.411     Pla   2.7203 -0.62880
1038     0     74   198.4   16.2  0.00811  8.349  0.942     Pla   4.0311  0.12435
1039     0    121   191.7   16.4  0.00722  2.101  0.199     Pla   3.8000  0.00000
Nouveau bloc:
      JOUR  O3obs  MOCAGE  TEMPE    RMH2O   NO2     NO STATION  VentMOD  VentANG
1040     1     93   221.2   16.7  0.00919  2.29  0.202     Pla   2.3022  0.04345
```

Figure 4-35 : Lecture du fichier ozone.csv paquet par paquet

Remarque

Notre fichier ozone.csv est de petite taille, mais le but ici est de montrer la technique de lecture paquet par paquet d'un fichier qui est applicable exactement de la même manière sur n'importe quel fichier de n'importe quelle taille.

Cette technique de lecture des fichiers par paquet permet de parcourir des fichiers de taille beaucoup plus grande que la taille de la mémoire vive.

Si, en plus de parcourir toutes les lignes d'un fichier de taille supérieure à ce que permet la taille de votre mémoire vive, vous souhaitez capturer un maximum d'informations sur l'ensemble des données de ce grand fichier, puis si vous souhaitez travailler sur ces informations, alors il serait intéressant de créer un nouveau DataFrame dans lequel vous allez stocker uniquement les colonnes qui vous intéressent avec, si c'est possible, des résumés sur les données du fichier initial.

Par exemple, le code suivant permet de lire le fichier *ozone.csv* par paquets de 200 lignes et de créer un DataFrame `myDataset` dans lequel il sera stocké uniquement les trois colonnes `O3obs`, `MOCAGE` et `TEMPE` après avoir calculé les moyennes des colonnes suivant l'agrégation par la colonne `STATION`.

```
import pandas as pd

myColumns = ["O3obs","MOCAGE","TEMPE"]
myDataset = pd.DataFrame(columns = myColumns)

for bloc in pd.read_csv('Data\ozone.csv', chunksize = 200):
   bloc = bloc.drop( columns=["JOUR", "RMH2O", "NO2", "NO",
                              "VentMOD", "VentANG"])
   results = bloc.groupby(["STATION"]).mean()
   myDataset = pd.concat([myDataset, results[myColumns]])

print(myDataset)
```

Dans ce programme, à chaque lecture d'un bloc de 200 lignes du fichier *ozone.csv*, ce bloc sera réduit aux quatre colonnes `STATION`, `O3obs`, `MOCAGE` et `TEMPE`, puis la moyenne des colonnes `O3obs`, `MOCAGE` et `TEMPE` sera calculée suivant l'agrégation sur la colonne `STATION`, et enfin le résultat sera ajouté au DataFrame `myDataset`. De cette façon, à la fin de l'exécution de toutes les itérations de lecture de blocs, le DataFrame `myDataset` contiendra les moyennes calculées des trois colonnes `O3obs`, `MOCAGE` et `TEMPE` sur l'ensemble du fichier *ozone.csv*.

© Editions ENI - All rights reserved

Remarque

Le fait de commencer chaque itération de lecture des blocs du fichier ozone.csv par la suppression des colonnes `JOUR`, `RMH2O`, `NO2`, `NO`, `VentMOD` *et* `VentANG` *permet d'optimiser les calculs de la méthode* `mean` *qui ne seront dans ce cas réalisés que sur les trois colonnes restantes qui nous intéressent, à savoir les colonnes* `O3obs`, `MOCAGE` *et* `TEMPE`.

Ce programme affiche le résultat suivant :

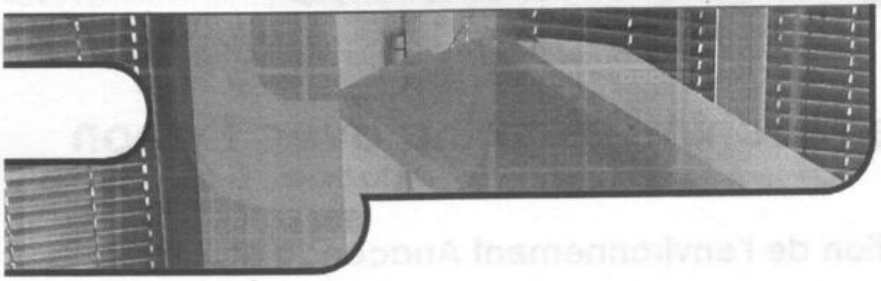
```
          O3obs      MOCAGE      TEMPE
Aix  119.497487  135.400000  25.766332
Als   46.000000   67.200000  15.000000
Als  106.830000  101.995500  23.752500
Als   69.571429   88.490476  20.871429
Cad  131.452514  137.932961  27.218994
Cad  121.913043  134.721739  24.386957
Ram  105.197740  113.839548  22.473446
Pla  129.149701  154.136527  21.386228
Ram   93.969697  101.796970  21.790909
Pla   94.048780  149.468293  20.239024
```

Figure 4-36 : Résumé du fichier ozone.csv paquet par paquet

Une fois que nous avons ajouté un par un les résumés d'un grand fichier dans un DataFrame, à la fin des itérations de lecture des blocs, nous aurions obtenu le résumé de l'ensemble du fichier initial.

11. Conclusion

Ce chapitre nous a permis d'aborder la bibliothèque Pandas. Bien sûr, il y a bien plus de choses à dire sur cette bibliothèque, mais nous avons évoqué la plupart des notions nécessaires pour travailler avec un jeu de données. Ainsi, nous avons abordé des notions telles que la création et la manipulation des DataFrames, le filtrage des données d'un DataFrame suivant un ou plusieurs critères, la restructuration et la modification d'un DataFrame, les opérations de tri ainsi que le calcul des statistiques de position sur les données d'un DataFrame, et enfin, nous avons vu comment réaliser des résumés des données d'un DataFrame et comment lire un fichier de grande taille.



Chapitre 2-4
Travailler avec Jupyter

1. Objectif du chapitre

L'environnement Jupyter est une application web dont l'origine est le projet IPython. On peut utiliser Jupyter pour programmer dans plus de quarante langages de programmation tels que Julia, Python, R (d'où le nom Jupyter), ou encore le langage Scala.

Ce chapitre aborde les fonctionnalités de Jupyter qui permettent de créer et de manipuler des notebooks ; il aborde également le concept des widgets dans Jupyter. Ces derniers peuvent être d'une grande utilité, notamment lorsque nous souhaitons réaliser des illustrations plus dynamiques et plus conviviales !

À la fin de ce chapitre, le lecteur aura abordé :

- l'installation de la distribution Anaconda et de Jupyter,
- un bref tour d'horizon des fonctionnalités de Jupyter,
- la manipulation et la gestion des documents dans Jupyter,
- la manipulation et l'utilisation des notebooks Jupyter,
- une introduction aux widgets Jupyter.

2. Installation de l'environnement Anaconda et Jupyter

Anaconda est sans doute l'une des distributions les plus connues et les plus utilisées pour le déploiement des environnements de développement en Data Science avec le langage Python. L'installation d'Anaconda s'accompagne de l'installation par défaut de la plupart des modules utilisés dans des projets d'analyse de données avec les statistiques et avec les algorithmes du Machine Learning.

La suite de cette section montre comment installer l'environnement Anaconda en local sur votre ordinateur, puis la section suivante est consacrée à l'utilisation de l'un des environnements de travail les mieux appréciés par les Data Scientists : l'environnement Jupyter.

Pour installer Anaconda sur votre ordinateur, suivez les quelques étapes ci-après :

- Rendez-vous sur le site d'Anaconda : https://anaconda.org. Vous êtes dirigé sur la page d'accueil d'Anaconda.

Remarque

Sur cette page d'accueil, vous êtes invité à créer un compte personnel Anaconda. Vous n'allez pas avoir besoin d'un tel compte pour la suite de ce livre. Cependant, créez-le si vous souhaitez découvrir les fonctionnalités proposées avec la distribution Anaconda sur le cloud.

- En haut à droite sur la page d'accueil d'Anaconda, cliquez sur le lien **Download Anaconda**. Vous êtes dirigé sur la page de présentation générale d'Anaconda et des téléchargements des fichiers d'installation.
- Cherchez et cliquez sur le bouton **Download**. Vous êtes envoyé vers la partie de la page où se trouvent les différents liens de téléchargement des fichiers d'installation d'Anaconda comme sur la figure ci-dessous.

© Editions ENI - All rights reserved

Vous constatez qu'Anaconda peut être téléchargé et installé sur les trois plateformes Windows, Mac OS et Linux.

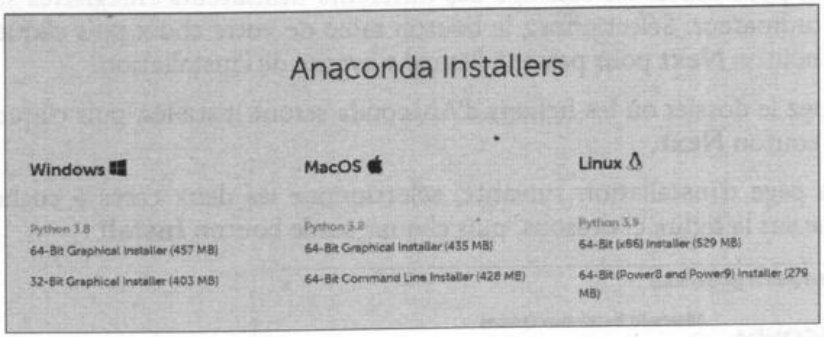


Figure 5-1 : Les versions d'Anaconda

- Cliquez sur le lien de téléchargement correspondant à votre système d'exploitation afin de télécharger le fichier d'installation sur votre machine.

Pendant le téléchargement de ce fichier, vous êtes invité à vous inscrire sur le site Anaconda afin de bénéficier, entre autres, de la version cloud d'Anaconda. Comme promis, dans ce livre, nous allons uniquement utiliser des outils qui tournent en local sur votre machine.

La plateforme utilisée pour les besoins de cet ouvrage est Windows. Si tel est votre cas, alors continuez à suivre les étapes suivantes pour lancer et installer Anaconda. Si vous travaillez sur Linux ou Mac OS, la procédure d'installation d'Anaconda ne devrait pas être très différente de celle présentée ci-dessous.

- Une fois que le fichier d'installation est complètement téléchargé, exécutez le programme d'installation d'Anaconda.
- Pour des raisons de sécurité, votre système d'exploitation vous demande de confirmer l'exécution du programme d'installation d'Anaconda lors de son lancement. Cliquez sur le bouton **Run**.
- Cliquez sur le bouton **Next** sur la page d'accueil du programme d'installation d'Anaconda.
- Lisez et validez les termes d'utilisation de la solution Anaconda en cliquant sur le bouton **I Agree**.

- Dans la page suivante, le programme d'installation vous demande d'indiquer si la solution, une fois installée, doit être disponible uniquement pour vous ou bien pour toutes les sessions des différents utilisateurs enregistrés sur votre ordinateur. Sélectionnez le bouton radio de votre choix puis cliquez sur le bouton **Next** pour passer à l'étape suivante de l'installation.
- Indiquez le dossier où les fichiers d'Anaconda seront installés, puis cliquez sur le bouton **Next**.
- Sur la page d'installation suivante, sélectionnez les deux cases à cocher comme sur la figure ci-dessous, puis cliquez sur le bouton **Install** :

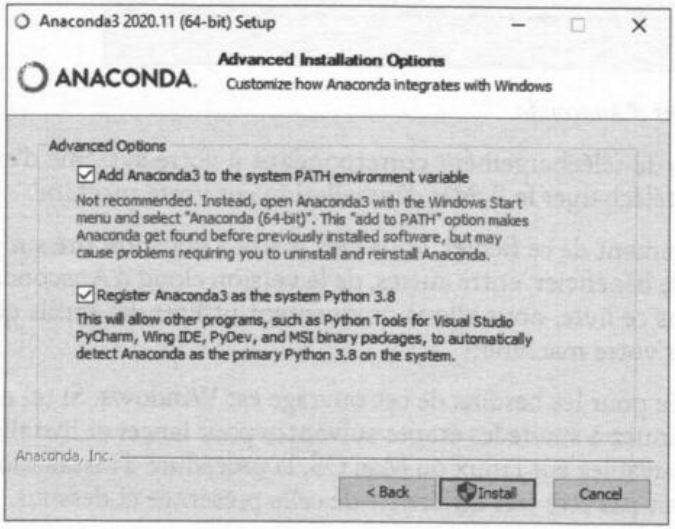


Figure 5-2 : Page de lancement de l'installation d'Anaconda

La première case à cocher indique au programme d'installation d'ajouter les chemins d'accès vers les différents programmes Anaconda dans la variable système PATH qui permet à votre système d'exploitation de savoir où se situent les exécutables Anaconda. La deuxième case à cocher permet de positionner et d'associer la version 3.8 de Python à Anaconda comme étant la version par défaut lorsque l'interpréteur Python sera utilisé par les différents outils de développement tels que PyCharm ou Visual Studio.

© Editions ENI - All rights reserved

Remarque

Dans le deuxième chapitre, lorsque nous avons étudié le langage Python, nous avons installé une version de Python supérieure à la version de Python incluse lors de l'installation d'Anaconda. L'installation d'Anaconda a comme conséquence la suppression de la version précédemment installée pour la remplacer avec la version proposée par la distribution Anaconda.

- Une fois que l'installation est terminée, cliquez sur le bouton **Next** afin de passer à la dernière étape de ce programme d'installation.
- Dans la nouvelle page qui s'affiche, en fonction des outils déjà installés sur votre plateforme, le programme d'installation d'Anaconda vous propose d'installer des outils de développement couramment utilisés avec le langage Python tels que PyCharm ou Visual Studio. Cliquez sur le bouton **Next**.
- Enfin, sélectionnez les deux cases à cocher sur la dernière page d'installation d'Anaconda, puis cliquez sur **Finish**. Le fait que vous ayez sélectionné ces deux cases à cocher vous ouvre dans votre navigateur par défaut de votre système d'exploitation les deux pages web correspondant respectivement à la page des tutoriels et de documentation d'Anaconda.
- Pour vérifier qu'Anaconda est installé correctement sur votre ordinateur, cliquez sur le bouton **Démarrer** de Windows et lancez la console **Anaconda Prompt (Anaconda3)** :

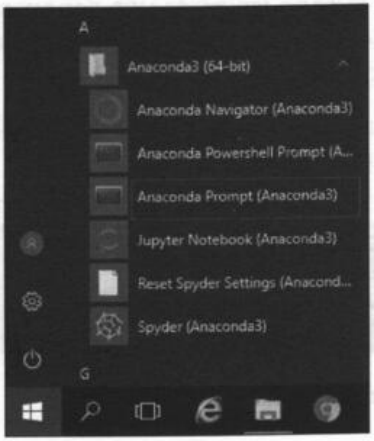


Figure 5-3 : Lancement de la console Anaconda

Une fenêtre correspondant à la console Anaconda s'ouvre, comme sur la figure suivante :

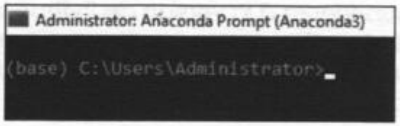


Figure 5-4 : Lancement de la console Anaconda

Remarquez que l'invite de commandes de cette console n'est pas dans la forme classique de l'invite de commandes habituelle de Windows. Cette fois, avec la console Anaconda, l'invite commence avec `(base)` ou lieu du `C:\....`

En effet, la notation `(base)` est le nom de l'environnement virtuel par défaut d'Anaconda. Un environnement virtuel Python peut être vu comme un simple environnement de développement Python dans lequel nous pouvons installer un ensemble de modules Python compatibles entre eux.

Si pour un nouveau projet Python nous avons besoin de versions différentes des modules Python déjà installés, alors il suffirait de créer un autre environnement virtuel dans lequel nous pouvons installer les modules Python avec leurs versions requises par ce nouveau projet.

En d'autres termes, les environnements virtuels Python permettent d'avoir sur la même machine plusieurs environnements Python isolés les uns des autres et qui permettent d'installer et d'utiliser une bibliothèque Python avec des versions différentes entre les différents environnements.

▶ Dans la console Anaconda, entrez la commande `python` comme dans la figure ci-dessous, afin de lancer l'interpréteur Python.

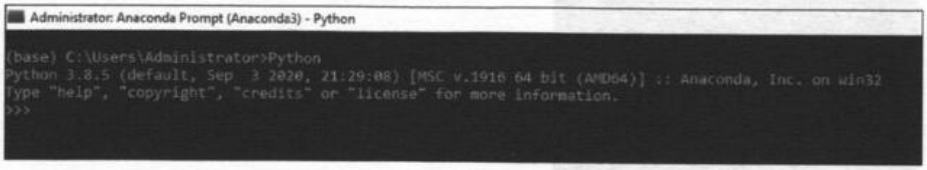


Figure 5-5 : Lancement de l'interpréteur Python via la console Anaconda

© Editions ENI - All rights reserved

Remarque

Au moment de la rédaction de ce livre, c'est bien la version 3.8.5 de Python qui est incluse avec l'installation de la dernière distribution d'Anaconda ; ce peut être différent dans votre cas.

- Dans l'interpréteur Python, saisissez les cinq instructions Python suivantes afin de vérifier que ces bibliothèques sont bien installées.

```
import numpy
import scipy
import matplotlib
import pandas
import statsmodels
```

Si tout est correctement installé, alors vous devriez avoir le même résultat que celui de la figure ci-après :

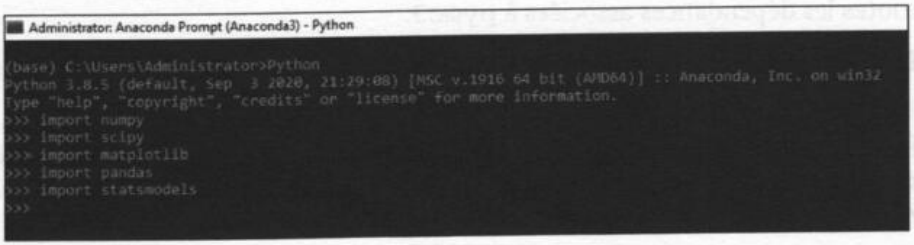


Figure 5-6 : Test de l'installation des bibliothèques telles que Pandas à la suite de l'installation d'Anaconda

- Tentez d'importer la bibliothèque `pymc3` et visualisez le résultat.

Dans le cas présent, la bibliothèque `pymc3` n'a pas été installée par défaut lors de l'installation d'Anaconda sur l'ordinateur (voir la figure ci-dessous) :

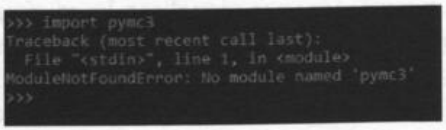


Figure 5-7 : Message d'erreur à la suite de l'importation du module `pymc3`

- Si cela est également votre cas, alors saisissez la commande `quit()` ou appuyez simultanément sur les deux touches [Ctrl] et [Attn] (sur un clavier AZERTY, la touche [Attn] se trouve généralement sur la même touche que [Pause]), pour quitter l'interpréteur Python.

Pour installer une nouvelle librairie, vous pouvez utiliser la commande `conda install` suivie du nom de la librairie que vous souhaitez installer.

- Pour installer la bibliothèque `pymc3`, saisissez la commande suivante dans la console Anaconda :

```
conda install pymc3
```

Cette commande télécharge les paquets nécessaires pour l'installation de `pymc3`.

Avant l'installation de `pymc3`, vous êtes invité à accepter l'installation de toutes les dépendances associées à `pymc3`.

- Appuyez sur la touche **y** pour accepter et finaliser l'installation de la bibliothèque `pymc3`.

Félicitations, vous avez installé Anaconda en local sur votre machine et vous savez comment installer de nouvelles bibliothèques Python via la console Anaconda !

L'installation d'Anaconda inclut l'installation de l'application Jupyter que nous allons utiliser dans la suite de ce livre pour nos programmes Python.

3. Travailler avec Jupyter

L'environnement Jupyter et ses notebooks peuvent être assimilés à un cahier numérique dans lequel vous notez vos programmes et visualisez leurs résultats. En effet, avec les notebooks Jupyter, une fois que vous avez saisi et exécuté du code, non seulement vous gardez une trace visuelle des résultats de ces codes, mais en plus vous pouvez ajouter des annotations aux différentes parties du code afin d'apporter des explications.

© Editions ENI - All rights reserved

Ces annotations sont réalisées avec le format Markdown qui permet, dans un notebook, de rédiger du texte en gras ou en italique, d'ajouter différents éléments tels que des formules mathématiques, des liens vers d'autres pages sur le Web, des titres ou encore des images. Tous ces éléments permettent de transformer nos codes Python, qui habituellement ne sont digestibles que par des informaticiens, en de jolies présentations dignes de ce nom ! Voyons maintenant à quoi ressemble concrètement Jupyter et comment l'utiliser.

- Pour lancer l'application Jupyter, cliquez sur le bouton **Démarrer** de Windows puis cliquez sur le lien **Jupyter Notebook (Anaconda3)** comme sur la figure suivante :

Figure 5-8 : Lancement de l'application Jupyter

Un terminal de commandes est lancé pour initialiser et monitorer l'exécution de l'application Jupyter. Au bout de quelques secondes, une fenêtre de votre navigateur par défaut s'ouvre sur la page d'accueil de l'environnement Jupyter avec l'URL locale localhost:8888/tree, comme sur la figure ci-dessous :

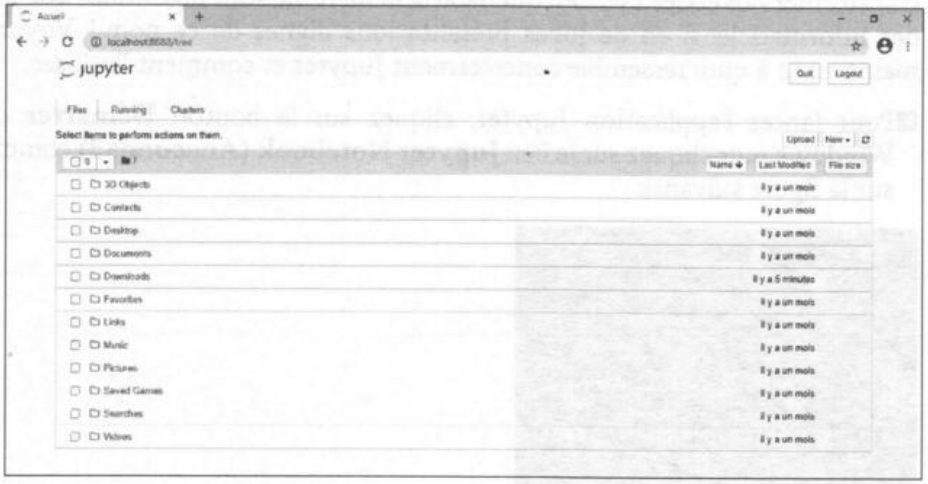


Figure 5-9 : Page d'accueil Jupyter

3.1 Les documents dans Jupyter

La page d'accueil de l'environnement Jupyter affiche la liste des éléments (dossiers et fichiers) du dossier racine qui est associé à votre session utilisateur sur votre ordinateur. Par exemple, sous Windows, si vous êtes l'administrateur de votre ordinateur, alors les documents qui seront affichés à la page d'accueil Jupyter sont les documents du dossier *C:\Users\Administrator*.

Sur cette page d'accueil, les deux boutons en haut à droite permettent respectivement d'arrêter le noyau Jupyter et de se déconnecter.

© Editions ENI - All rights reserved

Au centre de cette page, vous disposez des trois onglets suivants :

- **Files** : cet onglet affiche le contenu du dossier en cours. Par défaut, le dossier en cours correspond au dossier racine de votre session utilisateur, comme nous l'avons mentionné précédemment. C'est cet onglet qui vous permet de naviguer entre les différents dossiers et d'ouvrir vos notebooks ou tout autre fichier.
- **Running** : cet onglet affiche les notebooks et les terminaux en cours d'exécution. Nous y reviendrons plus tard avec un exemple d'utilisation de cet onglet.
- **Clusters** : cet onglet permet de mettre en place une architecture de calcul distribué avec Jupyter. Cette fonctionnalité est intéressante lorsque nous réalisons des calculs qui peuvent être parallélisés. Les détails de réalisation des calculs distribués ne font pas partie du périmètre couvert dans ce livre, néanmoins il faut savoir que grâce à cette possibilité de création de clusters, vous pouvez écrire du code en local sur votre machine et l'exécuter en utilisant un ensemble de processeurs qui, eux, tournent sur des machines distantes.

3.1.1 Créer un dossier

Pour créer un nouveau dossier dans Jupyter, suivez les étapes ci-dessous.

- Cliquez sur le bouton **New** situé à droite de l'onglet **Files**, puis, dans le menu contextuel qui s'affiche, cliquez sur le lien **Folder** :

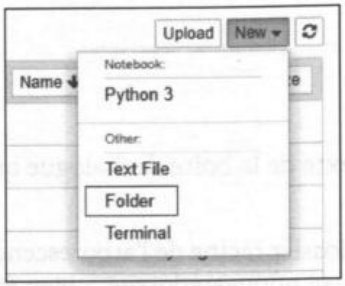


Figure 5-10 : Créer un nouveau dossier dans Jupyter

De retour à la page d'accueil Jupyter, vous constatez qu'un nouveau dossier a été créé et nommé par défaut **Untitled Folder**.

3.1.2 Renommer un dossier

Pour changer le nom du dossier **Untitled Folder** que vous venez de créer, sélectionnez-le en cochant la case à cocher correspondante, puis cliquez sur le bouton **Rename** comme sur la figure ci-dessous :

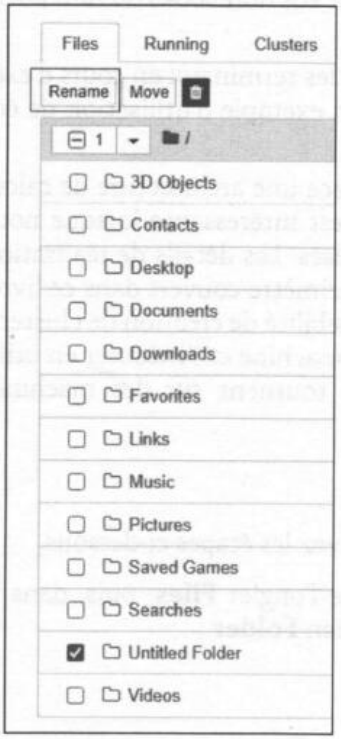


Figure 5-11 : Renommer un dossier dans Jupyter

- Saisissez le nom *TestJupyter* dans la zone de texte de la boîte de dialogue qui s'affiche puis cliquez sur le bouton **Rename**.

Vous êtes alors repositionné de nouveau sur le dossier racine de l'arborescence des dossiers et fichiers et vous constatez que votre nouveau dossier a bien été renommé en *TestJupyter*.

© Editions ENI - All rights reserved

3.1.3 Déplacer un dossier

Afin de déplacer un dossier d'un emplacement à un autre, suivez les étapes ci-dessous.

- Suivez la même procédure que précédemment afin de créer un deuxième dossier et renommez-le avec le nom *TestJupyter_Niveau_2*. Donc, pour l'instant, les deux dossiers *TestJupyter* et *TestJupyter_Niveau_2* sont au même niveau dans l'arborescence des dossiers et fichiers de Jupyter.
- Sélectionnez le nouveau dossier *TestyJupyter_Niveau_2* et cliquez sur le bouton **Move** situé en haut de l'arborescence des dossiers et fichiers juste à droite du bouton **Rename**.
- Dans la boîte de dialogue qui s'affiche, renseignez le nom du dossier *TestJupyter* précédé par une barre oblique, puis cliquez sur le bouton **Move**, comme dans la figure suivante :

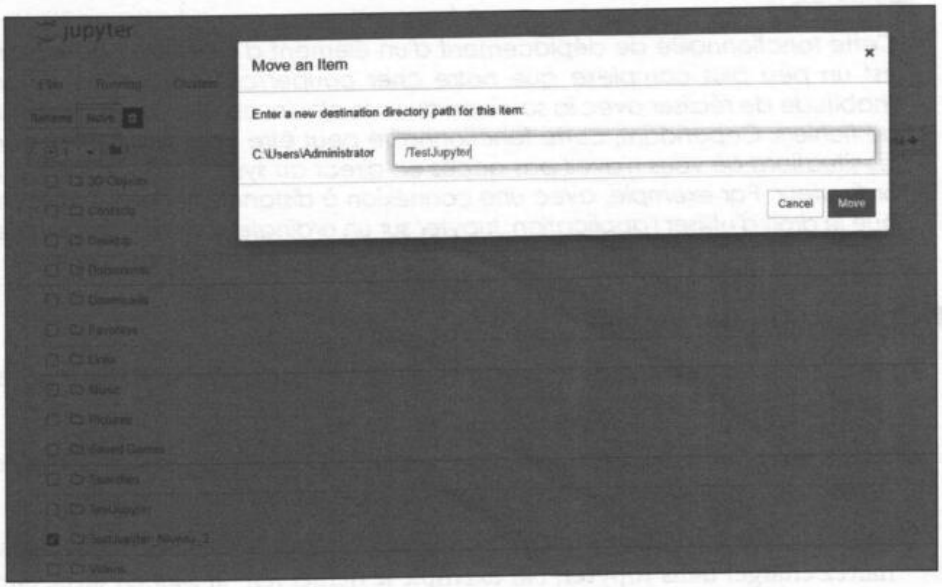


Figure 5-12 : Déplacement d'un dossier dans Jupyter

- De retour à l'arborescence des dossiers et fichiers, cliquez sur le dossier *TestJupyter* pour accéder à son contenu. Vous trouverez le dossier *TestJupyter_Niveau_2* comme étant un sous-dossier du dossier *TestJupyter*, comme sur la figure ci-dessous :

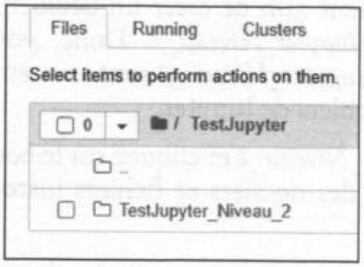


Figure 5-13 : Hiérarchie des deux dossiers TestyJupyter et Testyupyter_Niveau_2

Remarque

Cette fonctionnalité de déplacement d'un élément d'un endroit à un autre est un peu plus complexe que notre cher couper/coller que nous avons l'habitude de réaliser avec la souris ou avec le clavier en utilisant l'explorateur de fichiers. Cependant, cette fonctionnalité peut être très utile, surtout dans les situations où vous n'avez pas accès en direct au système de fichiers d'un ordinateur. Par exemple, avec une connexion à distance qui ne vous donne que le droit d'utiliser l'application Jupyter sur un ordinateur donné, vous n'allez pas pouvoir accéder à l'explorateur de fichiers de l'ordinateur en question.

3.1.4 Charger des documents

Pour charger un document de votre ordinateur, par exemple dans Jupyter, suivez les étapes ci-dessous.

- Sur la page de l'arborescence des dossiers et fichiers, cliquez sur le dossier *TestJupyter*, puis cliquez sur le bouton **Upload** situé en haut à droite.
- Dans la boîte de dialogue qui s'affiche, sélectionnez le fichier que vous souhaitez charger dans Jupyter, par exemple le fichier *test_upload.csv* situé dans le dossier *Data* de ce chapitre, puis cliquez sur le bouton **Open**.

© Editions ENI - All rights reserved

De retour dans l'arborescence des dossiers et fichiers, vous remarquez que le fichier *test_upload.csv* fait partie des éléments du dossier *TestJupyter*. Cependant, si vous accédez au dossier *TestJupyter* en utilisant l'explorateur de fichiers de votre ordinateur, vous constatez que le fichier *test_upload.csv* n'a pas été réellement copié dans ce dossier. En effet, lorsque dans Jupyter vous chargez un fichier avec le bouton **Upload**, il n'y a qu'un lien vers ce fichier qui est ajouté à la liste des éléments du dossier à partir duquel vous avez chargé ce fichier.

- Si vous souhaitez réaliser une copie du fichier ainsi chargé, cliquez sur le bouton **Upload** situé à droite du nom de ce fichier dans l'arborescence des dossiers et fichiers (voir la figure ci-dessous). À la suite de cette opération, notre fichier *test_upload.csv* est physiquement copié dans le dossier *TestJupyter*.

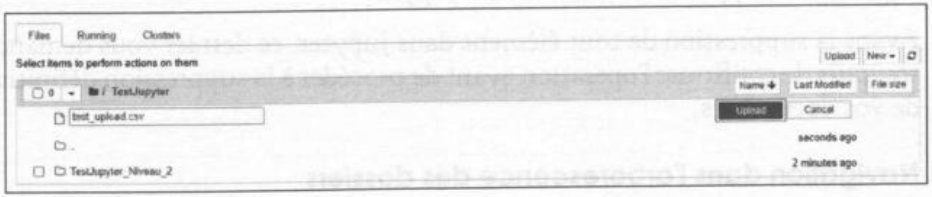


Figure 5-14 : Copie physique d'un fichier dans un dossier Jupyter

3.1.5 Supprimer des éléments

Pour supprimer des éléments de l'arborescence des dossiers et fichiers, il suffit de sélectionner ces éléments à supprimer, puis de cliquer sur le bouton symbolisé par une corbeille qui est situé en haut de l'arborescence.

Pour supprimer le dossier *TestJupyter_Niveau_2* et le fichier *test_upload.csv* de notre dossier *TestJupyter*, suivez les étapes ci-dessous.

- Accédez au dossier *TestJupyter*.

- Sélectionnez les deux éléments à supprimer en cochant les cases correspondantes, puis cliquez sur le bouton symbolisé avec une corbeille comme sur la figure ci-dessous :

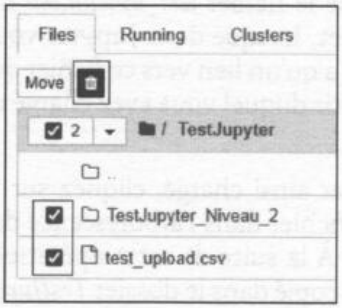


Figure 5-15 : Suppresion d'éléments dans Jupyter

Avant la suppression de tout élément dans Jupyter, ce dernier vous demande toujours de confirmer l'opération avant de procéder à la suppression définitive de vos documents.

3.1.6 Navigation dans l'arborescence des dossiers

Lors de la navigation entre les différents dossiers, un fil d'Ariane est disponible en haut de l'arborescence des dossiers et fichiers afin que vous puissiez vous repérer à tout moment et savoir à quel niveau de cette arborescence vous vous trouvez. Pour revenir au niveau du dossier racine, il suffit de cliquer sur le lien situé au début du fil d'Ariane, comme sur la figure suivante :

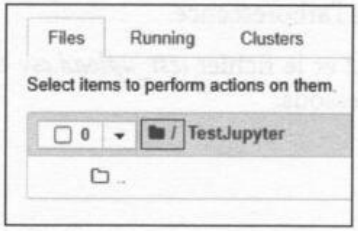


Figure 5-16 : Retour au dossier racine

© Editions ENI - All rights reserved

Remarque

Comme mentionné précédemment, toutes les opérations de manipulation de documents que nous avons réalisées dans cette section peuvent être effectuées en utilisant l'explorateur de fichiers de votre plateforme. Cependant, lorsque vous n'avez pas un accès direct ou pas suffisamment de droits sur le serveur où est installé Jupyter, il est important de savoir réaliser ces différentes manipulations via l'interface Jupyter.

3.1.7 Créer un notebook

Le notebook Jupyter est l'endroit où vous allez écrire vos codes Python. Vous pouvez créer autant de notebooks que nécessaire. Suivez les étapes ci-dessous pour créer votre premier notebook Jupyter.

- Dans l'interface Jupyter, positionnez-vous dans le dossier *TestJupyter*.
- Cliquez sur le bouton **New**, puis cliquez sur le lien **Python 3** :

Figure 5-17 : Création d'un notebook Jupyter

Un nouveau notebook est créé et il est automatiquement ouvert dans un nouvel onglet de votre navigateur, comme dans la figure ci-dessous :

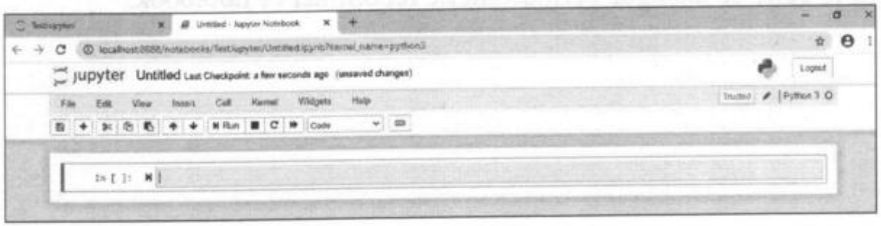


Figure 5-18 : Un nouveau notebook Jupyter

Félicitations, vous avez créé votre premier notebook Jupyter !

La section suivante aborde quelques fonctionnalités assez détaillées sur l'utilisation d'un notebook Jupyter.

3.2 Utiliser un notebook Jupyter

La section précédente a montré comment créer un notebook Jupyter. Cette section permet de découvrir les fonctionnalités les plus utilisées lorsqu'on écrit du code Python avec un notebook Jupyter. Pour cela, nous allons utiliser le notebook que nous avons créé précédemment.

Pour rappel, nous avons créé un nouveau notebook Jupyter dans le dossier *TestJupyter*. Pour découvrir les fonctionnalités de base d'un notebook Jupyter, les sous-sections suivantes montrent, entre autres, comment manipuler les cellules d'un notebook.

3.2.1 Renommer un notebook

Lors de la création de notre premier *notebook*, l'application Jupyter lui a automatiquement associé le nom par défaut *Untitled.ipynb*. Nous allons commencer par la modification du nom de notre nouveau notebook.

- Positionnez-vous dans le dossier *TestJupyter* et cliquez sur le notebook `Untitled.ipynb` pour l'ouvrir.
- Une fois que le notebook *Untitled.ipynb* est ouvert, positionnez le curseur de votre souris sur le nom de ce notebook situé juste à droite du logo Jupyter en haut à gauche de la page. Remarquez que le nom de ce notebook est automatiquement mis en valeur. Puis cliquez sur le nom **Untitled** pour ouvrir la boîte de dialogue permettant de renommer ce notebook.

© Editions ENI - All rights reserved

- Donnez un nom de votre choix à ce notebook, par exemple *MonPremierNotebook* et cliquez sur le bouton **Rename**, comme sur la figure ci-dessous :

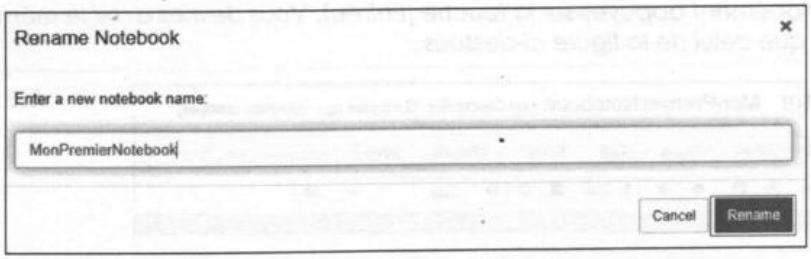


Figure 5-19 : Renommer un notebook Jupyter

Remarque

*Vous pouvez également renommer un notebook en cliquant sur le menu **File**, puis sur le bouton **Rename**.*

3.2.2 Les cellules Jupyter

L'élément le plus fondamental dans un notebook Jupyter est la cellule. Une cellule vous permet de saisir du code Python, du texte simple ou du texte avec des Markdown, de charger des images, etc. Lors de la création d'un nouveau notebook, une cellule est créée par défaut, encadrée dans un rectangle vert comme dans la figure 5-18.

Entrer du code Python

- Cliquez sur la zone de texte associée à l'unique cellule de votre notebook. Cela active cette cellule pour la rendre disponible pour la saisie de code Python ou la saisie de texte.
- Dans cette cellule sélectionnée, saisissez le code suivant :

```
print("Mon premier hello world dans mon premier notebook")
```

Remarque

Pour exécuter le code de cette cellule, maintenez la touche [Maj] appuyée et simultanément appuyez sur la touche [Entrée]. Vous devriez avoir le même résultat que celui de la figure ci-dessous :

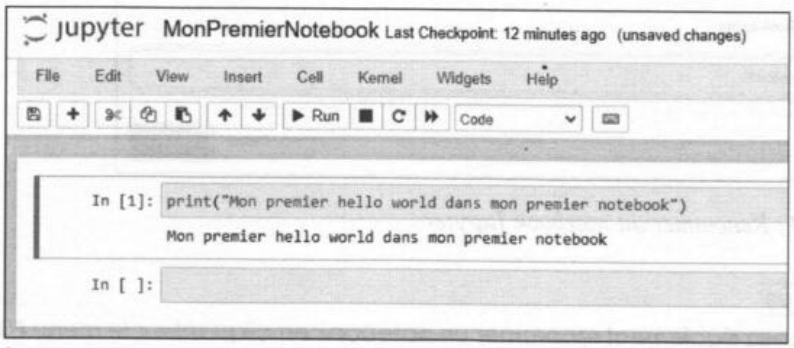


Figure 5-20 : Saisie de code Python dans une cellule Jupyter

Le résultat de l'exécution d'une cellule s'affiche immédiatement sous cette même cellule.

À chaque exécution d'une cellule, une nouvelle cellule sera créée.

Définir et appeler une fonction

- Cliquez sur la zone de texte de la nouvelle cellule qui est créée sous la première cellule et saisissez le code Python suivant :

```
def maFonction(a,b):
    return a+b
```

- Comme précédemment, maintenez la touche [Maj] appuyée et simultanément appuyez sur la touche [Entrée] afin d'exécuter le code de cette cellule. Bien sûr, cela ne produit aucun résultat si ce n'est l'enregistrement de la fonction `maFonction`.

© Editions ENI - All rights reserved

- Dans la nouvelle cellule, qui est automatiquement créée, commencez à saisir l'instruction Python qui permet d'appeler la fonction `maFonction` avec les deux arguments 1 et 3. Saisissez uniquement les deux lettres `ma`, puis appuyez sur la touche [Tab] de votre clavier. Cela affiche un menu contextuel dans lequel Jupyter vous propose une aide pour la saisie de votre code comme sur la figure ci-dessous :

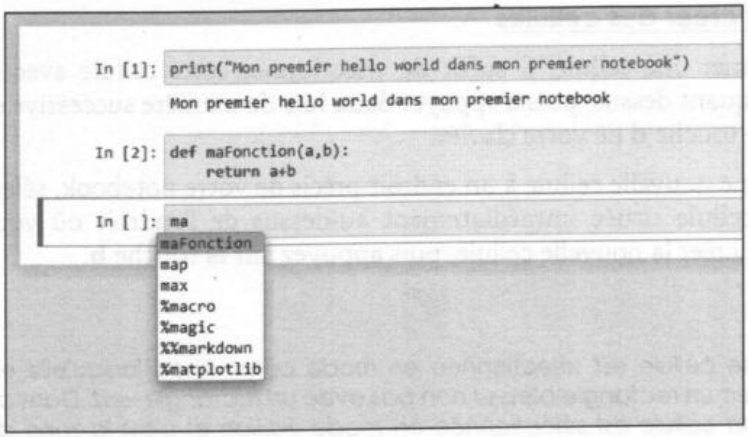


Figure 5-21 : Les cellules Jupyter et l'aide contextuelle

- Vous pouvez soit continuer à saisir par vous-même le reste de l'instruction, soit accepter l'aide proposée et sélectionner la bonne proposition parmi toutes celles présentes dans le menu contextuel affiché. À la fin, vous devez saisir l'instruction :

```
maFonction(1,3)
```

- Cette fois, nous allons exécuter cette dernière cellule en utilisant le menu des commandes Jupyter. Cliquez sur le menu **Cell**, puis sur le lien **Run Cells**. Cela exécute la cellule sélectionnée qui est, dans notre cas, la cellule d'appel à notre fonction `maFonction`. Le résultat est affiché immédiatement sous cette cellule.

Remarque

Comme dans la plupart des logiciels, vous remarquez que dans un notebook Jupyter toutes les opérations que nous réalisons avec la souris ou avec des raccourcis clavier peuvent être effectuées via les différents menus disponibles en haut de l'interface d'un notebook. Ainsi, l'exécution d'une cellule peut être réalisée avec la touche [Maj] et la touche [Entrée] ou via le menu ***Cell****.*

Supprimer et créer des cellules

- Pour supprimer une cellule, il suffit de sélectionner cette cellule avec la souris en cliquant dessus, puis d'appuyer deux fois de manière successive et rapide sur la touche **d** de votre clavier.
- Pour créer une nouvelle cellule à un endroit précis de votre notebook, sélectionnez la cellule située immédiatement au-dessus de l'endroit où vous souhaiteriez créer la nouvelle cellule, puis appuyez sur la touche **b**.

Remarque

Attention, une cellule est sélectionnée en mode commande lorsqu'elle est entourée avec un rectangle bleu et non pas avec un rectangle vert. Dans ce dernier cas, la cellule est sélectionnée en mode édition et c'est la zone de texte qui est activée afin de recevoir du code ou du texte.

Remarque

Les fonctions de suppression et de création de nouvelles cellules peuvent être réalisées respectivement via le menu ***Edit*** *et le menu* ***Insert****.*

Entrer du texte avec les Markdown

- Créez une nouvelle cellule.
- Sélectionnez cette nouvelle cellule et cliquez sur le menu **Cell**, puis cliquez sur le sous-menu **Cell Type** et sélectionnez la fonction **Markdown**.
- Saisissez le texte suivant dans la nouvelle cellule :

```
**Titre**

* point 1
* point 2
* point 3
```

© Editions ENI - All rights reserved

```
$\sqrt(2)$
```

Exécutez cette cellule. Vous devriez obtenir le même résultat que celui de la figure ci-dessous :

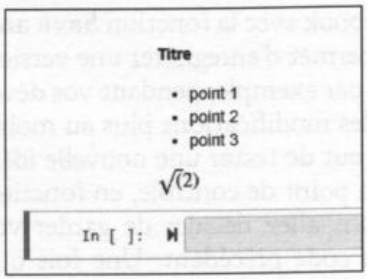


Figure 5-22 : Utilisation des Markdown dans un notebook Jupyter

Le type de cellule avec des Markdown sert principalement à documenter le code et les résultats d'un notebook. C'est très pratique pour des présentations, par exemple !

3.2.3 Les fonctionnalités d'un notebook

La section précédente a abordé les fonctionnalités de base utilisées lorsqu'on manipule un notebook Jupyter. Même si ce chapitre ne se veut pas être un manuel technique d'utilisation des notebooks Jupyter, cette section donne un aperçu des fonctionnalités que vous allez souvent utiliser et qui sont disponibles via les menus d'un notebook. L'idée est de vous donner une brève introduction de ces fonctions afin que vous ayez conscience de leur existence et de les utiliser au besoin.

Les fonctionnalités les plus utilisées et qui sont disponibles via les menus de notebook Jupyter sont les suivantes :

- **File** : ce menu regroupe les fonctionnalités qui s'appliquent sur les fichiers associés aux notebooks. Ainsi, avec les fonctionnalités de ce menu, vous pouvez :
  - créer une copie d'un notebook avec la fonction **Make a Copy**,
  - renommer le notebook avec la fonction **Rename**.

- télécharger un notebook dans un format parmi ceux disponibles, comme le format PDF ou le format HTML. Cette fonction est intéressante par exemple lorsque vous souhaitez envoyer les résultats de vos développements à une autre personne,
- enregistrer un point de contrôle d'un notebook avec la fonction **Save and Checkpoint**. Cette fonctionnalité vous permet d'enregistrer une version d'un notebook afin d'y revenir au besoin; par exemple, pendant vos développements, si vous souhaitez apporter des modifications plus au moins importantes à votre code Python dans le but de tester une nouvelle idée. Grâce à cette fonction de sauvegarde d'un point de contrôle, en fonction des résultats de votre nouveau code, vous allez décider de garder vos modifications ou bien de revenir à votre code précédent. Une fois que vous avez appliqué la fonction **Save and Checkpoint** sur un notebook, la version actuelle de ce notebook est enregistrée, et lorsque vous souhaitez revenir à cette version, vous devez utiliser la fonction **Revert to Checkpoint**.

- **Edit** : les fonctionnalités de ce menu concernent des opérations qu'on peut réaliser sur les cellules d'un notebook telles que les opérations de couper/copier, les opérations de suppression et de retour en arrière et les opérations qui permettent de fragmenter une cellule qui contient plusieurs instructions Python pour aboutir à des cellules de plus petite taille, ou encore effectuer des opérations de recherche avec des mots-clés. La plupart des fonctionnalités de ce menu sont similaires aux opérations que l'on retrouve dans le menu **Edit** classique des logiciels d'édition de code ou simplement d'édition de texte.
- **View** : ce menu permet de personnaliser l'aspect visuel de l'IHM (interface homme-machine) de l'application Jupyter, comme par exemple cacher/afficher les numéros de lignes dans les cellules.
- **Insert** : les fonctionnalités de ce menu permettent de créer de nouvelles cellules dans des endroits précis d'un notebook. La fonction **Insert Cell Above** crée une nouvelle cellule au-dessus de la cellule actuellement sélectionnée, tandis que la fonction **Insert Cell Below** crée une nouvelle cellule en dessous de la cellule actuellement sélectionnée.

© Editions ENI - All rights reserved

- **Cell** : les fonctionnalités de ce menu concernent l'exécution du code contenu dans des cellules, la typologie des cellules et l'affichage des résultats de l'exécution des cellules. Les fonctions les plus utilisées de ce menu sont les suivantes :
  - **Run Cells** : cette fonction lance l'exécution des cellules sélectionnées.
  - **Run All** : cette fonction lance l'exécution de toutes les cellules d'un notebook.
  - **Cell Type** : ce sous-menu permet de définir le type de la cellule actuellement sélectionnée. Par défaut, le type d'une cellule est le type Code.
  - **Current Outputs** : ce sous-menu contient des fonctionnalités qui concernent l'affichage des résultats associés aux exécutions du code des cellules.

    La première fonction **Toggle** permet de cacher la partie résultat d'une cellule sélectionnée si cette partie est dans un état affiché, sinon elle affiche la partie résultat si celle-ci est dans un état caché.

    La deuxième fonction **Toggle Scrolling** permet de cacher les deux ascenseurs horizontal et vertical lorsqu'ils sont affichés dans la partie résultat d'une cellule sélectionnée ; sinon, si ces deux ascenseurs sont cachés, alors cette fonction permet de les afficher si besoin est. En effet, ces deux ascenseurs ne seront affichés que si la taille nécessaire à la partie résultat d'une cellule est suffisamment grande pour qu'il y ait besoin de ces ascenseurs.

    La troisième fonction **Clear** permet d'effacer les résultats affichés dans la partie résultat d'une cellule sélectionnée.

Remarque

*Si vous effacez les résultats d'une cellule avec cette fonction **Clear**, alors pour afficher de nouveau les résultats attendus de l'exécution de cette même cellule, il faut exécuter son code, a contrario de la fonction **Toggle** qui permet de cacher puis de réafficher les résultats d'une cellule sans pour autant exécuter de nouveau le code de la cellule en question. La fonction **Clear** est intéressante lorsque vous apportez des modifications sur le code d'une cellule et que vous souhaitez visualiser le résultat de l'exécution du nouveau code. La fonction **Toggle** vous permet, par exemple, de présenter et d'expliquer le code d'une cellule ou de tout un notebook, puis de basculer vers un affichage où les résultats seront visibles, et cela sans refaire l'exécution du code contenu dans les cellules ; cela vous permet de réaliser des présentations fluides, surtout lorsque les cellules contiennent du code gourmand en matière de CPU.*

- **All Outputs** : ce menu contient exactement les mêmes fonctions que le menu **Current Outputs** à la différence que cette fois-ci les fonctions **Toggle**, **Toggle Scrolling** et **Clear** sont appliquées à l'ensemble des cellules d'un notebook et non pas seulement à la cellule actuellement sélectionnée.

- **Kernel** : le Kernel est un programme qui monitore et gère le code de vos notebooks. C'est ce programme qui gère tous les objets Python créés à la suite des exécutions des cellules. Vous pouvez utiliser le programme Kernel par défaut ou vous connecter à un autre programme Kernel, ou encore écrire votre propre Kernel et le connecter à l'application Jupyter. Bien sûr, dans ce livre, nous n'allons pas nous intéresser davantage au Kernel et nous allons nous contenter d'utiliser le Kernel par défaut. Cependant, les fonctions de ce menu les plus utilisées sont les suivantes :
  - **Interrupt** : cette fonction permet d'arrêter l'exécution d'une cellule anormalement longue ou tout simplement d'arrêter l'exécution d'une cellule, car vous souhaitez apporter une modification au code associé à cette cellule qui est en cours d'exécution.
  - **Restart** : cette fonction permet de réinitialiser le programme **Kernel**. Une fois que cette fonction **Restart** est exécutée, tous les objets créés lors des précédentes exécutions des cellules sont supprimés de la mémoire. Par exemple :

- Ouvrez le notebook *MonPremierNotebook.ipynb* et supprimez toutes les cellules déjà existantes.

- Saisissez puis exécutez l'instruction suivante dans une cellule :

```
n=4
```

- Puis saisissez et exécutez le code suivant dans une deuxième cellule :

```
print(n)
```

© Editions ENI - All rights reserved

Vous devriez obtenir le même résultat que celui de la figure ci-dessous :

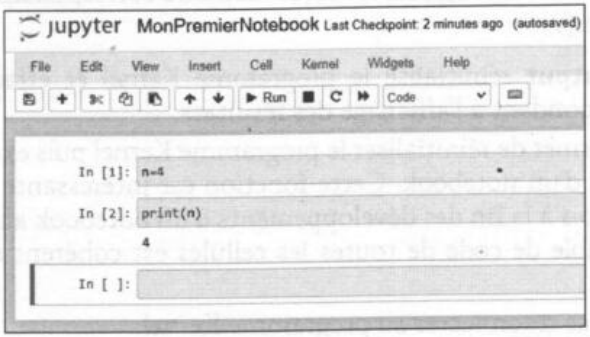


Figure 5-23 : Création et affichage d'une variable Python dans une cellule Jupyter

- Cliquez sur le menu **Kernel** puis lancez la fonction **Restart**.
- Cliquez sur le bouton **Restart** dans la boîte de dialogue de confirmation qui s'affiche.
- Sélectionnez la deuxième cellule qui contient l'instruction d'affichage de la variable n, puis exécutez cette cellule. Vous devriez obtenir un message d'erreur comme sur la figure ci-après.

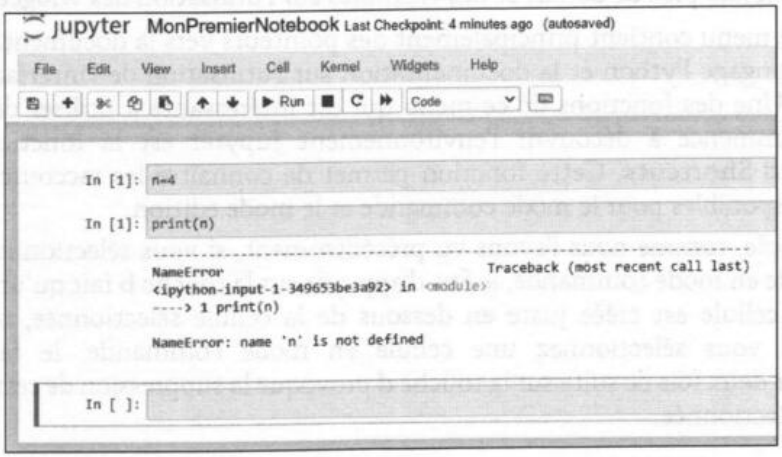


*Figure 5-24 : Le résultat de l'affichage d'une variable à la suite de l'exécution de la commande **Restart***

Ce message d'erreur vous indique que la variable n'est pas définie. En effet, à la suite de l'exécution de la fonction **Restart**, l'objet mémoire correspondant à la variable n a été supprimé.

- **Restart & Clear Output** réinitialise le programme Kernel et efface toutes les zones correspondant à l'affichage des résultats.
- **Restart & Run All** permet de réinitialiser le programme Kernel puis exécute toutes les cellules d'un notebook. Cette fonction est intéressante à utiliser au moins une fois à la fin des développements d'un notebook afin de vérifier que l'ensemble de code de toutes les cellules est cohérent et s'exécute correctement.
- **Reconnect** permet de se reconnecter au programme Kernel.
- **Shutdown** met fin à l'exécution du programme Kernel.
- **Change Kernel** permet de basculer d'un programme Kernel à un autre. Par défaut, vous ne disposez que de l'unique programme Kernel **Python 3**.

- **Widgets** : ce menu permet de gérer l'état mémoire associé aux widgets et également, avec la fonction **Embed Widgets**, nous pouvons télécharger le code HTML correspondant aux widgets utilisés dans un notebook, puis embarquer ce code HTML dans les pages d'une application web. La section suivante donne plus de détails et des exemples sur l'utilisation des widgets.
- **Help** : ce menu contient principalement des pointeurs vers la documentation du langage Python et la documentation sur l'utilisation de l'interface Jupyter. Une des fonctions de ce menu qui est intéressante à utiliser dès qu'on commence à découvrir l'environnement Jupyter est la fonction **Keyboard Shortcuts**. Cette fonction permet de connaître les raccourcis clavier disponibles pour le mode commande et le mode édition.

  Par exemple, comme nous l'avons vu précédemment, si vous sélectionnez une cellule en mode commande, le fait d'appuyer sur la touche **b** fait qu'une nouvelle cellule est créée juste en dessous de la cellule sélectionnée, ou encore, si vous sélectionnez une cellule en mode commande, le fait d'appuyer deux fois de suite sur la touche **d** provoque la suppression de cette cellule sélectionnée.

© Editions ENI - All rights reserved

3.3 Utiliser les widgets Jupyter

Les widgets sont des composants qui permettent d'enrichir les notebooks Jupyter afin de les rendre plus interactifs et plus dynamiques.

Dans cette section, nous allons découvrir quelques-uns de ces widgets. Pour cela, commencez par importer `ipywidgets` comme suit.

Ouvrez votre notebook *MonPremierNotebook.ipynb* et supprimez toutes les cellules déjà présentes, créez une nouvelle cellule, puis saisissez l'instruction suivante :

```
import ipywidgets as widgets
```

3.3.1 Le widget FloatSlider

Le widget `FloatSlider` permet d'afficher un intervalle de valeurs via un slider.

Remarque

Un slider est un composant graphique permettant de défiler l'affichage d'un ensemble de données. Les données peuvent être par exemple des images ou des nombres.

- Dans une nouvelle cellule Jupyter, saisissez le code suivant puis exécutez cette cellule.

```
widgets.FloatSlider(
    value=5,
    min=0,
    max=10.0,
    step=0.1,
    description='Slider:',
    orientation='horizontal',
    readout=True,
    readout_format='.1f',
)
```

L'attribut `value` permet de fixer la valeur initiale du slider; les deux attributs `min` et `max` permettent de fixer les bornes inférieure et supérieure de l'intervalle défini avec ce widget ; l'attribut `step` fixe le pas de déplacement dans l'intervalle ; l'attribut `description` affiche la description du widget ; l'attribut `orientation` fixe l'axe d'affichage du widget ; l'attribut `readout` indique si la valeur actuelle de l'intervalle doit être affichée ou pas ; enfin, l'attribut `readout_format` indique le format d'affichage des valeurs de l'intervalle.

Figure 5-25 : Affichage de l'intervalle `[0,10]` avec le widget `FloatSlider`

- Faites glisser vers la gauche ou vers la droite le bouton de commande de ce slider et observez le changement des valeurs affichées à droite du slider.

3.3.2 Associer une fonction à un slider

Dans le dernier exemple, nous avons utilisé un slider pour parcourir et afficher les valeurs d'un intervalle. Le plus intéressant serait d'accrocher notre slider à l'exécution d'une fonction ! L'exemple ci-dessous montre comment utiliser les valeurs d'un intervalle une par une afin d'exécuter une fonction spécifique et rendre l'exécution et l'affichage du résultat de cette fonction interactifs et dynamiques.

- Dans une nouvelle cellule Jupyter, saisissez le code suivant puis exécutez cette cellule :

```
slider = widgets.FloatSlider(
    value=5,
    min=0,
    max=10.0,
    step=0.1,
    disabled=False,
    description='x=:',
    orientation='horizontal',
    continuous_update=True,
    readout=True,
    readout_format='.1f',
)
```

© Editions ENI - All rights reserved

```
res_placeholder = widgets.Output()
def compute_x_2(change):
    with res_placeholder :
        res_placeholder.clear_output()
        print(f"x²={float(change['new'])**2}")
```

Ce code définit le slider `slider` comme nous l'avons fait précédemment, sauf que cette fois nous avons ajouté l'attribut `continuous_update=True` qui permet d'attacher à ce slider l'exécution d'une fonction de façon continue pendant que l'utilisateur glisse le bouton de commande de ce slider vers la gauche ou vers la droite.

Ce code définit également la variable `res_placeholder` associée à la zone d'affichage des widgets grâce à l'utilisation de la fonction `widgets.Output()`. Également, nous avons défini la fonction `compute_x_2` qui utilise la zone d'affichage `res_placeholder` pour afficher le résultat du carré de la valeur associée à la clé new du dictionnaire reçu en paramètre.

À ce stade, nous avons défini le slider `slider` d'un côté, et la fonction `compute_x_2` de l'autre.

▶Dans une nouvelle cellule Jupyter, saisissez le code suivant puis exécutez cette cellule :

```
slider.observe(compute_x_2, names='value')

display(slider,res_placeholder)
```

La fonction `observe` permet d'associer le slider `slider` avec la fonction `compute_x_2`. La deuxième instruction permet d'afficher le slider et la zone d'affichage des widgets. L'exécution de cette cellule affiche le slider initialisé à la valeur 5.

Si, par exemple, vous glissez le bouton de commande du slider tout à droite pour atteindre sa valeur maximale 10, vous obtenez le résultat de la figure suivante :

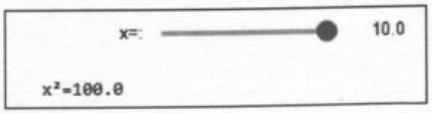


Figure 5-26 : Association d'un slider avec l'exécution d'une fonction

Remarquez la mise à jour dynamique de la zone d'affichage de ce widget au fur et à mesure que vous faites glisser le slider à gauche ou à droite.

Positionnez `continuous_update=False` dans la définition du slider `slider` et exécutez les deux dernières cellules, puis observez l'effet de cet attribut sur la zone d'affichage. Cette fois, vous constatez que la zone d'affichage du widget ne se met à jour qu'au moment où vous relâchez le bouton de commande du slider, c'est-à-dire que l'exécution de la fonction `compute_x_2` n'est relancée qu'au moment où vous arrêtez de bouger le slider !

Cet exemple a montré comment rendre l'exécution d'une fonction interactive et comment rendre l'affichage des résultats d'un programme plus dynamique. Ici, nous avons tout simplement affiché le résultat d'un calcul effectué sur les différentes valeurs renvoyées par un slider, mais nous aurions pu afficher la courbe de l'évolution de x^2 par rapport à x ou afficher le résultat d'une tout autre fonction plus complexe !

3.3.3 Le widget interact

Dans l'exemple précédent, nous avons vu comment associer l'exécution d'une fonction aux variations des valeurs d'un slider. De la même manière, nous pouvons associer l'exécution d'une fonction avec un widget `Play` afin d'exécuter cette fonction en utilisant les valeurs d'un intervalle de façon complètement automatisée. Ainsi, nous pouvons créer des animations pour illustrer le déroulement des étapes d'un programme. Dans l'exemple suivant, nous allons automatiser l'exécution d'une fonction qui calcule le cube de son paramètre en utilisant les valeurs d'un intervalle.

- Dans une nouvelle cellule Jupyter, saisissez le code suivant puis exécutez cette cellule :

```
from ipywidgets import interact
play = widgets.Play(
    value=1,
    min=1,
    max=20,
    step=1,
    interval=100,
    description="Test animation",
)
```

© Editions ENI - All rights reserved

Ce code crée l'objet `play` de type `widgets.Play`. Les attributs de l'objet `play` ont la même signification que les attributs de l'objet `FloatSlider` que nous avons vu précédemment. L'attribut `interval` permet de définir le temps de pause nécessaire entre le passage d'une valeur à une autre de l'intervalle `[1,20]`.

- Dans une nouvelle cellule Jupyter, saisissez le code suivant puis exécutez cette cellule :

```
def print_X_3(x):
    print(f"{x}**3 = {x**3}")
```

Ce code définit une fonction qui prend un argument et affiche le résultat du cube de cette valeur.

- Dans une nouvelle cellule Jupyter, saisissez l'instruction suivante, puis exécutez cette cellule :

```
interact(print_X_3,x=play)
```

- Cette instruction permet d'utiliser le widget `Interact` pour associer et exécuter la fonction `print_X_3` avec l'objet `play`. Vous obtenez le résultat de la figure suivante :

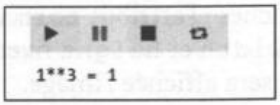


Figure 5-27 : Exemple d'utilisation du widget `Interact`

En cliquant sur le bouton symbolisé par le triangle orienté vers la droite, les valeurs de l'intervalle `[1, 20]` vont être utilisées une à une avec la fonction `print_X_3` et le résultat sera affiché à chaque fois, en espaçant les affichages de 100 millisecondes.

Remarque

Dans ces deux derniers exemples, lorsque nous avons utilisé les widgets `FloatSlider` et `Interact`, nous avons volontairement choisi des exemples basiques afin d'illustrer simplement leur utilisation. Dans la pratique, il est plus intéressant d'utiliser ces widgets avec des fonctions plus complexes, comme l'affichage de graphiques portant sur des données afin d'analyser et de comprendre les résultats d'un algorithme.

3.3.4 Le widget Image

Le widget `Image` permet de charger une image dans un notebook. C'est très intéressant, notamment lorsque votre présentation ou votre exposé nécessite des images afin d'expliquer des notions relatives au sujet traité dans votre notebook.

- Copiez le dossier *Exercices* contenant toutes les ressources de ce livre dans le dossier racine de Jupyter. Il doit être situé au même niveau que le dossier *TestJupyter* que vous avez créé au début de ce chapitre.
- Dans une nouvelle cellule Jupyter, du notebook *MonPremierNotebook.ipynb* qui, pour rappel, est contenu dans le dossier *TestJupyter*, saisissez le code suivant, puis exécutez cette cellule :

```
file = open("../Exercices/Chapitre_05/robot.jpg", "rb")
image = file.read()
widgets.Image(
    value=image,
    format='jpg',
    width=400,
    height=300,
)
```

L'attribut `value` indique le chemin vers l'image à afficher ; l'attribut `format` renseigne le type du fichier à afficher et les attributs `width` et `height` fixent respectivement la largeur et la hauteur de la zone où sera affichée l'image.

Après l'exécution de la cellule précédente, vous devriez voir afficher l'image *robot.jpg*.

3.3.5 Le widget DatePicker

Nous allons terminer la présentation des widgets avec un exemple de widget `DatePicker` qui permet de demander à un utilisateur de renseigner une date de manière interactive.

- Dans une nouvelle cellule Jupyter, saisissez le code suivant, puis exécutez cette cellule :

```
widgets.DatePicker(
    description='Date',
    disabled=False)
```

© Editions ENI - All rights reserved

Ce widget permet de choisir une date à partir d'un calendrier comme sur la figure ci-dessous :

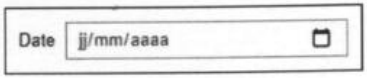


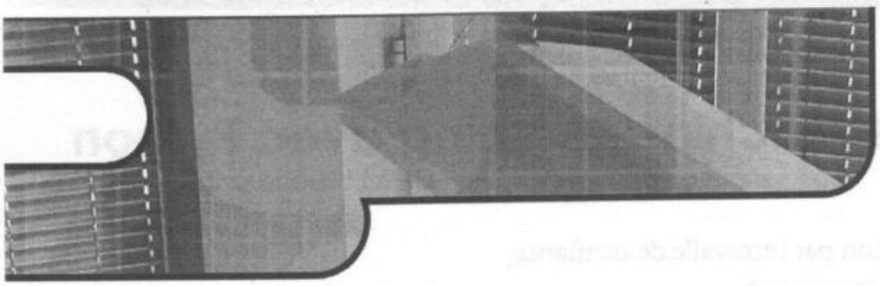
Figure 5-28 : Affichage du widget `DatePicker`

Remarque

Pour conclure ce chapitre de présentation de Jupyter, nous pouvons dire que, contrairement aux environnements de développement classiques, les notebooks Jupyter proposent nativement de sauvegarder les résultats des exécutions des programmes pour y revenir plus tard ou tout simplement pour montrer ces résultats lors d'une présentation. Cela peut éviter des heures de travail fastidieuses de préparation des supports de présentation lorsque nous souhaitons restituer les résultats de nos programmes Python.

4. Conclusion

Ce chapitre nous a permis d'aborder les notebooks Jupyter qui est l'un des environnements de travail préférés des Data Scientists. Nous avons commencé par l'installation de la distribution Anaconda, puis nous avons vu comment manipuler les documents et les notebooks Jupyter, et enfin nous avons découvert les widgets qui permettent de rendre un notebook Jupyter interactif et dynamique. Avant d'attaquer les algorithmes du Machine Learning, le chapitre suivant aborde les notions de statistiques de base nécessaires pour une meilleure compréhension des concepts sous-jacents à la Data Science.



Partie 3
Les statistiques

Chapitre 3-1
Statistiques

1. Objectif du chapitre

Les statistiques regroupent un ensemble de méthodes dédiées à l'échantillonnage de données ainsi qu'à leur analyse afin de tirer des conclusions et de comprendre les phénomènes sous-jacents à ces données. Ces méthodes statistiques font partie intégrante de la Data Science.

Il est quasiment impossible d'aborder l'ensemble des méthodes statistiques en un seul ouvrage vu leur diversité. Il existe plusieurs livres qui traitent des statistiques. L'objectif de ce chapitre est double : le premier est la présentation des outils statistiques élémentaires que tout Data Scientist devrait connaître, et le deuxième objectif est d'attirer l'attention du lecteur sur l'intérêt des statistiques et leur relation avec la Data Science. Ainsi, nous allons porter une attention particulière à la partie inférentielle des statistiques.

À la fin de ce chapitre, le lecteur aura abordé :

- les statistiques descriptives,
- les lois de probabilité,
- la loi normale et la loi normale centrée réduite,
- le principe de l'échantillonnage,
- le théorème central limite,
- l'estimation ponctuelle,

- l'estimation par intervalle de confiance,
- les tests d'hypothèses,
- le paradoxe de Simpson.

2. Les statistiques descriptives

Les statistiques descriptives permettent de résumer un ensemble de données de manière concise. Avec les statistiques descriptives, et pour un échantillon de valeurs $E = \{x_1, x_2, \dots x_n\}$, nous pouvons calculer certains paramètres afin de cerner la nature de la distribution associée aux valeurs x_i.

Ainsi, nous distinguons deux types de paramètres que nous pouvons calculer sur une série statistique de type quantitative : les paramètres de position et les paramètres de dispersion présentés dans les sous-sections suivantes.

2.1 Paramètres de position

Les paramètres de position permettent d'avoir une idée précise sur la nature du domaine de définition d'un ensemble de données. Ces paramètres de position, également appelés indicateurs de position, sont des nombres réels utilisés comme référence pour un ensemble $E = \{x_1, x_2, \dots x_n\}$.

2.1.1 La moyenne

La moyenne $\overline{X}$ associée à une série de valeurs $S = (x_1, x_2, \dots, x_n)$ se calcule comme suit $\overline{X} = \frac{1}{n}\sum_{i=1}^{n} x_i$.

La moyenne ainsi calculée correspond à la moyenne arithmétique. Il existe d'autres types de moyennes telles que la moyenne harmonique, la moyenne quadratique ou encore la moyenne géométrique. Généralement, en statistiques, la moyenne utilisée est la moyenne arithmétique.

© Editions ENI - All rights reserved

Remarque

Si la série S est un échantillon issu d'une population plus grande, alors il ne faut pas confondre la moyenne $\overline{X}$ calculée sur cet échantillon avec la moyenne de la population ! Dans la suite de ce chapitre, nous allons revenir sur la relation entre la moyenne d'un échantillon et la moyenne de la population.

2.1.2 Le mode

Le mode d'une série de valeurs est tout simplement la valeur qui apparaît le plus fréquemment.

Par exemple, dans la série de valeurs $S=(1, 2, 5, 2, 5, 5, 6, 8, 5, 9, 5)$, on dira que le mode est la valeur 5, car c'est bien cette valeur qui apparaît avec le plus d'occurrences. La valeur 5 apparaît cinq fois, la valeur 2 deux fois, puis les autres valeurs apparaissent une fois chacune.

La série S de notre exemple est dite unimodale, car elle dispose d'un seul mode.

La série $S^1=(1, 2, 5, 2, 5, 5, 2, 2, 5, 2, 5)$ est dite bimodale, car elle dispose de deux modes, à savoir le mode 5 et le mode 2.

2.1.3 La médiane

La médiane associée à une série de valeurs rangée dans l'ordre croissant $S = (x_1, x_2, \ldots, x_n)$ avec $x_i < x_{i+1}\ \forall\, i$ est une valeur qui partage toutes les valeurs de S en deux groupes de valeurs de taille égale. La valeur de la médiane peut ou pas faire partie de S.

Si les valeurs x_i de S sont des valeurs discrètes, alors la médiane se calcule comme suit :

- Si la taille n de la série S est impaire, alors nous pouvons écrire $n=2p+1$. Comme les valeurs de S sont ordonnées, la médiane est la $(p+1)^{\text{ième}}$ valeur. Dans ce cas, la médiane fait partie de la série S.

  Par exemple, pour la série $S=(1, 2, 5, 8, 15, 55, 68, 82, 125, 199, 500)$, la médiane est la valeur 55, alors que la moyenne est égale à 96,36.

- Si la taille n de la série S est paire, alors nous pouvons écrire $n=2p$. Comme les valeurs de S sont ordonnées, la médiane est la moyenne entre la $(p+1)^{ième}$ valeur et la $p^{ième}$ valeur. Dans ce cas, la médiane ne fait pas partie de la série S.

  Par exemple, pour la série $S=(1,2,5,8,15,55,68,82,125,199)$, la médiane est la valeur $\frac{15+55}{2} = 35$ et la moyenne est égale à 56.

Remarquez que dans des deux derniers exemples, les différences entre les moyennes et les médianes étaient importantes !

Si les valeurs x_i de S sont des valeurs continues, alors la médiane correspond à la valeur centrale que nous pouvons calculer par interpolation linéaire du centre des effectifs cumulés sur les x_i.

Par exemple, soit le tableau récapitulatif des notes obtenues par un groupe de 100 étudiants :

Notes	Effectifs
[0;5[	15
[5;7[	30
[7;11[	20
[11;16[	25
[16;20[	10
Total	100

À partir de ce tableau, nous allons procéder au calcul des effectifs cumulés comme suit :

Notes	Effectifs	Effectifs cumulés
[0;5[	15	15
[5;7[	30	45
[7;11[	20	65

© Editions ENI - All rights reserved

Notes	Effectifs	Effectifs cumulés
[11;16[	25	90
[16;20[	10	100

D'après la colonne des effectifs cumulés, nous avons :

- 15 étudiants qui ont une note en dessous de 5.
- 45 étudiants qui ont une note en dessous de 7.
- 65 étudiants qui ont une note en dessous de 11.
- 90 étudiants qui ont une note en dessous de 16.
- 100 étudiants qui ont une note en dessous de 20.

Le nombre total des étudiants est égal à 100, donc la moitié est égale à 50 étudiants. L'intervalle sur l'axe des effectifs cumulés où se situe la médiane est l'intervalle [45;65], puisque 50 appartient à cet intervalle.

Nous pouvons calculer la valeur de la médiane par interpolation linéaire comme sur la figure 6-1 suivante :

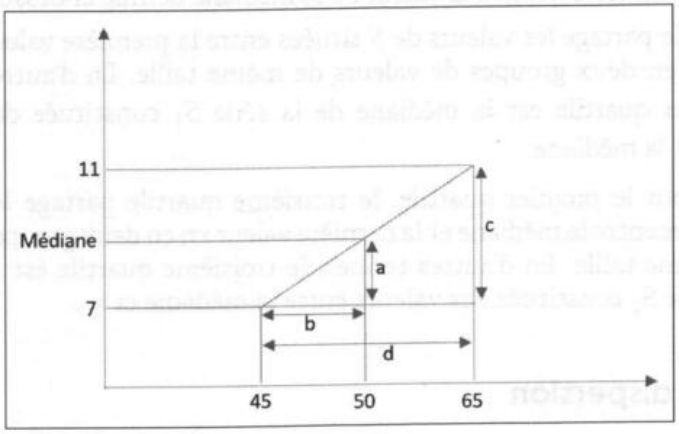


Figure 6-1 : Calcul de la médiane par interpolation linéaire du centre des effectifs cumulés

Grâce au théorème de Thalès, nous savons que $\frac{a}{b} = \frac{c}{d}$ (voir la figure ci-dessus).

a = médiane - 7.

b = 50 - 45 = 5.

c = 11 - 7 = 4.

d = 65 - 45=20.

En procédant au remplacement de a, b, c et d par leurs valeurs respectives, on obtient $\frac{médiane - 7}{5} = \frac{4}{20}$.

Donc la médiane est égale à 8.

2.1.4 Les quartiles

Pour une série de valeurs rangées dans l'ordre croissant $S = (x_1, x_2, \ldots, x_n)$ avec $x_i < x_{i+1}\ \forall\, i$, les quartiles sont définis par trois valeurs qui partagent les valeurs de la série S en quatre groupes de valeurs de même taille. Ces trois valeurs sont définies comme suit :

- Le deuxième quartile correspond à la valeur de la médiane définie ci-dessus.
- Le premier quartile partage les valeurs de S situées entre la première valeur x_1 et la médiane en deux groupes de valeurs de même taille. En d'autres termes, le premier quartile est la médiane de la série S_1 constituée des valeurs entre x_1 et la médiane.
- De même que pour le premier quartile, le troisième quartile partage les valeurs de S situées entre la médiane et la dernière valeur xn en deux groupes de valeurs de même taille. En d'autres termes, le troisième quartile est la médiane de la série S_2 constituée des valeurs entre la médiane et x_n.

2.2 Paramètres de dispersion

Les paramètres de dispersion permettent de comprendre la variabilité associée à une série de données quantitatives.

© Editions ENI - All rights reserved

2.2.1 La variance

La variance associée à un échantillon de données est une mesure qui nous renseigne sur la moyenne des carrés des écarts à la moyenne. On peut également dire que la variance est une valeur qui nous donne une idée sur la dispersion d'un ensemble de valeurs autour de leur moyenne.

Pour une série statistique $X = (x_1, x_2, ..., x_n)$, la variance $Var(X)$ se calcule comme suit : $Var(X) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{X})^2$.

Avec $\overline{X}$ la moyenne de la série X et qui, pour rappel, se calcule comme suit $\overline{X} = \frac{1}{n}\sum_{i=1}^{n} x_i$

À partir de la formule de $Var(X)$, on voit bien que la variance est une distance moyenne au carré qui sépare chaque élément x_i de sa moyenne $\overline{X}$.

2.2.2 Calcul de la variance avec la formule de Koenig

Le calcul de la variance $Var(X)$ avec la formule vue précédemment peut être compliqué à réaliser sans l'aide d'un logiciel. La formule de Koenig est une simplification de la formule précédente et qui permet de calculer la variance comme suit : $Var(X) = \frac{1}{n}\sum_{i=1}^{n} x_i{}^2 - \overline{X}^2$.

Avec cette formule, la variance est la différence entre la moyenne des carrés des x_i et le carré de la moyenne x_i.

Avec la formule de Koenig, le calcul de la variance devient plus facile et une simple calculatrice serait largement suffisante.

2.2.3 L'écart-type

Nous avons vu que la variance est la distance moyenne au carré d'un ensemble de valeurs par rapport à leur moyenne. L'écart-type σ est tout simplement la distance moyenne, ou la distance type, qui sépare les valeurs de leur moyenne et il se calcule à partir de la variance comme suit : $\sigma = \sqrt{Var(X)}$.

2.2.4 L'écart interquartile

L'écart interquartile correspond à la distance entre le troisième et le premier quartile.

Par exemple, pour la série $S=(1, 2, 5, 8, 15, 55, 68, 82,125, 199, 500)$, les quartiles sont les suivants :

- Le premier quartile est égal à 5.
- Le deuxième quartile est égal à 55, ce qui est également la médiane.
- Le troisième quartile est égal à 125.

Pour cette série S, l'espace interquartile est donc égal à 125-5 = 120.

La valeur de l'écart interquartile, aussi appelé l'espace interquartile ou bien l'étendue interquartile, nous renseigne sur l'ampleur de la dispersion des valeurs d'une série.

En combinant les mesures de la moyenne, du mode, de la médiane, de la variance, de l'écart-type et de l'écart interquartile, nous obtenons un résumé de l'ensemble des valeurs étudiées. Toutes ces mesures peuvent être calculées sur un ensemble de valeurs associées à une seule variable quantitative. Dans le chapitre Analyse en composantes principales, nous aborderons l'analyse en composantes principales, qui est une méthode permettant de résumer un ensemble de données définies avec plusieurs variables.

3. Les lois de probabilité

Une loi de probabilité permet de cerner le comportement d'une variable aléatoire. Dans le domaine des probabilités, une variable aléatoire dépend du hasard. Justement, c'est le comportement de ce hasard que l'on tente de décrire avec une loi de probabilité. Avec une loi de probabilité, nous pouvons calculer la probabilité qu'une variable aléatoire soit fixée à une valeur donnée.

Par exemple, si nous considérons une variable X associée au résultat obtenu après le lancer d'un dé à six chiffres, alors cette variable X sera appelée une variable aléatoire, puisque la survenue de l'un des six chiffres est un événement aléatoire.

© Editions ENI - All rights reserved

Si nous supposons que notre dé est parfait, c'est-à-dire que chacun des six chiffres est équiprobable avec une probabilité de $\frac{1}{6}$, alors la loi de la variable X est tout simplement $F(X) = \frac{1}{6}$.

Le choix d'une loi de probabilité est en fonction de la nature de la variable aléatoire étudiée et en fonction du phénomène associé à cette variable aléatoire. En effet, une variable aléatoire X peut être discrète ou continue et elle peut être définie dans un intervalle fini, semi-fini ou infini. Le phénomène associé à une variable aléatoire peut concerner des durées, des quantités comme le poids, des longueurs comme la taille ou toute autre mesure qui peut être observée et relative à un événement quelconque.

En résumé, une loi de probabilité décrit la manière dont sont distribuées les valeurs possibles d'une variable aléatoire X.

Il existe un nombre important de lois de probabilité. Loin d'être exhaustifs, parmi les plus usuelles nous pouvons citer :

- Loi de Bernoulli : décrit la distribution d'une variable aléatoire associée à un événement à deux issues possibles.
- Loi binomiale : décrit la distribution d'une variable aléatoire associée à un événement à deux issues possibles répété plusieurs fois avec remise, c'est-à-dire que le même événement peut survenir plusieurs fois.
- Loi de Poisson : décrit la distribution d'une variable aléatoire associée au nombre d'événements se produisant dans un intervalle de temps donné. Par exemple, le nombre de personnes supplémentaires dans une file d'attente au bout d'un temps donné.
- Loi uniforme : décrit la distribution d'une variable aléatoire associée à un ensemble d'événements équiprobables.
- Loi exponentielle : décrit la distribution d'une variable aléatoire associée au délai de la survenue d'un événement. Par exemple, le délai de l'arrivée d'une personne supplémentaire dans une file d'attente.

- Loi normale : est parmi les lois les plus utilisées dans le domaine de la modélisation en statistique. Elle est également appelée la courbe de Gauss ou encore la courbe en cloche. Cette loi permet de modéliser tout phénomène correspondant à la somme de plusieurs variables aléatoires indépendantes. Vu l'importance de cette loi, nous allons l'aborder en détail dans la sous-section suivante.
- Loi de Khi-carré : décrit la distribution d'une variable aléatoire associée à la somme de plusieurs lois normales.
- Loi de Student : décrit la distribution d'une variable aléatoire associée au quotient d'une variable distribuée suivant une loi normale et une variable distribuée suivant la loi de Khi-carré.

Remarque

Les lois de probabilité citées ci-dessus ne sont pas usuelles dans le sens qu'elles sont les plus rencontrées dans la pratique. Au contraire, dans les situations concrètes, nous pouvons rencontrer n'importe quelle loi de probabilité. Les lois citées ci-dessus sont, selon l'auteur, celles qui doivent absolument être connues et qui permettent de comprendre plus facilement bien d'autres lois de probabilité.

4. La loi normale

La loi normale permet de modéliser une distribution suivant une courbe de Gauss. Cette loi normale joue un rôle quasiment central en modélisation statistique du fait de ses relations avec plusieurs autres lois de probabilité, et aussi du fait du théorème central limite que nous allons aborder plus loin dans ce chapitre.

La distribution d'une variable aléatoire continue suivant la loi normale est décrite par la formule suivante : $F(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$.

Avec μ et σ respectivement la moyenne et l'écart-type de la variable aléatoire modélisée par cette formule.

© Editions ENI - All rights reserved

Lorsque la moyenne $\mu = 0$ et l'écart-type $\sigma = 1$, alors la formule précédente devient : $F(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$.

Et dans ce cas, nous parlons de loi normale centrée réduite. Cette loi normale centrée réduite est également appelée la loi normale standard.

De manière générale, pour désigner le fait qu'une variable aléatoire continue X suit une loi normale, nous utilisons la notation suivante : $X \sim \aleph(\mu, \sigma)$.

Pour un μ donné et un σ donné, nous pouvons tracer la courbe associée à la loi normale. Comme il y a une infinité de possibilités pour μ et σ, l'utilisation de la loi normale centrée réduite, avec $\mu = 0$ et $\sigma = 1$, devient intéressante, car les aires sous la courbe de la loi normale sont les mêmes, quelles que soient les valeurs de ces paramètres μ et σ.

Ainsi, le tracé de la courbe de la loi normale est sous la forme de la courbe de la figure suivante :

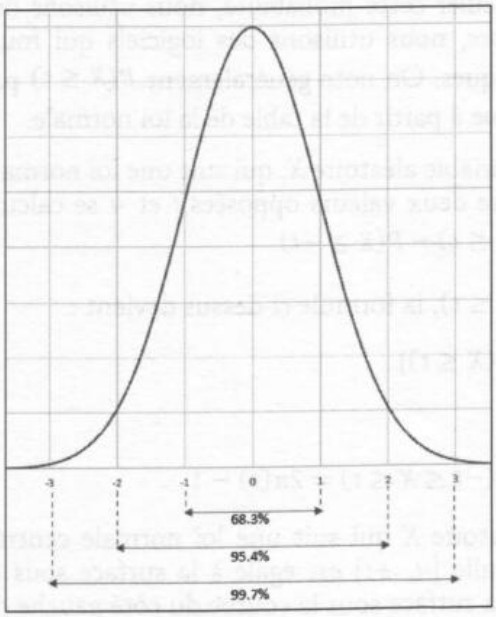


Figure 6-2 : Distribution d'une variable suivant la loi normale centrée réduite

On voit que la courbe de la distribution normale centrée réduite est symétrique autour de la moyenne $\mu = 0$.

La surface totale sous cette courbe est égale à 1 puisqu'elle représente une distribution de probabilités. Les valeurs sur l'axe des abscisses sont exprimées en écart-type, puisque nous avons représenté une version réduite de la loi normale.

La particularité d'une variable distribuée selon la loi normale est que 68,3 % de ses valeurs vont être comprises dans l'intervalle $[m - \sigma, m + \sigma]$, ce qui correspond à l'intervalle [-1;1] dans le cas d'une loi normale centrée réduite. Également, pour une telle variable aléatoire, 95,4 % des valeurs vont être comprises dans l'intervalle $[m - 2\sigma, m + 2\sigma]$, ce qui correspond à l'intervalle [-2;+2] dans le cas d'une loi normale centrée réduite et 99,7 % des valeurs vont être dans l'intervalle $[m - 3\sigma, m + 3\sigma]$, ce qui correspond à l'intervalle [-3;+3] dans le cas d'une loi normale centrée réduite.

La probabilité $P(X \leq t)$ se lit directement de la courbe en cloche associée à la loi normale. En pratique, pour calculer cette probabilité, nous utilisons des tables prédéfinies ou, mieux encore, nous utilisons des logiciels qui fournissent des fonctionnalités statistiques. On note généralement $P(X \leq t)$ par $\pi(t)$ et qui correspond à la valeur lue à partir de la table de la loi normale.

Également, la probabilité qu'une variable aléatoire X, qui suit une loi normale centrée réduite, soit comprise entre deux valeurs opposées t et $-t$ se calcule comme suit : $P(-t \leq X \leq t) = P(X \leq t) - P(X \geq -t)$

Sachant que $P(X \geq -t) = 1 - P(X \leq t)$, la formule ci-dessus devient :

$$P(-t \leq X \leq t) = P(X \leq t) - [1 - P(X \leq t)]$$

$$P(-t \leq X \leq t) = 2P(X \leq t) - 1$$

Ce que nous pouvons noter par : $P(-t \leq X \leq t) = 2\pi(t) - 1$.

La probabilité qu'une variable aléatoire X qui suit une loi normale centrée réduite soit comprise dans l'intervalle $[-t, +t]$ est égale à la surface sous la courbe du côté gauche de t moins la surface sous la courbe du côté gauche de $-t$ (voir la figure suivante) :

© Editions ENI - All rights reserved

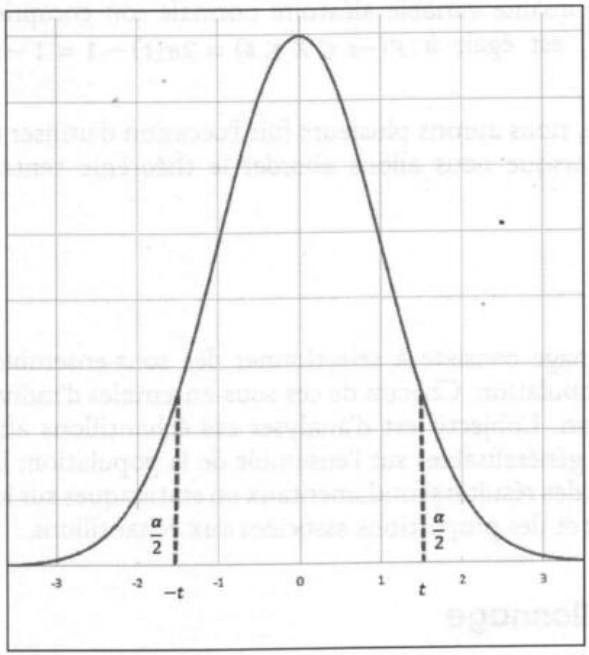


Figure 6-3: Probabilité d'appartenir à un intervalle [-t, t] d'une distribution normale

Dans cette dernière figure, on voit que la surface correspondant à la probabilité qu'une variable aléatoire appartienne à l'intervalle $[-t, +t]$ est égale à 1 moins la surface à droite de t et moins la surface à gauche de $-t$, ce qui est égal à α suivant notre figure 6-3.

On peut dire que dans cette figure la valeur de α correspond à un risque d'erreur, puisque même si $P(-t \leq X \leq t)$ est égale à la surface sous la courbe comprise à gauche de t et à droite de $-t$, il y a tout de même le risque que cette variable soit à l'extérieur de cette zone !

Et la probabilité associée à ce risque est égale à la surface sous la courbe située à droite de t, c'est-à-dire $\frac{\alpha}{2}$, plus la surface sous la courbe située à gauche de $-t$, également égale à $\frac{\alpha}{2}$. Cette dernière surface est notée dans la littérature par α.

En synthèse, la probabilité qu'une variable aléatoire normale soit comprise dans un intervalle [-t, +t] est égale à : $P(-t \leq X \leq t) = 2\pi(t) - 1 = 1 - \alpha$ (avec α un risque d'erreur)

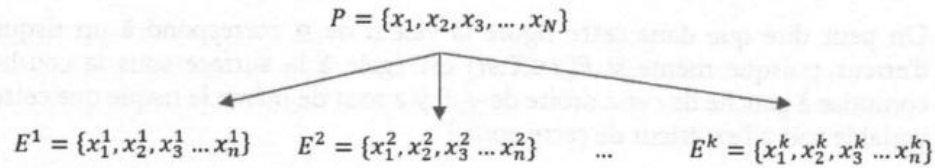
Dans la suite de ce chapitre, nous aurons plusieurs fois l'occasion d'utiliser la loi normale, notamment lorsque nous allons aborder le théorème central limite.

5. L'échantillonnage

La procédure d'échantillonnage consiste à sélectionner des sous-ensembles d'individus à partir d'une population. Chacun de ces sous-ensembles d'individus constitue un échantillon. L'objectif est d'analyser ces échantillons afin d'aboutir à des conclusions généralisables sur l'ensemble de la population. La suite de cette section donne des résultats fondamentaux en statistiques sur les distributions des moyennes et des proportions associées aux échantillons.

5.1 Principe de l'échantillonnage

Concrètement, l'action de l'échantillonnage consiste à réaliser des tirages d'individus à partir d'une population. Soit la population $P = \{x_1, x_2, x_3, \dots, x_N\}$ avec $x_i, i \in \{1,2,3, \dots N\}$ les individus de cette population et N le nombre d'individus de l'ensemble de la population. Sur cette population P, des échantillons peuvent être sélectionnés comme suit :



Avec E^q l'échantillon numéro q réalisé sur la population P, x_n^q le $n^{\text{ième}}$ individu de l'échantillon E^q et $\forall j, \forall i : (x_i^j \in P)$.

© Editions ENI - All rights reserved

Le calcul des moyennes, le calcul des écarts-types et le calcul des proportions avec un critère donné sur les échantillons $E^q, q \in \{1,2,3,...k\}$ permettent d'avoir une idée sur la répartition des données dans la population P.

Remarque

Dans la population P, le nombre d'individus, symbolisé par l'indice N dans la définition de la population ci-dessus, peut être fini ou infini. Si le nombre d'individus dans la population P est fini, alors cette population peut être assimilée à une population infinie lors de l'échantillonnage en utilisant des tirages successifs avec remise. En d'autres termes, après chaque tirage d'un individu x_j^i, avant de procéder au tirage du prochain individu, il faut d'abord remettre dans la population cet individu x_j^i de telle sorte qu'il puisse de nouveau être tiré avec la même probabilité que tous les autres individus.

5.2 Résultats sur la distribution des moyennes

Soit μ la moyenne de la population P et σ son écart-type. Si P est une population infinie ou assimilée à une population infinie en utilisant les tirages avec remise, et si $\overline{X_E}$ est la variable aléatoire associée aux moyennes des échantillons $\{\overline{X_{E^1}}, \overline{X_{E^2}}, \overline{X_{E^3}}, ..., \overline{X_{E^q}}, ..., \overline{X_{E^k}}\}$, avec $\overline{X_{E^q}} = \frac{1}{n}\sum_{i=1}^{n} x_i^q$ la moyenne du $q^{\text{ième}}$ échantillon, alors nous obtenons les résultats ci-dessous :

- L'espérance mathématique de la variable aléatoire $\overline{X_E}$ est égale à : $E(\overline{X_E}) = \mu$
- L'écart-type de la variable aléatoire $\overline{X_E}$ est le suivant $Ecart_type(\overline{X_E}) = \frac{\sigma}{\sqrt[2]{n}}$ (1)

  Avec n la taille des échantillons $E^q, q \in \{1,2,3,...k\}$ et σ l'écart-type de la population.

Si P est une population finie et que les tirages des échantillons sont réalisés sans remise, alors l'écart-type de la distribution des moyennes sera calculé avec la formule suivante : $Ecart_type(\overline{X_E}) = \frac{\sigma}{\sqrt[2]{n}} \sqrt[2]{\frac{N-n}{N-1}}$ (2)

Avec N la taille de la population, n la taille des échantillons et σ l'écart-type de la population. Dans la formule (2), le terme $\sqrt[2]{\frac{N-n}{N-1}}$ est appelé le coefficient d'exhaustivité.

Remarque

Remarquez que dans le résultat (2), lorsque N tend vers l'infini, $N \longrightarrow +\infty$, alors : $\sqrt[2]{\frac{N-n}{N-1}} \longrightarrow 1$, *donc* $Lim_{N\longrightarrow+\infty}\, Ecart_type(\overline{X_E}) = Lim_{N\longrightarrow+\infty} \frac{\sigma}{\sqrt[2]{n}} \sqrt[2]{\frac{N-n}{N-1}} = \frac{\sigma}{\sqrt[2]{n}}$, *ce qui est le même résultat que dans (1).*

Exemples numériques

Exemple avec une population finie et tirage avec remise

Soit une population P composée des individus suivants $P=\{1,2,3,4,5\}$. Le tableau ci-dessous donne tous les tirages d'échantillons E^q possibles et de taille 2 avec remise afin d'assimiler la population P à une population infinie ainsi que la moyenne associée à chaque échantillon. Nous avons choisi volontairement d'utiliser tous les échantillons possibles de la population P de taille 2 afin de vérifier la validité des deux équations (1) et (2) ci-dessus.

Échantillon	Prélèvement 1	Prélèvement 2	Moyenne
E^1	1	1	1
E^2	1	2	1.5
E^3	1	3	2
E^4	1	4	2.5
E^5	1	5	3
E^6	2	1	1.5
E^7	2	2	2
E^8	2	3	2.5
E^9	2	4	3

© Editions ENI - All rights reserved

Échantillon	Prélèvement 1	Prélèvement 2	Moyenne
E^{10}	2	5	3.5
E^{11}	3	1	2
E^{12}	3	2	2.5
E^{13}	3	3	3
E^{14}	3	4	3.5
E^{15}	3	5	4
E^{16}	4	1	2.5
E^{17}	4	2	3
E^{18}	4	3	3.5
E^{19}	4	4	4
E^{20}	4	5	4.5
E^{21}	5	1	3
E^{22}	5	2	3.5
E^{23}	5	3	4
E^{24}	5	4	4.5
E^{25}	5	5	5

Échantillonnage de la population $\boldsymbol{P=\{1,2,3,4,5\}}$ *avec remise*

Calcul de la moyenne de la population P et des moyennes $\overline{X_E}$:

– Calcul de la moyenne de la population P :

La moyenne de la population P se calcule comme suit $\mu = \frac{1+2+3+4+5}{5} = \frac{15}{5} = 3$.

– Calcul de la moyenne de la distribution associée à la variable aléatoire $\overline{X_E}$:

Le calcul de la moyenne des moyennes des 25 échantillons, c'est-à-dire la moyenne de la colonne "Moyenne" du tableau ci-dessus, donne le résultat suivant $\overline{X_E} = \frac{1}{25}\sum_{q=1}^{25}\overline{X_{Eq}} = 3$.

Donc, la moyenne de la population est égale à la moyenne de la variable aléatoire $\overline{X_E}$ associée aux moyennes des échantillons.

Calcul de l'écart-type de la population P et de l'écart-type de la variable $\overline{X_E}$:

– Nous allons commencer par le calcul de l'écart-type de la population P :

Après l'application de la formule de Koenig sur la population $P=\{1,2,3,4,5\}$, nous obtenons $\sqrt[2]{\frac{\sum_{i=1}^{5} x_i^2}{5} - E(x_i)^2} = \sqrt[2]{\frac{1+4+9+16+25}{5} - 3^2} = \sqrt[2]{11-9} = \sqrt[2]{2}$

– Calcul de l'écart-type de la distribution associée à la variable aléatoire $\overline{X_E}$:

Pour calculer l'écart-type de $\overline{X_E}$, nous allons également utiliser la formule de Koenig. Le tableau suivant reprend la colonne des moyennes du tableau précédent et donne également la colonne $\overline{X_E}^2$.

Échantillon	Moyenne=$\overline{X_E}$	$\overline{X_E}^2$
E^1	1	1
E^2	1.5	2.25
E^3	2	4
E^4	2.5	6.25
E^5	3	9
E^6	1.5	2.25
E^7	2	4
E^8	2.5	6.25

© Editions ENI - All rights reserved

Échantillon	Moyenne $=\overline{X_E}$	$\overline{X_E}^2$
E^9	3	9
E^{10}	3.5	12.25
E^{11}	2	4
E^{12}	2.5	6.25
E^{13}	3	9
E^{14}	3.5	12.25
E^{15}	4	16
E^{16}	2.5	6.25
E^{17}	3	9
E^{18}	3.5	12.25
E^{19}	4	16
E^{20}	4.5	20.25
E^{21}	3	9
E^{22}	3.5	12,25
E^{23}	4	16
E^{24}	4.5	20.25
E^{25}	5	25

Table des $\overline{X_E}^2$

À partir de la colonne $\overline{X_E}^2$, l'écart-type peut être calculé comme suit :

$$Ecart_type(\overline{X_E}) = \sqrt[2]{\frac{\sum_{q=1}^{k} \overline{X_{E^q}}^2}{k} - E(\overline{X_E})^2}$$

$$= \sqrt[2]{\frac{\sum_{q=1}^{25} \overline{X_{E^q}}^2}{25} - E(\overline{X_E})^2}$$

$$= \sqrt[2]{\frac{250}{25} - 3^2}$$

$$= \sqrt[2]{10 - 9} = 1$$

Le résultat de l'équation (1) se vérifie puisque, pour rappel $Ecart_type(\overline{X_E}) = \frac{\sigma}{\sqrt[2]{n}}$

Avec n la taille de l'échantillon et σ l'écart-type de la population. En procédant au remplacement des valeurs de n et σ, nous obtenons : $Ecart_type(\overline{X_E}) = \frac{\sqrt[2]{2}}{\sqrt[2]{2}} = 1$, ce qui correspond à l'écart-type de $\overline{X_E}$

Exemple avec une population finie et tirage sans remise

Soit $P=\{1, 2, 3, 4, 5\}$ la même population que dans l'exemple précédent. Le tableau ci-dessous donne tous les tirages d'échantillons E^q possibles et de taille 2 sans remise.

Échantillon	Prélèvement 1	Prélèvement 2	Moyenne
E^1	1	2	1.5
E^2	1	3	2
E^3	1	4	2.5
E^4	1	5	3
E^5	2	1	1.5
E^6	2	3	2.5

© Editions ENI - All rights reserved

Échantillon	Prélèvement 1	Prélèvement 2	Moyenne
E^7	2	4	3
E^8	2	5	3.5
E^9	3	1	2
E^{10}	3	2	2.5
E^{11}	3	4	3.5
E^{12}	3	5	4
E^{13}	4	1	2.5
E^{14}	4	2	3
E^{15}	4	3	3.5
E^{16}	4	5	4.5
E^{17}	5	1	3
E^{18}	5	2	3.5
E^{19}	5	3	4
E^{20}	5	4	4.5

Échantillonnage de la population $\boldsymbol{P=\{1,2,3,4,5\}}$ *sans remise*

Puisque la population est la même que dans l'exemple précédent, nous avons :

- La moyenne de la population $\mu = 3$.
- L'écart-type de la population $\sigma = \sqrt[2]{2}$.

Calcul de la moyenne de la distribution associée à la variable aléatoire $\overline{X_E}$:

Le calcul de la moyenne des moyennes de tous les 20 échantillons, c'est-à-dire la moyenne de la colonne "Moyenne" du tableau ci-dessus, donne le résultat suivant $\overline{X_E} = \frac{1}{20} \sum_{q=1}^{20} \overline{X_{Eq}} = 3$

Donc, la moyenne de la population est égale à la moyenne de la variable aléatoire $\overline{X_E}$ associée aux moyennes des échantillons.

Calcul des écarts-types de la distribution associée à la variable aléatoire $\overline{X_E}$:

Pour calculer l'écart-type de $\overline{X_E}$, nous allons utiliser la formule de Koenig. Le tableau suivant reprend la colonne des moyennes du tableau précédent et donne également la colonne $\overline{X_E}^2$.

Échantillon	Moyenne=$\overline{X_E}$	$\overline{X_E}^2$
E^1	1.5	2.25
E^2	2	4
E^3	2.5	6.25
E^4	3	9
E^5	1.5	2.25
E^6	2.5	6.25
E^7	3	9
E^8	3.5	12.25
E^9	2	4
E^{10}	2.5	6.25
E^{11}	3.5	12.25
E^{12}	4	16

© Editions ENI - All rights reserved

Échantillon	Moyenne= $\overline{X_E}$	$\overline{X_E}^2$
E^{13}	2.5	6.25
E^{14}	3	9
E^{15}	3.5	12.25
E^{16}	4.5	20.25
E^{17}	3	9
E^{18}	3.5	12.25
E^{19}	4	16
E^{20}	4.5	20.25

Table des $\overline{\mathbf{X_E}}^2$ *pour calculer l'écart-type*

À partir de la colonne $\overline{X_E}^2$, l'écart-type peut être calculé comme suit :

$$Ecart_type(\overline{X_E}) = \sqrt[2]{\frac{\sum_{q=1}^{k} \overline{X_{Eq}}^2}{k} - E(\overline{X_E})^2}$$

$$= \sqrt[2]{\frac{\sum_{q=1}^{20} \overline{X_{Eq}}^2}{20} - E(\overline{X_E})^2}$$

$$= \sqrt[2]{\frac{195}{20} - 3^2}$$

$$= \sqrt[2]{0.75}$$

Le résultat de l'équation (2) se vérifie puisque, pour rappel :

$$Ecart_type(\overline{X_E}) = \frac{\sigma}{\sqrt[2]{n}} \sqrt[2]{\frac{N-n}{N-1}}$$

Avec $\sqrt[2]{\frac{N-n}{N-1}}$ le coefficient d'exhaustivité puisque la population est finie et les tirages sont réalisés sans remise. Le paramètre n correspond à la taille des échantillons, le paramètre N correspond à la taille de la population et σ correspond à l'écart-type de la population. En procédant au remplacement des valeurs de n, N et σ nous obtenons : $Ecart_type(\overline{X_E}) = \frac{\sqrt[2]{2}}{\sqrt[2]{2}} \sqrt[2]{\frac{5-2}{5-1}} = \sqrt[2]{\frac{3}{4}} = \sqrt[2]{0.75}$, ce qui correspond à l'écart-type de $\overline{X_E}$.

5.3 Résultats sur la distribution des proportions

Soit p la proportion d'individus de la population P possédant une caractéristique donnée ζ. Si nous considérons les échantillons E^q de taille n prélevés à partir de la population P, avec $q \in \{1,2,3,\ldots,k\}$, et si f_{E^q} est la fréquence d'individus possédant la caractéristique ζ dans le $q^{\text{ième}}$ échantillon, alors nous obtenons les résultats ci-dessous sur la distribution des fréquences $F_E = \{f_{E^1}, f_{E^2}, f_{E^3}, \ldots, f_{E^k}\}$:

– L'espérance mathématique de la variable aléatoire F est la suivante :

$E(F_E) = p$

– L'écart-type de la variable aléatoire F est le suivant :

$$Ecart_type(F_E) = \sqrt[2]{\frac{p(1-p)}{n}} \quad (3)$$

Exemple numérique

Cet exemple reprend les données de l'exemple précédent avec une population P composée des individus $P=\{1, 2, 3, 4, 5\}$. La caractéristique ζ qui sera considérée est le nombre de chiffres impairs présents dans la population et dans les échantillons. Le tableau ci-dessous donne tous les tirages d'échantillons E^q possibles et de taille 2 avec remise afin d'assimiler la population P à une population infinie ainsi que la proportion des chiffres impairs de chaque tirage.

© Editions ENI - All rights reserved

Échantillon	Prélèvement 1	Prélèvement 2	Fréquence (f_{E^i})
E^1	1	1	1
E^2	1	2	0.5
E^3	1	3	1
E^4	1	4	0.5
E^5	1	5	1
E^6	2	1	0.5
E^7	2	2	0
E^8	2	3	0.5
E^9	2	4	0
E^{10}	2	5	0.5
E^{11}	3	1	1
E^{12}	3	2	0.5
E^{13}	3	3	1
E^{14}	3	4	0.5
E^{15}	3	5	1
E^{16}	4	1	0.5
E^{17}	4	2	0
E^{18}	4	3	0.5
E^{19}	4	4	0
E^{20}	4	5	0.5
E^{21}	5	1	1
E^{22}	5	2	0.5
E^{23}	5	3	1

Échantillon	Prélèvement 1	Prélèvement 2	Fréquence (f_{E^i})
E^{24}	5	4	0.5
E^{25}	5	5	1

Échantillonnage de la population ***P={1,2,3,4,5}***
avec remise pour calculer les fréquences

Dans la population *P*, les nombres impairs sont {1,3,5}, donc $p = \frac{3}{5} = 0.6$ et l'écart-type $\sigma = \sqrt[2]{2}$ comme dans l'exemple précédent.

En considérant le résultat de l'équation (3), le calcul de la moyenne et de l'écart-type des proportions du tableau ci-dessus devrait donner les résultats suivants :

- $E(F_E) = p = 0.6$
- $Ecart_type(F_E) = \sqrt[2]{\frac{p(1-p)}{n}} = \sqrt[2]{\frac{0.6*0.4}{2}} = \sqrt[2]{0.12}$

Vérifions si ces deux derniers résultats peuvent être obtenus à partir des données du dernier tableau ci-dessus.

Calcul de la moyenne des fréquences :

En utilisant les données du tableau des fréquences ci-dessus, la moyenne est calculée comme suit :

$$\mu_F = \frac{\sum_{i=1}^{k} f_{E^i}}{k}$$

$= \frac{15}{25} = 0.6$ (=*p*, comme dans la population *P*)

Calcul de l'écart-type des fréquences :

Pour calculer l'écart-type des fréquences, la formule de Koenig sera utilisée. Le tableau suivant reprend les fréquences du tableau ci-dessus et ajoute la colonne ${f_{E^i}}^2$.

© Editions ENI - All rights reserved

Échantillon	Fréquence (f_{E^i})	$f_{E^i}{}^2$
E^1	1	1
E^2	0.5	0.25
E^3	1	1
E^4	0.5	0.25
E^5	1	1
E^6	0.5	0.25
E^7	0	0
E^8	0.5	0.25
E^9	0	0
E^{10}	0.5	0.25
E^{11}	1	1
E^{12}	0.5	0.25
E^{13}	1	1
E^{14}	0.5	0.25
E^{15}	1	1
E^{16}	0.5	0.25
E^{17}	0	0
E^{18}	0.5	0.25
E^{19}	0	0
E^{20}	0.5	0.25
E^{21}	1	1
E^{22}	0.5	0.25

Échantillon	Fréquence (f_{E^i})	${f_{E^i}}^2$
E^{23}	1	1
E^{24}	0.5	0.25
E^{25}	1	1

Table des $\boldsymbol{{f_{E^i}}^2}$ *pour le calcul de l'écart-type de* $\boldsymbol{F}$

$$Ecart_type(F_E) = \sqrt[2]{\frac{\sum_{i=1}^{k} {f_{E^i}}^2}{k} - {\mu_F}^2}$$

$$= \sqrt[2]{\frac{12}{25} - 0.6^2}$$

$$= \sqrt[2]{0.48 - 0.36}$$

$= \sqrt[2]{0.12}$ (ce qui correspond au résultat attendu).

5.4 Théorème central limite

Soit la population P={x_1,x_2,x_3,…,x_N}, μ sa moyenne, σ son écart-type et p une proportion de cette population qui possède une caractéristique donnée ζ. Si la distribution des x_i suit une loi de probabilité $\mathcal{L}$, alors le théorème central limite établit, entre autres, les deux résultats suivants :

– La série statistique constituée des valeurs prises par la variable aléatoire $\overline{X_E}$, qui est associée à la moyenne des échantillons E^i avec $i \epsilon \{1,2,3,\ldots,k\}$, suit une loi normale avec les paramètres suivants : $\overline{X_E} \sim \aleph(\mu, \frac{\sigma}{\sqrt{n}})$ avec n la taille des échantillons E^i.

© Editions ENI - All rights reserved

– La série statistique constituée des valeurs prises par la variable aléatoire F_E, qui est associée à la proportion d'individus dans les échantillons E^i avec $i \epsilon \{1,2,3,\ldots,k\}$ et qui possèdent la caractéristique ζ, suit une loi normale avec les paramètres suivants :

$F_E \sim \aleph(p, \sqrt{\frac{p(1-p)}{n}})$, avec n la taille des échantillons E^i.

Remarque

Attention, ces deux résultats sont valides dans la mesure où la taille des échantillons prélevés est supérieure à 30, c'est-à-dire n > 30. En revanche, dans le cas particulier où la population $P=\{x_1,x_2,x_3,\ldots,x_N\}$ elle-même suit une loi normale $\mathcal{L} = P \sim \aleph(\mu,\sigma)$, alors, même si la taille des échantillons prélevés est inférieure à 30, ces résultats restent valables.

6. Les statistiques inférentielles

Sans perte de généralités, les statistiques inférentielles regroupent différentes techniques qui permettent d'estimer certaines caractéristiques d'une population à partir de l'analyse d'un seul échantillon. Cette estimation est le processus inverse de l'échantillonnage. En effet, lorsque l'échantillonnage a été abordé à la section précédente, nous avons considéré une population $P=\{x_1,x_2,x_3,\ldots,x_N\}$, pour laquelle la moyenne μ, l'écart-type σ et la proportion p d'individus possédant une caractéristique donnée ζ sont connus. Grâce au théorème central limite et à partir de ces trois paramètres connus, nous avons conclu des résultats sur les deux distributions des deux variables aléatoires associées respectivement aux moyennes des échantillons prélevés à partir de la population P et aux fréquences d'individus possédant la caractéristique ζ dans ces mêmes échantillons. Avec l'estimation, nous allons considérer une population $P=\{x_1,x_2,x_3,\ldots,x_N\}$ pour laquelle les paramètres μ, σ et p ne sont pas connus, et justement, grâce aux notions que nous allons aborder dans cette section, nous allons pouvoir estimer leurs valeurs à partir d'un seul échantillon. C'est en ce sens que l'estimation est le processus inverse de l'échantillonnage.

En d'autres termes, dans l'échantillonnage nous sommes partis d'une population pour avoir des conclusions sur les échantillons, et avec l'estimation, nous partons d'un échantillon afin d'aboutir à des conclusions sur la population.

Il existe deux méthodes d'estimation des paramètres d'une population à partir d'un échantillon : la première méthode consiste à estimer les paramètres μ, σ et p de façon ponctuelle, et la deuxième méthode consiste en l'estimation de ces paramètres avec des intervalles de confiance. La suite de cette section donne les détails de calcul de ces deux méthodes d'estimation.

6.1 Estimation ponctuelle

En considérant une population P et un échantillon de cette même population de n individus, $E=\{x_1,x_2,x_3,\ldots,x_n\}$, alors les paramètres inconnus μ, σ et p de la population P peuvent être estimés de manière ponctuelle comme suit :

- La moyenne μ : la moyenne de la population peut être estimée de manière ponctuelle à partir de la moyenne de l'échantillon E. Il suffit de calculer la moyenne de l'échantillon $\bar{X} = \frac{\sum_{i=1}^{n} x_i}{n}$ et de considérer l'estimation $\mu = \bar{X}$.
- L'écart-type σ : l'écart-type de la population peut être estimé de façon ponctuelle à partir de l'écart-type de l'échantillon E. Pour cela, on calcule l'écart-type de l'échantillon, avec par exemple la formule de Koenig :

$$Ecart_type(E) = \sqrt[2]{\frac{\sum_{i=1}^{n} x_i^2}{n} - \bar{X}^2}$$

L'estimation ponctuelle de l'écart-type σ de la population est calculée comme suit : $\sigma = Ecart_type(E) \cdot \sqrt[2]{\frac{n}{n-1}}$

- La proportion p possédant la caractéristique ζ : cette proportion p peut être estimée de manière ponctuelle à partir de la fréquence f d'individus de l'échantillon E possédant la caractéristique ζ. Il suffit de calculer cette fréquence f dans l'échantillon E et de considérer l'estimation $p=f$.

© Editions ENI - All rights reserved

Remarque

On dit que ces estimations de μ, σ et p sont ponctuelles car elles ne sont basées que sur un seul échantillon.

Exemple numérique

Dans cet exemple, nous allons considérer la population constituée des tigres du Bengale. Supposons que nous souhaitons estimer la moyenne associée à leur longueur, la queue comprise, et également estimer la proportion de ces tigres dont la longueur est supérieure ou égale à 2.6 mètres. Bien sûr, dans cet exemple, nous supposons que nous ne pouvons pas effectuer les mesures de la longueur de toute la population des tigres du Bengale, et justement, c'est pour cette raison que nous allons procéder à un tirage d'un échantillon de 55 tigres que nous allons utiliser pour estimer la moyenne et la proportion des tigres dont la taille est supérieure ou égale à 2.6 mètres de toute la population. Le tableau suivant donne les longueurs des 55 tigres de notre échantillon :

Longueur (x_i)	**Effectif (n_i)**	$n_i * x_i$	$n_i * {x_i}^2$
2.26	3	6.78	15.3228
2.31	5	11.55	26.6805
2.33	4	9.32	21.6800
2.39	2	4.78	11.4242
2.45	2	4.90	12.0050
2.49	3	7.47	18.6003
2.50	2	5.00	12.5000
2.53	3	7.59	19.2027
2.56	1	2.56	6.5536
2.60	1	2.60	6.7600
2.63	4	10.52	27.6676
2.65	2	5.30	14.0450
2.70	5	13.50	36.4500
2.72	2	5.44	14.7968

Longueur (x_i)	Effectif (n_i)	$n_i * x_i$	$n_i * {x_i}^2$
2.74	3	8.22	22.5228
2.85	2	5.70	16.2450
2.90	4	11.6	33.6400
3.00	1	3.00	9.0000
3.05	3	9.15	27.9075
3.10	3	9.30	28.8300
Total	55	144.28	381.8338

Table des tailles des tigres de l'échantillon

Remarque

Les données de ce tableau sont complètement artificielles, créées pour le seul besoin de nos exemples. Sinon, dans la vraie vie et d'après Wikipédia, la longueur des tigres du Bengale mâles varie entre 2.7 et 3.1 mètres et celle des femelles varie entre 2.3 et 2.7 mètres et leur population est constituée d'environ 3000 individus en 2019.

La colonne "Effectif" montre le nombre de tigres n_i qui ont la longueur indiquée dans la colonne "Longueur". La colonne $n_i * x_i$ va nous permettre de calculer la moyenne des tailles de notre échantillon et la colonne $n_i * {x_i}^2$ sera utilisée lors du calcul de l'écart-type avec la formule de Koenig.

La moyenne des tailles des tigres de notre échantillon est égale à : $\bar{X} = \frac{144,28}{55} \approx 2.6232$

L'estimation ponctuelle de μ, qui est la moyenne de la taille des tigres du Bengale, est $\mu = \bar{X}$. Donc μ=2,6232.

Pour calculer l'estimation ponctuelle de l'écart-type à la taille moyenne des tigres du Bengale, nous allons d'abord calculer l'écart-type à la moyenne de notre échantillon comme suit :

$$Ecart_type(E) = \sqrt{\frac{\sum_{j=1}^{55} {x_j}^2}{55} - \bar{X}^2}$$

© Editions ENI - All rights reserved

$$= \sqrt{\frac{\sum_{i=1}^{20} n_i {x_i}^2}{55} - \bar{X}^2}$$

$$\approx \sqrt{\frac{381.8338}{55} - 6.8811}$$

$$\approx \sqrt{6{,}9424 - 6.8811}$$

$$\approx \sqrt{0.0613}$$

$$\approx 0.2475$$

L'estimation ponctuelle de l'écart-type σ de la moyenne de la population des tigres du Bengale sera calculée à :

$$\sigma = Ecart_type(E) \,.\, \sqrt[2]{\frac{n}{n-1}}$$

$$\sigma \approx 0.2475 \,.\, \sqrt[2]{\frac{55}{55-1}}$$

$$\sigma \approx 0.2497$$

L'estimation ponctuelle de la proportion de tigres dont la longueur est supérieure ou égale à 2.6 dans la population des tigres du Bengale est égale à cette même proportion dans l'échantillon.

Dans notre échantillon, il y a 30 tigres dont la taille est supérieure ou égale à 2.6. Donc, la proportion de ces tigres est égale à 30/55 = 0.5454, soit une proportion de 54.54 %.

En résumé, à partir de notre échantillon de 55 tigres et avec l'estimation ponctuelle, nous avons obtenu les résultats suivants :

1. L'estimation de la moyenne de la taille de tous les tigres du Bengale est 2.6232 mètres.
2. L'écart-type à la moyenne est égal à 0.2497.
3. La proportion de la population des tigres du Bengale dont la taille est supérieure ou égale à 2.6 mètres est de 54.54 %.

6.2 Estimation de la moyenne par intervalle de confiance

L'estimation de la moyenne d'une population à partir d'un échantillon avec un intervalle de confiance consiste à calculer un intervalle $I_\alpha = [a, b]$ avec une probabilité d'erreur α pour que la moyenne de la population appartienne à cet intervalle.

Pour calculer un tel intervalle de confiance, nous allons utiliser le théorème central limite et les propriétés de la loi normale.

En effet, d'après le théorème central limite, la distribution de la variable $\overline{X_E}$ associée aux moyennes des échantillons suit une loi normale qui est, pour rappel, définie comme suit $\overline{X_E} \sim \aleph(\mu, \frac{\sigma}{\sqrt{n}})$

Avec n la taille de l'échantillon, μ la moyenne de la population et σ son écart-type.

En nous appuyant sur les propriétés de la loi normale vues précédemment au début de ce chapitre, nous pouvons écrire :

$P\left(\mu - t\frac{\sigma}{\sqrt{n}} \leq \overline{X_E} \leq \mu + t\frac{\sigma}{\sqrt{n}}\right) = 1 - \alpha$ (α est le risque d'erreur)

Dans cette dernière équation, nous avons encadré la variable aléatoire $\overline{X_E}$ avec deux termes dans lesquels il figure deux inconnues μ et σ. Si nous fixons les valeurs de ces deux inconnues μ et σ, alors pour un α donné, la valeur du paramètre t peut être déduite à partir de la table de la loi normale, puisque nous savons que $2\,\pi(t) - 1 = 1 - \alpha$.

Ce que nous recherchons est un intervalle de confiance pour la moyenne μ.

© Editions ENI - All rights reserved

Pour trouver cet intervalle en utilisant notre dernière équation, nous allons d'abord effectuer une manipulation afin de transférer $\overline{X_E}$ vers les termes à gauche et à droite de l'intervalle et positionner l'inconnue μ au centre de cet intervalle comme suit :

Nous allons ajouter $-\overline{X_E} - \mu$ à tous les termes de l'intervalle :

$$P\left(\mu - t\frac{\sigma}{\sqrt[2]{n}} - \overline{X_E} - \mu \leq \overline{X_E} - \overline{X_E} - \mu \leq \mu + t\frac{\sigma}{\sqrt[2]{n}} - \overline{X_E} - \mu\right) = 1 - \alpha$$

En simplifiant cette dernière équation, nous obtenons :

$$P\left(-t\frac{\sigma}{\sqrt[2]{n}} - \overline{X_E} \leq -\mu \leq t\frac{\sigma}{\sqrt[2]{n}} - \overline{X_E}\right) = 1 - \alpha$$

$$\Leftrightarrow P\left(\overline{X_E} - t\frac{\sigma}{\sqrt[2]{n}} \leq \mu \leq \overline{X_E} + t\frac{\sigma}{\sqrt[2]{n}}\right) = 1 - \alpha$$

On conclut que la moyenne de la population μ est située dans l'intervalle $I_\alpha = [\overline{X_E} - t\frac{\sigma}{\sqrt[2]{n}}, \overline{X_E} + t\frac{\sigma}{\sqrt[2]{n}}]$ avec un niveau de confiance de $1 - \alpha$.

Si nous fixons un risque d'erreur α, et si nous lisons la valeur de t avec la formule $2\,\pi(t) - 1 = 1 - \alpha$ à partir de la table de la loi normale, il suffit de fixer la valeur de σ pour définir complètement l'intervalle $I_\alpha = [\overline{X_E} - t\frac{\sigma}{\sqrt[2]{n}}, \overline{X_E} + t\frac{\sigma}{\sqrt[2]{n}}]$.

De ce fait, nous allons distinguer deux cas : le premier cas est celui où la valeur de l'écart-type σ de la population est connu et le deuxième est celui où l'écart-type de la population n'est pas connu.

Dans le premier cas, lorsque l'écart-type σ est connu, pour calculer l'intervalle de confiance de la moyenne de la population avec un seuil de risque α, il suffit de remplacer la valeur de t, lu à partir de la table de la loi normale, dans l'intervalle I_α pour que ce dernier soit complètement défini.

Dans le deuxième cas, lorsque l'écart-type σ est inconnu, pour calculer l'intervalle de confiance de la moyenne de la population avec un seuil de risque α, il est nécessaire de remplacer σ par son estimation ponctuelle qui est, pour rappel, définie comme suit $\sigma = Ecart_type(E) \cdot \sqrt[2]{\frac{n}{n-1}}$.

Avec *Ecart_type(E)* l'écart-type observé directement dans l'échantillon *E*.

En résumé, lorsque l'écart-type σ est inconnu, l'intervalle de confiance I_α devient $I_\alpha = \left[\overline{X_E} - t\frac{Ecart_type(E).\sqrt[2]{\frac{n}{n-1}}}{\sqrt[2]{n}}, \overline{X_E} + t\frac{Ecart_type(E).\sqrt[2]{\frac{n}{n-1}}}{\sqrt[2]{n}}\right]$

Avec ce nouvel intervalle, une fois que $\overline{X_E}$ et *Ecart_type(E)* sont remplacés par leurs valeurs calculées sur l'échantillon *E*, la valeur de *t* sera lue à partir de la table de la loi normale comme précédemment en utilisant la formule $2\,\pi(t) - 1 = 1 - \alpha$, avec la valeur de α fixée au seuil d'erreur souhaité.

Exemple numérique

Nous allons reprendre l'exemple du tableau précédent où sont renseignées les tailles de 55 tigres du Bengale et nous allons supposer que nous ne connaissons ni la moyenne de la taille de cette population ni son écart-type. Nous allons chercher un intervalle de confiance de la moyenne associée à la taille de la population des tigres du Bengale avec une probabilité de 95 %, c'est-à-dire un risque d'erreur de $\alpha = 0.05$, à partir de notre échantillon.

En d'autres termes, nous allons chercher un intervalle dans lequel serait définie la moyenne de la population des tigres du Bengale avec un niveau de confiance de 95 %.

Dans cet exemple, la taille de l'échantillon est de 55. Donc, l'hypothèse de la normalité de la loi associée à la distribution de la taille des tigres du Bengale n'est pas obligatoire puisque la taille de cet échantillon est supérieure à 30.

Nous avons calculé précédemment la moyenne de l'échantillon $\overline{X} = 2{,}6232$ et nous avons réalisé l'estimation ponctuelle de l'écart-type de la population $\sigma \approx 0{,}2497$.

Comme nous avons supposé que l'écart-type à la moyenne de la population n'est pas connu, l'intervalle de confiance recherché est défini par :

$$I_{0.05} = \left[\overline{X_E} - t\frac{Ecart_type(E).\sqrt[2]{\frac{n}{n-1}}}{\sqrt[2]{n}}, \quad \overline{X_E} + t\frac{Ecart_type(E).\sqrt[2]{\frac{n}{n-1}}}{\sqrt[2]{n}}\right]$$

© Editions ENI - All rights reserved

En procédant au remplacement dans cet intervalle de $\overline{X_E}$ par $\overline{X}=2.6232$ et au remplacement de $Ecart_type(E).\sqrt[2]{\frac{n}{n-1}}$ par $\sigma \approx 0.2497$, nous obtiendrons l'intervalle suivant : $I_{0.05} = \left[2.6232 - t\frac{0.2497}{\sqrt[2]{n}},\quad 2{,}6232 + t\frac{0.2497}{\sqrt[2]{n}}\right]$

Nous cherchons un intervalle avec un seuil de confiance de 95 %, c'est-à-dire un risque d'erreur $\alpha=0.05$. Comme cet intervalle est symétrique, la valeur de t sera calculée avec la formule :

$2\,\pi(t) - 1 = 1 - 0.05$

$\Leftrightarrow 2\,\pi(t) = 1.95$

$\Leftrightarrow \pi(t) = 0.975$

$\Leftrightarrow \pi(t) = \pi(1.96)$ (lu de la table de la loi normale avec $\alpha=0.05$)

$\Leftrightarrow t = 1.96$

On remplace dans l'intervalle I_α la valeur de t par 1,96 et la valeur de $\sqrt[2]{n}$ par $\sqrt[2]{55} = 7{,}4161$ pour définir complètement l'intervalle $I_{0.05}$ à :

$I_{0.05} = \left[2.6232 - 1{,}96\frac{0.2497}{7.4161},\ 2.6232 + 1.96\frac{0.2497}{7.4161}\right]$

$\Leftrightarrow I_{0.05} \approx [2.56,\ 2.69]$ (avec arrondissement des valeurs vers le haut)

En conclusion de cet exemple, nous pouvons dire que statistiquement parlant et en nous basant sur notre échantillon, il y a 95 % de chances pour que la taille moyenne des tigres du Bengale soit définie dans l'intervalle [2.56, 2.69]. Cela signifie également que si nous procédons à un autre échantillonnage, la moyenne des tailles des tigres du Bengale de ce nouvel échantillon aura une probabilité de 0.95 d'être incluse dans l'intervalle [2.56, 2.69]. Il faut bien noter que c'est la moyenne des échantillons qui est concernée par notre intervalle et non pas les valeurs des échantillons.

En effet, vous remarquez que les valeurs de l'échantillon utilisé dans notre exemple ne sont pas vraiment conformes à notre conclusion ! Dans notre échantillon, il n'y a que huit valeurs sur les 55 qui rentrent dans l'intervalle de confiance calculé avec un risque d'erreur $\alpha=0.05$!

Cependant, cette dernière constatation ne va pas invalider notre conclusion, car, comme nous l'avons dit précédemment, l'intervalle calculé concerne les moyennes des échantillons et non pas les valeurs associées à nos échantillons !

6.3 Estimation d'une proportion par intervalle de confiance

Dans la section sur l'estimation ponctuelle, nous avons vu que la proportion p d'une population P possédant une caractéristique ζ donnée peut être estimée par la fréquence d'individus possédant cette caractéristique dans un échantillon prélevé à partir de cette population. Cette section montre comment estimer cette même proportion avec un intervalle de confiance associé à un seuil de risque donné.

Avec le théorème central limite, nous savons que la série statistique constituée des valeurs prises par la variable aléatoire F_E, qui est associée à la proportion d'individus dans les échantillons E^i avec $i \epsilon \{1,2,3,\ldots,k\}$ et qui possèdent une caractéristique ζ suit une loi normale avec les paramètres suivants :

$$F_E \sim \aleph(p, \sqrt[2]{\frac{p(1-p)}{n}})$$

Avec n la taille des échantillons E^i et p la proportion inconnue des individus dans la population qui possèdent la caractéristique ζ.

Pour définir un intervalle de confiance de la proportion p à partir d'un échantillon, nous allons procéder comme précédemment lorsque nous avons défini l'intervalle de confiance pour la moyenne.

En nous appuyant sur les propriétés de la loi normale vues précédemment au début de ce chapitre, nous pouvons écrire $P\left(p - t\sqrt{\frac{p(1-p)}{n}} \leq F_E \leq p + t\sqrt{\frac{p(1-p)}{n}}\right) = 1 - \alpha$ (α est le risque d'erreur).

Nous allons ajouter $-F_E - p$ comme suit :

$$P\left(p - t\sqrt{\frac{p(1-p)}{n}} - F_E - p \leq F_E - F_E - p \leq p + t\sqrt{\frac{p(1-p)}{n}} - F_E - p\right) = 1 - \alpha$$

© Editions ENI - All rights reserved

Après simplification, nous obtenons l'équation suivante :

$$P\left(F_E - t\sqrt{\frac{p(1-p)}{n}} \leq p \leq F_E + t\sqrt{\frac{p(1-p)}{n}}\right) = 1 - \alpha$$

Donc l'intervalle de confiance de la proportion p est le suivant :

$$I_\alpha = \left[F_E - t\sqrt{\frac{p(1-p)}{n}}, \quad F_E + t\sqrt{\frac{p(1-p)}{n}}\right]$$

Avec un seuil d'erreur α.

Et avec :

- F_E la variable associée aux proportions de la caractéristique ζ dans les échantillons. Pour rappel, pour un échantillon donné, cette variable sera tout simplement égale à la proportion de la caractéristique ζ dans cet échantillon.
- t une valeur qui sera lue à partir de la table de la loi normale en utilisant la formule $2\,\pi(t) - 1 = 1 - \alpha$.
- $\sqrt{\frac{p(1-p)}{n}}$ l'écart-type de la distribution associée à la variable F_E.

En utilisant l'estimation ponctuelle de l'écart-type vue précédemment, nous pouvons considérer que l'estimation ponctuelle de l'écart-type $\sqrt{\frac{p(1-p)}{n}}$ est la suivante : $\sqrt{\frac{f(1-f)}{n}} \cdot \sqrt{\frac{n}{n-1}} = \sqrt{\frac{f(1-f)}{n-1}}$ (avec f l'estimation ponctuelle de la proportion p).

On remplace dans I_α la variable F_E par sa valeur f calculée sur un échantillon donné et $\sqrt{\frac{p(1-p)}{n}}$ par son estimation ponctuelle $\sqrt{\frac{f(1-f)}{n-1}}$ pour obtenir l'intervalle de confiance suivant $I_\alpha = \left[f - t\sqrt{\frac{f(1-f)}{n-1}}, \quad f + t\sqrt{\frac{f(1-f)}{n-1}}\right]$.

Donc, pour calculer cet intervalle à partir d'un échantillon de taille n, il suffit de calculer la valeur de f, qui correspond à la fréquence d'individus dans cet échantillon possédant la caractéristique ζ, puis, après avoir fixé un seuil d'erreur α, nous pouvons lire la valeur de t à partir de la table de la loi normale en utilisant la formule $2\,\pi(t) - 1 = 1 - \alpha$.

Exemple numérique

Nous allons reprendre l'exemple du tableau précédent où sont renseignées les tailles de 55 tigres du Bengale et nous allons chercher un intervalle de confiance de la proportion des tigres du Bengale qui ont une taille supérieure ou égale à 2,6 mètres avec une probabilité de 99 %, c'est-à-dire un risque d'erreur de α=0.01, puis nous allons rechercher le même intervalle, mais avec α=0.05.

Avec α=0.01 :

Comme nous l'avons vu précédemment, la forme générale de l'intervalle de confiance d'une proportion est la suivante :

$$I_{0.01} = \left[f - t\sqrt{\frac{f(1-f)}{n-1}},\quad f + t\sqrt{\frac{f(1-f)}{n-1}} \right]$$

L'intervalle $I_{0.01}$ recherché ici correspond à un intervalle de confiance à 99 % pour la proportion des tigres du Bengale dont la taille est supérieure à 2,6 mètres.

Pour calculer les bornes de cet intervalle, nous allons commencer par le remplacement de la valeur de f par 0.5454 et la valeur de n par 55.

Donc, l'intervalle précédent devient :

$$I_{0.01} = \left[0.5454 - t\sqrt{\frac{0.4546}{54}},\quad 0.5454 + t\sqrt{\frac{0.4546}{54}} \right]$$

$$\Leftrightarrow I_{0.01} = [0.5454 - t * 0.0917,\ 0.5454 + t * 0.0917]$$

© Editions ENI - All rights reserved

Nous cherchons un intervalle avec un seuil de confiance de 99 %, c'est-à-dire un risque d'erreur $\alpha=0.01$ Comme cet intervalle est symétrique, la valeur de t sera calculée avec la formule :

$$2\,\pi(t) - 1 = 1 - 0.01$$

$$\Leftrightarrow 2\,\pi(t) = 1.99$$

$$\Leftrightarrow \pi(t) = 0.995$$

$$\Leftrightarrow \pi(t) = \pi(2.575)$$

$$\Leftrightarrow t = 2.575$$

La valeur de $t=2.575$ est obtenue après la lecture inversée de la table de la loi normale.

Donc l'intervalle de confiance I_α pour $\alpha=0.01$ est le suivant :

$$\Leftrightarrow I_{0.01} = [0.5454 - 2.575 * 0.0917,\ 0.5454 + 2.575 * 0.0917]$$

$$\Leftrightarrow I_{0.01} = [0.5454 - 0.236,\ 0.5454 + 0.236]$$

$$\Leftrightarrow I_{0.01} = [0.3094,\ 0.781]$$

$$\Leftrightarrow I_{0.01} = [30.94\%,\ 78.10\%]$$

On conclut donc qu'avec $\alpha=0.01$, la proportion des tigres dont la taille est supérieure à 2.6 mètres dans la population des tigres du Bengale est incluse dans l'intervalle $I_{0.01}=[30.94\%, 78.10\%]$.

Avec $\alpha=0.05$:

Comme précédemment, nous allons commencer par le remplacement de la valeur de f par 0.5454 et la valeur de n par 55 dans l'intervalle I_α et aboutir à l'intervalle $\Leftrightarrow I_{0.05} = [0.5454 - t * 0.0917,\ 0.5454 + t * 0.0917]$

Nous cherchons un intervalle avec un seuil de confiance de 95 %, c'est-à-dire un risque d'erreur $\alpha=0.05$. Comme cet intervalle est symétrique, la valeur de t sera calculée avec la formule $2\,\pi(t) - 1 = 1 - 0.05$

Nous avons déjà réalisé ce calcul précédemment lors de l'exemple numérique d'estimation de la moyenne par un intervalle de confiance et nous avons trouvé que $t=1.96$.

Donc l'intervalle de confiance I_α pour $\alpha=0.05$ est le suivant :

$$\Leftrightarrow I_{0.05} = [0.5454 - 1.96 * 0.0917,\ 0.5454 + 1.96 * 0.0917]$$

$$\Leftrightarrow I_{0.05} = [0.5454 - 0.18,\ 0.5454 + 18]$$

$$\Leftrightarrow I_{0.05} = [0.3654,\ 0.7254]$$

$$\Leftrightarrow I_{0.05} = [36.54\%,\ 72.54\%]$$

On conclut donc qu'avec $\alpha=0.05$, la proportion des tigres dont la taille est supérieure à 2.6 mètres dans la population des tigres du Bengale est incluse dans l'intervalle $I_{0.05}=[36.54\%,72.54\%]$.

Remarque

Remarquez que l'intervalle de confiance obtenu avec $\alpha=0.01$ est plus large que celui obtenu avec $\alpha=0.05$. C'est tout à fait normal et attendu puisque nous avons vu que plus le seuil d'erreur qu'on se fixe est faible, plus large sera l'intervalle obtenu !

6.4 Test d'hypothèse

Dans les sections précédentes, nous avons abordé le théorème central limite qui nous a permis d'établir des résultats sur la distribution de la variable associée à la moyenne des échantillons et sur la distribution de la variable associée aux proportions d'individus répondant à un critère donné dans les échantillons. Également, nous avons vu comment réaliser une estimation ponctuelle et une estimation par intervalle de confiance de la moyenne et de la proportion d'individus possédant une caractéristique donnée dans une population à partir d'un échantillon. Cette section permet d'aller un peu plus loin dans les statistiques inférentielles en abordant la notion des tests d'hypothèse.

© Editions ENI - All rights reserved

Un test d'hypothèse est tout simplement une expérimentation dans laquelle nous allons poser une hypothèse donnée, puis nous allons réaliser des calculs sur un échantillon et comparer le résultat de ces calculs avec des résultats théoriques déjà établis. Cette comparaison des résultats observés sur un échantillon avec des résultats théoriques va nous permettre de confirmer ou de rejeter notre hypothèse.

Ainsi, avec les tests d'hypothèse, nous pouvons tester si la moyenne d'une mesure donnée dans un échantillon est conforme à la moyenne attendue, ou tester si deux échantillons donnés présentent des variances similaires, ou tester si deux échantillons proviennent de populations qui ont une même moyenne par rapport à une mesure donnée, ou encore tester s'il existe un lien de dépendance entre deux variables, etc.

6.4.1 Tests paramétriques

Lorsque les variables d'études sont quantitatives et que les lois associées aux distributions de ces variables sont connues, et si, de plus, la moyenne et l'écart-type de la population étudiée sont connus, alors les tests que nous réalisons sont qualifiés de tests paramétriques.

Parmi les tests paramétriques les plus classiques, nous pouvons citer le test de Fisher, le test de l'ANOVA et le test de Student.

Il faut également noter que lorsque le théorème central limite est applicable, les tests que nous réalisons sont aussi des tests paramétriques.

6.4.2 Tests non paramétriques

Les tests non paramétriques sont les tests d'hypothèse que nous réalisons lorsque les conditions d'application des tests paramétriques ne sont pas réunies. En d'autres termes, nous réalisons des tests non paramétriques lorsque nous ne pouvons pas réaliser des tests paramétriques.

Bien qu'ils soient intéressants et souvent applicables, les tests non paramétriques sont beaucoup moins puissants que les tests paramétriques.

Parmi les tests non paramétriques, nous pouvons citer le test de Mann-Whitney et le test de Kruskal-Wallis.

6.4.3 Construire un test d'hypothèse

Pour mettre en place un test statistique, nous pouvons suivre les étapes ci-dessous :

1. Poser une question.

Il s'agit de poser une problématique pour laquelle on cherche une réponse. Par exemple, dans une usine de conditionnement du café, on utilise une machine pour remplir des sacs d'un kilogramme avec du café. Lorsqu'elle est bien réglée, cette machine fabrique donc des sacs de café avec une moyenne $\mu = 1000g$ par sac. En procédant au prélèvement d'un échantillon de 100 sacs de café, nous constatons que la moyenne associée au poids des sacs est $m = 1020$ et qui est donc supérieure à un kilogramme. Il est tout à fait normal d'avoir des sacs avec des poids légèrement supérieurs ou légèrement inférieurs à un kilogramme, car il est en pratique impossible de remplir tous les sacs avec exactement 1000 g de café. En revanche, la moyenne des poids des sacs devrait être très proche de la valeur 1000 g, car la machine est réglée pour fabriquer des sacs à un kilogramme. Nous pouvons nous poser la question de savoir si la machine de remplissage des sacs avec du café est déréglée ou pas à partir du moment où la moyenne calculée sur un échantillon est différente d'un kilogramme. En effet, le fait que la moyenne m de notre échantillon soit supérieure à la moyenne μ attendue ne veut pas forcément dire que la machine en question est déréglée ! Cela peut être dû tout simplement aux fluctuations des poids des sacs de notre échantillon.

Le problème est que nous ne savons pas quelle est la différence entre la moyenne m et la moyenne μ à partir de laquelle nous pouvons décider si la machine de fabrication est déréglée ou pas. En d'autres termes, nous ne savons pas si la différence entre m et μ, si petite soit-elle, est significative ou pas.

Justement, dans cet exemple, un test statistique peut nous servir pour décider si la différence entre la moyenne observée sur un échantillon et la moyenne attendue est due au déréglage de la machine ou bien si cette différence est tout à fait normale, car elle est juste due au caractère aléatoire des poids des sacs.

© Editions ENI - All rights reserved

2. Poser les hypothèses H_0 et H_1.

En statistiques, un test d'hypothèse est souvent constitué de deux hypothèses : une première hypothèse H_0, qu'on appelle également l'hypothèse nulle, dans laquelle nous supposons que le phénomène étudié n'est pas significatif, et une deuxième hypothèse H_1, qui est l'hypothèse alternative de la première hypothèse, dans laquelle nous supposons que le phénomène étudié est significatif.

Ainsi, il est d'usage de définir les deux hypothèses H_0 et H_1 suivantes :

H_0 : le test n'est pas significatif.

H_1 : le test est significatif.

Par exemple, si nous revenons à notre exemple de la machine de dosage du café dans des sacs, ces deux hypothèses peuvent être les suivantes :

H_0: $\mu = m$

H_1: $\mu \neq m$

Sous l'hypothèse H_0 la machine n'est pas déréglée, et même si $m = 1020g$ est supérieur à $\mu = 1000$, cette différence n'est pas statistiquement significative ! Avec l'hypothèse H_0, on suppose que si nous prélevons plusieurs échantillons, la moyenne m va avoir tendance à s'approcher de la moyenne attendue μ. En revanche, l'hypothèse H_1 suppose qu'il y a une réelle différence entre m et μ et que cette différence ne peut pas s'expliquer par le caractère aléatoire associé aux poids des sacs.

Quel que soit le type de test que nous réalisons, qu'il s'agisse d'un test de conformité d'une moyenne comme dans notre exemple ou d'un test de variance, ou encore d'un test de dépendance entre deux variables, etc., l'hypothèse H_0 suppose qu'il n'y a pas de différence réelle entre les valeurs observées dans un échantillon et les valeurs attendues !

Remarque

Pour se rappeler du sens de l'hypothèse H_0, un moyen mnémotechnique est de retenir que le zéro du H_0 annule les différences ! On note cette hypothèse avec un zéro en indice, car justement elle correspond à l'hypothèse nulle !

3. Définir un seuil de risque α.

Lorsque nous effectuons un test d'hypothèse, quel que soit le résultat de ce test, nous ne serions jamais certains à 100 % de sa validité. Il y a toujours une probabilité, si infime soit-elle, de nous tromper. Ainsi, nous pouvons nous tromper en ayant rejeté l'hypothèse H_0 alors que celle-ci est vraie, comme nous pouvons nous tromper en ayant accepté l'hypothèse H_0 alors que celle-ci est fausse.

Il est d'usage d'utiliser le symbole α pour désigner le risque de se tromper en ayant rejeté H_0 à tort. Dans ce cas, on appelle cette erreur par l'erreur de première espèce.

Il est également d'usage d'utiliser le symbole β pour désigner le risque de se tromper en ayant accepté à tort l'hypothèse H_0, et dans ce cas on parle de l'erreur de deuxième espèce.

Pour réaliser un test statistique, il est nécessaire de fixer un seuil pour le risque de première espèce α alors qu'il n'est pas nécessaire de fixer une valeur pour l'erreur de deuxième espèce β. Cependant, même si β n'est pas nécessaire pour effectuer un test, il est intéressant, car il permet de connaître la puissance d'un test.

4. Calculer une statistique théorique.

En ayant considéré l'hypothèse nulle H_0 vraie et en ayant fixé une valeur du risque α, nous allons considérer que le paramètre étudié suit une loi de probabilité connue. Nous devrions choisir une loi de probabilité qui reflète l'écart entre les paramètres étudiés. Avec cette loi de probabilité, nous allons calculer une statistique qui correspond au résultat attendu sous l'hypothèse H_0.

Cette statistique calculée ici reflète la valeur théorique attendue du paramètre étudié en prenant en compte que l'hypothèse H_0 est vraie.

© Editions ENI - All rights reserved

5. Conclusion du test.

À ce stade, nous aurions d'un côté un paramètre calculé sur un échantillon ou sur plusieurs échantillons, et de l'autre côté la valeur de ce paramètre qu'on devrait avoir théoriquement en considérant que l'hypothèse H_0 est vraie avec un risque d'erreur de α. Il suffit de comparer ces deux résultats suivant la table de probabilités associée à la loi de probabilité considérée à l'étape précédente.

En fonction du résultat lu à partir de cette dernière table, nous déciderons si H_0 devrait être acceptée ou rejetée.

Remarque

Afin de mieux comprendre toutes ces étapes, un exemple numérique de test d'hypothèse sera réalisé à la fin de cette section.

À l'issue d'un test d'hypothèse, il y a deux possibilités : soit on rejette l'hypothèse H_0 en acceptant l'hypothèse H_1, soit on accepte l'hypothèse H_0. Il faut bien noter que dans ce dernier cas, le fait d'accepter l'hypothèse H_0 ne signifie pas que cette hypothèse est vraie ! Cela doit être compris comme étant statistiquement parlant, nous n'avons pas d'argument pour rejeter H_0. En d'autres termes, l'acceptation de l'hypothèse H_0 ne veut pas dire qu'elle est vraie, cela signifie juste qu'elle n'est pas rejetée.

6.5 Types de tests d'hypothèse

Sans être exhaustive, cette section liste quelques types de tests d'hypothèse qui font partie des tests d'hypothèse les plus répandus dans la pratique.

6.5.1 Test de conformité

Un test de conformité consiste à réaliser un test d'hypothèse afin de vérifier si la valeur observée dans un échantillon d'un paramètre correspond bien à une valeur de référence connue.

Généralement, les deux hypothèses d'un test de conformité se formulent comme suit :

- H_0 : la valeur du paramètre étudié observé dans un échantillon est conforme à la valeur de ce même paramètre connu dans la population.
- H_1 : la valeur du paramètre étudié observé dans un échantillon n'est pas conforme à la valeur de ce même paramètre connu dans la population.

L'exemple de la machine de remplissage du café dans des sacs que nous avons considéré précédemment est un exemple type de test de conformité, puisque dans cet exemple nous nous attendons à ce que la moyenne des poids des sacs de café soit égale à 1000 g ! Dans ce test, nous avons voulu vérifier si la moyenne $m=1020$ des poids des sacs d'un échantillon est conforme à la vraie moyenne $\mu=1000$.

6.5.2 Test d'adéquation

Un test d'adéquation consiste à supposer que la distribution associée à une variable aléatoire suit une loi donnée et à réaliser un test d'hypothèse afin de vérifier si, statistiquement parlant, cette variable aléatoire suit vraiment la loi supposée.

Généralement, les deux hypothèses d'un test d'adéquation se formulent comme suit :

- H_0 : X est une variable aléatoire distribuée suivant la loi L.
- H_1 : X n'est pas distribuée suivant la loi L.

Par exemple, supposons que nous disposons d'un échantillon composé de plusieurs valeurs associées à une variable aléatoire. En utilisant les valeurs de cet échantillon, nous pouvons réaliser un test d'hypothèse afin de vérifier si la variable aléatoire associée aux valeurs de notre échantillon suit une loi normale ou si cette variable aléatoire suit une loi de Poisson ou toute autre loi.

© Editions ENI - All rights reserved

6.5.3 Tests d'homogénéité

Un test d'homogénéité consiste à supposer que deux échantillons E^1 et E^2 sont issus d'une même population et de réaliser un test d'hypothèse afin de vérifier si, statistiquement parlant, ces deux échantillons sont réellement issus d'une même population.

Généralement, les deux hypothèses d'un test d'homogénéité se formulent comme suit :

- $H_0 : E^1$ et E^2 sont homogènes.
- $H_1 : E^1$ et E^2 sont hétérogènes.

Généralement, un tel test d'hypothèse est basé sur l'analyse des variances des deux échantillons E^1 et E^2.

Un exemple type est celui où nous comparons la précision de fabrication de deux machines différentes. L'objectif est de savoir si les deux machines ont la même précision ou pas. Un autre exemple intéressant est celui où nous comparons les performances de deux programmes différents, mais qui sont développés pour résoudre un même problème. Dans ce dernier exemple, on suppose que les performances considérées sont relatives au temps de calcul utilisé pour résoudre une instance du problème en question, nous allons exécuter les deux programmes plusieurs fois pour résoudre des instances différentes et nous allons noter le temps de calcul consommé par ces deux programmes sur chaque instance. À la fin de toutes les exécutions, nous aurions constitué deux échantillons. En comparant les variances associées à ces deux échantillons avec un test statistique, nous pouvons vérifier si les deux programmes consomment des temps de calcul comparables, ou bien s'il y a un des deux programmes qui serait meilleur qu'un autre.

Remarque

Sans entrer dans les détails, dans cet exemple de comparaison des performances de deux programmes, on peut réaliser un test bilatéral afin de vérifier si les variances des deux échantillons sont homogènes ou pas, mais on peut également réaliser un test unilatéral afin de vérifier si les performances d'un programme sont meilleures que l'autre !

Dans le cas où E^1 et E^2 portent sur les mêmes individus, on parle de test sur séries appariées. Cela est intéressant lorsque nous souhaitons étudier l'effet d'un traitement donné sur les individus en mesurant certains paramètres sur ces individus avant et après le traitement étudié.

6.5.4 Test d'indépendance de variables

Le test d'indépendance consiste à réaliser un test afin d'étudier l'existence d'un lien entre deux variables.

Si nous disposons d'un échantillon dans lequel nous avons relevé les valeurs de deux variables X et Y pour chaque individu et que nous souhaitons étudier l'influence de la variable X sur la variable Y, alors la nature du test d'indépendance à réaliser dépend de la nature de ces deux variables. Ainsi, nous distinguons les quatre cas suivants :

1. Si les deux variables X et Y sont qualitatives, nous pouvons utiliser le test de Khi-deux.
2. Si la variable X est qualitative et la variable Y est quantitative, nous pouvons réaliser une ANOVA (*Analyse Of Variance*).
3. Si les deux variables X et Y sont quantitatives, nous pouvons réaliser une régression linéaire. La régression linéaire est l'objet du chapitre La régression linéaire et polynomiale.
4. Si la variable X est quantitative et la variable Y est qualitative, nous pouvons réaliser une régression logistique (voir chapitre du même nom).

Dans les deux premiers cas, les deux hypothèses d'un test de dépendance se formulent comme suit :

- H_0 : la variable Y est complètement indépendantes de la variable X.
- H_1 : il existe un lien d'association entre les deux variables X et Y.

Attention, les tests d'indépendance entre deux variables ne permettent pas d'établir l'existence de relation de cause à effet entre les variables étudiées !

© Editions ENI - All rights reserved

D'ailleurs, dans la mesure où le test s'avère significatif, c'est-à-dire que l'hypothèse H_0 est rejetée, on dira qu'il existe une association ou un lien de dépendance entre les variables étudiées, mais pas une relation de dépendance entre elles.

Un exemple connu et qui illustre bien ce dernier cas est celui où nous étudions la dépendance entre l'âge et l'apparition de la rougeole dans un échantillon constitué d'un ensemble de personnes allant des nouveau-nés jusqu'aux personnes de 70 ans. Il est clair qu'un test de dépendance entre la variable âge et l'apparition de la rougeole sera significatif, c'est-à-dire que nous allons fort probablement rejeter l'hypothèse H_0 dans laquelle nous supposons qu'il n'y a pas de lien entre l'âge et l'apparition de la rougeole. Cependant, nous savons très bien que l'âge n'est pas la cause de la rougeole, puisque cette dernière est causée par un virus !

Dans les deux derniers cas, les cas 3 et 4, nous n'allons pas formuler un test d'hypothèse, mais plutôt construire un modèle qui explique la relation entre les variables étudiées.

6.6 Exemple numérique de test de conformité d'une moyenne

Dans cet exemple, nous allons supposer une entreprise pharmaceutique qui a obtenu une autorisation de mise sur le marché d'un médicament dont le principe actif est la molécule M. Lors de la phase des essais cliniques, ce médicament a été utilisé avec une concentration de la molécule M de moyenne $\mu = 5g$ et d'écart type $\sigma = 0.2$. Ce médicament a donc été produit avec des machines dans une usine et distribué dans des flacons dans différentes pharmacies. Dans l'usine de fabrication de ce médicament, la machine de dosage de la molécule M a été ajustée afin de respecter la moyenne μ.

Un point important à comprendre dans cet exemple est que μ correspond à une moyenne de dosage de la molécule M, donc si nous contrôlons ce dosage dans un flacon donné, il y a de fortes chances que le dosage de cette molécule M dans ce flacon soit légèrement supérieur ou légèrement inférieur à la moyenne μ.

Cependant, pour que ce médicament soit conforme au médicament validé, il est nécessaire que la moyenne de concentration de la molécule M calculée sur un échantillon de plusieurs flacons soit égale ou très proche de la moyenne de référence μ.

Au bout de quelques mois de distribution, on aimerait s'assurer que la machine de dosage de la molécule M produit toujours des flacons avec un dosage de la molécule conforme à la moyenne μ.

Supposons qu'au bout de six mois de production nous prélevons un échantillon de 150 flacons et sur lesquels nous constatons que la moyenne de concentration de la molécule M est de $m=5.09g$.

La question qu'on va se poser est la suivante :

Le fait que $m>\mu$ est-il dû au déréglage de la machine de dosage de la molécule M, auquel cas il est nécessaire de procéder à son ajustement, ou bien cette différence est-elle tout simplement due aux fluctuations des dosages relatifs à notre échantillon ?

Cette même question peut également être posée de la manière suivante :

Dans l'échantillon des 150 flacons prélevés, la moyenne $m>\mu$. Si nous procédons à plusieurs autres opérations d'échantillonnage, est-ce que dans chacun de ces nouveaux échantillons la moyenne de dosage de la molécule M serait supérieure à μ ?

En effet, supposons qu'après avoir contrôlé le dosage de la molécule M dans 30 échantillons, avec 150 flacons dans chacun de ces échantillons, nous constatons qu'à chaque fois la moyenne $m>\mu$. Dans ce cas, nous avons le droit de penser que la machine de dosage de la molécule M est déréglée ! En revanche, si sur les 30 échantillons nous constatons que tantôt $m>\mu$ et tantôt $\mu>m$, alors nous pouvons également penser que les différences entre μ et m sont dues aux fluctuations des niveaux de dosage dans les échantillons.

Le problème est que nous ne souhaitons pas réaliser le contrôle de 30 échantillons ! Nous souhaitons décider si la machine de dosage de la molécule M est déréglée ou pas en considérant uniquement notre premier et seul échantillon de 150 flacons et dans lequel nous avons mesuré $m=5.09g$.

Dans ce cas-là, nous allons réaliser un test d'hypothèse sur la moyenne m.

© Editions ENI - All rights reserved

Maintenant que nous avons bien posé notre question, nous allons poser nos deux hypothèses H_0 et H_1 comme suit :

$H_0 : m = \mu$

$H_1 : m \neq \mu$

Si elle est vraie, l'hypothèse H_0 veut dire que même si m=5.09 alors que la moyenne attendue est μ=5, cela ne signifie pas forcément que la machine de dosage est déréglée et que probablement, si nous prélevons d'autres échantillons, nous aurons une moyenne plus proche de μ.

A contrario, l'hypothèse H_1, si elle est vraie, veut dire que la machine de dosage est déréglée et qu'elle va avoir tendance à produire des médicaments dont le dosage moyen de la molécule M est significativement supérieur ou inférieur au dosage attendu.

Nos deux hypothèses H_0 et H_1 sont posées, nous devons nous fixer un seuil d'erreur α que nous pouvons assumer et avec lequel nous pouvons nous tromper si l'on accepte l'hypothèse nulle H_0.

Nous allons fixer α=0.01.

On suppose que la population dont est issu notre échantillon de 150 flacons suit une loi normale, sous l'hypothèse H_0 nous pouvons donc utiliser le théorème central limite et confirmer que :

$\overline{X_E} \sim \aleph(5, \frac{0.2}{\sqrt{150}})$ (avec μ=5 et σ=0.2)

$\overline{X_E} \sim \aleph(5,\ 0.016)$

Bien sûr, cette dernière écriture n'est vraie que sous l'hypothèse H_0, c'est-à-dire que $\overline{X_E} \sim \aleph(5,\ 0.016)$ dans la mesure où notre échantillon des 150 flacons est issu d'une population dont la moyenne de concentration de la molécule M est μ=5.

En considérant que H_0 est vraie, nous allons déterminer un intervalle de confiance avec α=0.01 dans lequel devrait être définie la moyenne associée à la variable aléatoire $\overline{X_E}$.

Pour rappel, la forme générale de cet intervalle est définie comme suit :

$$I_\alpha = [\overline{X_E} - t\frac{\sigma}{\sqrt[2]{n}}, \quad \overline{X_E} + t\frac{\sigma}{\sqrt[2]{n}}]$$

Avec α=0.01, nous savons que la valeur de t=2.575. Donc :

$$I_{0.01} = [5 - 2.575 * 0.016,\ 5 + 2.575 * 0.016]$$

$$\Leftrightarrow I_{0.01} = [4.9588,\ 5.0412]$$

Nous pouvons dire que la moyenne de la concentration de la molécule M dans notre échantillon de 150 flacons a une probabilité de 99 % d'être incluse dans l'intervalle $I_{0.01}$=[4.9588, 5.0412].

La moyenne observée dans notre échantillon est m=5.09 et elle est située en dehors de notre intervalle $I_{0.01}$. Donc, nous concluons que l'hypothèse H_0 doit être rejetée au risque de nous tromper de 0.01. Avec ce résultat, nous pouvons affirmer que notre échantillon de 150 flacons est bien fabriqué par une machine déréglée et il est donc nécessaire de procéder à son ajustement !

Dans cet exemple, nous avons volontairement choisi des données qui permettent d'aboutir au rejet de H_0 avec α=0.01. Cependant, nous pouvons rechercher la valeur de p avec laquelle nous pourrons toujours rejeter H_0. Cette valeur représente la plus petite valeur possible du risque α et avec laquelle nous pouvons toujours rejeter l'hypothèse H_0.

À titre indicatif, pour notre exemple p=0.00088903 et cette valeur a été calculée avec le logiciel Excel. On peut donc dire avec un risque d'erreur proche de zéro que notre échantillon a été fabriqué avec une machine déréglée !

6.7 Le paradoxe de Simpson

Dans la pratique, lorsque nous étudions le lien supposé existant entre deux variables, nous sommes souvent confrontés à un paradoxe statistique dans lequel nous aboutissons à des conclusions sur l'ensemble des données qui sont en contradiction avec ce que nous observons en considérant ces mêmes données en petits groupes séparés.

© Editions ENI - All rights reserved

Pour comprendre et savoir détecter dans un jeu de données la présence du paradoxe de Simpson, analysons l'exemple de la figure ci-dessous :

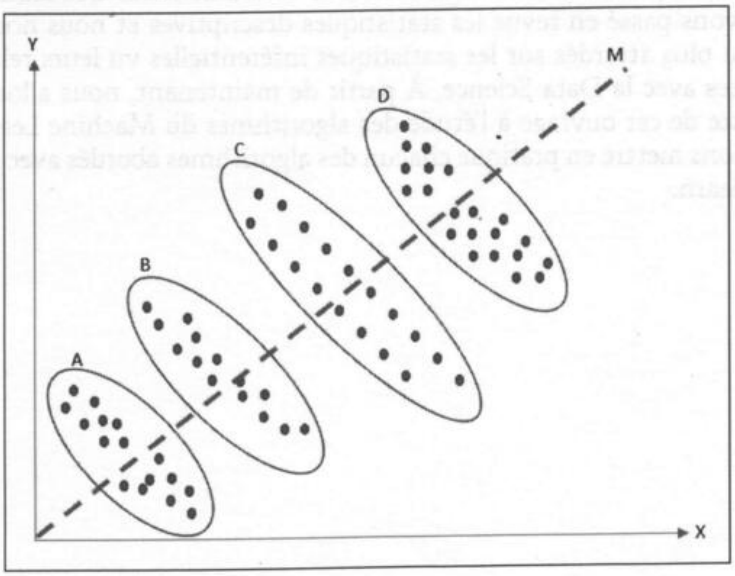


Figure 6-4: Illustration du paradoxe de Simpson

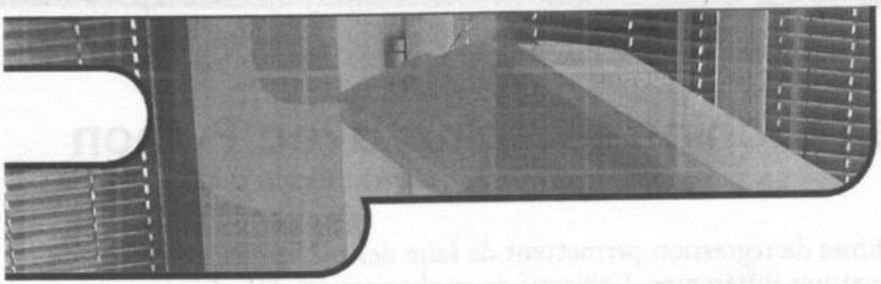
Dans cet exemple, un nuage de points est défini par les deux variables X et Y. Si nous considérons les points du groupe A, nous constatons que plus ces points ont une valeur élevée pour la variable X, moins l'est leur valeur pour la variable Y. La même remarque peut être observée pour les points des trois autres groupes B, C et D. Cependant, si nous modélisons l'ensemble de tous les points des quatre groupes avec un seul modèle, nous constatons que plus la valeur de la variable X est élevée, plus la valeur de la variable Y est également élevée (voir le modèle représenté par la droite M dans la figure) ce qui est en contradiction avec ce que nous observons au sein des petits groupes A, B, C et D ! C'est ce type de contradiction que nous appelons paradoxe de Simpson.

Généralement, ce type de contradiction entre la conclusion générale et les conclusions locales vient du fait que lorsque nous avons considéré les données dans leur ensemble, nous n'avons pas tenu compte d'un facteur de confusion.

7. Conclusion

Ce chapitre nous a permis d'aborder les notions fondamentales des statistiques. Nous avons passé en revue les statistiques descriptives et nous nous sommes un peu plus attardés sur les statistiques inférentielles vu leurs relations immédiates avec la Data Science. À partir de maintenant, nous allons consacrer le reste de cet ouvrage à l'étude des algorithmes du Machine Learning et nous allons mettre en pratique chacun des algorithmes abordés avec la librairie Scikit-learn.

© Editions ENI - All rights reserved



Partie 4
Les grands algorithmes du Machine Learning

Chapitre 4-1
La régression linéaire et polynomiale

1. Objectif du chapitre

Pour résumer, le premier chapitre a permis d'aborder la Data Science de manière théorique ; les deux suivants ont permis d'acquérir une première expérience avec le langage Python et les bibliothèques NumPy et Pandas ; le quatrième chapitre vous a permis de faire vos premiers pas dans l'utilisation de l'environnement Jupyter et le chapitre suivant d'aborder les notions fondamentales des statistiques. Le résultat des courses de ces six premiers chapitres est que vous êtes fin prêt pour attaquer les algorithmes du Machine Learning en commençant par la réalisation de modèles de régression linéaire et polynomiale dans ce présent chapitre.

Le mot «régression» trouve son origine dans une étude publiée par Sir Francis Galton en 1885. Dans cette étude très célèbre, Galton remarque que les pères dont la taille est supérieure à la moyenne ont des fils plus petits qu'eux. A contrario, les pères dont la taille est inférieure à la moyenne ont des fils plus grands qu'eux. De ce constat vient à Galton l'idée de la régression vers la moyenne.

Les algorithmes de régression permettent de faire des prévisions sur des problèmes de natures différentes. L'objectif de ce chapitre est d'étudier les principes fondamentaux pour comprendre comment fonctionnent ces algorithmes et réaliser des exemples concrets.

À la fin de ce chapitre, le lecteur aura abordé :

- le principe de la régression linéaire simple, la régression linéaire multiple, la régression polynomiale,
- la méthode des moindres carrés,
- une introduction à l'algorithme de la descente de gradient,
- l'introduction de la bibliothèque Scikit-learn,
- la construction de modèles de régression linéaire multiple,
- la construction de modèles de régression polynomiale,
- la normalisation de données avec Scikit-learn,
- l'évaluation de modèles,
- la cross-validation avec la méthode KFold,
- l'étude du coefficient de détermination R^2,
- le problème du surapprentissage,
- les régularisations Lasso et Ridge.

2. La régression linéaire simple

La régression linéaire simple concerne l'étude de la variation d'une variable à expliquer en fonction d'une et une seule variable explicative. Dans la section La Data Science - Modélisation du problème, nous avons vu que le principe général des algorithmes du Machine Learning supervisés est basé sur l'estimation, à partir d'un échantillon, d'une fonction F qui serait à l'origine des résultats observés. Dans le cas de la régression linéaire simple, ce principe s'applique avec l'hypothèse que la fonction F est linéaire et ne dépend que d'une seule variable.

© Editions ENI - All rights reserved

En d'autres termes, si nous disposons de n observations sous la forme de couples $x_i \in \mathbb{R}, y_i \in \mathbb{R}$, avec y_i le résultat correspondant à l'observation x_i, alors chacun de ces résultats y_i peut être obtenu, du moins approché, par une fonction $F(x_i)$ linéaire. Concrètement, en régression linéaire simple, la fonction F est supposée égale à $F(x_i) = ax_i + a_0$ avec $a \in \mathbb{R}$, $a_0 \in \mathbb{R}$ avec $a \in \mathbb{R}$, $a_0 \in \mathbb{R}$.

Cette définition de *F* propose qu'entre *x* et *y* existe une relation affine qui peut être représentée par une droite $ax_i + a_0$ appelée la droite de régression. Tout le but du jeu est de trouver des bons estimateurs pour les deux paramètres a et a_0.

2.1 La régression linéaire simple de point de vue géométrique

Dans l'exemple illustré par le graphique de la figure 7-1, le point *Pi* est défini par son abscisse x_i et son ordonnée y_i. Cela veut dire que pour l'observation x_i nous avons obtenu le résultat y_i.

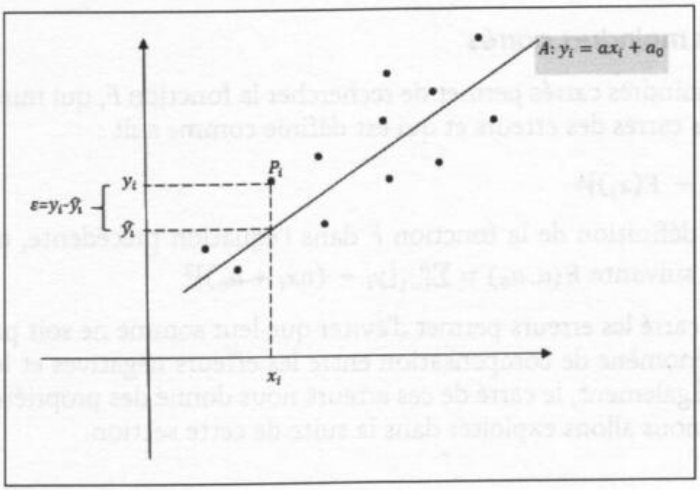


Figure 7-1 : exemple d'une droite de régression linéaire

Trouver des bons estimateurs de a et a_0 revient à trouver la droite de régression $A: F(x_i) = ax_i + a_0$ la plus proche des points x_i, y_i). Bien sûr, les valeurs retournées par la fonction F seront toutes sur la droite de régression A et ne coïncideront pas avec les valeurs y_i. En effet, une fois que les valeurs de a et a_0 sont fixées, alors pour le point P_i d'abscisse x_i, la fonction $F(x_i)$ nous donne l'estimation de son ordonnée $\widehat{y_i}$ (voir la figure ci-dessus) qui peut être différente de y_i. Cette différence est appelée l'erreur de l'estimation ou le résidu et elle est notée généralement par ε.

La droite de régression recherchée est celle qui minimise ces résidus. La section suivante aborde l'une des méthodes les plus utilisées pour obtenir la droite de régression.

2.2 La régression linéaire simple de point de vue analytique

La méthode la plus utilisée pour trouver la valeur du paramètre a et la valeur du paramètre a_0 est la méthode des moindres carrés.

2.2.1 La méthode des moindres carrés

La méthode des moindres carrés permet de rechercher la fonction F, qui minimise la somme des carrés des erreurs et qui est définie comme suit :

$$E(a, a_0) = \sum_{i=1}^{n} [y_i - F(x_i)]^2$$

En remplaçant la définition de la fonction F dans l'équation précédente, on obtient l'équation suivante $E(a, a_0) = \sum_{i=1}^{n} [y_i - (ax_i + a_0)]^2$

Le fait d'élever au carré les erreurs permet d'éviter que leur somme ne soit pas biaisée par un phénomène de compensation entre les erreurs négatives et les erreurs positives. Également, le carré de ces erreurs nous donne des propriétés intéressantes que nous allons exploiter dans la suite de cette section.

© Editions ENI - All rights reserved

La fonction E dépend uniquement des deux paramètres a et a_0. Trouver le minimum de cette fonction revient à trouver le point qui annule sa dérivée partielle par rapport au paramètre a et trouver le point qui annule sa dérivée partielle par rapport au paramètre a_0. Cela nous donne les deux équations suivantes :

$$\frac{dE}{da_0} = -2 \sum_{i=1}^{n} [y_i - (ax_i + a_0)] \quad (1)$$

$$\frac{dE}{da} = -2 \sum_{i=1}^{n} [y_i - (ax_i + a_0)] x_i \quad (2)$$

La fonction E atteint son minimum aux points d'annulation des deux équations (1) et (2). À partir de l'équation (1), on obtient :

$$\frac{dE}{da_0} = 0 \Leftrightarrow -2 \sum_{i=1}^{n} [y_i - (ax_i + a_0)] = 0$$

$$\Leftrightarrow \sum_{i=1}^{n} [y_i - (ax_i + a_0)] = 0$$

$$\Leftrightarrow \sum_{i=1}^{n} y_i - \sum_{i=1}^{n} (ax_i + a_0) = 0$$

$$\Leftrightarrow \sum_{i=1}^{n} y_i = \sum_{i=1}^{n} (ax_i + a_0)$$

$$\Leftrightarrow \sum_{i=1}^{n} y_i = a \sum_{i=1}^{n} x_i + na_0$$

$$\Leftrightarrow \frac{\sum_{i=1}^{n} y_i}{n} = a \frac{\sum_{i=1}^{n} x_i}{n} + \frac{na_0}{n}$$

Sachant que $\frac{\sum_{i=1}^{n} y_i}{n} = \bar{y}$ la moyenne des y_i et $\frac{\sum_{i=1}^{n} x_i}{n} = \bar{x}$ la moyenne des x_i, on obtient de l'équation (3) : $\bar{y} = a\bar{x} + a_0$ (4) (4)

Le point dont l'abscisse est la moyenne des x_i et dont l'ordonnée est la moyenne des y_i passe donc bien par la droite de régression.

À partir de l'équation (2), on obtient :

$$\frac{dE}{da} = 0 \Leftrightarrow -2 \sum_{i=1}^{n} [y_i - (ax_i + a_0)] x_i = 0$$

$$\Leftrightarrow \sum_{i=1}^{n} [x_i y_i - (ax_i^2 + a_0 x_i)] = 0$$

$$\Leftrightarrow \sum_{i=1}^{n} x_i y_i = \sum_{i=1}^{n} ax_i^2 + \sum_{i=1}^{n} a_0 x_i \quad (5)$$

À partir de l'équation (4), nous avons $a_0 = \overline{y} - a\,\overline{x}$. En remplaçant la valeur de a_0 dans (5), on obtient :

$$\sum_{i=1}^{n} x_i y_i = \sum_{i=1}^{n} a x_i^2 + \sum_{i=1}^{n} (\overline{y} - a\,\overline{x}) x_i$$

$$\Leftrightarrow \sum_{i=1}^{n} x_i y_i = \sum_{i=1}^{n} a x_i^2 + \sum_{i=1}^{n} x_i\,\overline{y} - \sum_{i=1}^{n} a x_i\,\overline{x}$$

$$\Leftrightarrow \sum_{i=1}^{n} x_i y_i = a \sum_{i=1}^{n} x_i^2 + \overline{y} \sum_{i=1}^{n} x_i - a\overline{x} \sum_{i=1}^{n} x_i$$

$$\Leftrightarrow \frac{\sum_{i=1}^{n} x_i y_i}{n} = a \frac{\sum_{i=1}^{n} x_i^2}{n} + \overline{y} \frac{\sum_{i=1}^{n} x_i}{n} - a \frac{\overline{x} \sum_{i=1}^{n} x_i}{n} \quad (6)$$

En remplaçant dans (6) tous les termes $\frac{\sum_{i=1}^{n} x_i}{n}$ par $\overline{x}$ et en factorisant la variable a, on obtient :

$$\frac{\sum_{i=1}^{n} x_i y_i}{n} = a\left(\frac{\sum_{i=1}^{n} x_i^2}{n} - \overline{x}^2\right) + \overline{x}\,\overline{y}$$

$$\Leftrightarrow a\left(\frac{\sum_{i=1}^{n} x_i^2}{n} - \overline{x}^2\right) = \frac{\sum_{i=1}^{n} x_i y_i}{n} - \overline{x}\,\overline{y}$$

$$\Leftrightarrow a = \frac{COV(X,Y)}{VAR(X)} \quad (7)$$

Avec $VAR(X)$ la variance de la variable X et $COV(X,Y)$ la covariance de la variable X et la variable Y.

En synthèse, pour trouver les valeurs des paramètres a et b il suffirait d'utiliser les équations (4) et (7). L'équation (7) permet de calculer la valeur de a, puis il suffit d'utiliser cette valeur de a dans l'équation (4) pour trouver la valeur de a_0.

2.2.2 Quelques considérations statistiques sur les données

En théorie, avant de faire des estimations sur les paramètres d'un modèle linéaire simple, les conditions d'applicabilité ci-dessous doivent être vérifiées :

- L'existence d'une relation de corrélation entre la variable explicative et la variable à expliquer. Cette relation n'est pas forcément une relation de causalité.

© Editions ENI - All rights reserved

- La variance des différentes distributions associées aux échantillons de la variable à expliquer est la même, quelles que soient les valeurs de la variable explicative.
- Les valeurs prises par la variable explicative sont exactes. Si ce n'est pas le cas, alors les erreurs sur les mesures prises doivent être négligeables.
- Les valeurs prises par la variable à expliquer ne sont pas dépendantes les unes des autres.
- La variable à expliquer est distribuée suivant une loi normale.

Remarque

Ces considérations statistiques sont très importantes. Cependant, dans la réalité, lorsqu'on aborde un problème de régression, on utilise des outils qui calculent la droite de régression et qui nous donnent des mesures de performance. Notre niveau de confiance accordé aux résultats est souvent basé sur ces mesures de performance.

3. La régression linéaire multiple

La régression linéaire multiple est une généralisation immédiate de la régression linéaire simple. En régression linéaire multiple, la fonction F que l'on souhaite estimer ne dépend plus d'une seule variable, mais de plusieurs.

Si nous disposons de n couples de la forme $(X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,m}) \in \mathbb{R}^m, y_i \in \mathbb{R})$, avec y_i le résultat obtenu pour l'observation $X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,m})$, alors la fonction que l'on souhaite estimer sera de la forme générale ci-dessous :

$$F(x_{i,1}, x_{i,2}, \dots, x_{i,m}) = a_1 x_{i,1} + a_2 x_{i,2} + \cdots + a_m x_{i,m} + a_0$$

L'objectif est d'estimer le vecteur $A = (a_1, a_2, \dots, a_m, a_0)$ tel que la fonction F soit la plus proche des y_i.

3.1 La méthode des moindres carrés pour la régression multiple

Tout comme pour la régression linéaire simple, la méthode des moindres carrés peut être utilisée pour la recherche du vecteur A et la fonction à minimiser sera définie comme suit :

$$E = \sum_{i=1}^{n} [y_i - F(x_{i,1}, x_{i,2}, \ldots, x_{i,m})]^2 \quad (8)$$

Les mêmes étapes peuvent être suivies à la différence qu'ici nous allons avoir $(m+1)$ dérivées partielles au lieu de deux. Après un certain calcul, en annulant et en simplifiant ces m dérivées partielles, nous obtenons la solution suivante :

$$\hat{A} = (X^T X)^{-1} X^T Y \quad (9)$$

Avec :

- $\hat{A}$ l'estimateur du vecteur $A = (a_0, a_1, a_2, \ldots, a_m)$
- X la matrice de toutes les observations qui est de la forme :

$$X = \begin{pmatrix} x_{1,1}, x_{1,2} & \cdots & x_{1,m} \\ \vdots \quad \vdots & \ddots & \vdots \\ x_{n,1}, x_{n,2} & \cdots & x_{n,m} \end{pmatrix}$$

- X^T la matrice transposée de la matrice X
- Y le vecteur des résultats de toutes les observations de la forme

$$Y = \begin{pmatrix} y_1 \\ y_2 \\ . \\ . \\ . \\ y_n \end{pmatrix}$$

Remarque

À noter que l'équation (9) suppose que la matrice X^{TX} est inversible.

© Editions ENI - All rights reserved

3.2 La méthode de la descente de gradient

En plus de la contrainte de la réversibilité de la matrice X^{TX}, l'utilisation de la méthode des moindres carrés peut s'avérer trop consommatrice en temps de calcul lorsque le nombre d'observations n et le nombre de variables m sont très grands.

L'algorithme de la descente de gradient est particulièrement adapté lorsque justement la taille du volume de données devient importante. Pour trouver une bonne estimation du vecteur $A = (a_0, a_1, a_2, \ldots, a_m)$, cet algorithme suit les étapes ci-dessous :

1: Initialisation aléatoire du vecteur $A = (a_0, a_1, a_2, \ldots, a_m)$,
2: Pour chaque a_i faire

$$a_i = a_i + \alpha \frac{dE}{da_i} \quad (10)$$

3: Si pas de convergence, reprendre au niveau (2), sinon Fin.

Avec E la fonction d'erreur définie dans l'équation (8) et α un facteur d'apprentissage. Plus ce facteur est grand, plus grand est le pas du déplacement de l'algorithme et plus vite la convergence surviendra. En revanche, avec un facteur d'apprentissage très grand, l'algorithme peut rater l'optimum global.

Remarque

Ce livre n'aborde pas les aspects théoriques associés aux problèmes d'optimisation, car cela dépasse notre sujet. Néanmoins, la connaissance des notions fondamentales de l'optimisation en mathématiques devrait intéresser toute personne souhaitant aller plus loin dans la compréhension des algorithmes du Machine Learning.

L'algorithme de la décente de gradient commence par initialiser les paramètres du modèle $A = (a_0, a_1, a_2, \ldots, a_m)$. Cette initialisation peut être complètement aléatoire ou choisie en utilisant une heuristique. Puis, l'algorithme entre dans une boucle d'itérations. À chaque itération, les valeurs des a_i seront recalculées en fonction de l'erreur actuelle et le facteur α (voir l'équation 10). Une fois qu'un point de convergence est atteint, l'algorithme s'arrête.

Dans le cas où l'espace de recherche n'est pas convexe, les résultats de cet algorithme dépendent fortement de la valeur initiale du vecteur A et du facteur d'apprentissage α et l'algorithme peut être piégé dans un optimum local.

3.3 Exemple de régression linéaire multiple

3.3.1 Définition du jeu de données utilisées

Dans cet exemple, nous allons construire un modèle linéaire multiple pour prédire la pollution de l'air dans une ville à partir d'un jeu de données. Ce jeu de données est constitué de treize variables explicatives et d'une variable à expliquer (voir le fichier *ozone.csv* du dossier *Data* de ce chapitre). Ces variables explicatives sont définies comme suit :

- Les variables `T9`, `T12` et `T15` correspondent respectivement à des prélèvements de température à 9 h, 12 h et 15 h.
- Les variables `Ne9`, `Ne12` et `Ne15` correspondent respectivement à des prélèvements de nébulosité à 9 h, 12 h et 15 h.
- Les variables `Vx9`, `Vx12` et `Vx15` correspondent respectivement à des prélèvements de vent à 9 h, 12 h et 15 h.
- Deux variables qualitatives `Vent` et `Pluie`. Ces deux variables ne seront pas utilisées lors de l'estimation des paramètres de notre modèle, puisqu'elles ne sont pas de type quantitatif. Néanmoins, pour les inclure dans une expérimentation, il suffirait de les transformer en valeurs numériques. Par exemple, les valeurs `sec` de la variable `Pluie` peuvent être remplacées par la valeur `1`, et les valeurs `Pluie` de cette même variable `Pluie` peuvent être remplacées par la valeur `2`. De même, nous pouvons transformer les valeurs de la variable `Vent` comme suit : `Nord = 1`, `Sud = 2`, `Est = 3` et `West = 4`.
- La variable `Date` correspond aux dates des prélèvements. Comme pour les deux variables `Vent` et `Pluie`, cette variable ne sera pas utilisée lors de l'estimation du modèle.
- Enfin, la variable `Max03v` correspond à la mesure maximum de la concentration d'ozone de la veille.

© Editions ENI - All rights reserved

Dans cet exemple, la variable à expliquer est la variable `Maxo3`. L'objectif du modèle que nous allons construire est donc de prédire la valeur que prendra le lendemain la variable `Max03` en fonction des valeurs enregistrées aujourd'hui pour les dix variables explicatives `T9`, `T12`, `T15`, `Ne9`, `Ne12`, `Ne15`, `Vx9`, `Vx12`, `Vx15` et `Maxo3v`, puisque nous n'allons pas utiliser les trois variables `Date`, `Vent` et `Pluie`.

3.3.2 Régression linéaire multiple avec Scikit-learn

Scikit-learn est une bibliothèque composée de plusieurs modules dédiés à la Data Science. Elle est issue des laboratoires de recherche comme l'Inria et du milieu universitaire français.

Scikit-learn propose d'utiliser un grand nombre d'algorithmes du Machine Learning de façon simple et intuitive. La mise en œuvre d'un algorithme avec cette bibliothèque peut être synthétisée par les étapes suivantes :

1. Instanciation d'un algorithme : par exemple un algorithme de régression linéaire comme nous allons le voir dans ce chapitre, ou un algorithme Random Forest ou autre.
2. Lancement de la fonction d'apprentissage avec l'algorithme instancié à l'étape (1) en lui fournissant un jeu de données sur lequel l'apprentissage va s'effectuer.
3. Réalisation des prédictions avec le modèle obtenu à l'étape (2).
4. Évaluation des performances du modèle sur un jeu de données de test en utilisant le résultat des prédictions obtenues à l'étape (3).

En plus des algorithmes d'apprentissage, Scikit-learn fournit également un nombre important de modules qui permettent de préparer et de manipuler les données.

Dans ce chapitre, nous allons dérouler toutes les étapes mentionnées ci-dessus pour la construction de modèles linéaires et polynomiaux. Les chapitres suivants seront réservés pour l'application d'autres types d'algorithmes mais, comme vous allez le constater, quelle que soit la nature des algorithmes appliqués, avec Scikit-learn nous suivons toujours les mêmes étapes.

3.3.3 Importer les modules Scikit-learn

Afin de construire un modèle de régression linéaire multiple avec Scikit-learn, suivez les étapes ci-après :

- Commencez par la création d'un nouveau notebook Jupyter. Vous pouvez utiliser le notebook *07-régression_linéaire.ipynb* contenu dans le sous-dossier *Code* de ce chapitre.
- Dans la première cellule de votre notebook, saisissez les instructions suivantes puis exécutez cette cellule afin d'importer les modules nécessaires à la réalisation de votre modèle.

```
import pandas as pd
from math import sqrt
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn import preprocessing
```

Les trois premiers modules importés sont `pandas` pour la manipulation des données, `math` pour calculer des racines carrées et `matplotlib.pyplot` pour réaliser des graphiques. Les autres modules importés sont définis comme suit :

- `sklearn.model_selection` : ce module nous permettra d'avoir accès à la fonction `train_test_split` qui nous sera utile pour la séparation de nos données en deux groupes : un premier groupe de données pour l'entraînement d'un modèle et un deuxième groupe de données pour l'évaluation d'un modèle.
- `sklearn.linear_model` : ce module nous donnera accès à la fonction `LinearRegression` qui sera utile pour instancier un algorithme de régression linéaire.
- `sklearn.metrics` : ce module nous donnera accès aux deux fonctions `mean_squared_error` et `r2_score` qui, comme leur nom l'indique, vont nous servir pour le calcul de la racine carrée de la moyenne des erreurs d'un modèle et de la métrique de performance *R2* qui sera détaillée dans la suite de ce chapitre.

© Editions ENI - All rights reserved

– `preprocessing` : ce module va nous donner accès à la fonction de normalisation de données dont nous aurons besoin afin de représenter nos données dans des échelles relatives à la moyenne zéro et leur écart-type.

3.3.4 Lecture des données dans un DataFrame

Pour lire les données de notre exemple, suivez les étapes ci-après.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin de lire votre jeu de données dans un DataFrame :

```
data = pd.read_csv('../Data/ozone.csv', sep=',')
```

- Dans une nouvelle cellule, saisissez et exécutez l'instruction suivante afin d'isoler dans un vecteur séparé la variable à expliquer `y`.

```
y = data["MaxO3"]
```

- Dans une nouvelle cellule, saisissez et exécutez l'instruction suivante afin de ne garder dans le DataFrame `data` que les dix variables explicatives `T9`, `T12`, `T15`, `Ne9`, `Ne12`, `Ne15`, `Vx9`, `Vx12`, `Vx15` et `Max03v` :

```
data= data[
          ['T9', 'T12', 'T15', 'Ne9', 'Ne12', 'Ne15', 'Vx9',
           'Vx12', 'Vx15', 'MaxO3v']
]
```

- Dans une nouvelle cellule, saisissez et exécutez l'instruction suivante afin d'afficher les cinq premières et les cinq dernières lignes du DataFrame `data` :

```
data
```

Vous obtenez le résultat de la figure suivante :

	T9	T12	T15	Ne9	Ne12	Ne15	Vx9	Vx12	Vx15	MaxO3v
0	15.6	18.5	18.4	4	4	8	0.6946	-1.7101	-0.6946	84
1	17.0	18.4	17.7	5	5	7	-4.3301	-4.0000	-3.0000	87
2	15.3	17.6	19.5	2	5	4	2.9544	1.8794	0.5209	82
3	16.2	19.7	22.5	1	1	0	0.9848	0.3473	-0.1736	92
4	17.4	20.5	20.4	8	8	7	-0.5000	-2.9544	-4.3301	114
...	...	...	...	...	...	...	...	...	...	...
107	13.3	17.7	17.8	3	5	6	0.0000	-1.0000	-1.2856	76
108	16.2	20.8	22.1	6	5	5	-0.6946	-2.0000	-1.3681	71
109	16.9	23.0	22.6	6	4	7	1.5000	0.8682	0.8682	77
110	16.9	19.8	22.1	6	5	3	-4.0000	-3.7588	-4.0000	99
111	15.7	18.6	20.7	7	7	7	0.0000	-1.0419	-4.0000	83

112 rows × 10 columns

Figure 7-2 : extrait du jeu de données `data`

3.3.5 Normalisation des données

En analysant visuellement les quelques enregistrements ci-dessus du DataFrame `data`, nous pouvons constater que les variables `T9`, `T12` et `T15` sont définies dans des échelles relativement comparables entre elles, puisque ces trois variables mesurent la température, mais elles sont très différentes des échelles dans lesquelles sont définies les variables `Ne9`, `Ne12`, `Ne15`. Ces différences sont encore plus importantes entre les variables `T9`, `T12`, `T15` et les variables `Vx9`, `Vx12`, `Vx15`. De même, les échelles des variables `Ne9`, `Ne12` et `Ne15` sont clairement différentes de celles des variables `Vx9`, `Vx12` et `Vx15`. Enfin, l'échelle dans laquelle est définie la variable `MaxO3v` est beaucoup plus large que les échelles de toutes les autres variables.

Pour avoir le cœur net quant aux échelles de définition de nos dix variables, nous allons calculer les statistiques de base de notre DataFrame `data`.

- Dans une nouvelle cellule, saisissez et exécutez l'instruction suivante afin d'afficher les statistiques de base associées aux données du DataFrame `data` :

```
data.describe()
```

© Editions ENI - All rights reserved

Vous obtenez le résultat de la figure ci-dessous :

	T9	T12	T15	Ne9	Ne12	Ne15	Vx9	Vx12	Vx15	MaxO3v
count	112.000000	112.000000	112.000000	112.000000	112.000000	112.000000	112.000000	112.000000	112.000000	112.000000
mean	18.360714	21.526786	22.627679	4.928571	5.017857	4.830357	-1.214346	-1.611004	-1.690683	90.571429
std	3.122726	4.042321	4.530859	2.594915	2.281860	2.332259	2.632742	2.795673	2.810198	28.276853
min	11.300000	14.000000	14.900000	0.000000	0.000000	0.000000	-7.878500	-7.878500	-9.000000	42.000000
25%	16.200000	18.600000	19.275000	3.000000	4.000000	3.000000	-3.276450	-3.564700	-3.939200	71.000000
50%	17.800000	20.550000	22.050000	6.000000	5.000000	5.000000	-0.866000	-1.879400	-1.549650	82.500000
75%	19.925000	23.550000	25.400000	7.000000	7.000000	7.000000	0.694600	0.000000	0.000000	106.000000
max	27.000000	33.500000	35.500000	8.000000	8.000000	8.000000	5.196200	6.577800	5.000000	166.000000

Figure 7-3 : table des statistiques de bases du jeu de données `data`

En comparant ces statistiques, on voit plus clairement et de façon plus certaine les différences des distributions associées aux dix variables. Par exemple, on note que les valeurs minimales et maximales des variables `T9`, `T12` et `T15` sont très différentes des valeurs minimales et maximales des variables `Ne9`, `Ne12`, `Ne15` et ces différences sont plus importantes avec les valeurs minimales et maximales des variables `Vx9`, `Vx12` et `Vx15`.

Également, on voit que les valeurs minimales et maximales des variables `T9`, `T12`, `T15` sont peu différentes et ces différences sont liées aux horaires correspondant aux prises de ces températures. Le matin, la température `T9` est généralement inférieure à la température `T12` prise à 12h, et cette dernière est généralement inférieure à la température `T15` qui est prise à 15h.

Remarque

Notez que ce jeu de données ne contient que 112 observations. Ce n'est pas handicapant pour construire un modèle, mais cela reste un nombre relativement bas pour espérer aboutir à un modèle performant.

Afin de remédier à ce problème d'échelle de définition des variables, nous allons procéder à leur normalisation. Pour cela, nous utilisons la normalisation standard qui consiste, pour une variable donnée, à retirer la moyenne de chaque valeur de cette variable et diviser le résultat de cette soustraction par l'écart-type de cette même variable. Ainsi, les valeurs des variables sont représentées par leurs distances en termes d'écart-type à leur moyenne zéro. Pour réaliser cette normalisation standard, suivez les étapes ci-après.

Dans une nouvelle cellule, saisissez et exécutez le code de la fonction `normalization` ci-dessous et l'instruction qui permet d'appeler cette fonction avec le DataFrame `data` comme paramètre :

```
def normalization(dataToNormalize):
    columns = dataToNormalize.columns
    for col in columns:
        x = dataToNormalize[[col]].values.astype(float)
        standard_normalization = preprocessing.StandardScaler()
        res = standard_normalization.fit_transform(x)
        dataToNormalize[col]=res

normalization(data)
```

Cette fonction récupère une par une les variables d'un DataFrame reçu en paramètre, puis elle transforme les valeurs de ces variables en utilisant la normalisation standard. Cette normalisation standard est réalisée par un algorithme créé avec la fonction `preprocessing.StandardScaler()` qui est appliqué sur une variable x et le résultat de cette transformation sera enregistré dans le DataFrame reçu en paramètre à la place des valeurs d'origine de la variable qui a subi la transformation.

Comme précédemment, visualisez le DataFrame data avec l'instruction suivante :

```
data
```

Vous obtenez le résultat suivant :

	T9	T12	T15	Ne9	Ne12	Ne15	Vx9	Vx12	Vx15	MaxO3v
0	-0.888045	-0.752140	-0.937279	-0.359451	-0.448069	1.365152	0.728338	-0.035506	0.356046	-0.233441
1	-0.437704	-0.778969	-1.092469	0.027850	-0.007861	0.934456	-1.188782	-0.858374	-0.468910	-0.126870
2	-0.984547	-0.975785	-0.693408	-1.133553	-0.007861	-0.357531	1.590540	1.254113	0.790522	-0.304488
3	-0.695042	-0.453946	-0.028306	-1.520753	-1.768585	-2.080415	0.839060	0.703625	0.542276	0.050748
4	-0.309035	-0.255151	-0.493878	1.188953	1.312765	0.934456	0.272551	-0.482587	-0.943449	0.832266
...	...	...	...	...	...	...	...	...	...	...
107	-1.627892	-0.950935	-1.070299	-0.746552	-0.007861	0.503760	0.463320	0.219536	0.144795	-0.517629
108	-0.695042	-0.180602	-0.116987	0.414751	-0.007861	0.073064	0.198303	-0.139768	0.115306	-0.695247
109	-0.469871	0.366086	-0.006136	0.414751	-0.448069	0.934456	1.035829	0.890788	0.914663	-0.482105
110	-0.469871	-0.429097	-0.116987	0.414751	-0.007861	-0.788327	-1.062836	-0.771710	-0.825457	0.299413
111	-0.855878	-0.727290	-0.427368	0.801852	0.872556	0.934456	0.463320	0.204481	-0.825457	-0.268964

112 rows x 10 columns

Figure 7-4 : le jeu de données `data` après la normalisation

© Editions ENI - All rights reserved

- De nouveau, exécutez l'instruction suivante afin de visualiser les mêmes statistiques que précédemment avec les données transformées :

```
data.describe()
```

Vous obtenez les mêmes résultats que les suivants :

	T9	T12	T15	Ne9	Ne12	...	Ne15	Vx9	Vx12	Vx15	MaxO3v
count	1.120000e+02	1.120000e+02	1.120000e+02	1.120000e+02	1.120000e+02		1.120000e+02	1.120000e+02	1.120000e+02	1.120000e+02	1.120000e+02
mean	-6.215266e-16	-2.111406e-16	3.568574e-17	-1.040834e-16	-1.179612e-16		-1.675247e-16	3.419883e-17	2.924248e-17	-8.524927e-17	6.046750e-17
std	1.004494e+00	1.004494e+00	1.004494e+00	1.004494e+00	1.004494e+00		1.004494e+00	1.004494e+00	1.004494e+00	1.004494e+00	1.004494e+00
min	-2.271236e+00	-1.870365e+00	-1.713231e+00	-1.907854e+00	-2.208904e+00		-2.080415e+00	-2.542636e+00	-2.251932e+00	-2.612687e+00	-1.725430e+00
25%	-5.950420e-01	-7.272901e-01	-7.432910e-01	-7.485517e-01	-4.480995e-01		-7.883274e-01	-7.867737e-01	-7.019695e-01	-8.037238e-01	-6.952468e-01
50%	-1.803663e-01	-2.427259e-01	-1.280717e-01	4.147509e-01	-7.860858e-03		7.306449e-02	1.329075e-01	-9.643571e-02	5.041172e-02	-2.867259e-01
75%	5.031874e-01	5.027576e-01	6.145254e-01	8.018518e-01	8.725563e-01		9.344564e-01	7.283376e-01	5.788388e-01	6.043282e-01	5.480777e-01
max	2.779019e+00	2.975278e+00	2.853802e+00	1.188953e+00	1.312765e+00		1.365152e+00	2.445874e+00	2.942264e+00	2.391559e+00	2.679491e+00

Figure 7-5 : les statistiques de base du dataset `data` après la normalisation

Cette fois, on voit bien que les moyennes de toutes les variables sont très proches de zéro et que les valeurs minimales et maximales de toutes les variables sont bien comparables et peu différentes.

3.3.6 Construction d'un modèle linéaire

Avant d'appliquer un algorithme de régression linéaire, nous devons d'abord construire un jeu de données d'entraînement et un jeu de données de test que nous utiliserons pour l'évaluation d'un modèle.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin de séparer les données du DataFrame `data` en deux jeux de données distincts :

```
x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)
```

À la fin de l'exécution de l'instruction ci-dessus, les variables `x_train` et `y_train` vont représenter le jeu de données d'entraînement. `x_train` contiendra les variables explicatives et `y_train` contiendra la variable à prédire. De même, `x_test` et `y_test` représentent le jeu de données de test, `x_test` sera considéré comme étant des données à utiliser par un modèle pour prédire les valeurs contenues dans le vecteur `y_test`.

Le paramètre `test_size` permet d'indiquer le taux des observations qui seront affectées au jeu de données de test. La fonction `train_test_split()` va tout simplement sélectionner de manière pseudo-aléatoire 20 % des observations de l'ensemble global de toutes les observations et les affecter au jeu de données de test.

Cet algorithme de séparation de données va essayer d'équilibrer la répartition des données du vecteur y entre `y_train` et `y_test`. Autrement dit, la distribution associée aux données de `y_train` va être la plus proche possible de la distribution associée aux données de `y_test`. Par exemple, nous n'allons pas avoir dans `y_train` des valeurs de la variable `MaxO3` plus grandes que toutes les valeurs dans `y_test` ou vice versa, un certain équilibre sera garanti par cette fonction `train_test_split()`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin d'instancier un algorithme de régression linéaire :

```
regression_alg = LinearRegression()
```

À ce stade, nous avons tous les ingrédients nécessaires pour entraîner un modèle à la prédiction de notre variable `MaxO3`. En effet, `regression_alg` représente un algorithme linéaire et `x_train` et `y_train` représentent des données propres et prêtes à être utilisées.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
regression_alg.fit(x_train, y_train)
```

Cette dernière instruction permet à l'algorithme `regression_alg` de s'entraîner pour comprendre comment les données de `x_train` peuvent être liées ensemble afin d'expliquer les résultats associés aux données de `y_train`. Chaque ligne de `x_train` explique la ligne correspondante dans `y_train`. Le résultat de cette instruction est un modèle qui peut être utilisé pour prédire la concentration de l'air en ozone en partant des valeurs des dix variables explicatives.

© Editions ENI - All rights reserved

3.3.7 Évaluation d'un modèle linéaire

Pour rappel, pendant l'entraînement d'un modèle linéaire multiple, l'objectif recherché est l'estimation des coefficients qui seront associés à nos variables. Une fois que l'entraînement est achevé et avant d'évaluer un modèle sur le jeu de données de test, nous pouvons réaliser le calcul des métriques de performance de ce modèle sur le jeu de données d'entraînement. Cela va nous donner une idée de la qualité des prédictions du modèle acquise pendant l'entraînement.

Dans notre exemple, nous allons utiliser les deux métriques de performance RMSE et le coefficient de détermination R^2, abordés au premier chapitre, que nous allons expliquer en détail par la suite.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les trois instructions ci-dessous :

```
train_predictions = regression_alg.predict(x_train)

print(f"RMSE = {round(sqrt(mean_squared_error(y_train,
train_predictions)),2)}")

print(f"R2_score = {round(r2_score(y_train,
train_predictions),2)}")
```

Grâce à la fonction `regression_alg.predict()`, la variable `train_predictions` va recevoir les prédictions de notre modèle sur les données `x_train`. La deuxième instruction de ce code permet de calculer la moyenne des carrés des erreurs, avec la fonction `mean_squared_error()`, et d'afficher sa racine carrée en utilisant la fonction `sqrt()` avec un niveau de précision de deux chiffres après la virgule. La deuxième instruction permet d'afficher le *R2* du modèle.

L'exécution du code ci-dessus a donné le résultat ci-après :

```
RMSE = 14.17
R2_score = 0.72
```

Ces résultats montrent que notre modèle donne des prédictions qui s'écartent en moyenne de `14.17` des vraies observations contenues dans le jeu de données d'entraînement.

La métrique de performance R^2 nous donne une idée sur la comparaison de ce modèle avec un modèle théorique qui prédit toujours la valeur moyenne des valeurs observées. Comme nous l'avons dit précédemment, dans la suite de ce chapitre nous allons expliquer en détail le R^2, mais pour l'instant il faut juste noter que plus le R^2 est proche de la valeur `1`, meilleures sont ses prédictions, et plus sa valeur est proche de la valeur `0`, plus ses prédictions sont comparables avec les prédictions d'un modèle qui prédit toujours la valeur moyenne des observations, et ce quelles que soient les valeurs des variables explicatives. Dans ce dernier cas, ce modèle aurait un pouvoir de prédiction faible.
Notez aussi que la valeur de R^2 peut être négative et nous allons en découvrir les raisons dans la suite de ce chapitre. La valeur de R^2 de notre modèle est `0.72`, ce qui indique que notre modèle a un niveau de prédiction de bonne qualité.

Rappelons que dans le code précédent les deux métriques RMSE et R^2 ont été mesurées sur le jeu de données d'entraînement. Ces données sont connues par notre modèle puisque ce sont ces mêmes données qui ont été utilisées par notre algorithme pour estimer les paramètres du modèle. Voyons à présent quelles sont les valeurs de ces deux métriques de performance sur le jeu de données de test qui contient des données qui n'ont jamais été vues par notre modèle !

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les trois instructions ci-dessous :

```
test_predictions = regression_alg.predict(x_test)

print(f"RMSE = {round(sqrt(mean_squared_error(y_test,
test_predictions)),2)}")

print(f"R2_score = {round(r2_score(y_test,
test_predictions),2)}")
```

Le résultat obtenu est le suivant :

```
RMSE = 11.66
R2_score = 0.85
```

Ces résultats montrent que notre modèle a un pouvoir de prédiction relativement intéressant puisque le *R2* est proche de la valeur `1`.

© Editions ENI - All rights reserved

Notez que ces résultats peuvent être sensiblement différents d'une exécution à une autre. En effet, les différences d'une exécution à une autre dans notre cas précis sont dues à la fonction `train_test_split` qui construit des jeux de données d'entraînement et de test différents selon les exécutions. Comme nous ne disposons pas d'un grand volume de données, même des petites différences entre les données d'entraînement et celles de test peuvent jouer un rôle suffisamment important pour aboutir à des modèles très différents.

Pour afficher les coefficients associés à notre modèle, procédez comme suit.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction ci-dessous :

```
regression_alg.coef_
```

Cette instruction retourne les dix coefficients associés à nos dix variables explicatives comme suit :

```
array([-0.6975262, 7.0173408, 4.3514018, -6.3002497,
-1.00444703, 0.51129768, 1.29647116, 0.82860687, 1.19131626,
10.47335291])
```

Ces coefficients correspondent aux $a_1, a_2, \ldots, a_m$ de la formule recherchée qui est pour rappel :

$$F(x_{i,1}, x_{i,2}, \ldots, x_{i,m}) = a_1 x_{i,1} + a_2 x_{i,2} + \cdots + a_m x_{i,m} + a_0$$

L'ordre de ces coefficients est associé à l'ordre des variables dans le DataFrame `data`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction ci-dessous afin d'afficher la constante a_0 utilisée dans notre modèle linéaire (voir la formule ci-dessus) :

```
regression_alg.intercept_
```

Le résultat s'affiche comme suit :

```
89.8519832376055
```

Cette constante et les dix coefficients ci-dessus déterminent la forme du modèle.

Nous pouvons également visualiser dans un graphique les prédictions de notre modèle en fonction des vraies valeurs observées. Pour cela, suivez les étapes ci-après.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions ci-dessous afin de visualiser les prédictions de notre modèle en fonction des valeurs observées du jeu de données de test :

```
plt.scatter(y_test, test_predictions, color='black')
plt.title("les prédictions du modèle vs la réalité")
plt.xlabel("les valeurs observées")
plt.ylabel("Les prédictions")
plt.plot([40.0, 160.0], [40.0, 160.0], 'red', lw=1)
plt.show()
```

Ces instructions affichent le graphique ci-dessous :

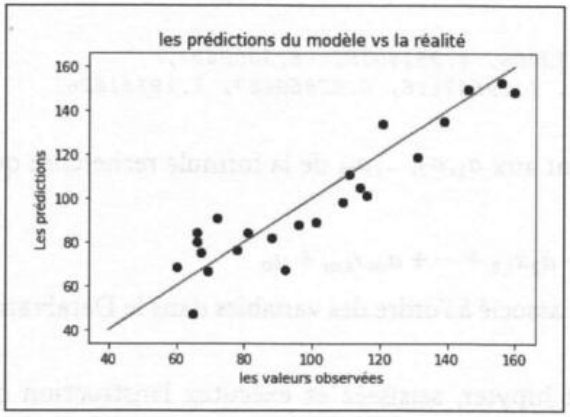


Figure 7-6 : représentation des prédictions en fonction des observations du modèle de régression linéaire

La ligne rouge obtenue avec la fonction `plt.plot(...)` correspond au modèle parfait qui prédit de manière exacte les valeurs du jeu de données de test. Les points noirs correspondent aux prédictions de notre modèle.

Ce graphique montre que notre modèle donne des prédictions relativement proches de la réalité puisque les points noirs ne sont pas très loin de la droite rouge.

© Editions ENI - All rights reserved

Remarque

Pour rappel, ces résultats peuvent être sensiblement différents d'une exécution à une autre.

3.3.8 Évaluer le futur comportement d'un modèle

Comme les résultats obtenus dans notre exemple peuvent sensiblement être différents d'une exécution à une autre, il serait intéressant de se poser la question sur le pouvoir prédictif de notre modèle une fois déployé en production. En effet, on aimerait bien se faire une idée de la qualité des prédictions de notre modèle qui sera la plus proche possible de la réalité.

Pour cela, nous pouvons exécuter notre programme plusieurs fois et calculer le RMSE et le R^2 à chaque exécution puis réaliser la moyenne des ces deux métriques relatives à toutes les exécutions. Cela nous donnera une idée plus précise de la qualité moyenne que va avoir notre modèle en production. Pour calculer les moyennes de RMSE et de R^2 sur plusieurs exécutions, suivez les étapes ci-après.

- Dans le même notebook que le précédent, définissez la fonction `average_result` suivante dans une nouvelle cellule :

```
def average_result(nb_run):
    average_rmse = 0
    average_r2 = 0
    for i_run in range(nb_run):
        x_train, x_test, y_train, y_test = train_test_split(data,
        y, test_size=0.2)

        regression_alg = LinearRegression()
        regression_alg.fit(x_train, y_train)

        test_predictions = regression_alg.predict(x_test)

        i_run_rmse = sqrt(mean_squared_error(y_test,
                          test_predictions))
        i_run_r2 = r2_score(y_test, test_predictions)

        print(f"Run {i_run} : RMSE = {round(i_run_rmse,2)} -
              R2_score = {round(i_run_r2,2)}")
```

```
        average_rmse = average_rmse + i_run_rmse
        average_r2 = average_r2 + i_run_r2
    average_rmse = average_rmse / nb_run
    average_r2 = average_r2 / nb_run

    print(f"Moyenne : RMSE = {round(average_rmse,2)} - R2_score =
          {round(average_r2,2)}")
```

Cette fonction permet d'automatiser les exécutions de la fonction `regression_alg.fit()` avec des jeux de données d'entraînement et de test différents à chaque fois. En effet, nous savons qu'à chaque exécution de cette fonction les données des variables `x_train`, `x_test`, `y_train`, `y_test` sont différentes.

Cette fonction commence par l'initialisation des deux moyennes `average_rmse` et `average_r2` avec la valeur `0`. Puis, dans une boucle `for`, la variable `i_run` prend tour à tour les valeurs de `0` jusqu'à la valeur `(nb_run-1)`, avec `nb_run` qui correspond à l'unique paramètre de cette fonction et qui représente le nombre d'exécutions que nous souhaitons réaliser.

Au début de chaque itération de cette boucle, la fonction `train_test_split(...)` génère des données `x_train`, `y_train`, `x_test` et `y_test` différentes. Le reste du code de cette boucle est décrit comme suit :

- Une instance de l'algorithme est créée avec la fonction `LinearRegression()`.
- La fonction `fit()` est lancée sur le jeu de données d'entraînement `x_train`, `y_train`.
- La fonction `predict()` est utilisée pour effectuer des prédictions sur les données de test `x_test`.
- La métrique RMSE est calculée pour l'itération en cours et stockée dans la variable `i_run_rmse`.
- Le RMSE et le *R2* de l'exécution en cours sont affichés.
- La variable *average_rmse* est mise à jour pour avoir le cumul de tous les RMSE de toutes les exécutions.

© Editions ENI - All rights reserved

Et enfin, à la fin de cette fonction, les moyennes de `average_rmse` et de `average_r2` sont calculées et affichées.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction ci-dessous afin d'exécuter la fonction `average_result` dix fois :

```
average_result(10)
```

L'exécution de la dernière instruction donne un résultat similaire au résultat ci-après :

```
Run 0 : RMSE = 21.33 - R2_score = 0.51
Run 1 : RMSE = 15.3 - R2_score = 0.72
Run 2 : RMSE = 13.81 - R2_score = 0.59
Run 3 : RMSE = 19.45 - R2_score = 0.57
Run 4 : RMSE = 18.5 - R2_score = 0.61
Run 5 : RMSE = 19.9 - R2_score = 0.66
Run 6 : RMSE = 14.43 - R2_score = 0.68
Run 7 : RMSE = 19.16 - R2_score = 0.6
Run 8 : RMSE = 12.04 - R2_score = 0.86
Run 9 : RMSE = 17.82 - R2_score = 0.68
Moyenne : RMSE = 17.17 - R2_score = 0.65
```

La moyenne de RMSE sur les dix modèles est égale à `17.17` et la moyenne de R^2 sur les dix modèles est égale à `0.65`. Sachant que nous ne disposons au total que de 112 observations dans l'ensemble du dataset utilisé, nous pouvons supposer que cette moyenne du RMSE sera plus au moins proche de la moyenne du RMSE qui sera obtenue en production. La même supposition peut être émise à l'égard de la moyenne du R^2. En effet, lorsque nous avons utilisé la fonction `train_test_split()`, nous avons fixé le paramètre `test_size` à 20 % ; c'est-à-dire que chacune des dix itérations sera réalisée avec un nombre d'observations dans le jeu de données de test égal, plus ou moins, à 22. Avec un peu de chance, chacune des 112 observations aura l'occasion d'appartenir aux données de test à un moment donné sur les dix itérations.

Remarque

Contrairement aux résultats obtenus précédemment avec un seul modèle, ces derniers résultats de RMSE et du R2 obtenus après la construction de ces dix modèles devront être très proches des résultats que vous obtenez avec un nombre similaire de modèles.

Voici les résultats obtenus avec 20 modèles :

```
Moyenne : RMSE = 15.35 - R2_score = 0.62
```

Voici les résultats obtenus avec 30 modèles :

```
Moyenne : RMSE = 14.17 - R2_score = 0.69
```

Voici les résultats avec 50 modèles :

```
Moyenne : RMSE = 15.93 - R2_score = 0.65
```

Et enfin avec 100 modèles :

```
Moyenne : RMSE = 15.02 - R2_score = 0.65
```

On voit très bien que le fait d'augmenter le nombre de modèles ne change pas les moyennes du RMSE et du R^2. Cela renforce l'idée que ces résultats seront similaires à ceux qui seront observés dans la réalité.

Notez que dans notre dernier programme le fait de modifier les données d'entraînement aboutit quasiment à chaque fois à un modèle différent. Lorsque ces modèles sont sensiblement différents, c'est probablement lié à la petite taille de notre DataFrame. Cela montre le pouvoir qu'ont les données sur les modèles et, de manière générale, en Machine Learning, les modèles peuvent être vus comme des fonctions dérivées directement des données. En résumé, retenons de ce dernier programme (sur lequel nous reviendrons plus tard dans ce livre) que la forme d'un modèle n'est pas uniquement influencée par l'algorithme utilisé et ses paramètres, mais aussi par les données utilisées lors de l'entraînement !

3.3.9 Cross-validation avec KFold

Lors du dernier programme réalisé, nous avons construit plusieurs modèles différents de régression linéaire en nous appuyant sur la fonction `train_test_split()`, qui génère des données d'entraînement et de test différentes à chaque exécution. Avec cette fonction de séparation de données, nous avons fixé un taux de données de test à 20 %, ce qui a comme effet de sélectionner, de manière pseudo-aléatoire, 22 observations pour les données de test sur les 112 observations présentes dans le dataset.

© Editions ENI - All rights reserved

Même si les données générées avec cette fonction sont globalement différentes d'une exécution à une autre, il se peut qu'une partie plus au moins importante des données de test ainsi générées soit souvent identique ! Par exemple, rien n'empêche que sur dix exécutions de cette fonction, certaines observations apparaissent dans huit, voire dix jeux de données de test. Cela signifie qu'avec la fonction `train_test_split()` nous ne maîtrisons pas le processus de répartition des observations entre les données d'entraînement et les données de test ; c'est là que la cross-validation avec la méthode KFold prend tout son sens !

Avec la méthode KFold, abordée au premier chapitre, nous pouvons séparer un dataset en plusieurs paquets de données. Puis plusieurs modèles peuvent être réalisés en utilisant à chaque fois un des paquets générés comme données de test et les autres paquets comme données d'entraînement. De cette manière, nous sommes certains que, d'un côté, les modèles sont tous évalués en utilisant un jeu de données de test complètement différent, et de l'autre côté, chacune des données du dataset aura servi, une seule fois, comme données de test.

Justement, dans cette section nous allons montrer comment utiliser la méthode KFold avec Scikit-learn. Suivez les étapes ci-après.

- Revenez au notebook précédent. Dans une nouvelle cellule, saisissez et exécutez l'instruction suivante pour importer la fonction `KFold` afin de l'utiliser plus tard :

```
from sklearn.model_selection import KFold
```

Nous allons commencer par montrer le fonctionnement de la méthode KFold sur un petit exemple simple, puis allons l'appliquer sur notre exemple précédent afin de générer des modèles de prédiction de la concentration de l'air avec la molécule `MaxO3`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
kf = KFold(n_splits=2, shuffle=False)
```

Dans cet exemple, la variable `kf` correspond à un objet `KFold` préconfiguré avec le paramètre `n_splits=2`, ce qui veut dire qu'on souhaite construire deux paquets à partir d'un dataset donné. Ce dataset sera désigné plus tard

lors de l'utilisation de la méthode `split()`. Nous avons également fixé le paramètre `shuffle` à `False` pour indiquer que lorsque les données seront séparées, il faudrait se baser sur les données ordonnées telles qu'elles sont dans le dataset. Nous allons observer l'effet de fixer ce dernier paramètre à `True` dans le prochain exemple.

▶ Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
for train_index, test_index in kf.split(data):
    print("Les indices de train_index = ", train_index)
    print("Les indices de test_index = ", train_index)
    print("\n\n")
```

La méthode `split()` d'un objet `KFold`, qui est appliquée ici sur notre jeu de données data, retourne une liste de couples (`train_index`, `test_index`). Chacun de ces couples contient deux listes `train_index` et `test_index`. La liste `train_index` contient des indices parmi ceux présents dans le dataset `data`. La liste `test_index` contient les indices qui ne sont pas présents dans la première liste `train_index`.

L'exécution du code précédent donne le résultat suivant :

```
Les indices de train_index =  [ 56  57  58  59  60  61  62  63
64  65  66  67  68  69  70  71  72  73 74  75  76  77  78  79  80
81  82  83  84  85  86  87  88  89  90  91 92  93  94  95  96  97
98  99 100 101 102 103 104 105 106 107 108 109 110 111]

Les indices de test_index =  [ 0  1  2  3  4  5  6  7  8  9 10 11
12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
55]


Les indices de train_index =  [ 0  1  2  3  4  5  6  7  8  9 10 11
12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
55]

Les indices de test_index =  [ 56  57  58  59  60  61  62  63  64
65  66  67  68  69  70  71  72  73 74  75  76  77  78  79  80  81
82  83  84  85  86  87  88  89  90  91 92  93  94  95  96  97  98
99 100 101 102 103 104 105 106 107 108 109 110 111]
```

© Editions ENI - All rights reserved

Comme vous pouvez le constater, il y a deux datasets construits à partir du dataset `data` :

- Un premier dataset dans lequel la liste `train_index` contient les indices de `56` jusqu'à `111` et la liste `test_index` contient les indices de `0` jusqu'à `55`.
- Un deuxième dataset dans lequel, cette fois-ci, la liste `train_index` contient les indices de `0` jusqu'à `55` et la liste `test_index` contient les indices de `56` jusqu'à `111`.

On voit très bien que les deux datasets construits avec `KFold` contiennent des données complètement différentes dans la liste `test_index`.

En résumé, afin de construire deux modèles et de les tester sur deux jeux de données de test complètement différents, il suffit d'utiliser des données de test correspondant aux indices de la liste `test_index` à chaque fois.

Voyons maintenant quel est l'effet du paramètre `shuffle` lorsqu'il est fixé à `True`.

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
kf = KFold(n_splits=2, shuffle=True)
for train_index, test_index in kf.split(data):
    print("Les indices de train_index = ", train_index)
    print("Les indices de test_index = ", train_index)
    print("\n\n")
```

Cette fois, l'argument `shuffle` est fixé à `True`. Vous obtenez des résultats similaires aux suivants :

```
Les indices de train_index =  [  1   2   3   4   6   7   8   9  10
 11  15  16  21  22  23  24  25  27 31  33  36  37  41  42  44  49
 50  53  55  56  60  61  63  64  66  67 70  71  72  74  76  77  79
 80  81  88  91  92  94  96  97 100 101 103 107 111]

Les indices de test_index =  [  0   5  12  13  14  17  18  19  20
 26  28  29  30  32  34  35  38  39 40  43  45  46  47  48  51  52
 54  57  58  59  62  65  68  69  73  75 78  82  83  84  85  86  87
 89  90  93  95  98  99 102 104 105 106 108 109 110]


 Les indices de train_index =  [  0   5  12  13  14  17  18  19  20
```

```
 26  28  29  30  32  34  35  38  39 40  43  45  46  47  48  51  52
 54  57  58  59  62  65  68  69  73  75 78  82  83  84  85  86  87
 89  90  93  95  98  99 102 104 105 106 108 109 110]

Les indices de test_index =  [  1   2   3   4   6   7   8   9  10
 11  15  16  21  22  23  24  25  27 31  33  36  37  41  42  44  49
 50  53  55  56  60  61  63  64  66  67 70  71  72  74  76  77  79
 80  81  88  91  92  94  96  97 100 101 103 107 111]
```

Comme vous pouvez le constater, cette fois les données ont été mélangées avant de construire les deux datasets ! C'est pour cette raison que dans les listes `train_index` et `test_index` nous ne retrouvons pas que des indices qui se suivent comme lorsque le paramètre `shuffle` a été fixé à `False`.

Un point important à noter ici et que le nombre de données qui seront affectées à la liste `train_index` et à la liste `test_index` est en fonction du nombre de datasets à construire et qui est fixé avec le paramètre `n_splits`. Dans l'exemple précédent, le nombre d'éléments dans les deux listes est de 56, car le nombre de datasets à construire a été fixé à 2 et le nombre total de données est égal à 112. Voyons ce qui se passe si nous fixons le nombre de datasets à construire à 3.

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
kf = KFold(n_splits=3, shuffle=False)
for train_index, test_index in kf.split(data):
    print(f"Le nombre d'éléments dans train_index =
          {train_index.shape[0]}")
    print(f"Le nombre d'éléments dans test_index =
          {test_index.shape[0]}\n")")
```

Cette fois, ce ne sont pas les données contenues dans les listes `train_index` et `test_index` qui sont affichées, mais plutôt le nombre d'éléments qu'elles contiennent grâce à l'attribut `shape`.

Le code précédent donne le résultat ci-dessous :

```
Le nombre d'éléments dans train_index = 74
Le nombre d'éléments dans test_index = 38

Le nombre d'éléments dans train_index = 75
Le nombre d'éléments dans test_index = 37
```

© Editions ENI - All rights reserved

```
Le nombre d'éléments dans train_index = 75
Le nombre d'éléments dans test_index = 37
```

Vous remarquez que le nombre d'éléments dans la liste `train_index` est supérieur au nombre d'éléments dans la liste `test_index`. En effet, dans cet exemple, la méthode `split` divise les données en trois paquets, tel qu'il a été fixé par le paramètre `n_splits = 3`. Dans chaque paquet, il y aura donc un tiers des indices présents dans le dataset `data`. Pour chaque dataset construit, c'est-à-dire à chaque itération de la boucle `for`, la méthode `split` dépose deux paquets de données dans la liste `train_index` et un seul paquet dans la liste `test_index`.

Si le paramètre `n_splits` est fixé à 5 au lieu de 3, alors la méthode `split()` va diviser les indices présents dans le dataset `data` en cinq paquets, et à chaque itération de la boucle `for` elle retournera quatre paquets dans la liste `train_index` et un paquet dans la liste `test_index`. Donc, à partir du moment où le nombre de paquets `n_splits` est supérieur à 2, la première liste retournée par la méthode `split` contient toujours plus d'éléments que la deuxième liste. C'est pour cette raison que dans les exemples précédents nous avons nommé la première liste avec le nom `train_index` afin d'indiquer que ce sont les valeurs de cette liste qui seront appropriées pour récupérer les observations qui seront utilisées pour l'entraînement d'un modèle, puisque, en règle générale, il faut utiliser 75 % ou 80 % des données pour l'entraînement d'un modèle et ne laisser que 20 % à 25 % de ces données pour l'évaluation d'un modèle.

Il faut noter que le rôle de `KFold` est de répartir les indices d'un dataset dans des listes séparées et c'est à nous d'utiliser ces listes afin de retrouver les observations correspondant à ces indices ! D'ailleurs, cette méthode `KFold` peut être utilisée à d'autres fins que la construction de modèles. En réalité, à chaque fois que dans un programme nous avons besoin d'éclater un dataset en un ensemble de sous-datasets, la méthode `KFold` peut être utilisée. Dans la suite de cette section, nous allons montrer comment utiliser `KFold` pour construire et tester des modèles de prédiction linéaires.

Maintenant que nous avons vu comment fonctionne la méthode `KFold`, nous allons pouvoir l'utiliser pour générer plusieurs modèles avec la garantie de tester chacun de ces modèles sur des données complètement différentes de celles utilisées pour tester les autres modèles.

Cette façon de tester nos modèles sur des données différentes permet d'avoir une idée plus précise du comportement moyen que va avoir un modèle une fois déployé en production.

Pour appliquer `KFold` sur notre exemple de modèle de prédiction de la concentration de l'air en ozone, suivez les étapes ci-dessous.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez le code de la fonction `create_evaluate_model` suivante :

```
def create_evaluate_model(index_fold, x_train, x_test, y_train,
y_test):
    regression_alg = LinearRegression()
    regression_alg.fit(x_train, y_train)
    test_predictions = regression_alg.predict(x_test)
    rmse = sqrt(mean_squared_error(y_test, test_predictions))
    r2 = r2_score(y_test, test_predictions)
    print(f"Run {index_fold} : RMSE = {round(rmse,2)} - R2_score
          = {round(r2,2)}")
    return (rmse, r2)
```

Cette fonction ne fait rien de nouveau par rapport à ce que nous avons vu précédemment. Elle reçoit les deux jeux de données d'entraînement et de test, `x_train`, `x_test`, `y_train`, `y_test`, puis elle applique un modèle de régression linéaire sur les données d'entraînement et elle teste le modèle obtenu sur les données de test. Pour finir, cette fonction affiche les résultats correspondant au RMSE et R^2 en préfixant cet affichage avec la valeur de `index_fold` reçue en paramètre. Cette fonction retourne également dans un tuple le couple de valeurs `(rmse, r2)`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
nb_model = 5
kf = KFold(n_splits=nb_model, shuffle=False)

index_fold = 0
average_rmse = 0
average_r2 = 0

for train_index, test_index in kf.split(data):

    x_train, x_test = data.iloc[train_index],
                      data.iloc[test_index]
```

© Editions ENI - All rights reserved

```
    y_train, y_test = y[train_index], y[test_index]

    current_rmse, current_r2 = create_evaluate_model(index_fold,
                                x_train, x_test, y_train, y_test)

    average_rmse = average_rmse + current_rmse
    average_r2 = average_r2 + current_r2

    index_fold = index_fold + 1
average_rmse = average_rmse / nb_model
average_r2 = average_r2 / nb_model

print(f"Moyenne : RMSE = {round(average_rmse,2)} - R2_score =
{round(average_r2,2)}")
```

Ce code utilise `KFold` avec un nombre de paquets égal à 5. Puis, comme nous l'avons fait précédemment, à chaque itération de la boucle `for`, les valeurs des deux listes `train_index` et `test_index` sont récupérées et utilisées pour construire les deux jeux de données `(x_train, y_train)` et `(x_test, y_test)`. Puis la fonction `create_evaluate_model` est appelée en lui passant les paramètres correspondant aux deux jeux de données d'entraînement et de test ainsi que la valeur de `index_fold` associée à l'indice de l'itération en cours de la boucle `for`. Le résultat retourné par la fonction `create_evaluate_model` est utilisé pour calculer et afficher la moyenne des deux métriques RMSE et R^2 comme nous l'avons fait précédemment.

Le code précédent donne ce résultat :

```
Run 0 : RMSE = 15.25 - R2_score = 0.55
Run 1 : RMSE = 18.13 - R2_score = 0.2
Run 2 : RMSE = 19.2 - R2_score = 0.74
Run 3 : RMSE = 16.31 - R2_score = 0.68
Run 4 : RMSE = 8.4 - R2_score = 0.62
Moyenne : RMSE = 15.46 - R2_score = 0.56
```

On voit que la moyenne du RMSE est très proche de la moyenne obtenue lorsque nous avions généré précédemment des données d'entraînement et de test en nous basant sur des exécutions multiples de la fonction `train_test_split`. En revanche, la valeur moyenne de R^2 est sensiblement plus faible avec la méthode `KFold`.

En règle générale, `KFold` donne des estimations fiables par rapport au comportement qu'on peut rencontrer avec un modèle une fois mis en production, et c'est d'autant plus vrai que la taille des données d'entraînement et de test est plus grande.

Remarque

Dans cette section, nous avons abordé la méthode `KFold` en l'appliquant dans le cadre de génération de modèles linéaires. Dans la suite de cet ouvrage, nous aurons l'occasion de réutiliser cette méthode dans le cadre de génération de modèles de classification.

Remarque

Pour rappel, vous trouverez le code de tous ces modèles de régression linéaire multiple dans le notebook 07-régression_linéaire.ipynb contenu dans le sous-dossier Code de ce chapitre.

4. La régression polynomiale

La section précédente a montré que la régression linéaire multiple est une généralisation de la régression linéaire simple. Dans le même ordre d'idée, la régression polynomiale est elle-même une généralisation de la régression linéaire multiple. En effet, un modèle de régression polynomiale est un polynôme à plusieurs variables. Par exemple, pour un modèle polynomial d'ordre 2, la fonction F sera définie comme suit :

$$F(x_{i,1}, x_{i,2}, \ldots, x_{i,m}) = a_1 x_{i,1}^2 + a_2 x_{i,1}^1 + a_3 x_{i,2}^2 + a_4 x_{i,2}^1 + \ldots + a_{2m-1} x_{i,m}^2 + a_{2m} x_{i,m}^1 + a_0$$

Les modèles polynomiaux permettent de décrire des relations entre les variables qui ne sont pas linéaires. C'est là où réside leur avantage majeur vis-à-vis des modèles de régression linéaire. Cependant, ces modèles nécessitent des temps de calcul plus importants et ils sont sujets au problème de surapprentissage de façon relativement importante.

© Editions ENI - All rights reserved

4.1 Exemple de régression polynomiale

4.1.1 Construction d'un modèle polynomial

Dans ce nouvel exemple, nous allons utiliser le même jeu de données que celui utilisé lorsque nous avons développé des modèles de régression linéaire précédemment dans ce chapitre. Nous allons donc construire des modèles de régression polynomiale pour la prédiction de la pollution de l'air avec l'ozone.

Pour développer notre premier modèle de régression polynomiale, suivez les étapes ci-après.

- Commencez par la création d'un nouveau notebook Jupyter. Vous pouvez utiliser le notebook *07-régression_polynomiale.ipynb* contenu dans le sous-dossier *Code* de ce chapitre.
- Dans la première cellule de votre notebook, saisissez les instructions suivantes puis exécutez cette cellule afin d'importer les modules nécessaires à la réalisation de votre modèle de régression polynomiale :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn import preprocessing
from math import sqrt
import matplotlib.pyplot as plt

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
```

Les sept premières instructions d'import ci-dessus sont exactement les mêmes que celles utilisées dans les exemples précédents de ce chapitre. Les deux nouveaux modules utilisés ici sont les suivants :

- `sklearn.pipeline` : ce module peut être vu comme un outil qui permet le développement de modèles polynomiaux de manière plus simple.
- `PolynomialFeatures` : ce module permet de calculer les formes polynomiales des différentes variables d'un dataset.

Notez que nous aurions pu combiner la dernière instruction d'import avec une autre instruction d'import déjà existante et qui correspond à la directive où nous avons importé le module `preprocessing`, mais nous avons préféré séparer ces deux directives d'import afin de mettre en avant ce nouveau module importé.

Un modèle de régression polynomiale se calcule avec la même démarche que pour un modèle de régression simple ou multiple. La seule différence réside dans l'utilisation du module `PolynomialFeatures` qui permet d'ajouter de nouvelles variables à un dataset. Ces nouvelles variables correspondent aux différents calculs de puissance des variables existantes ; par exemple, pour une variable x, ce module va calculer x^2, x^3, x^4, etc. Le degré des polynômes ainsi calculé est défini par un paramètre de ce même module `PolynomialFeatures`.

▶ Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
data = pd.read_csv('../Data/ozone.csv', sep=',')

y = data["MaxO3"]

data= data[
          ['T9', 'T12', 'T15', 'Ne9', 'Ne12', 'Ne15', 'Vx9',
           'Vx12', 'Vx15', 'MaxO3v']
]

def normalization(dataToNormalize):
    columns = dataToNormalize.columns
    for col in columns:
        x = dataToNormalize[[col]].values.astype(float)
        standard_normalization = preprocessing.StandardScaler()
        res = standard_normalization.fit_transform(x)
        dataToNormalize[col]=res

normalization(data)
```

Le code précédent est exactement le même que celui de l'exemple de régression linéaire multiple, puisqu'il concerne uniquement la partie de lecture et de normalisation des données. Nous n'allons donc pas l'expliquer dans les détails.

© Editions ENI - All rights reserved

À ce stade, nous disposons donc de notre jeu de données `data` qui contient les variables explicatives et le vecteur y qui contient les données de la variable à expliquer, qui correspondent, pour rappel, aux valeurs observées de la variable `MaxO3`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.20)
```

Cette instruction permet, comme nous l'avons également vu précédemment, de séparer le dataset en deux jeux de données : un jeu de données d'entraînement (`x_train, y_train`) et un jeu de données de test (`x_test, y_test`), avec la taille du jeu de données de test égale à 20 % de la taille totale du dataset `data`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
polynomial_features = PolynomialFeatures(degree=2)
```

Cette dernière instruction permet de créer un objet qui aura la charge de calculer les puissances de 2 de toutes les variables explicatives.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
poly_regression_alg = LinearRegression()
```

L'objet `poly_regression_alg` représente notre algorithme de régression.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model = Pipeline([
                ("polynomial_features", polynomial_features),
                ("linear_regression", poly_regression_alg)
        ])
```

L'objet `model` est associé à une combinaison entre l'algorithme `poly_regression_alg` de régression et l'objet `polynomial_features` qui sera utilisé pour le calcul des puissances de 2 des variables.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin de lancer le calcul du modèle :

```
model.fit(x_train, y_train)
```

C'est lors de l'exécution de cette dernière instruction que l'objet `polynomial_features` calcule les puissances de 2 de toutes les variables explicatives et ajoute ces nouvelles variables ainsi calculées aux variables existantes, puis l'objet `poly_regression_alg` lance la procédure de calcul du modèle.

Une fois l'exécution de la fonction `fit` terminée et avant d'évaluer le modèle obtenu sur les données de test, nous allons d'abord jeter un coup d'œil au comportement de ce modèle vis-à-vis des données d'entraînement ! L'objectif de cette opération est de voir quel est le niveau ou le degré d'apprentissage acquis par ce modèle par rapport aux données utilisées lors de l'entraînement.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant pour afficher le RMSE et le R^2 obtenus après l'application du modèle polynomial sur les données d'entraînement :

```
train_predictions = model.predict(x_train)

print(f"RMSE = {round(sqrt(mean_squared_error(y_train,
train_predictions)),2)}")

print(f"R2_score = {round(r2_score(y_train,
train_predictions),2)}")
```

Remarquez bien que dans ce dernier code la fonction `predict` est utilisée avec les données `x_train` et non pas avec les données `x_test`.

Les résultats obtenus sont les suivants :

```
RMSE = 5.96
R2_score = 0.96
```

Remarque

Vous pouvez obtenir des résultats différents en fonction de la répartition des observations entre les données d'entraînement et les données de test par la fonction `train_test_split`.

© Editions ENI - All rights reserved

On voit très bien que notre modèle possède un niveau de prédiction élevé sur les données d'entraînement ! C'est normal dans la mesure où ces données sont bien connues par notre modèle, puisque ce sont elles qui sont utilisées pour fixer les paramètres de ce même modèle.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant pour visualiser la relation entre les valeurs prédites par votre modèle et les vraies valeurs observées `y_train` relatives aux données d'apprentissage :

```
plt.scatter(y_train, train_predictions, color='black')
plt.title("les prédictions du modèle vs la réalité")
plt.xlabel("les valeurs observées")
plt.ylabel("Les prédictions")
plt.plot([40.0, 160.0], [40.0, 160.0], 'red', lw=1)
plt.show()
```

Les résultats sont affichés dans la figure suivante :

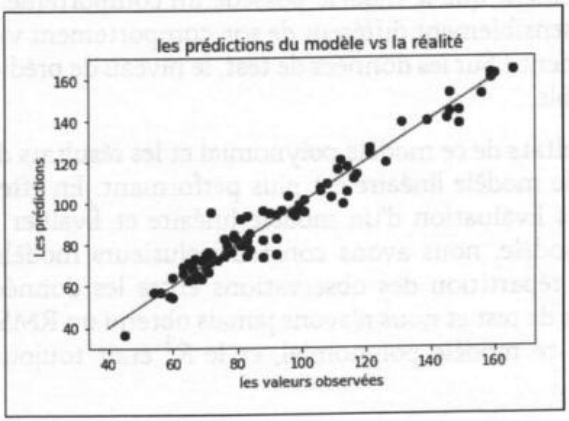


Figure 7-7 : représentation des prédictions en fonction des observations sur les données d'apprentissage du modèle polynomial d'ordre 2

Cette figure confirme visuellement que le modèle obtenu est bien conforme aux données qu'il a utilisées lors de la phase d'apprentissage. En effet, on constate que les points noirs se resserrent bien autour de la droite rouge qui, elle, correspond à un modèle parfait qui prédit avec exactitude les valeurs observées.

Voyons maintenant comment ce même modèle se comporte vis-à-vis des données de test qu'il n'a pas rencontrées lors de la phase d'apprentissage.

▶ Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant afin d'afficher le RMSE et le R^2 après l'application du modèle pour la prédiction de la variable `MaxO3` associée aux données de test :

```
test_predictions = model.predict(x_test)
print(f"RMSE = {round(sqrt(mean_squared_error(y_test,
test_predictions)),2)}")
print(f"R2_score = {round(r2_score(y_test,
test_predictions),2)}")
```

Le code précédent donne les résultats suivants :

```
RMSE = 40.26
R2_score = -2.05
```

Ces résultats montrent clairement que le modèle possède un comportement vis-à-vis des données de test sensiblement différent de son comportement vis-à-vis des données d'entraînement ! Sur les données de test, le niveau de prédiction de ce modèle est très faible.

La comparaison entre les résultats de ce modèle polynomial et les résultats du modèle linéaire montre que le modèle linéaire est plus performant. En effet, dans les sections précédentes Évaluation d'un modèle linéaire et Évaluer le futur comportement d'un modèle, nous avons construit plusieurs modèles linéaires en faisant varier la répartition des observations entre les données d'apprentissage et les données de test et nous n'avons jamais obtenu un RMSE aussi élevé que le RMSE de ce modèle polynomial, et le R^2 était toujours positif !

Il est assez déroutant de voir un indice de performance noté avec une lettre au carré R^2 qui soit négatif. La section suivante explique comment calculer le R^2 et montre aussi dans quelle situation il peut être négatif.

© Editions ENI - All rights reserved

- Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant pour visualiser la relation entre les valeurs prédites par votre modèle et les vraies valeurs observées `y_test` relatives aux données de test :

```
plt.scatter(y_test, test_predictions, color='black')
plt.title("les prédictions du modèle vs la réalité")
plt.xlabel("les valeurs observées")
plt.ylabel("Les prédictions")
plt.plot([40.0, 160.0], [40.0, 160.0], 'red', lw=1)
plt.show()
```

Ce code affiche le graphique suivant :

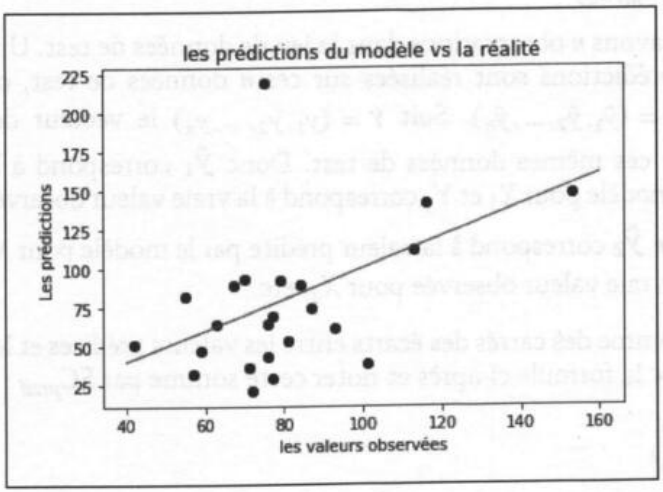


Figure 7-8 : représentation des prédictions en fonction des observations sur les données de test du modèle polynomial d'ordre 2

Les deux derniers graphiques montrent clairement que ce modèle polynomial a des comportements très divergents entre les données d'entraînement et les données de test. Cette fois, sur les données de test, les points noirs ne se resserren plus autour de la droite rouge comme sur la figure précédente lorsque nous avions tracé le même graphique sur les données d'entraînement.

4.1.2 Le coefficient de détermination R^2

L'évaluation d'un modèle consiste à utiliser ce modèle pour prédire les valeurs de la variable à expliquer en utilisant un jeu de données de test.

Soit F la fonction correspondant à un modèle de régression obtenu après une phase d'apprentissage sur un ensemble de données. Pour toute observation $X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,m})$ du jeu de données de test, on obtient la prédiction $\hat{y}_i$:

$$F(X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,m})|P) = \hat{y}_i$$

Avec : $P = (a_1, a_2, \dots, a_m, a_0)$

Supposons que nous avons n observations dans le jeu de données de test. Une fois que toutes les prédictions sont réalisées sur ces n données de test, on obtient le vecteur $\hat{Y} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)$. Soit $Y = (y_1, y_2, \dots, y_n)$ le vecteur des valeurs observées sur ces mêmes données de test. Donc $\hat{y}_1$ correspond à la valeur prédite par le modèle pour X_1 et Y_1 correspond à la vraie valeur observée pour X_1, de même que $\hat{y}_2$ correspond à la valeur prédite par le modèle pour X_2 et y_2 correspond à la vraie valeur observée pour X_2, etc.

On peut calculer la somme des carrés des écarts entre les valeurs prédites et les valeurs observées avec la formule ci-après et noter cette somme par SC_{pred} :

$$SC_{pred} = \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

Également, nous pouvons calculer la somme des carrés des écarts des valeurs observées par rapport à leur moyenne $\bar{Y}$ et noter cette somme par SC_{obs} :

$$SC_{obs} = \sum_{i=1}^{n} (\bar{Y} - y_i)^2$$

Avec $\bar{Y} = \frac{\sum_{i=1}^{n} y_i}{n}$

© Editions ENI - All rights reserved

Le coefficient de détermination R^2 se calcule comme suit :

$$R^2 = 1 - \frac{SC_{pred}}{SC_{obs}}$$

Ce coefficient de détermination peut être interprété comme étant la proportion de la variance de la variable à prédire expliquée par le modèle.

On peut distinguer trois cas particuliers par rapport aux valeurs possibles de R^2, à savoir :

1. Lorsque $SC_{pred}=0$, cela signifie que le modèle réalise des prédictions parfaites de la variable à prédire en utilisant les données X_i, c'est-à-dire $\hat{Y} = Y$. Dans ce cas, on obtient $R^2 = 1 - \frac{0}{SC_{obs}} = 1 - 0 = 1$.

Donc, lorsque $R^2=1$, cela signifie que le modèle explique parfaitement l'influence des variables X_i sur la variance de la distribution associée à la variable à expliquer.

2. Lorsque $SC_{pred}=SC_{obs}$, cela signifie que le modèle n'explique aucunement la variable à prédire à partir des données X_i, puisqu'on aurait obtenu les mêmes résultats avec un modèle correspondant à une fonction constante qui retourne toujours la moyenne $\bar{Y}$ et cela quelles que soient les valeurs des variables explicatives. Dans ce cas, on obtient :

$$R^2 = 1 - \frac{SC_{pred}}{SC_{obs}} = 1 - \frac{SC_{obs}}{SC_{obs}} = 1 - 1 = 0$$

Donc lorsque $R^2=0$, le modèle n'explique pas du tout l'influence des variables X_i sur la variation de la variable à expliquer.

3. Lorsque $SC_{pred} > SC_{obs}$ on obtient $\frac{SC_{pred}}{SC_{obs}} > 1$

   Donc R^2 est égal à 1 moins une valeur supérieure à 1, ce qui donne une valeur de R^2 négative.

   Lorsque $R^2 < 0$, cela signifie que les prédictions sont sensiblement loin des vraies valeurs observées et donc le modèle utilisé est de très mauvaise qualité.

4.1.3 R^2 et les valeurs extrêmes

Il faut noter que R^2 est très sensible aux valeurs extrêmes. Supposons un modèle *F* qui réalise les prédictions suivantes :

$(\hat{y}_1 = y_1, \hat{y}_2 = y_2, \ldots \hat{y}_{(n-1)} = y_{(n-1)}, \hat{y}_n \gg y_n)$

Ici, le modèle trouve les valeurs exactes pour les valeurs observées $(y_1, y_2, \ldots, y_{(n-1)})$, sauf pour la valeur y_n où il a prédit une valeur beaucoup plus grande. Supposons que $\hat{y}_n - y_n$ est beaucoup plus grande que la variance totale de la variable à prédire *Y*. Cela va mener à un S^2 négatif puisque SC_{pred} sera supérieur à SC_{obs}. Donc, théoriquement, il suffit qu'un modèle se trompe de manière significative sur la prédiction de quelques valeurs pour que le coefficient de détermination R^2 soit négatif !

En synthèse, le coefficient de détermination est intéressant puisqu'il donne une idée sur le degré d'utilisation des variables explicatives pour prédire la variable à expliquer, mais il faut être vigilant sur l'effet des valeurs extrêmes !

4.1.4 Modèle polynomial et surapprentissage

Dans la section précédente, nous avons calculé le RMSE et le R^2 d'un modèle polynomial de degré égal à 2 et nous avons constaté que ce modèle donne des prédictions de bonne qualité sur les données d'apprentissage ; en revanche, il donne des prédictions de très mauvaise qualité sur les données de test.

© Editions ENI - All rights reserved

Dans cette section, nous allons construire un autre modèle polynomial d'ordre 3 afin de voir si oui ou non le fait d'augmenter l'ordre d'un modèle polynomial améliore la qualité de ses prédictions.

Pour réaliser un modèle polynomial d'ordre 3, suivez les quelques étapes ci-dessous.

- Dans le même notebook Jupyter où vous avez réalisé le modèle polynomial d'ordre 2 précédemment, repérez la cellule où vous avez instancié l'objet `polynomial_features`, puis, dans cette même cellule, remplacez la valeur du paramètre `degree` avec la valeur 3, comme ci-après, et exécutez cette cellule afin que votre modification prenne effet :

```
polynomial_features = PolynomialFeatures(degree=3)
```

- Repérez et exécutez la cellule où la méthode `fit` a été utilisée afin de relancer le processus de calcul des polynômes d'ordre 3 ainsi que le processus d'apprentissage sur les données `(x_train, y_train)`.

Cette fois, lors du lancement de la méthode `fit`, l'objet va calculer pour chaque variable x_i la puissance de deux x_i^2 et la puissance de trois x_i^3.

- Une fois que l'exécution de la méthode `fit` est terminée, exécutez la cellule où le modèle `model` est appliqué pour faire des prédictions sur le jeu de données d'entraînement `x_train` et où vous avez affiché le RMSE et le R^2 associés à ces prédictions.

Ce modèle polynomial d'ordre 3 a donné les résultats suivants sur les données d'apprentissage :

```
RMSE = 0.0
R2_score = 1.0
```

On voit que les prédictions de ce modèle polynomial d'ordre 3 épousent parfaitement les vraies valeurs observées !

Ces résultats signifient que ce modèle parvient à expliquer la totalité de la variance de la variable à prédire en se basant sur les variables explicatives.

- Réexécutez la cellule permettant d'afficher le graphique où les valeurs prédites sont tracées en fonction des vraies valeurs observées sur le jeu de données d'entraînement.

Vous obtenez un résultat similaire au résultat de la figure ci-dessous :

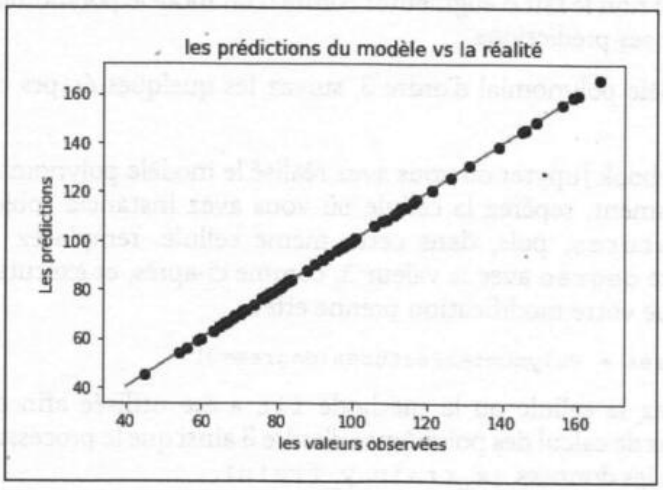


Figure 7-9 : représentation des prédictions en fonction des observations sur les données d'apprentissage du modèle polynomial d'ordre 3

Sur ce dernier graphique, on remarque que les points noirs coïncident exactement avec la droite rouge qui, elle, représente un modèle parfait. Ce n'est bien sûr pas une surprise puisque nous savons déjà que le RMSE est égal à 0. Voyons maintenant ce que donne ce modèle polynomial de degré 3 sur les données de test.

- Réexécutez la cellule où le modèle `model` est appliqué pour faire des prédictions sur le jeu de données d'entraînement `x_test` et où vous avez affiché le RMSE et le R^2 associés à ces prédictions.

Les résultats obtenus sont les suivants :

```
RMSE = 21.7
R2_score = 0.11
```

Ici, on voit que sur les données de test ce modèle polynomial d'ordre 3 donne des résultats meilleurs que ceux obtenus avec le modèle polynomial d'ordre 2 que nous avons construit précédemment.

© Editions ENI - All rights reserved

Cependant, vu les résultats parfaits obtenus avec ce modèle polynomial d'ordre 3 sur les données d'entraînement, on aurait espéré des résultats meilleurs que ces résultats obtenus sur les données de test.

- Réexécutez la cellule qui permet d'afficher le graphique où les valeurs prédites sont tracées en fonction des vraies valeurs observées sur le jeu de données de test :

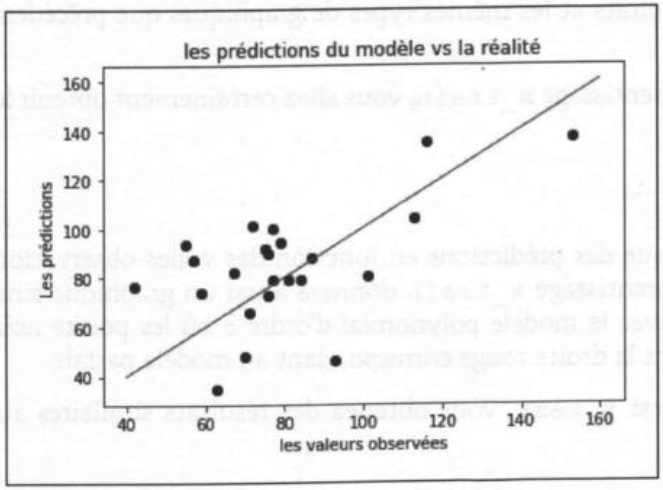


Figure 7-10 : représentation des prédictions en fonction des observations sur les données de test du modèle polynomial d'ordre 3

La comparaison entre les deux derniers graphiques montre que les performances irréprochables obtenues par ce modèle polynomial d'ordre 3 sur les données d'entraînement sont dues au surapprentissage et non pas à une découverte d'une corrélation réelle entre les variables explicatives et la variable à expliquer.

Pour constater ce phénomène de surapprentissage de manière encore plus importante, nous allons suivre les mêmes étapes que précédemment pour construire un modèle polynomial d'ordre 6.

- Repérez la cellule où vous avez instancié l'objet `polynomial_features`, puis, dans cette même cellule, remplacez la valeur du paramètre `degree` avec la valeur 6, comme ci-après, et exécutez cette cellule afin que votre modification prenne effet :

```
polynomial_features = PolynomialFeatures(degree=6)
```

- Réexécutez toutes les cellules situées après la cellule précédente afin d'afficher les mêmes résultats et les mêmes types de graphiques que précédemment.

Sur les données d'apprentissage `x_train`, vous allez certainement obtenir les résultats suivants :

```
RMSE = 0.0
R2_score = 1.0
```

Le traçage du graphique des prédictions en fonction des vraies observations sur les données d'apprentissage `x_train` donnera aussi un graphique similaire à celui obtenu avec le modèle polynomial d'ordre 3 où les points noirs épousent parfaitement la droite rouge correspondant au modèle parfait.

Sur les données de test `x_test`, vous obtenez des résultats similaires aux résultats suivants :

```
RMSE = 192.92
R2_score = -69.13
```

Remarquez que le R^2 est négatif et s'éloigne de 0 de façon importante. Cela veut tout simplement dire que les prédictions de ce modèle sur les données de test s'écartent de façon importante des vraies valeurs observées.

Vous allez également obtenir un graphique similaire au suivant sur les données de test où les valeurs prédites sont tracées en fonction des vraies valeurs observées.

© Editions ENI - All rights reserved

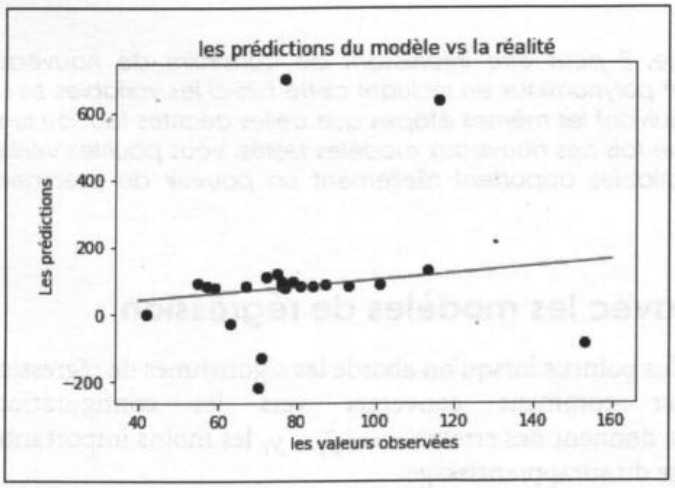


Figure 7-11 : représentation des prédictions en fonction des observations sur les données de test du modèle polynomial d'ordre 6

Ce graphique montre clairement que le niveau des prédictions est très faible et certaines prédictions sont aberrantes. En effet, le modèle prédit des valeurs de `MaxO3` qui s'éloignent de la droite rouge de manière importante et il prédit aussi des valeurs négatives, chose qui est impossible puisque la concentration de l'air en ozone ne peut être que positive !

En résumé, on peut parler du problème de surapprentissage lorsqu'un modèle possède un niveau de prédiction élevé sur les données d'apprentissage, mais présente un niveau de prédiction faible sur les données de test. C'est pour cette raison que la cross-validation est intéressante à mettre en œuvre de manière systématique, car elle nous permet justement d'avoir une idée plus précise du comportement que va avoir un modèle une fois déployé en production pour ainsi nous éviter de mauvaises surprises.

Remarque

En guise d'exercice, il peut être intéressant de construire de nouveaux modèles linéaires et polynomiaux en incluant cette fois-ci les variables `Data`, `Pluie` *et* `Vent` *en suivant les mêmes étapes que celles décrites tout au long de ce chapitre. Une fois ces nouveaux modèles testés, vous pourriez vérifier si ces nouvelles variables apportent réellement un pouvoir de prédiction supplémentaire.*

5. Aller plus loin avec les modèles de régression

Un des aspects les plus pointus lorsqu'on aborde les algorithmes de régression est de déterminer comment converger vers les configurations $(a_1, a_2, \ldots, a_m, a_0)$ qui donnent des erreurs $\varepsilon = \hat{y}_i - y_i$ les moins importantes et qui évitent le piège du surapprentissage.

Lorsque nous avons abordé la régression linéaire au début de ce chapitre, nous avons vu que la fonction d'erreur à minimiser est la suivante :

$$E = \sum_{i=1}^{n} [y_i - F(x_{i,1}, x_{i,2}, \ldots, x_{i,m})]^2$$

L'approche la plus connue qui permet de minimiser la fonction de calcul d'erreur E tout en évitant le problème de surapprentissage est l'introduction du concept de régularisation.

Il existe deux régularisations très utilisées avec les modèles de régression : la régularisation L_1, également appelée régularisation Lasso, et la régularisation L_2, ou régularisation Ridge.

5.1 La régularisation Lasso

Avec la régularisation Lasso (*Least Absolute Shrinkage and Selection Operator*), la fonction d'erreur à minimiser devient la suivante :

$$E_{Lasso} = \sum_{i=1}^{n} [y_i - F(x_{i,1}, x_{i,2}, \ldots, x_{i,m})]^2 + \lambda \sum_{i=0}^{m} |a_i|$$

© Editions ENI - All rights reserved

La différence entre E et E_{Lasso} est que dans E_{Lasso} nous avons ajouté le terme $\lambda \sum_{i=0}^{m} |a_i|^2$ pour sanctionner davantage les solutions avec des valeurs de $(a_1, a_2, \ldots, a_m, a_0)$ plus grandes.

5.2 La régularisation Ridge

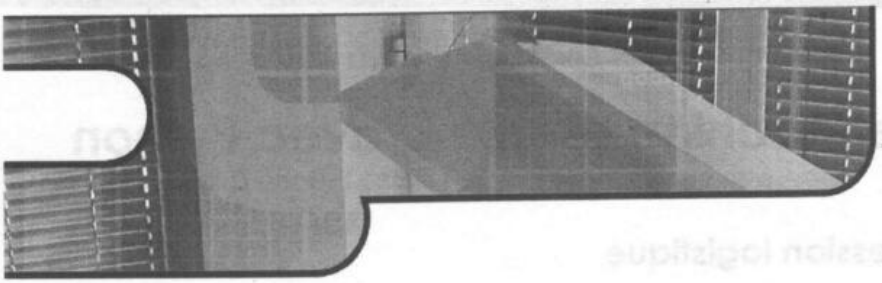
Avec la régularisation Ridge, les grandes valeurs de $(a_1, a_2, \ldots, a_m, a_0)$ sont davantage sanctionnées et la fonction d'erreur à minimiser devient la suivante :

$$E_{Ridge} = \sum_{i=1}^{n} [y_i - F(x_{i,1}, x_{i,2}, \ldots, x_{i,m})]^2 + \lambda \sum_{i=0}^{m} a_i{}^2$$

Remarquez que dans les deux cas de régularisation, Lasso et Bridge, lorsque la valeur de λ est fixée à 0, alors $E = E_{Lasso} = E_{Ridge}$.

6. Conclusion

Ce chapitre a permis d'introduire la régression linéaire et la régression polynomiale. Dans un premier temps, la régression linéaire a été abordée du point de vue géométrique pour donner l'intuition des algorithmes de régression, et dans un second temps la méthode des moindres carrés a été expliquée en détail. Puis des exemples concrets de régression multiple et de régression polynomiale ont été traités avec Scikit-learn. Nous avons profité des exemples réalisés afin d'aborder les notions de normalisation de données, de cross-validation et de surapprentissage et avons aussi détaillé le coefficient de détermination R^2. Le chapitre suivant montre comment un algorithme de régression peut être utilisé pour développer un modèle de classification en introduisant la régression logistique.



Chapitre 4-2
La régression logistique

1. Objectif du chapitre

Le modèle de la régression logistique est un cas particulier de modèle linéaire généralisé et c'est l'un des modèles de classification les plus répandus. Il est utilisé dans des domaines tels que le domaine médical, le domaine social ou le domaine bancaire : par exemple, détecter la présence ou non d'une tumeur chez un patient, expliquer un risque suicidaire après une dépression nerveuse, ou encore prédire si un client pourra ou non rembourser son crédit. De façon plus générale, avec la régression logistique, on cherche à détecter les prémisses de la survenue d'un phénomène.

L'objectif de ce chapitre est d'expliquer les notions de base de la régression logistique et de montrer un exemple de modèle de régression logistique développé avec Scikit-learn.

À la fin de ce chapitre, le lecteur aura abordé :

- une justification et l'intuition des algorithmes de régression logistique,
- quelques notions théoriques de la régression logistique,
- l'application de la régression logistique pour prédire les survivants du naufrage du Titanic en utilisant la bibliothèque Scikit-learn,
- le seuil de décision d'un modèle de régression logistique,
- l'algorithme One-vs-All.

2. La régression logistique

Le chapitre précédent a abordé les concepts de la régression. Nous avons vu que si nous disposons de n couples de la forme $(X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,m}) \in \mathbb{R}^m, y_i \in \mathbb{R})$, avec y_i le résultat obtenu pour l'observation $X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,m})$, alors on peut construire un modèle prédictif linéaire de la forme $F(x_{i,1}, x_{i,2}, \ldots, x_{i,m}) = a_1 x_{i,1} + a_2 x_{i,2} + \cdots + a_m x_{i,m} + a_0$

Ce modèle permet de prédire les y_i futurs pour les nouvelles observations X_i. Le problème est que si $Y = \{y_i / i = 1..n\}$ est une variable binaire ou une variable qualitative, alors il n'est plus question de prédire un nombre réel, mais plutôt la classe associée à une observation. Dans ce dernier cas, le but recherché est un modèle de classification, et les modèles de régression linéaire ne sont plus adaptés puisque la fonction linéaire $F(x_{i,1}, x_{i,2}, \ldots, x_{i,m})$ est définie dans $\mathbb{R}$.

Comme nous allons le voir dans la suite de ce chapitre, l'idée de la régression logistique est de transformer le problème de l'explication d'une variable continue en un problème d'explication d'une probabilité. En d'autres termes, les algorithmes de régression logistique sont une adaptation des algorithmes de régression afin de ne plus prédire une valeur réelle, mais plutôt une classe d'appartenance associée à une observation.

Supposons que nous disposons de n observations de deux variables. La première variable indique l'âge d'une personne et la deuxième indique son état de santé. Pour une personne donnée, la deuxième variable vaut 1 si elle est malade et elle vaut 0 si cette personne est saine.

© Editions ENI - All rights reserved

Supposons que ces deux variables sont réparties comme dans la figure suivante :

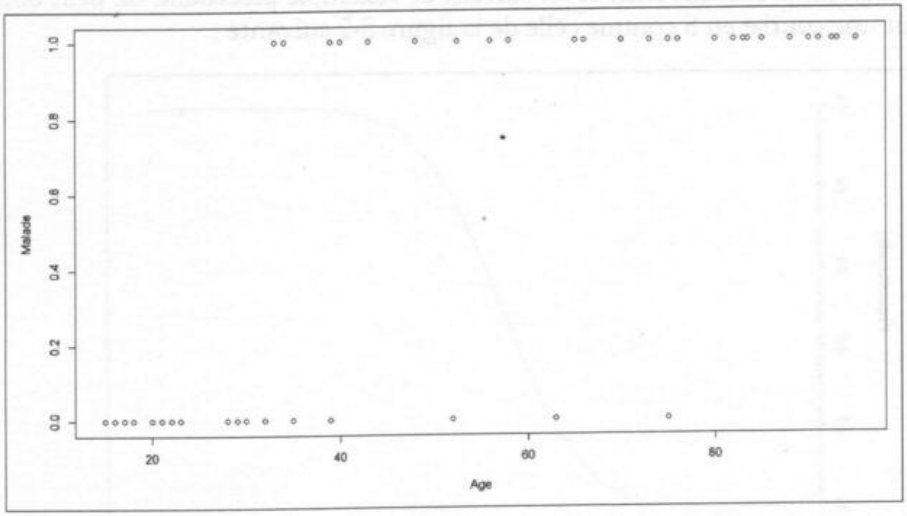


Figure 8-1 : exemple de la répartition de la variable binaire `Malade` *en fonction des valeurs de la variable* `Age`

Si nous essayons d'établir un modèle linéaire simple pour prédire l'état de santé d'une personne à partir de son âge, nous sommes tout de suite confrontés au problème de la non-continuité de la variable que nous souhaitons prédire et qui ne peut prendre que l'une des deux valeurs : `0` pour sain ou `1` pour malade.

Pour contourner ce problème, il suffit de découper la variable Age en plusieurs classes et de calculer pour chaque classe la proportion de malades dans cette classe. En procédant ainsi et en partant de l'exemple précédent, on peut obtenir une courbe en S comme celle de la figure 8-2 suivante :

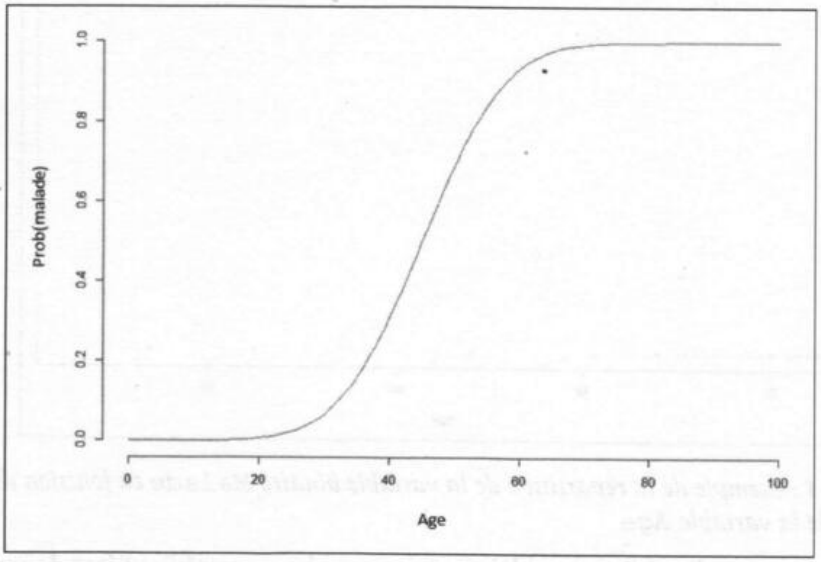


Figure 8-2 : la probabilité d'être malade en fonction de l'âge

Avec une telle courbe, pour une valeur de la variable Age, on obtient une valeur $v \in [0,1]$ indiquant la probabilité que la personne soit malade. Maintenant, au lieu de rechercher un modèle pour prédire si une personne est malade ou non, nous cherchons un modèle pour prédire la probabilité qu'une personne soit malade à partir de son âge. Finalement, nous recherchons donc un modèle qui prédit une valeur continue, et cette fois les modèles de régression seront adaptés.

Même si les valeurs de notre variable à prédire sont devenues continues, on ne peut toujours pas proposer un modèle de régression linéaire, puisque les fonctions linéaires prennent leurs valeurs dans $\mathbb{R}$.

© Editions ENI - All rights reserved

En d'autres termes, un modèle linéaire peut proposer des valeurs supérieures à 1 ou inférieures à 0, ce qui ne correspond pas à des probabilités :

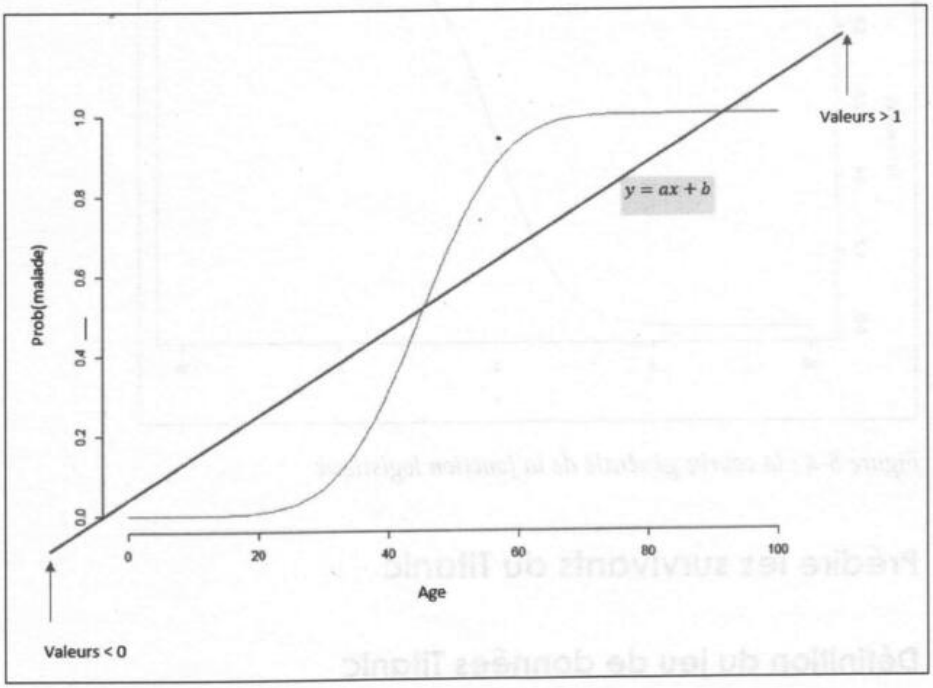


Figure 8-3 : un modèle linéaire pour prédire la probabilité d'être malade

Pour contourner ce nouveau problème, nous avons besoin d'une fonction qui sait transformer le résultat d'une fonction linéaire en une valeur comprise dans l'intervalle `[0,1]`. Les fonctions qui savent faire ce genre de transformation s'appellent des fonctions sigmoïdes.

La fonction sigmoïde la plus simple et qui porte le nom de la fonction logistique est définie comme suit $\frac{1}{1+e^{-F(x_{i,1},x_{i,2},\ldots,x_{i,m})}}$

La figure ci-dessous donne la représentation graphique de cette fonction.

Quelle que soit la valeur prise par la fonction $F(x_{i,1}, x_{i,2}, \ldots, x_{i,m})$, cette fonction logistique nous retourne une valeur dans l'intervalle `[0,1]`.

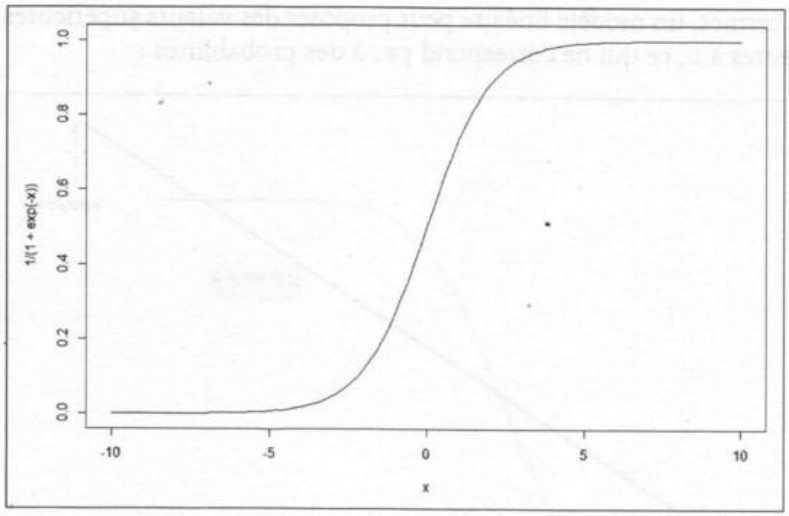


Figure 8-4 : la courbe générale de la fonction logistique

3. Prédire les survivants du Titanic

3.1 Définition du jeu de données Titanic

Dans cet exemple, un jeu de données très connu dans la communauté des Data Scientists sera utilisé. Il s'agit d'un sous-ensemble des passagers du fameux navire Titanic. L'objectif de cet exemple est de construire un modèle qui sait prédire pour un passager particulier s'il a survécu ou pas à ce drame. Ce jeu de données indique pour chaque passager les informations suivantes :

- `PassengerId` : identifiant d'un passager.
- `Survived` : variable binaire indiquant si le passager a survécu au drame ou non.
- `Pclass` : variable indiquant la classe de la cabine. Elle prend ses valeurs parmi les valeurs `1`, `2` et `3` qui correspondent respectivement à la première, seconde et troisième classe.

© Editions ENI - All rights reserved

- `Sex` : variable indiquant le sexe du passager, `Male` ou `Female`.
- `Age` : variable indiquant l'âge du passager.
- `SibSp` : variable indiquant si le passager a des frères, des sœurs, un époux ou une épouse à bord du bateau.
- `Parch` : variable indiquant si le passager a des parents ou des enfants à bord du bateau.
- `Fare` : prix du ticket.
- `Embarked` : port d'embarcation (`C` = Cherbourg, `Q` = Queenstown, `S` = Southampton).

En résumé, l'objectif est de prédire la variable `Survived` à partir des valeurs des variables `Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare` et `Embarked`. La variable `PassengerId` ne sera pas utilisée, car nous supposons qu'elle n'a pas d'impact sur la variable `Survived`.

3.2 Réalisation du modèle de régression logistique

Afin de construire un modèle de régression logistique avec Scikit-learn pour prédire la variable `Survived`, suivez les étapes ci-après.

- Commencez par la création d'un nouveau notebook Jupyter, vous pouvez utiliser le notebook *08-régression_logistique.ipynb* contenu dans le sous-dossier *Code* de ce chapitre.

3.2.1 Chargement des modules Scikit-learn

- Dans la première cellule de votre notebook, saisissez les instructions suivantes, puis exécutez cette cellule afin d'importer les modules nécessaires à la réalisation de notre modèle :

```
import matplotlib.pyplot as plt
%matplotlib inline
import pandas as pd
from math import sqrt
import seaborn as sb
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import  LogisticRegression
from sklearn.metrics import precision_recall_fscore_support as score
import warnings
```

```
warnings.filterwarnings('ignore')
```

Nous avons rencontré la plupart de ces modules importés dans les chapitres précédents. Les nouveaux modules sont les suivants :

- `seaborn` : comme la bibliothèque `matplotlib`, ce module permet de visualiser les données et de réaliser des graphiques intéressants. Nous allons utiliser ce module pour afficher la matrice de corrélation des variables.
- `LabelEncoder` : cette classe permet de transformer certaines de nos variables lorsque nous réalisons l'étape de Feature Engineering.
- `LogisticRegression` : cette classe sert à instancier l'algorithme de régression logistique qui est le sujet de ce chapitre.
- `Warnings` : ce module permet de spécifier avec l'instruction `warnings.filterwarnings('ignore')` notre souhait de ne pas afficher les messages d'alerte associés aux manipulations que nous allons réaliser sur notre jeu de données. Bien sûr, ce n'est pas recommandé dans un vrai projet, ici nous l'avons fait pour des soucis de simplicité.

3.2.2 Lecture des données

Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin de lire votre jeu de données dans un DataFrame :

```
data = pd.read_csv('../Data/train.csv', sep=',')
data = data[['Survived', 'Pclass', 'Sex', 'Age', 'SibSp',
'Parch', 'Fare', 'Embarked']]
data.isnull().sum()
```

Le code précédent charge la table stockée dans le fichier *train.csv* dans le DataFrame `data`, puis sélectionne les variables qui seront exploitées dans notre modèle. Il s'agit de toutes les variables du fichier *train.csv* sauf la première variable `PassengerId`. Ce code se termine par l'affichage du nombre de données nulles par variable avec l'utilisation de la fonction `isnull()`. Cette fonction retourne un DataFrame composé des mêmes variables et du même nombre de lignes que le DataFrame `data`, mais les valeurs des variables seront toutes booléennes. Dans le DataFrame retourné par la fonction `isnull`, une valeur `True` indique que la valeur correspondant dans le DataFrame `data` est nulle et la valeur `False` indique que la valeur correspondant dans le DataFrame `data` est renseignée.

© Editions ENI - All rights reserved

La fonction `sum()` permet tout simplement de calculer le nombre de valeurs `True` par variable, c'est-à-dire le nombre de valeurs nulles par variable.

L'exécution de la cellule précédente affiche ce résultat :

```
Survived      0
Pclass        0
Sex           0
Age         177
SibSp         0
Parch         0
Fare          0
Embarked      2
dtype: int64
```

Ce résultat montre que la variable `Age` contient 177 valeurs manquantes et la variable `Embarked` contient deux valeurs manquantes; les autres variables ne contiennent pas de valeurs manquantes.

3.2.3 Traitement des valeurs manquantes

Lorsqu'une variable présente des valeurs manquantes, nous pouvons choisir de supprimer les observations associées à ces valeurs manquantes ou bien renseigner les valeurs manquantes avec de nouvelles valeurs. Dans notre exemple, nous allons renseigner les deux valeurs manquantes de la variable `Embarked` avec une valeur arbitraire sélectionnée à partir des valeurs déjà existantes et nous allons renseigner les valeurs manquantes de la variable `Age` avec la valeur médiane calculée à partir des valeurs présentes de cette même variable. Pour réaliser ces deux opérations du remplacement des valeurs manquantes, suivez les étapes ci-dessous.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les trois instructions suivantes :

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data['Age'].fillna(data['Age'].median(), inplace=True)
data.isnull().sum()
```

En utilisant la fonction `fillna()`, la première instruction permet de remplacer les valeurs manquantes de la variable `Embarked` avec la valeur de la première observation de cette même variable et la deuxième instruction permet de remplacer les valeurs manquantes de la variable `Age` avec la valeur de la médiane qui est calculée avec la fonction `median()`.

La troisième instruction permet d'afficher le nombre de valeurs nulles pour chacune des variables comme nous l'avons fait précédemment. Le résultat de cette dernière instruction est le suivant :

```
Survived    0
Pclass      0
Sex         0
Age         0
SibSp       0
Parch       0
Fare        0
Embarked    0
dtype: int64
```

Ce nouveau résultat montre qu'il n'y a plus de valeurs manquantes dans notre jeu de données.

3.2.4 Transformation de variables

Les deux variables `Fare` et `Age` présentent des valeurs définies dans des échelles relativement différentes des échelles dans lesquelles sont définies les autres variables. Pour pallier ce problème, nous pouvons, par exemple, procéder à la normalisation de toutes les variables ou redéfinir les deux variables `Age` et `Fare` dans des échelles plus ou moins comparables avec les échelles des autres variables. Dans cet exemple, nous allons choisir la dernière option.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les trois instructions suivantes :

```
data['FareBin'] = pd.qcut(data['Fare'], 4)
data['AgeBin'] = pd.cut(data['Age'].astype(int), 6)
data.head(5)
```

© Editions ENI - All rights reserved

La première instruction permet de créer une nouvelle variable `FareBin` à partir des quantiles de la variable `Fare`. La deuxième instruction permet de créer une nouvelle variable `AgeBin` à partir de la fragmentation de la variable `Age` en six paquets.

La dernière instruction permet d'afficher les cinq premières lignes de notre jeu de données afin de visualiser le résultat avec les nouvelles variables créées. Le résultat est le suivant :

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	FareBin	AgeBin
0	0	3	male	22.0	1	0	7.2500	S	(-0.001, 7.91]	(13.333, 26.667]
1	1	1	female	38.0	1	0	71.2833	C	(31.0, 512.329]	(26.667, 40.0]
2	1	3	female	26.0	0	0	7.9250	S	(7.91, 14.454]	(13.333, 26.667]
3	1	1	female	35.0	1	0	53.1000	S	(31.0, 512.329]	(26.667, 40.0]
4	0	3	male	35.0	0	0	8.0500	S	(7.91, 14.454]	(26.667, 40.0]

Figure 8-5 : fragmentation des variables `Fare` et `Age` en paquets

Maintenant que les deux variables `Fare` et `Age` sont fragmentées en paquets, nous pouvons ajouter de nouvelles variables catégorielles comme suit.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes :

```
label = LabelEncoder()
data['Labeled_AgeBin'] = label.fit_transform(data['AgeBin'])
data['Labeled_FareBin'] = label.fit_transform(data['FareBin'])
data['FamilySize'] = data['Parch']+ data['SibSp']+1
data['IsAlone'] = 1
data['IsAlone'].loc[data['FamilySize'] > 1] = 0
data.head(5)
```

La première instruction crée l'objet `label` qui est un encodeur de variable et qui est utilisé dans les deux instructions suivantes afin d'associer les valeurs des deux variables `AgeBin` et `FareBin` avec leurs catégories correspondantes.

La quatrième instruction permet de créer la nouvelle variable `FamilySize` en calculant, pour une personne donnée, le nombre de personnes appartenant à sa famille et présentes sur le bateau en incluant la personne elle-même dans ce calcul.

La cinquième instruction crée la nouvelle variable `IsAlone` qui est initialisée avec la valeur `1`, puis l'instruction suivante affecte la valeur `0` à toutes les observations associées aux personnes qui n'ont aucun membre de leur famille sur le bateau.

La dernière instruction affiche les cinq premières lignes (voir ci-dessous) de notre jeu de données avec les variables nouvellement créées :

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	FareBin	AgeBin	Labeled_AgeBin	Labeled_FareBin	FamilySize	IsAlone
0	0	3	male	22.0	1	0	7.2500	S	(-0.001, 7.91]	(13.333, 26.667]	1	0	2	0
1	1	1	female	38.0	1	0	71.2833	C	(31.0, 512.329]	(26.667, 40.0]	2	3	2	0
2	1	3	female	26.0	0	0	7.9250	S	(7.91, 14.454]	(13.333, 26.667]	1	1	1	1
3	1	1	female	35.0	1	0	53.1000	S	(31.0, 512.329]	(26.667, 40.0]	2	3	2	0
4	0	3	male	35.0	0	0	8.0500	S	(7.91, 14.454]	(26.667, 40.0]	2	1	1	1

Figure 8-6 : ajout des variables `Labeled_AgeBin`, `Labeled_FareBin`, `FamilySize` et `IsAlone`)

3.2.5 Sélection des variables

Avant d'appliquer un algorithme du Machine Learning quelconque, il est très intéressant de n'utiliser que les variables peu ou pas du tout corrélées entre elles. C'est évidemment intéressant, car les variables fortement corrélées sont souvent porteuses d'informations redondantes. Afin de repérer facilement les variables fortement corrélées, nous allons afficher la matrice des corrélations deux à deux des variables en utilisant `seaborn` comme suit.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes :

```
sb.heatmap(data.corr(),annot=True,cmap='RdYlGn',linewidths=0.2)
fig=plt.gcf()
fig.set_size_inches(9,8)
plt.show()
```

Dans la première instruction, la fonction `heatmap` de `seaborn` permet de tracer un graphique intéressant à partir des résultats de la fonction `corr`.

© Editions ENI - All rights reserved

Les instructions suivantes permettent d'afficher la figure ci-dessous :

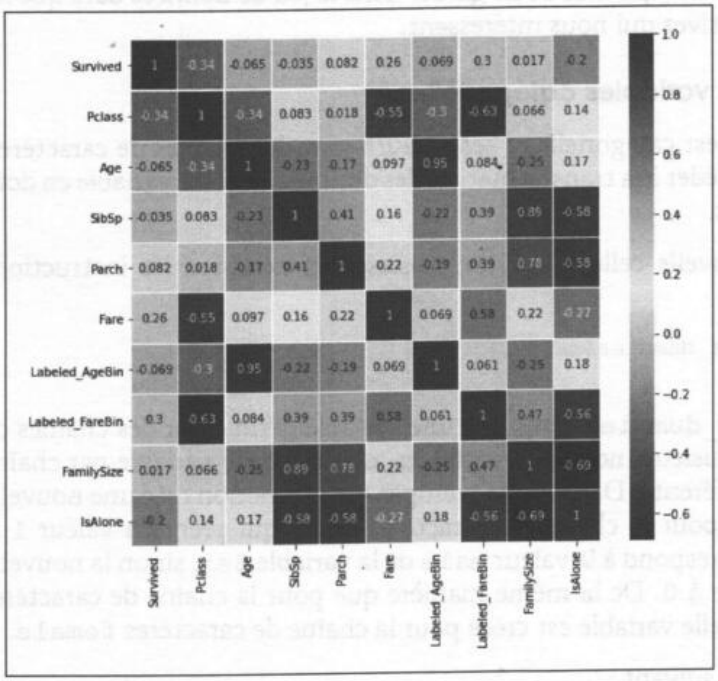


Figure 8-7 : matrice des corrélations des variables numériques

Ce graphique permet de repérer d'un seul coup d'œil les variables fortement corrélées, puisque celles-ci correspondent aux cases en vert foncé.

Pour la suite de notre exemple, nous n'allons garder que les variables `Survived`, `Pclass`, `Sex`, `Embarked`, `Labeled_AgeBin`, `Labeled_FareBin`, `FamilySize` et `IsAlone` grâce aux deux instructions à suivre.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes :

```
y = data['Survived']
data= data[['Pclass', 'Sex', 'Embarked', 'Labeled_AgeBin',
            'Labeled_FareBin' , 'FamilySize', 'IsAlone']]
```

La première instruction permet d'isoler la variable à prédire `Survived` et la deuxième instruction permet de ne garder dans le jeu de données data que les variables explicatives qui nous intéressent.

3.2.6 Traitement des variables catégorielles

La variable `Sex` est catégorielle et ses valeurs sont des chaînes de caractères. Nous allons procéder à la transformation des données de cette variable en données numériques.

▶ Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes :

```
data = pd.get_dummies(data)
data
```

La fonction `get_dummies` remplace une variable définie par des chaînes de caractères par plusieurs nouvelles variables, une nouvelle variable par chaîne de caractères différente. Dans notre exemple, cette fonction crée une nouvelle variable binaire pour la chaîne de caractères `male` qui prend la valeur `1` si l'observation correspond à la valeur `male` de la variable `Sex`, sinon la nouvelle variable est égale à `0`. De la même manière que pour la chaîne de caractères `male`, une nouvelle variable est créée pour la chaîne de caractères `female`.

Le résultat est le suivant :

	Pclass	Labeled_AgeBin	Labeled_FareBin	FamilySize	IsAlone	Sex_female	Sex_male	Embarked_C	Embarked_Q	Embarked_S
0	3	1	0	2	0	0	1	0	0	1
1	1	2	3	2	0	1	0	1	0	0
2	3	1	1	1	1	1	0	0	0	1
3	1	2	3	2	0	1	0	0	0	1
4	3	2	1	1	1	0	1	0	0	1
...	...	...	...	...	...	...	...	...	...	...
886	2	2	1	1	1	0	1	0	0	1
887	1	1	2	1	1	1	0	0	0	1
888	3	2	2	4	0	1	0	0	0	1
889	1	1	2	1	1	0	1	1	0	0
890	3	2	0	1	1	0	1	0	1	0

Figure 8-8 : transformation des données catégorielles en données numériques

© Editions ENI - All rights reserved

Les deux nouvelles variables correspondant aux valeurs `female` et `male` sont respectivement les deux variables `Sex_female` et `Sex_male`.

3.2.7 Entraînement du modèle logistique

L'entraînement d'un modèle de régression logistique se fait exactement de la même manière que l'entraînement du modèle de régression que nous avons vu dans le chapitre précédent.

Pour construire un modèle de régression logistique, appliquez le code ci-dessous.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes :

```
x_train, x_test, y_train, y_test = train_test_split(data, y,
                                                    test_size=0.25)
log_regression_alg = LogisticRegression()
log_regression_alg.fit(x_train, y_train)
test_predictions = log_regression_alg.predict(x_test)
precision, recall, _, _ = score(y_test, test_predictions,
                                pos_label=1, average='binary')
accuracy = round((test_predictions==y_test).sum() /
                 len(test_predictions),3)

print('Precision: {} / Recall: {} / Accuracy:
       {}'.format(round(precision, 3), round(recall, 3),
       accuracy))
```

La première instruction permet de construire deux jeux de données : un pour l'entraînement du modèle et un deuxième pour l'évaluer.

La deuxième instruction permet d'instancier l'objet `log_regression_alg` à partir de la classe `LogisticRegression` qui représente notre algorithme de régression logistique. Les deux instructions suivantes permettent, dans l'ordre, d'appliquer l'algorithme `log_regression_alg` sur les données d'entraînement et de réaliser les prédictions sur le jeu de données de test. Les deux instructions suivantes permettent de calculer les métriques `precision`, `recall` et `accuracy` associées à notre modèle. La dernière instruction permet d'afficher les résultats associés à ces trois métriques de performance :

```
Precision: 0.805 / Recall: 0.75 / Accuracy: 0.83
```

Génial ! Sans trop d'efforts de paramétrage ni d'analyse poussée de notre jeu de données, nous sommes parvenus à prédire correctement 75 % de cas positifs avec une précision de 80,5 % !

Bien sûr, le code de la dernière cellule Jupyter ci-dessus produit des résultats différents d'une exécution à une autre et cela est dû à l'aspect pseudo-aléatoire associé à la séparation des données.

3.2.8 Le seuil de décision

À la section La régression logistique de ce chapitre, nous avons vu que l'objectif de la fonction logistique est de transformer les résultats de la fonction $F(x_{i,1}, x_{i,2}, \ldots, x_{i,m})$, définis dans l'ensemble des réels $\mathbb{R}$, en des résultats assimilés à des probabilités comprises dans l'intervalle `[0,1]`.

La question à se poser est de savoir comment interpréter les valeurs retournées par la fonction logistique. Par exemple, dans le cas d'un modèle de classification binaire, quel est le seuil de la fonction logistique à partir duquel nous allons considérer les cas positifs ?

Nous allons revenir à notre exemple de prédiction des survivants du Titanic afin d'illustrer concrètement les résultats de la fonction logistique.

- À la suite de la dernière cellule Jupyter de l'exemple, saisissez et exécutez les instructions suivantes :

```
prob_test_predictions = log_regression_alg.predict_proba(x_test)
prob_test_predictions
```

La première instruction permet de récupérer et de stocker dans la variable `prob_test_predictions` toutes les probabilités, ou tous les résultats, issues de la fonction logistique de notre modèle. La deuxième instruction permet d'afficher ces probabilités.

© Editions ENI - All rights reserved

Vous obtenez des résultats similaires à ceux de la figure suivante :

```
prob_test_predictions
array([[0.81120636, 0.18879364],
       [0.75883583, 0.24116417],
       [0.90414919, 0.09585081],
       [0.25551182, 0.74448818],
       [0.72364166, 0.27635834],
       [0.91339745, 0.08660255],
       [0.7094652 , 0.2905348 ],
       [0.40700561, 0.59299439],
       [0.11972731, 0.88027269],
       [0.82124258, 0.17875742],
       [0.86260535, 0.13739465],
       [0.16964147, 0.83035853],
       [0.90184185, 0.09815815],
       [0.82124258, 0.17875742],
       [0.42002763, 0.57997237],
       [0.54165893, 0.45834107],
       [0.08030536, 0.91969464],
       [0.08505877, 0.91494123],
       [0.72505992, 0.27494008],
```

Figure 8-9 : les probabilités relatives aux données de test associées aux deux classes du modèle de prédiction des survivants du Titanic

Ces résultats sont constitués d'un DataFrame avec deux colonnes. La première colonne correspond aux probabilités que les observations soient associées à la classe `0`, c'est-à-dire à la classe des personnes qui n'ont pas survécu au naufrage, alors que la deuxième colonne correspond aux probabilités que les observations soient associées à la classe `1`, c'est-à-dire à la classe des survivants du Titanic.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les deux instructions suivantes :

```
prob_test_predictions =
log_regression_alg.predict_proba(x_test)[:,1]
bin_class_1 = (prob_test_predictions >=0.5).astype(int)
```

La première instruction permet de stocker dans la variable `prob_test_predictions` uniquement les probabilités associées à la classe des positifs, c'est-à-dire la classe des survivants. La deuxième instruction crée un tableau avec le nom `bin_class_1` dont chaque case contient la valeur 0 si la probabilité

dans la case du même index du tableau `prob_test_predictions` est inférieure à 0.5, sinon cette case contient la valeur `1`.

Avec les deux dernières cellules Jupyter, nous avons tout simplement construit un tableau binaire, des 0 et des 1, à partir des probabilités des résultats obtenus avec notre modèle sur les données de test. Le code suivant permet de comparer ce nouveau tableau binaire avec le tableau, qui lui aussi est binaire, retourné par la méthode `predict` que nous avons utilisée précédemment.

▶ Dans une nouvelle cellule Jupyter, saisissez et exécutez les deux instructions suivantes :

```
compare = (bin_class_1 == test_predictions).all()
compare
```

Le code précédent donne ce résultat :

```
True
```

Ce résultat montre que la fonction de prise de décision de l'algorithme utilise le seuil 0.5 comme valeur par défaut afin de prendre la décision et classer une observation.

En d'autres termes, le modèle de régression logistique calcule la fonction $F(x_{i,1}, x_{i,2}, \ldots, x_{i,m}) = a_1 x_{i,1} + a_2 x_{i,2} + \cdots + a_m x_{i,m} + a_0$ avec comme objectif d'obtenir avec la fonction $\frac{1}{1+e^{-F(x_{i,1}, x_{i,2}, \ldots, x_{i,m})}}$ des valeurs inférieures à 0,5 pour les observations négatives et des valeurs supérieures à 0,5 pour les observations positives. C'est pour cette raison qu'il n'est pas recommandé de manipuler ce seuil pour construire une fonction de prise de décision qui serait basée sur une valeur de seuil autre que 0,5.

Cependant, dans certains cas, notamment lorsque les données d'entraînement présentent un fort déséquilibre dans les valeurs prises par les variables explicatives ou un fort déséquilibre entre le nombre d'observations positives et négatives, alors le modèle de régression logistique peut aboutir à une fonction de prise de décision, basée sur le seuil 0.5, peu performante. Dans ces cas-là, il peut être intéressant d'utiliser une fonction de prise de décision basée sur un seuil autre que le seuil 0.5.

© Editions ENI - All rights reserved

Le code suivant permet d'utiliser un seuil différent, autre que 0.5, pour la prédiction des survivants du Titanic.

▪ Dans une nouvelle cellule Jupyter, saisissez et exécutez les deux instructions suivantes :

```
bin_class_2 = (prob_test_predictions >=0.3).astype(int)
precision, recall, _, _ = score(y_test, bin_class_2, pos_label=1,
average='binary')
accuracy = round((bin_class_2==y_test).sum() /
len(bin_class_2),3)
print('Precision: {} / Recall: {} / Accuracy:
{}'.format(round(precision, 3), round(recall, 3), accuracy))
```

La première instruction construit un tableau binaire avec la valeur `0` si la probabilité d'une observation est inférieure à 0.3, sinon la valeur `1` si cette probabilité est supérieure ou égale à 0.3. Les trois instructions suivantes sont exactement les mêmes que celles utilisées précédemment afin de calculer et afficher les métriques de performance `precision`, `recall` et `Accuracy`.

Les résultats sont les suivants :

```
Precision: 0.698 / Recall: 0.841 / Accuracy: 0.794
```

Pour terminer cette section sur le seuil de décision, il faut bien noter les deux points suivants :

- Le seuil 0.5 est utilisé pour la répartition de la façon la plus correcte possible des observations des données d'entraînement dans leurs classes respectives. De ce fait, le seuil 0.5 est adapté de façon plus au moins optimale à ces données d'entraînement, mais pas forcément optimale pour les données de test ou les données de production ! D'où l'intérêt dans certains cas d'utiliser un seuil différent du seuil par défaut.
- Le choix d'utiliser un seuil de décision différent de 0.5 doit être fait avec beaucoup de précautions et uniquement dans des cas particuliers où la fonction de décision par défaut donne des résultats médiocres. Également, il existe différentes techniques qui permettent de sélectionner le meilleur seuil. Dans cette section, nous avons montré la méthode la plus simple consistant à tester des valeurs autres que la valeur par défaut.

4. L'algorithme One-vs-All

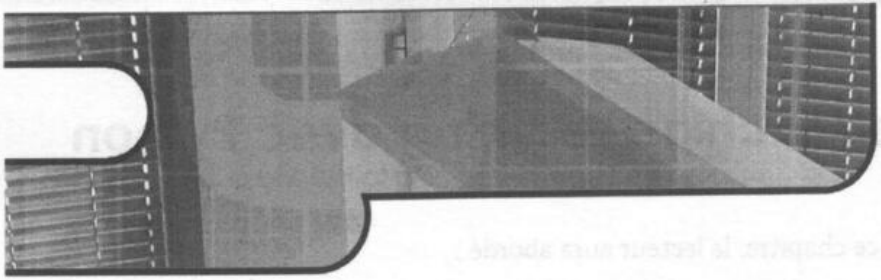
Ce chapitre s'est focalisé sur la régression logistique binomiale, qui traite des modèles dont la variable à prédire est binaire. Cependant, la régression logistique peut être généralisée à des variables prédictives multiclasses, qui peuvent donc prendre un nombre de modalités supérieur à deux. Dans ce dernier cas, l'algorithme One-vs-All est sans doute l'algorithme le plus utilisé.

Supposons que la variable Y à prédire peut prendre ses valeurs dans l'ensemble $\{m_1, m_2, m_3\}$. Pour calculer la probabilité que $Y = \{m_1\}$, cet algorithme va construire deux groupes : le premier groupe $g_1 = \{m_1\}$ et le deuxième groupe $g_2 = \{m_2, m_3\}$. Maintenant, le problème est transformé en un problème binaire où il faudrait prédire si $Y = g_1$ ou $Y = g_2$. On suit le même raisonnement pour calculer la probabilité que $Y = \{m_2\}$ avec $g_1 = \{m_2\}$ et $g_2 = \{m_1, m_3\}$. Enfin, pour calculer la probabilité que $Y = \{m_3\}$, on fixe $g_1 = \{m_3\}$ et $g_2 = \{m_1, m_2\}$. Une fois que ces trois probabilités sont calculées, il suffit de choisir la probabilité la plus élevée afin de prédire la classe d'une observation.

5. Conclusion

La régression logistique est un modèle très répandu dans le monde du Machine Learning. Il trouve son application dans des domaines divers et variés. Ce chapitre a permis de donner l'intuition de ce modèle ainsi que d'aborder quelques concepts justifiant plus au moins sa complexité vis-à-vis des modèles linéaires. Il nous a également permis de voir comment construire un modèle de régression logistique avec Scikit-learn pour prédire les survivants du naufrage du Titanic. Enfin, nous avons expliqué l'algorithme One-vs-All qui permet de traiter les problèmes de classification multiclasse.

© Editions ENI - All rights reserved



Chapitre 4-3
Arbres de décision et Random Forest

1. Objectif du chapitre

Les deux derniers chapitres ont présenté les notions de base des algorithmes de régression linéaire et de régression logistique. Ce chapitre va aborder les arbres de décision et les algorithmes de type Random Forest.

Les arbres de décision permettent de découper un ensemble d'observations en groupes homogènes en se basant sur des règles appliquées sur les variables descriptives.

Les algorithmes Random Forest permettent d'étendre les notions d'arbres de décision afin de construire des modèles prédictifs plus stables.

Parmi les algorithmes les plus connus pour la construction d'arbres de décision, on peut citer CART (*Classification And Regression Tree*) qui a été introduit par Leo Breiman dans la première moitié des années 1980. Les algorithmes de forêts aléatoires ou Random Forest ont quant à eux été introduits au tout début des années 2000.

En plus de leur efficacité, les algorithmes de Random Forest et les arbres de décision sont très faciles à comprendre et très intuitifs.

L'objectif de ce chapitre est d'introduire les concepts de base des arbres de décision et des algorithmes Random Forest. Un exemple d'application d'un algorithme Random Forest sera également développé.

À la fin de ce chapitre, le lecteur aura abordé :

- les notions de base des arbres de décision,
- les notions de base de l'algorithme Random Forest,
- l'application de Random Forest avec Scikit-learn pour prédire si un e-mail donné est un spam ou pas.

1.1 Construction d'un arbre de décision

Les algorithmes Random Forest sont basés sur les arbres de décision. C'est pour cette raison que cette section montre, sur un exemple, le schéma général de construction d'un arbre de décision.

Prenons une base d'apprentissage constituée d'un ensemble d'observations représentées par les carrés et les triangles de la figure 9-1 ci-après. Ces observations sont définies dans un espace à deux dimensions.

Supposons que pour cet exemple les deux variables prédictives sont la variable X, qui représente l'abscisse d'une observation, et la variable Y représentant l'ordonnée d'une observation. L'objectif avec un arbre de décision est de construire un système de prise de décision qui va permettre de catégoriser une observation afin de savoir si c'est un carré ou un triangle à partir de ses deux coordonnées X et Y.

© Editions ENI - All rights reserved

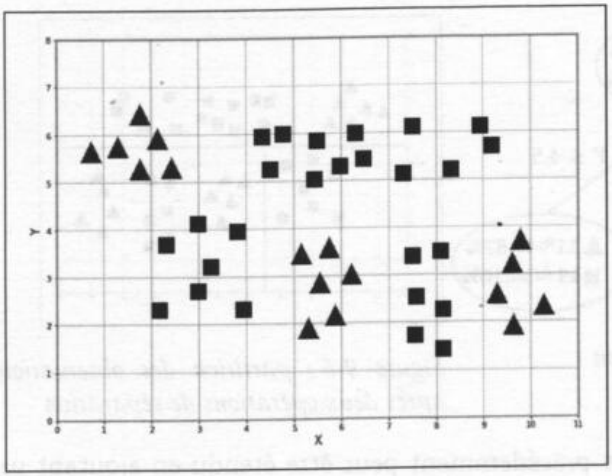


Figure 9-1 : exemple d'un jeu de données défini dans un espace à deux dimensions

Pour construire un arbre de décision et prédire si une nouvelle observation est un carré ou un triangle, on procède comme suit :

- La construction d'un arbre de décision commence par poser le nœud racine. Pour notre exemple, le nœud racine peut être représenté ainsi :

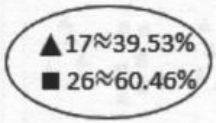


Figure 9-2 : la racine de l'arbre de décision

Le nœud racine contient toutes les observations de la base de données d'apprentissage. Dans ce nœud racine, les triangles représentent approximativement 39.53 % des observations totales et les carrés représentent approximativement 60.46 %.

- À partir du nœud racine, l'ensemble des observations peut être divisé en deux groupes. Par exemple, le premier groupe serait le groupe des observations avec la valeur de la variable $Y>4.5$ et le deuxième avec la valeur de $Y\leq 4.5$. Ces deux groupes vont représenter les deux nœuds enfants du nœud racine comme dans la figure suivante.

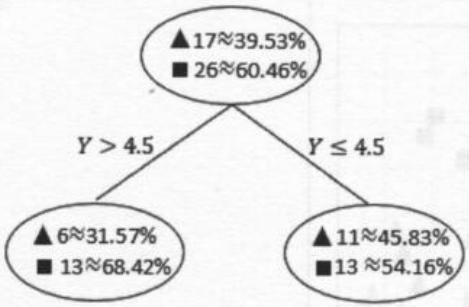


Figure 9-3 : arbre de décision d'une profondeur égale à 1

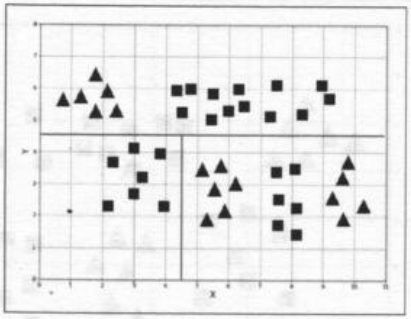


Figure 9-6 : partition des observations après deux opérations de séparation

– L'arbre partiel construit précédemment peut être étendu en ajoutant une règle de séparation des observations du nœud enfant à droite du nœud racine comme suit :

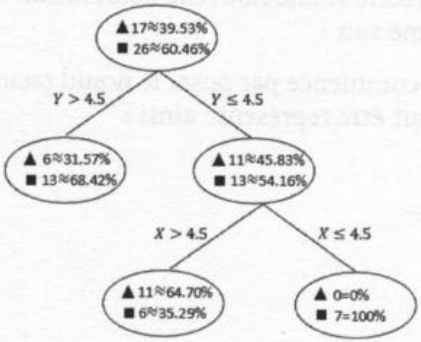


Figure 9-5 : arbre de décision d'une profondeur égale à 2

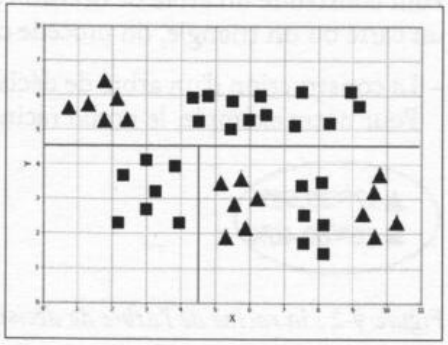


Figure 9-6 : partition des observations après deux opérations de séparation

De telles opérations de séparation des observations au niveau de chaque nœud peuvent être réitérées jusqu'à l'obtention de l'arbre représenté sur la figure 9-7 suivante.

© Editions ENI - All rights reserved

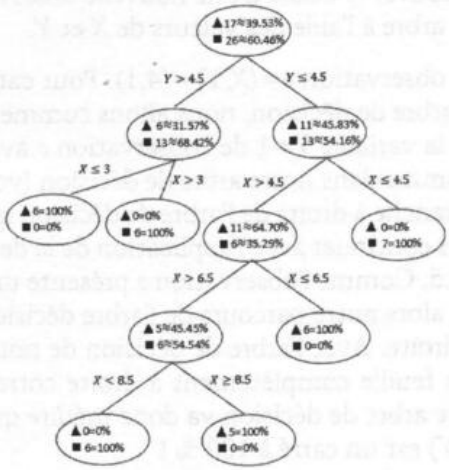


Figure 9-7 : arbre de décision d'une profondeur égale à 4

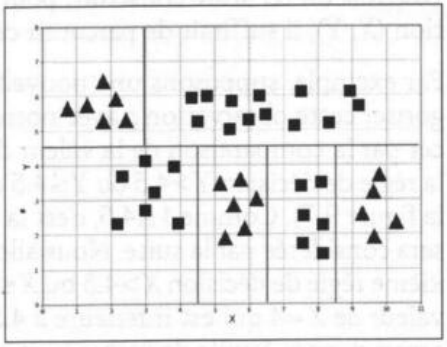


Figure 9-8 : partition des observations après quatre opérations de séparation

Au final, un arbre de décision est constitué des éléments suivants :

1. Une racine qui représente toutes les observations d'apprentissage.
2. Des sommets pendants communément appelés les feuilles et qui sont situés tout en bas des différentes branches de l'arbre de décision.
3. Des niveaux intermédiaires entre la racine et les feuilles. Chaque niveau intermédiaire correspond à la sélection d'une variable et certaines de ses valeurs.

1.2 Prédire la classe d'appartenance avec un arbre de décision

Le dernier arbre construit, celui de la figure 9-7, correspond à un système de décision basé sur l'ensemble des observations initiales. Chaque feuille de cet arbre correspond à une règle de classification.

Une fois un tel arbre construit, pour trouver la classe d'une nouvelle observation (X, Y), il suffirait de parcourir cet arbre à l'aide des valeurs de X et Y.

Par exemple, supposons une nouvelle observation $o=(X,Y)=(4,1)$. Pour catégoriser cette observation o avec notre arbre de décision, nous allons commencer par la comparaison de la valeur de la variable $Y=1$ de l'observation o avec la règle de décision $Y>4.5$ ou $Y\leq 4.5$ comme dans notre arbre de décision (voir la figure 9-7). Comme $1\leq 4.5$, c'est la branche à droite de l'arbre de décision qui sera considérée par la suite. Nous allons continuer avec l'application de la deuxième règle de décision $X>4.5$ ou $X\leq 4.5$. Comme l'observation o présente une valeur de $X=4$ qui est inférieure à 4.5, alors notre parcours de l'arbre décision arrive dans la feuille de l'arbre tout à droite. Avec l'arbre de décision de notre exemple, un parcours qui aboutit à la feuille complètement à droite correspond à une décision 100 % carré. Notre arbre de décision va donc prédire que l'observation avec les coordonnées (X,Y) est un carré à 100 % !

Dans cet exemple, l'arbre de décision est un système de classification qui permet de catégoriser les observations soit dans la classe des carrés, soit dans la classe des triangles. Dans le cas d'un problème de régression, la même approche peut être appliquée pour la construction de l'arbre de décision. Pour prédire le résultat d'une nouvelle observation, il suffirait de parcourir l'arbre de décision, comme pour un problème de classification, et la valeur à prédire correspondrait à la moyenne des résultats des observations groupées dans la feuille en bas de l'arbre.

1.3 Considérations théoriques sur les arbres de décision

Si vous souhaitez implémenter vos propres arbres de décision, c'est-à-dire que vous ne souhaitez pas utiliser les algorithmes des API telles que Scikit-learn, alors il est important de comprendre les concepts théoriques relatifs à la sélection des couples variable/valeur au niveau de chaque nœud et les concepts théoriques relatifs à la taille ou à la profondeur des arbres de décision. Ces deux concepts sont expliqués brièvement dans la section suivante.

© Editions ENI - All rights reserved

1.3.1 Choix de la variable de segmentation

Lorsque nous avons construit l'arbre de décision de l'exemple introductif au début de ce chapitre, le choix des couples variable/valeur à utiliser au niveau des nœuds de l'arbre de décision était arbitraire. Ce n'est cependant pas recommandé, car une telle stratégie de sélection des variables au niveau des nœuds conduit généralement à la construction d'arbres de décision peu performants.

La littérature regorge d'un nombre assez important de méthodes de sélection de la variable à utiliser au niveau des nœuds lors de la construction des arbres de décision. Nous allons expliquer l'une des méthodes les plus utilisées et qui est basée sur la statistique de Khi-2.

Supposons une variable Y à prédire et un ensemble de variables explicatives $X=\{X^1,X^2,...X^i...\}$. Cette méthode, appelée la méthode CHAID (*CHi-squared Automatic Interaction Detection*), propose de construire un tableau de contingence pour tous les couples de variables Y/X^i, c'est-à-dire un tableau de contingence pour Y/X^1, un tableau de contingence pour Y/X^2, etc. Pour une variable X^i, la forme générale du tableau de contingence est la suivante :

Y/X^i	v_1^i	v_2^i	....	v_p^i		v_k^i	$\sum$
Y_1							
Y_2							
Y_q			...	e_{qp}	...		n_q
Y_m							
$\sum$				n_p			n

Figure 9-9 : tableau de contingence de la variable à prédire Y et une variable explicative X^i

Où k est le nombre de valeurs possibles pour la variable X^i et m le nombre de valeurs possibles pour la variable à prédire Y. La valeur e_{qp} correspond au nombre d'observations pour lesquelles la variable $X^i=v_p^i$ et la variable à prédire $Y=Y_q$ et n est le nombre total des observations.

Une fois que le tableau de contingence d'une variable est calculé, la statistique de Khi-2 sera calculée avec la formule suivante :

$$\chi^2 = \sum_{q=1}^{m}\sum_{p=1}^{k}\frac{\left[e_{qp} - \frac{n_p * n_q}{n}\right]^2}{\frac{n_p * n_q}{n}}$$

La statistique χ^2 varie de 0 à $+\infty$ et plus cette statistique est grande, plus une variable a de chances d'être sélectionnée. Cette statistique, comme nous allons le voir à la section suivante, permet d'étudier l'écart à l'indépendance de deux variables, ici les deux variables Y/X^i.

La statistique de Khi-2 ci-dessus présente l'inconvénient de donner un avantage d'être sélectionnées aux variables explicatives ayant un nombre élevé de valeurs possibles. C'est pour cette raison que cette statistique est souvent normalisée avec le nombre de degrés de liberté. Une des méthodes les plus utilisées pour normaliser la statistique de Khi-2 est la normalisation par la statistique de Tschuprow qui se calcule avec la formule suivante :

$$t = \frac{\chi^2}{n\sqrt{(m-1)*(k-1)}}$$

Par définition, le domaine de définition de t de Tschuprow est `[0;1]`.

Parmi toutes les variables, c'est celle qui présente la plus grande valeur de t de Tschuprow qui sera sélectionnée en tant que variable de segmentation.

Il existe un nombre important de travaux scientifiques où plusieurs méthodes ont été proposées pour la recherche de la meilleure variable à choisir pour la segmentation au niveau des nœuds des arbres de décision. Énumérer toutes ces méthodes mérite un ouvrage dédié. Également, la méthode présentée ci-dessus est applicable sur des variables discrètes. Lorsque les variables sont continues, une discrétisation sur ces variables est d'abord opérée avant d'entamer la procédure de sélection de la variable de segmentation.

1.3.2 Profondeur d'un arbre de décision

Lors de la construction d'un arbre de décision, les opérations de sélection de variables au niveau des différents nœuds seront répétées jusqu'à ce qu'un critère d'arrêt soit atteint.

© Editions ENI - All rights reserved

La structure d'un arbre de décision est principalement définie par le choix des variables de segmentation d'une part, et sa taille ou sa profondeur d'autre part.

La profondeur d'un arbre de décision correspond au nombre de niveaux qu'il contient.

À titre indicatif, l'arbre de décision que nous avons construit dans l'exemple illustratif de la figure 9-7 est de taille 5 puisque la branche la plus profonde, qui est la branche à droite, est définie par cinq niveaux. Dans cet exemple, l'arbre de décision construit ne contient que des feuilles pures, c'est-à-dire des feuilles dont les observations sont soit à 100 % des triangles, soit à 100 % des carrés. On dit également que ces feuilles sont cohérentes à 100 %.

Dans la pratique, les feuilles des arbres de décision ne sont pas toujours pures et même loin de là. Le critère d'arrêt de l'expansion d'un arbre de décision au niveau d'un nœud peut être atteint à partir du moment où les observations contenues dans ce nœud présentent un niveau de cohérence raisonnable permettant de prendre une décision sans trop d'ambiguïté. Par exemple, imaginons que dans notre exemple précédent nous ayons un plus grand nombre de triangles et de carrés et qu'à un moment donné de la construction de l'arbre de décision nous ayons atteint un nœud où il y a 10000 triangles et 10 carrés. Dans ce cas, on peut raisonnablement considérer ce nœud comme une feuille et arrêter l'expansion de cette branche de l'arbre de décision en cours de construction. Bien sûr, cette feuille sera associée à une décision qui va classer les futures observations dans la catégorie des triangles.

Un arbre de décision de petite taille va présenter le problème de sous-apprentissage et un arbre de décision de grande taille va présenter le problème de surapprentissage.

Le taux d'erreurs d'un arbre de décision sur les données d'apprentissage va diminuer constamment à mesure que la taille de cet arbre de décision devient de plus en plus grande. D'ailleurs, ce taux d'erreur atteindra la valeur zéro au plus tard lorsque la taille de l'arbre de décision sera égale au nombre d'observations d'apprentissage.

Cependant, en règle générale, le taux d'erreurs sur les données de test va diminuer jusqu'à une certaine taille de l'arbre de décision pour atteindre un plateau, puis à partir d'une taille plus grande de l'arbre de décision, ce taux d'erreurs va commencer à augmenter, comme illustré dans la figure 9-10 ci-après :

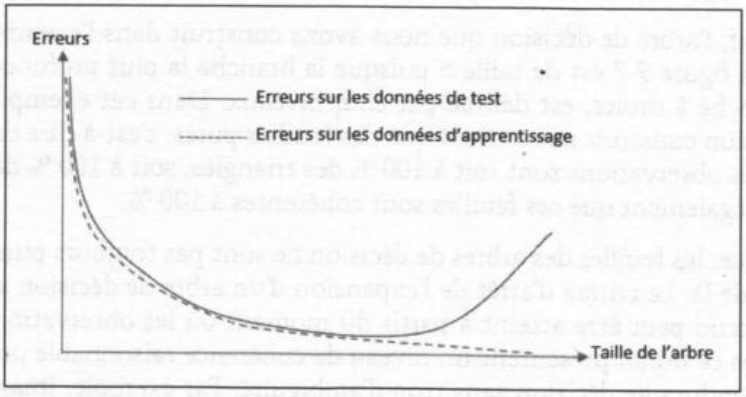


Figure 9-10 : évolution du nombre d'erreurs sur les données d'apprentissage et les données de test en fonction de la profondeur d'un arbre de décision

Tout l'enjeu des méthodes de recherche de la profondeur optimale d'un arbre de décision est de détecter les valeurs correspondant au plateau associé au taux d'erreurs sur les données de test. Bien sûr, l'idéal est de trouver la plus petite valeur sur l'axe de la taille de l'arbre pour laquelle le taux d'erreurs est la première valeur sur le plateau.

Pour trouver la taille d'un arbre de décision correspondant au plateau associé aux erreurs sur les données de test (voir la figure 9-10), il existe principalement deux approches. La première approche consiste à arrêter l'expansion de l'arbre de décision en réalisant des opérations de pré-élagage sur les branches de l'arbre en cours de construction. La deuxième approche consiste à réduire la taille de l'arbre de décision une fois que celui-ci est construit en réalisant des opérations de post-élagage. Les paragraphes suivants introduisent brièvement ces deux approches.

© Editions ENI - All rights reserved

L'approche pré-élagage

Le pré-élagage est une méthode qui permet de limiter l'expansion de l'arbre de décision durant sa construction en appliquant une règle d'arrêt définie au préalable au niveau des nœuds. C'est-à-dire que lorsque l'algorithme de construction de l'arbre de décision arrive au niveau d'un nœud donné, la règle d'arrêt est testée, et si cette règle se révèle applicable, alors le nœud en cours de traitement est considéré comme étant une feuille de l'arbre de décision et donc cet arbre de décision ne peut plus s'étendre au-delà de cette feuille.

L'une des règles d'arrêt les plus utilisées est la règle basée sur le test de Khi-2. Dans la section précédente, nous avons montré comment la statistique de Khi-2 est utilisée pour sélectionner la variable de segmentation au niveau d'un nœud donné. La variable sélectionnée sera utilisée pour l'expansion de l'arbre de décision à partir d'un nœud et créer ainsi de nouvelles branches. Plus précisément, nous avons vu que c'est la variable qui maximise le t de Tschuprow qui sera sélectionnée.

Supposons que c'est la variable X^i qui a donné la valeur maximale de t de Tschuprow au niveau d'un nœud donné. Dans ce cas, la règle d'arrêt basée sur le test de Khi-2 fonctionne comme suit :

1. Mise en place d'un test statistique avec l'hypothèse nulle associée au fait que la variable X^i est indépendante avec la variable à prédire.
2. Comparaison de la statistique de Khi-2 avec une valeur de seuil théorique. Ce seuil correspond au risque d'erreur de première espèce.
3. Si la statistique de Khi-2 calculée est supérieure au seuil théorique, alors la segmentation sera acceptée et donc de nouvelles branches seront créées.

Toute la difficulté de cette méthode réside dans le choix du seuil à utiliser avec la règle d'arrêt. Si la valeur de ce seuil est trop restrictive, alors l'arbre de décision sera sous-dimensionné; et si a contrario sa valeur est trop laxiste, alors l'arbre de décision sera surdimensionné. L'un des inconvénients de cette approche est justement le fait que la règle d'arrêt est basée sur une valeur théorique qui n'est pas en lien direct avec l'objectif ciblé avec l'arbre de décision en cours de construction.

Cette méthode qui consiste à fixer une règle d'arrêt au niveau des nœuds est appelée méthode d'arrêt locale, car la règle d'arrêt est applicable au niveau des nœuds localement sans prendre en compte l'état global de l'arbre en cours de construction. Cette dernière caractéristique est un autre inconvénient majeur de cette méthode.

Malgré ces deux inconvénients cités ci-dessus, cette méthode de pré-élagage basée sur le test de Khi-2 donne des résultats intéressants dans la pratique.

Bien d'autres méthodes de pré-élagage existent. Parmi les méthodes les plus utilisées, hormis la méthode de Khi-2 citée ci-dessus, on trouve les méthodes basées sur les effectifs présents dans les nœuds. Nous pouvons par exemple fixer un nombre d'observations à partir duquel les nœuds ne seront plus segmentés. Généralement, c'est en réalisant plusieurs essais que l'on parvient à trouver de bonnes valeurs des effectifs au niveau des nœuds.

L'approche post-élagage

Avec cette approche, l'arbre de décision est construit en deux étapes principales : la première étape consiste à construire des arbres de décision dont les feuilles sont le plus cohérentes possible, et la deuxième consiste à appliquer une règle de comparaison entre les arbres de tailles différentes.

Parmi les méthodes qui implémentent l'approche post-élagage, on trouve les méthodes basées sur une estimation du taux d'erreurs calculé en réalisant le classement des données d'apprentissage ou le classement des données de test. Le choix d'utiliser les données d'apprentissage ou les données de test est en fonction des méthodes.

Il faut noter que l'approche post-élagage est apparue avec la méthode CART, mentionnée lors de l'introduction de ce chapitre, et depuis, plusieurs versions de cette approche ont vu le jour.

© Editions ENI - All rights reserved

2. Problème de surapprentissage avec un arbre de décision

Nous avons abordé le problème de surapprentissage au premier chapitre où nous avons montré un exemple illustratif relatif au problème de surapprentissage. Avec L'algorithme de construction des arbres de décision, le problème de surapprentissage est souvent rencontré lorsque la taille de ces arbres est importante. En effet, les modèles associés à des arbres de décision complexes présentent un risque plus élevé d'épouser exactement la forme des données d'apprentissage !

Pour mieux comprendre et faire l'association entre un arbre de décision complexe et le problème de surapprentissage, il suffit d'imaginer le cas extrême d'un arbre de décision qui contient un nombre de feuilles égal au nombre d'observations présentes dans le jeu de données d'apprentissage ! Avec un tel arbre de décision, où chaque feuille correspond à une observation, le pouvoir de généralisation est généralement médiocre. En effet, dans ce cas, cet arbre de décision va réaliser des prédictions correctes à 100 % sur les données d'apprentissage, mais ce ne sera certainement pas le cas sur les données de test.

3. Random Forest

Comme mentionné au début de ce chapitre, l'objectif ici est d'introduire les notions de base des arbres de décision et de Random Forest sans trop entrer dans les détails. L'objectif principal est de comprendre le fonctionnement général de ces algorithmes et de les appliquer à un exemple concret avec Scikit-learn.

Si nous observons de plus près les étapes de construction d'un arbre de décision que nous avons utilisées dans l'exemple précédent, nous constatons qu'à chacune de ces étapes, nous aurions pu choisir des règles de décision différentes. En effet, à partir d'un même jeu de données d'apprentissage, nous pouvons construire des arbres de décision qui seraient sensiblement différents les uns des autres. Ces différences peuvent influencer de façon significative les résultats issus de l'utilisation de ces arbres de décision. Ainsi, des arbres de décision construits à partir des mêmes données d'apprentissage peuvent donner des résultats de décision contradictoires !

Sans perdre en généralités, l'algorithme Random Forest propose de construire plusieurs arbres de décision à partir d'un ensemble d'échantillons tirés de façon aléatoire à partir de la base de données d'apprentissage. Ainsi, pour chaque échantillon tiré de la base d'apprentissage, un arbre de décision sera construit. Le nombre d'arbres à construire est un des paramètres les plus importants de l'algorithme Random Forest.

Une fois que ces arbres de décision sont construits, pour prédire la classe d'une nouvelle observation, l'algorithme Random Forest interroge chacun des arbres de décision, et c'est le résultat majoritaire qui l'emportera.

Dans l'exemple de la section précédente, supposons que l'algorithme Random Forest a été utilisé avec un nombre d'arbres à construire égal à 10. Supposons aussi que pour une nouvelle observation (X, Y), huit arbres ont prédit que cette observation est un triangle et deux d'entre eux ont prédit que cette nouvelle observation est un carré. Dans ce cas, la nouvelle observation sera prédite en tant que triangle, car cela correspond au choix de la majorité des arbres de décision.

Comme l'algorithme Random Forest construit plusieurs arbres à partir d'échantillons différents, il permet de combler l'instabilité des prédictions qu'aurait fait un simple arbre de décision.

4. Exemple de Random Forest avec Scikit-learn

Cette section montre comment appliquer un algorithme de Random Forest sur un jeu de données issu du site https://archive.ics.uci.edu/ml/datasets/spambase.

Il s'agit dans cet exemple de construire un modèle pour prédire si un e-mail est un spam ou pas. La base d'apprentissage est constituée d'un jeu de données défini par 4601 observations, et chaque observation correspond à un e-mail défini par 58 variables, voir le fichier spambase.csv.

© Editions ENI - All rights reserved

Ces variables sont définies comme suit :

- Les 48 premières variables sont des variables numériques continues définies dans l'intervalle [0,100]. Chaque variable indique le taux de présence d'un mot. Ce taux est calculé comme suit :

$$100 . \frac{Le\ nombre\ d'occurences d'un\ mot\ dans\ un\ e-mail}{Le\ nombre\ total\ de\ mots\ dans\ l'e-mail}.$$

  Un mot est une suite de caractères alphanumériques limités par un caractère non alphanumérique ou par le caractère de fin de chaîne de caractères.

- Six variables numériques continues définies dans l'intervalle [0,100]. Chaque variable indique le taux de présence d'un caractère. Ce taux est calculé comme suit :

$$100 . \frac{Le\ nombre\ d'occurences d'un\ caractère\ dans\ un\ e-mail}{Le\ nombre\ total\ de\ caractères\ dans\ l'e-mail}$$

- Une variable numérique continue définie dans l'intervalle [1, ...] qui indique la moyenne des longueurs des mots écrits en lettres majuscules.
- Une variable numérique discrète définie dans l'intervalle [1, ...] qui indique la longueur du mot le plus long écrit en lettres majuscules.
- Une variable numérique discrète définie dans l'intervalle [1, ...] qui indique la somme des longueurs des mots écrits en lettres majuscules d'un l'e-mail.
- Une variable nominale qui est égale à la valeur 1 lorsque l'e-mail est un spam et égale à 0 lorsque l'e-mail n'est pas un spam.

Sur les 4601 e-mails de cette base d'apprentissage, 1813 sont des spams. Pour construire un modèle prédictif sur cette base d'apprentissage, veuillez suivre les étapes ci-après.

▶ Créez un nouveau notebook Jupyter, puis saisissez et exécutez les instructions suivantes dans une nouvelle cellule; vous pouvez également utiliser le notebook *Random_Forest-01.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_recall_fscore_support as
score
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv("../Data/spambase.csv", sep=',', header=None)
email_class = data[57]
data.drop([57],axis = 1, inplace=True)
```

Le code de cette première cellule Jupyter commence par importer les modules Python nécessaires à la réalisation de notre exemple. Le module `sklearn.ensemble` va nous permettre d'utiliser la classe `RandomForestClassifier` afin de créer une instance de l'algorithme `RandomForest`. Nous n'allons pas détailler le rôle des autres modules importés, car nous les avons déjà abordés dans les chapitres précédents.

Ce code permet également la lecture des données stockées dans le fichier *spambase.csv*.

L'avant-dernière instruction permet de stocker le caractère à prédire dans la variable `email_class` qui désigne la catégorie d'un e-mail et qui est égale à 1 si un e-mail est un spam, sinon elle est égale à 0. La dernière instruction permet de supprimer le caractère à prédire du DataFrame `data` afin de ne laisser dans ce jeu de données que les variables explicatives.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
x_train, x_test, y_train, y_test = train_test_split(data,
email_class, test_size=0.25)
```

Cette instruction va nous permettre de créer deux jeux de données : un premier jeu de données d'apprentissage formé avec les deux DataFrames `x_train` et `y_train` et un deuxième jeu de donnée de test formé avec les deux DataFrames `x_test` et `y_test`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les deux instructions suivantes :

```
alg_DecisionTree = RandomForestClassifier(n_estimators=1,
                                          max_depth=60, n_jobs=-1)
dt_model = alg_DecisionTree.fit(x_train, y_train)
```

© Editions ENI - All rights reserved

La première instruction crée l'objet `alg_DecisionTree` qui représente une instance de l'algorithme Random Forest. Cet objet a été initialisé avec la valeur `n_estimators=1`. Ce paramètre permet d'indiquer le nombre d'arbres de décision à utiliser et c'est ce qui justifie le suffixe `DecisionTree` dans le nom de la variable associée à cet objet.

Le paramètre `max_depth` permet d'indiquer la profondeur maximale que les arbres de décision sont autorisés à avoir.

Le paramètre `n_jobs` permet d'indiquer le nombre de processeurs à exploiter lors de la construction des arbres de décision. Lorsque ce paramètre est affecté avec la valeur `-1`, cela indique notre souhait d'exploiter tous les processeurs disponibles.

Ce dernier code va construire un seul arbre de décision puisque `n_estimators=1` ; de ce fait, dans cet exemple le paramètre `n_jobs` a peu d'intérêt.

La deuxième instruction permet tout simplement de lancer la procédure de création de l'unique arbre de décision à partir des données `x_train` et `y_train` indiquées comme arguments de la fonction `fit`. Une fois construit, cet arbre de décision sera stocké dans la variable `dt_model`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les deux instructions suivantes :

```
dt_predictions = dt_model.predict(x_test)
precision, recall, fscore, support = score(y_test,
                dt_predictions, pos_label=1, average='binary')
```

La première instruction permet d'utiliser l'arbre de décision `dt_model` construit à l'étape précédente afin de prédire les catégories des e-mails stockés dans le jeu de données de test `x_test`. Les résultats de ces prédictions seront stockés dans la variable `dt_predictions`.

La deuxième instruction permet tout simplement de calculer les valeurs des quatre métriques de performance `precision`, `recall`, `fscore`, `support`. Par la suite, nous allons nous intéresser uniquement aux deux métriques `precision` et `recall` avec la métrique `accuracy` que nous allons calculer.

Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
print('Precision: {} / Recall: {} / Accuracy:
      {}'.format(round(precision, 3), round(recall, 3),
      round((dt_predictions==y_test).sum()/
      len(dt_predictions),3)))
```

Cette dernière instruction permet de calculer et d'afficher les trois métriques de performance `precision`, `recall` et `accuracy`. Dans cet exemple, l'arbre de décision construit donne les résultats suivants :

```
Precision: 0.829 / Recall: 0.868 / Accuracy: 0.879
```

Ces résultats sont relativement intéressants puisque le modèle construit permet de détecter approximativement 86.8 % des spams contenus dans les données de test. En revanche, comme nous l'avions évoqué au premier chapitre lorsque nous avons abordé le concept du cross-validation, une seule exécution d'un algorithme du Machine Learning sur un jeu de données ne permet pas d'avoir un modèle robuste et stable une fois déployé en production. Afin qu'un modèle gagne toute notre confiance, il est plus judicieux d'utiliser la technique de cross-validation. La suite de cet exemple montre comment réaliser la cross-validation avec Scikit-learn.

Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes :

```
from sklearn.model_selection import KFold, cross_val_score

alg_RandomForest = RandomForestClassifier(n_estimators=1,
                   max_depth=60, n_jobs=-1)

k_fold = KFold(n_splits=4)

cross_val_score(alg_RandomForest, data, email_class, cv=k_fold,
                scoring='accuracy', n_jobs=-1)
```

La première instruction permet d'importer la classe `KFold` et la fonction `cross_val_score` qui permettent de réaliser la cross-validation de façon simple et intuitive.

© Editions ENI - All rights reserved

La deuxième instruction permet de créer une instance de l'algorithme Random Forest comme nous l'avons réalisé lors de l'exemple précédent. Notez le fait que nous avons encore affecté la valeur `1` à l'argument `n_estimators`, ce qui signifie que l'algorithme va encore construire un seul arbre de décision par modèle durant la cross-validation.

La troisième instruction permet de créer l'objet `k_fold` de type `KFold` avec l'argument `n_splits=4` qui indique le nombre de paquets à utiliser avec la cross-validation.

La dernière ligne de ce code permet de lancer le processus de la cross-validation avec la fonction `cross_val_score`. Cette fonction reçoit en arguments l'algorithme `alg_RandomForest` qui sera appliqué à chaque étape de la cross-validation ainsi que les données d'apprentissage, qui sont représentées ici par les arguments `data` et `email_class`, l'objet `k_fold` pour indiquer quels sont les paquets de données à utiliser, l'argument `scoring='accuracy'` indique la métrique sur laquelle sera basée la comparaison des modèles construits à chaque étape de la cross-validation, et enfin l'argument `n_jobs=-1` pour autoriser ce programme à exploiter tous les processeurs disponibles.

Ce dernier code s'est terminé avec les résultats suivants :

```
array([0.73848827, 0.84956522, 0.89304348, 0.76956522])
```

Ces quatre résultats correspondent aux quatre valeurs de la métrique de performance `accuracy` associées aux quatre modèles construits avec la cross-validation. La moyenne de l'`accuracy` de ces quatre modèles est de 81,26 %. Voyons maintenant ce que va donner la cross-validation appliquée avec la valeur `60` de l'argument `n_estimators`.

- Revenez dans la cellule Jupyter précédente et modifiez la valeur du paramètre `n_estimators` pour la fixer à la valeur `60`, puis exécutez cette cellule. Vous obtenez des résultats plus ou moins proches des résultats ci-dessous :

```
array([0.81754996, 0.92173913, 0.96434783, 0.81826087])
```

Cette fois, la moyenne de la métrique `accuracy` des quatre modèles construits est égale à 88.04 %.

Nous constatons que le fait d'avoir construit soixante arbres de décision et non pas un seul arbre de décision permet d'améliorer les résultats.

Remarque

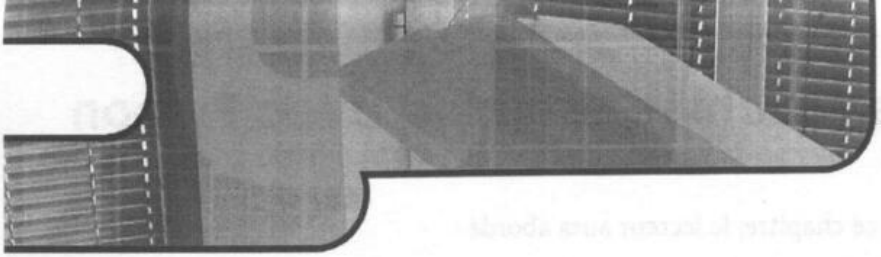
Attention, dans l'absolu, de telles améliorations de résultats ne nous permettent pas de confirmer qu'un algorithme est plus efficace qu'un autre ! Pour avoir plus de confiance dans nos comparaisons, il serait toujours plus judicieux de procéder à des tests statistiques ! En effet, dans notre exemple, le fait que le deuxième code permette d'avoir de meilleurs résultats peut tout simplement être le fruit du hasard lié à la manière dont les données sont réparties entre les quatre paquets utilisés avec la méthode de cross-validation. Néanmoins, l'expérience pratique montre que l'utilisation de plusieurs arbres de décision au lieu d'un seul donne quasiment toujours de meilleurs résultats.

Les deux paramètres les plus importants lors de l'utilisation de l'algorithme Random Forest sont les paramètres `n_estimators` et `max_depth`. Pour trouver des valeurs intéressantes pour ces deux paramètres, il est nécessaire de réaliser plusieurs tests. Avec *Scikit-learn*, en utilisant *GridSearchCV* nous pouvons facilement réaliser plusieurs exécutions d'un même algorithme en faisant varier les valeurs de certains paramètres. Nous allons découvrir *GridSearchCV* au chapitre Support Vector Machine lorsque nous aborderons l'algorithme ainsi nommé. L'utilisation de *GridSearchCV* est similaire, quel que soit l'algorithme utilisé.

5. Conclusion

Ce chapitre a abordé les notions de base des arbres de décision et de l'algorithme Random Forest. Une application de Random Forest a été réalisée avec Scikit-learn pour la détection des e-mails indésirables. Tout au long de ce chapitre, nous avons insisté sur le fait que l'algorithme de construction des arbres de décision présente les deux inconvénients majeurs d'être très sensible aux observations de départ d'une part, et très sensible au choix des variables et des valeurs à utiliser pour la segmentation au niveau des nœuds d'autre part. Nous avons également abordé quelques méthodes de choix des variables de segmentation et nous avons discuté des méthodes de recherche de la profondeur optimale d'un arbre de décision.

© Editions ENI - All rights reserved



Chapitre 4-4
L'algorithme k-means

1. Objectif du chapitre

Les chapitres précédents ont abordé des exemples de deux types d'algorithmes de Machine Learning : les algorithmes de régression et de classification. Ce chapitre porte sur l'algorithme k-means, appelé l'algorithme des k-moyennes en français, qui est un algorithme simple à comprendre et qui fait partie des algorithmes de clustering les plus connus et les plus utilisés.

L'algorithme k-means a été introduit par J. McQueen en 1967. C'est un algorithme non supervisé qui permet de répartir un ensemble de n observations en k clusters. L'objectif après l'application de l'algorithme k-means sur un jeu de données est que chaque cluster contienne des observations homogènes et que deux observations de deux clusters différents soient hétérogènes.

Les domaines d'application de l'algorithme k-means sont nombreux. Par exemple, il est très utilisé pour la segmentation des clients à des fins de marketing, ou encore pour l'isolation des motifs dans les images, car justement les images présentent souvent des régions homogènes, notamment en matière d'intensité lumineuse.

De manière générale, le succès de l'algorithme k-means et de ses versions réside dans sa simplicité et sa capacité à traiter des données de grande taille.

À la fin de ce chapitre, le lecteur aura abordé :

Le fonctionnement de k-means via des illustrations.

- Les étapes principales de l'algorithme k-means classique.
- L'application de l'algorithme k-means avec Scikit-learn.
- L'impact des valeurs extrêmes sur les performances de l'algorithme k-means.
- La recherche de la valeur optimale du paramètre *K* de l'algorithme k-means.
- Les avantages et les inconvénients ainsi que les variantes de l'algorithme k-means.

2. k-means du point de vue géométrique

Comme précisé au début de ce chapitre, l'algorithme k-means est très intuitif et simple à comprendre. Avant d'entrer dans les détails, il faut noter que k-means, comme tous les algorithmes de clustering, ne nécessite pas l'étiquetage des données, car c'est une procédure non supervisée.

De façon informelle, étant donné *n* observations à répartir sur *k*clusters, k-means choisit initialement, de manière aléatoire, *k* observations parmi les *n* observations, comme étant les centres des *k* clusters recherchés. Chacune des *n* observations sera associée au cluster dont le centre est le plus proche parmi les *k* centres choisis initialement. Une fois que toutes les observations sont associées à leurs clusters respectifs, le centre de chaque cluster est recalculé en fonction des observations qu'il contient. Puis, de nouveau, chacune des observations est associée au cluster dont le centre est le plus proche de cette observation par rapport à tous les centres des autres clusters. Ces opérations de recalcul des centres des clusters puis d'association des observations aux clusters les plus proches sont répétées jusqu'à ce qu'un critère d'arrêt soit atteint.

L'algorithme k-means utilise une fonction pour calculer les distances entre les observations et les centres des clusters. Ce calcul des distances peut être basé sur la distance euclidienne, la distance de Manhattan ou toute autre fonction permettant de mesurer la dissimilarité entre les observations.

© Editions ENI - All rights reserved

Pour mieux comprendre cet algorithme de clustering, cette section déroule l'algorithme k-means sur un exemple simple. Soit six observations a, b, c, d, e et f à répartir sur deux clusters C_1 et C_2 ; supposons que la distance utilisée est la distance euclidienne classique. Ces six observations sont définies dans un espace à deux dimensions et leurs coordonnées sont indiquées dans le tableau suivant :

Axes	a	b	c	d	e	f
x	2	4	2	4	10	10
y	4	4	2	2	2	4

Figure 10-1 : un simple jeu de données avec leurs coordonnées en deux dimensions

Remarque

Pour rappel, la distance euclidienne entre deux observations A=(xA,yA) et B=(xB,yB) est calculée grâce à la formule :

$$\sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

Avant de faire un traitement quelconque sur les données, il est toujours intéressant de les visualiser lorsque c'est possible. Dans cet exemple, les données sont définies dans un espace à deux dimensions, donc elles peuvent être facilement visualisées sur deux axes comme dans la figure 10-2 ci-dessous.

Remarque

Même lorsque les données sont définies dans un espace à grande dimension, supérieur à deux ou à trois dimensions, il existe des méthodes qui permettent de les visualiser en deux ou trois dimensions, avec une perte d'informations qu'on espère minime. Ces méthodes sont appelées les méthodes de réduction de domaines. Le chapitre Analyse en composantes principales présente l'analyse du même nom, qui est l'une des méthodes de réduction de domaines les plus connues et qui permet d'avoir une vue en deux dimensions des données définies initialement dans un espace à grande dimension.

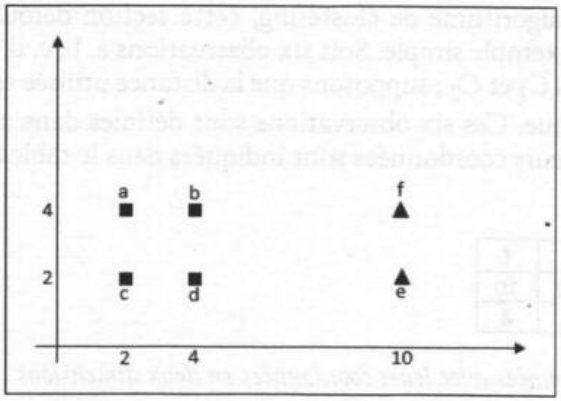


Figure 10-2 : représentation graphique en deux dimensions des données

Ce graphique montre clairement que les observations a, b, c et d, représentées par des carrés, sont très proches entre elles au sens de la distance euclidienne, par rapport aux deux observations e et f. Également, les deux dernières observations, représentées par des triangles, sont très proches entre elles.

Pour cet exemple, en se basant donc sur la distance euclidienne, un algorithme de clustering efficace proposerait certainement de répartir ces six observations dans les deux clusters C_1 et C_2 comme dans la figure 10-3 ci-dessous.

En effet, avec la distance euclidienne, cette répartition est optimale. La section suivante définit de façon plus formelle la notion de solution optimale pour un algorithme k-means et pour un nombre de clusters fixe.

© Editions ENI - All rights reserved

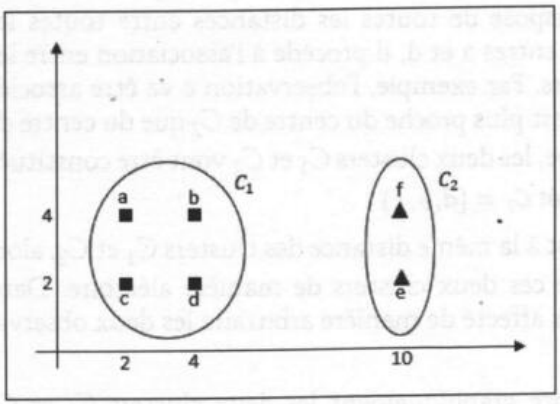


Figure 10-3 : répartition des six points dans les deux classes C_1 et C_2

En suivant les étapes classiques de l'algorithme k-means, le résultat optimal de la figure 10-3 peut être obtenu comme suit :

1. L'algorithme k-means commence initialement par sélectionner de façon aléatoire deux observations parmi les six observations disponibles. Les deux observations ainsi sélectionnées vont être considérées comme les centres des deux clusters recherchés C_1 et C_2. Ici, nous supposons que l'algorithme k-means recherche un nombre de clusters qui est égal à 2.

   Dans cet exemple, supposons que les deux observations a et d sont sélectionnées aléatoirement. Ces deux observations vont être considérées comme les centres respectifs des clusters C_1 et C_2. L'algorithme k-means calcule les distances entre chacune des six observations avec les centres a et d. Les résultats sont reportés dans le tableau ci-dessous :

Les centres des clusters	a	b	c	d	e	f
Centre de C_1=a=(2,4)	0	2	2	2.8284	8.2462	8
Centre de C_2=d=(4,2)	2.8284	2	2	0	6	6.3245

Figure 10-4 : distances entre les observations et les centres de C_1 et C_2

2. Une fois que k-means dispose de toutes les distances entre toutes les observations et les deux centres a et d, il procède à l'association entre les observations et les clusters. Par exemple, l'observation e va être associée au cluster C_2, puisqu'elle est plus proche du centre de C_2 que du centre de C_1. À la suite de cette étape, les deux clusters C_1 et C_2 vont être constitués comme suit $C_1 = \{a, b, c\}$ et $C_2 = \{d, e, f\}$.

   Lorsqu'une observation est à la même distance des clusters C_1 et C_2, alors elle est affectée à l'un de ces deux clusters de manière aléatoire. Dans notre exemple nous avons affecté de manière arbitraire les deux observations b et c au cluster C_1.

   La figure suivante présente graphiquement les deux clusters C_1 et C_2 obtenus à la suite de cette étape :

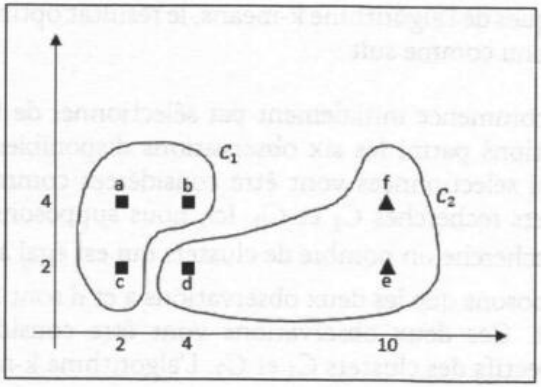


Figure 10-5 : affectation des observations aux clusters C_1 et C_2 après la première itération

3. Lors de la première étape, l'algorithme k-means a choisi de façon aléatoire a et d en tant que centres des deux clusters recherchés. À l'étape 2, les deux clusters C_1 et C_2 ont été concrètement construits.

© Editions ENI - All rights reserved

À cette étape, l'algorithme k-means calcule le centre du cluster C_1 en utilisant les observations a, b et c et calcule le centre du cluster C_2 en utilisant les observations d, e et f.

Ainsi : le centre de $C_1 = \left(\frac{1}{3}(2+2+4), \frac{1}{3}(4+4+2)\right) = (2.66, 3.33)$

le centre de $C_2 = \left(\frac{1}{3}(4+10+10), \frac{1}{3}(2+2+4)\right) = (8, 2.66)$

Les centres des deux clusters C_1 et C_2 sont représentés avec le symbole étoile dans la figure 10-6 ci-dessous :

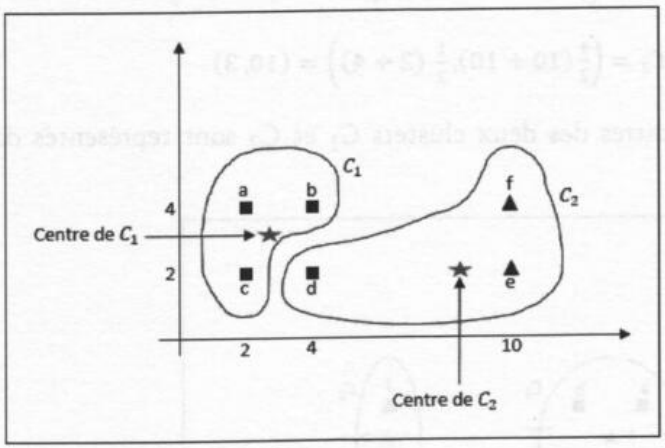


Figure 10-6 : les centres des clusters C_1 et C_2

4. À cette étape, les six observations sont réparties de nouveau sur les deux clusters C_1 et C_2 en se basant sur les nouveaux centres calculés à l'étape précédente. Les distances entre les observations et les centres de C_1 et C_2 sont reportées dans le tableau suivant :

Les centres des clusters	a	b	c	d	e	f
Centre de C_1= (2.66,3.33)	0.9404	1.4981	1.4847	1.8879	7.4595	7.3505
Centre de C_2= (8,2.66)	6.1478	4.2184	6.0361	4.0540	2.1060	2.4074

Figure 10-7 : distances entre les observations et les centres de C_1 et C_2

5. Après la dernière répartition, les deux clusters deviennent :

$C_1 = \{a, b, c, d\}$ et $C_2 = \{e, f\}$

Remarque

Remarquez que les deux clusters C_1 et C_2 construits à cette étape sont différents des deux clusters construits à l'étape 2:

6. Avec les deux nouveaux clusters C_1 et C_2, les centres deviennent :

Le centre de $C_1 = \left(\frac{1}{4}(2+4+2+4), \frac{1}{4}(4+4+2+2)\right) = (3,3)$

Le centre de $C_2 = \left(\frac{1}{2}(10+10), \frac{1}{2}(2+4)\right) = (10,3)$

Les nouveaux centres des deux clusters C_1 et C_2 sont représentés dans la figure suivante :

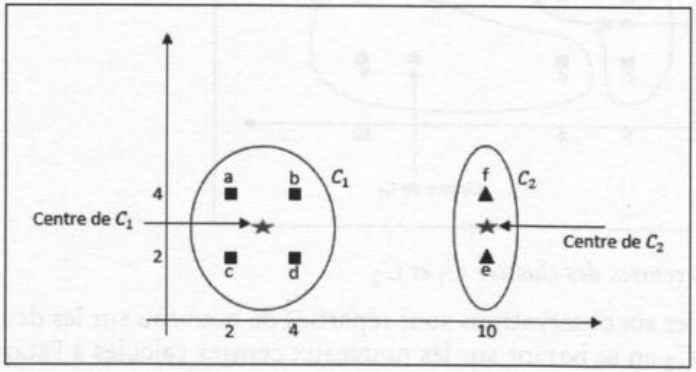


Figure 10-8 : les centres des clusters C_1 et C_2

© Editions ENI - All rights reserved

7. De nouveau, les distances entre les observations et les nouveaux centres des deux clusters C_1 et C_2 sont calculées. Le tableau suivant donne les distances des six observations avec les nouveaux centres calculés à l'étape précédente :

Les centres des clusters	a	b	c	d	e	f
Centre de C_1= (3,3)	1.4142	1.4142	1.4142	1.4142	7.0710	7.0710
Centre de C_2= (10,3)	8.0622	6.0827	8.0622	6.0827	1	1

Figure 10-9 : distances entre les observations et les centres des clusters C_1 et C_2

8. Avec les distances du tableau précédent, les six observations sont réparties sur les deux clusters C_1 et C_2 comme suit $C_1 = \{a, b, c, d\}$ et $C_2 = \{e, f\}$.

Ces derniers clusters C_1 et C_2 sont exactement les mêmes que les clusters obtenus à l'étape 5. Par conséquent, l'algorithme s'arrête puisque, s'il recalcule les centres de ces nouveaux clusters, il trouvera les mêmes résultats que ceux obtenus à l'étape 6.

Les deux clusters C_1 et C_2 obtenus constituent la solution optimale de cet exemple. Il faut noter que si les centres initiaux choisis à l'étape 1 avaient été par exemple $\{b, d\}$ et non pas $\{a, d\}$, les résultats auraient été très différents. Ce dernier constat sera confirmé dans la suite de ce chapitre, lorsque l'algorithme k-means sera appliqué avec Scikit-learn.

3. k-means du point de vue algorithmique

L'exemple de la section précédente montre le mécanisme de base de l'algorithme k-means. Les deux paramètres principaux de cet algorithme sont le nombre de clusters *k* à construire et la fonction utilisée pour le calcul des distances entre les observations. Le schéma général de l'algorithme k-means est présenté ci-dessous :

```
Entrées : un ensemble d'observations X = {X1, X2, ..., Xi, ..., Xn}
          un nombre entier K. correspondant au nombre de
          clusters à construire
          une fonction de distance 𝔽
```

```
Sortie : une partition 𝒫 de l'ensemble X en K clusters
1- Initialisation aléatoire des centres C̄ᵢ, 1 ≤ i ≤ K
2- Répété :
    a) Pour chaque observation Xᵢ, calculer les distances 𝔽(Xᵢ, C̄ⱼ)
       pour tous les j=1..K
    b) Association de chaque observation Xᵢ au cluster
       dont le centre est le plus proche de Xᵢ
    c) Recalculer les centres C̄ᵢ, 1 ≤ i ≤ K
3- Reprendre au point (2) jusqu'à ce que les centres C̄ᵢ, 1 ≤ i ≤ K
deviennent stables
```

L'algorithme k-means prend en entrée un ensemble d'observations $\{X_1, X_2, \ldots, X_i, \ldots, X_n\}$, un paramètre k qui correspond au nombre de clusters que l'algorithme doit construire et une fonction de distance $\mathbb{F}$.

La première étape consiste en l'initialisation aléatoire des centres des k clusters, c'est-à-dire que k-means tire aléatoirement k observations qui seront considérées comme les k centres des k clusters recherchés. Puis il réalise un ensemble d'itérations jusqu'à ce qu'un critère d'arrêt soit atteint.

À chaque itération, en utilisant la fonction de distance choisie, k-means calcule la distance entre chaque observation X_i et les k centres. Pour la première itération, ces k centres sont ceux tirés aléatoirement à la première étape. Chaque X_i sera associé au cluster dont le centre est le plus proche de X_i. À ce stade, k-means aura construit une première partition en k clusters des observations $\{X_1, X_2, \ldots, X_i, \ldots, X_n\}$. L'algorithme k-means s'arrête lorsque les k clusters deviennent stables, c'est-à-dire lorsque l'algorithme obtient des clusters identiques entre deux itérations.

Comme montré dans l'exemple précédent, les résultats de l'algorithme k-means sont fortement influencés par le choix des k centres initiaux. Pour aboutir à une répartition efficace, il faudrait exécuter l'algorithme k-means plusieurs fois et sélectionner la partition la plus performante.

© Editions ENI - All rights reserved

Pour une valeur de k fixe, c'est-à-dire pour un nombre de clusters fixe, la meilleure partition est celle qui minimise la quantité suivante et qui correspond à l'inertie totale associée à l'ensemble des clusters $\sum_{i=1}^{K} \sum_{x \in C_i} d^2(x, \overline{C_i})$

Avec $\overline{C_i}$ le centre du cluster C_i et $d^2(x, \overline{C_i})$ le carré de la distance séparant le point x du centre de son cluster C_i.

Cette quantité correspond aux carrés des distances entre les observations et les centres des clusters auxquels elles appartiennent. À chaque itération de l'algorithme k-means, cette quantité s'approche de son minimum.

À la fin de son exécution, l'algorithme k-means ne garantit pas l'optimalité de la partition trouvée. En effet, la convergence de l'algorithme k-means est fortement influencée par le choix des centres initiaux, et souvent il converge vers des optimums locaux.

4. Application de k-means avec Scikit-learn

L'application de l'algorithme k-means avec Scikit-learn est très simple et intuitive ! Les exemples ci-dessous montrent comment appliquer cet algorithme sur des jeux de données afin de regrouper les données dans des clusters.

Notez dès à présent que dans les exemples de cette section le nombre de clusters à construire, c'est-à-dire la valeur du paramètre k, est connu à l'avance. Dans la suite de ce chapitre, nous allons voir comment tenter de trouver la valeur optimale de ce paramètre k.

Commençons avec un premier exemple dans lequel nous utilisons les mêmes données que dans l'illustration de la figure 10-2 précédente. Pour réaliser cet exemple, suivez les étapes ci-après.

- Créez un nouveau notebook Jupyter, puis saisissez et exécutez les instructions suivantes dans une nouvelle cellule ; vous pouvez également utiliser le notebook *k-means-01.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
%matplotlib inline
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt

data = pd.read_csv('../Data/exemple_simple_kmeans-1.csv', sep=',' )
plt.figure(1, figsize=(10, 10))
plt.scatter(data.X, data.Y,s=60)
plt.ylim(0,5)
```

Ces instructions vont tout simplement permettre de charger les modules nécessaires à notre exemple et de lire les données du fichier *exemple_simple_k-means-1.csv*. Ces données seront affichées dans un graphique. Remarquez l'import du module `KMeans` qui sera utilisé pour instancier et utiliser l'algorithme k-means.

L'exécution de ce code affiche un graphique comme celui de la figure ci-dessous :

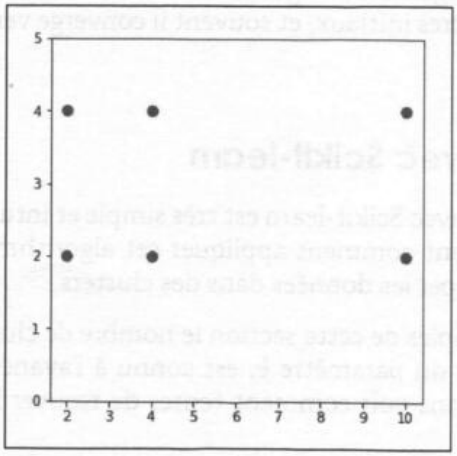


Figure 10-10 : représentation des données en deux dimensions

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les deux instructions suivantes :

```
model = KMeans(n_clusters=2)
model.fit(data)
```

Dans ce dernier code, la variable `model` permet de créer une instance de l'algorithme k-means. Ici, nous l'avons initialisée avec le paramètre `n_clusters=2`. Ce paramètre permet d'indiquer le nombre de clusters à construire et qui est égal à 2 pour notre exemple.

© Editions ENI - All rights reserved

Une fois que nous avons une instance de l'algorithme k-means, pour construire des clusters à partir d'un jeu de données, il suffit de lancer la fonction `fit` en lui indiquant le jeu de données dans lequel les clusters seront recherchés.

À la fin de l'exécution de la fonction `fit`, les clusters sont construits.

Notez bien que, contrairement à tous les algorithmes que nous avons abordés lors des chapitres précédents, cette fois nous n'avons pas eu à séparer nos données pour la création d'un jeu de données d'entraînement et d'un jeu de données de test. En effet, ce n'est pas nécessaire avec k-means vu que, pour rappel, nous réalisons un apprentissage non supervisé !

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model.labels_
```

L'attribut `labels_` contient un tableau dont le nombre d'éléments est égal au nombre de données analysées avec la fonction `fit`.

L'exécution de cette dernière cellule affiche le résultat suivant :

```
array([0, 0, 0, 0, 1, 1])
```

Ce résultat correspond tout simplement aux libellés associés à nos données. Nous avons six points à répartir dans deux clusters. Les quatre premiers points sont donc associés au cluster `0` et les deux derniers points sont associés au cluster `1`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model.cluster_centers_
```

L'attribut `cluster_centers_` stocke les coordonnées des centres des clusters construits.

Cette instruction affiche le résultat suivant :

```
array([[3.,3.],
       [10., 3.]])
```

Ce résultat montre que le centre du premier cluster a comme coordonnées `(3, 3)` et le centre du deuxième cluster est défini par `(10, 3)`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model.inertia_
```

L'attribut `inertia_` correspond à l'inertie totale des deux clusters. L'exécution de cette dernière instruction donne le résultat suivant :

```
10.0
```

Cela veut dire que la somme des carrés des distances qui séparent les différents points des centres de leurs clusters respectifs est égale à 10.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes afin de visualiser nos résultats :

```
colormap = np.array(['green','blue'])
plt.figure( figsize=(5, 5))
plt.scatter(data.X, data.Y,c=colormap[model.labels_],s=2)
plt.scatter(model.cluster_centers_[:,0],
model.cluster_centers_[:,1],c='red',s=20)
for i in range(len(model.labels_)) :
    plt.text(data.X[i], data.Y[i],model.labels_[i], fontsize=10 )
plt.ylim(0,5)
```

Ce code permet d'afficher les données représentées par des points de notre exemple dans un graphique avec l'étiquette ou le chiffre correspondant au cluster associé à chaque point. Également, les centres des deux clusters sont représentés sur ce graphique par des points de couleur rouge.

© Editions ENI - All rights reserved

Ce résultat correspond bien à celui souhaité, comme le montre la figure ci-dessous :

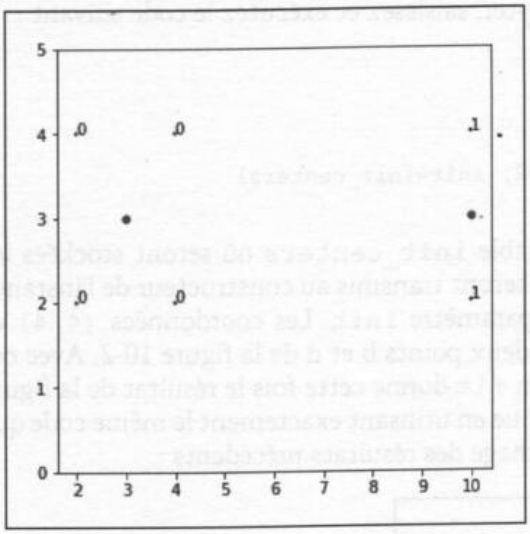


Figure 10-11 : k-means avec k=2 appliqué sur les données exemple_simple_kmeans-1.csv

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin d'appliquer le modèle sur de nouvelles données :

```
print(model.predict([[1,1],[11,1]]))
```

Cette instruction affiche le résultat suivant :

```
[0 1]
```

Dans ces résultats, le cluster prédit par le modèle pour le point défini par les coordonnées `(X=1, Y=1)` est le cluster `0` et le cluster prédit pour le point défini avec les coordonnées `(X=11, Y=1)` est le cluster `1`.

Lorsque k-means a été présenté au début de ce chapitre, nous avons mentionné l'importance du choix des centres des clusters initiaux. Pour illustrer l'impact de cette étape d'initialisation de k-means, nous allons exécuter de nouveau k-means sur les mêmes données que précédemment, mais en lui indiquant de manière explicite les centres des clusters initiaux.

Nous allons choisir les points b et d (voir la figure 10-2) comme étant les centres des deux clusters initiaux.

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
init_centers= np.array([
    [4,4],
    [4,2],
])
model = KMeans(n_clusters=2, init=init_centers)
model.fit(data)
```

Ce code permet de créer la variable `init_centers` où seront stockées les coordonnées de deux points qui seront transmis au constructeur de l'instance de l'algorithme k-means via le paramètre `init`. Les coordonnées `(4,4)` et `(4,2)` correspondent bien aux deux points b et d de la figure 10-2. Avec ces deux centres initiaux, la fonction `fit` donne cette fois le résultat de la figure suivante. Cette figure a été obtenue en utilisant exactement le même code que celui qui nous a servi pour l'affichage des résultats précédents :

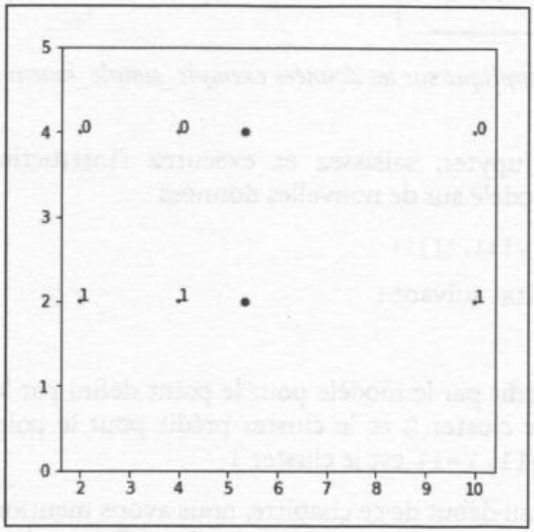


Figure 10-12 : k-means avec k=2 appliqué sur les données exemple_simple_kmeans-1.csv avec les centres initiaux b et d

© Editions ENI - All rights reserved

Cette fois, les observations ont été réparties différemment sur les deux clusters par rapport à l'exécution précédente.

Remarquez que cette nouvelle répartition est de moindre qualité que celle obtenue lors de la première exécution. En effet, dans cette dernière répartition, les trois observations en bas sont regroupées dans le même cluster et les observations en haut sont également regroupées dans un même cluster. Pourtant, au sens de la distance euclidienne, l'observation tout en bas à gauche est beaucoup plus proche de l'observation tout en haut à gauche que de l'observation tout en bas à droite !

Ces derniers résultats montrent concrètement l'importance du tirage initial des centres des clusters.

Lors du premier clustering réalisé, l'attribut était égal à `10`. Voyons quelle est la valeur de l'inertie totale associée au clustering de ce nouveau modèle.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante :

```
model.inertia_
```

Cette fois, le résultat est :

```
69.33333333333334
```

Force est de constater une grande différente entre les deux inerties des clusterings ! Plus les données sont réparties correctement, moindre est la valeur de l'inertie totale.

Voyons maintenant ce que donne la prédiction des clusters des deux points `(X=1,Y=1)` et `(X=11,Y=1)`.

- Dans une nouvelle cellule Jupyter, saisissez et exécutez l'instruction suivante afin de réaliser la prédiction :

```
print(model.predict([[1,1],[11,1]]))
```

Cette instruction affiche le résultat suivant :

```
[1 1]
```

Le point `(X=1,Y=1)` est beaucoup plus proche du point `a=(2,4)`, mais avec ce nouveau modèle il serait placé dans le même cluster que le point `(X=11,Y=1)` ! Ce qui prouve que ce nouveau modèle est de faible qualité.

5. L'algorithme k-means et les valeurs extrêmes

L'algorithme k-means utilise la notion de distance afin de calculer les centres des clusters recherchés. Bien sûr, la nature de la fonction de calcul de distance utilisée a une influence directe sur les résultats de k-means. Cependant, quelle que soit la fonction de distance utilisée, l'algorithme k-means est sensible aux valeurs extrêmes, comme nous allons le montrer dans l'exemple suivant.

- Créez un nouveau notebook, puis saisissez et exécutez le code suivant ; vous pouvez utiliser le notebook *k-means-02.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
data = pd.read_csv('../Data/exemple_simple_kmeans-2.csv', sep=',' )
plt.figure(1, figsize=(10, 10))
plt.scatter(data.X, data.Y,s=60)
plt.ylim(0,6)
plt.xlim(0,15)
```

© Editions ENI - All rights reserved

Ce bout de code permet tout simplement la lecture et l'affichage dans un graphique des données du fichier *exemple_simple_kmeans-2.csv*. Vous obtenez le graphique suivant :

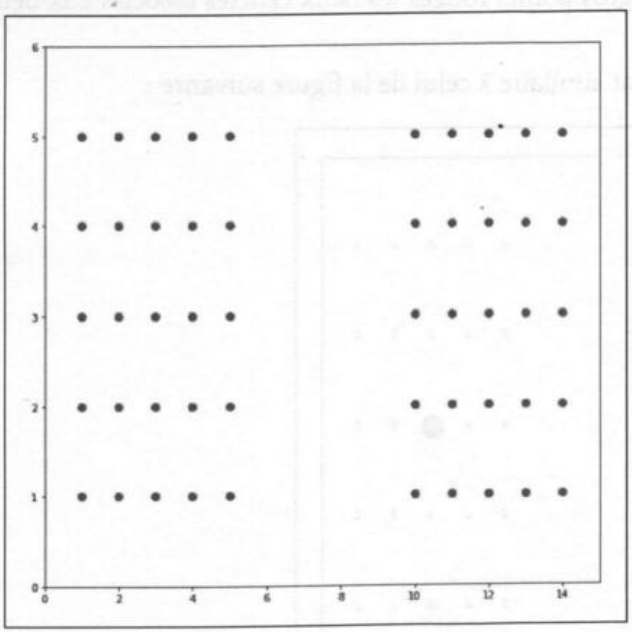


Figure 10-13 : représentation graphique des données définies dans le fichier exemple_simple_kmeans-2.csv

Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
model = KMeans(n_clusters=2)
model.fit(data)

colormap = np.array(['green','blue'])
plt.figure( figsize=(10, 10))
plt.scatter(data.X, data.Y,c=colormap[model.labels_],s=2)
plt.scatter(model.cluster_centers_[:,0],
model.cluster_centers_[:,1],c='red',s=500)
for i in range(len(model.labels_)) :
    plt.text(data.X[i], data.Y[i],model.labels_[i], fontsize=10 )
plt.ylim(0,6)
plt.xlim(0,15)
```

Les deux premières instructions de ce code appliquent l'algorithme k-means pour rechercher deux clusters. Le reste du code permet d'afficher les résultats obtenus en indiquant pour chaque point le numéro de son cluster. Également, ce code affiche avec de gros points rouges les deux centres associés aux deux clusters trouvés.

Vous obtenez un résultat similaire à celui de la figure suivante :

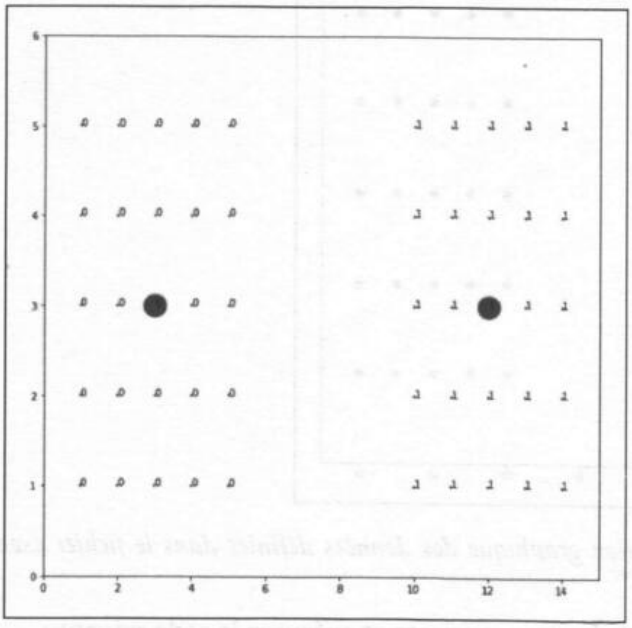


Figure 10-14 : résultat de k-means avec k=2 sur les données du fichier exemple_simple_kmeans-2.csv

Ce résultat correspond au résultat attendu puisqu'en observant le graphique de la figure 10-13, on distingue bien deux clusters.

Dans la suite de cet exemple, nous allons exécuter plus ou moins le même code que celui des deux dernières cellules sur un jeu de données qui est celui utilisé précédemment, auquel nous ajoutons un seul point que nous qualifions de point extrême au vu de ses coordonnées `(x, y)`.

© Editions ENI - All rights reserved

En effet, nous utilisons cette fois les données du fichier *exemple_simple_k-means-2-bis.csv* contenant toutes les données du fichier précédent plus un point avec les coordonnées (X=10, Y=40).

- Dans le même notebook, créez une nouvelle cellule Jupyter puis saisissez et exécutez le code suivant :

```
data = pd.read_csv('../Data/exemple_simple_kmeans-2-bis.csv',
sep=',')
plt.figure(1, figsize=(10, 10))
plt.scatter(data.X, data.Y,s=60)
plt.ylim(0,45)
```

Ce code lit et affiche dans un graphique les points du fichier *exemple_simple_k-means-2-bis.csv*. L'exécution de cette dernière cellule donne le résultat de la figure suivante :

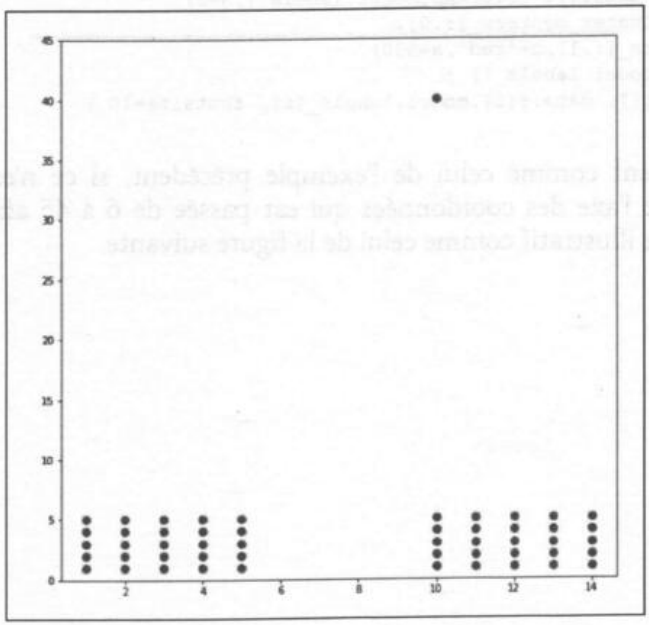


Figure 10-15 : affichage du nuage de points défini dans le fichier exemple_simple_k-means-2-bis.csv

La figure ci-dessus montre clairement que le nouveau point défini avec les coordonnées `(10,40)` est sensiblement loin de tous les autres points et il peut être considéré comme un point extrême. Le reste des points, autres que ce nouveau point, peuvent en toute légitimité former deux clusters différents comme dans l'exemple précédent. On peut se demander comment k-means va se comporter face à de telles situations si on lui demande de créer deux clusters. Va-t-il considérer les deux clusters comme dans l'exemple de la figure 10-14 en ajoutant le nouveau point au cluster à droite? Pour voir concrètement comment k-means réagira, poursuivons avec notre exemple.

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
model = KMeans(n_clusters=2)
model.fit(data)
colormap = np.array(['green','blue'])
plt.figure( figsize=(10, 10))
plt.scatter(data.X, data.Y,c=colormap[model.labels_],s=2)
plt.scatter(model.cluster_centers_[:,0],
model.cluster_centers_[:,1],c='red',s=500)
for i in range(len(model.labels_)) :
    plt.text(data.X[i], data.Y[i],model.labels_[i], fontsize=10 )
plt.ylim(0,45)
```

Ce code est exactement comme celui de l'exemple précédent, si ce n'est l'échelle d'affichage de l'axe des coordonnées qui est passée de 6 à 45 afin d'obtenir un graphique illustratif comme celui de la figure suivante.

© Editions ENI - All rights reserved

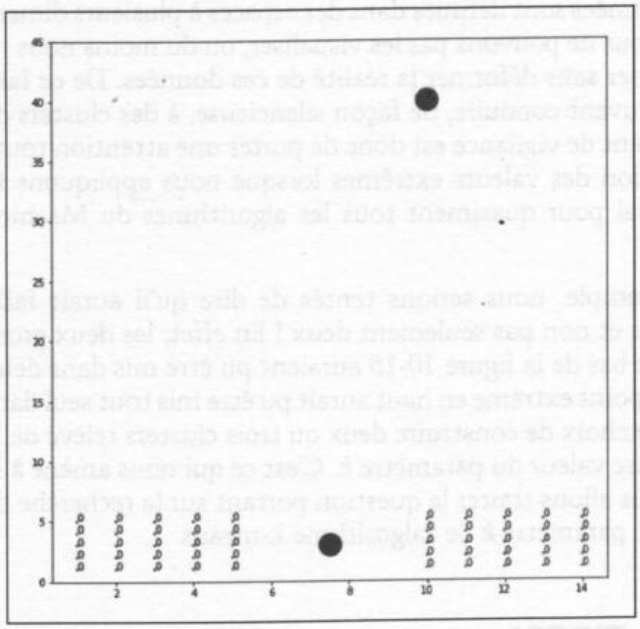


Figure 10-16 : résultat de k-means avec k=2 sur les données du fichier exemple_simple_kmeans-2-bis.csv

Ce résultat montre que le point extrême en haut forme un cluster à lui tout seul ! Et tous les autres points sont positionnés dans un seul et même cluster.

Bien sûr, des clusters comme ces deux derniers ne sont généralement pas souhaités. Il faut noter que dans notre exemple les données sont définies dans un espace à deux dimensions et c'est pour cette raison que nous pouvons facilement les visualiser et reconnaître que les deux clusters trouvés avec k-means dans cet exemple ne sont pas les meilleurs clusters que ne pouvons trouver dans l'absolu. Également, dans notre exemple, nous avons très peu de données pour réaliser des graphiques pédagogiques et montrer l'influence que peuvent avoir les valeurs extrêmes sur le fonctionnement de l'algorithme k-means.

Dans la pratique, les données sont définies dans des espaces à plusieurs dimensions, à tel point que nous ne pouvons pas les visualiser, ou du moins nous ne pouvons pas les visualiser sans déformer la réalité de ces données. De ce fait, les valeurs extrêmes peuvent conduire, de façon silencieuse, à des clusters de moindre qualité. Un point de vigilance est donc de porter une attention toute particulière à la détection des valeurs extrêmes lorsque nous appliquons k-means, et cela reste vrai pour quasiment tous les algorithmes du Machine Learning.

Dans notre dernier exemple, nous serions tentés de dire qu'il aurait fallu rechercher trois clusters et non pas seulement deux ! En effet, les deux groupements de données en bas de la figure 10-15 auraient pu être mis dans deux clusters différents et le point extrême en haut aurait pu être mis tout seul dans un troisième cluster. Le choix de construire deux ou trois clusters relève de la recherche d'une meilleure valeur du paramètre *k*. C'est ce qui nous amène à la section suivante où nous allons traiter la question portant sur la recherche de la valeur optimale de ce paramètre *k* de l'algorithme k-means.

6. Choisir le k de k-means

A priori, lorsque nous sommes face à un grand volume de données sur lequel nous souhaitons appliquer l'algorithme k-means, nous ignorons quel est le nombre de clusters qu'il serait judicieux de construire. On peut imaginer d'exécuter l'algorithme k-means avec plusieurs valeurs du paramètre *k* et de choisir le partitionnement des données le plus pertinent ! Justement, toute la problématique réside dans la définition de ce qu'est un partitionnement pertinent !

Dans la réalité, l'algorithme k-means est appliqué dans l'un des deux contextes suivants.

© Editions ENI - All rights reserved

1. Le premier contexte concerne le cas où nous avons une idée, plus ou moins précise, de ce que nous recherchons dans les données. Par exemple, une entreprise de vente de produits électroménagers, tels que les machines à laver ou les lave-vaisselles, utilise k-means afin de réaliser une segmentation de ses clients dans le but de détecter les profils éventuellement intéressés par certaines options de ses appareils. Ici, k-means peut être orienté afin que le clustering soit réalisé autour des options des produits proposés. Pour cela, il suffit de donner plus de poids aux variables décrivant les options de ces produits afin d'indiquer à k-means que les clients qui achètent plus ou moins les mêmes options soient considérés comme faisant partie des mêmes clusters. Dans cet exemple, une fois que k-means est exécuté plusieurs fois, avec à chaque fois une valeur différente du paramètre *k*, nous pouvons analyser les clusters construits en nous appuyant sur l'idée que les clients d'un même cluster auraient tendance à acheter des produits avec les mêmes options. Si, par exemple, avec une valeur de *k* égale à 3 les clusters retournés par k-means contiennent des profils très divergents par rapport aux options de leurs produits achetés, alors nous pouvons considérer que la valeur 3 pour le paramètre *k* n'est pas suffisante pour construire des clusters intéressants ; il serait donc judicieux d'essayer avec des valeurs supérieures à 3 pour ce paramètre *k*. Dans ce cas, notre analyse pour trouver des valeurs intéressantes du paramètre *k* est basée sur un point de vue purement métier. Bien sûr, cela ne nous garantit en rien que l'augmentation de la valeur du paramètre *k* aboutira à des clusters de meilleure qualité. D'ailleurs, il n'est pas rare que k-means soit utilisé sans succès lorsque nous avons un point de vue métier sur ce que nous recherchons, mais cela a le mérite de nous donner le pouvoir de décider si le clustering réalisé par k-means est intéressant ou pas par rapport à l'objectif.

2. Le deuxième contexte concerne le cas où nous n'avons aucune idée de ce que nous recherchons. Dans ce cas, nous avons un grand volume de données à partir duquel nous allons tenter d'extraire de nouvelles connaissances avec k-means. En pratique, k-means est lancé plusieurs fois, avec des valeurs différentes du paramètre *k* pour chaque exécution, puis nous analysons les clusters construits afin de voir s'il n'y aurait pas une information intéressante à exploiter. Ici nous supposons que pour chaque valeur du paramètre *k*, k-means a trouvé un ou plusieurs facteurs qui l'ont poussé à regrouper les données dans les clusters construits. Donc, pour chaque cluster, nous essayons de comprendre pourquoi k-means a regroupé les données ensemble dans ce cluster et tentons également de savoir pourquoi certaines données sont séparées dans des clusters différents. Cela peut mettre en lumière des informations sur nos données que nous n'avions pas forcément avant la construction de ces clusters !

   Dans ce deuxième contexte d'utilisation de l'algorithme k-means, nous n'avons rien pour nous guider dans notre choix des valeurs du paramètre *k*. Cependant, certaines méthodes peuvent nous aider et nous orienter vers des valeurs intéressantes du paramètre *k*. Justement, les deux prochaines sous-sections expliquent deux de ces méthodes les plus utilisées.

Quel que soit le contexte dans lequel k-means est appliqué, les méthodes d'aide au choix des valeurs du paramètre *k* seront applicables et intéressantes.

Remarque

Il faut noter dès à présent que les deux méthodes présentées ci-dessous sont purement mathématiques et leur utilisation est à conjuguer avec notre analyse qui, elle, n'a de sens que si elle est orientée métier.

© Editions ENI - All rights reserved

6.1 Déterminer k avec la méthode Elbow

La méthode Elbow (méthode du coude en français) est parmi les méthodes les plus couramment utilisées afin d'indiquer la qualité d'un clustering. Elle est simple et intuitive, car basée uniquement sur le calcul de l'inertie totale d'un clustering complet.

Pour trouver le nombre optimal de clusters à construire avec l'algorithme k-means, la méthode Elbow propose d'exécuter plusieurs fois k-means en variant le paramètre *k* dans un intervalle de valeurs, par exemple `k=[1,2,...10]`, puis de calculer l'inertie totale de chaque clustering et afficher la courbe associée aux variations de cette inertie par rapport aux variations de la valeur du paramètre *k*. Avec la courbe ainsi construite, nous espérons trouver la valeur optimale du paramètre *k*.

Remarque

Pour rappel, la formule de calcul de l'inertie d'une partition est donnée précédemment dans ce chapitre à la section k-means du point de vue algorithmique.

La méthode Elbow en pratique

Le code suivant donne un exemple d'utilisation de la méthode Elbow pour déterminer le nombre de clusters à considérer avec k-means.

- Créez un nouveau notebook, puis saisissez et exécutez le code suivant ; vous pouvez utiliser le notebook *k-means-03.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
%matplotlib inline
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
data=pd.read_csv('../Data/exemple_simple_kmeans-3.csv', sep=',' )
plt.figure(1, figsize=(10, 10))
plt.scatter(data.X, data.Y,s=60)
```

Ce code permet d'importer les modules nécessaires à notre exemple et d'afficher dans un graphique les données du fichier *exemple_simple_kmeans-3.csv* (voir la figure suivante). Ces données sont définies par deux variables `X` et `Y`.

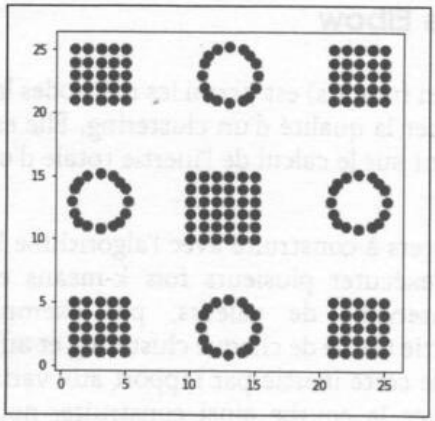


Figure 10-17 : représentation graphique des données du fichier exemple_simple_k-means-3.csv

Cette figure montre clairement que ces données sont organisées en neuf groupes. Pour ce jeu de données, nous pouvons affirmer sans hésiter qu'un clustering correct serait donc organisé en neuf clusters !

Bien sûr, ici comme dans l'exemple précédent, insistons sur le fait que les données que nous avons utilisées sont définies dans un espace à deux dimensions, nous permettant de les visualiser facilement et de savoir qu'un clustering avec neuf clusters serait le plus intéressant. Cependant, dans la réalité, la visualisation des données dans un graphique à deux dimensions n'est pas aussi facile, voire impossible. Ici, nous avons choisi d'utiliser des données simples afin d'illustrer le fonctionnement de l'algorithme k-means.

Pour jouer le jeu, supposons que nous n'avons pas vu le graphique 10-17 ci-dessus ; nous allons compter uniquement sur la méthode Elbow pour nous indiquer la bonne valeur du paramètre *k* ! Pour cela, poursuivons avec notre exemple.

© Editions ENI - All rights reserved

Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
fig, axs = plt.subplots(4, 3)
inertie_lst = []
k=1
for row in range(4):
   for col in range(3):
       print("k= ",k)
       model = KMeans(n_clusters=k)
       model.fit(data)
       inertie_lst.append(model.inertia_)
       centers = model.cluster_centers_
       axs[row,col].set_title(f"k = {str(k)}, inertie =
                              {model.inertia_}")
       axs[row,col].scatter(centers[:,0],
                            centers[:,1],c='red',s=100)
       axs[row,col].scatter(data.X, data.Y,c='#000000',s=1)
       for i in range(len(model.labels_)) :
           axs[row,col].text(data.X[i], data.Y[i],
                             model.labels_[i])
       k+=1

fig.set_figheight(20)
fig.set_figwidth(15)
```

Ce dernier code permet d'exécuter douze fois l'algorithme k-means en variant à chaque exécution la valeur du paramètre *k* en commençant par la valeur `k=1` jusqu'à la valeur `k=12`.

La première instruction permet de créer une grille de douze graphiques, quatre lignes et trois colonnes, où seront illustrés les résultats de chacune des douze exécutions de k-means. La liste `inertie_lst` permet de stocker le contenu de l'attribut `inertia_` associé à chaque modèle construit après chaque exécution. Les deux boucles avec les deux variables `row` et `col` permettent de placer les résultats de chaque exécution dans la case correspondante de la grille des graphiques. La boucle avec la variable `i` permet de mentionner sur chaque graphique le numéro de cluster de chaque point. Également, ce code permet d'afficher dans chaque graphique de chaque résultat les centres des clusters avec un grand point rouge. Cela va nous permettre de situer le centre de chaque groupement de données.

Les deux dernières instructions permettent tout simplement d'ajuster la taille des graphiques. L'exécution de ce code donne les résultats illustrés dans les quatre figures suivantes :

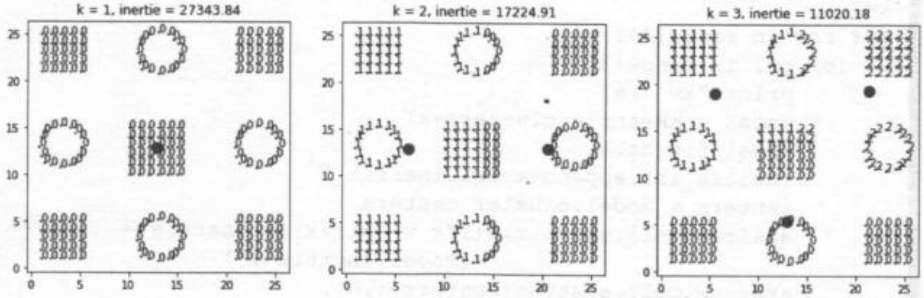


Figure 10-18 : clustering avec k-means et avec k=1, k=2 et k=3 sur les données exemple_simple_kmeans-3.csv

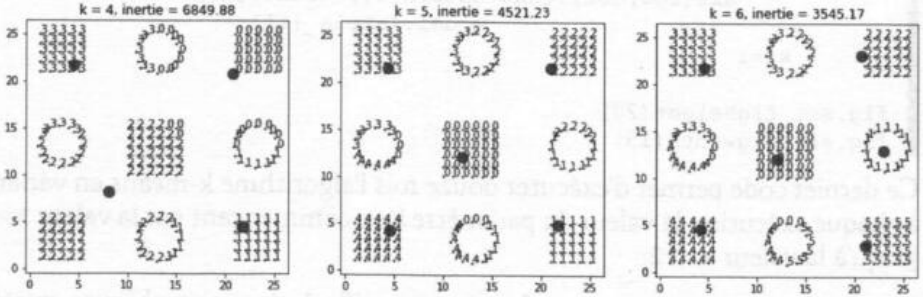


Figure 10-19 : clustering avec k-means et avec k=4, k=5 et k=6 sur les données exemple_simple_kmeans-3.csv

© Editions ENI - All rights reserved

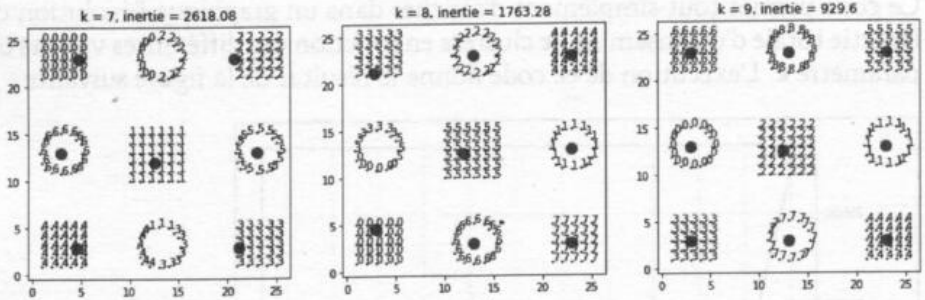


Figure 10-20 : clustering avec k-means et avec k=7, k=8 et k=9 sur les données exemple_simple_kmeans-3.csv

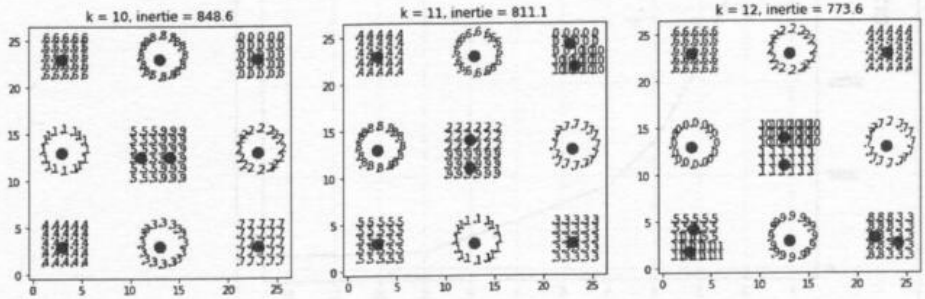


figure 10-21 : clustering avec k-means et avec k=10, k=11 et k=12 sur les données exemple_simple_kmeans-3.csv

Ces résultats montrent que plus la valeur du paramètre *k* augmente, c'est-à-dire plus il y a de clusters, plus la valeur de l'inertie totale est faible. Justement, grâce à la liste `inertie_lst` qui contient les inerties des douze exécutions de k-means, nous allons pouvoir tracer la courbe associée à la méthode Elbow.

▶Dans une nouvelle cellule Jupyter, saisissez et exécutez le code suivant :

```
fig, ax = plt.subplots(1)
ax.set_xticks(np.arange(15))
ax.plot(range(1,13), inertie_lst)
ax.set_xlabel("k", fontsize=20)
ax.set_ylabel("Inertie", fontsize=20)
fig.set_figheight(8)
fig.set_figwidth(8)
plt.grid()
```

Ce code permet tout simplement d'afficher dans un graphique l'évolution de l'inertie totale d'un ensemble de clusters en fonction des différentes valeurs du paramètre *k*. L'exécution de ce code donne le résultat de la figure suivante :

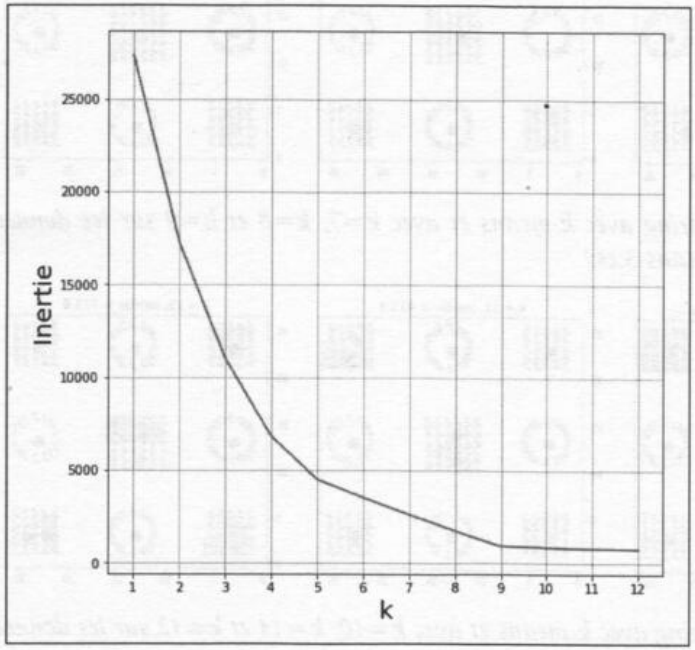


Figure 10-22 : courbe d'évolution de l'inertie totale en fonction de la valeur de k avec l'application de k-means sur les données exemple_simple_kmeans-3.csv

Sur la courbe de la dernière figure, on voit que le chiffre 9 correspond à une valeur particulière. En effet, à partir de la valeur de k=9, la valeur de l'inertie totale est relativement stable. On peut clairement observer sur cette figure que la valeur 9 correspond à la dernière valeur qui a permis une diminution drastique de l'inertie.

En analysant cette courbe de manière plus approfondie, on remarque que l'inertie diminue au fur et à mesure que la valeur du paramètre k devient plus grande. En revanche, on constate que cette diminution de l'inertie est brutale avant la valeur 9 du paramètre *k* ; puis cette diminution devient plus lente après la valeur 9.

© Editions ENI - All rights reserved

Cette courbe peut être vue comme un bras en flexion dont l'épaule, qui n'est pas représentée dans le graphique, se situerait tout en haut à gauche, et la main, qui n'est pas non plus représentée dans le graphique, serait située tout en bas à droite. La valeur 9 serait le coude de ce bras imaginaire, d'où le nom de cette méthode !

Dans cet exemple, on voit que la valeur 9 correspond au coude du bras associé à cette courbe. Nous pouvons donc supposer que la valeur 9 du paramètre k est intéressante et que nos données seraient bel et bien organisées en neuf clusters. C'est ainsi que la méthode Elbow nous aide à déterminer le bon nombre de clusters.

Dans la suite de ce chapitre, nous allons voir que la méthode Elbow n'est pas toujours exploitable, car la courbe de l'évolution de l'inertie en fonction des valeurs du paramètre k n'est pas toujours facile à lire et à interpréter comme dans notre exemple. C'est pour cette raison que l'utilisation de la méthode Elbow est souvent combinée avec l'utilisation d'autres indicateurs tels que le coefficient de silhouette que nous allons découvrir ci-après.

6.2 Déterminer k avec le coefficient de silhouette

La méthode du coefficient de silhouette repose elle aussi sur l'exécution de k-means avec plusieurs valeurs du paramètre k, puis de réaliser des calculs sur les clusters construits à chaque exécution afin de déterminer le coefficient de silhouette de chaque partitionnement. Une fois calculé, ce coefficient de silhouette sera utilisé pour mesurer la qualité d'une partition.

Supposons k clusters associés à une exécution de k-means. Pour calculer le coefficient de silhouette sur l'ensemble de cette partition, il est nécessaire de calculer d'abord le coefficient de silhouette de chaque point x_i de chaque cluster.

Le coefficient de silhouette x_i d'un point x_i se calcule comme suit :

$$-1 \leq s(x_i) = \frac{b(x_i) - a(x_i)}{\max(a(x_i), b(x_i))} \leq 1 \quad (1)$$

Avec $a(x_i)$ et $b(x_i)$ définis comme suit :

- $a(x_i)$ est un coefficient de cohésion du point x_i avec les autres points de son propre cluster et qui se calcule comme suit :

$$a(x_i) = \frac{1}{|C^{x_i}| - 1} \sum_{j \in C^{x_i}, j \neq i} d(x_i, x_j)$$

Où C^{x_i} est le cluster d'appartenance du point x_i et donc $|C^{x_i}|$ représente le nombre d'éléments, ou le nombre de points, dans ce cluster.

Le facteur $a(x_i)$ correspond à la distance moyenne qui sépare le point x_i de tous les autres points de son cluster d'appartenance. De ce fait, $a(x_i)$ permet de comparer les distances intracluster. En règle générale, un clustering efficace est un clustering qui présente des facteurs $a(x_i)$ avec des valeurs faibles.

- $b(x_i)$ est le coefficient de séparation ou de divergence d'un point x_i avec le cluster voisin de son cluster d'appartenance. Il se calcule comme suit :

$$b(x_i) = min_{C \neq C^{x_i}} \frac{1}{|C|} \sum_{x_j \in C} d(x_i, x_j)$$

Pour calculer cette formule pour un point x_i, il est nécessaire de calculer les distances qui séparent ce point x_i avec tous les autres points de tous les clusters autres que le cluster d'appartenance de x_i. Ainsi, pour un cluster C autre que C^{x_i}, nous pouvons calculer la distance moyenne qui sépare le point x_i et les points de ce cluster C. C'est le cluster qui donne la plus petite valeur de cette distance moyenne qui sera considéré comme étant le cluster voisin du cluster C^{x_i}.

Le facteur $b(x_i)$ est aussi appelé la distance moyenne inter-clusters puisqu'il s'intéresse aux distances entre un point et les points des autres clusters.

© Editions ENI - All rights reserved

Dans la formule (1) ci-dessus, si la valeur de $s(x_i)$ est proche de 1, alors le point x_i est bien classé, et si la valeur de $s(x_i)$ est proche de -1, alors le point x_i est fort probablement mal classé. En revanche, si $s(x_i)$ est proche de 0, alors le point x_i peut être classé dans les deux clusters, celui où il est classé ou bien le cluster voisin le plus proche.

Avec les différents coefficients de silhouette $s(x_i)$, nous pouvons maintenant calculer le coefficient de silhouette moyen de toute une partition comme suit :

$$S = \frac{1}{k}\sum_{j=1}^{k}\frac{1}{|C^j|}\sum_{i\in C^j} s(x_i)$$

Le coefficient de silhouette S correspond à la moyenne des moyennes des coefficients $s(x_i)$. C'est-à-dire qu'une fois que la moyenne des $s(x_i)$ est calculée au sein de chaque cluster, nous obtiendrons k moyennes, S est la moyenne de toutes ces k moyennes.

La méthode du coefficient de silhouette en pratique

À travers un exemple, nous allons montrer comment calculer le coefficient de silhouette pour une partition et comment cette mesure peut nous aider dans notre prise de décision quant à la sélection de la valeur du paramètre k de l'algorithme k-means. Pour la réalisation de ce nouvel exemple de code, réutilisons le même jeu de données que dans l'exemple précédent.

Remarque

Le code de cet exemple est quasiment le même que le code de l'exemple précédent. De ce fait, les explications ci-dessous sont concentrées uniquement sur les différences entre ces deux codes.

- Créez un nouveau notebook, puis saisissez et exécutez les mêmes instructions d'import de modules que dans l'exemple précédent en ajoutant l'instruction d'import ci-dessous ; vous pouvez utiliser le notebook *k-means-04.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
from sklearn.metrics import silhouette_score
```

La fonction `silhouette_score` nous permet de calculer le coefficient de silhouette associé à une partition de données.

- Dans une nouvelle cellule Jupyter, utilisez le même code que dans l'exemple précédent afin de lire les données du fichier *exemple_simple_kmeans-3.csv* et d'exécuter l'algorithme k-means avec des valeurs du paramètre `k=[2,3,4,...,13]` ainsi que pour calculer les coefficients de silhouette associés à chaque exécution comme dans le code suivant :

```
data = pd.read_csv('../Data/exemple_simple_kmeans-3.csv',
                   sep=',')
plt.figure(1, figsize=(5, 5))
plt.scatter(data.X, data.Y,s=60)

fig, axs = plt.subplots(4, 3)
inertie_lst = []
silhouette_lst = []

k=2 #Ici, la première valeur du paramètre k est égale à 2.

for row in range(4):
    for col in range(3):
        print("k= ",k)
        model = KMeans(n_clusters=k)
        model.fit(data)
        inertie_lst.append(model.inertia_)

        #Calcul du coefficient de silhouette
        silhouette_lst.append(silhouette_score(data,
                                               model.labels_))

        centers = model.cluster_centers_
        axs[row,col].set_title(f"k = {str(k)}, inertie =
                               {round(model.inertia_,2)}")
        axs[row,col].scatter(centers[:,0],
                             centers[:,1],c='red',s=100)
        axs[row,col].scatter(data.X, data.Y,c='#000000',s=1)
        for i in range(len(model.labels_)) :
            axs[row,col].text(data.X[i], data.Y[i],
                                         model.labels_[i])
        k+=1

fig.set_figheight(20)
fig.set_figwidth(15)
```

© Editions ENI - All rights reserved

Les deux seules différences entre ce nouveau code et celui de l'exemple précédent sont les suivantes :

- Le paramètre *k* varie entre la valeur 2 et la valeur 13. La première valeur est égale à 2 car le coefficient de silhouette n'a d'intérêt que si le nombre de clusters est strictement supérieur à 1. La dernière valeur du paramètre *k* est naturellement égale à 13, puisqu'avec ce code nous réalisons douze exécutions de k-means afin d'avoir douze graphiques.
- L'utilisation de la fonction `silhouette_score` permet de calculer le coefficient de silhouette de chaque exécution de k-means avec une valeur différente du paramètre *k*. Le coefficient de silhouette associé à chaque exécution de k-means est ajouté à la liste `silhouette_lst`.

À la fin de l'exécution de la dernière cellule Jupyter, nous avons stocké dans la liste `silhouette_lst` toutes les valeurs du coefficient de silhouette.

▶ Dans une nouvelle cellule Jupyter, utilisez le code suivant afin d'afficher le graphique de l'évolution du coefficient de silhouette en fonction des valeurs du paramètre *k*.

```
fig, ax = plt.subplots(1)
ax.set_xticks(np.arange(15))
ax.plot(range(2,14), silhouette_lst)
ax.set_xlabel("k", fontsize=20)
ax.set_ylabel("Silhouette", fontsize=20)
fig.set_figheight(8)
fig.set_figwidth(8)
plt.grid()
```

L'exécution de ce code donne le graphique ci-dessous :

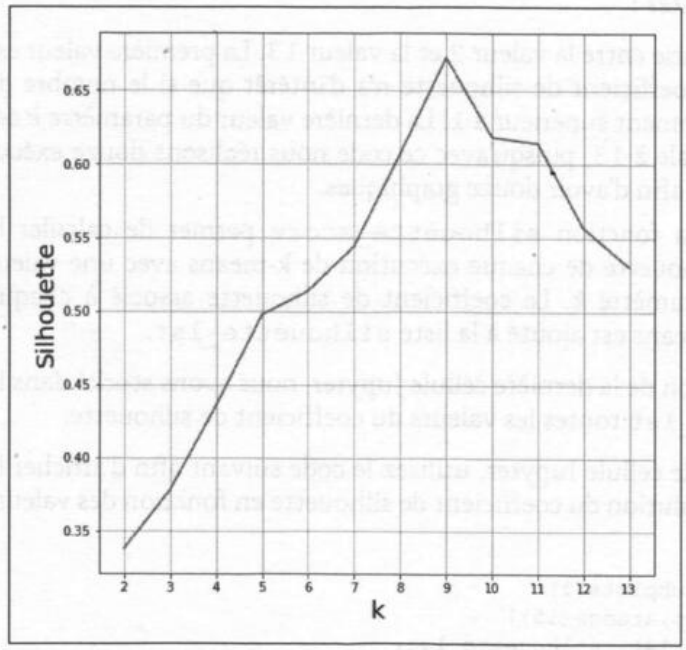


Figure 10-23 : courbe d'évolution du coefficient de silhouette en fonction de la valeur de k avec l'application de k-means sur les données exemple_simple_kmeans-3.csv

Ce graphique montre que la valeur du coefficient de silhouette moyen atteint une valeur maximale avec la valeur du paramètre `k=9`.

Dans ces deux derniers exemples, nous avons choisi un jeu de données qui permet d'illustrer l'utilisation des deux métriques de performance d'un clustering basées sur les méthodes Elbow et du coefficient de silhouette. Bien que ces deux métriques nous permettent de mesurer la qualité d'un clustering de manière quantitative, elles ne peuvent cependant pas être utilisées pour automatiser complètement le processus de prise de décision pour la détection de la meilleure valeur du paramètre *k*. À l'état actuel des choses, la détection du bon nombre de clusters à considérer requiert l'intervention d'experts humains ! Ces deux métriques sont plutôt un support qui nous conforte dans notre décision quant à la valeur du paramètre *k*.

© Editions ENI - All rights reserved

Il faut noter également que les deux métriques présentées ci-dessus ne sont pas spécifiques à l'algorithme k-means et peuvent être utilisées pour mesurer la qualité d'un clustering construit avec n'importe quel autre algorithme.

7. Les limites de k-means

Dans cette section, nous allons montrer un exemple où l'algorithme k-means ne réussira pas à répartir les données dans des clusters de façon efficace. Cela va nous permettre de mettre en lumière les faiblesses de cet algorithme. Également, ce nouvel exemple illustrera un cas où les deux métriques Elbow et le coefficient de silhouette ne nous seront pas d'une grande aide !

En effet, comme nous allons le voir, les performances de l'algorithme k-means dépendent directement de l'organisation des données.

- Pour la réalisation de ce nouvel exemple, réutilisez votre notebook Jupyter créé lors de l'exemple précédent ou réutilisez le notebook *k-means-04.ipynb*. Dans les deux cas, la seule modification nécessaire est de remplacer le nom du fichier *exemple_simple_kmeans-3.csv* par *exemple_simple_kmeans-4.csv* à partir duquel vous allez lire et utiliser un nouveau jeu de données. Une fois cette modification effectuée, exécutez toutes les cellules de ce notebook afin de pouvoir visualiser les résultats. Ces derniers sont présentés ci-après.

La visualisation de ce nouveau jeu de données est illustrée dans la figure suivante :

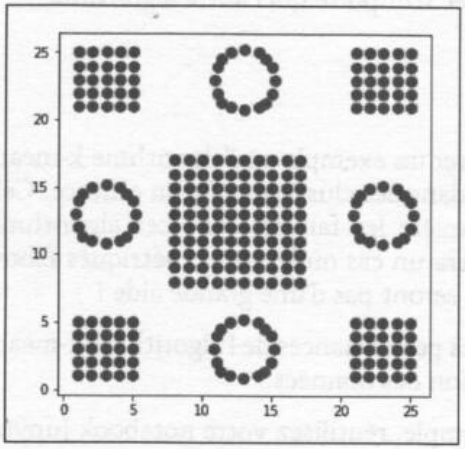


Figure 10-24 : représentation graphique des données du fichier exemple_simple_k-means-4.csv

Ce nouveau jeu de données ressemble beaucoup au jeu de données utilisé précédemment (voir figure 10-17), sauf que cette fois le groupe de données central contient un peu plus de points et il se rapproche grandement des groupes de données sur les bords. Nous allons voir que malgré cette différence de données relativement minime, elle aura un impact majeur sur le fonctionnement de l'algorithme k-means !

La visualisation du graphique de la figure 10-24 montre nettement que les données sont organisées dans neuf groupes. Cependant, avec la valeur `k=9`, k-means n'a pas réussi à positionner correctement tous les points comme le montre la figure 10-25. En effet, ces résultats montrent que les points du lot central ont été répartis dans les trois clusters représentés avec les chiffres 4, 7 et 8.

On note également que le centre du cluster représenté avec le chiffre 4 se situe en dehors du cercle formé par les points du lot au milieu en bas. Aussi, le centre du cluster représenté avec le chiffre 7 est quasiment situé sur le bord du cercle formé par les points du lot au milieu à droite.

© Editions ENI - All rights reserved

On peut expliquer ces résultats en observant que les points situés tout en bas du lot central sont plus proches des points en haut du lot au milieu en bas que des points situés tout en haut de ce même lot central ! Il en est de même pour les points situés tout à gauche du lot central qui sont plus proches des points à droite du lot au milieu à gauche que des points à droite de ce lot central !

Bien sûr, force est de constater que ces résultats sont de moindre qualité que ceux de la figure 10-20 avec k=9.

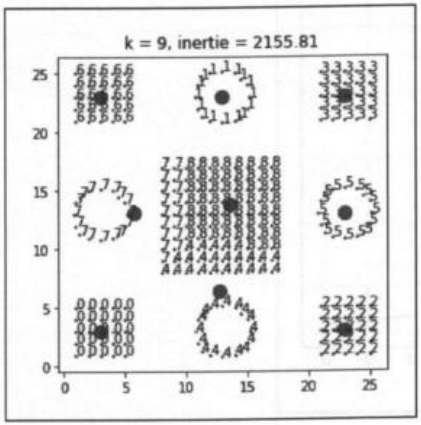


Figure 10-25 : clustering avec k-means et avec k=9 sur les données exemple_simple_kmeans-4.csv

La figure suivante trace la courbe Elbow correspondant à l'évolution de l'inertie totale en fonction de la valeur du paramètre *k* :

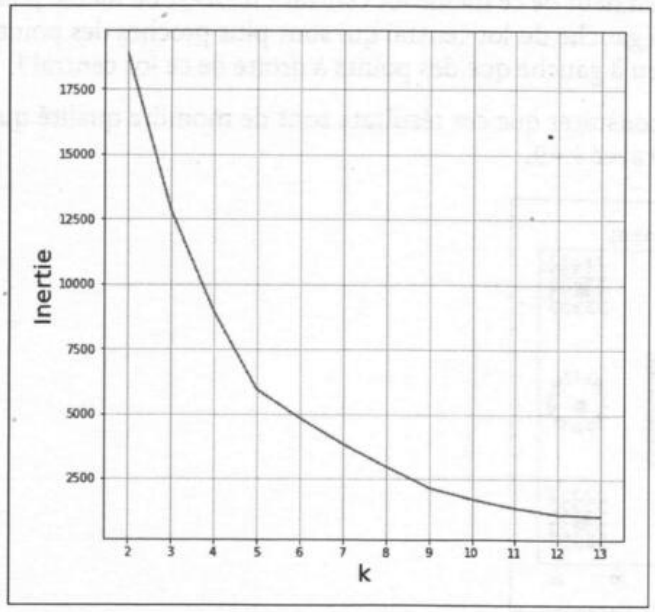


Figure 10-26 : courbe d'évolution de l'inertie en fonction de la valeur de k avec l'application de k-means sur les données exemple_simple_kmeans-4.csv

Comparativement à la courbe de la figure 10-22, cette dernière nous laisse incertains quant au choix de la valeur du paramètre Aller à la page précédente, car, pour rappel, le fait que nous sachions que `k=9` est la valeur optimale n'est dû qu'au fait que nous pouvons visualiser ces données. Ici, dans cette nouvelle courbe, nous aurions du mal à choisir entre la valeur `9` et la valeur `12` !

La figure suivante montre la courbe de l'évolution du coefficient de silhouette en fonction des différentes valeurs de *k*.

© Editions ENI - All rights reserved

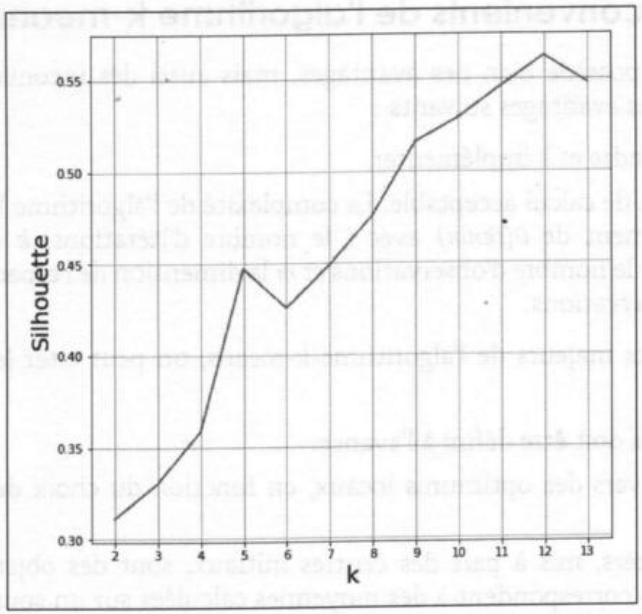


Figure 10-27 : courbe d'évolution du coefficient de silhouette en fonction de la valeur de k avec l'application de k-means sur les données exemple_simple_kmeans-4.csv

La courbe de l'évolution du coefficient de silhouette décrit un pic pour une valeur de `k=12` !

Cet exemple prouve que si notre analyse pour trouver la meilleure valeur du paramètre *k* n'était basée que sur les deux critères de performance Elbow et le coefficient de silhouette, alors nous n'aurions pas forcément considéré que `k=9` est la meilleure valeur !

En synthèse, l'utilisation de méthodes de mesure de la qualité d'un clustering, telles que les méthodes Elbow et du coefficient de silhouette, ne doit pas éliminer complètement l'analyse par un expert du contenu des différents clusters ! En effet, seule une analyse métier peut confirmer la véracité d'un clustering, et toutes les méthodes mathématiques de mesure de qualité d'un clustering ne devraient être utilisées que comme un support et une aide à la décision.

8. Avantages et inconvénients de l'algorithme k-means

L'algorithme k-means possède bien des avantages, mais aussi des inconvénients. On peut citer les avantages suivants :

- Il est facile à comprendre et à implémenter.
- Il implique un temps de calcul acceptable. La complexité de l'algorithme k-means est effectivement de $O(iknm)$ avec i le nombre d'itérations, k le nombre de clusters, n le nombre d'observations et m la dimension de l'espace de définition des observations.

Parmi les inconvénients majeurs de l'algorithme k-means, on peut citer les points suivants :

- Le nombre de clusters doit être défini à l'avance.
- Il converge souvent vers des optimums locaux, en fonction du choix des centres initiaux.
- Les centres des clusters, mis à part des centres initiaux, sont des objets inexistants puisqu'ils correspondent à des moyennes calculées sur un sous-ensemble d'observations à chaque itération.
- Une forte influence des valeurs aberrantes sur les résultats.
- Il donne des résultats médiocres pour les données qui ne sont pas linéairement séparables.
- Il n'est pas adapté aux données non numériques.

© Editions ENI - All rights reserved

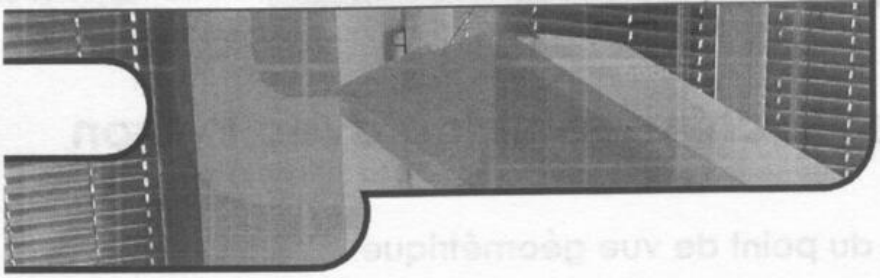
9. Quelques versions de l'algorithme k-means

Pour pallier les inconvénients de l'algorithme k-means dans sa version classique abordée dans ce chapitre, plusieurs versions ont été développées. Sans entrer dans les détails, on peut citer parmi ces versions :

- L'algorithme k-medoids : cet algorithme suit le même schéma général que l'algorithme k-means, sauf que k-medoids choisit les centres des clusters parmi les observations déjà présentes. Par rapport à k-means, il présente les deux avantages majeurs suivants :
  - Il est beaucoup moins sensible aux valeurs aberrantes.
  - Les distances entre les observations peuvent être calculées au début de l'algorithme puisque les centres des clusters sont représentés par des observations déjà existantes. C'est un avantage dans la mesure où le nombre d'itérations nécessaires est supérieur au nombre d'observations !
- L'algorithme k-modes : cet algorithme est une adaptation de l'algorithme k-means pour traiter les données de type non numériques. Le centre d'un cluster est calculé en fonction des fréquences des modalités majoritaires présentes dans ce cluster.
- L'algorithme CLARA : cet algorithme est adapté aux données de très grande taille. Il procède par application d'une version de k-means appelée PAM (*Partitioning Around Medoids*) sur plusieurs échantillons des observations initiales.

10. Conclusion

Après avoir présenté le fonctionnement de l'algorithme k-means via des illustrations graphiques, ce chapitre a présenté quelques exemples de code d'utilisation de k-means avec Scikit-learn en portant une attention particulière à la notion de mesure de qualité d'un clustering avec la présentation des deux méthodes Elbow et du coefficient de silhouette. Également, ce chapitre a abordé les limites de l'algorithme k-means via un exemple et évoqué ses avantages et ses inconvénients. Nous avons conclu avec une brève présentation de quelques versions de l'algorithme k-means.



Chapitre 4-5
Support Vector Machine

1. Objectif du chapitre

L'algorithme SVM fut introduit par Vapnik en 1995 et il est basé sur la théorie de Vapnik-Chervonenkis. Théoriquement, les SVM sont une généralisation des modèles de classification linéaires. Les SVM ont été appliqués avec beaucoup de succès dans divers domaines et spécialement dans le domaine de la reconnaissance de formes à partir d'images et le domaine de la bio-informatique.

Dans la littérature francophone, l'algorithme SVM est le plus souvent nommé «algorithme séparateur à vaste marge». En effet, les algorithmes SVM se distinguent de la plupart des autres algorithmes par la maximisation de la marge qui sépare deux classes de données. Ce chapitre tente d'expliquer les SVM d'un point de vue intuitif, mais aussi analytique, tout en mettant l'accent sur cet aspect de maximisation de la marge séparatrice.

À la fin de ce chapitre, le lecteur aura abordé :

- une compréhension intuitive de l'algorithme SVM,
- les SVM du point de vue théorique et analytique,
- la notion de fonction noyau,
- le théorème de Mercer,
- l'application de l'algorithme SVM pour la réalisation d'un modèle de détection de fraudes de cartes de crédit à partir d'un jeu de données réel contenant des données relatives à des transactions bancaires.

2. Le SVM du point de vue géométrique

Les SVM sont des algorithmes supervisés qui, au début, ont été développés pour résoudre des problèmes de classification binaire. Ils ont été généralisés pour prédire des variables quantitatives.

Dans le cas de la classification binaire, les SVM recherchent un hyperplan discriminant de marge optimale entre deux classes. Pour comprendre l'idée véhiculée par cet algorithme, cette section donne une explication intuitive basée sur des illustrations.

Dans la figure ci-dessous, deux ensembles sont représentés sur quatre graphiques à deux dimensions : le premier est composé de neuf carrés vides et le deuxième ensemble est composé de neuf cercles vides.

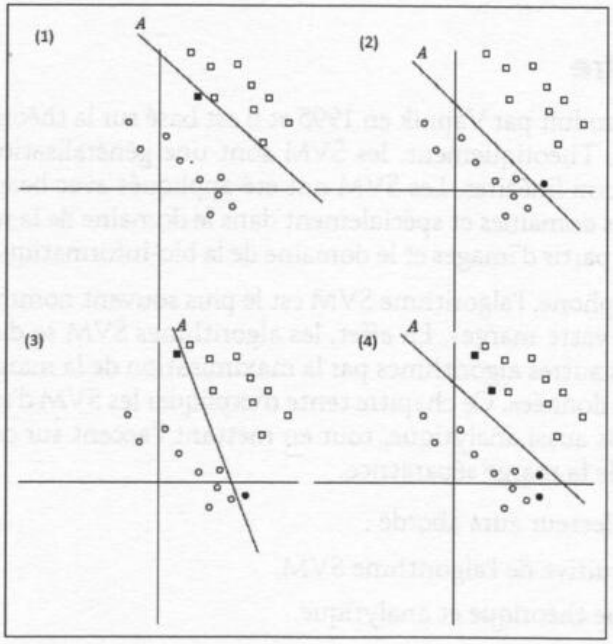


Figure11-1 : exemples de modèles linéaires

© Editions ENI - All rights reserved

Supposons que ces carrés et ces cercles vides forment la base de données d'apprentissage et que les carrés et les cercles noirs (pleins) sont les données futures, des données encore inconnues au moment de la phase d'apprentissage.

Sans prendre en compte ces données futures, le modèle représenté par la droite *(A)* dans chacun des quatre repères sépare toutes les données d'apprentissage de façon correcte. Cependant, des différences existent entre ces quatre modèles. En effet, bien qu'ils soient égaux vis-à-vis des données d'apprentissage, ils n'ont pas le même pouvoir de généralisation sur les données futures. Ainsi :

- Dans le repère (1), le modèle considère à tort que le point carré noir est un cercle puisqu'il est du côté gauche de la droite *(A)*.
- Dans le repère (2), le modèle considère à tort que le cercle plein est un carré puisqu'il est du côté droit de la droite *(A)*.
- Dans le repère (3), le carré noir et le cercle noir sont mal classés puisqu'ils sont du mauvais côté de la droite *(A)*.
- Dans le repère (4), toutes les données futures, cercles et carrés pleins, sont correctement classées.

Parmi ces quatre modèles, celui du repère (4) est le plus performant puisque non seulement il classe tous les exemples d'apprentissage correctement, mais en plus il classe correctement les exemples des données futures.

De façon intuitive, le modèle représenté par la droite *(A)* du repère (4) présente une meilleure capacité de généralisation, car cette droite est positionnée de façon à séparer correctement les données d'apprentissage, les carrés et les cercles vides. De plus, cette droite *(A)* est le plus loin possible de toutes ces données.

C'est justement cette idée impliquant de trouver un hyperplan le plus loin possible des données séparées qui est exploitée par le SVM. Les sections suivantes détaillent comment le SVM trouve un tel hyperplan.

La démarche de SVM

La figure 11-2 ci-après explique comment le SVM procède pour classer les données futures ou les données de production. Cette figure reprend le même exemple que celui du repère (4) de la figure précédente. Par exemple, pour trouver la classe d'appartenance du point P_1, le SVM effectue la projection de ce point sur un vecteur $\vec{w}$ normal à la droite *(A)*. En fonction du résultat de cette projection, le SVM prend la décision sur la classe d'appartenance de *P*1. Cette décision est prise en se basant sur les deux règles suivantes :

Si $\overrightarrow{p_1}.\vec{w} \geq B$, alors P_1 est un carré.

Si $\overrightarrow{p_1}.\vec{w} \leq B$, alors P_1 est un cercle.

La valeur de la constante B et le vecteur $\vec{w}$ sont calculés sur les données d'apprentissage. La section suivante montre comment ces deux éléments sont calculés.

On note que si $\overrightarrow{p_1}.\vec{w} = B$, alors le point sera considéré simultanément comme étant un cercle et un carré !

La projection de P_1 se situe au-dessus de la droite *(A)*, donc le point *P*1 sera classé dans la classe des carrés.

Cet exemple illustre bien l'importance de la position de la droite séparatrice *(A)*. En effet, cette droite peut être décalée légèrement en diagonale vers le haut ou en diagonale vers le bas, tout en gardant les données d'apprentissage bien séparées. En revanche, le point P_1 peut être classé comme un carré ou comme un cercle, en fonction de la position de cette droite séparatrice.

En d'autres termes, le SVM ne se contente pas de trouver un hyperplan qui sépare correctement les données d'apprentissage, il cherche en plus l'hyperplan qui s'apprête le mieux à la généralisation.

Le but du SVM est donc de trouver les éléments $\vec{w}$ et C en utilisant les données d'apprentissage, puis appliquer les deux règles de décision précédemment citées pour classer les données futures.

© Editions ENI - All rights reserved

Plusieurs algorithmes, autres que les algorithmes SVM, recherchent un hyperplan séparant correctement les données de la base d'apprentissage. Les SVM, comme évoqué lors des exemples précédents, recherchent la position optimale de cet hyperplan.

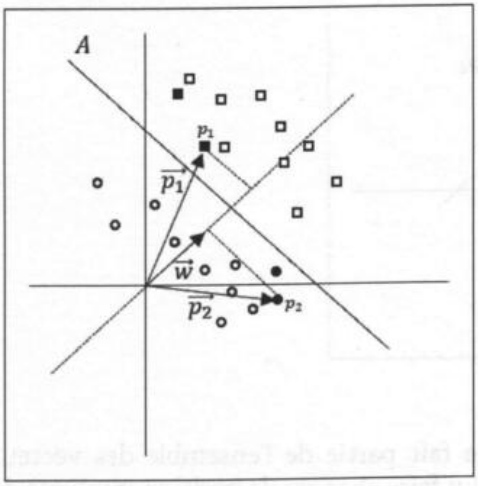


Figure11-2 : classification des données de production par le SVM

Cette position optimale peut être obtenue en maximisant les deux distances d_1 et d_2 (voir figure 11-3 ci-dessous). En d'autres termes, le SVM cherche à maximiser la marge qui sépare la classe des carrés de la classe des cercles. Dans l'exemple de cette figure, cette marge optimale entre les deux classes peut être représentée par les deux droites S_1 et S_2 et elle est obtenue en se basant sur les points p_1, p_2 et p_3. En effet, la droite *(A)* est placée à l'endroit où elle est la plus éloignée de p_1 et p_2 d'un côté, et p_3 de l'autre côté. Le SVM a choisi les points p_1, p_2 et p_3 pour déterminer la position de la droite *(A)*, car ces trois points représentent les frontières entre la classe des cercles et la classe des carrés. Ces points sont appelés les vecteurs supports, d'où le nom de Support Vector Machine !

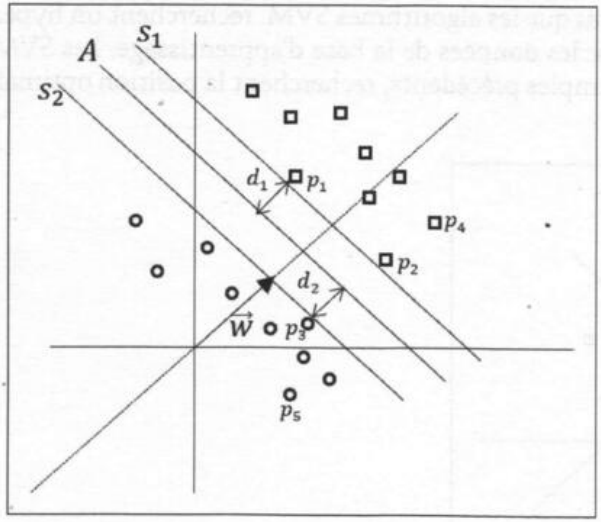


Figure 11-3 : séparateur à vaste marge

Un point de la base d'apprentissage fait partie de l'ensemble des vecteurs supports si et seulement si le fait de lui faire changer de position implique des modifications de la marge maximum séparant les deux classes.

Par exemple, dans la figure 11-3, supprimer le point p_3 a pour conséquence de déplacer la droite S_2 vers la gauche et ainsi d'élargir encore plus la marge qui sépare les deux classes. p_3 est donc un vecteur support. Pour la même raison, les points p_1 et p_2 sont également des vecteurs supports. En revanche, les deux points p_4 et p_5 n'en sont pas puisque le fait de les supprimer ne changera pas l'étendue de la marge qui sépare les deux classes.

3. Le SVM du point de vue analytique

L'objectif de l'algorithme SVM est de trouver l'hyperplan qui sépare en deux classes avec la marge maximale les données de la base d'apprentissage. Cette section montre la démarche analytique pour atteindre cet objectif.

© Editions ENI - All rights reserved

Supposons que les deux classes à séparer sont la classe des exemples positifs et celle des exemples négatifs. Nous avons vu qu'une fois l'hyperplan séparateur trouvé, pour déterminer la classe d'appartenance d'un point x_i, le SVM utilise une constante C et le vecteur $\vec{w}$, qui est un vecteur normal à l'hyperplan, comme suit :

Si $\vec{w}.\vec{x_i} \geq B$, alors x_i est positif.

Si $\vec{w}.\vec{x_i} \leq B$, alors x_i est négatif.

Si nous remplaçons B par $-b$, nous obtenons :

$\vec{w}.\vec{x_i} + b \geq 0$, pour les futurs points x_i positifs

$\vec{w}.\vec{x_i} + b \leq 0$, pour les futurs points x_i négatifs

Avec les deux inéquations précédentes, déterminer la classe d'un point x_i revient à étudier le signe de la quantité $\vec{w}.\vec{x_i} + b$. S'il est positif, alors x_i appartient à la classe des points positifs, sinon il appartient à la classe des points négatifs. En résumé, la classe d'un point x_i est déterminée par :

$Sign[\vec{w}.\vec{x_i} + b]$ (1)

L'équation (1) constitue donc la règle de décision que le SVM utilise pour la prédiction de la classe d'appartenance des futurs points. Cette règle est basée sur les deux inconnues $\vec{w}$ et b. L'objectif de la suite de cette section est d'expliquer les étapes analytiques nécessaires pour trouver les valeurs de ces deux inconnues.

Pour les points de la base d'apprentissage, la recherche des deux paramètres $\vec{w}$ et b peut être basée sur les deux contraintes suivantes :

$\vec{w}.\vec{x_i} + b \geq 1$, pour les points de la base d'apprentissage positifs (2)

$\vec{w}.\vec{x_i} + b \leq -1$ pour les points de la base d'apprentissage négatifs (3)

Les deux dernières contraintes permettent de restreindre la recherche de $\vec{w}$ et b de façon à ce que les données d'apprentissage positives soient éloignées des données d'apprentissages négatives et créer une frontière entre les deux classes.

Pour unifier ces deux contraintes, une nouvelle variable y_i est introduite, définie comme suit :

$y_i = 1$ pour les x_i positifs

$y_i = -1$ pour les x_i négatifs

Si on multiplie les deux équations (2) et (3) par y_i, on obtient :

Pour les $\vec{x}_i$ positifs et $y_i = 1$, $y_i(\vec{w}.\vec{x}_i + b) \geq 1$ (4)

Pour les $\vec{x}_i$ négatifs et $y_i = -1$, $y_i(\vec{w}.\vec{x}_i + b) \leq -1$ (5)

On multiplie l'inéquation (5) par -1 et on obtient : $y_i(\vec{w}.\vec{x}_i + b) \leq -1$ (6) avec $y_i = 1$

Remarquez que l'inéquation (6) est la même que l'inéquation (4). La même inéquation est donc obtenue pour les exemples positifs et les exemples négatifs. Ainsi, on peut résumer les deux inéquations (4) et (5) comme suit :

$y_i(\vec{w}.\vec{x}_i + b) - 1 \geq 0$

Pour les points les plus proches de l'hyperplan recherché, on impose la contrainte suivante $y_i(\vec{w}.\vec{x}_i + b) - 1 = 0$ (7).

Les points qui réalisent l'équation (7) sont ceux qui sont positionnés sur la frontière de la marge séparant la classe positive et la classe négative. Maximiser la marge entre ces deux classes revient à maximiser la quantité suivante :

$(\overrightarrow{x_+} - \overrightarrow{x_-}).\frac{\vec{w}}{||\vec{w}||} = \frac{(\overrightarrow{x_+}\ \vec{w} - \overrightarrow{x_-}\ \vec{w})}{||\vec{w}||}$ (8) (voir la figure 11-4)

© Editions ENI - All rights reserved

En utilisant l'équation (7), on obtient :

$\overrightarrow{x_+}.\vec{w} = 1 - b$, car $y_i = 1$ et $x+$ est un exemple positif.

$\overrightarrow{x_-}.\vec{w} = -b - 1$, car $y_i = -1$ et $x-$ est un exemple négatif.

Donc (8) devient $\frac{1-b+1+b}{||\vec{w}||} = \frac{2}{||\vec{w}||}$.

Le but est de maximiser la quantité $\frac{2}{||\vec{w}||}$ qui correspond à la marge séparant les exemples positifs des exemples négatifs. Pour des besoins de simplification mathématique, la quantité $\frac{2}{||\vec{w}||}$ sera remplacée par $\frac{1}{2}||\vec{w}||^2$.

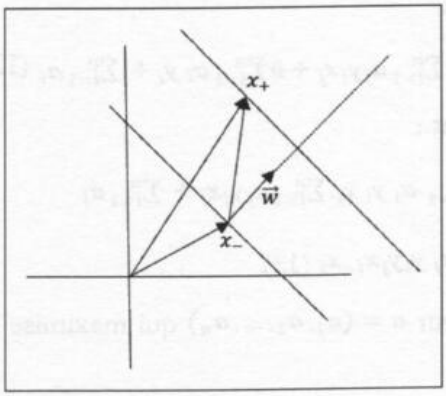


Figure 11-4 : la distance qui sépare les points positifs des points négatifs

En résumé, le problème à résoudre pour trouver le séparateur à vaste marge est le suivant $\begin{cases} Min\ \frac{1}{2}||\vec{w}||^2 \\ y_i(\vec{w}.\vec{x_i} + b) - 1 = 0 \end{cases}$

Pour résoudre ce problème, la méthode du dual, en introduisant des multiplicateurs de Lagrange, peut être utilisée.

Après l'application de cette méthode, le problème est reformulé comme suit :

$$L(w,\alpha) = \frac{1}{2}||\vec{w}||^2 - \sum_{i=1}^{n} \alpha_i[y_i(\vec{w}.\vec{x}_i + b) - 1] \quad (9)$$

Pour rechercher le maximum de la fonction L, il faudrait rechercher les points qui annulent les dérivées par rapport à w et b.

$$\frac{dL}{d\vec{w}} = \vec{w} - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \Rightarrow \vec{w} = \sum_{i=1}^{n} \alpha_i y_i x_i \quad (10)$$

$$\frac{dL}{db} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \Rightarrow \sum_{i=1}^{n} \alpha_i y_i = 0 \quad (11)$$

L'équation (10) montre que $\vec{w}$ recherché n'est rien d'autre qu'une combinaison linéaire des exemples de la base d'apprentissage. En remplaçant $\vec{w}$ obtenu en (10) dans l'équation (9), on obtient :

$$\frac{1}{2}\sum_{i=1}^{n} \alpha_i y_i x_i . \sum_{j=1}^{n} \alpha_j y_j x_j - \sum_{i=1}^{n} \alpha_i y_i x_i . \sum_{i=1}^{n} \alpha_j y_j x_j + b\sum_{i=1}^{n} \alpha_i y_i + \sum_{i=1}^{n} \alpha_i \quad (12)$$

En utilisant (11), l'équation (12) devient :

$$L(w,\alpha) = \frac{1}{2}\sum_{i=1}^{n} \alpha_i y_i x_i . \sum_{j=1}^{n} \alpha_j y_j x_j - \sum_{i=1}^{n} \alpha_i y_i x_i . \sum_{i=1}^{n} \alpha_j y_j x_j + \sum_{i=1}^{n} \alpha_i$$

$$\Rightarrow \quad L(w,\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i . x_j \quad (13)$$

L'objectif est donc de trouver un vecteur $\alpha = (\alpha_1, \alpha_2, \ldots, \alpha_n)$ qui maximise la fonction $L(w,\alpha)$ définie en (13).

Le problème d'optimisation à résoudre pour trouver l'hyperplan à vaste marge est le suivant :

$$\begin{cases} Max\ L(w,\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i . x_j \\ \text{Avec} \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \quad \text{et} \quad \alpha_i \geq 0, \forall i \in \{12, \ldots, n\} \end{cases}$$

Ce problème montre que la recherche de l'hyperplan à vaste marge dépend des produits scalaires entre les vecteurs (les données) de la base d'apprentissage.

© Editions ENI - All rights reserved

Remarque

Ne soyez pas inquiet si vous rencontrez des difficultés pour comprendre la formulation de ce dernier problème d'optimisation, car cela ne vous empêchera pas d'utiliser les algorithmes SVM pour réaliser des modèles de prédiction ! En effet, pour comprendre une telle formulation, il est nécessaire d'avoir des connaissances en optimisation, qui est une branche à part entière en mathématiques.

4. Données non linéairement séparables

Jusqu'à présent, dans ce chapitre, l'algorithme SVM tel qu'il a été présenté traite uniquement des données linéairement séparables. Ainsi, la figure 11-2 a montré que pour trouver la classe d'appartenance d'un nouveau point, le modèle SVM procède à la projection de ce point sur l'axe portant le vecteur normal de l'hyperplan qui sépare linéairement les données de la base d'entraînement.

Dans la plupart des problèmes posés dans la vraie vie, les données sont rarement linéairement séparables. Pour contourner ce problème, il existe une solution consistant à appliquer une transformation sur les données.

L'idée de cette transformation est de passer par un espace intermédiaire dans lequel les données sont linéairement séparables et dans lequel le SVM peut être appliqué exactement comme l'ont montré les sections précédentes, puis restituer les résultats dans l'espace d'origine.

La figure suivante donne un exemple très simple de données non linéairement séparables. En effet, aucune droite ne peut séparer correctement les cercles d'un côté et les triangles de l'autre côté.

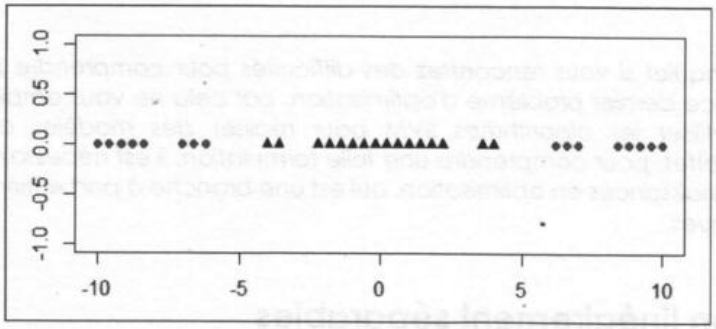


Figure 11-5 : exemple de données non linéairement séparables

Les données de cet exemple sont définies dans un espace à une dimension. Chaque point est associé à son abscisse x qui correspond à sa coordonnée sur un unique axe. La figure 11-6 donne le résultat de la transformation de ces points avec une fonction $\Phi(x) = (x, y = x^2)$. Les nouveaux points sont cette fois linéairement séparables !

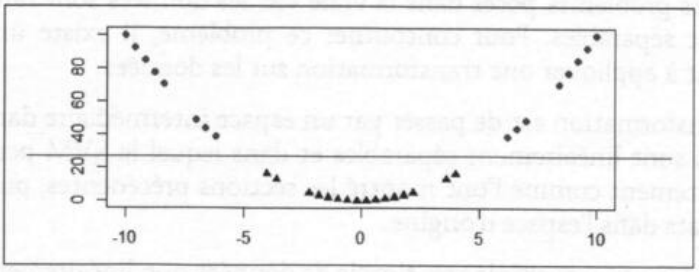


Figure 11-6 : transformation des points (x) par l'application $\Phi(x) = (x, y = x^2)$

Le but de cet exemple simple, mais pédagogique, est de montrer la possibilité d'utiliser une fonction Φ pour transformer des données non linéairement séparables et les rendre linéairement séparables.

Dans la réalité, les transformations Φ nécessaires pour rendre les données linéairement séparables sont plus complexes que la transformation définie dans cet exemple. Généralement, la fonction Φ transforme les données définies dans un espace à m_1 dimensions vers un autre espace à m_2 dimensions avec $m_2 > m_1$.

© Editions ENI - All rights reserved

Plus la fonction de transformation Φ est complexe, plus la résolution du problème d'optimisation pour trouver l'hyperplan à vaste marge devient compliquée. En effet, le problème à résoudre avec la transformation Φ devient le suivant :

$$P = \begin{cases} Max\ L(w,\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j\, y_i y_j\, \Phi(x_i).\Phi(x_j) \\ \\ \text{Avec } \sum_{i=1}^{n} \alpha_i y_i = 0 \text{ et } \alpha_i \geq 0, \forall\, i \in \{1,2,\dots,n\} \end{cases}$$

Également, avec une fonction de transformation Φ complexe, prédire la classe d'appartenance des données futures peut nécessiter des temps de calcul assez importants. En effet, une fois que w est fixé, prédire la classe d'appartenance d'une nouvelle donnée X revient à vérifier le signe de $Sign[\, w.X + b\,]$, voir l'équation (1)

En remplaçant w par sa valeur $\sum_{i=1}^{n} \alpha_i y_i x_i$, voir l'équation (10), on obtient :

$$Sign[\, \sum_{i=1}^{n} \alpha_i y_i x_i\,.X + b\,] = Sign[\, \sum_{i=1}^{n} \alpha_i y_i (x_i\,.X) + b\,]$$

Avec la transformation Φ, la dernière formule devient :

$$Sign[\, \sum_{i=1}^{n} \alpha_i y_i (\Phi(x_i).\Phi(X)) + b\,]$$

Le temps de calcul de cette dernière formule dépend directement de la complexité de la fonction Φ.

4.1 Le Kernel Trick

L'idée d'appliquer une transformation sur les données pour sortir d'un espace de données non linéairement séparable et le rendre linéairement séparable est intéressante en soi. Cependant, comme évoqué à la section précédente, la complexité de cette transformation Φ peut causer un problème lié au coût des calculs associés à la résolution du problème d'optimisation P, et également au coût des calculs associés aux prédictions sur les données de production.

Pour contourner ce problème, il existe une solution permettant d'appliquer la transformation Φ sans connaître sa définition !

Par exemple, soit $X = (x, y)$ et $\Phi(x, y) = (x^2, \sqrt{2}\,x\,y, y^2)$, alors :

$$\begin{aligned}\Phi(x,y).\Phi(x',y') &= (x^2, \sqrt{2}\,x\,y, y^2).(x'^2, \sqrt{2}\,x'y', y'^2)\\ &= x^2x'^2 + \sqrt{2}\,x\,y\,\sqrt{2}\,x'y' + y^2y'^2\\ &= (xx')^2 + 2\,x\,x'\,y\,y' + (yy')^2\\ &= [(x,y).(x',y')]^2\end{aligned}$$

Pour calculer le produit scalaire $\Phi(x, y).\Phi(x', y')$, il n'est donc pas nécessaire de savoir que $\Phi(x, y) = (x^2, \sqrt{2}\,x\,y, y^2)$ et $\Phi(x', y') = (x'^2, \sqrt{2}\,x'\,y', y'^2)$. Il suffit de calculer $[(x, y).(x', y')]^2$ qui n'est finalement rien d'autre que le produit scalaire des deux vecteurs élevés au carré. C'est ce qu'on appelle l'astuce du noyau, communément appelée en anglais **Kernel Trick**.

De façon plus générale, au lieu de calculer toutes les transformations $\Phi(X)$ pour réaliser ensuite les produits scalaires $\Phi(X).\Phi(Y)$, il suffit de définir une fonction noyau $K(X, Y)$ simple à calculer, mais qui correspond en réalité à un produit scalaire dans un espace où les données sont linéairement séparables.

Avec cette astuce du noyau, le problème du SVM à résoudre devient :

$$P = \begin{cases} Max\ L(w,\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j\, y_i y_j\, K(x_i, x_j) \\ \\ \text{Avec}\ \ \sum_{i=1}^{n} \alpha_i y_i = 0 \ \ \text{et}\ \ \alpha_i \geq 0, \forall\, i \in \{1,2,\ldots,n\} \end{cases}$$

Pour trouver la classe d'un nouveau point X, la solution est sous la forme :

$Sign[\sum_{i=1}^{n} \alpha_i y_i K(x_i . X) + b]$, voir l'équation (1)

© Editions ENI - All rights reserved

4.2 La condition de Mercer

L'utilisation d'une fonction noyau K permet donc de passer d'un espace où les données n'étaient pas linéairement séparables à un espace généralement beaucoup plus grand, où les données peuvent devenir linéairement séparables.

La section précédente a montré qu'avec le Kernel Trick la connaissance de la forme des données dans l'espace d'arrivée de la fonction K n'est pas nécessaire. En revanche, il faudrait s'assurer que cette même fonction K représente bien un produit scalaire dans cet espace d'arrivée.

Pour qu'une fonction $K: \mathrm{X} * \mathrm{X} \rightarrow \mathbb{R}$: soit une fonction noyau, il faut qu'elle vérifie la condition de Mercer dont l'énoncé est comme suit :

Une fonction symétrique $K(X,Y)$ est un noyau si :

$[K(X_i, X_j)]_{i,j}$ est une matrice semi-définie positive $\forall\, i, j$.

Si cette condition est vérifiée, alors $\exists\, \Phi: \mathrm{X} \rightarrow \mathrm{F}$ tels que $K(X,Y) = \Phi(\mathrm{X}).\Phi(\mathrm{Y})$

Cette condition, en pratique difficile à vérifier, garantit que $K(X,Y)$ représente bien un produit scalaire dans l'espace F.

Remarque

La condition de Mercer a été trouvée ou inventée par James Mercer en 1909.

4.3 Exemple de fonction noyau

Plusieurs fonctions noyaux existent. La liste non exhaustive ci-dessous recense les plus utilisées :

1. La fonction noyau polynomiale : $K(X,Y) = (X.Y + C)^n$.
2. La fonction noyau gaussienne : $K(X,Y) = \exp\left(-\frac{||X-Y||^2}{2\sigma^2}\right)$.
3. La fonction noyau sigmoïde : $K(X,Y) = \tanh\left((a(X.Y) - b)\right)$. La condition de Mercer n'est vérifiée que pour certaines valeurs de a et certaines valeurs de b.

5. Détecter les fraudes de cartes de crédit

Cet exemple concerne la réalisation d'un modèle pour détecter les fraudes de cartes de crédit. Il s'agit d'entraîner un modèle sur un ensemble de données contenant des transactions saines et des transactions frauduleuses.

5.1 Les données des transactions de cartes de crédit

Nous allons utiliser un jeu de données issu du site kaggle.com. Il s'agit d'un dataset du monde réel contenant 284807 transactions par carte de crédit. Parmi ces transactions, 492 représentent des fraudes. Vous trouverez ce jeu de données dans le fichier *creditcard.csv* sur le site https://www.kaggle.com/mlg-ulb/creditcardfraud.

Ce jeu de données est défini par vingt-huit variables anonymes $V_1, V_2, \ldots, V_{28}$ qui sont les résultats d'une ACP réalisée sur les données d'origine, plus trois autres variables `Time`, `Amount` et `Class` définies comme suit :

- `Time` : représente le temps en secondes passé depuis la première transaction du dataset.
- `Amount` : indique la somme d'argent concernée par la transaction.
- `Class` : est la variable binaire à expliquer. Elle indique si la transaction est frauduleuse ou pas. Elle vaut `0` si la transaction est normale, sinon elle vaut `1` si la transaction est frauduleuse.

Remarque

ACP est l'acronyme de l'Analyse en Composantes Principales, qui est une méthode permettant de résumer un ensemble de données. Le chapitre Les réseaux de neurones est entièrement consacré à cette méthode.

Remarque

Pour des raisons de confidentialité, les données à l'origine de l'ACP ne sont pas divulguées sur le site kaggle.com.

Il faut noter d'ores et déjà que la variable `Class` est très mal équilibrée, car le taux des transactions frauduleuses est très faible, il est égal à 0.172 %.

© Editions ENI - All rights reserved

Par conséquent, l'utilisation de l'indicateur AUC pour évaluer un quelconque modèle sur ce jeu de données ne sera pas appropriée. Dans ce cas, c'est l'utilisation de la courbe Précision/Recall qui sera recommandée.

Ce jeu de données a été publié dans : *Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson and Gianluca Bontempi. Calibrating Probability with Undersampling for Unbalanced Classification.In Symposium on Computational Intelligence and Data Mining (CIDM), IEEE, 2015.*

5.2 Application de l'algorithme SVM pour la détection des transactions bancaires frauduleuses

5.2.1 Application de l'algorithme SVM sur les données creditcard.csv

- Comme dans les précédents chapitres, lorsque vous avez appliqué des algorithmes tels que les algorithmes de régression et ceux de type Random Forest, pour appliquer un algorithme SVM, commencez par importer les bibliothèques nécessaires comme le montre le code suivant ; vous pouvez utiliser le code contenu dans le notebook *svm-01.ipynb* du dossier Code de ce chapitre :

```
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support as
score
from sklearn.model_selection import train_test_split
from sklearn  import svm
import numpy as np
```

La nouveauté de ce code est l'import de la bibliothèque `svm` qui, comme son nom l'indique, contient les fonctions nécessaires pour l'utilisation de l'algorithme SVM.

- Chargez les données contenues dans le fichier creditcard.csv pour les stocker dans le DataFrame `data_origin` avec l'instruction suivante :

```
data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')
```

Tout au long de ce chapitre, nous allons appliquer l'algorithme SVM plusieurs fois en apportant à chaque fois des modifications aux données et/ou aux paramètres du SVM. L'objectif est de montrer l'impact de ces modifications sur la qualité des modèles obtenus.

Dans un premier temps, nous allons appliquer l'algorithme SVM sur les données dans leur état d'origine.

Pour cela, avec l'instruction suivante, créez un nouveau DataFrame `data` dans lequel vous stockez toutes les variables explicatives :

```
data= data_origin[['Time', 'Amount', 'V1', 'V2', 'V3', 'V4',
'V5', 'V6', 'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14',
'V15', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23',
'V24', 'V25', 'V26', 'V27', 'V28']].copy()
```

Remarque

L'objectif de créer un nouveau DataFrame est de conserver dans la variable `data_origin` les données dans leur état d'origine afin qu'elles soient disponibles lors des prochaines applications de l'algorithme SVM, car vous pouvez réaliser tous les exemples de code de ce chapitre dans un seul et même notebook.

- De la même manière, stockez les données de la variable à expliquer dans un DataFrame y à l'aide de l'instruction suivante :

```
y =data_origin['Class']
```

- Puis créez le jeu de données d'entraînement et le jeu de données de test avec l'instruction suivante :

```
x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)
```

Ici, nous avons spécifié que 20 % des données seront stockées dans le jeu de données de test composé par `x_test` et `y_test` et donc 80 % des données seront stockées dans le jeu de données d'entraînement `x_train` et `y_train`.

Pour vous convaincre que ces données sont bien séparées avec ces proportions, les quatre instructions suivantes affichent la taille des données d'entraînement et de test :

```
print('La taille des données d\'entraînement X =
{}'.format(np.shape(X_train)))

print('La taille des données d\'entraînement Y =
{}'.format(np.shape(Y_train)))

print('La taille des données de test X =
```

© Editions ENI - All rights reserved

```
{}'.format(np.shape(X_test)))

print('La taille des données de test Y =
{}'.format(np.shape(Y_test)))
```

Le résultat de l'exécution de la cellule Jupyter précédente est le suivant :

```
La taille des données d'entraînement X = (227845, 30)
La taille des données d'entraînement Y = (227845,)
La taille des données de test X = (56962, 30)
La taille des données de test Y = (56962,)
```

Cela nous permet de vérifier que le DataFrame `x_train` contient bien 30 variables et 227845 données de transactions bancaires et `y_train` leurs libellés associés et que le DataFrame `x_test` contient également 30 variables correspondant à 56962 transactions bancaires et `y_test` leurs libellés associés.

Un point très important à vérifier également lors de la création des jeux de données d'entraînement et de test est la répartition des observations positives entre ces deux jeux de données. Ce contrôle peut être réalisé en affichant le nombre de cas positifs présents dans `y_train` et le nombre de cas positifs conservés dans le jeu de données de test `y_test`. En d'autres termes, il est important de savoir comment les cas positifs recherchés sont répartis entre les données d'entraînement et les données de test.

Pour notre exemple, nous pouvons afficher le nombre de transactions frauduleuses contenues dans `y_train` et celles contenues dans `y_test`.

▶Pour cela, utilisez les deux instructions suivantes :

```
print('Transactions frauduleuses dans y_train = ',sum(y_train))
print('Transactions frauduleuses dans y_test = ',sum(y_test))
```

Voici le résultat de l'exécution de ce code :

```
Transactions frauduleuses dans y_train =  395
Transactions frauduleuses dans y_test =  97
```

On constate que le rapport d'environ 20 % a bien été respecté par la procédure de séparation des données réalisée avec la fonction `train_test_split`.

Il faut noter que ce résultat obtenu avec la fonction `train_test_split` peut être légèrement différent de celui que vous obtenez.

Maintenant que le jeu de données est prêt, nous allons appliquer l'algorithme SVM. L'instruction suivante permet de créer une instance de l'algorithme SVM :

```
alg_svm= svm.SVC(kernel = 'linear')
```

À ce stade, nous disposons d'un objet `alg_svm`, qui représente notre algorithme SVM et qui ne demande qu'à ce qu'on lui fournisse les données d'entraînement.

Remarquez la valeur `linear` affectée à l'argument `kernel` lors de la construction de l'objet `alg_svm` associé à notre algorithme SVM. La valeur `linear` de l'argument `kernel` permet d'indiquer que c'est la version linéaire de l'algorithme SVM qui sera utilisée. En d'autres termes, l'algorithme SVM sera appliqué sur des données qui sont supposées être linéairement séparables.

Dans la suite de cette section, nous allons appliquer des versions de l'algorithme SVM avec des valeurs autres que `linear` pour le paramètre `kernel`.

L'instruction suivante permet de lancer la procédure d'apprentissage sur le jeu de données d'entraînement défini par les DataFrames `x_train` et `y_train` :

```
alg_svm.fit(x_train, y_train)
```

L'exécution de l'instruction précédente peut prendre plusieurs minutes, car le jeu de données d'apprentissage est assez important puisqu'il est composé de 227845 individus, chaque individu étant défini par 30 variables explicatives.

À la fin de cette phase d'apprentissage, le résultat suivant s'affiche :

```
SVC(kernel='linear')
```

L'exécution de la fonction `fit` se termine par l'affichage de la valeur utilisée avec le paramètre `kernel`.

À ce stade, le modèle a bien défini un hyperplan qui sépare les deux classes associées aux données d'entraînement. Comme nous l'avons vu au début de ce chapitre, cet hyperplan est principalement défini par les vecteurs supports ! Vous pouvez accéder à l'ensemble de tous les vecteurs supports en utilisant l'attribut `support_vectors_` de l'objet `alg_svm`.

© Editions ENI - All rights reserved

L'instruction suivante affiche le nombre des vecteurs supports trouvés par notre algorithme :

```
print(f"Le nombre de vecteurs supports est de :
{len(alg_svm.support_vectors_)}")
```

Dans notre exemple, la dernière instruction affiche le résultat suivant :

```
Le nombre de vecteurs supports est de : 533
```

Bien sûr, ce nombre de vecteurs supports dépend directement du jeu de données d'entraînement et des paramètres de l'algorithme SVM. Si vous exécutez de nouveau le code précédent à partir de la cellule Jupyter contenant l'appel de la fonction `train_test_split`, vous allez fort probablement aboutir à un jeu de données différent de celui associé à l'exécution précédente de ce même code. Cette différence dans le jeu de données d'entraînement, si minime soit-elle, peut engendrer un nombre de vecteurs supports différent après l'exécution de la fonction `fit`.

Nous pouvons même afficher le nombre des vecteurs supports par classe, par exemple avec l'instruction suivante :

```
print(f"Le nombre de vecteurs supports par classe :
{alg_svm.n_support_}")
```

Cette instruction affiche le résultat suivant :

```
Le nombre de vecteurs supports par classe : [266 267]
```

Pour notre exemple, le premier élément de la liste affichée correspond à la classe `0` et le deuxième élément correspond à la classe `1` ; d'ailleurs, 266 + 267 = 533. Pour en avoir le cœur net, vous pouvez utiliser l'instruction suivante :

```
print(f"Les classes ou les catégories manipulées sont :
{alg_svm.classes_}")
```

Ce qui produit ce résultat :

```
Les classes ou les catégories manipulées sont : [0 1]
```

L'étape de l'apprentissage sur les données d'entraînement est terminée. Nous allons donc pouvoir utiliser le modèle résultant afin d'effectuer des prédictions sur le jeu de données de test défini par `x_test` et `y_test`.

L'instruction suivante permet de réaliser des prédictions sur ce jeu de données de test :

```
predictions = alg_svm.predict(x_test)
```

La variable `predictions` est tout simplement la liste des prédictions du modèle sur les données de test `x_test`.

L'instruction suivante permet de calculer le nombre de cas positifs sur le jeu de données de test estimé par notre modèle.

```
print('Le nombre de cas positifs est : ', sum(predictions))
```

Ce qui produit le résultat suivant :

```
Le nombre de cas positifs est : 40
```

Bien sûr, il est possible que ce résultat soit légèrement différent de celui que vous obtenez avec votre code.

Pour évaluer notre modèle en nous basant sur les prédictions réalisées sur le jeu de données de test, nous utilisons les deux métriques de performance `precision` et `recall`. Pour calculer la `precision` et le `recall` nous allons utiliser la fonction `score` comme suit :

```
precision, recall, _, _ = score(y_test, predictions, pos_label=1,
average='binary')
```

La fonction `score` confronte tout simplement les prédictions réalisées par notre modèle sur les données de test et qui sont stockées dans le tableau `predictions` avec la réalité renseignée dans `y_test`.

On remarque le paramètre `pos_label` positionné à la valeur `1` afin d'indiquer la valeur qui sera utilisée pour le calcul des valeurs du `recall` et de `precision`, c'est-à-dire que la classe des cas positifs est désignée par la valeur `1`.

Nous allons également calculer le taux des prédictions correctes global `accuracy` à l'aide de l'instruction suivante :

```
accuracy = round((predictions==y_test).sum() /
len(predictions),3)
```

© Editions ENI - All rights reserved

À l'exécution de cette dernière instruction, la variable `accuracy` contient le taux de succès global obtenu par notre modèle.

L'instruction suivante permet enfin d'afficher nos trois métriques de performance en indiquant que les résultats doivent être affichés avec un arrondi de la partie décimale à trois positions après la virgule :

```
print('Precision: {} / Recall: {}./ Accuracy:
{}'.format(round(precision, 3), round(recall, 3), accuracy))
```

Le résultat de la dernière instruction est le suivant :

```
Precision: 0.7 / Recall: 0.289 / Accuracy: 0.999
```

On voit que le `recall` est très faible. Cela signifie que seulement 28.9 % des transactions frauduleuses contenues dans le jeu de données de test ont été détectées par notre modèle.

La valeur de `precision` est égale à 70 %. Cela correspond à la probabilité qu'une transaction soit réellement frauduleuse lorsqu'elle est détectée comme telle par notre modèle.

On remarque que le taux de succès global de réussite de notre modèle est très élevé, il est égal à 99.9 %. Malgré cela, les niveaux respectifs des métriques `recall` et `precision` sont relativement faibles. Cela est lié au fait que la variable à expliquer est déséquilibrée, car le nombre de transactions normales est beaucoup plus important que le nombre de transactions frauduleuses.

- Saisissez dans une cellule Jupyter le nom de la variable `alg_svm` suivi d'un point, puis appuyez sur la touche [Tab] de votre clavier pour afficher et explorer les différents attributs de cet objet ainsi que les différents paramètres utilisés par l'algorithme lors de la phase d'apprentissage.

Remarque

Remarquez au passage le nombre assez important de ces paramètres. Notez également dès maintenant que tous ces paramètres ne sont pas compatibles entre eux, dans le sens où certains ne sont utilisables que sous certaines conditions. Par exemple, le paramètre `Gamma` ne sera pas utilisé par l'algorithme si la valeur `linear` est affectée au paramètre `kernel`. Nous allons aborder un peu plus en détail ces paramètres dans la suite de ce chapitre.

5.2.2 Application du SVM sur un sous-ensemble de creditcard.csv

Les données sur lesquelles nous avons appliqué le SVM dans l'exemple précédent présentent une anomalie que nous pouvons remarquer par une simple visualisation des données du fichier *creditcard.csv*. En effet, on peut facilement voir que les deux variables `Time` et `Amount` sont définies dans des intervalles beaucoup plus larges que les vingt-huit autres variables.

Dans ce nouvel exemple, nous allons appliquer l'algorithme SVM exactement de la même manière que dans l'exemple précédent, sauf que cette fois-ci nous n'allons pas inclure ces deux variables `Time` et `Amount`. Le code suivant, défini dans le notebook *svm-02.ipynb* du dossier *Code* de ce chapitre reprend globalement les mêmes instructions que le code précédent :

```
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support as
score
from sklearn.model_selection import train_test_split
from sklearn  import svm
import numpy as np

data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')

data= data_origin[['V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7',
'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16',
'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V25',
'V26', 'V27', 'V28']].copy()

y =data_origin['Class']

x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)

alg_svm= svm.SVC(kernel = 'linear')

alg_svm.fit(x_train, y_train)

predictions = alg_svm.predict(x_test)

precision, recall, _, _ = score(y_test, predictions, pos_label=1,
average='binary')

accuracy = round((predictions==y_test).sum()/len(predictions),3)
```

© Editions ENI - All rights reserved

```
print('Precision: {} / Recall: {} / Accuracy:
{}'.format(round(precision, 3), round(recall, 3), accuracy))

print('Le modèle prédit {} transactions
frauduleuses'.format(sum(predictions)))

print('Dans les données de test, il y a {} transactions
frauduleuses'.format(sum(Y_test)))
```

Les seules différences avec le code précédent résident dans la définition de la variable `data` qui cette fois-ci ne contient pas les deux variables `Time` et `Amount`.

L'exécution de ce nouveau programme donne les résultats suivants :

```
Precision: 0.827 / Recall: 0.827 / Accuracy: 0.999
Le modèle prédit 104 transactions frauduleuses
Dans les données de test, il y a 104 transactions frauduleuses
```

La première chose que nous pouvons remarquer est que sans l'utilisation des deux variables `Time` et `Amount`, les deux métriques de performance `recall` et `precision` sont nettement améliorées !

Remarque

Ces résultats peuvent être surprenants, mais attention, cela ne signifie pas que les deux variables que nous avons exclues de cette nouvelle expérimentation n'ont pas de pouvoir prédictif pour la détection des transactions frauduleuses. De plus, dans les deux exemples précédents, nous avons posé l'hypothèse que les transactions, notamment les transactions frauduleuses, sont complètement indépendantes les unes par rapport aux autres et qu'il n'y a aucun lien d'ordre chronologique entre elles. C'est du fait de cette hypothèse que nous n'avons pas considéré ce jeu de données comme étant une série temporelle dont l'ordre chronologique serait défini par la variable `Time`. Dans la réalité, il peut être intéressant d'exploiter un tel jeu de données en tant que série temporelle ! Nous ne l'avons pas fait dans nos exemples afin de nous concentrer sur l'application du SVM de la façon la plus simple et la plus pédagogique possible.

Afin de renforcer votre compréhension des deux métriques de performance `recall` et `precision`, un point intéressant à observer dans ces nouveaux résultats est que le nombre de transactions réellement frauduleuses dans le jeu de données de test est égal à 104 et le modèle a en effet prédit 104 transactions frauduleuses. Cependant, cela ne signifie pas que ce sont les 104 transactions réellement frauduleuses qui ont été détectées par le modèle ; et cela se confirme par un `recall` égal à 82.7 % et non pas égal à 100 % ! Bien sûr, dans cet exemple, le fait que le nombre de transactions réellement frauduleuses soit égal au nombre de transactions frauduleuses prédites par le modèle est un simple hasard !

5.2.3 Application du SVM sur des données normalisées

Dans les deux exemples précédents, nous avions appliqué l'algorithme SVM pour construire un modèle de détection des transactions bancaires frauduleuses sans nous poser de questions sur la nature des distributions associées aux variables qui définissent le jeu de données utilisé.

En effet, dans le premier exemple, nous avons utilisé les données dans leur état d'origine, et dans le deuxième exemple nous avons tout simplement exclu les deux variables `Time` et `Amount` sans pour autant analyser ces variables en détail.

Si nous prêtons un peu plus d'attention à la forme de nos données, nous remarquons les différences des échelles de définitions entre les deux variables `Time` et `Amount` et les vingt-huit autres variables, en allant de `V1` jusqu'à `V28`. Ces différences d'échelles entre les variables posent bien sûr un gros problème puisque l'algorithme SVM utilise les valeurs des variables pour détecter les vecteurs supports et ainsi tracer les frontières entre les cas positifs et négatifs, c'est-à-dire délimiter les frontières entre les transactions frauduleuses et les transactions normales. Ce décalage d'échelle de définition des variables peut facilement conduire à des erreurs de calcul importantes.

© Editions ENI - All rights reserved

Jetons un coup d'œil à la nature des distributions associées à nos variables.

- Commencez avec le code suivant, par la lecture du DataFrame `data` comme lors du premier exemple, et créez un nouveau notebook Jupyter ou utilisez le code du notebook *stats.ipynb* contenu dans le dossier Code de ce chapitre :

```
import pandas as pd
from matplotlib import pyplot
import math
%matplotlib inline

data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')
data= data_origin[['Time','Amount','V1', 'V2', 'V3', 'V4', 'V5',
'V6', 'V7', 'V8', 'V9', 'V10',
       'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18',
'V19', 'V20',
       'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27',
'V28']].copy()
```

Nous allons utiliser la fonction `describe()` afin de visualiser dans une seule table les statistiques de base associées à toutes les variables. L'instruction suivante permet d'afficher la table de description de nos données :

```
data.describe()
```

Le résultat de l'instruction précédente est résumé dans la table suivante :

	Time	Amount	V1	V2	V3	V4	V5	V6	V7	V8
count	284807.000000	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	94813.859575	88.349619	3.919560e-15	5.688174e-16	-8.769071e-15	2.782312e-15	-1.552563e-15	2.010663e-15	-1.694249e-15	-1.927028e-16
std	47488.145955	250.120109	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00	1.237094e+00	1.194353e+00
min	0.000000	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01	-4.355724e+01	-7.321672e+01
25%	54201.500000	5.600000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01	-5.540759e-01	-2.086297e-01
50%	84692.000000	22.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	4.010308e-02	2.235804e-02
75%	139320.500000	77.165000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01	5.704361e-01	3.273459e-01
max	172792.000000	25691.160000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	1.205895e+02	2.000721e+01

8 rows × 30 columns

Figure 11-7 : les statistiques de base des variables `Time`, `Amount` et des variables de V1 à V28

Une simple visualisation de cette table permet de constater que la variable `Time` est définie dans l'intervalle `[min, max]=[0, 172792]` et la variable `Amount` est définie dans l'intervalle `[min, max]=[0, 25691.16]`, alors que toutes les autres variables sont définies avec des valeurs très proches de `0`. En effet, les variables de `V1` à `V28` ont chacune pour moyenne une valeur très proche de la valeur `0`.

Par exemple, si nous visualisons dans un histogramme la distribution de la variable `V1` avec le code suivant, nous obtenons le diagramme de la figure ci-dessous :

```
bins = [-60,-50,-40,-30,-20,-10,-9,-8,-7,-6,-5,-4,-3,-2,-1,0,1,2]
pyplot.figure(figsize=(10,7))
pyplot.hist(data['V1'], bins, alpha=0.5,label='V1')
pyplot.legend(loc='upper right')
pyplot.show()
```

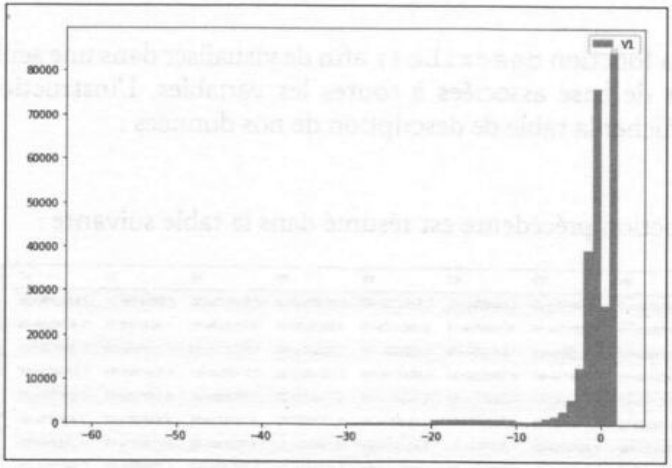


Figure 11-8 : l'histogramme associé à la variable `V1`

Et si nous visualisons le diagramme de la variable `Amount` avec le code suivant, nous obtenons le résultat sur la figure 11-9 ci-après :

```
bins = [0,5,10,15,20,25,30,50,75,100,125,150,175,200,300,400,500]
pyplot.figure(figsize=(10,7))
pyplot.hist(data['Amount'], bins, alpha=0.5,  label='Amount')
pyplot.legend(loc='upper right')
pyplot.show()
```

© Editions ENI - All rights reserved

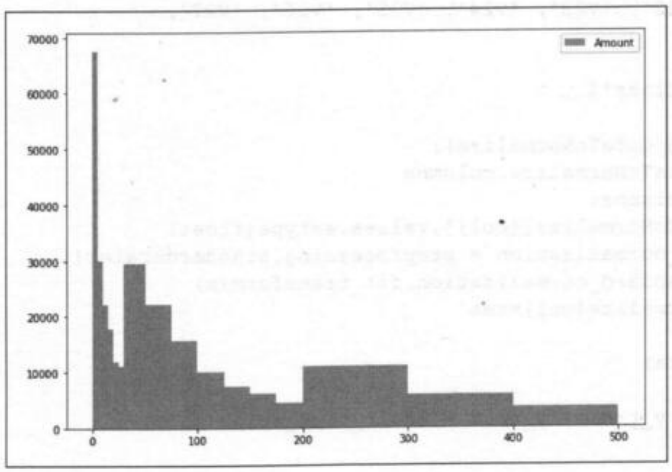


Figure 11-9 : l'histogramme associé à la variable `Amount`

Les deux diagrammes précédents montrent clairement et visuellement la différence des deux distributions associées aux deux variables `Amount` et `V1` et nous obtenons des différences similaires si nous comparons cette variable `Amount` avec chacune des variables de `V2` à `V28`.

Dans l'exemple suivant, nous allons procéder à la normalisation de toutes les variables du jeu de données du fichier *creditcard.csv*, sauf la variable `Time` qui ne sera pas utilisée. Puis nous allons appliquer l'algorithme SVM exactement de la même manière que dans l'exemple précédent.

▶ Pour cela, utilisez le code suivant, créez un nouveau notebook ou utilisez le notebook *svm-03.ipynb* contenu dans le dossier *Code* de ce chapitre :

```
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support as score
from sklearn.model_selection import train_test_split
from sklearn  import svm
import numpy as np
from sklearn import preprocessing

data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')
data= data_origin[['Amount','V1', 'V2', 'V3', 'V4', 'V5', 'V6',
'V7', 'V8', 'V9', 'V10',
       'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18',
'V19', 'V20',
```

```
        'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27',
'V28']].copy()

y =data_origin['Class']

def normalization(dataToNormalize):
    columns = dataToNormalize.columns
    for col in columns:
        x = dataToNormalize[[col]].values.astype(float)
        standard_normalization = preprocessing.StandardScaler()
        res = standard_normalization.fit_transform(x)
        dataToNormalize[col]=res

normalization(data)

x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)

alg_svm= svm.SVC(kernel = 'linear')

alg_svm.fit(x_train, y_train)

predictions = alg_svm.predict(x_test)

precision, recall, _, _ = score(y_test, predictions, pos_label=1,
average='binary')

accuracy = round((predictions==y_test).sum() / len(predictions),3)

print('Precision: {} / Recall: {} / Accuracy: {}'.format(round(preci-
sion, 3), round(recall, 3), accuracy))

print('Le modèle prédit {} transactions
frauduleuses'.format(sum(predictions)))

print('Dans les données de test, il y a {} transactions
frauduleuses'.format(sum(y_test)))
```

La seule différence entre ce nouveau code et le code de l'exemple précédent réside dans la définition et l'utilisation de la fonction `normalization`.

© Editions ENI - All rights reserved

Cette fonction normalise les colonnes d'un DataFrame qu'elle reçoit en argument comme suit :

- Récupération de toutes les variables du DataFrame reçu en paramètre.
- Pour chaque valeur de chaque variable, cette fonction soustrait la moyenne de la variable en question et divise le résultat par l'écart-type de cette même variable.

Mis à part la normalisation des variables, le reste de ce dernier code est similaire au code précédent. Une exécution de ce code a donné le résultat ci-après :

```
Precision: 0.867 / Recall: 0.802 / Accuracy: 0.999
Le modèle prédit 98 transactions frauduleuses
Dans les données de test, il y a 106 transactions frauduleuses
```

5.3 Les paramètres de l'algorithme SVM

L'algorithme SVM utilise plusieurs paramètres et la quête qui permet de trouver les valeurs optimales pour tous ces paramètres n'est pas une tâche facile. En réalité, les meilleures valeurs possibles pour les paramètres d'un algorithme SVM dépendent directement des données utilisées. De ce fait, il est nécessaire de réaliser plusieurs tests et de sélectionner les valeurs de ces paramètres qui permettent d'aboutir au meilleur modèle.

Dans la prochaine section, nous allons nous intéresser aux paramètres les plus couramment manipulés, dont la recherche du meilleur modèle. Il s'agit des paramètres `Kernel`, `C` et `Gamma`. Ces trois paramètres vont être expliqués, puis nous donnerons un exemple de code permettant d'utiliser *GridSearchCV* pour la recherche de valeurs intéressantes pour les paramètres `Kernel`, `C` et `Gamma`.

5.3.1 Le paramètre Kernel pour la variation de la fonction noyau

Le paramètre `Kernel` permet de désigner le type de la fonction noyau à utiliser. Les valeurs possibles pour ce paramètre sont les suivantes :

- `Linear` : la valeur que nous avons utilisée lors de tous les précédents exemples de ce chapitre. On l'utilise lorsque nous supposons que les données d'entraînement sont linéairement séparables et qu'aucune transformation de ces données n'est nécessaire. En d'autres termes, le Kernel Trick n'est pas nécessaire.
- `Poly` : la fonction noyau est une transformation polynomiale définie avec la formule suivante $(\gamma < x, y > +r)^d$

  Où γ, r et d sont trois paramètres spécifiés respectivement avec les arguments `Gamma`, `coef0` et `degree`.
- `Rbf` : cette fonction signifie *Radial Basis Function*, ou fonction à base radiale en français, et est définie comme suit $\exp(-\gamma||x-y||^2)$

  Avec γ un paramètre.

  Dans la littérature, on trouve souvent que cette fonction `Rbf` est définie par $\exp(-\frac{||X-Y||^2}{2\sigma^2})$

  Cependant, ces deux dernières formules sont en réalité les mêmes ; dans la documentation de Scikit-learn, le terme $\frac{1}{2\sigma^2}$ a simplement été remplacé par γ et c'est pour cette raison que dans la littérature on parle plutôt du paramètre γ et non du paramètre σ comme dans Scikit-learn.
- `Sigmoid` : cette fonction est définie comme suit $tanh(\gamma < x, y > +r)$

  Où γ et r sont deux paramètres spécifiés respectivement avec les arguments `Gamma` et `coef0`.
- Fonction définie par l'utilisateur : vous pouvez également définir votre propre fonction noyau et la transmettre en argument pour le constructeur d'instances de l'algorithme SVM, de la même manière que lorsque nous avons utilisé la valeur `linear` dans nos précédents exemples.

© Editions ENI - All rights reserved

Pour cela, il suffit de définir votre propre fonction permettant de réaliser le Kernel Trick. Cette fonction doit recevoir deux arguments correspondant à deux observations et retourne une valeur correspondant à la distance entre ces deux observations dans l'espace de plus grande dimension associé à votre fonction.

Remarque

Pour rappel, dans la mesure où votre fonction utilise la technique du Kernel Trick, vous n'avez nullement besoin de calculer les coordonnées de ces deux observations dans l'espace de plus grande dimension, cela est fait de manière implicite par la fonction que vous avez utilisée.

5.3.2 Le paramètre C

Nous avons vu que pour construire un modèle de classification binaire, l'algorithme SVM cherche un hyperplan le plus loin possible des observations des deux classes en question. La recherche d'un tel hyperplan passe par la résolution d'un problème d'optimisation où la bande séparatrice des deux classes est la plus vaste possible et où les données d'entraînement sont parfaitement classées. Cependant, dans certains cas, il serait plus intéressant de se satisfaire d'un hyperplan qui permet de classer correctement un grand nombre de données d'entraînement, mais pas toutes les données, et où la bande séparant les deux classes est de plus grande marge !

Considérons les points de la figure suivante représentés dans un plan à deux dimensions où les deux classes à séparer par un SVM sont la classe des carrés et la classe des cercles. Supposons que les données d'entraînement sont les carrés et les cercles vides et les données de production, ou les données de test, sont les carrés remplis.

Dans cet exemple, on voit que le modèle représenté avec la droite oblique sépare parfaitement les données d'entraînement, c'est-à-dire que les carrés et les cercles vides sont classés correctement à 100 %. En revanche, les carrés pleins sont malheureusement considérés comme étant des cercles puisqu'ils sont du côté gauche de la droite associée à ce modèle.

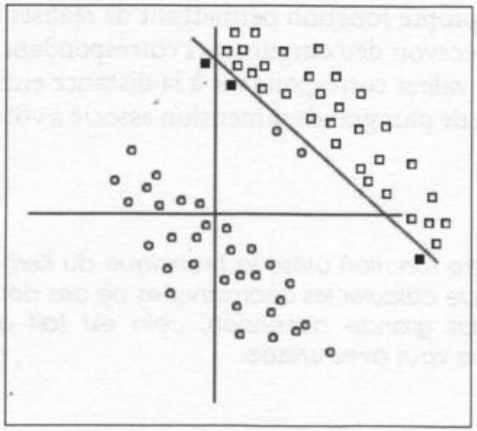


Figure 11-10 : exemple d'un modèle SVM non tolérant aux erreurs de classification

L'exemple précédent montre un cas où un modèle de type SVM peut avoir des performances maximales sur les données d'entraînement, mais des performances médiocres en production ! Généralement, nous faisons face à de telles situations lorsque nos modèles rencontrent le problème de surapprentissage.

Dans l'exemple précédent, on voit que pour obtenir un modèle avec un pouvoir de généralisation plus intéressant, nous devons construire un modèle où la marge séparant les carrés des cercles est plus vaste, au détriment d'une classification parfaite des données d'entraînement, comme dans l'exemple de la figure suivante.

© Editions ENI - All rights reserved

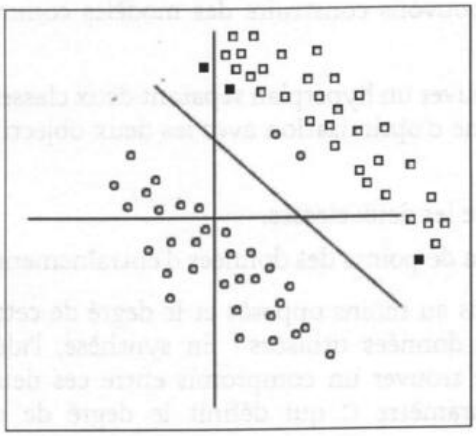


Figure 11-11 : exemple d'un modèle SVM tolérant aux erreurs de classification

En effet, cette dernière figure montre un modèle, représenté par la droite oblique, où les carrés et les cercles vides ne sont pas parfaitement séparés, puisqu'il y a deux cercles vides qui sont placés du mauvais côté de la droite, mais ce nouveau modèle a clairement un pouvoir prédictif beaucoup plus intéressant que le modèle précédent. Dans ce nouveau modèle, les carrés pleins sont cette fois-ci correctement classés.

Concrètement, l'algorithme SVM classique fonctionne de façon à trouver un hyperplan où toutes les données d'entraînement sont correctement classées au détriment du pouvoir de généralisation sur les données futures !

Par conséquent, lorsque nous utilisons l'algorithme SVM classique, nous supposons, ou nous espérons, que les données futures seront relativement loin des frontières tracées pas l'hyperplan correspondant au modèle. En effet, lorsque les données de test ou les données de production sont suffisamment loin de ces frontières, elles sont correctement classées malgré le fait que la marge séparatrice soit étroite. Par contre, si ces données sont très proches de ces mêmes frontières, alors le modèle risque de leur associer des classes erronées.

Pour pallier ce problème, en 1995, **Cortes** et **Vapnik** ont proposé une version du SVM, connue sous le nom de Soft Margin, où l'algorithme SVM accepte de mal classer certaines données d'entraînement en faveur de la construction d'un hyperplan à plus grande marge !

Avec cette nouvelle version, nous pouvons construire des modèles comme celui de l'exemple de la figure 11-11.

Avec la version Soft Margin, pour trouver un hyperplan séparant deux classes, l'algorithme SVM résout un problème d'optimisation avec les deux objectifs suivants :

- Accroître la marge séparatrice entre les deux classes.
- Classer correctement un maximum de points des données d'entraînement.

Bien sûr, ces deux objectifs sont plus au moins opposés et le degré de cette opposition dépend directement des données utilisées ! En synthèse, l'idée véhiculée par le Soft Margin est de trouver un compromis entre ces deux objectifs. C'est justement l'hyperparamètre `C` qui définit le degré de ce compromis.

Le paramètre `C` définit la pénalité associée à chaque observation d'entraînement mal classée. Par conséquent, nous pouvons déduire les deux points suivants :

1. Plus la valeur du paramètre `C` est élevée, moins large est la marge séparatrice, et le modèle tend à classer correctement un maximum de données, voire toutes les données d'entraînement. Dans ce cas, le risque est d'aboutir à un modèle avec des performances élevées sur les données d'entraînement, mais des performances médiocres sur les données de test et de production.
2. Plus la valeur du paramètre `C` est petite, plus grande est la marge séparatrice, puisque l'algorithme accepte plus facilement de mal classer certaines observations. Dans ce cas, le modèle n'est pas parfait sur les données d'entraînement, mais il a des capacités de généralisation plus intéressantes.

Il faut noter que la valeur du paramètre `C` n'est pas utilisée en tant que telle pour pénaliser une observation mal classée. La pénalité d'une observation mal classée se calcule en fonction de la valeur de ce paramètre `C` et de la distance de cette observation à l'hyperplan considéré. Les observations mal classées n'ont donc pas toutes la même pénalité.

© Editions ENI - All rights reserved

Pour revenir aux deux exemples précédents, une valeur trop élevée du paramètre C a une probabilité élevée d'aboutir au modèle de la figure 11-10 et une valeur faible a une probabilité importante d'aboutir au modèle de la figure 11-11.

5.3.3 Le paramètre Gamma

Le paramètre Gamma est utilisé uniquement avec des fonctions Kernel non linéaires et il permet d'indiquer le niveau d'influence de chaque observation du jeu de données d'entraînement sur la recherche de l'hyperplan associé au modèle. Lorsque la valeur de ce paramètre est élevée, pour que deux observations soient considérées comme faisant partie d'un même groupement de points, elles doivent être très proches entre elles. A contrario, lorsque la valeur du paramètre Gamma est suffisamment faible, les observations sont réparties dans des groupes de façon quasiment linéaire !

D'une certaine manière, ce paramètre Gamma permet de contrôler le niveau de courbure de l'hyperplan séparant les différentes classes. La figure 11-12 suivante montre une illustration du résultat d'un modèle qui serait obtenu avec une valeur du paramètre Gamma élevée.

Remarque

Pour rappel, ce type d'hyperplan avec des courbures n'existe pas et aucun SVM ne cherche réellement une fonction qui donne de telles courbures. Lorsque vous trouvez dans la littérature ce type d'hyperplan, il s'agit en réalité d'une représentation graphique des résultats retournés par un modèle SVM, car même lorsque les données ne sont pas linéairement séparables, le SVM utilise le Kernel Trick que nous avons abordé plus haut dans ce chapitre, afin de transformer ces données en données linéairement séparables, et cela sans pour autant calculer les nouvelles coordonnées des données dans l'espace de plus grande dimension.

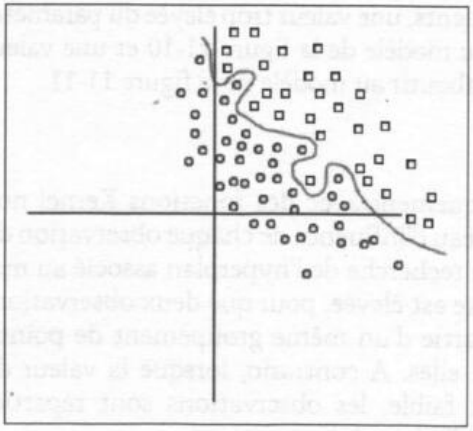


Figure 11-12 : exemple d'un modèle SVM avec une valeur élevée du paramètre Gamma

Les mêmes données que celles de la figure précédente peuvent aboutir à un hyperplan linéaire, comme dans la figure suivante, en utilisant une petite valeur du paramètre `Gamma`.

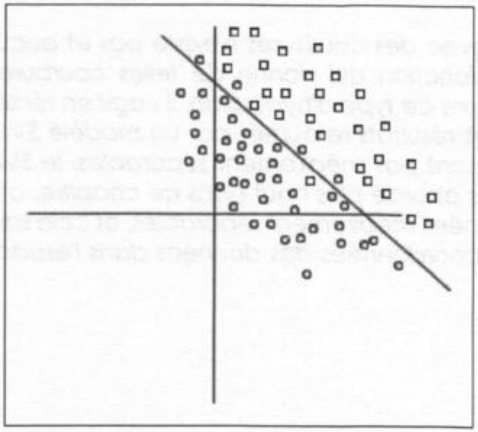


Figure 11-13 : exemple d'un modèle SVM avec une petite valeur du paramètre Gamma

Les deux figures précédentes montrent que plus la valeur de `Gamma` est élevée, plus le risque du surapprentissage est lui aussi élevé !

© Editions ENI - All rights reserved

5.3.4 Le paramètre C versus le paramètre Gamma

Nous avons vu que les deux paramètres `C` et `Gamma` influencent la forme de l'hyperplan associé à un modèle SVM. Lorsque la fonction Kernel utilisée est linéaire, nous avons simplement besoin d'optimiser la valeur du paramètre `C`, puisque le paramètre `Gamma` n'est pas applicable. En revanche, si la fonction Kernel utilisée est la fonction `Rbf`, alors nous avons réellement besoin d'optimiser les valeurs des deux paramètres `C` et `Gamma`. Cependant, si la valeur du paramètre `Gamma` est grande, alors le paramètre `C` devient négligeable.

Les valeurs communément utilisées avec ces deux paramètres `C` et `Gamma` sont les suivantes $\begin{matrix} 0.1 < C < 100 \\ 0.0001 < gamma < 10 \end{matrix}$.

Bien sûr, les valeurs optimales de ces deux paramètres dépendent directement des données utilisées et elles peuvent être différentes de celles mentionnées ici.

5.3.5 Tuning des hyperparamètres d'un SVM avec GridSearchCV

Afin d'illustrer le réglage d'hyperparamètres d'un algorithme SVM, dans ce nouvel exemple de construction d'un modèle de détection des transactions frauduleuses, nous allons appliquer cet algorithme SVM plusieurs fois avec à chaque fois des valeurs différentes de certains de ses paramètres en utilisant *GridSearchCV*.

- Pour la réalisation de cet exemple, créez un nouveau notebook et saisissez le code suivant ou utilisez le notebook *svm-04.ipynb* contenu dans le dossier *Code* de ce chapitre.
- Commencez par importer les modules suivants :

```
import pandas as pd
from sklearn.metrics import precision_recall_fscore_support as score
from sklearn.model_selection import train_test_split
from sklearn  import svm
import numpy as np
from sklearn import preprocessing
from sklearn.model_selection import GridSearchCV
```

Comme nous allons le voir dans la suite de ce nouveau programme, la classe `GridSearchCV` nous permet de construire de façon simple un pipeline où nous allons pouvoir spécifier un jeu de valeurs pour un ensemble de paramètres et appliquer un algorithme, qui sera un algorithme SVM dans notre cas, autant de fois que de combinaisons de ces valeurs de paramètres possibles. De plus, pour chaque combinaison de valeurs des paramètres spécifiés, l'algorithme choisi sera appliqué avec la technique de cross-validation. Le code suivant permet de lire et de normaliser les mêmes données que celles utilisées lors du dernier exemple. La dernière instruction de ce code permet tout simplement de construire les deux jeux de données d'entraînement (`x_train`, `y_train`) et de test (`x_test`, `y_test`).

```
data_origin = pd.read_csv('../Data/creditcard.csv', sep=',')
data= data_origin[['Amount','V1', 'V2', 'V3', 'V4', 'V5', 'V6',
'V7', 'V8', 'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15',
'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24',
'V25', 'V26', 'V27', 'V28']].copy()

y =data_origin['Class']

normalization(data)

x_train, x_test, y_train, y_test = train_test_split(data, y,
test_size=0.2)
```

Le code suivant permet de construire une instance d'un algorithme SVM avec le nom `alg_svm` et un dictionnaire `params` dans lequel nous avons indiqué trois informations, à savoir :

1. La clé `kernel` correspond à une liste contenant deux valeurs `rbf` et `linear`.

2. La clé `gamma` correspond à une liste de six valeurs.

3. La clé `C` correspond à une liste de six valeurs.

```
alg_svm= svm.SVC()
params = {'kernel': ['rbf'],
          'gamma': [0.0001,0.001,0.005,0.01,1,10],
          'C':[0.1,1,5,10,50,100]}
```

© Editions ENI - All rights reserved

Le dictionnaire `params` sera utilisé afin d'indiquer tous les paramètres avec lesquels nous souhaitons exécuter l'algorithme `alg_svm` ! Au total, il y aura 36 exécutions de cet algorithme en utilisant à chaque exécution des valeurs différentes des paramètres `Gamma` et `C` à partir de leurs listes respectives.

L'instruction suivante permet justement de construire une instance de l'objet `hyper_params_grid` qui va jouer le rôle d'un workflow. Pour cet objet de type `GridSearchCV`, nous avons indiqué les paramètres suivants :

- `alg_svm` pour indiquer que c'est cet algorithme, qui est un algorithme SVM, qui sera utilisé.
- `params` pour indiquer le jeu de valeurs des paramètres `kernel`, `gamma` et `C` à utiliser.
- `cv =2` afin de préciser le nombre de paquets qui seront utilisés avec la technique de cross-validation.
- `scoring='accuracy'` afin de choisir le critère de performance qui sera utilisé pour la comparaison des différents modèles qui seront construits.
- `n_jobs=-1` pour indiquer notre souhait d'utiliser tous les processeurs disponibles lors de la phase d'entraînement qui sera lancée plus tard avec la fonction `fit`.

```
hyper_params_grid = GridSearchCV(alg_svm, params, cv=2, scoring
='accuracy', n_jobs=-1)
```

Il ne nous reste plus qu'à lancer l'entraînement de l'algorithme `alg_svm` sur les données d'entraînement en utilisant la fonction `fit` comme dans l'instruction suivante :

```
hyper_params_models = hyper_params_grid.fit(x_train, y_train)
```

À la fin de l'exécution de la fonction `fit`, nous pouvons afficher les paramètres qui ont servi à la construction du meilleur modèle avec l'instruction suivante :

```
hyper_params_models.best_params_
```

En effet, l'affichage de l'attribut `best_params_` montre les résultats suivants :

```
{'C': 10, 'gamma': 0.005, 'kernel': 'rbf'}
```

Si nous n'avions pas spécifié des valeurs précises pour les deux paramètres `Gamma` et `C`, alors l'algorithme `alg_svm` aurait tout simplement utilisé les valeurs par défaut de ces deux paramètres et qui correspondent à la valeur `1` pour le paramètre `C` et à la valeur $\frac{1}{n_feature * X.var()}$ pour le paramètre `Gamma` où $n_feature$ est égal au nombre de variables et $X.var()$ la variance du jeu de données traité.

Ces derniers résultats montrent que les meilleures valeurs trouvées pour ces deux paramètres `Gamma` et `C` sont différentes de celles utilisées par défaut.

En effet, pour le paramètre `C`, c'est évident, car la meilleure valeur est égale à `10`, alors que la valeur par défaut est égale à `1`. Pour le paramètre `Gamma`, l'exécution du programme précédent sans fixer explicitement la valeur de `Gamma` s'est terminée en indiquant que la valeur du paramètre `Gamma` qui a été utilisée est d'environ `1.47`.

Remarque

Pour vérifier les valeurs par défaut de ces deux paramètres, il suffit d'exécuter un des exemples des sections précédentes, puis d'afficher les valeurs des attributs `alg_svm.C` et `alg_svm._gamma`.

L'instruction suivante nous permet justement de récupérer le meilleur modèle dans la variable `model`.

```
model = hyper_params_models.best_estimator_
```

Maintenant que nous avons ce meilleur modèle, nous n'avons qu'à l'utiliser pour réaliser les prédictions sur les données de test et afficher les métriques de performance `precision`, `recall` et `accuracy` comme nous l'avons fait lors des exemples précédents :

```
predictions = model.predict(x_test)

precision, recall, _, _ = score(y_test, predictions, pos_label=1,
average='binary')

accuracy = round((predictions==y_test).sum() /
len(predictions),3)

print('Precision: {} / Recall: {} / Accuracy:
```

© Editions ENI - All rights reserved

```
{}'.format(round(precision, 3), round(recall, 3), accuracy))

print('Le modèle prédit {} transactions
frauduleuses'.format(sum(predictions)))

print('Dans les données de test, il y a {} transactions
frauduleuses'.format(sum(y_test)))
```

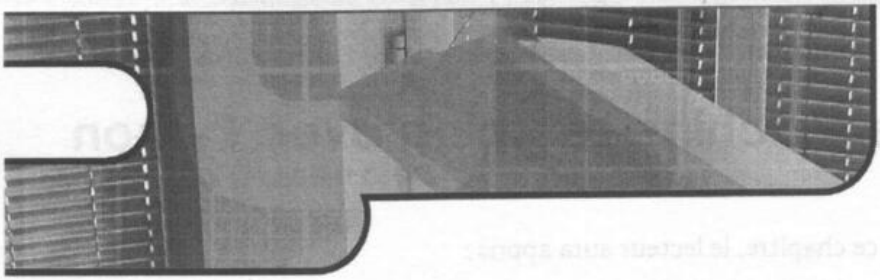
Les deux dernières instructions permettent d'afficher à titre indicatif le nombre de transactions détectées par le modèle en tant que transactions frauduleuses et le nombre de transactions qui sont réellement frauduleuses présentes dans les données de test. Ce modèle donne les résultats suivants :

```
Precision: 0.975 / Recall: 0.762 / Accuracy: 1.0
Le modèle prédit 79 transactions frauduleuses
Dans les données de test, il y a 101 transactions frauduleuses
```

Ces résultats montrent un niveau de la métrique `recall` intéressant puisqu'il est de 76.2 %, mais surtout ces résultats sont encore plus intéressants que ceux que nous avons obtenus avec tous les autres exemples précédents de ce chapitre puisque cette fois la métrique `precision` est quasiment parfaite !

6. Conclusion

L'algorithme SVM est l'un des algorithmes les plus sophistiqués parmi les algorithmes d'apprentissage automatique. Les deux premières sections ont abordé les SVM par des illustrations géométriques pour fournir au lecteur une compréhension intuitive de ces algorithmes SVM. La troisième section a introduit les notions mathématiques pour expliquer le problème d'optimisation sous-jacent aux SVM. La quatrième section a abordé les aspects théoriques relatifs à la fonction noyau utilisée dans le cadre des SVM pour traiter les cas où les données ne sont pas linéairement séparables. Nous avons terminé ce chapitre avec des exemples permettant de montrer d'une part comment appliquer les SVM sur un problème en utilisant Scikit-learn, et d'autre part comment manipuler les données et régler les paramètres des SVM afin d'améliorer les performances des modèles.



Chapitre 4-6
Analyse en composantes principales

1. Objectif du chapitre

Les algorithmes du Machine Learning permettent principalement de réaliser des regroupements, des classifications et des régressions dans l'objectif d'analyser les données et/ou de construire des modèles prédictifs sur ces données. Le plus souvent, ces données sont définies dans des espaces de très grande dimension. Cela cause deux problèmes majeurs, à savoir :

- Plus la dimension est élevée, plus les données sont éparses, ce qui rend la tâche des algorithmes de prédiction plus compliquée et très sensible à la généralisation.
- La représentation graphique de ces données devient quasiment impossible puisque les capacités de visualisation de l'œil humain sont limitées à un nombre de dimensions $d \leq 3$.

À partir de ce constat, le besoin de réduire la dimension des données devient important. L'objectif de ce chapitre est d'introduire l'ACP (Analyse en Composantes Principales) qui est l'une des méthodes les plus utilisées pour la réduction de dimensions.

À la fin de ce chapitre, le lecteur aura appris :

- ce qu'est l'ACP et à quoi elle sert,
- les aspects analytiques associés à l'ACP,
- comment mettre en œuvre une ACP et interpréter ses résultats,
- l'utilisation de Scikit-learn pour réaliser une ACP.

2. Pourquoi l'ACP ?

Les données de grande dimension posent des problèmes liés à la pertinence des modèles prédictifs. Mais avant tout, cette grande dimension pose des problèmes de visualisation. En effet, l'œil humain est un outil très puissant qui, à partir d'un simple graphique et en quelques fractions de seconde, peut détecter des regroupements d'individus et des liaisons entre les variables. Ces informations sur les individus et les variables peuvent naturellement être exploitées et utilisées pour la construction de modèles prédictifs performants. Le problème est que si les données sont exprimées dans un espace de dimension supérieur à trois, l'œil humain devient inutile. Dans la pratique, le plus souvent, les données appartiennent à des espaces nettement supérieurs à cette limite humaine. Pour en être convaincu, il suffirait par exemple de faire un tour sur le site www.kaggle.com où plusieurs compétitions sont proposées aux Data Scientists et aux amateurs de l'intelligence artificielle. Ces compétitions portent sur des problèmes réels issus du monde des entreprises et des laboratoires de recherche.

© Editions ENI - All rights reserved

Il existe plusieurs méthodes qui permettent de synthétiser les informations d'un nuage de points dans un espace de dimension beaucoup plus petit que l'espace initial, sans pour autant perdre beaucoup d'informations. L'analyse en composantes principales est une de ces méthodes.

L'ACP procède à la projection d'un nuage de points défini dans un espace E à d dimensions dans un autre espace F à p dimensions avec $d=p$. Mais alors, pourquoi diable cette projection puisque d équivaut à p ?

La particularité de l'espace F est que ses axes sont ordonnés de sorte que le premier axe porte le maximum d'informations sur le nuage de points initial par rapport à tous les autres axes. Le deuxième axe porte le maximum d'informations sur le nuage de points par rapport aux (p-2) axes restants. Le troisième axe porte le maximum d'informations sur le nuage de points par rapport aux (p-3) axes restants, et ainsi de suite. En d'autres termes, dans le nouvel espace où les données sont projetées, l'information véhiculée par le nuage de points est concentrée principalement sur les premiers axes ; en tout cas, c'est ce que l'on espère obtenir avec l'ACP.

Une fois l'espace F calculé, il suffit de choisir les deux premiers axes pour représenter graphiquement le nuage de points dans un plan à deux dimensions, tout en conservant un maximum d'informations sur le nuage initial défini dans l'espace E.

En synthèse, l'ACP permet d'approcher un nuage de points X, inaccessible graphiquement, par un autre nuage de points Y qui, lui, est représentable sur deux ou trois axes sans trop déformer le nuage initial X. Les sections suivantes montrent comment l'ACP procède pour atteindre un tel objectif.

3. L'ACP du point de vue géométrique

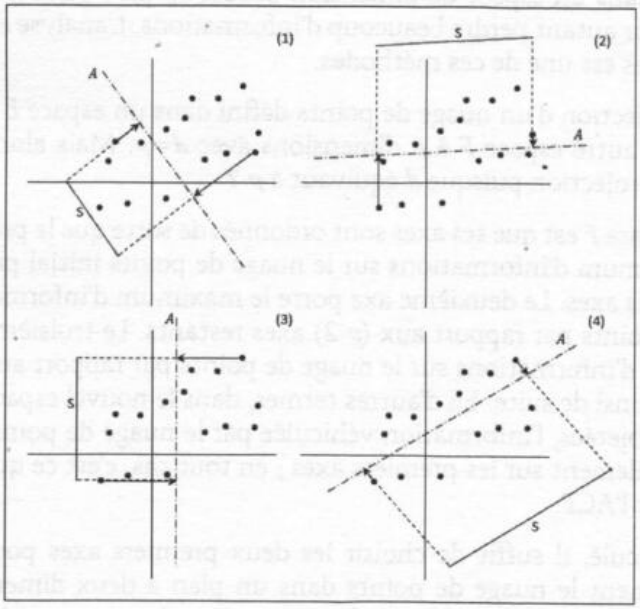


Figure 12-1 : projection d'un nuage de points sur différents axes

Afin de donner une vision géométrique et intuitive de l'ACP, considérons le nuage de 15 points défini dans un espace à deux dimensions et représenté dans les quatre graphiques de la figure 12-1. Pour passer de cet espace à deux dimensions à un espace plus petit à une dimension, il faudrait procéder à la projection sur un seul axe de tous les points du nuage. Comme il existe une infinité d'axes de projection possibles, la question qui se pose immédiatement est la suivante : comment choisir cet axe pour qu'à la fin de la projection l'information totale présente dans le nuage de points initial soit déformée le moins possible sur ce nouvel axe ?

Dans chacun des graphiques ci-dessous, le nuage de points est projeté sur un axe *A* différent. Vous remarquerez que la différence entre les quatre projections se situe essentiellement au niveau de la taille du segment *S* contenant toute la projection du nuage sur l'axe *A*.

© Editions ENI - All rights reserved

La projection d'un nuage de points défini dans un espace à plusieurs dimensions dans un autre espace de dimension inférieure ne peut que déformer l'information présente dans ce nuage projeté. Cependant, plus la variance totale du nuage de points projeté s'approche de la variance totale du nuage de points initial, moindre serait la déformation de l'information.

Dans notre exemple, c'est la projection du graphique (4) qui est la plus intéressante, puisqu'elle contient le segment S le plus grand. Intuitivement, la taille du segment S correspond à la variabilité, ou la variance, des points projetés. En d'autres termes, plus le segment contenant la projection est grand, plus la variance du nuage initial est conservée.

Dans un espace à d dimensions, pour réaliser une projection optimale qui conserve le maximum de variance du nuage projeté, il suffit donc de trouver un axe A_1 sur lequel la variance des points projetés est maximale. Bien sûr, la variance projetée sur cet axe va être inférieure à la variance du nuage initial. Pour récupérer le reste de la variance ainsi perdue, il suffit de calculer un deuxième axe A_2, orthogonal au premier axe A_1 déjà trouvé, et sur lequel la projection est de nouveau optimale par rapport à la variance restante à récupérer. Par définition, la variance portée par le deuxième axe est inférieure à la variance portée par le premier axe. De même, si $d>2$, alors la somme des deux variances projetées sur les deux premiers axes est strictement inférieure à la variance totale du nuage initial. Nous pouvons continuer à chercher des axes $A_3, A_4,..., A_d$. L'ensemble de tous ces axes $A_1, A_2..., A_d$ sont orthogonaux entre eux deux à deux. La variance totale portée par tous ces axes est égale à la variance du nuage de points initial.

Les données traitées en ACP sont centrées afin d'avoir un nuage de points réparti autour de sa moyenne. Centrer un nuage de points revient à soustraire la moyenne à toutes les valeurs observées. De plus, si les variables sont exprimées dans différentes unités de mesure, alors les variables doivent être réduites. Réduire une variable revient à diviser toutes ses valeurs sur son écart-type de sorte que les valeurs résultantes expriment les écarts à la moyenne et non pas des valeurs absolues. Une fois que les variables sont réduites, elles deviennent indépendantes des unités de mesure.

Remarque

Dans la littérature française sur le sujet de l'ACP, on parle souvent d'inertie et de centre de gravité pour désigner respectivement la variance et le point moyen d'un nuage de points. La variance concerne la dispersion des valeurs prises par une variable autour de sa moyenne, et l'inertie concerne la dispersion d'individus (observations sur plusieurs variables) autour de l'individu moyen (l'individu constitué des moyennes de toutes les variables). Les termes «inertie» et «centre de gravité» sont probablement dus à l'usage de l'ACP dans le domaine de la physique, et particulièrement dans la mécanique.

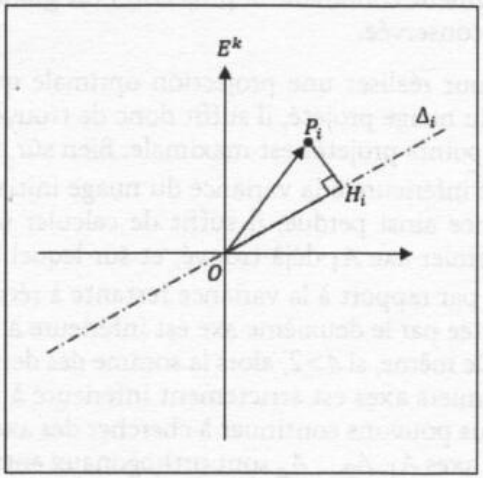


Figure 12-2 : projection d'un point P_i défini dans l'espace E à K dimensions sur l'axe Δ_i

4. L'ACP du point de vue analytique

Le premier objectif de l'ACP est de trouver le premier axe Δ_1 le plus proche du nuage de points étudié (voir la figure 12-2 ci-dessus). En d'autres termes, nous recherchons un axe Δ_1 tel que la quantité : $\frac{1}{n}\sum_{i=1}^{n} d(P_i, H_i)$ est maximale avec $d(P_i, H_i)$ la distance entre le point P_i et sa projection orthogonale sur l'axe Δ_1 et n le nombre de points dans le nuage.

© Editions ENI - All rights reserved

En appliquant le théorème de Pythagore, cela revient à maximiser la quantité :

$max\ \frac{1}{n} \sum_{i=1}^{n} d(0, H_i)^2$ (1)

car : $d(0, P_i)^2 = d(0, H_i)^2 + d(P_i, H_i)^2$

Pour maximiser la formule (1), il suffit de trouver un vecteur v_1 qui a la même direction que l'axe Δ_1 et qui maximise la formule suivante :

$max\ \frac{1}{n} \sum_{i=1}^{n} < OP_i, v_1 >^2$ (2)

Avec $< OP_i,\ v_i >$ le produit scalaire entre le vecteur associé au point P_i et le vecteur $\overrightarrow{v_1}$. Pour simplifier la formulation, nous allons noter OP_i par simplement P_i.

Sachant que : $< P_i,\ v_1 >^2 = {}^t v_1 P_i\ {}^t P_i v_1$ (3)

En remplaçant (3) dans (2), on obtient : $max \frac{1}{n} \sum_{i=1}^{n} {}^t v_1 P_i\ {}^t P_i v_1$ (4)

Le produit scalaire est symétrique, donc la formule (4) devient :

$max\ {}^t v_1 \left(\frac{1}{n} \sum_{i=1}^{n} P_i\ {}^t P_i \right) v_1$ (5)

Le terme $\frac{1}{n} \sum_{i=1}^{n} P_i\ {}^t P_i$ n'est rien d'autre que la matrice de variance/covariance S du nuage de points initial, ou la matrice des corrélations si le nuage de points est centré et réduit. Donc la formule (5) devient : $max\ {}^t v_1\, S\, v_1$ (6).

Le problème modélisé avec la formule (6) est un problème d'optimisation quadratique avec $v_1 = (v_1^1, v_1^2, v_1^3 \ldots v_1^p)$ les inconnues du problème. Ce problème admet une infinité de solutions puisque, pour tout vecteur v_1 maximisant la formule (6), nous pouvons trouver un autre vecteur $w_1 = a.v_1$ avec $a \in \mathbb{R}$ qui, lui aussi, maximise la formule (6). Pour n'avoir qu'une seule solution, il suffit de rajouter la contrainte $|v_1| = 1$. Cette contrainte permet d'obtenir le vecteur unitaire de l'axe recherché Δ_1 et le produit scalaire $P.v_1$ nous donne la projection orthogonale du point P sur l'axe Δ_1.

Donc notre problème (6) d'optimisation devient $\left\{ \begin{array}{c} max\ {}^t v_1 S\, v_1 \\ avec\ {}^t v_1 . v_1 = 1 \end{array} \right\}$ (7)

Pour résoudre ce problème, la méthode des multiplicateurs de Lagrange peut être appliquée et la formule (7) devient :

$$L(v_1^1, v_1^2, v_1^3 \ldots v_1^p) = {}^t v_1 S\, v_1 - \lambda_1({}^t v_1 . v_1 - 1)$$

Après avoir effectué toutes les dérivées matricielles partielles, nous obtenons :

$$S\, v_1 - \lambda_1 v_1 = 0$$

Donc : $S\, v_1 = \lambda_1 v_1$ (8)

La formule (8) montre que λ_1 n'est rien d'autre qu'une valeur propre de S et v_1 le vecteur propre de la matrice S associé à λ_1.

Si nous multiplions les deux termes de la formule (8) par ${}^t v_1$, nous obtenons ${}^t v_1\, S\, v_1 = \lambda_1$ (9).

Puisque ${}^t v_1\, \lambda_1 v_1 = \lambda_1\, {}^t v_1 v_1 = \lambda_1$

Le terme à gauche de la formule (9) n'est rien d'autre que la quantité que nous voulions maximiser. Donc, de (8) et (9), nous pouvons conclure que λ_1 correspond à la plus grande valeur propre de S et v_1 le vecteur propre qui lui est associé.

Une fois que Δ_1, défini par v_1 est trouvé, pour rechercher le deuxième axe principal Δ_2 orthogonal à Δ_1, il suffit de résoudre le problème suivant :

$$\left\{ \begin{array}{c} max\ {}^t v_2 S\, v_2 \\ avec\ {}^t v_2 . v_2 = 1 \\ et\ {}^t v_1 . v_2 = 0 \end{array} \right\}$$

La contrainte ${}^t v_1 . v_2 = 0$ exprime le fait que v_2 est orthogonal à v_1.

Comme pour la recherche de v_1, en utilisant la méthode des multiplicateurs de Lagrange, nous allons trouver un λ_2 qui correspond à la deuxième plus grande valeur propre de S et v_2 le vecteur propre de S associé à λ_2.

© Editions ENI - All rights reserved

En poursuivant les mêmes étapes pour trouver v_1 et v_2, nous pouvons rechercher tous les autres axes restants.

Les formules précédentes montrent que les meilleurs axes sur lesquels un nuage de points peut être projeté en le déformant le moins possible correspondent aux vecteurs propres associés à la matrice sous-jacente à ce nuage. Cependant, pour calculer les valeurs et les vecteurs propres de S, il est inutile de résoudre le problème d'optimisation vu plus haut. Il suffit de réaliser une décomposition singulière de la matrice X_j^i.

Une des propriétés les plus remarquables de l'ACP est la propriété de l'emboîtement des solutions. Cette propriété annonce que le plan défini par deux axes et qui porte le maximum de variance, par rapport à tout autre plan, contient obligatoirement les deux premiers axes qu'on aurait trouvés si nous les avions recherchés un par un avec la méthode présentée ci-dessus dans cette section !

5. Indicateurs de la qualité de la représentation des données

Les sections précédentes ont montré qu'une ACP est une projection d'un nuage de points qui permet d'avoir une vue simple, représentable en deux ou trois dimensions, et synthétique puisqu'elle résume les données étudiées. Cependant, l'ACP n'est pas une fin en soi, il faudrait donner un sens métier à ces résultats. Par exemple, à partir des résultats, nous pouvons constater des regroupements d'individus, ou encore de fortes liaisons, corrélations entre certaines variables. Il existe des indicateurs qui peuvent nous aider à l'interprétation des résultats d'une ACP. Comme nous allons le voir par la suite, ces indicateurs sont liés soit aux individus, soit aux variables.

5.1 Indicateurs liés aux individus

Une fois que le nuage de points est projeté dans un nouvel espace formé par les composantes principales, nous disposons de plusieurs métriques permettant de nous indiquer la qualité de la représentation des individus par cette projection.

Ainsi, les trois sections suivantes montrent comment calculer :

- le score d'un individu ;
- la contribution d'un individu à l'information apportée par une composante principale ;
- la qualité de la représentation d'un individu par une composante principale.

5.1.1 Score des individus

Le score d'un individu sur une composante principale est tout simplement sa coordonnée sur l'axe correspondant. De façon générale, le score d'un individu P_i sur la composante principale Δ_j est le produit scalaire $< \vec{P_i}, \vec{v_j} > = \sum_{k=1}^{p} P_i^k v_j^k$ avec v_j le $j^{ième}$ vecteur propre du nuage de points et qui est également le vecteur directeur de Δ_j. Le score d'un individu sur toutes les composantes principales peut être représenté par le vecteur suivant :

$$\vec{V} = (< \vec{P_i}, \vec{v_1} >, < \vec{P_i}, \vec{v_2} >, \ldots < \vec{P_i}, \vec{v_p} >)$$

5.1.2 Qualité de la représentation des individus

La qualité de la représentation d'un individu Pi par la composante principale j est calculée par la formule suivante $QLT_j(P_i) = \frac{\left(\sum_{k=1}^{p} P_i^k v_j^k\right)^2}{\sum_{j=1}^{p}\left(\sum_{k=1}^{p} P_i^k v_j^k\right)^2}$

$QLT_j(P_i)$ correspond au score de l'individu P_i sur la composante principale j au carré divisé par la somme des scores au carré de cet individu sur toutes les composantes principales. Plus cette valeur est élevée, mieux est représenté l'individu P_i sur l'axe j.

5.1.3 Contribution des individus

La contribution d'un individu P_i à la variance totale représentée par une composante principale j est donnée par la formule suivante $CTR_j(P_i) = \frac{{}^t v_j P_i \, {}^t P_i v_j}{n \lambda_j}$

© Editions ENI - All rights reserved

Avec λ_j la valeur propre associée à la composante principale j et v_j le vecteur propre associé à la valeur propre λ_j. Pour rappel, ${}^t v_j P_i \; {}^t P_i v_j$ est tout simplement la projection du point Pi sur le facteur j.

De façon intuitive, $CTR_j(P_i)$ nous donne la part de l'individu P_i dans la variance expliquée par la $j^{ième}$ composante principale.

5.2 Indicateurs liés aux variables

Tout comme pour les individus, nous disposons de plusieurs métriques permettant d'indiquer la qualité de la représentation des variables par l'ACP. Chacune des composantes principales peut être considérée comme une nouvelle variable qui vient s'ajouter aux anciennes, c'est-à-dire aux variables initiales. Étudier le rôle et l'influence des anciennes variables sur l'ACP revient à étudier leurs liaisons avec ces nouvelles variables.

5.2.1 Le cercle des corrélations

Le cercle des corrélations sert à illustrer graphiquement les liens entre les anciennes et les nouvelles variables. Pour obtenir un cercle des corrélations, il suffit de suivre les étapes ci-après :

1. Choisir deux axes principaux Δ_i et Δ_j.
2. Calculer les coefficients de corrélations entre les anciennes variables et les deux nouvelles variables F_i et F_j engendrées respectivement par les deux axes principaux Δ_i et Δ_j.
3. Tracer un cercle dont le diamètre est égal à 1.
4. Représenter les variables par des vecteurs dont les coordonnées sont les coefficients de corrélation calculés en point 2.

Dans l'exemple de la figure 12-3, les variables X_1, X_3 sont très fortement corrélées positivement avec l'axe Δ_i. La variable X_2 est corrélée avec l'axe Δ_i mais négativement.

La variable X_3 est très faiblement corrélée avec l'axe Δ_j tandis que X_1 et X_2 sont très fortement corrélées positivement avec cet axe. La variable X_4 est très faiblement corrélée avec les deux axes principaux Δ_i et Δ_j.

Toujours dans le même exemple, nous constatons que les deux variables X_1 et X_2 ne sont pas corrélées entre elles puisque les deux vecteurs les représentant dans le cercle des corrélations sont orthogonaux.

Remarque

Un coefficient de corrélation faible entre deux variables X_i et X_j permet uniquement de conclure que ces deux variables ne sont pas liées linéairement. En effet, même dans le cas où ces deux variables seraient liées par une liaison non linéaire, leur coefficient de corrélation peut être très faible.

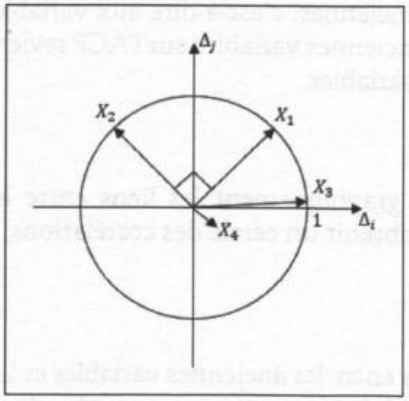


Figure 12-3 : exemple d'un cercle des corrélations entre les quatre variables X_1, X_2, X_3 et X_4 et deux composantes principales Δ_i et Δ_j.

5.2.2 Qualité de la représentation des variables

La qualité d'une représentation d'une variable par rapport à un axe Δ_i est tout simplement le carré du coefficient de corrélation entre cette variable X et la nouvelle variable artificielle F_i engendrée par Δ_i.

© Editions ENI - All rights reserved

5.2.3 Contribution des variables

La contribution d'une variable à un axe principal Δ_i est égale à la qualité de la représentation de cette variable X par rapport à l'axe Δ_i divisé par la valeur propre associée à ce même axe Δ_i.

6. Exemple d'ACP avec Python

Afin de mieux comprendre les aspects théoriques abordés dans ce chapitre, nous allons réaliser une ACP sur un exemple illustratif. Nous avons volontairement choisi un jeu de données pour lequel nous avons une bonne idée des résultats que nous devrions obtenir avec l'ACP.

Ce jeu de données est défini dans le fichier *notes_étudiants.csv* du dossier Data de ce chapitre et il est composé de 33 observations. Chaque observation correspond aux résultats d'un étudiant dans 10 matières différentes. Les individus qui composent notre nuage de points sont donc des étudiants, et les variables sont les matières suivantes : Mathématiques, Physique, Sciences naturelles, Histoire, Géographie, Philosophie, Français, Anglais, Allemand et Sport. Également, chaque étudiant est identifié de façon unique avec la variable `id`.

En réalité, nous avons construit manuellement les données de cet exemple afin de dissimuler intentionnellement un certain nombre d'informations et de voir si l'ACP permettrait de retrouver toutes ces informations à l'aide de graphiques tracés sur des plans à deux dimensions.

Ainsi, les données de cet exemple ont été construites de la manière suivante :

- Domaines des variables :
  - Toutes les variables sont définies dans l'intervalle `[2,20]` sauf la variable `Sport` qui est définie dans l'intervalle `[5,10]`.
- Les profils des étudiants : nous avons créé sept profils représentés avec les groupes `G1`, `G2`, `G3`, `G4`, `G5`, `G6` et `G7` définis ci-après.
  - Le profil formé par le groupe `G1:{id=4, id=6, id=12, id=28, id=32}`, composé de cinq étudiants qui ont de bonnes notes dans les six matières Mathématiques, Physique, Sciences naturelles, Histoire, Géographie et Philosophie. Ces notes sont comprises dans l'intervalle `[16,20]`.

Aussi, les étudiants de ce groupe ont des notes proches de la moyenne qui varie dans l'intervalle `[8,13]` dans les matières Français, Anglais, Allemand.

- Le profil formé par le groupe `G2:{id=1, id=2, id=22, id=24, id=27}`, composé de cinq étudiants qui ont de bonnes notes dans les trois matières Mathématiques, Physique et Sciences naturelles. Ces notes sont comprises dans l'intervalle `[16,20]`. Les étudiants de ce groupe ont des notes proches de la moyenne qui varie dans l'intervalle `[8,13]` dans les matières Histoire, Géographie, Philosophie, Français, Anglais et Allemand.
- Le profil formé par le groupe `G3:{id=5, id=9, id=16, id=25, id=33}`, composé de cinq étudiants qui ont de bonnes notes dans les six matières Mathématiques, Physique, Sciences naturelles, Français, Anglais et Allemand. Ces notes sont comprises dans l'intervalle `[16,20]`. Les étudiants de ce groupe ont des notes proches de la moyenne qui varie dans l'intervalle `[8,13]` dans les matières Mathématiques, Histoire, Géographie et Philosophie.
- Le profil formé par le groupe `G4:{id=7, id=10, id=11, id=13, id=14, id=21, id=29, id=31}`, composé de huit étudiants qui ont de bonnes notes dans les trois matières Histoire, Géographie et Philosophie. Ces notes sont comprises dans l'intervalle `[16,20]`. Les étudiants de ce groupe ont des notes proches de la moyenne qui varie dans l'intervalle `[8,13]` dans les matières Mathématiques, Physique, Sciences naturelles, Français, Anglais et Allemand.
- Le profil formé par le groupe `G5:{id=3, id=8, id=15, id=18, id=19, id=23, id=26, id=30}`, composé de huit étudiants qui ont de bonnes notes dans les trois matières Français, Anglais et Allemand. Ces notes sont comprises dans l'intervalle `[16,20]`. Les étudiants de ce groupe ont des notes proches de la moyenne qui varie dans l'intervalle `[8,13]` dans les matières Mathématiques, Physique, Sciences naturelles, Histoire, Géographie et Philosophie.
- Le profil formé par le groupe `G6:{id=20}`, composé d'un seul étudiant pour lequel nous avons attribué des notes faibles qui varient dans l'intervalle `[3,7]`.

© Editions ENI - All rights reserved

- Le profil formé par le groupe `G7:{id=17}`, composé d'un seul étudiant que nous allons considérer comme l'étudiant le plus fort puisque nous lui avons attribué la meilleure note 10 dans la matière Sport et la meilleure note 20 dans toutes les autres matières.

Remarque

Les notes pour la variable `Sport` ont été attribuées de manière complètement arbitraire indépendamment des sept groupes ci-dessus.

- Corrélation des variables :
  - Les variables associées aux trois matières Mathématiques, Physique et Sciences naturelles ont été volontairement corrélées positivement, c'est-à-dire que plus un étudiant a de bonnes notes dans l'une de ces trois matières, plus il a de bonnes notes dans les deux autres matières.
  - Les variables associées aux trois matières Histoire, Géographie et Philosophie ont été volontairement corrélées positivement.
  - Les variables associées aux trois matières Français, Anglais et Allemand ont été volontairement corrélées positivement.
  - La variable associée à la matière Sport n'est corrélée avec aucune des neuf autres variables et elle est, pour rappel, attribuée aux étudiants de manière aléatoire.

En résumé, nous avons un nuage de points défini dans un espace à dix dimensions dans lequel nous avons codifié sept profils d'étudiants, à savoir `G1`, `G2`, `G3`, `G4`, `G5`, `G6` et `G7` et quatre profils de variables, à savoir `M1`, `M2`, `M3` et `Sport`. En nous appuyant uniquement sur des graphiques à deux dimensions réalisés directement sur les données du fichier *notes_étudiants.csv*, nous aurions beaucoup de mal à extraire toutes les informations qui y sont cachées.

Alors, l'ACP va-t-elle nous permettre de débusquer toutes ces informations en un clin d'œil sur de simples graphiques en deux dimensions ? C'est ce que nous allons découvrir dans la suite de cet exemple.

Remarque

Avant d'aller plus loin, insistons sur le fait que les données de cet exemple ont été soigneusement créées afin d'obtenir des résultats illustratifs dans le but de comprendre l'intérêt de l'ACP. Dans la pratique, l'ACP ne parviendrait pas forcément à extraire les informations qui seraient dissimulées dans un jeu de données. De plus, l'intérêt de l'ACP est d'autant plus important que la taille des données est grande, mais nous avons volontairement choisi d'utiliser un jeu de données de petite taille afin de simplifier l'analyse et la compréhension des notions abordées.

- Pour appliquer une ACP, commencez par importer les bibliothèques nécessaires comme le montre le code suivant ; vous pouvez utiliser le code contenu dans le notebook *acp.ipynb* du dossier *Code* de ce chapitre :

```
%matplotlib inline
import pandas as pd
from sklearn.preprocessing import StandardScaler
import numpy as np
from matplotlib import pyplot as plt
from sklearn.decomposition import PCA

data = pd.read_csv("../Data/notes_étudiants.csv", sep=',',
index_col='id' )
n = data.shape[0]
p = data.shape[1]
```

Remarquez l'import du module `sklearn.decomposition` où se trouve le type `PCA` qui va nous servir pour instancier l'algorithme permettant l'application d'une ACP sur un jeu de données.

Ce code permet de lire les données du fichier *notes_étudiants.csv* dans le DataFrame `data`. Nous avons indiqué lors de la lecture de ces données que la variable `id` doit être utilisée en tant qu'index. Cela aura également comme effet l'exclusion de cette variable de la liste des colonnes du DataFrame `data`.

Les deux variables `n` et `p` stockent respectivement le nombre d'observations et le nombre de colonnes du DataFrame `data`. Donc, à la fin de l'exécution de ce bout de code, la variable `n` sera égale à `33` et la variable `p` sera égale `10`, puisque pour rappel la variable `id` n'est plus considérée en tant que colonne !

© Editions ENI - All rights reserved

Avant d'appliquer une ACP, il est primordial de procéder au centrage et à la réduction des données, puisque la variable `Sport` n'est pas définie dans le même intervalle que les neuf autres variables. Les deux instructions suivantes permettent de mettre à l'échelle toutes les observations et de stocker le résultat dans le nouveau DataFrame `scaled_data`.

```
s_scaler = StandardScaler()
scaled_data = s_scaler.fit_transform(data)
```

Nous pouvons vérifier que les nouvelles données sont mises à l'échelle en affichant les moyennes des dix variables avec l'instruction suivante :

```
print(np.mean(scaled_data,axis=0))
```

Les moyennes de nos dix variables sont les suivantes :

```
[ 1.31208176e-16  4.37360585e-17 -1.07657990e-16  9.08364293e-17
-1.95130107e-16 -1.48029737e-16 -9.42007415e-17  1.61486985e-16
-1.68215610e-16  8.99953512e-17]
```

Comme attendu, ces dix moyennes sont très proches de la valeur 0. Également, l'écart-type de chacune des dix variables de ce nouveau jeu de données est égal à l'unité, comme nous pouvons le vérifier avec l'instruction suivante :

```
print(np.std(scaled_data,axis=0,ddof=0))
```

Cette instruction affiche le tableau ci-dessous dont les valeurs correspondent respectivement aux écarts-types des dix variables du DataFrame `scaled_data`.

```
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

À présent, le moment est venu d'appliquer l'ACP sur les données mises à l'échelle ! Les deux instructions suivantes appliquent la procédure de l'ACP via l'objet `pca` de type `PCA` sur les données `scaled_data` :

```
pca = PCA()
coords = pca.fit_transform(scaled_data)
```

La fonction `fit_transform` retourne les coordonnées de chaque individu analysé dans les nouveaux axes créés par l'ACP ! Pour rappel, le nombre de ces axes serait égal au nombre d'axes d'origine, c'est-à-dire que, dans notre exemple, le tableau `coords` aura les mêmes dimensions que les données `scaled_data`.

La différence entre `coords` et `scaled_data` réside dans le fait que les deux ou les trois premières variables de `coords` vont porter plus d'informations sur les données analysées que les informations portées par les deux ou trois premières variables de `scaled_data` ; en tout cas, c'est ce que nous espérons avec l'application de l'ACP.

6.1 Déterminer le nombre de facteurs pertinents

Après avoir réalisé une ACP, il existe plusieurs méthodes permettant de décider du nombre de facteurs ou du nombre d'axes à utiliser pour l'interprétation des résultats. Dans cette section, nous allons analyser certains graphiques pour nous aider à nous fixer les idées sur le nombre d'axes à retenir pour l'analyse de nos données. Également, nous allons utiliser par la suite la méthode du «test des bâtons brisés» de Legendre & Legendre (1983) pour nous aider dans ce choix du nombre d'axes à retenir.

L'instruction suivante affiche le taux de l'inertie totale expliquée par chacun des dix axes construits avec l'application de l'ACP.

```
print(pca.explained_variance_ratio_)
```

L'attribut `explained_variance_ratio_` correspond aux taux des inerties expliquées par les différents axes. Les résultats sont les suivants :

```
[0.39895152 0.29268582 0.16848099 0.0951434  0.01518182
0.00992171 0.00778928 0.00633025 0.0030013  0.00251392]
```

Ces résultats montrent que le premier axe explique à lui tout seul 39.89 % de l'inertie totale, le deuxième axe explique à lui tout seul 29.26 % de l'inertie totale, et ainsi de suite.

Ces résultats illustrent très bien l'idée de l'ACP qui consiste à concentrer un maximum d'inertie sur les premiers axes. Le premier axe concentre un maximum d'inertie, puis, sur le total de l'inertie restante, l'ACP récupère un maximum dans le deuxième axe, et ainsi de suite.

Remarquez que les cinq derniers axes portent des taux d'inertie quasiment proches de zéro !

© Editions ENI - All rights reserved

Les deux instructions suivantes calculent et affichent les dix valeurs propres associées aux dix axes de l'ACP :

```
eigenvalues = (n-1)/n*pca.explained_variance_
print(eigenvalues)
```

L'attribut `explained_variance_` correspond à la quantité de l'inertie expliquée par les dix axes de l'ACP. Nous avons multiplié ces quantités par $\frac{(n-1)}{n}$ afin d'apporter un ajustement pour que ces quantités correspondent exactement aux valeurs propres associées aux dix axes de l'ACP.

Ces valeurs propres sont les suivantes :

```
[3.98951516 2.92685816 1.68480992 0.95143398 0.15181819
0.09921712 0.07789282 0.06330248 0.03001295 0.02513922]
```

Avec le code suivant, nous allons tracer un premier graphique où les quantités d'inertie expliquées seront affichées en fonction des axes.

```
fig, axes = plt.subplots(figsize=(8,8))
axes.set_xticks(np.arange(11))
axes.plot(range(1,p+1),eigenvalues)
plt.title("Les valeurs propres en fonction des facteurs")
```

Ce code affiche la courbe ci-dessous :

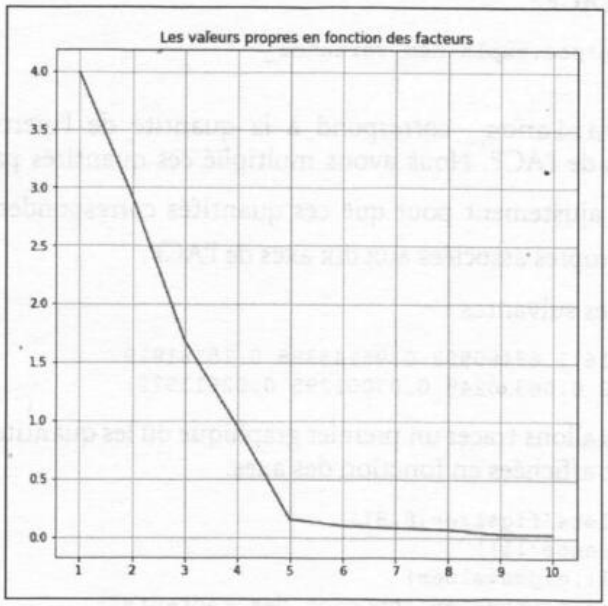


Figure 12-4 : représentation des valeurs propres en fonction des axes de l'ACP

Cette courbe est intéressante puisqu'elle permet de renforcer notre compréhension de l'ACP. Elle montre clairement que la quantité de l'inertie expliquée est nettement plus importante sur les premiers axes que sur les derniers.

Cette courbe montre qu'à partir du cinquième axe la courbe forme un plateau dont la coordonnée est quasiment nulle !

Une courbe encore plus intéressante que la précédente est celle où nous pouvons visualiser les cumuls des taux d'inertie expliquée. Le code suivant permet de tracer cette courbe :

```
fig, axes = plt.subplots(figsize=(8,8))
axes.set_xticks(np.arange(11))
axes.plot(np.arange(1,p+1),np.cumsum(pca.explained_variance_ratio
                                     _))
plt.title("Cumul de la variance expliquée vs nombre de
          facteurs")
```

© Editions ENI - All rights reserved

Ce code affiche le graphique ci-dessous :

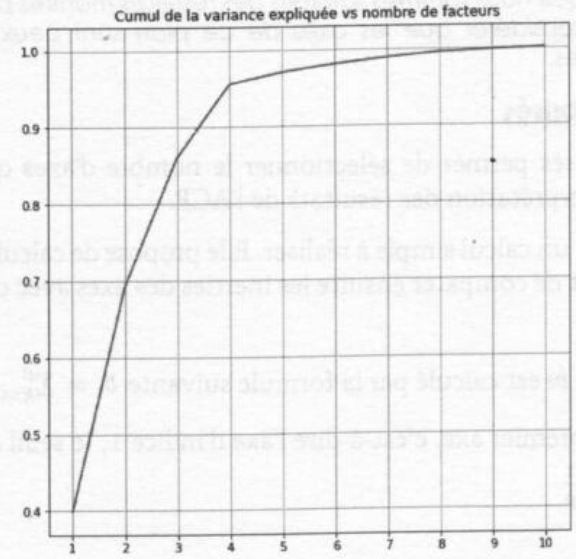


Figure 12-5 : représentation des cumuls de l'inertie expliquée par les plans formés par l'emboîtement des axes en commençant par les premiers axes et en allant vers les derniers axes

Cette courbe permet d'affirmer visuellement que le plan formé par les deux premiers axes explique environ 70 % de l'inertie totale et que l'hyperplan formé par les trois premiers axes explique à lui seul un peu plus de 85 % de l'inertie totale !

Cette courbe montre que le choix d'utiliser les deux premiers axes pour l'analyse et l'interprétation des résultats de l'ACP est déjà intéressant !

Bien sûr, le choix du nombre d'axes à considérer pour l'interprétation des résultats d'une ACP dépend du sens que nous allons donner aux informations portées par ces axes ! Dans notre exemple, la question à nous poser est de savoir si les 70 % d'inertie portée par le plan formé avec les deux premiers axes sont suffisants ou non ?

Remarque

De manière générale, lorsque nous sommes satisfaits des résultats montrés par un plan, nous pouvons considérer que les axes de ce plan sont ceux à conserver pour nos analyses.

La méthode des bâtons brisés

La méthode des bâtons brisés permet de sélectionner le nombre d'axes qui serait intéressant pour l'interprétation des résultats de l'ACP.

Cette méthode est basée sur un calcul simple à réaliser. Elle propose de calculer un seuil théorique par axe et de comparer ensuite les inerties des axes avec ces seuils.

Pour un axe *i* donné, le seuil *b*i est calculé par la formule suivante $b_i = \sum_{k=i}^{p} \frac{1}{k}$.

Ainsi, par exemple pour le premier axe, c'est-à-dire l'axe d'indice 1, le seuil *b*1 sera égal à $\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{p}$

Avec p le nombre d'axes.

On voit que la formule de calcul de ces seuils ne tient compte que du nombre d'axes et non des valeurs des variables d'origine ni des valeurs correspondant aux coordonnées des individus dans les nouveaux axes formés par l'ACP.

Avec cette méthode, pour qu'un axe soit retenu, il faudrait que la quantité d'inertie portée par cet axe soit strictement supérieure à la valeur théorique qui lui est associée. En d'autres termes, pour qu'un axe *i* soit retenu, il faudrait que $\lambda_i > b_i$.

Dans notre exemple, pour que le premier axe soit retenu, il doit valider l'inéquation suivante :

$$\lambda_1 > b_1 \Leftrightarrow 3.98951516 > \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} + \frac{1}{9} + \frac{1}{10}$$

$$\Leftrightarrow 3.98951516 > 2.928968$$

Donc le premier axe est retenu !

© Editions ENI - All rights reserved

Le code suivant permet de calculer les seuils théoriques pour tous les axes :

```
b_i = 1/np.arange(p,0,-1)
cumul_b_i = np.cumsum(b_i)
cumul_b_i = cumul_b_i[::-1]
print(pd.DataFrame({'Val.Propre':eigenvalues,'Seuils':cumul_b_i}))
```

La dernière instruction affiche tout simplement côte à côte les inerties des axes et leurs seuils théoriques. Le résultat est le suivant :

```
    Inertie    Seuils
0  3.989515  2.928968
1  2.926858  1.928968
2  1.684810  1.428968
3  0.951434  1.095635
4  0.151818  0.845635
5  0.099217  0.645635
6  0.077893  0.478968
7  0.063302  0.336111
8  0.030013  0.211111
9  0.025139  0.100000
```

Ces résultats montrent que l'inéquation $\lambda_i > b_i$ est validée par les trois premiers axes, mais pas les autres.

Avec la méthode des bâtons brisés, il n'y a donc que les trois premiers axes qui seraient intéressants à prendre en compte pour l'interprétation des résultats de l'ACP de notre exemple.

6.2 Interprétation des résultats sur les individus

Pour interpréter les résultats de l'ACP de notre exemple par rapport aux individus analysés, nous allons représenter graphiquement tous les étudiants en procédant à leur projection sur les deux premiers axes, puis nous allons mesurer la qualité de la représentation de chaque individu sur chacun des axes ainsi que leur contribution à ces axes.

6.2.1 Représentation des individus

Nous allons commencer par la représentation de tous les étudiants dans un plan formé par les deux premiers axes. Pour rappel, le plan formé par ces deux premiers axes couvre à lui seul environ 70 % de l'inertie totale présente dans les données d'origine.

Le code suivant trace un graphique pour visualiser les 33 étudiants suivant leurs coordonnées sur les deux premiers axes de l'ACP :

```
fig, axes = plt.subplots(figsize=(8,8))
axes.set_xlim(-3,4)
axes.set_ylim(-4,6)
plt.plot([-3,4],[0,0],color='black',linewidth=1)
plt.plot([0,0],[-4,6],color='black',linewidth=1)

for i in range(n):
    plt.annotate(data.index[i],(coords[i,0],coords[i,1]))
```

Pour rappel, `coords` est le tableau à deux dimensions dans lequel nous avons stocké les résultats de la fonction `fit_transform` lorsque nous avons appliqué l'ACP. Les lignes de ce tableau correspondent aux étudiants et les colonnes correspondent aux coordonnées de ces étudiants sur les nouveaux axes créés par l'ACP.

Pour ce graphique, ce code utilise les deux premières colonnes de `coords`, à savoir `coords[i,0]` et `coords[i,1]` et qui correspondent donc à la projection de tous les étudiants sur les deux premiers axes. La fonction `annotate` permet de renseigner sur le graphique la valeur de l'`id` de chaque étudiant afin de connaître l'emplacement des étudiants sur ce graphique.

© Editions ENI - All rights reserved

La figure ci-dessous montre le résultat de ce code :

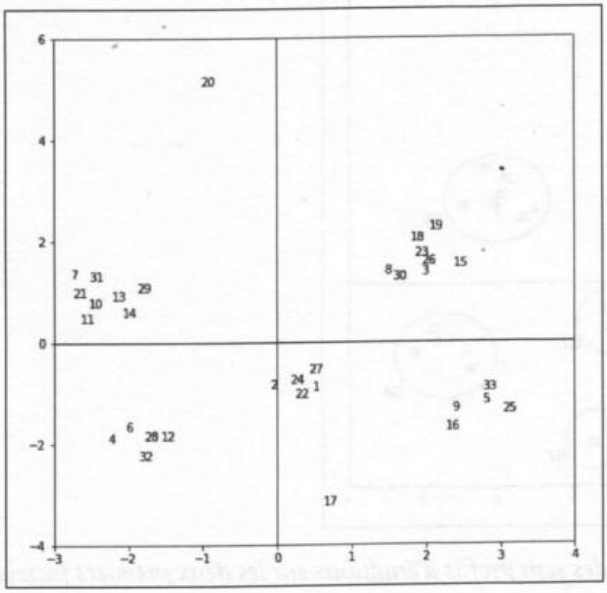


Figure 12-6 : représentation des 33 étudiants sur les deux premiers facteurs

Ce graphique montre clairement que sur ce plan les étudiants sont répartis dans sept groupes. Quelle bonne surprise !

Comme le montre la figure ci-dessous, ces sept groupes d'étudiants correspondent exactement aux sept profils d'étudiants que nous avions cachés intentionnellement dans le jeu de données initial.

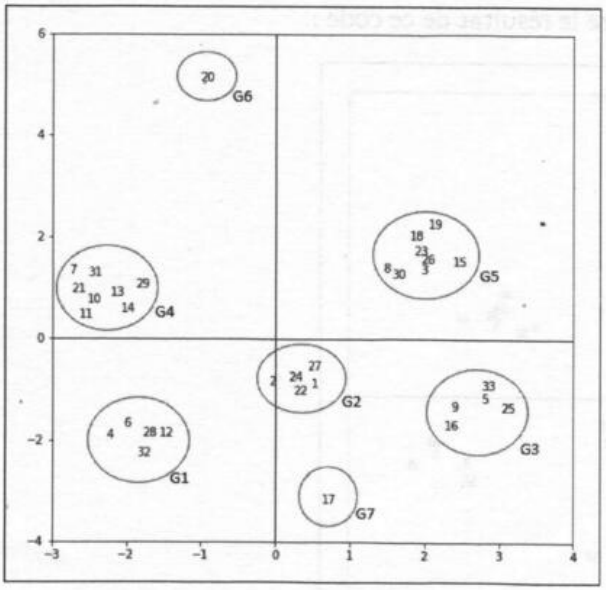


Figure 12-7 : représentation des sept profils d'étudiants sur les deux premiers facteurs

Sur le graphique 12-7, nous remarquons que les étudiants des deux groupes `G3` et `G5` ont des scores ou des coordonnées élevés sur le premier axe, qui est l'axe horizontal ; en revanche, ces deux mêmes groupes ont des coordonnées relativement opposées sur le deuxième axe, qui est l'axe vertical sur ce graphique.

Un peu de la même manière que pour les deux groupes `G3` et `G5`, les groupes `G1` et `G4` ont des valeurs négatives et proches les unes des autres sur le premier axe ; en revanche, ces deux groupes sont également opposés sur le deuxième axe.

Les étudiants du groupe 2 ont des coordonnées relativement proches de zéro sur les deux premiers axes !

L'étudiant avec l'`id=17` est celui que nous avons qualifié d'étudiant le plus fort ! Sur ces deux premiers axes, cet étudiant est mis un peu à l'écart de tous les autres, mais avec une coordonnée relativement proche de zéro sur le premier axe et une coordonnée relativement extrême sur le deuxième axe.

© Editions ENI - All rights reserved

Le fait que cet étudiant soit mis à l'écart des autres nous surprend peu puisque nous savons qu'il a des notes maximales dans toutes les matières, sauf la matière Sport.

De façon un peu plus extrême que l'étudiant avec `id=17`, l'étudiant avec l'`id=20` est également mis à l'écart de tous les autres étudiants. Nous remarquons que cet étudiant est placé de manière plus ou moins opposée à l'étudiant avec l'`id=17` sur ces deux premiers axes. Ici aussi nous ne sommes pas surpris, puisque nous savons que cet étudiant a les notes les plus faibles dans toutes les matières, sauf la matière Sport.

Avec ces résultats, nous pouvons affirmer que les profils que nous avons cachés manuellement dans les données ont été capturés par l'ACP !

Grâce à l'ACP, nous avons pu visualiser sur un plan à deux dimensions des données qui étaient définies dans un espace à dix dimensions !

Si vous utilisez les données initiales, c'est-à-dire les données du fichier *notes_étudiants.csv*, et si vous souhaitez réaliser une représentation graphique sur un plan à deux dimensions de tous ces étudiants, alors quelles que soient les deux variables, parmi ces dix variables initiales, que vous auriez choisies pour cette représentation, vous n'auriez jamais pu obtenir une répartition en sept groupes comme la répartition de la figure 12-7.

En effet, si nous utilisons les données initiales, quelles que soient les deux variables sélectionnées pour une représentation graphique sur un plan à deux dimensions, les informations qui sont renseignées dans les autres variables non sélectionnées ne seront pas représentées dans ce graphique ! C'est justement dans ce dernier constat que l'ACP trouve tout son intérêt, car elle permet de concentrer un maximum d'informations sur les premiers axes.

La figure 12-7 nous a permis de visualiser les 33 étudiants sur le plan formé par les deux premiers axes. Bien sûr, les résultats de cette représentation graphique sont satisfaisants puisqu'ils correspondent aux résultats attendus. Dans la pratique, il est souvent nécessaire de réaliser d'autres représentations graphiques sur des plans à deux dimensions en utilisant différents axes que les deux premiers. Par exemple, certaines informations cachées dans les données initiales peuvent se révéler à nous sur un plan qui serait formé avec le premier et le troisième axe.

6.2.2 Calcul de la qualité de la représentation des individus

Nous allons procéder au calcul de la qualité de la représentation de chaque étudiant sur chacun des axes. Le code suivant permet de réaliser ces calculs en utilisant la formule vue plus haut dans ce chapitre lorsque nous avons abordé la notion du score des individus.

```
total_dist = np.sum(scaled_data **2,axis=1)
cos2 = coords**2
for j in range(p):
    cos2[:,j] = cos2[:,j]/total_dist

print(pd.DataFrame({'id':data.index,'COS2_1':cos2[:,0],
'COS2_2':cos2[:,1], 'COS2_3':cos2[:,2]}))
```

Ce code calcule la somme de l'inertie totale des données initiales `scaled_data` et stocke le résultat dans la variable `total_dist`. Puis le tableau à deux dimensions `cos2` enregistre le rapport entre le carré de chaque coordonnée sur les nouveaux axes divisé sur l'inertie totale. Au final, les lignes de `cos2` correspondent aux étudiants et chaque colonne correspond aux qualités de la représentation de chaque étudiant sur l'axe associé à cette colonne.

Le code précédent affiche les cinq premières lignes et les trois premières colonnes de `cos2`.

```
    Id    COS2_1    COS2_2    COS2_3
0    1  0.041582  0.162667  0.636153
1    2  0.001479  0.152025  0.610155
2    3  0.296143  0.129340  0.008793
3    4  0.496391  0.377519  0.066270
4    5  0.633967  0.129396  0.043465
...
```

On voit que l'étudiant avec l'`id=5` est mieux représenté sur le premier axe que sur les deux autres axes, alors que l'étudiant avec l'`id=1` est beaucoup mieux représenté sur le troisième axe !

Si vous analysez ces résultats par rapport aux étudiants du groupe `G2`, c'est-à-dire les étudiants avec `id` égal à une des valeurs `1`, `2`, `22`, `24` ou `27`, vous allez observer que la qualité de leur représentation est beaucoup plus élevée sur le troisième axe !

© Editions ENI - All rights reserved

Peut-on supposer que le troisième axe représente mieux le profil scientifique associé aux bonnes notes dans les trois matières Mathématiques, Physique et Sciences naturelles ? Nous allons apporter une réponse claire à cette question avec l'analyse des résultats du point de vue des variables.

Si nous faisons la somme de chaque ligne de `cos2` (voir l'instruction suivante), on obtient la valeur 1 à chaque fois :

```
print(np.sum(cos2,axis=1))
```

La somme des lignes de `cos2` donne les résultats suivants :

```
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
1. 1. 1. 1. 1. 1.]
```

Cela prouve que l'inertie totale portée par les variables initiales est complètement récupérée dans les nouveaux axes de l'ACP !

6.2.3 Calcul de la contribution des individus

Pour chaque étudiant, le code suivant calcule sa contribution dans le calcul des nouveaux axes.

```
ctr = coords**2
for j in range(p):
    ctr[:,j] = ctr[:,j]/(n*eigenvalues[j])

print(pd.DataFrame({'id':data.index,'CTR_1':ctr[:,0],
'CTR_2':ctr[:,1],'CTR_3':ctr[:,2]}))
```

Le carré de chaque coordonnée dans les nouveaux axes est renseigné dans le tableau `ctr`, puis chaque élément de ce tableau est divisé par $n.\lambda$.

```
   id     CTR_1     CTR_2     CTR_3
0   1  0.001779  0.009484  0.064431
1   2  0.000063  0.008791  0.061296
2   3  0.028793  0.017141  0.002025
3   4  0.039229  0.040667  0.012401
4   5  0.057563  0.016015  0.009345
...
```

6.3 Interprétation des résultats sur les variables

L'interprétation des résultats d'une ACP est basée sur le degré de corrélation entre les variables initiales et les nouvelles variables associées aux nouveaux axes créés par la procédure de l'ACP.

6.3.1 Tracer un cercle des corrélations

Pour tracer un cercle des corrélations, nous avons besoin de calculer les niveaux de corrélation des dix variables initiales avec les dix variables engendrées par les axes de l'ACP. Le code suivant permet justement de construire une matrice dans laquelle les lignes représentent les variables initiales et les colonnes les nouvelles variables associées aux axes de l'ACP. Ainsi, chaque élément de cette matrice correspond au degré de corrélation entre une des variables initiales et une des nouvelles variables.

```
sqrt_eigenvalues = np.sqrt(eigenvalues)
corvar = np.zeros((p,p))
for i in range(p):
    corvar[:,i] = pca.components_[i,:] * sqrt_eigenvalues[i]
```

L'attribut `components_` est un tableau dans lequel chaque ligne est un vecteur propre. Chaque vecteur propre est associé à un et un seul axe de l'ACP. L'instruction suivante permet d'afficher les niveaux de corrélation entre nos dix variables et les trois premières variables artificielles associées aux trois premiers axes de l'ACP.

```
print(pd.DataFrame({'id':data.columns,'COR_1':corvar[:,0],'COR_2'
:corvar[:,1], 'COR_3':corvar[:,2]}))
```

Le résultat est le suivant :

```
                  id     COR_1     COR_2     COR_3
0       Mathématiques  0.191638 -0.913991  0.283831
1            Physique  0.253727 -0.906291  0.230384
2  Sciences_naturelles  0.157963 -0.939141  0.119398
3            Histoire -0.785478 -0.337560 -0.454168
4          Géographie -0.791712 -0.285526 -0.464489
5         Philosophie -0.689147 -0.401413 -0.465376
6             Français  0.875155 -0.038146 -0.442291
7              Anglais  0.758663 -0.040075 -0.626080
8             Allemand  0.845973 -0.163645 -0.474515
9                Sport -0.296001  0.041052 -0.292510
```

© Editions ENI - All rights reserved

Pour interpréter facilement cette matrice de corrélation, le code suivant trace le cercle des corrélations entre les dix variables initiales et les deux premiers axes de l'ACP.

```
fig, axes = plt.subplots(figsize=(8,8))
axes.set_xlim(-1,1)
axes.set_ylim(-1,1)
for j in range(p):

plt.annotate(data.columns.get_loc(data.columns[j]),(corvar[j,0],
corvar[j,1]))

plt.plot([-1,1],[0,0],color='black',linewidth=1)
plt.plot([0,0],[-1,1],color='black',linewidth=1)

circle = plt.Circle((0,0),1,color='blue',fill=False)
axes.add_artist(circle)
plt.title("Cercle des corrélations")
```

Le résultat est le suivant :

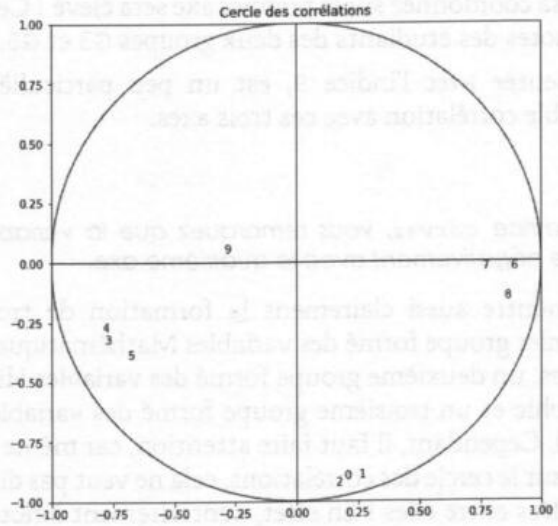


Figure 12-8 : cercle des corrélations des dix variables avec les deux premiers facteurs

Ces résultats montrent sans ambiguïté que :

- Les matières Mathématiques, Physique et Sciences naturelles, représentées avec les indices `0`, `1` et `2`, sont fortement corrélées négativement avec le deuxième axe. Maintenant, nous comprenons mieux pourquoi dans la figure 12-7 les trois groupes `G1`, `G2` et `G3`, composés d'étudiants forts dans ces trois matières, ont été représentés sous la valeur `0` du deuxième axe.
- Les trois matières Histoire, Géographie et Philosophie, représentées avec les indices `3`, `4` et `5`, sont fortement corrélées avec le premier axe. Cela signifie que plus un étudiant a de meilleures notes dans l'une de ces trois matières, plus son score ou sa coordonnée sur le premier axe sera plus faible avec une valeur négative ; c'est-à-dire que sa note sera plus à gauche sur la figure 12-7. Cela se vérifie en observant les notes des étudiants des deux groupes `G1` et `G4`.
- Les trois matières Français, Anglais et Allemand, représentées avec les indices `6`, `7` et `8`, sont fortement corrélées positivement avec le premier axe ; c'est-à-dire que plus un étudiant a de meilleures notes dans l'une de ces trois matières, plus son score ou sa coordonnée sur le premier axe sera élevé ! Cela se vérifie en observant les notes des étudiants des deux groupes `G3` et `G5`.
- La variable `Sport`, représentée avec l'indice `9`, est un peu particulière puisqu'elle présente une faible corrélation avec ces trois axes.

Remarque

Si vous affichez toute la matrice `corvar`, vous remarquez que la variable `Sport` est fortement corrélée négativement avec le quatrième axe.

Ce cercle des corrélations montre aussi clairement la formation de trois groupes de variables : un premier groupe formé des variables Mathématiques, Physique et Sciences naturelles, un deuxième groupe formé des variables Histoire, Géographie, et Philosophie et un troisième groupe formé des variables Français, Anglais et Allemand. Cependant, il faut faire attention, car même si des variables sont regroupées sur le cercle des corrélations, cela ne veut pas dire qu'elles sont réellement corrélées entre elles ! En effet, contrairement à notre exemple où les variables proches sur le cercle des corrélations sont réellement corrélées entre elles, dans la réalité, deux variables peuvent être très proches entre elles sur le cercle des corrélations sans pour autant qu'il y ait une quelconque relation entre elles dans la réalité.

© Editions ENI - All rights reserved

6.3.2 Calcul de la qualité de la représentation des variables

Le code ci-après calcule la qualité de la représentation des variables sur les dix axes, puis affiche les résultats obtenus sur les quatre premiers axes :

```
cos2_var = corvar**2
print(pd.DataFrame({'id':data.columns,'COS2_1':cos2_var[:,0],
'COS2_2':cos2_var[:,1], 'COS2_3':cos2_var[:,2]}))
```

Comme le montre ce code, la qualité de la représentation d'une variable se calcule directement par le carré des corrélations de cette variable avec chacun des axes de l'ACP !

Le résultat est le suivant :

```
                id    COS2_1    COS2_2    COS2_3    COS2_4
0      Mathématiques  0.036725  0.835379  0.080560  0.000075
1            Physique  0.064377  0.821363  0.053077  0.008473
2  Sciences_naturelles  0.024952  0.881986  0.014256  0.014912
3           Histoire  0.616976  0.113947  0.206268  0.000721
4         Géographie  0.626808  0.081525  0.215750  0.026472
5         Philosophie  0.474923  0.161132  0.216575  0.065122
6           Français  0.765897  0.001455  0.195621  0.005378
7            Anglais  0.575570  0.001606  0.391976  0.004797
8           Allemand  0.715670  0.026780  0.225165  0.006523
9              Sport  0.087617  0.001685  0.085562  0.818960
```

Les valeurs ci-dessus correspondent bien à ce que nous avons observé sur le cercle des corrélations, c'est-à-dire que les trois premières matières sont très bien représentées sur le deuxième axe, tandis que les six matières suivantes sont bien représentées sur le premier axe, et enfin la matière Sport n'est pas bien représentée sur ces trois premiers axes ; en revanche, elle est très bien représentée sur le quatrième axe, c'est d'ailleurs la seule à être bien représentée sur ce dernier axe !

6.3.3 Calcul des contributions des variables

Le code suivant permet de calculer la contribution des variables initiales à la construction des axes de l'ACP :

```
ctrvar = cos2_var
for i in range(p):
    ctrvar[:,i] = ctrvar[:,i]/eigenvalues[i]
print(pd.DataFrame({'id':data.columns,'CTR_1':ctrvar[:,0],'CTR_2'
:ctrvar[:,1],'CTR_3':ctrvar[:,2]}))
```

Le calcul de la contribution d'une variable sur un axe donné est obtenu par la division entre la qualité de la représentation de cette variable sur cet axe et la valeur propre associée à cet axe.

Les contributions des variables initiales sur les quatre premiers axes sont affichées ci-dessous :

```
                 id     CTR_1     CTR_2     CTR_3     CTR_4
0       Mathématiques  0.009205  0.285418  0.047816  0.000079
1             Physique  0.016137  0.280630  0.031503  0.008905
2  Sciences_naturelles  0.006254  0.301342  0.008461  0.015673
3             Histoire  0.154649  0.038931  0.122428  0.000758
4           Géographie  0.157114  0.027854  0.128056  0.027823
5          Philosophie  0.119043  0.055053  0.128545  0.068447
6              Français  0.191977  0.000497  0.116109  0.005653
7               Anglais  0.144271  0.000549  0.232653  0.005042
8              Allemand  0.179388  0.009150  0.133644  0.006856
9                 Sport  0.021962  0.000576  0.050784  0.860764
```

Nous pouvons retenir de ces résultats que les contributions et les qualités de représentation des variables par rapport aux axes d'une ACP ne sont pas forcément corrélées. Par exemple, la matière Anglais a une meilleure qualité de représentation sur le premier axe que sur le troisième, tandis que cette même matière contribue plus à la construction du troisième axe qu'à celle du premier axe.

Remarquez que la matière Sport a une contribution très importante au quatrième axe !

© Editions ENI - All rights reserved

6.4 Représentation de nouveaux individus

Une fois qu'une ACP est réalisée sur un jeu de données, nous pouvons projeter de nouvelles données sur les axes de l'ACP.

Pour montrer comment représenter de nouvelles données en fonction des facteurs d'une ACP, le code suivant lit les notes de trois étudiants qui ont les identifiants `id=34`, `id=35` et `id=36` à partir du fichier *notes_nouveaux_étudiants.csv*, puis il procède à leur normalisation et leur projection sur l'hyperplan constitué des dix axes de notre ACP. Cette projection se réalise en appliquant tout simplement la fonction `transform` de l'objet `pca` qui, pour rappel, nous a servi pour la réalisation de l'ACP au début de cet exemple.

```
new_data = pd.read_csv("../Data/notes_nouveaux_étudiants.csv",
sep=',', index_col='id' )
Z_new_data = s_scaler.transform(new_data)
coords_new_data = pca.transform(Z_new_data)
```

Les coordonnées sur les axes de l'ACP des nouvelles données sont enregistrées dans le tableau `coords_new_data`. Le code suivant permet de tracer un graphique où les nouvelles données et les données initiales sont représentées ensemble sur le plan formé par les deux premiers axes :

```
fig, axes = plt.subplots(figsize=(8,8))
axes.set_xlim(-3,4)
axes.set_ylim(-4,6)

for i in range(n):
    plt.annotate(data.index[i],(coords[i,0],coords[i,1]))

for i in range(coords_new_data.shape[0]):

plt.annotate(new_data.index[i],(coords_new_data[i,0],coords_new_
data[i,1]),color='r')


plt.plot([-3,4],[0,0],color='black',linewidth=1)
plt.plot([0,0],[-4,6],color='black',linewidth=1)
```

Le résultat est illustré sur la figure 12-9 ci-dessous :

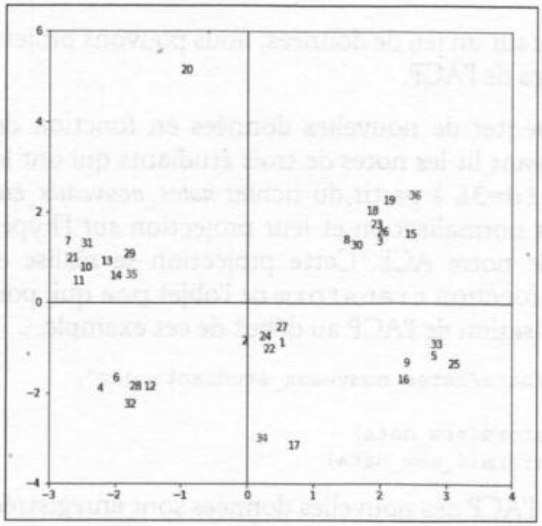


Figure 12-9 : représentation de nouvelles données à côté des données initiales en fonction de leurs coordonnées sur les deux premiers axes

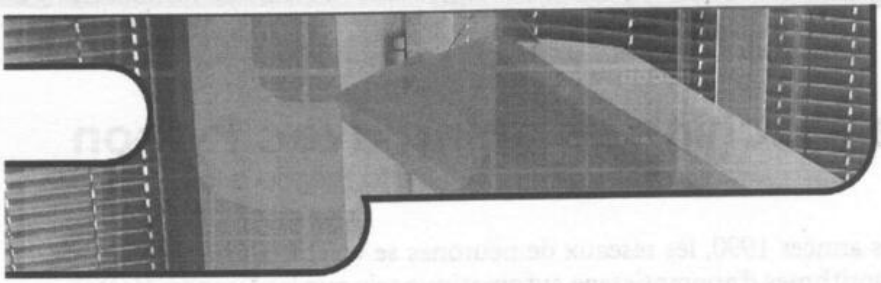
Sans ouvrir manuellement le fichier *notes_nouveaux_étudiants.csv* pour visualiser les notes des trois nouveaux étudiants, grâce à leur projection et leur représentation sur les deux premiers axes nous pouvons deviner les profils auxquels ils ont le plus de chances d'appartenir !

Ainsi, l'étudiant avec l'`id=34` est potentiellement un étudiant fort dans toutes les matières, puisqu'il est placé non loin de l'étudiant avec l'`id=17`. L'étudiant avec l'`id=35` est potentiellement fort dans les matières Histoire, Géographie et Philosophie et il a potentiellement des notes moyennes dans les autres matières puisqu'il est proche des étudiants du groupe `G4:{id=7, id=10, id=11, id=13, id=14, id=21, id=29, id=31}`. L'étudiant avec l'`id=36` est potentiellement fort dans les trois matières Français, Anglais et Allemand et il a potentiellement des notes moyennes dans les autres matières, puisque sa représentation graphique le place près des étudiants du groupe `G5:{id=3, id=8, id=15, id=18, id=19, id=23, id=26, id=30}`. Vous pouvez vérifier ces prédictions en ouvrant cette fois le fichier *notes_nouveaux_étudiants.csv*.

© Editions ENI - All rights reserved

7. Conclusion

Dans ce chapitre, nous avons abordé l'Analyse en Composantes Principales. Nous avons commencé par expliquer les objectifs de l'ACP et les problèmes qu'elle tente de résoudre. Nous avons essayé de donner l'intuition derrière l'ACP, puis nous avons donné la preuve formelle que les vecteurs propres de la matrice de données sous-jacente au nuage de points analysé correspondent aux axes principaux de l'ACP. Nous avons proposé de réaliser une ACP sur un exemple illustratif afin d'assimiler les concepts théoriques et de montrer comment Scikit-learn facilite l'application d'une ACP.



Chapitre 4-7
Les réseaux de neurones

1. Objectif du chapitre

Comme vous pouvez l'imaginer, les réseaux de neurones artificiels trouvent leur origine et leur inspiration dans les neurones biologiques, notamment dans les neurones humains. En effet, en plein milieu de la Seconde Guerre mondiale, en 1943, Warren McCulloch et Walter Pitts proposèrent une première formalisation du neurone. Une forme plus sophistiquée de ce formalisme a été introduite en 1957 par Frank Rosenblatt avec le modèle du perceptron. Outre les limites technologiques de l'époque, en 1969, Marvin Lee Minsky et Seymour Papert démontrent les limites théoriques des modèles de classification basés sur les réseaux de neurones à traiter les problèmes non linéaires. En conséquence de ces limites, technologiques et théoriques, les réseaux neuronaux ont été mis à l'écart pendant une bonne dizaine d'années.

En 1982, John Joseph Hopfield introduit un modèle de réseaux de neurones récurrents et, en 1986, le fameux modèle de perceptron multicouche basé sur la rétropropagation du gradient a été introduit par David Rumelhart. Une version similaire à ce dernier modèle a été introduite par Yann LeCun durant les années 1980. Suite à ces travaux, les réseaux de neurones connaissent un nouvel engouement donnant naissance à des applications concrètes dans des domaines divers.

À la fin des années 1990, les réseaux de neurones se voient concurrencés par d'autres algorithmes d'apprentissage automatique tels que les Support Vector Machines. Ces dernières années, avec l'évolution fulgurante de la technologie, la pratique des réseaux de neurones dépasse largement le cadre de la recherche et trouve sa place dans quasiment tous les domaines de l'intelligence artificielle appliqués dans les entreprises et dans divers métiers.

Aujourd'hui, les algorithmes de Deep Learning, formé par des architectures complexes de réseaux de neurones, sont utilisés avec beaucoup de succès, en particulier dans le domaine de la reconnaissance de formes dans les images.

Ce chapitre introduit les notions de base des réseaux de neurones en se fondant sur l'un des formalismes les plus connus, à savoir les réseaux de neurones avec rétropropagation.

À la fin de ce chapitre, le lecteur aura abordé :

- la notion de neurone formel,
- la notion de réseau de neurones,
- l'algorithme de réseau de neurones avec rétropropagation,
- la construction d'un réseau de neurones avec Scikit-learn.

2. Modélisation d'un neurone artificiel

Avant toute chose, il faut savoir que les modèles de neurones artificiels ne représentent pas fidèlement le fonctionnement des neurones biologiques. En effet, ces derniers présentent des structures complexes tant au niveau d'un neurone isolé qu'au niveau des interactions interneuronales. La tâche qui consiste à étudier le fonctionnement d'un cerveau, humain ou pas, reste encore et avant tout un travail exploratoire et difficile.

© Editions ENI - All rights reserved

2.1 Le neurone biologique

Le neurone biologique a une structure similaire à celle représentée dans la figure 13-1 suivante :

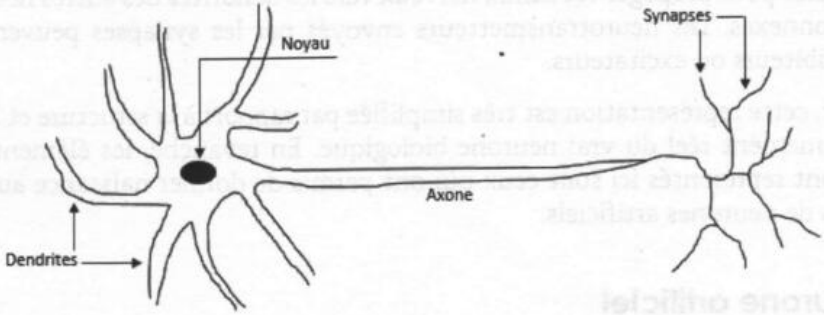


Figure 13-1 : représentation d'un neurone biologique

Cette figure montre qu'un neurone est composé essentiellement des éléments suivants :

- Les dendrites, qui servent d'interface en entrée pour un neurone. Concrètement, ces dendrites reçoivent l'influx nerveux provenant des autres neurones.
- Le noyau, où ont lieu les réactions électrochimiques. En fonction de l'intensité des signaux qui arrivent à travers les dendrites, le noyau génère et envoie un influx nerveux électrique à travers l'axone.
- L'axone joue le rôle de support de l'influx nerveux électrique qui sera transmis aux terminaisons synaptiques. La taille de l'axone peut aller de quelques fois à plusieurs milliers de fois la taille du noyau !
- Les synapses, qui transmettent l'influx nerveux électrique provenant de l'axone. Cette transmission est réalisée en libérant des neurotransmetteurs qui seront capturés par les dendrites des autres cellules neuronales. Ces neurotransmetteurs peuvent être excitateurs ou inhibiteurs.

En résumé, un neurone biologique reçoit l'influx nerveux via ses dendrites. En fonction de l'intensité de ce flux, le noyau du neurone décide d'envoyer ou pas un influx nerveux électrique à travers l'axone.

Une fois que l'influx nerveux a atteint les synapses, des neurotransmetteurs sont libérés pour propager cet influx nerveux vers les dendrites des autres neurones connexes. Les neurotransmetteurs envoyés par les synapses peuvent être inhibiteurs ou excitateurs.

Bien sûr, cette représentation est très simplifiée par rapport à la structure et le fonctionnement réel du vrai neurone biologique. En revanche, les éléments qui y sont représentés ici sont ceux qui ont permis de donner naissance aux modèles de neurones artificiels.

2.2 Le neurone artificiel

Le modèle de neurone artificiel qui est inspiré du neurone biologique et généralement utilisé dans la littérature est représenté dans la figure suivante :

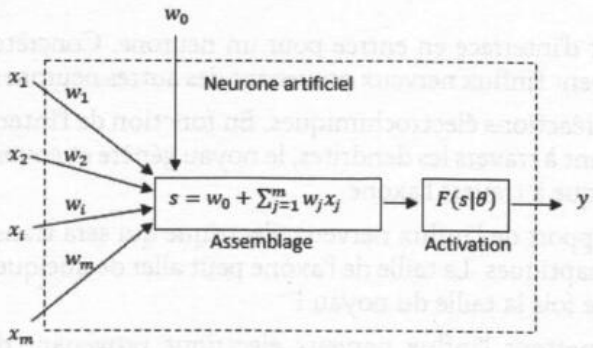


Figure 13-2 : représentation d'un neurone formel

À l'instar du neurone biologique, le neurone artificiel reçoit un ensemble d'entrées et retourne un résultat en sortie. Pour aboutir à ce résultat, le neurone artificiel applique une fonction d'assemblage sur les entrées, puis, sur le résultat de cette fonction d'assemblage, il applique une fonction d'activation qui donne le résultat final du neurone.

© Editions ENI - All rights reserved

Sur le schéma de la figure 13-2 ci-dessus, le neurone artificiel reçoit en entrée un vecteur $X_i = (x_1, x_2, \ldots, x_m)$. En utilisant un vecteur de poids $W_i = (w_0, w_1, w_2, \ldots, w_m)$, ce neurone applique un produit scalaire en guise de fonction d'assemblage $s = w_0 + \sum_{j=1}^{m} w_j x_j$. Le poids w_i avec `i=[1,m]` correspond à la force de la liaison entre le neurone et la valeur x_i de X_i. Contrairement à tous les autres poids w_i, le poids w_0 n'est associé à aucune valeur du vecteur d'entrée X_i. En réalité, le poids w_0 représente un biais.

Sur le résultat de la fonction d'assemblage, un neurone applique une fonction d'activation qui est généralement sous la forme $F(s|\theta)$ où θ est un seuil d'activation du neurone. Le résultat à la sortie d'un neurone correspond au résultat de cette fonction d'activation.

Dans la littérature, les fonctions d'assemblage utilisées généralement sont soit le produit scalaire entre le vecteur d'entrée et le vecteur des poids, soit la mesure de distance entre ces deux vecteurs.

Plusieurs fonctions d'activation existent. Les trois fonctions suivantes font partie des plus utilisées :

1. La fonction sigmoïde $F(s) = \frac{1}{1+e^s}$. Cette fonction transforme le résultat de la fonction d'assemblage en une probabilité.
2. La fonction d'activation à seuil $F(s|\theta) = \begin{cases} 1 \; si \; s \geq \theta \\ 0 \; si \; s < \theta \end{cases}$
3. La fonction radiale $F(s) = \sqrt{\frac{1}{2\pi}} e^{(-\frac{s^2}{2})}$

Un exemple concret sera expliqué dans la suite de ce chapitre pour comprendre le déroulement des calculs en partant des entrées jusqu'à la sortie d'un neurone.

Dans la pratique, il est plus simple de considérer le biais w_0 comme étant une valeur au même titre que les valeurs x_i et fixer le poids de la relation entre un neurone et ce biais par la valeur 1 comme le montre la figure 13-3 suivante :

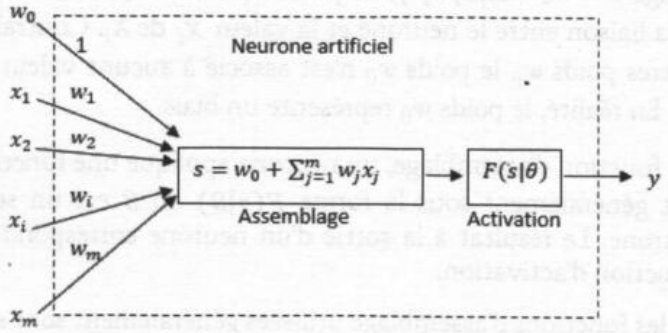


Figure 13-3 : représentation d'un neurone formel avec l'introduction du biais à travers un neurone supplémentaire

Les neurones artificiels se distinguent entre eux par le choix au sein de chaque neurone de la fonction d'assemblage et de la fonction d'activation.

3. Architecture d'un réseau de neurones

La section précédente a montré qu'un neurone artificiel reçoit en entrée des données et, après avoir opéré une transformation sur ces données, il donne en sortie un résultat. Ce résultat peut être utilisé en entrée d'un neurone qui à son tour produira un autre résultat. Un réseau de neurones est constitué d'un ensemble de neurones répartis sur un ensemble de couches. L'architecture d'un réseau de neurones est déterminée par la nature des liens entre ses neurones et aussi la manière dont ils sont répartis.

© Editions ENI - All rights reserved

La figure 13-4 suivante donne un exemple d'un réseau de neurones avec une couche d'entrée, deux couches cachées et une couche de sortie :

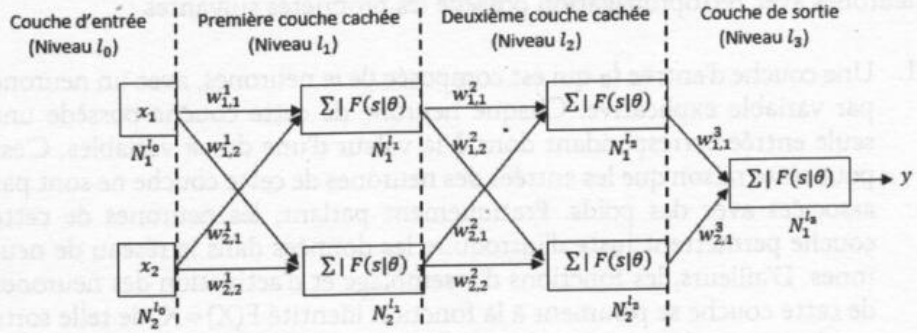


Figure 13-4 : un réseau de neurones avec deux couches cachées

Pour rendre plus simple la compréhension des notions qui vont être abordées dans la suite de ce chapitre, nous allons adopter les notations ci-dessous qui sont illustrées sur la figure 13-4.

- Un réseau de neurones est constitué des couches $C = \{l_0, l_1, l_2 \dots l_L\}$ avec $|C| = L + 1$. Le nombre de couches est donc égal à $(L+1)$.
- Chaque couche l_i d'un réseau de neurones est associée à un ensemble de neurones $N^{l_i} = \{N_1^{l_i}, N_2^{l_i}, N_3^{l_i} \dots\}$.
- Chaque neurone $N_j^{l_i}$ de la couche l_i est associé à un ensemble de poids $W_j^{l_i} = \{w_{1,j}^{l_i}, w_{2,j}^{l_i}, w_{3,j}^{l_i} \dots w_{q,j}^{l_i}\}$ avec q le nombre de neurones dans la couche précédente $l_{(i-1)}$.

De façon générale, un réseau de neurones est caractérisé par le nombre de couches qui le constituent, le nombre de neurones dans chaque couche, la nature des relations entre les neurones et les fonctions d'assemblage et d'activation de chaque neurone.

L'exemple donné dans la figure 13-4 correspond à une instance du modèle de réseaux de neurones avec rétropropagation, qui est l'un des types de réseaux de neurones les plus utilisés et les plus détaillés dans la littérature.

Son algorithme sera présenté à la section suivante.

Pour des données d'entraînement à m variables explicatives, le réseau de neurones avec rétropropagation possède les propriétés suivantes :

1. Une couche d'entrée l_0 qui est composée de m neurones, avec un neurone par variable explicative. Chaque neurone de cette couche possède une seule entrée correspondant donc à la valeur d'une des m variables. C'est pour cette raison que les entrées des neurones de cette couche ne sont pas associées avec des poids. Pratiquement parlant, les neurones de cette couche permettent juste d'introduire les données dans le réseau de neurones. D'ailleurs, les fonctions d'assemblage et d'activation des neurones de cette couche se résument à la fonction identité F(X)=X, de telle sorte que les sorties de ces neurones correspondent exactement à leurs entrées.

   Par exemple, dans la figure 13-4, si le neurone $N_1^{l_0}$ reçoit la valeur 5 en entrée, alors il va renvoyer cette même valeur 5 aux neurones de la couche l_1.
2. Les neurones d'une même couche ne sont pas connectés entre eux, c'est-à-dire que pour une couche l_i, il n'existe aucun poids reliant les neurones $N^{l_i} = \{N_1^{l_i}, N_2^{l_i}, N_3^{l_i} \dots\}$.
3. L'entrée de chaque neurone d'une couche cachée est connectée à tous les neurones de la couche précédente et sa sortie est connectée à tous les neurones de la couche suivante.

   Plus formellement, mis à part la couche d'entrée l_0 et la couche de sortie l_L, nous avons : $\forall i, \forall j, \forall N_j^{l_i}, \exists N_k^{l_{i-1}}$ avec $w_{k,j}^{l_i}$ et $\exists N_{k'}^{l_{i+1}}$ avec $w_{j,k'}^{l_i}$

Remarque

En règle générale, quelle que soit la nature d'un réseau de neurones, de type rétropropagation ou non, lorsque dans une couche cachée chaque entrée de chaque neurone est connectée à tous les neurones de la couche précédente et que chaque sortie de chaque neurone est connectée à tous les neurones de la couche suivante, on dit que cette couche est une couche dense.

© Editions ENI - All rights reserved

4. La couche de sortie fournit la réponse du réseau de neurones à une observation $X_i = (x_1, x_2, \ldots, x_m)$. Pour les problèmes de classification à plusieurs classes, le réseau de neurones aura un neurone de sortie par classe cible.

4. L'algorithme de rétropropagation

Le réseau de neurones avec rétropropagation est un algorithme itératif. Les étapes principales de cet algorithme se présentent comme suit :

Entrées : - Un ensemble d'observations $X = \{X_1, X_2, \ldots, X_i, \ldots, X_n\}$
- Un ensemble de résultats $Y = \{y_1, y_2, \ldots, y_i, \ldots, y_n\}$
avec y_i le résultat connu de l'observation X_i
- Une fonction d'activation $F(s|\theta)$
avec θ le seuil d'activation
- Un réseau de neurones *RN* composé de $L \geq 2$ couches;
Une couche d'entrée l_0 he de sortie l_L
et au moins une couche cachée
- Un taux d'apprentissage α

Sorties : le réseau de neurones *RN*
avec les ensembles de poids $W_j^{l_i}$ ajustés.

a) Pour chaque couche l_i avec i allant de 1 à L faire :
Initialisation aléatoire des poids $w_{i,j}^{l_i}$

b) Répété jusqu'à ce qu'un critère d'arrêt soit atteint :
Pour chaque couple (X_i, y_i) faire :

//Initialisation de la couche d'entrée
1- Affecter à chaque neurone j de la couche d'entrée l_0
la valeur de $x_{i,j}$ ($x_{i,j}$ la valeur de la variable j de X_i)

//Propagation avant
2- Pour chaque couche l_i avec i allant de 1 à L faire :
Pour chaque neurone $N_j^{l_i}$ de la couche l_i faire :
- $s = \sum_{k=0}^{q} w_{k,j}^{l_i} v_{k,j}$ (q=le nombre de neurones de la

couche précédente ($l_{(i-1)}$), $v_{k,j}$= la valeur de la $k^{ième}$ entrée du neurone $N_j^{l_i}$). Pour k=0, $w_{0,j}^{l_i}$ représente le biais du neurone $N_j^{l_i}$ et $v_{0,j}$=1

- $res_{N_j^{l_i}} = F(s|\theta)$ (donc $res_{N_j^{l_i}}$ est la valeur de sortie du neurone $N_j^{l_i}$

//Calcul de l'erreur des neurones de la couche de sortie

3- Pour chaque neurone $N_j^{l_L}$ de la couche de sortie faire :

$\Delta_j^{l_L} = y_i - res_{N_j^{l_L}}$

//Propagation arrière

4- Pour chaque couche l_i avec i allant de L-1 à 1 faire :

Pour chaque neurone $N_j^{l_i}$ de la couche l_i faire :

$\Delta_j^{l_i} = res_{N_j^{l_i}} * \left(1 - res_{N_j^{l_i}}\right) * \sum_{k=1}^{p}(w_{j,k}^{(l+1)}\Delta_k^{l_{(i+1)}})$

(p=le nombre de neurones dans la couche suivante ($l_{(i-1)}$) et $\Delta_k^{l_{(i+1)}}$ l'erreur du neurone $N_k^{l_{(i+1)}}$)

//Mise à jour des poids du réseau

5- Pour chaque couche l_i allant de 1 à L faire :

Pour chaque poids $w_{j,k}^{l_i}$ faire :

//la règle d'apprentissage

$w_{j,k}^{l_i} = w_{j,k}^{l_i} + \alpha * res_{N_j^{(l_{i-1})}} * \Delta_k^{l_i}$

L'algorithme de réseau de neurones avec rétropropagation admet en paramètres une base d'apprentissage $X = \{X_1, X_2, \ldots, X_i, \ldots, X_n\}$ et $Y = \{y_1, y_2, \ldots, y_i, \ldots, y_n\}$, un réseau de neurones avec les propriétés vues à la section précédente et une fonction d'activation $F(s|\theta)$. Il passe par deux étapes globales. La première étape, l'étape (a) dans le pseudo-code ci-dessus, permet d'initialiser les valeurs des poids $w_{i,j}$. Plusieurs stratégies d'initialisation peuvent être adoptées.

© Editions ENI - All rights reserved

La deuxième étape, l'étape (b), constitue le cœur de l'algorithme et elle est composée d'une boucle principale qui prend fin lorsqu'un critère d'arrêt est atteint. Ce critère d'arrêt peut être par exemple un nombre d'itérations fixé au préalable ou un critère relatif à la convergence du réseau. Chaque itération de cette boucle permet de parcourir tous les exemples de la base d'apprentissage.

Ainsi, pour chaque exemple $(X_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,m}), y_i)$, cet algorithme exécute cinq étapes qui sont définies dans l'ordre comme suit :

- Étape (1) : affectation des neurones de la couche d'entrée avec les valeurs des variables $(x_{i,1}, x_{i,2}, \ldots, x_{i,m})$. En effet, cette couche d'entrée (voir figure 13-4) est composée de m neurones, un neurone pour chaque variable explicative. La particularité des neurones de cette couche d'entrée est qu'ils n'ont ni fonction d'assemblage ni fonction d'activation. Ces neurones transmettent en sortie leur valeur d'entrée. C'est cette couche qui permet d'introduire les observations dans le réseau de neurones.
- Étape (2) : cette étape est composée de deux boucles imbriquées pour réaliser une propagation avant. La première boucle parcourt une par une les couches l_i du réseau, sauf la couche d'entrée l_0. La boucle imbriquée parcourt un par un les neurones $N_j^{l_i}$ de la couche l_i en cours de traitement. Pour chaque neurone $N_j^{l_i}$, l'algorithme calcule la fonction d'assemblage $s = \sum_{k=0}^{q} w_{k,j}^{l_i} v_{k,j}$, puis il calcule la fonction d'activation $res_{N_j^{l_i}} = F(s|\theta)$. La valeur de q correspond au nombre de neurones dans la couche précédente $l_{(i-1)}$ et $v_{k,j}$ est la k$^{\text{ième}}$ entrée du neurone $N_j^{l_i}$ et qui correspond également au résultat du neurone $N_k^{l_{(i-1)}}$ de la couche précédente.

  À la fin de cette étape (2), l'algorithme a calculé les résultats de sortie de tous les neurones du réseau de neurones incluant les neurones de la couche de sortie. Pour rappel, un réseau de neurones peut avoir plusieurs neurones dans la couche de sortie si le problème à résoudre est un problème de classification multiclasse.

- Étape (3) : pour chaque neurone $N_j^{l_L}$ de la couche de sortie l_L, l'algorithme calcule la différence entre la valeur prédite par ce neurone $res_{N_j^{l_L}}$ et la valeur attendue y_i en utilisant la simple formule $\Delta_j^{l_L} = y_i - res_{N_j^{l_L}}$.
- Étape (4) : cette étape est composée de deux boucles imbriquées pour réaliser une propagation arrière des erreurs. La première boucle parcourt une par une les couches l_i du réseau, sauf les couches d'entrée et de sortie l_0 et l_L. La boucle imbriquée parcourt un par un les neurones $N_j^{l_i}$ de la couche l_i en cours de traitement. Pour chaque neurone $N_j^{l_i}$, l'algorithme calcule le delta $\Delta_j^{l_i}$ par rapport aux deltas des neurones devant lui en utilisant la formule $\Delta_j^{l_i} = res_{N_j^{l_i}} * \left(1 - res_{N_j^{l_i}}\right) * \sum_{k=1}^{p}(w_{j,k}^{(l+1)} \Delta_k^{l_{(i+1)}})$

  Avec $res_{N_j^{l_i}}$ le résultat de neurone $N_j^{l_i}$ lors de la propagation avant précédente et $k = 1..p$ l'indice des neurones de la couche suivante.

  Cette formule, aussi appelée la règle d'apprentissage, est une simplification du calcul des dérivées partielles en chaîne des résultats intermédiaires entre le neurone $N_j^{l_i}$ et les neurones des couches suivantes auxquels il est connecté de façon immédiate ou de façon transitive.
- Étape (5) : une fois que la propagation des erreurs est appliquée à tous les neurones des couches allant de la couche $l_{(L-1)}$ jusqu'à la couche l_1, c'est-à-dire que chaque neurone de ces couches possède un delta $\Delta_j^{l_i}$, les poids du réseau vont être mis à jour avec la formule $w_{j,k}^{l_i} = w_{j,k}^{l_i} + \alpha * res_{N_j^{(l_{i-1})}} * \Delta_k^{l_i}$

  Le paramètre α, appelé le taux d'apprentissage, permet de rythmer la vitesse de convergence du réseau.

Une fois que ces cinq étapes sont exécutées pour tous les exemples de la base d'apprentissage, l'algorithme vérifie si un critère d'arrêt est atteint. Si aucun critère d'arrêt n'est atteint, alors un autre parcours de tous les exemples de la base d'apprentissage sera réalisé et les poids $w_{j,k}^{l_i}$ seront de nouveau réajustés.

© Editions ENI - All rights reserved

Une fois qu'un critère d'arrêt est atteint, l'algorithme retourne un réseau de neurones avec les poids $w_{j,k}^{l_i}$ ajustés.

Afin de comprendre cet algorithme de façon plus approfondie, la section suivante montre étape par étape un exemple concret de déroulement de la rétropropagation sur une observation donnée.

Exemple de propagation avant et arrière

Dans l'exemple qui suit, un réseau de neurones est utilisé pour montrer le déroulement de la propagation avant, la propagation arrière et la mise à jour des poids de l'algorithme de rétropropagation vue précédemment.

Supposons que le réseau de neurones de cet exemple permette de réaliser une classification binaire qui associe pour une entrée donnée, soit la classe `0` soit la classe `1`. Également, supposons qu'une observation de la base d'apprentissage $X=(1,2)$ appartienne à la classe $y=1$ et que :

- le réseau de neurones est composé de deux couches cachées,
- chaque couche cachée contient deux neurones,
- la fonction d'assemblage au niveau de chaque neurone est le produit scalaire entre le vecteur composé des entrées et le vecteur composé des poids reliés à un neurone,
- la fonction d'activation est la sigmoïde $\frac{1}{1+e^{-F(x)}}$,
- le taux d'apprentissage $\alpha=0.01$,
- l'étape (a) permet l'initialisation aléatoire des poids de ce réseau de neurones comme sur la figure 13-5 ci-après. Habituellement, les poids d'un réseau de neurones sont initialisés avec des petites valeurs comprises dans l'intervalle `[-0.001, 0.001]`. Cependant, pour simplifier les illustrations de cet exemple, nous avons volontairement choisi des valeurs entières,
- pour des raisons de simplification, nous avons volontairement omis les neurones associés aux biais.

Il nous reste donc à dérouler l'étape (b). Dans cet exemple illustratif, nous déroulons cette étape (b) uniquement sur une seule observation $X=(1,2)$. Nous allons donc réaliser une seule itération de la boucle imbriquée de cette

étape (b), puisqu'une seule itération de cette boucle est suffisante pour montrer concrètement le déroulement de la propagation avant, de la propagation arrière des erreurs, ainsi que le calcul de l'ajustement des poids du réseau.

L'étape (1) de la grande étape (b) consiste à affecter les neurones de la couche d'entrée avec les valeurs de l'observation $X=(1,2)$.

L'état du réseau après l'exécution de cette étape (1) de l'algorithme de rétropropagation est le suivant :

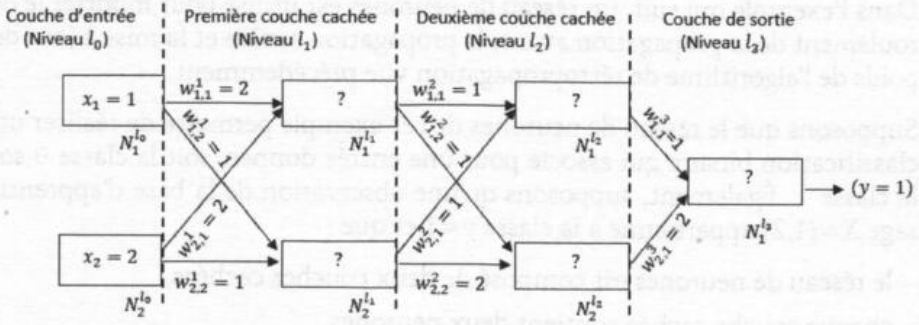


Figure 13-5 : exemple d'un réseau de neurones après les étapes d'initialisation des poids et l'affectation des neurones de la couche d'entrée

Déroulement de l'étape (2) :

- Calcul des sorties des neurones de la première couche cachée l_i :

$$res_{N_1^{l_1}} = \frac{1}{1+e^{-(2*1+2*2)}} = \frac{1}{1+e^{-6}} \approx \frac{1}{1+0.00247875} \approx 0.99752737$$

$$res_{N_2^{l_1}} = \frac{1}{1+e^{-(1*1+1*2)}} = \frac{1}{1+e^{-3}} \approx \frac{1}{1+0.04978706} \approx 0.95257413$$

- Calcul des sorties des neurones de la deuxième couche cachée l_2 :

$$res_{N_1^{l_2}} = \frac{1}{1+e^{-\left(1*res_{N_1^{l_1}}+1*res_{N_2^{l_1}}\right)}} = \frac{1}{1+e^{-(1*0.99752737+1*0.95257413)}} = \frac{1}{1+e^{-1.9501015}}$$

$$\approx \frac{1}{1+0.14225963} \approx 0.87545771$$

© Editions ENI - All rights reserved

$$res_{N_2^{l_2}} = \frac{1}{1+e^{-(1*res_{N_1^{l_1}} + 2*res_{N_2^{l_1}})}} = \frac{1}{1+e^{-(1*0.99752737+2*0.95257413)}} = \frac{1}{1+e^{-2.90267563}}$$

$$\approx \frac{1}{1+0.05487619} \approx 0.94797854$$

– Calcul de la sortie de la couche l_3 qui est la couche de sortie du réseau :

$$res_{N_1^{l_3}} = \frac{1}{1+e^{-(1*res_{N_1^{l_2}} + 2*res_{N_2^{l_2}})}} = \frac{1}{1+e^{-(1*0.87545771+2*0.94797854)}} = \frac{1}{1+e^{-2.77141479}}$$

$$\approx \frac{1}{1+0.06257341} \approx 0.94111144$$

L'état du réseau à la fin de la propagation avant est présenté dans la figure suivante :

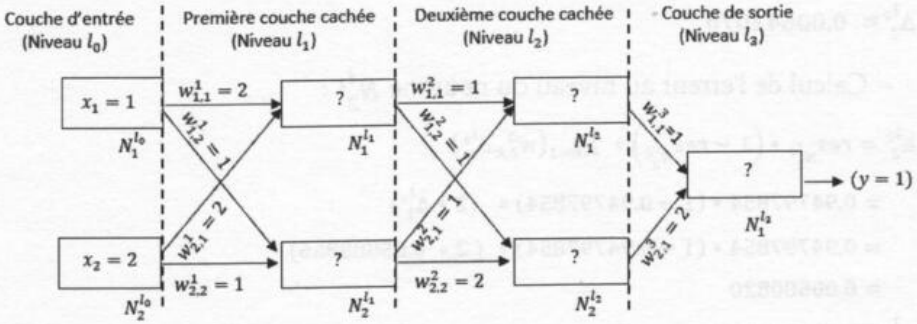


Figure 13-6 : l'état du réseau après la propagation avant

Déroulement de l'étape (3) :

Cette étape consiste à calculer le delta des neurones de sortie du réseau. Dans notre exemple, il y a un seul neurone dans la couche de sortie. Le résultat attendu est (y=1), alors que le résultat de sortie de ce réseau de neurones est égal à 0.94111144. Nous allons donc calculer l'erreur du seul neurone de sortie comme suit $\Delta_1^{l_3} = y - res_{N_1^{l_3}} = 1 - 0.94111144$

Déroulement de l'étape (4) :

Maintenant, nous allons propager l'erreur du neurone de la couche de sortie $\Delta_1^{l_3}$ en réalisant une propagation arrière. Cette propagation de l'erreur sera réalisée sur les deux couches cachées l_2 et l_1.

– Calcul des deltas des neurones de la deuxième couche cachée l_2 :

– Calcul de l'erreur au niveau du neurone $N_1^{l_2}$:

$$\Delta_1^{l_2} = res_{N_1^{l_2}} * \left(1 - res_{N_1^{l_2}}\right) * \sum_{k=1}^{1}(w_{1,k}^{3}\Delta_k^{l_3})$$

$$= 0.87545771 * (1 - 0.87545771) * (1 * \Delta_1^{l_3})$$

$$= 0.87545771 * (1 - 0.87545771) * (1 * 0.05888856)$$

$$\approx 0.00642070$$

$$\Delta_1^{l_2} \approx 0.00642070$$

– Calcul de l'erreur au niveau du neurone $N_2^{l_2}$:

$$\Delta_2^{l_2} = res_{N_2^{l_2}} * \left(1 - res_{N_2^{l_2}}\right) * \sum_{k=1}^{1}(w_{2,k}^{3}\Delta_k^{l_3})$$

$$= 0.94797854 * (1 - 0.94797854) * (2 * \Delta_1^{l_3})$$

$$= 0.94797854 * (1 - 0.94797854) * (2 * 0.05888856)$$

$$\approx 0.00580820$$

$$\Delta_2^{l_2} \approx 0.00580820$$

– Calcul des deltas des neurones de la deuxième couche cachée l_1 :

– Calcul de l'erreur au niveau du neurone $N_1^{l_1}$:

$$\Delta_1^{l_1} = res_{N_1^{l_1}} * \left(1 - res_{N_1^{l_1}}\right) * \sum_{k=1}^{2}(w_{1,k}^{2}\Delta_k^{l_2})$$

$$= 0.99752737 * (1 - 0.99752737) * [(1 * \Delta_1^{l_2}) + (1 * \Delta_2^{l_2})]$$

$$= 0.99752737 * (1 - 0.99752737) * [(1 * 0.00642070) + (1 * 0.00580820)]$$

$$\approx 0.00003016$$

© Editions ENI - All rights reserved

$\Delta_1^{l_1} \approx 0.00003016$

– Calcul de l'erreur au niveau du neurone $N_2^{l_1}$:

$$\Delta_2^{l_1} = res_{N_2^{l_1}} * \left(1 - res_{N_2^{l_1}}\right) * \sum_{k=1}^{2}(w_{2,k}^2 \Delta_k^{l_2})$$

$$= 0.95257413 * (1 - 0.95257413) * [(1 * \Delta_1^{l_2}) + (2 * \Delta_2^{l_2})]$$

$$= 0.95257413 * (1 - 0.95257413) * [(1 * 0.00642070) + (2 * 0.00580820)]$$

$$\approx 0.00081485$$

$\Delta_2^{l_1} \approx 0.00081485$

À la fin de l'exécution de cette étape (4), l'état du réseau devient comme suit :

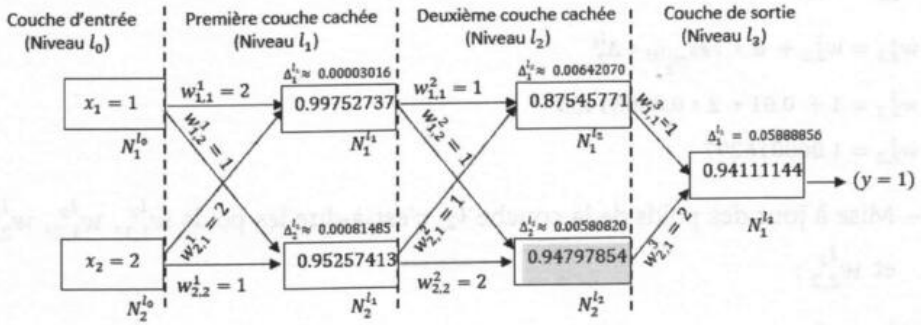


Figure 13-7 : l'état du réseau après la propagation arrière des erreurs

Déroulement de l'étape (5) :

– Ajustement des poids du réseau de neurones. Pour chaque poids du réseau, le calcul s'effectue en utilisant la formule $w_{j,k}^{l_i} = w_{j,k}^{l_i} + \alpha * res_{N_j^{(l_{i-1})}} * \Delta_k^{l_i}$ avec α=0.01.

– Mise à jour des poids de la couche l_1, c'est-à-dire les poids $w_{1,1}^{l_1}$, $w_{1,2}^{l_1}$, $w_{2,1}^{l_1}$ et $w2,2l_1$:

$$w_{1,1}^1 = w_{1,1}^1 + \alpha * res_{N_1^{(l_0)}} * \Delta_1^{l_1}$$

$$w_{1,1}^1 = 2 + 0.01 * 1 * 0.00003016$$

$$w_{1,1}^1 \approx 2.0000003016$$

$$w_{1,2}^1 = w_{1,2}^1 + \alpha * res_{N_1^{(l_0)}} * \Delta_2^{l_1}$$

$$w_{1,2}^1 = 1 + 0.01 * 1 * 0.00081485$$

$$w_{1,2}^1 = 1.0000081485$$

$$w_{2,1}^1 = w_{2,1}^1 + \alpha * res_{N_2^{(l_0)}} * \Delta_1^{l_1}$$

$$w_{2,1}^1 = 2 + 0.01 * 2 * 0.00003016$$

$$w_{2,1}^1 = 2.0000006032$$

$$w_{2,2}^1 = w_{2,2}^1 + \alpha * res_{N_2^{(l_0)}} * \Delta_2^{l_1}$$

$$w_{2,2}^1 = 1 + 0.01 * 2 * 0.00081485$$

$$w_{2,2}^1 = 1.000016297$$

– Mise à jour des poids de la couche l_2, c'est-à-dire les poids $w_{1,1}^{l_2}$, $w_{1,2}^{l_2}$, $w_{2,1}^{l_2}$ et $w_{2,2}^{l_2}$:

$$w_{1,1}^2 = w_{1,1}^2 + \alpha * res_{N_1^{(l_1)}} * \Delta_1^{l_2}$$

$$w_{1,1}^2 = 1 + 0.01 * 0.99752737 * 0.00642070$$

$$w_{1,1}^2 \approx 1.0000640482$$

$$w_{1,2}^2 = w_{1,2}^2 + \alpha * res_{N_1^{(l_1)}} * \Delta_2^{l_2}$$

$$w_{1,2}^2 = 1 + 0.01 * 0.99752737 * 0.00580820$$

$$w_{1,2}^2 \approx 1.0000579383$$

$$w_{2,1}^2 = w_{2,1}^2 + \alpha * res_{N_2^{(l_1)}} * \Delta_1^{l_2}$$

$$w_{2,1}^2 = 1 + 0.01 * 0.95257413 * 0.00642070$$

$$w_{2,1}^2 \approx 1.0000611619$$

© Editions ENI - All rights reserved

$$w_{2,2}^{2} = w_{2,2}^{2} + \alpha * res_{N_2^{(l_1)}} * \Delta_2^{l_2}$$

$$w_{2,2}^{2} = 2 + 0.01 * 0.95257413 * 0.00580820$$

$$w_{2,2}^{2} \approx 2.0000553274$$

– Mise à jour des poids de la couche l_3, c'est-à-dire les poids $w_{1,1}^{l_3}$, $w2,1l_3$:

$$w_{1,1}^{3} = w_{1,1}^{3} + \alpha * res_{N_1^{(l_2)}} * \Delta_1^{l_3}$$

$$w_{1,1}^{3} = 1 + 0.01 * 0.87545771 * 0.94111144$$

$$w_{1,1}^{3} \approx 1.0082390326$$

$$w_{2,1}^{3} = w_{2,1}^{2} + \alpha * res_{N_2^{(l_2)}} * \Delta_1^{l_3}$$

$$w_{2,1}^{3} = 2 + 0.01 * 0.94797854 * 0.94111144$$

$$w_{2,1}^{3} \approx 2.0089215344$$

À la fin de l'exécution de l'étape (5), le réseau devient comme suit :

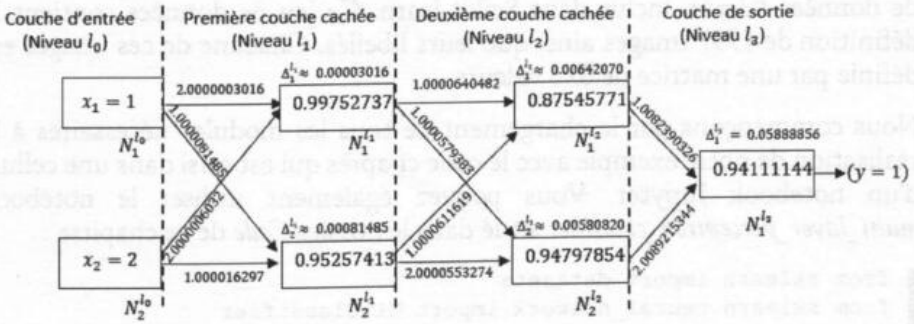


Figure 13-8 : l'état du réseau de neurones après la mise à jour des poids

Remarquez que les poids initiaux de la figure 13-5 et les poids obtenus de la dernière figure 13-8, après une seule propagation avant et une seule propagation arrière, sont légèrement différents. Dans cet exemple, nous avons déroulé l'algorithme de la rétropropagation sur une seule donnée d'entraînement `[x=(1,2), y=1]`. En pratique, cette procédure est répétée autant de fois qu'il y a de données d'entraînement à chaque boucle imbriquée de l'étape (b) de l'algorithme de rétropropagation.

Une fois que toutes les observations d'entraînement ont été utilisées pour mettre à jour les poids d'un réseau de neurones, un critère d'arrêt est mesuré afin de savoir si un autre parcours de la boucle imbriquée de l'étape (b) est nécessaire ou pas.

Par exemple, si nous constatons que les cinq dernières itérations de cette boucle imbriquée n'améliorent pas la qualité des résultats au-delà d'un certain seuil fixé au préalable, alors on peut décider que le réseau de neurones est dans une situation de convergence et que le programme devrait s'arrêter, car quel que soit le nombre d'itérations supplémentaires que nous aurions réalisées, la qualité des résultats ne serait jamais améliorée.

5. Exemple d'un réseau de neurones avec Scikit-learn

Dans cet exemple, nous allons appliquer un réseau de neurones basé sur le principe de la rétropropagation abordé précédemment. Ce réseau de neurones a comme objectif d'apprendre à reconnaître les chiffres écrits à la main. Pour entraîner et tester le réseau de neurones de cet exemple, nous utilisons le jeu de données `Digit` inclus dans Scikit-learn. Ce jeu de données contient la définition de 1797 images ainsi que leurs libellés. Chacune de ces images est définie par une matrice de 8x8 valeurs.

Nous commençons par le chargement de tous les modules nécessaires à la réalisation de notre exemple avec le code ci-après qui est saisi dans une cellule d'un notebook Jupyter. Vous pouvez également utiliser le notebook *multi_layer_perceptron_classifier* situé dans le dossier *Code* de ce chapitre :

```
from sklearn import datasets
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import matplotlib.colors
```

La première instruction `from sklearn import datasets` permet d'importer le module `datasets` où est défini, entre autres, le jeu de données `Digit`. La deuxième instruction importe la classe `MLPClassifier` à partir du module `sklearn.neural_network` qui va nous permettre de définir notre réseau de neurones.

© Editions ENI - All rights reserved

La fonction `accuracy_score` importée à partir de `sklearn.metrics` permet de mesurer la métrique de performance `accuracy` et les deux dernières instructions permettent d'importer les outils nécessaires pour réaliser des graphiques.

Le code suivant permet de charger le jeu de données `Digit` dans `data`.

```
data = datasets.load_digits()
dir(data)
```

L'instruction `dir(data)` permet d'afficher les propriétés du jeu de données `data`. Le résultat de cette instruction sera affiché comme suit :

```
['DESCR', 'data', 'feature_names', 'frame', 'images', 'target',
'target_names']
```

Les deux seules propriétés qui nous intéressent pour cet exemple sont les attributs suivants :

- `images` : contient 1797 matrices de 8x8 valeurs. Chaque matrice correspond à l'image d'un chiffre entre 0 et 9.
- `Target` : vecteur où les classes associées aux images sont définies. Par exemple `target[10]` va contenir le chiffre associé à l'image `images[10]`.

Remarque

Notez que tous les jeux de données de `sklearn.datasets` sont très intéressants, car non seulement ils permettent d'appliquer et d'apprendre les algorithmes du Machine Learning, mais en plus ces données sont propres et prêtes à être utilisées sans perdre de temps dans la recherche des valeurs manquantes et/ou aberrantes ou autres !

L'instruction suivante permet de confirmer que les 1797 images ont bien été chargées :

```
data.images.shape
```

Le résultat de cette instruction est le suivant :

```
(1797, 8, 8)
```

Également, l'instruction suivante permet de confirmer que l'attribut `target` contient bien les classes des images :

```
print(f"Il y a {len(data.target)} classes dans data.target =
{data.target}")
```

Cette instruction affiche le résultat suivant :

```
Il y a 1797 classes dans data.target = [0 1 2 ... 8 9 8]
```

Vous pouvez également accéder aux matrices associées aux images, comme le montre l'instruction suivante qui affiche la matrice associée à la toute première image :

```
print(data.images[0])
```

Les valeurs des pixels de la première image sont affichées ci-après :

```
[[ 0.  0.  5. 13.  9.  1.  0.  0.]
 [ 0.  0. 13. 15. 10. 15.  5.  0.]
 [ 0.  3. 15.  2.  0. 11.  8.  0.]
 [ 0.  4. 12.  0.  0.  8.  8.  0.]
 [ 0.  5.  8.  0.  0.  9.  8.  0.]
 [ 0.  4. 11.  0.  1. 12.  7.  0.]
 [ 0.  2. 14.  5. 10. 12.  0.  0.]
 [ 0.  0.  6. 13. 10.  0.  0.  0.]]
```

En analysant les différentes zones de cette matrice, nous pouvons percevoir l'allure du chiffre zéro. Justement, la classe associée à cette première image est la classe représentée avec le chiffre 0 que nous pouvons vérifier en affichant la valeur de `data.target[0]` qui est égale à 0 !

À titre indicatif, afin de visualiser certaines des images du jeu de données `Digit`, le code ci-dessous permet de construire un graphique de douze images :

```
def plot_digits(start):
    fig = plt.figure(figsize=(10,10))
    cmap = matplotlib.colors.ListedColormap(['red', 'black'])
    for im in range(12):
        plt.subplot(3,4,im+1)
        title = str(start+im) + ":" + str(data.target[start+im])
        plt.title(title)
        plt.imshow(data.images[start+im], cmap=cmap)
        plt.axis('off')
    plt.show()
```

© Editions ENI - All rights reserved

```
plot_digits(1000)
```

La fonction `plot_digits` affiche les douze images successives du data.images en commençant par la première image située à la position indiquée par `start` qui est le seul argument de cette fonction.

La dernière instruction de ce code est un appel à la fonction `plot_digits` avec comme argument la valeur `1000`. Le résultat de cet appel est le suivant :

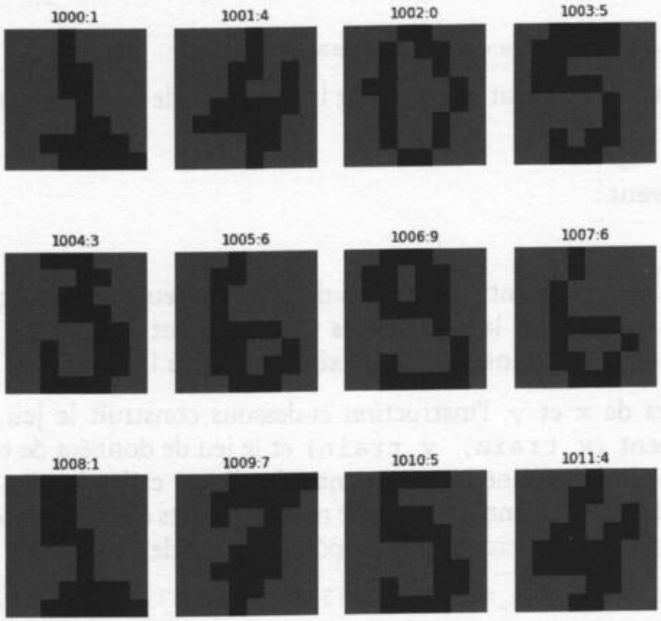


Figure 13-9 : les images situées entre l'indice `1000` et l'indice `1012` avec leurs libellés associés

Le graphique de la figure 13-9 affiche les douze images successives à partir de l'image `1000` en affichant également la position et le libellé de chacune de ces images.

Dans notre exemple, nous allons appliquer un réseau de neurones simple suivant la procédure de la rétropropagation. Dans un tel algorithme, chacun des nœuds de la première couche, ou de la couche d'entrée, reçoit une valeur.

Sachant que les données dont nous disposons dans notre exemple sont organisées sous forme de matrices, il est donc nécessaire d'aplatir ou de transformer ces matrices sous forme de vecteurs.

Le code suivant permet de réorganiser les images sous forme de tableaux. La première instruction permet de sauvegarder dans la variable y les classes associées aux images et la variable x permet de sauvegarder chacune des matrices dans un seul vecteur.

```
y = data.target
x = data.images.reshape((len(data.images), -1))
```

L'instruction suivante affiche tout simplement la structure de la variable x :

```
x.shape
```

Le résultat est le suivant :

```
(1797, 64)
```

Cela signifie que les données sont constituées de 1797 vecteurs avec chaque vecteur de taille 64. Pour rappel, la taille de ces vecteurs vient du fait que les matrices `data.images` sont constituées de 8x8=64 valeurs !

À partir des données de x et y, l'instruction ci-dessous construit le jeu de données d'entraînement `(x_train, y_train)` et le jeu de données de test `(x_test, y_test)`. Les données situées entre l'indice `0` et l'indice `1346` seront utilisées pour l'entraînement du réseau de neurones et les données situées entre les indices `1347` et `1796` seront utilisées pour la phase de validation.

```
x_train, y_train, x_test, y_test  = x[:1347], y[:1347], x[1347:],
y[1347:]
```

À ce stade, nous sommes prêts pour la création de notre réseau de neurones. À cet effet, le code suivant utilise la classe `MLPClassifier` afin d'instancier un réseau de neurones suivant les paramètres fournis en arguments.

```
mlp_classifier = MLPClassifier(hidden_layer_sizes=(20,),
activation='logistic',solver='sgd', tol=0.01,n_iter_no_change=30,
random_state=1, alpha=0.0001, learning_rate_init=.1,
verbose=True)
```

À l'exécution de ce dernier code, la variable `mlp_classifier` représente une instance de notre réseau de neurones.

© Editions ENI - All rights reserved

Les arguments utilisés pour la création de ce réseau de neurones sont définis comme suit :

- `hidden_layer_sizes` : ce paramètre est un tuple qui indique l'architecture du réseau de neurones. Dans notre exemple, nous avons positionné ce paramètre à `20` pour indiquer que le réseau de neurones que nous souhaitons mettre en place est constitué d'une couche cachée définie par vingt neurones. Cela signifie que notre réseau de neurones dans son ensemble sera constitué de trois couches au total : une couche d'entrée, la couche cachée définie par ce paramètre et la couche de sortie. Si nous voulions mettre une architecture avec, par exemple, trois couches cachées dont la première couche serait constituée de quarante neurones, la deuxième couche cachée constituée de dix neurones, puis une troisième couche cachée constituée de vingt neurones, alors ce paramètre serait fixé ainsi : `hidden_layer_sizes=(40,10,30)`. Il faut bien se rappeler que ce paramètre ne définit ni la couche d'entrée ni la couche de sortie.
- `activation` : ce paramètre indique la fonction d'activation à utiliser au sein des neurones des couches cachées. Nous pouvons choisir parmi les quatre fonctions suivantes :
  - `identity` : pour indiquer que la valeur à la sortie d'un neurone est la même qu'à l'entrée.
  - `logistic` : c'est la fameuse fonction logistique que nous avons vue au chapitre La régression logistique.
  - `tanh` : c'est la fonction tangente hyperbolique.
  - `relu` : pour *Rectified Linear* Unit qui est définie avec $\mathcal{F}(x) = \max(0, x)$.
- `solver` : ce paramètre indique la fonction à utiliser pour l'ajustement des poids du réseau de neurones. Cette fonction peut être l'une des trois fonctions `lbfgs`, `sgd` ou `adam`. Nous n'allons pas entrer dans les détails de ces fonctions, car chacune d'elles mériterait largement qu'on lui consacre un livre entier... au minimum ! Cependant, il faut retenir que ces méthodes font partie principalement des méthodes d'optimisation stochastiques issues du monde de la programmation mathématique.

- `tol` : ce paramètre doit être considéré conjointement avec le paramètre `n_iter_no_change`. La valeur de `tol` correspond à un seuil d'amélioration des erreurs sur les données d'entraînement. Si au bout de `n_iter_no_change` itérations successives du réseau de neurones le résultat des prédictions sur les données d'entraînement ne s'améliore pas d'une valeur supérieure ou égale à la valeur de `tol`, alors l'algorithme s'arrête et considère que le réseau de neurones a convergé.
- `n_iter_no_change` : comme expliqué précédemment, ce paramètre correspond au nombre d'itérations successives du réseau de neurones qui devrait améliorer la qualité des prédictions par au moins une valeur égale à `tol`.
- `random_state` : ce paramètre permet de fixer la première valeur du générateur de nombres aléatoires utilisé par l'algorithme afin de pouvoir reproduire les mêmes résultats lorsque l'algorithme est appliqué sur les mêmes données d'entraînement.
- `alpha` : ce paramètre correspond à un terme de régularisation de type L2.
- `learning_rate_init` : ce paramètre indique le taux d'apprentissage initial qui sera utilisé pour la mise à jour des poids du réseau de neurones.
- `verbose` : lorsqu'il est égal à `True`, ce paramètre indique notre souhait de visualiser les résultats au fur et à mesure des itérations du réseau de neurones.

L'instruction suivante permet de réaliser les itérations du réseau de neurones sur les données d'entraînement en utilisant la fameuse fonction `fit` :

```
mlp_classifier.fit(x_train,y_train)
```

À la fin de l'exécution de la fonction `fit`, nous pouvons utiliser le réseau de neurones afin de réaliser les prédictions sur les données de test pour ainsi évaluer son efficacité. La première instruction du code ci-dessous permet de prédire les classes associées aux données de test `x_test` en utilisant notre réseau de neurones et la deuxième instruction calcule la valeur de la métrique de performance `accuracy`.

```
predictions = mlp_classifier.predict(x_test)
accuracy_score(y_test, predictions)
```

© Editions ENI - All rights reserved

Le résultat est affiché ci-dessous :

```
0.9222222222222223
```

Ce résultat montre que notre réseau de neurones a été capable de prédire correctement à hauteur de 92.22 % les classes des données de test.

Bien sûr, ces résultats peuvent être améliorés en testant notre réseau de neurones avec plusieurs paramètres différents.

À titre de comparaison, les deux instructions suivantes affichent les vraies classes des images situées entre l'indice 10 et l'indice 19 ainsi que les classes prédites pour ces mêmes images :

```
print(f"     y_test[10:20] = {y_test[10:20]}")
print(f"predictions[10:20] = {predictions[10:20]}")
```

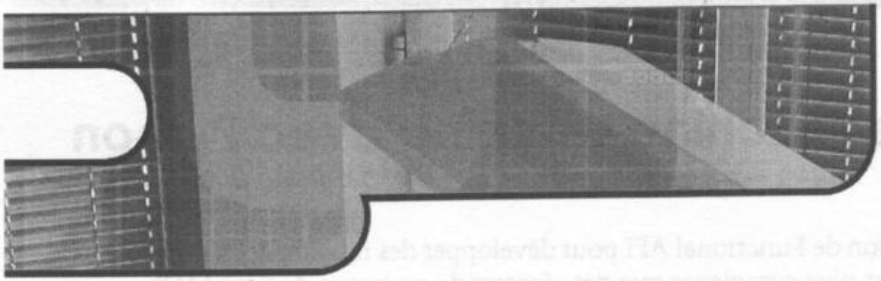
Le résultat est le suivant :

```
     y_test[10:20] = [1 5 0 9 5 2 8 2 0 0]
predictions[10:20] = [1 5 0 9 6 2 8 2 0 0]
```

Ces résultats nous montrent que sur ces dix images, il n'y a que l'image correspondant au chiffre 5 qui a été mal prédite !

6. Conclusion

Ce chapitre a introduit les concepts de base des réseaux de neurones. La structure formelle d'un neurone et la structure des réseaux de neurones ont été abordées dans un premier temps, puis l'algorithme de réseau de neurones avec rétropropagation a été expliqué à travers le déroulement d'un exemple complet d'une propagation avant et d'une propagation arrière de cet algorithme. Enfin, nous avons appliqué la bibliothèque Scikit-learn pour mettre en place un réseau de neurones pour la reconnaissance des chiffres à partir des images.



Partie 5
Le Deep Learning et le traitement automatique du langage

Chapitre 5-1
Le Deep Learning avec TensorFlow

1. Objectif du chapitre

Le chapitre précédent nous a permis d'aborder les concepts de base des réseaux de neurones. L'objectif de ce chapitre est double. D'une part, il va nous permettre d'aller un peu plus loin dans les concepts des réseaux de neurones tels que les réseaux de neurones à convolution ou le Transfer Learning, et d'autre part, il va nous permettre d'aborder la bibliothèque TensorFlow, qui est sans doute l'une des bibliothèques les plus utilisées pour implémenter les concepts sous-jacents des réseaux de neurones. Bien sûr, un seul chapitre ne va certainement pas être suffisant pour couvrir le sujet de Deep Learning dans sa globalité ni pour aborder tous les détails techniques de la bibliothèque TensorFlow. Le but est évidemment d'évoquer les concepts les plus fondamentaux du Deep Learning et les techniques les plus couramment utilisées avec TensorFlow de sorte que le lecteur, à la suite de ce chapitre, pourra sans difficulté aller plus loin dans sa formation traitant de ces deux sujets.

À la fin de ce chapitre, le lecteur aura abordé :

- une introduction aux concepts généraux du Deep Learning,
- une introduction à la bibliothèque TensorFlow 2,
- l'utilisation de Sequential API pour développer des réseaux de neurones de type MLP,

- l'utilisation de Functional API pour développer des réseaux de neurones plus évolués et plus complexes que des réseaux de neurones de type MLP,
- la gestion des critères d'arrêt de la phase d'apprentissage d'un réseau de neurones,
- l'utilisation de TensorBoard,
- les réseaux de neurones à convolution,
- l'exploitation d'un réseau de neurones déjà existant,
- le Transfer Learning.

2. Le Deep Learning : notions générales

Pendant longtemps, les chercheurs en réseaux de neurones ont cru qu'un réseau de neurones avec une couche d'entrée, une couche de sortie et une ou deux couches cachées suffirait à modéliser la plupart des phénomènes étudiés dans ce domaine. Mais avec la multiplicité et la fréquence croissante des sources de données, les tailles des bases d'apprentissage ont beaucoup augmenté ces dernières années, notamment lors de la dernière décennie. Avec ces données d'apprentissage plus importantes en matière de taille, les réseaux de neurones complexes composés de plusieurs couches cachées ont montré une efficacité telle qu'aujourd'hui, dans certains domaines, il est très difficile d'envisager de développer des modèles d'apprentissage automatique sans l'utilisation de ces réseaux de neurones multicouches. En effet, les réseaux de neurones constitués de plusieurs couches cachées sont devenus des champions jamais égalés dans des domaines tels que la reconnaissance d'objets dans des images et la reconnaissance vocale.

Traiter le sujet du Deep Learning dans sa globalité avec toutes les versions techniques possibles proposées dans la littérature est quasiment impossible. Rien que l'évocation des grandes lignes des principes de ce domaine nécessiterait tout un ouvrage dédié en plusieurs tomes! Cependant, dans ce chapitre, nous allons mettre en pratique les principes les plus répandus en matière de Deep Learning via des exemples en utilisant TensorFlow.

Mais avant d'aborder les détails techniques d'implémentation des réseaux de neurones complexes, cette section nous permet de passer en revue certains des principes les plus élémentaires du Deep Learning.

© Editions ENI - All rights reserved

Lorsque nous parlons de Deep Learning, ou de l'apprentissage profond, il s'agit bien de réseaux de neurones constitués d'une ou plusieurs couches d'entrée, d'une ou plusieurs couches de sortie et de deux ou plusieurs couches cachées.

La figure suivante illustre l'architecture d'un réseau de neurones à plusieurs couches cachées.

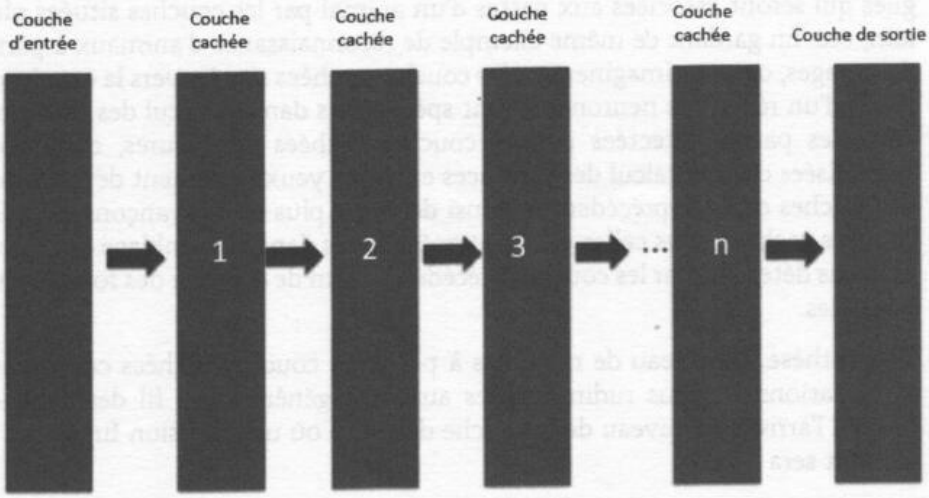


Figure 14-01 : schéma général d'un réseau de neurones dit de Deep Learning

Dans un tel réseau de neurones, les couches se succèdent les unes après les autres de manière linéaire tel que nous l'avons vu lorsque nous avons abordé les réseaux de neurones de type MLP (*Multilayer Perceptron*). Cependant, dans les réseaux de neurones de type Deep Learning, dont les MLP font partie, les couches peuvent ne pas être empilées les unes derrière les autres de façon linéaire Comme nous allons le découvrir dans ce chapitre, les couches d'un Deep Learning peuvent être connectées suivant certaines conditions, ou connectées de façon cyclique, etc.

Dans un réseau de neurones dit de Deep Learning, nous retrouvons généralement un nombre de couches cachées supérieur à deux. Chacune des couches cachées d'un Deep Learning sera spécialisée dans la résolution d'une partie du problème à solutionner.

Par exemple, pour un problème de reconnaissance des animaux à partir des images, on peut imaginer que dans un réseau de neurones semblable à celui de la figure ci-dessus, les premières couches cachées seront spécialisées dans la détection de formes circulaires et/ou ovales. Ces formes seront associées aux yeux dans les couches cachées plus loin dans ce réseau de neurones. Également, les premières couches cachées peuvent détecter des formes fines et longues qui seront associées aux pattes d'un animal par les couches situées plus loin, etc. En gardant ce même exemple de reconnaissance d'animaux à partir des images, on peut imaginer que les couches cachées situées vers la couche de sortie d'un réseau de neurones seront spécialisées dans le calcul des distances entre les pattes détectées par les couches cachées antérieures, ou encore spécialisées dans le calcul des distances entre les yeux également détectés par les couches cachées précédentes. Ainsi de suite, plus nous avançons dans les couches cachées, plus celles-ci seront spécialisées dans l'assemblage des informations détectées par les couches précédentes afin de détecter des formes plus générales.

En synthèse, un réseau de neurones à plusieurs couches cachées capture les informations des plus rudimentaires aux plus générales au fil des couches jusqu'à l'arrivée au niveau de la couche de sortie où une décision finale sur le résultat sera prise.

Bien sûr, pour un problème donné, il ne suffit pas d'augmenter le nombre de couches cachées pour augmenter les performances d'un réseau de neurones. Généralement, le fait d'augmenter le nombre de couches cachées d'un réseau de neurones de manière aléatoire ne fait qu'augmenter la redondance des informations déjà capturées par des couches situées au début de ce réseau, voire pire, la multiplicité arbitraire des couches cachées peut détériorer les performances d'un réseau de neurones. En effet, plus les données traversent de couches, plus le risque est élevé de perdre le sens de certaines informations véhiculées dans ces données.

Nous pouvons imaginer n'importe quelle architecture pour assembler et connecter les différentes couches qui composent un réseau de neurones. Cependant, il existe certaines architectures et certains types de couches étudiés dans la littérature qu'il convient de connaître. Loin d'être exhaustives, les quelques sous-sections suivantes évoquent certains des principes très répandus lorsqu'il s'agit des réseaux de neurones.

© Editions ENI - All rights reserved

Nous n'allons pas entrer dans les détails techniques de ces notions qui seront juste abordées dans l'objectif de prendre conscience de leur existence et ainsi comprendre les exemples de code qui seront examinés plus loin dans ce chapitre.

2.1 Réseau de neurones avec plusieurs couches d'entrée

Comme évoqué ci-dessous, au fur et à mesure que les données traversent les couches d'un réseau de neurones, le risque de perdre le sens de certaines informations cachées dans les données initiales est élevé. En effet, les données à la sortie d'une couche donnée ne correspondent pas aux données à son entrée. De ce fait, si certaines informations présentes dans les données initiales sont capitales pour la prise de décision par rapport au problème à résoudre, alors il serait dommage de ne pas pouvoir exploiter ces informations le moment venu. Pour pallier ce problème, nous pouvons imaginer un réseau de neurones composé de plusieurs couches d'entrée, comme l'exemple de la figure ci-dessous, de sorte que les données initiales seront injectées non seulement dans la couche d'entrée, mais aussi dans certaines couches cachées situées plus loin dans le réseau.

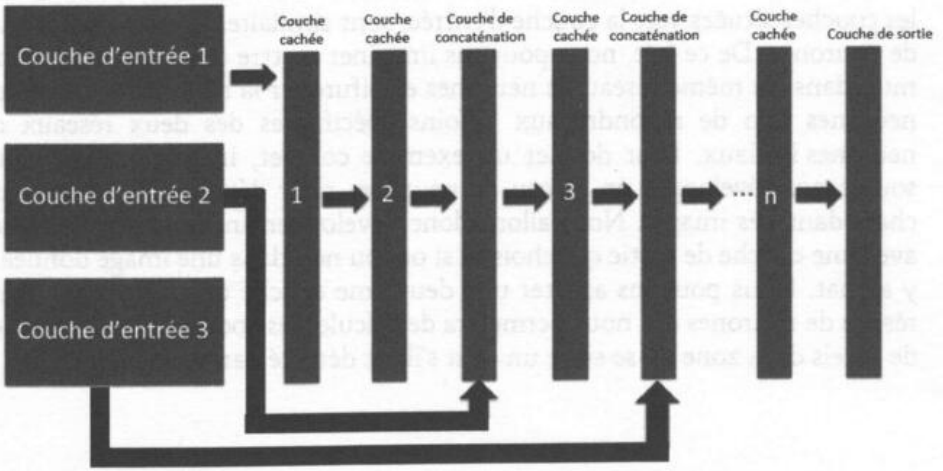


Figure 14-02 : réseau de neurones avec plusieurs couches d'entrée

Bien sûr, nous pouvons imaginer de réinjecter les mêmes données initiales utilisées au niveau de la couche d'entrée dans des couches situées plus loin, ou bien nous pouvons récupérer une partie des données injectées dans la couche d'entrée et les injecter dans une autre couche. Également, en fonction des besoins, nous pouvons injecter des données complètement différentes entre celles utilisées au niveau de la couche d'entrée et celles qui seront injectées directement dans des couches situées devant la couche d'entrée.

Ainsi, dans l'exemple de la figure ci-dessus, la couche cachée numéro 3 va recevoir des données issues de la sortie de la couche cachée numéro 2, mais aussi des données issues de la couche d'entrée numéro 2 non traitées par aucune couche du réseau.

2.2 Réseau de neurones avec plusieurs couches de sortie

De la même manière que des réseaux de neurones peuvent avoir plusieurs couches d'entrée, un réseau de neurones peut avoir plusieurs couches de sortie, comme le montre l'exemple de la figure suivante.

Parmi les besoins qui peuvent nous amener à utiliser plusieurs couches de sortie, c'est celui où les premières couches d'un réseau de neurones, c'est-à-dire les couches situées vers la couche d'entrée, sont similaires entre deux réseaux de neurones. De ce fait, nous pouvons imaginer mettre ces couches en commun dans un même réseau de neurones et bifurquer la suite de ce réseau de neurones afin de répondre aux besoins spécifiques des deux réseaux de neurones initiaux. Pour donner un exemple concret, imaginons que nous souhaitons développer un réseau de neurones pour détecter la présence de chats dans des images. Nous allons donc développer un réseau de neurones avec une couche de sortie qui choisira si oui ou non dans une image donnée il y a chat. Nous pouvons ajouter une deuxième couche de sortie à ce même réseau de neurones qui nous permettra de calculer les coordonnées en termes de pixels de la zone où se situe un chat s'il est détecté dans une image.

© Editions ENI - All rights reserved

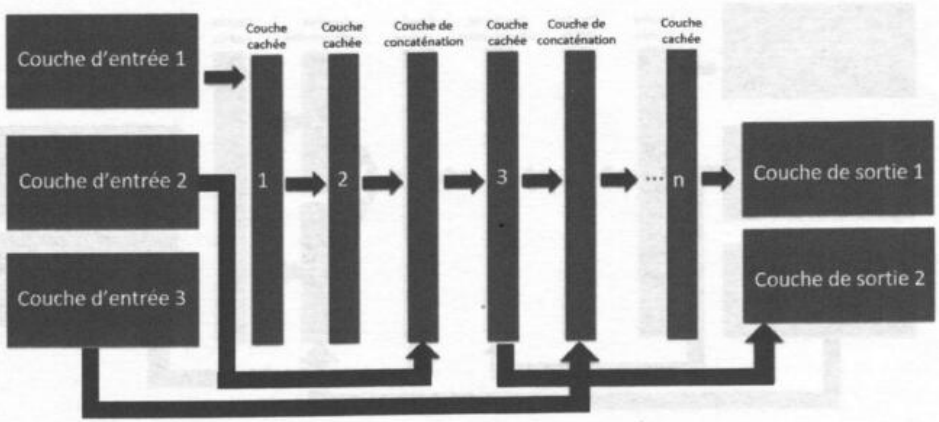


Figure 14-03 : réseau de neurones avec plusieurs couches d'entrée et plusieurs couches de sortie

2.3 Réseau de neurones avec des branchements conditionnels

Même si son utilisation reste rare et uniquement pour répondre à des besoins spécifiques, dans un réseau de neurones nous pouvons connecter les couches entre elles en fonction de la survenue de certains événements. Dans l'exemple de la figure suivante, la couche cachée numéro 3 se connecte à la couche cachée numéro 4 ou la couche cachée numéro 5 en fonction d'une condition placée à la sortie de cette couche cachée numéro 3.

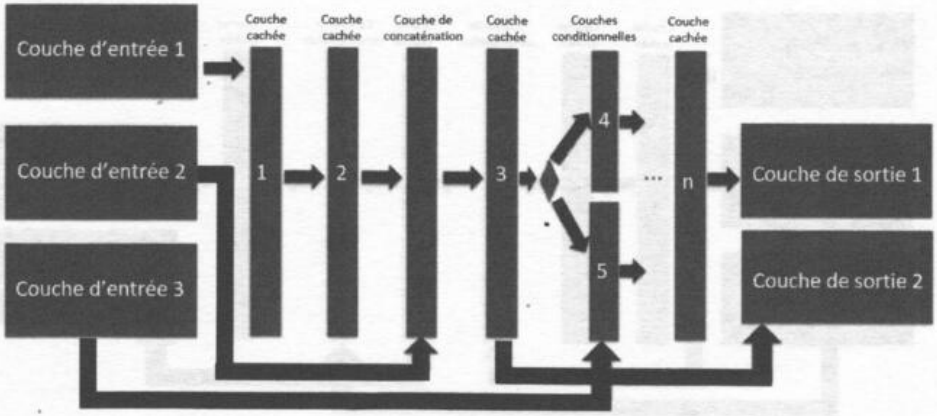


Figure 14-04 : réseau de neurones avec des branchements conditionnels

2.4 Réseau de neurones avec de la récurrence RNN

Également, on peut retrouver des connexions entre les couches d'un réseau de neurones qui forment un cycle (voir la figure ci-dessous). Ce type de branchement en boucle est souvent rencontré lorsqu'on traite des séries temporelles avec des réseaux de neurones de type RNN (*Recurrent Neural Network*). En effet, avec les séries temporelles, les valeurs futures sont liées aux valeurs passées. Dans ce cas, les boucles d'un réseau de neurones permettent de réutiliser les prédictions réalisées sur des données pour calculer les futures prédictions !

© Editions ENI - All rights reserved

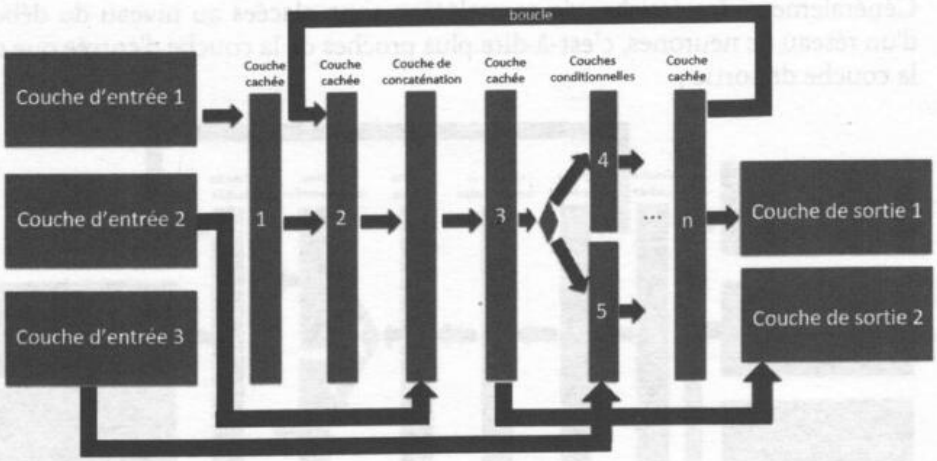


Figure 14-05 : réseau de neurones avec des cycles

2.5 Réseau de neurones avec couches de convolution CNN

Les couches de convolution sont sans doute la technique qui a contribué le plus à hisser les réseaux de neurones parmi les technologies les plus efficaces dans le domaine de l'apprentissage automatique ; du moins, dans le domaine de la reconnaissance des objets à partir des images. Un réseau de neurones convolutif ou en anglais *Convolutional Neural Network* (CNN) est un réseau de neurones qui contient au moins une couche de convolution.

Une couche à convolution consiste en une couche qui va appliquer des filtres sur les données qu'elle reçoit à son entrée afin de détecter des informations élémentaires. Par exemple, dans un réseau de neurones de reconnaissance des chiffres manuscrits, une couche de convolution recherche dans une image donnée la présence de cercles, afin de détecter des 0 ou une partie de l'un des chiffres 8 et 9, ou bien cette couche de convolution cherche la présence de formes d'arcs afin de déceler la présence du chiffre 2 ou du chiffre 3, etc.

Une couche de convolution permet donc de rechercher des informations très élémentaires qui seront exploitées par les couches devant elle afin de détecter des informations plus générales.

Généralement, les couches de convolution sont placées au niveau du début d'un réseau de neurones, c'est-à-dire plus proches de la couche d'entrée que de la couche de sortie :

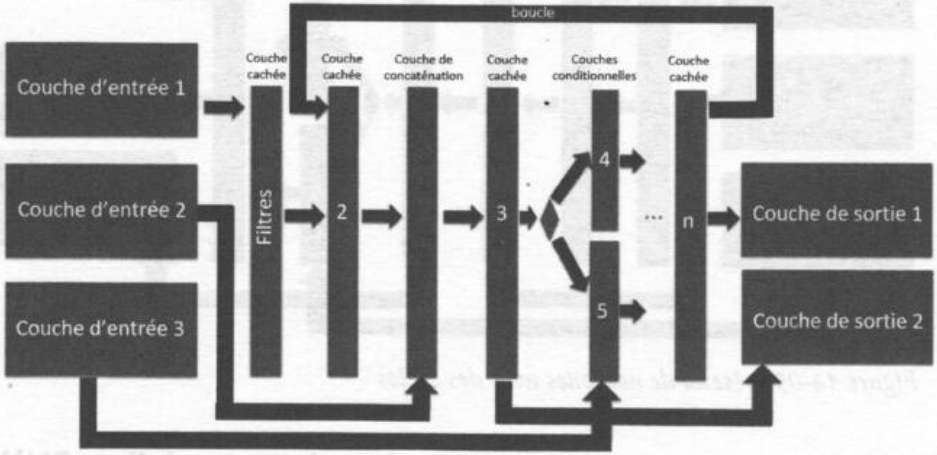


Figure 14-06 : réseau de neurones de convolution

Dans le cadre de la reconnaissance d'objets dans des images, les couches de convolution apportent une solution au problème liée à la position des objets recherchés dans les images. En effet, un des problèmes les plus fréquents lorsqu'on travaille dans le domaine de la reconnaissance des objets vient du fait que généralement les images utilisées pour l'entraînement d'un réseau de neurones sont centrées par rapport aux objets en question. De ce fait, lors de l'application d'un tel réseau de neurones pour la classification d'une image donnée, si l'objet recherché se présente dans une zone située vers les bords de cette image et non pas vers le centre, alors cela peut provoquer des erreurs de classification, car notre réseau de neurones focalisera la recherche des objets dans le centre des images comme il a été entraîné à le faire.

Le même problème que le précédent est rencontré par rapport à l'orientation des objets dans les images. Si lors de l'entraînement d'un réseau de neurones nous utilisons uniquement des objets orientés horizontalement, alors, pendant les prédictions, si un objet est présent dans une image mais orienté autrement qu'horizontalement, il peut ne pas être détecté.

© Editions ENI - All rights reserved

Les filtres appliqués par les couches de convolution permettent justement de scruter une image à la recherche d'informations élémentaires, quelles que soient leur position et leur orientation.

Plus concrètement, dans le cadre de la reconnaissance d'objets, un filtre est une matrice de petite taille associée à une forme élémentaire donnée. Cette matrice sera utilisée pour balayer une image de la gauche vers la droite et du haut vers le bas afin de déceler la présence de la forme élémentaire associée à ce filtre dans cette image.

2.6 Éviter le surapprentissage avec les couches Dropout

Le Dropout est une technique simple et efficace qui permet potentiellement d'éviter le problème de surapprentissage pour un réseau de neurones. Cette technique, qui peut être utilisée à n'importe quel niveau d'un réseau de neurones (voir la figure suivante), consiste à désactiver certains poids d'une couche donnée au moment où les données traversent cette couche.

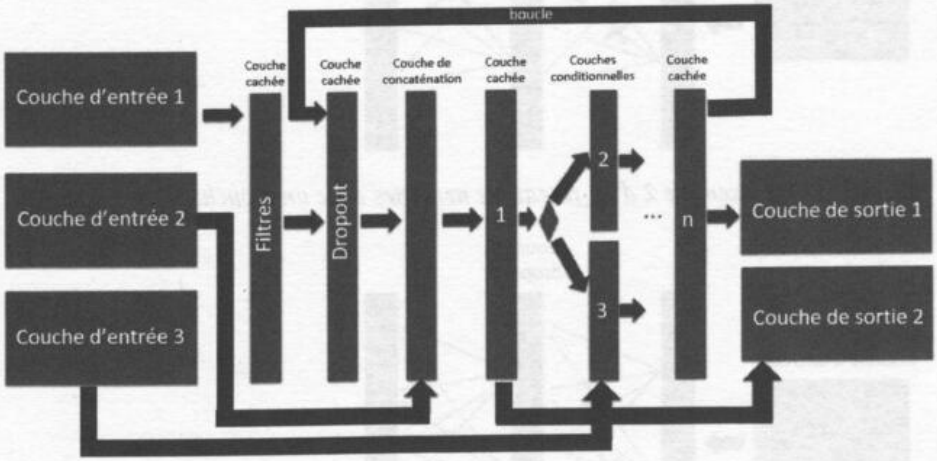


Figure 14-07 : réseau de neurones avec une couche Dropout

Les trois figures suivantes montrent un exemple d'application d'une couche de type Dropout en désactivant à chaque fois deux poids parmi les résultats de la couche précédente.

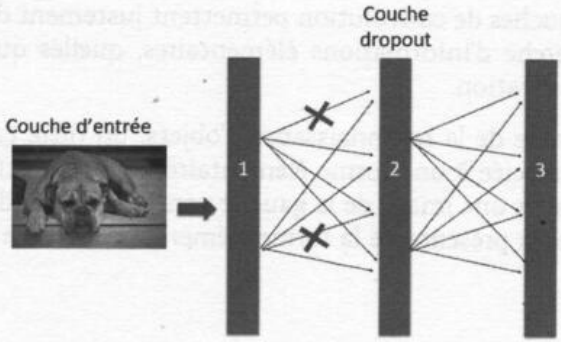


Figure 14-08 : exemple 1 d'un réseau de neurones avec une couche Dropout

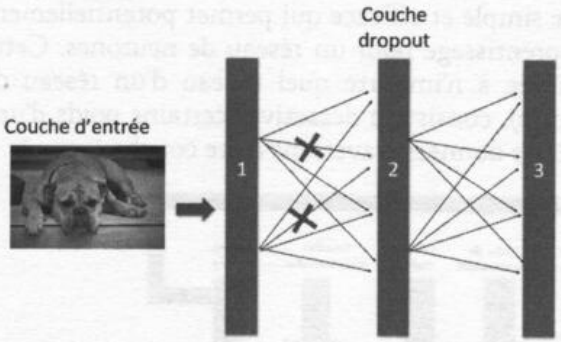


Figure 14-09 : exemple 2 d'un réseau de neurones avec une couche Dropout

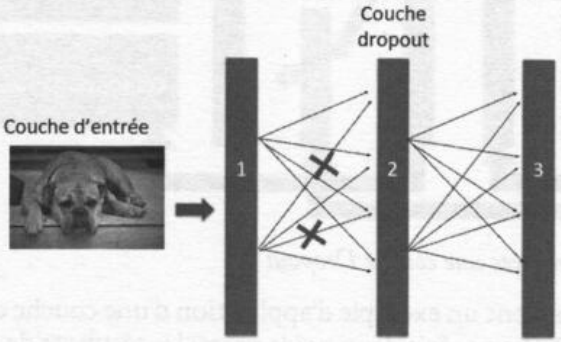


Figure 14-10 : exemple 3 d'un réseau de neurones avec une couche Dropout

© Editions ENI - All rights reserved

Une couche Dropout désactive de manière aléatoire un taux de poids parmi les sorties de la couche qui la précède. Ce taux est tout simplement un paramètre propre à chaque couche de type Dropout. Ainsi, une fois ce paramètre fixé, à chaque fois qu'une donnée traverse le réseau de neurones, la couche Dropout choisit les poids à désactiver de manière aléatoire. Bien sûr, les poids à désactiver sont différents d'une propagation avant à une autre.

L'idée derrière la technique de Dropout est de pousser un maximum de poids à participer à la phase d'apprentissage et d'éviter d'atteindre rapidement des points de convergence.

2.7 Le Transfer Learning

Le Transfer Learning, ou l'apprentissage par transfert, est une technique très répandue dans le domaine des réseaux de neurones. Il s'agit tout simplement de réutiliser une partie d'un réseau de neurones déjà existant et qui est spécialisé dans la résolution d'un problème donné pour résoudre un autre problème.

Pour comprendre le concept du Transfer Learning, supposons que nous souhaitons résoudre un problème d'apprentissage automatique donné et que la résolution de ce problème passe par la résolution chronologique des deux sous-problèmes A et B (autrement dit, nous ne pouvons pas résoudre le sous-problème B avant d'avoir résolu le sous-problème A). Pour résoudre notre problème, on peut procéder à la construction d'un réseau de neurones dont les premières couches vont réaliser un apprentissage pour la résolution du problème A et les couches suivantes vont apprendre à résoudre le problème B. Supposons maintenant que nous disposons d'un réseau de neurones qui est déjà entraîné pour résoudre un problème composé des deux sous-problèmes A et C, c'est-à-dire que les premières couches de ce réseau de neurones sont déjà entraînées pour résoudre le problème A et les couches suivantes sont entraînées pour résoudre le problème C. La technique du Transfer Learning propose d'utiliser les premières couches du réseau de neurones déjà existant, soit les couches qui savent résoudre le problème A, et d'ajouter à ces couches existantes des nouvelles couches qui vont résoudre le problème B. De cette sorte, nous n'aurions pas à entraîner un réseau de neurones pour la résolution du problème A !

Les six figures suivantes illustrent schématiquement les étapes globales à réaliser afin d'exploiter le concept de Transfer Learning pour résoudre un problème composé de deux sous-problèmes.

La première étape consiste à charger en mémoire le réseau de neurones déjà existant.

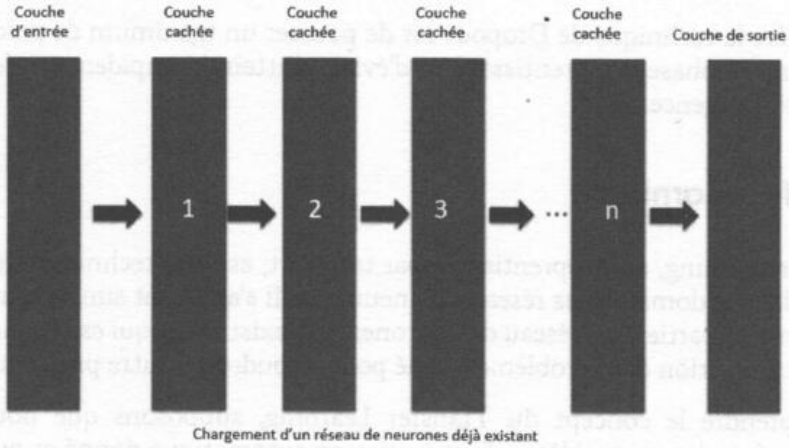


Figure 14-11 : étape 1 - procédure du Transfer Learning

La deuxième étape consiste à supprimer les couches les plus proches de la couche de sortie et qui ne sont pas utiles pour le problème que nous souhaitons résoudre. En d'autres termes, nous n'allons garder que les couches entraînées pour la résolution du premier sous-problème.

© Editions ENI - All rights reserved

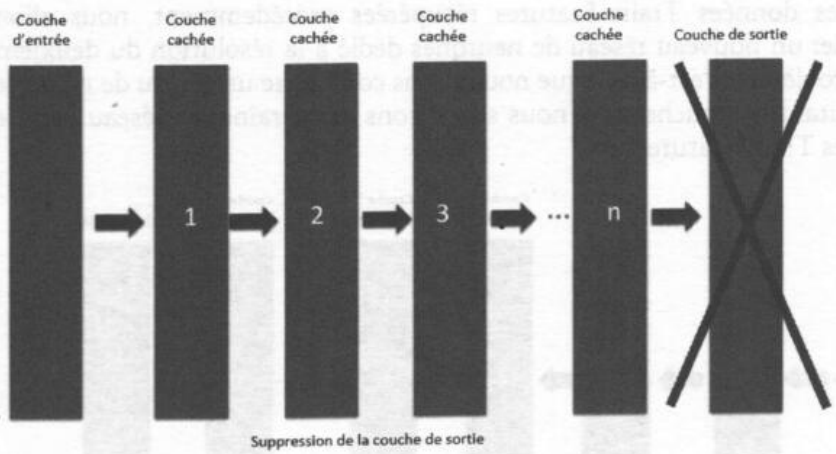


Figure 14-12 : étape 2 - procédure du Transfer Learning

La troisième étape consiste à utiliser les données à notre disposition afin de récupérer les informations extraites avec les couches utiles du réseau de neurones chargé précédemment. Ces informations extraites vont constituer les données d'entrée pour les couches que nous allons ajouter et qui seront spécialisées dans la résolution du deuxième sous-problème.

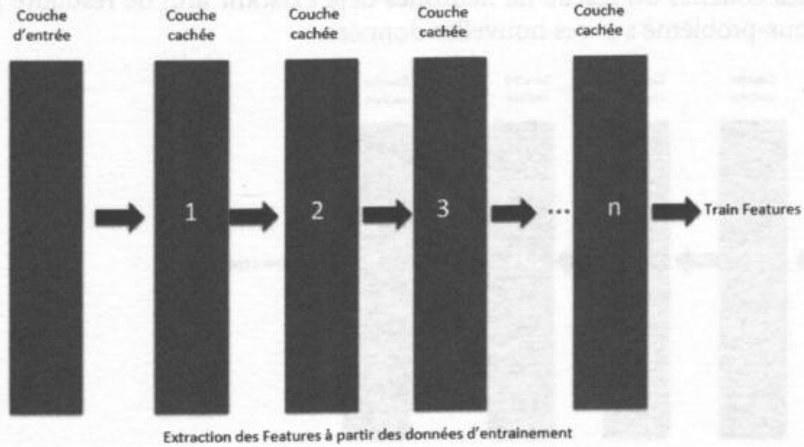


Figure 14-13 : étape 3 - procédure du Transfer Learning

Avec les données Train Features récupérées précédemment, nous allons entraîner un nouveau réseau de neurones dédié à la résolution du deuxième sous-problème ; c'est-à-dire que nous allons construire un réseau de neurones avec autant de couches que nous souhaitons et entraîner ce réseau avec les données Train Features.

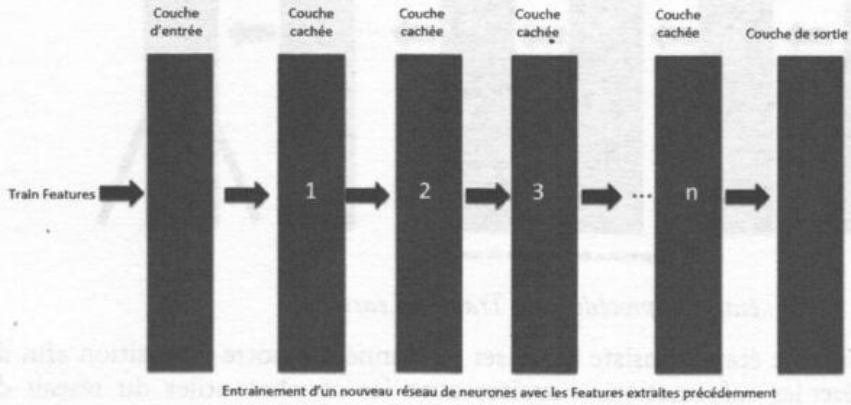


Figure 14-14 : étape 4 - procédure du Transfer Learning

Pour réaliser des prédictions avec de nouvelles données, nous devons d'abord réutiliser les couches du réseau de neurones déjà existant afin de résoudre le premier sous-problème sur ces nouvelles données.

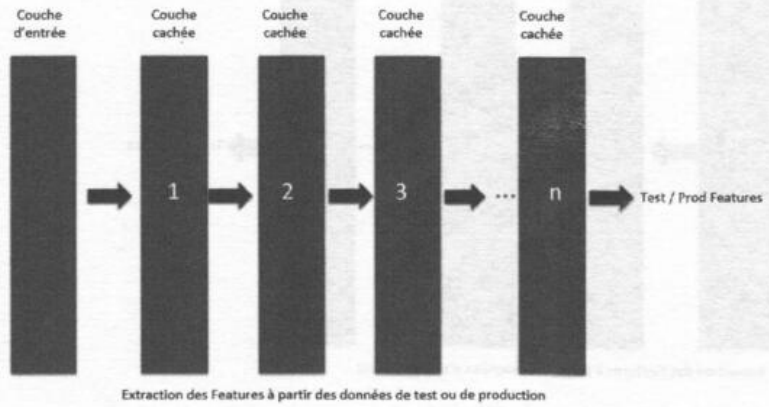


Figure 14-15 : étape 5 - procédure du Transfer Learning

© Editions ENI - All rights reserved

Avec les données extraites en utilisant les premières couches du réseau de neurones déjà existant, nous pouvons maintenant résoudre le deuxième sous-problème en utilisant le nouveau réseau de neurones.

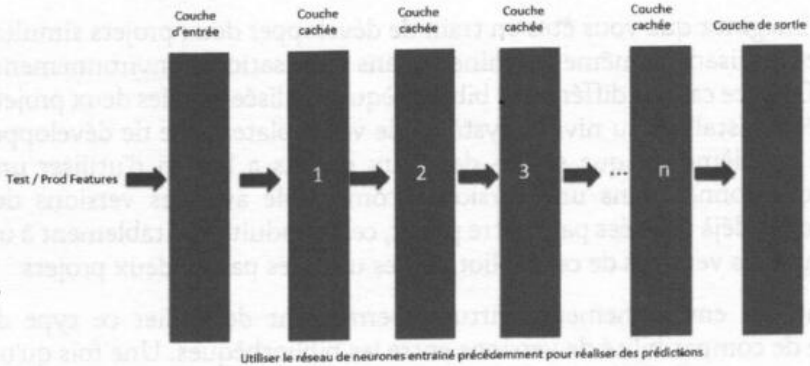


Figure 14-16 : étape 6 - procédure du Transfer Learning

Si ces étapes théoriques utilisées pour expliquer le Transfer Learning ne sont pas claires, sachez qu'un exemple concret et complet sera développé dans la suite de ce chapitre.

3. Introduction à TensorFlow

3.1 Installer TensorFlow

TensorFlow peut être installé en utilisant la commande `pip install` classique qui permet d'ajouter de nouvelles bibliothèques dans votre environnement de travail. Cependant, en procédant ainsi, vous risquez de vous retrouver avec un environnement dans lequel certaines des bibliothèques déjà installées seraient incompatibles avec la version de TensorFlow ainsi installée.

À notre connaissance, la méthode d'installation de TensorFlow la plus simple et la plus sûre est d'utiliser un environnement virtuel Python conjointement avec Anaconda. Les deux sous-sections suivantes montrent comment créer un environnement virtuel Python et comment y installer TensorFlow.

3.1.1 Créer un environnement virtuel

Lorsque nous développons nos projets avec Python, il est fortement recommandé d'utiliser les environnements virtuels.

En effet, imaginez que vous êtes en train de développer deux projets simultanément en utilisant la même machine et sans l'utilisation d'environnements virtuels. Dans ce cas, les différentes bibliothèques utilisées par les deux projets doivent être installées au niveau système de votre plateforme de développement. Le problème est que si l'un des deux projets a besoin d'utiliser une bibliothèque donnée dans une version incompatible avec les versions des bibliothèques déjà utilisées par l'autre projet, cela conduit inévitablement à un conflit entre les versions de ces bibliothèques utilisées par les deux projets.

Justement, les environnements virtuels permettent de pallier ce type de problème de compatibilité de versions entre les bibliothèques. Une fois qu'un environnement virtuel est créé, il sera utilisé comme environnement de travail pour les projets utilisant les mêmes versions de bibliothèques.

Ainsi, les bibliothèques nécessaires pour un projet donné seront installées dans un environnement virtuel et ces bibliothèques ne seront visibles que par les projets développés dans ce même environnement virtuel.

Nous recommandons vivement de créer un environnement virtuel pour chaque projet. Ainsi, si nous avons deux projets en cours de développement, nous allons créer deux environnements virtuels différents, un pour chaque projet. De cette façon, si l'un des deux projets a besoin d'utiliser une bibliothèque dans une version incompatible avec des bibliothèques déjà utilisées par l'autre projet, cela ne cause aucun problème puisque cette bibliothèque sera installée dans un environnement virtuel dédié.

Pour créer un nouvel environnement virtuel, nous allons utiliser une commande Anaconda. Pour rappel, les étapes d'installation d'Anaconda ont été abordées au tout début du chapitre Travailler avec Jupyter. Suivez les étapes ci-dessous pour créer à un nouvel environnement virtuel avec la version 3.6 de Python.

© Editions ENI - All rights reserved

- Créez un nouvel environnement virtuel en exécutant la commande suivante via l'invite de commandes Anaconda (voir la figure 5-3 du chapitre Travailler avec Jupyter) :

```
conda create -name TensorFlow_Learning python=3.6
```

Nous créons cet environnement virtuel avec la version 3.6 de Python car, à notre connaissance, les versions supérieures peuvent présenter des problèmes de compatibilité avec TensorFlow.

À la suite de l'exécution de cette commande, Anaconda vous demande de confirmer la création de l'environnement virtuel **TensorFlow_Learning**. À la confirmation de la création de cet environnement virtuel, vous devriez voir le message suivant affiché dans votre console Anaconda :

```
Downloading and Extracting Packages
pip-21.1.3           | 1.8 MB    |
############################################################| 100%
sqlite-3.36.0        | 780 KB    |
############################################################| 100%
certifi-2021.5.30    | 140 KB    |
############################################################| 100%
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate TensorFlow_Learning
#
# To deactivate an active environment, use
#
#     $ conda deactivate
```

Lancez **Anaconda Navigator** comme sur la figure suivante :

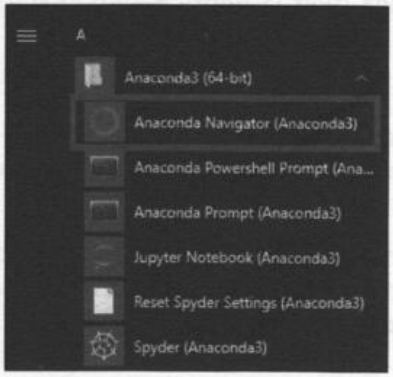


Figure 14-17 : lancement d'Anaconda Navigator

Après le lancement d'Anaconda Navigator, vous devriez voir la page suivante affichée :

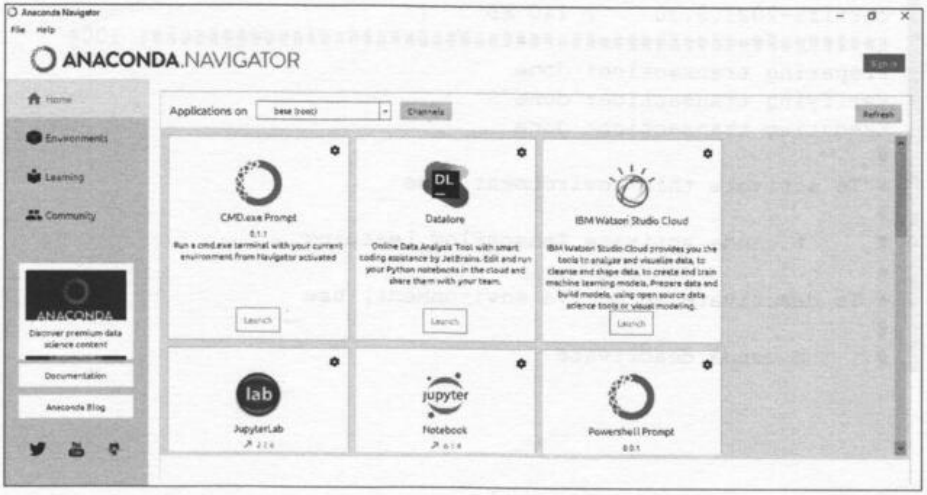


Figure 14-18 : page d'accueil d'Anaconda Navigator

La page d'accueil d'Anaconda Navigator est positionnée par défaut sur l'environnement virtuel base (root), qui est l'environnement virtuel créé automatiquement lors de l'installation d'Anaconda.

© Editions ENI - All rights reserved

Dans les chapitres précédents, à chaque fois que nous avons utilisé l'application Jupyter, elle a été lancée dans l'environnement base (root) qui était le seul environnement virtuel existant dans votre environnement.

- Cliquez sur le lien **Environments** sur le panneau à gauche de la page d'accueil d'Anaconda. Sur la nouvelle page qui s'affiche, sélectionnez votre nouvel environnement virtuel **TensorFlow_Learning** situé sur le panneau au centre de cette page, juste en dessous de l'environnement **base (root)**. Ce lien vers l'environnement virtuel nouvellement créé est encadré en rouge sur la figure ci-dessous :

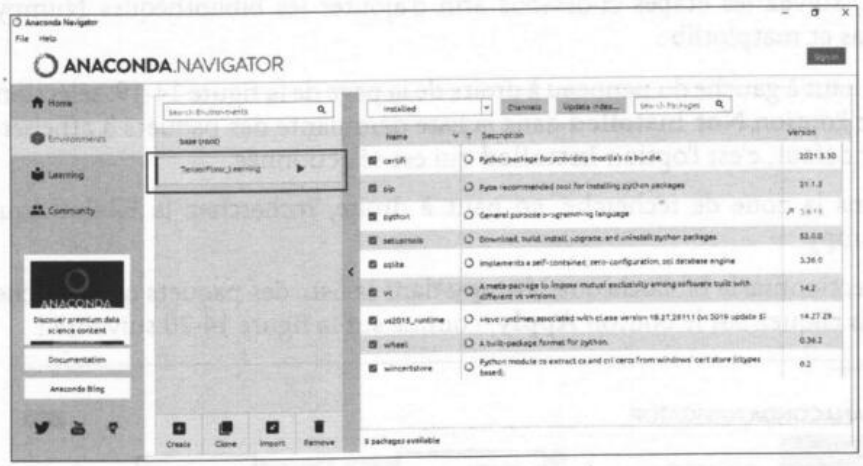


Figure 14-19 : l'environnement virtuel TensorFlow_Learning

Le grand panneau situé à droite de cette page correspond à la liste des paquets installés par défaut lors de la création de l'environnement virtuel **TensorFlow_Learning** avec la version 3.6 de Python.

Remarque

Pour rappel, dans le chapitre Le langage Python nous avons installé la dernière version en date de Python, qui était la version 3.9. Cette version est associée au compte système de votre plateforme. En revanche, la version de Python installée dans notre environnement virtuel ***TensorFlow_Learning*** *est associée uniquement à cet environnement, ce qui veut dire que sur votre ordinateur les deux versions de Python (les versions 3.9 et 3.6) existent et il n'y a aucune interaction entre elles.*

Remarque

*L'environnement virtuel **TensorFlow_Learning** a été installé avec la version 3.6 de Python car, pour l'instant, cette version de Python est préconisée lorsque nous utilisons TensorFlow et elle évite de nombreux problèmes de compatibilité.*

3.1.2 Installer des bibliothèques dans un environnement virtuel avec Anaconda

L'installation de nouvelles bibliothèques avec Anaconda Navigator est triviale. Ainsi, suivez les étapes ci-dessous afin d'ajouter les bibliothèques Numpy, pandas et matplotlib :

- En haut à gauche du panneau à droite de la page de la figure 14-19, sélectionnez l'option **Not installed** dans la liste déroulante des paquets à afficher ; par défaut, c'est l'option **Installed** qui est sélectionnée.
- Dans la zone de recherche, en haut à droite, recherchez la bibliothèque Numpy.
- Sélectionnez la bibliothèque **Numpy** dans la liste des paquets qui s'affiche, puis cliquez sur le bouton **Apply**, comme sur la figure 14-20 suivante :

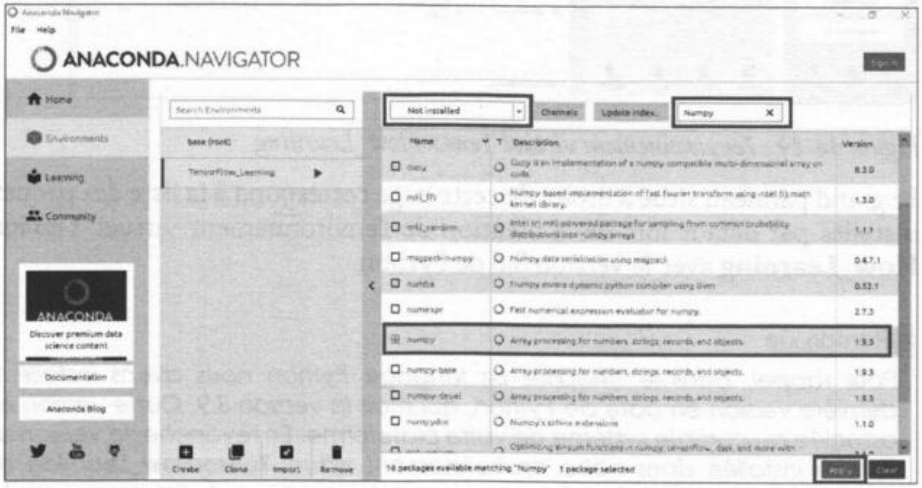


Figure 14-20 : installation de la bibliothèque Numpy

© Editions ENI - All rights reserved

- Lorsque Anaconda vous demande de confirmer l'installation des dépendances, cliquez sur le bouton **Apply**.
- De retour sur la page d'installation des paquets, pour vérifier si la bibliothèque Numpy est bien installée, sélectionnez l'option **Installed** dans la liste déroulante du panneau à droite. Vous devriez constater que la bibliothèque Numpy est installée comme sur la figure ci-dessous :

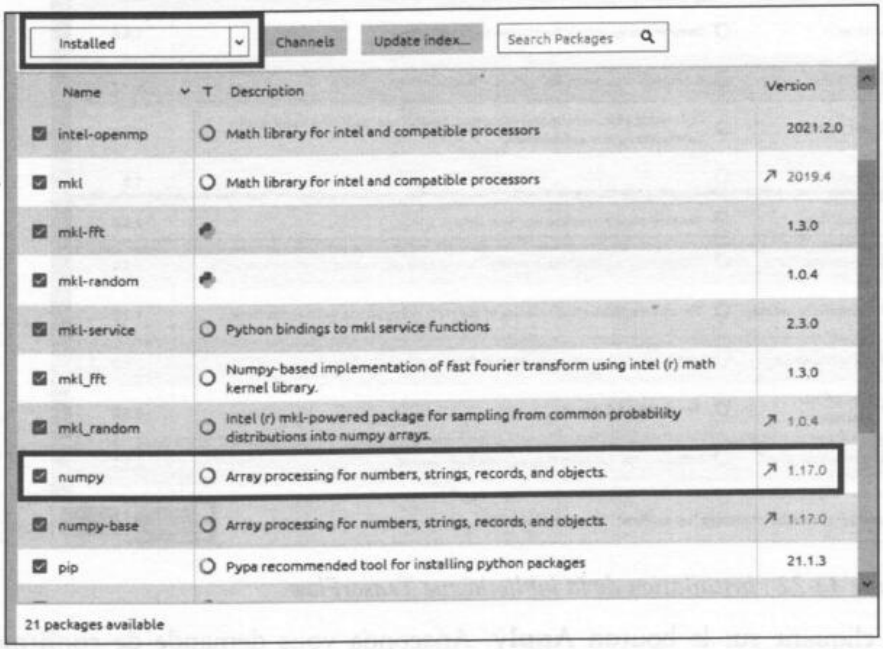


Figure 14-21 : la bibliothèque Numpy installée

- Effectuez de nouveau les étapes précédentes afin d'installer les bibliothèques Pandas, matplotlib, scikit-learn et leurs dépendances.

3.1.3 Installer la bibliothèque TensorFlow

Pour l'installation de la bibliothèque TensorFlow, suivez exactement les mêmes étapes que les étapes d'installation des bibliothèques précédentes :

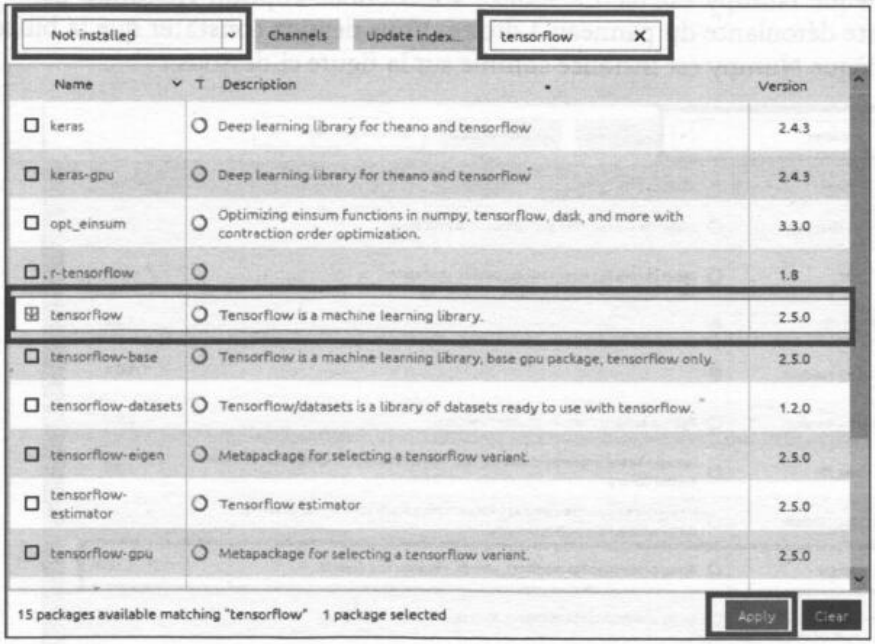


Figure 14-22 : installation de la bibliothèque TensorFlow

En cliquant sur le bouton **Apply**, Anaconda vous demande de confirmer l'installation des dépendances nécessaires. Une fois TensorFlow installé, vous êtes prêt à développer vos réseaux de neurones avec cette bibliothèque !

Remarque

Nous ne pouvons qu'insister sur le fait que toute autre méthode d'installation de TensorFlow outre celle évoquée ici peut facilement aboutir à des problèmes de compatibilité. En effet, lorsque nous saisissons le nom d'une bibliothèque à installer dans la zone de recherche, Anaconda nous propose uniquement d'installer les bibliothèques dont les versions sont compatibles avec les bibliothèques déjà installées dans notre environnement.

© Editions ENI - All rights reserved

3.1.4 Tester TensorFlow

- Sur la page **Home** d'Anaconda, sélectionnez l'environnement **TensorFlow_Learning**, puis installez l'application Jupyter en cliquant sur le bouton **Install**, comme sur la figure ci-dessous :

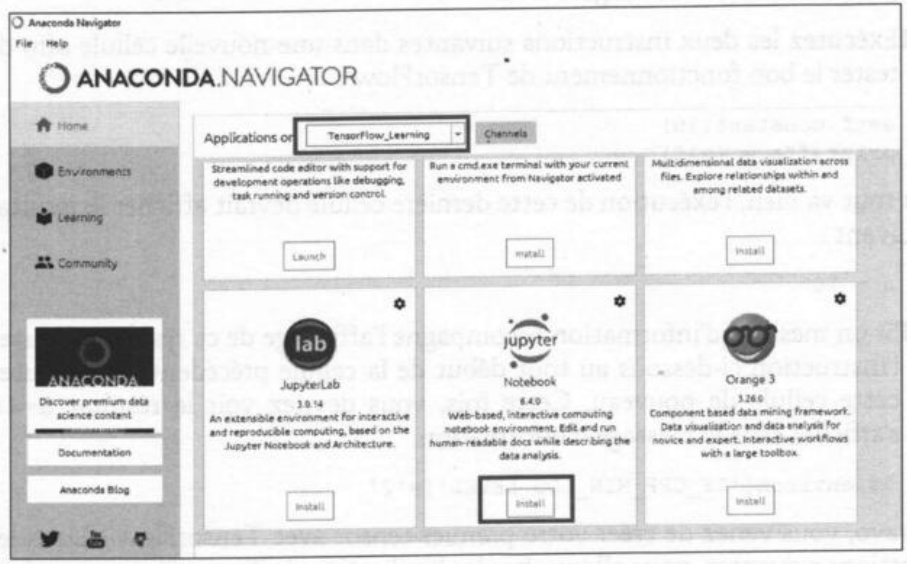


Figure 14-23 : installation de Jupyter dans l'environnement virtuel TensorFlow_Learning

- Une fois l'installation terminée, lancez l'application Jupyter en cliquant sur le bouton **Launch** sous la vignette de l'application Jupyter. Cela permet d'exécuter l'application Jupyter dans le contexte de l'environnement virtuel **TensorFlow_Learning**.

- Pour tester si la bibliothèque TensorFlow est correctement installée, créez un nouveau notebook Jupyter et saisissez les instructions suivantes dans une nouvelle cellule ; vous pouvez également utiliser le notebook *Test_tensorflow.ipynb* du dossier *Code* de ce chapitre.

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
```

Si aucun message d'erreur n'est affiché, alors toutes les bibliothèques que nous avons ajoutées précédemment à l'environnement **TensorFlow_Learning** ont été correctement chargées !

- Exécutez les deux instructions suivantes dans une nouvelle cellule afin de tester le bon fonctionnement de TensorFlow :

```
a=tf.constant(10)
print(f"a = {a}")
```

Si tout va bien, l'exécution de cette dernière cellule devrait afficher le résultat suivant :

```
a = 10
```

- Si un message d'information accompagne l'affichage de ce résultat, ajoutez l'instruction ci-dessous au tout début de la cellule précédente et exécutez cette cellule de nouveau. Cette fois, vous devriez voir le résultat `a=10` s'afficher sans le message d'information.

```
os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

Bravo, vous venez de créer votre premier tensor avec TensorFlow ! Dans les sections suivantes, nous allons aborder l'utilisation de TensorFlow pour réaliser des opérations élémentaires de l'algèbre, mais surtout nous allons réaliser des exemples pour développer des réseaux de neurones plus complexes que ceux vus dans le chapitre précédent.

3.2 Opérations élémentaires avec les tensors

Cette section permet d'aborder TensorFlow à travers des exemples simples où nous allons manipuler les objets de base de TensorFlow, à savoir les tensors. Ainsi, nous allons réaliser des opérations élémentaires avec ces tensors. Pour les exemples de cette section, vous pouvez créer votre propre notebook Jupyter avec les codes qui seront présentés ou utiliser directement le notebook *Opérations_TensorFlows.ipynb*.

© Editions ENI - All rights reserved

3.2.1 Travailler avec les tensors

- Dans une nouvelle cellule Jupyter, saisissez et exécutez les instructions suivantes afin de charger les bibliothèques tensorflow et numpy.

```
import tensorflow as tf
import os
import numpy as np

os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

Pour rappel, la dernière instruction de cette cellule permet d'éviter l'affichage inutile de certains messages d'informations de tensorflow.

- Dans une nouvelle cellule, saisissez et exécutez les deux instructions suivantes :

```
B = tf.constant([[1.,2.,3],[4,5,6],[7,8,9]])

print(f"B = {B}")
```

Dans ce code, la variable `B` est un tensor qui est associé à un tableau à deux dimensions. Les valeurs de `B` ont été déclarées avec les deux valeurs réelles `1.` et `2.` et toutes les autres valeurs ont été fixées avec des valeurs entières.

L'exécution de cette cellule affiche le résultat suivant :

```
B = [[1. 2. 3.]
     [4. 5. 6.]
     [7. 8. 9.]]
```

Les valeurs de `B` qui ont été déclarées avec des valeurs entières ont toutes été transformées en valeurs réelles ! Il suffit qu'il y ait une valeur réelle pour que TensorFlow considère que toutes les valeurs sont réelles. En effet, c'est tout à fait logique puisqu'avec le type des valeurs réelles on peut représenter toute valeur entière, mais pas l'inverse.

L'attribut `shape` d'un tensor permet de connaître les dimensions avec lesquelles ce tensor est défini et l'attribut `dtype` permet de connaître le type de données associé au tensor. Les deux instructions suivantes permettent d'afficher ces deux informations relatives au tensor `B`.

```
print(f"B.shape = {B.shape}")
print(f"B.dtype = {B.dtype}")
```

Le résultat est le suivant :

```
B.shape = (3, 3)
B.dtype = <dtype: 'float32'>
```

Remarquez que par défaut le type des réels utilisé par TensorFlow est `float32`, contrairement à la bibliothèque numpy qui considère par défaut que le type des réels est le type `float64`.

Pour accéder aux valeurs d'un tensor, on utilise la même notation que celle utilisée pour accéder aux valeurs d'un tableau numpy !

L'instruction suivante permet d'afficher les lignes du tensor `B` en ignorant la première ligne, c'est-à-dire la ligne d'indice `0`, et les deux premières colonnes en ignorant la dernière colonne.

```
print(f"{B[1:,:2]}")
```

Le résultat est le suivant :

```
[[4. 5.]
 [7. 8.]]
```

Les opérations arithmétiques peuvent être réalisées comme avec les tableaux numpy. Par exemple, le code suivant permet d'ajouter la valeur `100` au tensor `B`.

```
print(f"B+100 =\n {B+100}")
```

Cela aura comme conséquence d'ajouter la valeur `100` à chaque élément du tableau à deux dimensions associé au tensor `B`. Le résultat sera le suivant :

```
B+100 =
 [[101. 102. 103.]
 [104. 105. 106.]
 [107. 108. 109.]]
```

L'opération d'addition précédente aurait pu être réalisée en utilisant la méthode `add` de TensorFlow comme suit :

```
print(f"B+100 =\n {tf.add(B,100)}")
```

En réalité, lorsque nous avons utilisé le symbole +, c'était bien la méthode `add` qui était réellement appelée et exécutée.

© Editions ENI - All rights reserved

Nous pouvons créer des tensors à partir de tableaux numpy. L'exemple suivant montre comment créer le tensor `tf_tensor` à partir du tableau numpy `np_array_1`.

```
np_array_1 = np.array([1.,2.,3.,4.,5.,6.,7.,8.,9.,10.])
tf_tensor = tf.constant(np_array_1)
print("np_array_1 :", np_array_1)
print("tf_tensor",tf_tensor)
```

Ce programme affiche le résultat suivant :

```
np_array_1 : [ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
tf_tensor :  tf.Tensor([ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.],
shape=(10,), dtype=float64)
```

Remarquez que cette fois TensorFlow a créé le tensor `tf_tensor` avec le type `float64` et non pas le type `float32` comme il aurait dû le faire, puisque les nombres réels sont représentés par défaut avec le type `float32` ! C'est tout à fait normal car le tensor `tf_tensor` a été créé à partir d'un tableau numpy, et dans ce dernier les nombres réels sont représentés avec le type `float64` et `TensorFlow` a réutilisé le type de `np_array_1`, donc le type `float64`, pour la création de `tf_tensor`.

On peut également réaliser l'opération inverse à l'opération précédente en créant un tableau numpy à partir d'un tensor ! Le code suivant permet de créer le tableau numpy `np_array_2` à partir du tensor `tf_tensor`.

```
np_array_2 = tf_tensor.numpy()
print("np_array_2 : ", np_array_2)
```

Le résultat est le suivant :

```
np_array_2 :  [ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
```

Un point important auquel il faut faire attention est le type des tensors lorsque nous les utilisons dans des opérations arithmétiques.

Par exemple, le code suivant va générer une erreur à l'exécution, car le type du tensor `a` est `float64` alors que le type du tensor `b` est le type par défaut, à savoir `float32`.

```
a=tf.constant(10., dtype="float64")
b = tf.constant(10.)
print(f"a+b = {a+b}")
```

En effet, le code de cette dernière cellule, si vous tentez de l'exécuter, va lancer une erreur puisque pour TensorFlow il n'est pas possible d'additionner des `float64` avec des `float32` !

Contrairement à ce que nous avons l'habitude de faire avec les langages de programmation classiques où nous pouvons réaliser des opérations arithmétiques entre des nombres réels même s'ils ne sont pas de même taille, TensorFlow considère que le fait de tenter une opération arithmétique entre des données de tailles différentes est dû à une erreur de conception.

Ainsi, pour pouvoir réaliser l'opération d'addition précédente, il serait nécessaire de transformer l'un des deux opérandes vers le type de l'autre opérande. Le code suivant transforme l'opérande `a` qui était de type `float64` vers le type `float32`.

```
a=tf.constant(10., dtype="float64")
b = tf.constant(10.)

a=tf.cast(a, tf.float32 )
print(f"a+b = {a+b}")
```

Cette cellule affichera le résultat `a+b = 20.0`. Le calcul de ce résultat a été possible cette fois car les deux opérandes de l'opération d'addition sont de même type `float32`.

Il y a deux raisons principales pour lesquelles TensorFlow considère que c'est le type `float32` qui représente par défaut les valeurs réelles et non pas le type `float64`. Ces deux raisons sont les suivantes :

- Le type `float32` est suffisant dans la plupart des situations pour mener correctement des calculs.
- Évidemment, le type `float64` est beaucoup plus consommateur d'espace mémoire que le type `float32`. Sachant que les flux de calculs tels que ceux réalisés avec les réseaux de neurones utilisent beaucoup de données, l'optimisation de l'espace mémoire devient une nécessité.

© Editions ENI - All rights reserved

3.2.2 Les tensors variables

Les tensors que nous avons eu l'occasion de manipuler à la section précédente sont des constantes, c'est-à-dire qu'une fois qu'ils ont été créés, nous ne pouvons pas modifier leurs valeurs.

Les tensors dont les valeurs sont modifiables peuvent être créés avec la méthode `Variable`. Le code suivant crée le tensor `tensor_1` en l'initialisant avec un tableau à deux dimensions, puis modifie les valeurs de ce tableau en utilisant la méthode `assign`.

```
tensor_1 = tf.Variable([[1,2,3],[4,5,6],[7,8,9]])

print(f"tensor_1 = \n{tensor_1}")

tensor_1.assign(tensor_1 * 100)
print(f"tensor_1 = \n{tensor_1}")
```

Ce code affiche la valeur du tensor `tensor_1` avant et après sa modification comme suit :

```
tensor_1 =
<tf.Variable 'Variable:0' shape=(3, 3) dtype=int32, numpy=
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])>

tensor_1 =
<tf.Variable 'Variable:0' shape=(3, 3) dtype=int32, numpy=
array([[100, 200, 300],
       [400, 500, 600],
       [700, 800, 900]])>
```

3.2.3 Initialiser les tensors

Il existe des méthodes de TensorFlow qui permettent d'initialiser les valeurs d'un tensor. Par exemple, le code suivant utilise la méthode `zeros` pour créer un tensor et l'associer à un tableau à deux dimensions, trois lignes et trois colonnes, dont toutes les valeurs sont égales à 0.

```
tensor_3 = tf.zeros((3,3))
print(f"tensor_3 = \n{tensor_3}")
```

Le résultat est le suivant :

```
tensor_3 =
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

On peut également initialiser un tensor avec des valeurs générées de manière aléatoire comme dans le code suivant :

```
tensor_4 = tf.random.uniform((4,4), minval=1, maxval=100, seed=1)
print(f"tensor_4 = \n{tensor_4}")
```

Ce code génère un tensor associé à un tableau de quatre lignes et quatre colonnes dont les valeurs seront générées de manière pseudo-aléatoire, mais comprises entre 1 et 100. Le résultat est le suivant :

```
tensor_4 =
[[42.450756 20.282463 93.04293  17.566793]
 [95.555885 55.12257  29.931704 99.2678  ]
 [64.860146 90.37877  63.63836  44.409744]
 [71.43016  27.630108 94.333725 44.25939 ]]
```

Dans ce dernier exemple de méthodes d'initialisation des tensors, nous avons utilisé la méthode `fill` afin de choisir explicitement la valeur avec laquelle nous souhaitons initialiser le tensor `tensor_4` et qui est la valeur 5.

```
 tensor_4 = tf.fill((3,3), value = 5)
print(f"tensor_4 = \n{tensor_4}")
```

Le résultat qui sera affiché est le suivant :

```
tensor_4 =
[[5 5 5]
 [5 5 5]
 [5 5 5]]
```

3.2.4 Opérations algébriques avec les tensors

Avec TensorFlow, nous pouvons effectuer toutes les opérations algébriques que nous pouvons réaliser avec les tableaux numpy. À titre d'exemple, le code suivant réalise la multiplication des deux matrices associées aux deux tensors `tensor_5` et `tensor_6`. La multiplication est réalisée grâce à la méthode `matmul` de TensorFlow.

© Editions ENI - All rights reserved

```
tensor_5 = tf.fill((2,3), 1)
tensor_6 = tf.fill((3,2), 2)
print(f"tensor_5 = \n{tensor_5}")
print(f"tensor_6 = \n{tensor_6}")

tensor_7 = tf.matmul(tensor_5, tensor_6)
print(f"tensor_7 = \n{tensor_7}")
```

Le résultat de cette multiplication est stocké dans le tensor `tensor_7`. Cette cellule affiche le résultat suivant :

```
tensor_5 =
[[1 1 1]
 [1 1 1]]
tensor_6 =
[[2 2]
 [2 2]
 [2 2]]
tensor_7 =
[[6 6]
 [6 6]]
```

4. Les réseaux de neurones avec Sequential API

Cette classe permet de définir un objet qui sera associé à un réseau de neurones dont les couches sont connectées les unes aux autres de manière linéaire en partant de la couche d'entrée jusqu'à la couche de sortie en passant par les couches cachées.

Chaque couche cachée d'un réseau de neurones de type `models.Sequential` reçoit ses entrées à partir des sorties de la couche située juste derrière elle, et les sorties de cette couche cachée seront connectées uniquement aux entrées de la couche située tout de suite devant elle.

Nous allons réaliser un exemple de réseau de neurones en utilisant Sequential API. Ce modèle de réseau de neurones que nous allons réaliser sera spécialisé dans la classification des images.

4.1 Charger les données

Pour les besoins de cet exemple, nous allons utiliser les données du dataset `fashion_mnist`. Ces données sont un ensemble d'images dont chacune représente un produit parmi dix produits différents. Une fois entraîné, le réseau de neurones de cet exemple a pour objectif de reconnaître le produit présent dans une image donnée parmi dix produits possibles.

- Créez un nouveau notebook Jupyter, puis saisissez et exécutez le code suivant dans une nouvelle cellule ; vous pouvez également utiliser directement le notebook *Sequential_classification.ipynb*.

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

Ces instructions permettent de charger les différentes bibliothèques nécessaires à la réalisation de notre exemple.

Remarquez l'import de la bibliothèque Keras à partir de TensorFlow. En effet, Keras est la bibliothèque de haut niveau officielle de développement des réseaux de neurones avec TensorFlow et cela depuis la version 2 de TensorFlow.

Avant la version 2, pour utiliser Keras conjointement avec TensorFlow pour le développement de réseaux de neurones, il aurait fallu installer TensorFlow, puis installer Keras. Mais depuis la version 2 de TensorFlow, une version de Keras a été intégrée à TensorFlow et est dédiée aux développements des réseaux de neurones.

Il ne faut donc pas confondre la bibliothèque Keras, qui peut être utilisée pour le développement de réseaux de neurones conjointement avec TensorFlow ou Theano ou autre framework, et la bibliothèque Keras intégrée à TensorFlow et qui, elle, ne fonctionne qu'avec TensorFlow.

© Editions ENI - All rights reserved

- Dans une nouvelle cellule, saisissez et exécutez le code suivant afin de charger le jeu de données `fashion_mnist` à partir des datasets présents dans `keras.datasets`.

```
fashion_mnist_data = keras.datasets.fashion_mnist
(all_x_train, all_y_train), (x_test, y_test) =
                                fashion_mnist_data.load_data()

all_x_train = all_x_train.astype('float32')

x_test = x_test.astype('float32')
```

À la suite de l'exécution de la cellule précédente, nous aurons construit deux jeux de données : un premier jeu de données `all_x_train` avec ses classes associées `all_y_train` qui nous servira pour entraîner un réseau de neurones et un deuxième jeu de données `x_test` avec ses classes associées `y_test` qui nous servira pour évaluer un réseau de neurones. Également, ces deux jeux de données ont été reformatés avec le type `float32`.

Les instructions suivantes affichent la taille du jeu de données `all_x_train`, la taille d'une image de ce jeu de données, le type associé à chaque pixel de ses images et la taille du jeu de données `x_test`.

```
print(f"all_x_train.shape = {all_x_train.shape}")
print(f"all_x_train[0].shape = {all_x_train[0].shape}")
print(f"all_x_train[0].dtype = {all_x_train[0].dtype}")

print(f"x_test.shape = {x_test.shape}")
```

Le résultat est le suivant :

```
all_x_train.shape = (60000, 28, 28)
all_x_train[0].shape = (28, 28)
all_x_train[0].dtype = float32
x_test.shape = (10000, 28, 28)
```

Nous avons donc 60000 images dans le jeu de données `all_x_train` et chacune de ces images est représentée par une matrice de 28 x 28 dont chaque valeur est de type `float32` et nous avons 10000 images dans le jeu de données `x_test`.

- Dans une nouvelle cellule, saisissez et exécutez le code suivant afin de séparer le jeu de données `all_x_train` en deux groupes de données pour ainsi avoir un jeu de données à utiliser comme étant les données de validation. Nous allons voir par la suite à quoi vont nous servir ces données de validation.

```
x_validation, x_train = all_x_train[:5000] / 255.0,
all_x_train[5000:] / 255.0
y_validation, y_train = all_y_train[:5000], all_y_train[5000:]
```

À la suite de l'exécution de la dernière cellule, nous aurons construit les deux nouveaux jeux de données `x_validation` et `x_train`. Dans le jeu de données `x_validation` nous aurons stocké les 5 000 premières images de `all_x_train` et dans le jeu de données `x_train` nous aurons stocké les 55000 images restantes. Le fait de diviser les pixels de toutes ces images par 255 permet de normaliser leurs valeurs.

Les instructions suivantes affichent les mêmes informations que celles affichées précédemment sur le jeu de données `all_x_train`, mais cette fois les informations affichées concernent les images de `x_train`.

```
print(f"x_train.shape = {x_train.shape}")
print(f"x_train[0].shape = {x_train[0].shape}")
print(f"x_train[0].dtype = {x_train[0].dtype}")
```

Le résultat est le suivant :

```
x_train.shape = (55000, 28, 28)
x_train[0].shape = (28, 28)
x_train[0].dtype = float32
```

On voit bien que nous avons 55 000 images dans le jeu de données `x_train`.

© Editions ENI - All rights reserved

Le code suivant permet de stocker dans la liste `fashion_mnist_class_names` les vrais noms des produits associés aux images de nos données. En effet, les classes des images indiquées dans les tableaux `y_validation`, `y_train` et `y_test` sont des valeurs entre 0 et 9 et la liste `fashion_mnist_class_names` va nous permettre d'associer des noms de produits à ces valeurs.

```
fashion_mnist_class_names = ["T-shirt/top", "Trouser",
"Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag",
"Ankle boot"]
```

Le code suivant permet d'afficher une association entre les cinq premières valeurs numériques de `x_train` avec leurs noms de produits de la liste `fashion_mnist_class_names`.

```
for cls in range(5):
   print(cls, ":",fashion_mnist_class_names[y_train[cls]])
```

Le résultat est le suivant :

```
0 : Coat
1 : T-shirt/top
2 : Sneaker
3 : Ankle boot
4 : Ankle boot
```

Le code suivant permet d'afficher les deux premières images de `x_train` avec leur nom de produit.

```
for i in range(2):
   my_img= x_train[i]
   my_img_class = y_train[i]
   my_img_class_name = fashion_mnist_class_names[my_img_class]
   plt.imshow(my_img)
   plt.title(my_img_class_name)
   plt.show()
```

L'exécution de cette cellule affiche un résultat comme sur la figure suivante :

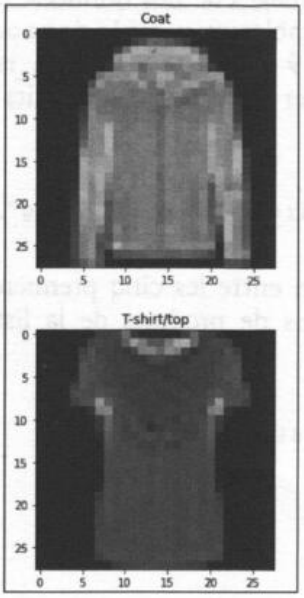


Figure 14-24 : les deux premières images de `x_train`

En résumé, à ce stade nous avons construit les trois jeux de données suivants :

- `(x_train, y_train)` : constitué de 55 000 images avec leurs classes associées et qui sera utilisé pour l'entraînement d'un réseau de neurones.
- `(x_validation, y_validation)` : constitué de 5 000 images avec leurs classes associées et qui sera utilisé pour évaluer le réseau de neurones à la fin de chaque itération lors de la phase d'entraînement d'un réseau de neurones.
- `(x_test, y_test)` : constitué de 10 000 images avec leurs classes associées et qui sera utilisé pour évaluer le réseau de neurones à la fin de toutes les itérations de la phase d'entraînement d'un réseau de neurones.

© Editions ENI - All rights reserved

4.2 Définir un MLP avec Sequential API

Pour définir l'architecture de notre réseau de neurones, nous allons utiliser la bibliothèque Keras dans sa version intégrée à TensorFlow.

Le code suivant permet de définir la structure de notre réseau de neurones.

```
model1 = keras.models.Sequential()
model1.add(keras.layers.Flatten(input_shape=[28, 28]))
model1.add(keras.layers.Dense(300, activation="relu"))
model1.add(keras.layers.Dense(150, activation="relu"))
model1.add(keras.layers.Dense(10, activation="softmax"))
```

Ce code commence par créer l'objet `model1` avec la classe `models.Sequential` qui correspond, pour rappel, à un réseau de neurones dont les couches sont connectées les unes aux autres de manière linéaire. Puis nous avons défini pour ce réseau de neurones `model1` une couche d'entrée, deux couches cachées et une couche de sortie définies comme suit :

1. La couche d'entrée de type `Flatten`. Ce type de couche permet d'aplatir les tableaux à deux dimensions qui contient les données sous-jacentes aux images pour les transformer en tableau à une seule dimension. Cette couche de type `Flatten`, lorsqu'elle reçoit une matrice 28 x 28 valeurs en entrée, produira à sa sortie un tableau de 784 valeurs. L'argument `input_shape` permet justement de définir la taille des données en entrée.
2. Deux couches cachées : une première couche cachée avec 300 neurones et avec la fonction d'activation `relu` suivie d'une deuxième couche cachée avec 150 neurones et également avec la fonction d'activation `relu`. Ces deux couches cachées sont de type `Dense`. Le type `Dense` permet de définir des couches dont les neurones sont connectés à tous les neurones de la couche précédente.

3. La couche de sortie de type `Dense` et qui est définie avec 10 neurones et la fonction d'activation `softmax`. Cette couche de sortie a besoin de dix neurones car le modèle de classification que nous sommes en train de développer doit reconnaître la présence dans une image d'un produit parmi dix produits. En d'autres termes, nous souhaitons développer un modèle de classification pour dix classes différentes, donc la dernière couche doit avoir dix neurones. Chacun des dix neurones de la couche de sortie sera entraîné pour reconnaître l'un des dix produits !

Nous n'allons pas entrer dans les détails de la définition des fonctions d'activation `relu` et `softmax` car cela n'est pas l'objectif de ce chapitre, cependant il faut savoir que la fonction `relu` permet d'avoir des couches cachées efficaces et la fonction `softmax` permet d'obtenir la probabilité du produit qui a le plus de chances d'être celui qui figure dans une image donnée. Généralement, lorsque le problème à résoudre est un problème de classification où le nombre de classes est supérieur à 2 et où nous souhaitons associer une et une seule classe pour chaque image, alors la fonction `softmax` devrait être utilisée.

Saisissez l'instruction suivante afin d'afficher un résumé du réseau `model1`.

```
model1.summary()
```

Le résultat sera le suivant :

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 300)               235500
_________________________________________________________________
dense_1 (Dense)              (None, 150)               45150
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1510
=================================================================
Total params: 282,160
Trainable params: 282,160
Non-trainable params: 0
_________________________________________________________________
```

Figure 14-25 : résultat de la méthode `summary` sur le réseau de neurones model1

© Editions ENI - All rights reserved

À gauche de cette figure, on voit les noms attribués par défaut par TensorFlow aux quatre couches de notre réseau de neurones `model1`. Ces noms sont `flatten`, `dense`, `dense_1` et `dense_2`.

La colonne `Output Shape` montre le nombre de résultats à la sortie de chaque couche. En résumé, nous avons les 784 poids associés à la sortie de la couche d'entrée `flatten` puisque, comme mentionné précédemment, cette couche transforme une matrice de 28 x 28 en un tableau à une dimension, les 300 sorties associées aux résultats des 300 neurones de la première couche cachée `dense`, les 150 sorties associées aux résultats des 150 neurones de la deuxième couche cachée `dense_1`, et enfin, les 10 sorties associées aux résultats des 10 neurones de la couche de sortie `dense_2`.

La colonne `Param` indique le nombre de poids et de biais attachés à chaque couche. Ainsi, la couche d'entrée `flatten` a 0 poids et biais. La première couche cachée `dense` a 235 500 poids et biais. En effet, les 784 sorties au niveau de la couche d'entrée `flatten` sont connectées à chacun des 300 neurones de cette couche cachée `dense`, ce qui fait 784 * 300 = 235 200 poids auxquels il faut ajouter les 300 biais associés à cette couche, un biais par neurone comme nous l'avons vu au chapitre précédent. Donc, au total, la couche `dense` aura 235 200 + 300= 235 500 poids !

De même, la couche cachée `dense_1` aura 300 * 150 = 45 000 poids issus de la couche cachée précédente plus 150 biais associés à cette couche, ce qui fait un total de 45 150 poids et biais associés à cette deuxième couche cachée `dense_1`. De la même manière, le nombre de poids et biais de la couche de sortie `dense_2` est égal à 150 * 10 + 10 = 1 510.

4.3 Accéder aux éléments d'un réseau de neurones

Une fois que les couches d'un réseau de neurones sont définies, on peut accéder aux informations de chaque couche. L'exemple de code suivant montre comment accéder aux valeurs des poids et aux valeurs des biais de la deuxième couche cachée `dense_1`.

```
hidden_ = model1.get_layer('dense_1')

weights, biases = hidden_.get_weights()
```

```
print("Les poids :\n ")
print(weights)
print(weights.shape)
print("Les biais :\n ")
print(biases)
print(biases.shape)
```

Ce code récupère dans la variable `hidden_` la deuxième couche cachée en utilisant la méthode `get_layer`. Puis la méthode `get_weights` retourne les poids de cette deuxième couche cachée dans la variable `weights` et les biais de cette même couche cachée dans la variable `biases`.

La figure suivante montre les valeurs des poids et des biais de la couche cachée `dense_1`.

```
Les poids :

[[ 0.0660976   0.05660684 -0.07136895 ...  0.0424329  -0.09606344
  -0.05489517]
 [ 0.08349757  0.11541742  0.0792702  ...  0.05178729  0.00928503
  -0.03378094]
 [ 0.00138865 -0.0233416  -0.00802676 ... -0.10342614 -0.00844335
  -0.02871398]
 ...
 [-0.08509746  0.05107844  0.06101492 ... -0.11332393 -0.100563
  -0.0649552 ]
 [-0.02766114 -0.10531881  0.09312865 ... -0.02262264  0.11483601
   0.10447685]
 [-0.10813703 -0.00813232  0.01048039 ...  0.00991713  0.04582493
  -0.07162592]]
(300, 150)

Les biais :

[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0.]
(150,)
```

Figure 14-26 : les poids et les biais du réseau de neurones `model`

On voit que les poids de cette deuxième couche cachée sont stockés dans un tableau de 300 x 150, c'est-à-dire une matrice. Chaque colonne de cette matrice correspond aux entrées de l'un des 150 neurones de cette couche cachée.

© Editions ENI - All rights reserved

Les 150 biais de cette couche ont tous été initialisés avec la valeur 0.

En réalité, les biais de toutes les couches sont initialisés par défaut avec la valeur 0.

4.4 Initialisation des poids et des biais d'un réseau de neurones

On peut utiliser les fonctions de `tensorflow.keras.initializers` afin d'initialiser les poids et les biais des couches d'un réseau de neurones. Par exemple, le code suivant permet de créer un deuxième réseau de neurones `model2` avec la même structure que le réseau de neurones `model1`, mais cette fois les poids et les biais de la deuxième couche cachée de `model2` sont initialisés avec des valeurs autres que les valeurs par défaut.

```
model2 = keras.models.Sequential()
model2.add(keras.layers.Flatten(input_shape=[28, 28]))
model2.add(keras.layers.Dense(300, activation="relu"))

my_weights_initializer =
      tf.keras.initializers.RandomNormal(mean=100., stddev=1.)

my_bias_initializer= tf.keras.initializers.Ones()

model2.add(keras.layers.Dense(150, activation="relu" ,\
          kernel_initializer = my_weights_initializer, \
          bias_initializer = my_bias_initializer))
model2.add(keras.layers.Dense(10, activation="softmax"))

model2.summary()
```

Dans ce code, la variable `my_weights_initializer` est associée à des poids initialisés avec la fonction `RandomNormal` qui permet d'obtenir des valeurs distribuées suivant la loi normale centrée avec une moyenne indiquée avec l'argument `mean` qui est fixé ici à `100` et un écart-type indiqué par l'argument `stddev` qui est fixé ici à `1`.

Également, la variable `my_bias_initializer` est initialisée avec des valeurs égales à `1`, puisque c'est la fonction `Ones` qui est utilisée.

Lors de la création de la deuxième couche cachée avec la méthode add, la variable my_weights_initializer a été affectée à l'argument kernel_initializer et la variable my_bias_initializer a été affectée à l'argument bias_initializer.

Pour ce nouveau réseau de neurones model2, la méthode summary affiche des résultats comme ceux de la figure suivante :

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten_1 (Flatten)          (None, 784)               0
_________________________________________________________________
dense_3 (Dense)              (None, 300)               235500
_________________________________________________________________
dense_4 (Dense)              (None, 150)               45150
_________________________________________________________________
dense_5 (Dense)              (None, 10)                1510
=================================================================
Total params: 282,160
Trainable params: 282,160
Non-trainable params: 0
_________________________________________________________________
```

Figure 14-27 : résultat de la méthode summary sur le réseau de neurones model2

Cette fois, la deuxième couche cachée est nommée dense_4. Le code suivant permet d'afficher les poids et les biais de cette couche.

```
hidden_ = model2.get_layer('dense_4')
weights, biases = hidden_.get_weights()
print("Les poids :\n ")
print(weights)
print(weights.shape)
print("Les biais :\n ")
print(biases)
print(biases.shape)
```

© Editions ENI - All rights reserved

Le résultat est le suivant :

```
Les poids :

[[101.781296  99.22688  101.06561  ... 100.05606   99.85397  100.30503 ]
 [101.71722   98.91859   99.06323  ... 101.620674  97.878395 101.26123 ]
 [ 98.25745   98.168045  99.38385  ... 100.782646  99.40467  100.490974]
 ...
 [101.12513  100.24388   99.547424 ... 101.528244  99.6419   100.93199 ]
 [100.34876  101.85785   99.90284  ...  99.7295    100.85832  101.88643 ]
 [ 99.76423   98.82184   98.95094  ... 100.01098   100.09875   98.182724]]
(300, 150)

Les biais :

[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1.]
(150,)
```

Figure 14-28 : les poids et les biais du réseau de neurones model2

On voit dans ces résultats que les poids de la deuxième couche cachée `dense_4` sont centrés autour de la valeur `100` et les biais ont été positionnés avec la valeur `1`.

4.5 Compiler un réseau de neurones

Une fois que la structure d'un réseau de neurones est mise en place, avant d'introduire les données dans ce réseau pour entamer la phase d'entraînement, il est nécessaire de le compiler d'abord.

Avec TensorFlow, compiler un réseau de neurones revient à indiquer les trois informations suivantes :

1. La fonction `loss` à utiliser, c'est-à-dire indiquer comment calculer les erreurs à la sortie de la couche de sortie.
2. Indiquer la méthode d'optimisation à utiliser lors des propagations arrière des erreurs.
3. Les métriques de performance que nous souhaitons utiliser pour mesurer l'efficacité de l'apprentissage du réseau de neurones.

Le code suivant montre un exemple de compilation du réseau de neurones `model1` :

```
model1.compile(loss="sparse_categorical_crossentropy",
optimizer="sgd", metrics=["accuracy"])
```

À la suite de l'exécution de l'instruction précédente, notre réseau de neurones `model1` est prêt pour recevoir les données d'entraînement !

Nous n'allons pas entrer dans les détails de toutes les valeurs possibles pour les paramètres `loss`, `optimizer` et `metrics` car ceci dépasse la portée de ce livre. Néanmoins, la fonction `sparse_categorical_crossentropy` est utilisée comme fonction de calcul de perte lorsque nous avons, comme dans notre exemple, un problème de classification où la classe d'appartenance à la sortie du réseau de neurones est donnée par un vecteur de probabilités. En effet, dans cet exemple, à la sortie de notre réseau de neurones chaque résultat correspond à un vecteur de dix probabilités, une probabilité pour chaque produit d'être le produit présent dans une image.

4.6 Entraîner un réseau de neurones

À ce stade, l'entraînement de notre réseau de neurones peut être lancé. Pour cela, il suffit d'utiliser la méthode `fit` comme dans le code suivant :

```
results = model1.fit(x_train, y_train, epochs=40,\
                     validation_data=(x_validation, y_validation))
```

Les deux premiers arguments de la méthode `fit` indiquent le jeu de données d'entraînement. L'argument `epochs` indique le nombre d'itérations souhaité pour la phase d'entraînement, qui est fixé ici à `40`. Pour rappel, à chaque itération les 55 000 images du jeu de données d'entraînement vont traverser le réseau de neurones. L'argument `validation_data` indique les données à utiliser à la fin de chaque itération afin d'évaluer les performances du réseau de neurones à la suite de cette itération. L'évaluation du réseau de neurones à la suite de chaque itération sur les données de validation sera calculée en fonction des métriques de performance indiquées avec l'argument `metrics` lors de la compilation du réseau de neurones.

© Editions ENI - All rights reserved

À l'exécution de la dernière cellule, vous devriez voir s'afficher les résultats de chaque itération de la phase d'entraînement. La figure suivante montre les résultats des cinq dernières itérations :

```
Epoch 36/40
55000/55000 [==============================] - 3s 59us/sample - loss: 0.2053 - accuracy: 0.9261 - val_loss: 0.3136 - val_accu
racy: 0.8914
Epoch 37/40
55000/55000 [==============================] - 3s 57us/sample - loss: 0.2032 - accuracy: 0.9284 - val_loss: 0.2895 - val_accu
racy: 0.8948
Epoch 38/40
55000/55000 [==============================] - 3s 58us/sample - loss: 0.1988 - accuracy: 0.9292 - val_loss: 0.3116 - val_accu
racy: 0.8906
Epoch 39/40
55000/55000 [==============================] - 3s 56us/sample - loss: 0.1957 - accuracy: 0.9303 - val_loss: 0.3016 - val_accu
racy: 0.8938
Epoch 40/40
55000/55000 [==============================] - 3s 59us/sample - loss: 0.1928 - accuracy: 0.9304 - val_loss: 0.3265 - val_accu
racy: 0.8860
```

Figure 14-29 : résultat des itérations de la phase d'entraînement du réseau de neurones `model1`

Chacune des lignes de ces résultats affiche les informations suivantes, en allant de la gauche vers la droite :

- Le numéro de l'itération, par exemple 40/40 pour la dernière itération.
- La progression du passage des données d'entraînement dans le réseau. Cette information est la même pour toutes les lignes des résultats puisque c'est toujours le même nombre de données qui vont traverser le réseau de neurones à chaque itération. Dans notre exemple, lorsque la dernière image traverse le réseau de neurones `model1`, cette information sera égale à 55 000.
- Le temps consommé par l'itération en cours.
- La valeur de la fonction utilisée pour calculer les erreurs à la sortie du réseau de neurones. Dans notre exemple, cette information est égale à 0.1928 lors de la dernière itération.
- Les valeurs des métriques utilisées pour l'évaluation du réseau de neurones sur les données d'entraînement. Dans notre exemple, nous avons utilisé la métrique `accuracy` qui est égale à 0.9304 à la fin de la dernière itération.
- Les deux dernières informations correspondent respectivement à la valeur de la fonction de calcul des erreurs et à la valeur du calcul des métriques de performance sur les données de validation. Dans notre exemple, ces deux valeurs sont égales respectivement à 0.3265 et 0.8860.

Cela signifie que notre réseau de neurones parvient à classer correctement 88.60 % des 5 000 images du jeu de données de validation.

4.7 Analyser les résultats de l'entraînement d'un réseau de neurones

Avec les résultats obtenus pendant la phase d'entraînement d'un réseau de neurones, nous pouvons tracer les quatre courbes intéressantes suivantes :

1. La courbe d'évolution de la fonction de calcul des pertes à la sortie du réseau de neurones au fil des itérations par rapport aux données d'entraînement.
2. La courbe d'évolution de la fonction de calcul des pertes à la sortie du réseau de neurones au fil des itérations par rapport aux données de validation.
3. La courbe d'évolution des performances au fil des itérations par rapport aux données d'entraînement.
4. La courbe d'évolution des performances au fil des itérations par rapport aux données de validation.

Le code suivant permet de tracer ces quatre courbes dans un même graphique :

```
import pandas as pd
pd.DataFrame(results.history).plot(figsize=(8, 5))
plt.grid(True)
plt.gca().set_ylim(0, 1)
plt.show()
```

© Editions ENI - All rights reserved

La figure suivante montre les quatre courbes obtenues suite à l'entraînement précédent du réseau de neurones `model1`.

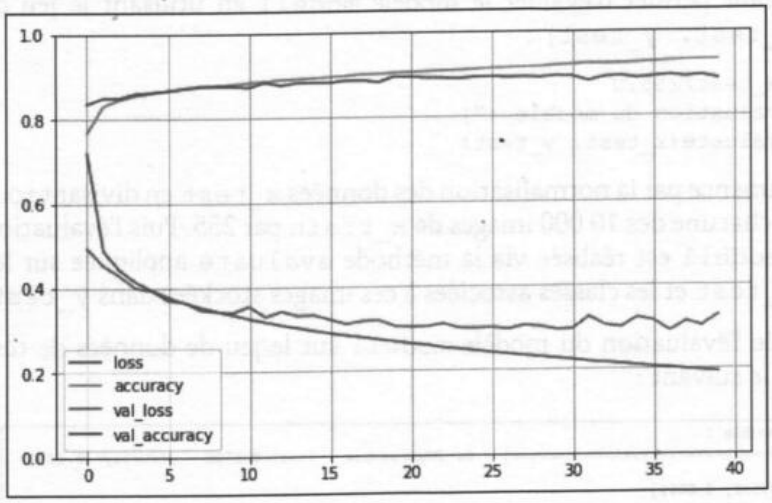


Figure 14-30 : courbes d'évolution des performances de `model1` pendant la phase d'entraînement

Sur ces quatre courbes, nous pouvons observer que le calcul des pertes à la sortie du réseau de neurones `model1` est en baisse au fur et à mesure des itérations, et ce sur les données d'entraînement et sur les données de validation. Également, le calcul de la métrique de performance `accuracy` augmente au fur et à mesure des itérations sur les données d'entraînement et également sur les données de validation.

Une observation qui est également intéressante à faire sur ce graphique est que l'évolution de la métrique `accuracy` sur les données d'entraînement et celle sur les données de validation sont très similaires, puisque leurs tracés respectifs sont proches l'un de l'autre et se suivent au fil des itérations. Cela indique qu'il y a une très forte probabilité que notre réseau de neurones ne soit pas en situation de surapprentissage.

4.8 Évaluer un réseau de neurones

Le code suivant permet d'évaluer le modèle `model1` en utilisant le jeu de données (`x_test, y_test`) :

```
x_test= x_test/255.0
print("Évaluation du modèle :")
model1.evaluate(x_test, y_test)
```

Ce code commence par la normalisation des données `x_test` en divisant tous les pixels de chacune des 10 000 images de `x_train` par 255. Puis l'évaluation du modèle `model1` est réalisée via la méthode `evaluate` appliquée sur les images de `x_test` et les classes associées à ces images stockées dans `y_test`.

Le résultat de l'évaluation du modèle `model1` sur le jeu de données de test `x_test` est le suivant :

```
Evaluation du modèle :
10000/10000 [==============================] - 0s 30us/sample - loss: 0.3590 - accuracy: 0.8811

[0.3590307104289532, 0.8811]
```

Figure 14-31 : résultat de l'évaluation de `model1` sur les données de test `x_test`

Ces résultats montrent que sur le jeu de données de test notre réseau de neurones `model1` parvient à classer correctement 88.11 % des images !

4.9 Prédire avec un réseau de neurones pour la classification

Pour les besoins de notre exemple, afin de montrer comment réaliser des prédictions sur des images, nous allons récupérer les sept premières images de `x_test` et tester les réponses de notre réseau de neurones `model1` comme suit :

```
x_new = x_test[0:7]
y_prob = model1.predict(x_new)
print(f"y_prob :\n{y_prob.round(2)}")
y_prediction = model1.predict_classes(x_new)
print(f"\ny_prediction :\n {y_prediction}")
```

Pour chacune des sept images, `model1` va nous retourner un tableau de dix probabilités !

© Editions ENI - All rights reserved

La figure suivante montre le résultat de `model1` sur ces sept images :

```
y_prob :
[[0.   0.   0.   0.   0.   0.   0.01 0.   0.99]
 [0.   0.   1.   0.   0.   0.   0.   0.   0.   ]
 [0.   1.   0.   0.   0.   0.   0.   0.   0.   ]
 [0.   1.   0.   0.   0.   0.   0.   0.   0.   ]
 [0.45 0.   0.01 0.   0.   0.   0.53 0.   0.   ]
 [0.   1.   0.   0.   0.   0.   0.   0.   0.   ]
 [0.   0.   0.   0.   0.98 0.   0.01 0.   0.   ]]

y_prediction :
 [9 2 1 1 6 1 4]
```

Figure 14-32 : résultat des prédictions de `model1` sur les sept premières images de `x_test`

Cette dernière figure montre que, par exemple pour la première image, notre réseau de neurones prédit pour chacun des dix produits possibles la probabilité d'être celui présent dans cette image. Ainsi, toujours pour cette première image, le réseau de neurones `model1` prédit une probabilité proche de la valeur 0 pour les sept premiers produits et également la probabilité 0 pour le produit numéro 8. En revanche, notre réseau de neurones `model1` prédit que la première image correspond au produit numéro 7 avec la probabilité de 0.01 et cette même image correspond au produit numéro 9 avec la probabilité 0.99. Justement, c'est parce que pour chaque image notre réseau de neurones associe un vecteur de dix probabilités que nous avons choisi la fonction `loss = sparse_categorical_crossentropy`.

Afin de comparer les prédictions de notre réseau de neurones avec les vraies valeurs correspondant aux images du jeu de données de test, le code suivant affiche les noms des classes prédites à côté des vrais noms de classes des sept premières images du jeu de données de test :

```
print(f"        Prédictions :
{np.array(fashion_mnist_class_names)[y_prediction]}")
y_truth = y_test[0:7]
print(f"Les vraies classes :
{np.array(fashion_mnist_class_names)[y_truth]}")
```

Le résultat est le suivant :

```
      Prédictions : ['Ankle boot' 'Pullover' 'Trouser' 'Trouser' 'Shirt' 'Trouser' 'Coat']
Les vraies classes : ['Ankle boot' 'Pullover' 'Trouser' 'Trouser' 'Shirt' 'Trouser' 'Coat']
```

Figure 14-33 : comparaison des prédictions de `model1` sur les sept premières images de `x_test` avec les vraies classes de ces images

On voit que notre réseau de neurones a prédit correctement les classes des sept premières images de `x_test`.

5. Utiliser Functional API

Functional API permet de développer des réseaux de neurones dont la structure est plus complexe que la structure des réseaux de neurones que nous pouvons développer avec Sequential API. Avec Functional API, les couches d'un réseau de neurones ne sont pas forcément empilées les unes derrière les autres de manière linéaire comme avec Sequential API.

Dans cette section, nous allons développer un réseau de neurones avec trois couches d'entrée, trois couches cachées et deux couches de sortie. Afin de diversifier nos exemples de réseaux de neurones développés avec TensorFlow, nous allons développer cette fois un réseau de neurones pour une régression linéaire. L'architecture du réseau de neurones que nous allons réaliser dans cet exemple correspond à l'architecture de la figure suivante :

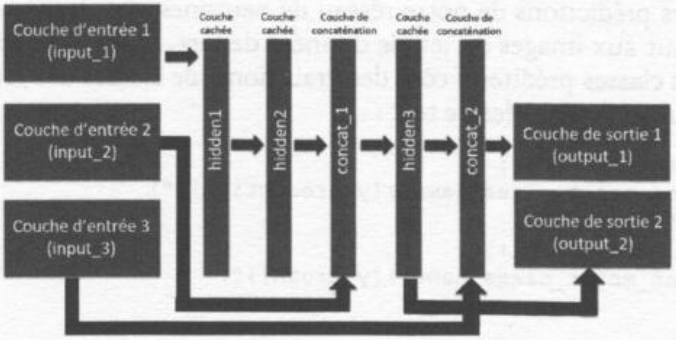


Figure 14-34 : réseau de neurones avec trois couches d'entrée, trois couches cachées et deux couches de sortie

© Editions ENI - All rights reserved

5.1 Un modèle Functional API avec plusieurs couches d'entrée

Pour les besoins de ce nouvel exemple, nous allons utiliser les données du dataset `fetch_california_housing` que nous allons importer de la bibliothèque Scikit-learn. Ce jeu de données concerne le prix d'appartements en Californie en fonction de certaines caractéristiques relatives à ces appartements. Nous n'allons pas entrer dans les détails de définition de ces données car ce qui nous intéresse est de montrer comment utiliser Functional API afin de réaliser l'architecture du réseau de neurones de la figure 14-34 ci-dessus.

▶ Pour réaliser cet exemple, créez un nouveau notebook, puis suivez les étapes de code ci-après ; vous pouvez également utiliser directement le notebook *Functional_Regression.ipynb*.

Le code suivant permet de lire les données que nous allons utiliser à partir de `fetch_california_housing`.

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


os.environ['TF_CPP_MIN_LOG_LEVEL']='2'


housing = fetch_california_housing()
all_x_train, x_test, all_y_train, y_test =
train_test_split(housing.data, housing.target)


x_train, x_validation, y_train, y_validation =
train_test_split(all_x_train, all_y_train)
scaler = StandardScaler()
```

```
x_train_scaled = scaler.fit_transform(x_train)
x_validation_scaled = scaler.transform(x_validation)
x_test_scaled = scaler.transform(x_test)
```

À la fin de l'exécution de ce dernier code, les variables `x_train_scaled`, `x_validation_scaled` et `x_test_scaled` vont contenir respectivement les données que nous allons utiliser pour entraîner notre réseau de neurones, les données de validation que nous allons utiliser pour l'évaluation du réseau de neurones à la fin de chaque cycle et les données de test. Ces trois jeux de données ont été normalisés en utilisant la classe `StandardScaler` comme nous l'avons fait lors des chapitres précédents.

Le code suivant commence la construction de notre réseau de neurones par la définition des trois couches d'entrée `input_1`, `input_2` et `input_3`. Pour cela, nous avons utilisé des couches de type `keras.layers.Input`.

```
input_1 = keras.layers.Input(shape=x_train_scaled.shape[1:])
input_2 = keras.layers.Input(shape=[5])
input_3 = keras.layers.Input(shape=[3])
```

Remarquez que nous avons défini l'entrée de `input_1` avec toutes les variables de `x_train`, alors que l'entrée de `input_2` est limitée aux cinq premières variables de `x_train` et `input_3` est limitée aux variables allant de la sixième variable jusqu'à la dernière variable de `x_train`.

Conformément à notre objectif représenté dans la figure 14-34 plus haut, cette répartition des variables entre ces trois couches d'entrée va nous permettre d'introduire toutes les données de `x_train` dans la première couche cachée avec `input_1`, puis de réintroduire les cinq premières variables directement au niveau de la troisième couche cachée avec `input_2`, et enfin les variables à partir de la sixième variable vont être réintroduites directement au niveau de la première couche de sortie avec `input_3`.

Remarque

Dans cet exemple, la stratégie de répartition des variables entre les trois couches d'entrée est réalisée de manière complètement aléatoire puisque, pour rappel, l'objectif n'est pas de construire un modèle performant mais plutôt de montrer comment utiliser Functional API pour réaliser des réseaux de neurones complexes.

© Editions ENI - All rights reserved

Le code suivant permet de définir la première couche cachée `hidden1` de type `Dense` avec 60 neurones et avec la fonction d'activation `relu`. Cette première couche cachée prend ses entrées à partir de la couche d'entrée `input_1`.

Remarquez que `input_1` a été connecté à l'entrée de `hidden1` en l'indiquant entre parenthèses comme si cette couche `input_1` représentait le paramètre d'une fonction ! C'est par rapport à cette notation qui permet de connecter les couches entre elles que cette API est appelée Functional API, puisque ces connexions se réalisent comme si c'étaient des appels à des fonctions.

De la même manière que `hidden1`, la deuxième couche cachée `hidden2` est définie de type `Dense` et avec 30 neurones et la fonction d'activation `relu`. La sortie de la couche cachée `hidden1` est connectée à l'entrée de la couche cachée `hidden2`.

La couche `concat_1` est de type `keras.layers.concatenate`. Cette couche permet tout simplement de regrouper les sorties des deux couches `input_2` et `hidden2`, cela va nous permettre de connecter ces deux sorties à la prochaine couche cachée qui sera définie juste après.

```
hidden1 = keras.layers.Dense(60, activation="relu")(input_1)
hidden2 = keras.layers.Dense(30, activation="relu")(hidden1)
concat_1 = keras.layers.concatenate([input_2, hidden2])
```

Le code suivant permet de définir la troisième couche cachée `hidden3` avec 10 neurones et avec la fonction d'activation `relu`. Cette troisième couche cachée prend ses entrées à partir de la sortie de la couche `concat_1`.

La sortie de cette dernière couche cachée `hidden3` est regroupée avec la sortie de la couche d'entrée `input_3`, grâce à la couche de concaténation `concat_2`.

```
hidden3 = keras.layers.Dense(10, activation="relu")(concat_1)
concat_2 = keras.layers.concatenate([input_3, hidden3])
```

À ce stade, il ne nous reste plus qu'à définir les deux couches de sortie de notre réseau de neurones.

5.2 Un modèle Functional API avec plusieurs couches de sortie

Le code suivant permet de définir deux couches `output_1` et `output_2` de type `Dense` chacune composée d'un seul neurone.

L'entrée de la couche de sortie `output_1` est connectée à la sortie de la couche `concat_2` et l'entrée de la couche de sortie `output_2` est connectée à la sortie de la troisième couche cachée `hidden3`, toujours conformément à notre objectif de la figure 14-34.

Ces deux couches de sortie ont été définies avec un seul neurone chacune car l'objectif de ce réseau de neurones est de réaliser une régression linéaire, donc à la sortie de ces deux couches nous allons avoir des prédictions de nombres réels et pour cela nous n'avons besoin que d'un seul neurone dans chacune des deux couches.

```
output_1 = keras.layers.Dense(1)(concat_2)
output_2 = keras.layers.Dense(1)(hidden3)
```

Dans cet exemple, nous avons utilisé deux couches de sortie qui vont être utilisées pour la prédiction d'un même objectif, à savoir le prix d'un appartement en fonction de certaines variables.

Cependant, même si ces deux couches de sortie donnent une évaluation d'un même objectif, nous pouvons voir l'intérêt de réintroduire les données de `input_3` à la première couche de sortie `output_1` en comparant les résultats obtenus au niveau de cette sortie avec ceux obtenus au niveau de la sortie `output_2`.

Nous sommes enfin prêts pour définir l'ensemble de notre réseau de neurones. La variable `model` du code suivant correspond à notre réseau de neurones où nous avons précisé avec l'argument `inputs` que ce modèle prend ses entrées à partir des trois couches `input_1`, `input_2` et `input_3` et nous avons précisé avec l'argument outputs que les sorties de ce réseau de neurones sont les couches `output_1` et `output_2`.

```
model = keras.models.Model(inputs=[input_1, input_2, input_3],
outputs=[output_1, output_2])
```

© Editions ENI - All rights reserved

Le code suivant permet de compiler notre modèle. Cette compilation se réalise de la même manière que pour le modèle de classification d'images précédent, sauf que cette fois la fonction `loss` utilisée est `mse` (*mean squared error*), la fonction d'optimisation est `rmsprop` et la métrique de performance utilisée est `mae` (*mean absolute error*).

```
model.compile(loss="mse", optimizer="rmsprop", metrics=["mae"])
```

Avant de lancer la méthode `fit` pour entraîner notre réseau de neurones, nous devons d'abord préparer les données à injecter dans nos trois couches d'entrée. Pour la première couche d'entrée `input_3` nous allons utiliser toutes les variables, donc les données de `x_train_scaled` seront conformes. En revanche, pour `input_2` il est nécessaire d'extraire les cinq premières variables de `x_train_scaled` et pour `input_3` il est nécessaire d'extraire le reste des variables à partir de la sixième variable. Ces mêmes opérations seront nécessaires pour les données de validation et les données de test.

Le code suivant permet de réaliser toutes ces opérations pour ainsi préparer les entrées des deux couches `input_2` et `input_3`.

```
x_train_2, x_train_3 = x_train_scaled[:, :5],
x_train_scaled[:, 5:]
x_validation_2, x_validation_3 = x_validation_scaled[:, :5],
x_validation_scaled[:, 5:]
x_test_2, x_test_3 = x_test_scaled[:, :5], x_test_scaled[:, 5:]
```

`x_train_2` et `x_train_3` seront utilisés pour `input_2` et `input_3` lors de la phase d'entraînement, `x_validation_2` et `x_validation_3` seront utilisés pour `input_2` et `input_3` lors de l'évaluation du modèle à la fin de chaque itération, et enfin `x_test_2` et `x_test_3` seront utilisés pour `input_2` et `input_3` pour l'évaluation finale du réseau de neurones.

Le code suivant permet de lancer la phase d'entraînement avec 100 itérations :

```
ressults = model.fit((x_train_scaled , x_train_2, x_train_3),
                     [y_train, y_train], epochs=100,\
                     validation_data=((x_validation_scaled,
                                       x_validation_2,
                                       x_validation_3),
                                      [y_validation,y_validation]))
```

À la fin de chaque itération, l'évaluation de la fonction `loss` ainsi que de la métrique `mae` au niveau des deux couches de sortie seront affichées.

Le code suivant permet d'évaluer les deux couches de sortie en utilisant les données de `x_test_scaled`, `x_test_2`, `x_test_3`.

```
res_eval = model.evaluate((x_test_scaled, x_test_2, x_test_3),
[y_test, y_test])
```

Le résultat obtenu est le suivant :

```
5160/5160 [==============================] - 0s 24us/sample -
loss: 0.6470 - dense_8_loss: 0.3207 - dense_9_loss: 0.3244 -
dense_8_mae: 0.3584 - dense_9_mae: 0.3590
```

On voit que les résultats obtenus au niveau des deux couches de sortie sont très proches en matière de `mae`.

Pour utiliser ce modèle et réaliser des prédictions, nous devons également fournir les données pour les trois couches d'entrée `inpt_1`, `input_2` et `input_3` comme le montre l'exemple de code suivant.

Ce code utilise des données à partir de `x_test_scaled` pour illustrer comment réaliser des prédictions avec ce réseau de neurones à plusieurs couches d'entrée et plusieurs couches de sortie. Ainsi, nous avons utilisé les trois premières lignes de `x_test_scaled` pour réaliser trois prédictions !

```
x_new_1, x_new_2, x_new_3 = x_test_scaled[:3],
x_test_scaled[:3, :5], x_test_scaled[:3, 5:]
y_pred = model.predict((x_new_1, x_new_2,x_new_3))

print(f"y_pred = {y_pred}")
```

Le résultat est le suivant :

```
y_pred = [array([[1.0698228],[0.8772676],[1.882385 ]],
          dtype=float32),
          array([[1.0703032],[0.84909767],[1.891499  ]],
          dtype=float32)]
```

La première ligne de ce résultat correspond aux prédictions de `output_1` et la deuxième ligne aux prédictions de `output_2`.

© Editions ENI - All rights reserved

6. Opérations avancées sur les réseaux de neurones

6.1 Monitorer un réseau de neurones

6.1.1 Contrôler les critères d'arrêt avec les callbacks

- Pour réaliser cet exemple, créez un nouveau notebook, puis suivez les étapes de code ci-après ; vous pouvez également utiliser directement le notebook *Cifar10.ipynb*.

Le code suivant commence par charger les images du dataset `cifar10` à partir de `keras.datasets`. Ce jeu de données contient des images dont chacune correspond à un des objets définis dans la liste `cifar10_class_names`.

Pour montrer quelques fonctionnalités de `tensorflow.keras` intéressantes et couramment utilisées, nous allons développer un exemple dans lequel nous allons construire un réseau de neurones de classification d'images. Ce réseau de neurones, une fois entraîné, devrait classer une image donnée dans l'une des dix classes listées dans `cifar10_class_names`.

Après avoir chargé le jeu de données `cifar10`, ce code définit le réseau de neurones `model` avec deux couches cachées en utilisant Sequential API selon plus au moins les mêmes étapes que celles que nous avons suivies lorsque nous avons développé le réseau de neurones de classification des images de `fashion_mnist`.

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
import numpy as np

os.environ['TF_CPP_MIN_LOG_LEVEL']='2'

cifar10_class_names = {
   0:"Plane",
   1:"Car",
   2:"Bird",
   3:"Cat",
   4:"Deer",
```

```
    5:"Dog",
    6:"Frog",
    7:"Horse",
    8:"Boat",
    9:"Truck",
}

cifar10 = keras.datasets.cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train = x_train.astype("float32")
x_test = x_test.astype("float32")

x_train= x_train/255
x_test = x_test /255

y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)

model = keras.Sequential()

model.add(keras.layers.Flatten(input_shape=(32,32,3)))
model.add(keras.layers.Dense(500, activation='relu'))
model.add(keras.layers.Dense(250, activation='relu'))
model.add(keras.layers.Dense(10, activation="softmax"))

model.compile(loss="categorical_crossentropy", optimizer="adam",
metrics=["accuracy"] )
```

La différence majeure entre la structure de ce réseau de neurones et la structure du réseau de neurones de classification des images du dataset `fashion_mnist` précédent réside dans l'utilisation de la fonction `loss categorical_crossentropy` au lieu de la fonction `sparse_categorical_crossentropy`. En effet, cette fois les résultats attendus au niveau de la couche de sortie correspondent à des valeurs binaires renvoyées par chaque neurone de la couche de sortie.

© Editions ENI - All rights reserved

Lorsque nous avons lancé les phases d'entraînement de nos deux derniers réseaux de neurones, nous avons fixé le nombre d'itérations total que chacun de ces deux réseaux de neurones doit réaliser. En réalité, un réseau de neurones peut atteindre un point de convergence bien avant d'avoir consommé toutes les itérations fixées avec le paramètre `epochs`. Et dans ce cas où un réseau de neurones a convergé, il n'est plus nécessaire de continuer le déroulement des itérations restantes, puisque la probabilité que ce réseau de neurones puisse sortir de l'optimum local sous-jacent à cette convergence est très faible, voire nulle.

Pour éviter de continuer un grand nombre d'itérations inutilement, nous pouvons utiliser la fonction callback `EarlyStopping` comme le montre le code suivant :

```
early_stopping = keras.callbacks.EarlyStopping(patience=10,
                 restore_best_weights=True)
```

Ce code permet de créer l'objet `early_stopping` qui sera utilisé plus tard au moment du lancement de la méthode `fit`. L'argument `patience` permet d'indiquer le nombre d'itérations successives sans amélioration de la métrique de performance qui va être utilisée comme indice de convergence. Cela signifie que, dans notre exemple, si au bout de dix itérations successives la métrique de performance n'est pas améliorée, alors nous allons considérer que notre réseau de neurones a atteint un point de convergence et donc nous allons arrêter la phase d'entraînement. L'argument `restore_best_weights` permet d'assigner aux poids du réseau de neurones les valeurs qui ont permis d'obtenir le meilleur résultat connu au moment de la convergence. Le calcul de la métrique de performance qui sera utilisé conjointement avec cette fonction callback sera réalisé sur les données de validation. Donc, si nous utilisons une telle fonction callback, nous sommes obligés d'utiliser des données de validation au moment du lancement de la phase d'entraînement. Nous allons voir dans la suite de cette section comment utiliser cette fonction callback et nous allons observer l'effet potentiel de son utilisation en tant que critère d'arrêt.

Il existe plusieurs fonctions callback proposées avec Keras selon les besoins. Nous pouvons également développer nos propres fonctions callback qui seront exécutées pendant la phase d'entraînement.

6.1.2 TensorBoard

TensorBoard est un outil intéressant qui permet de visualiser graphiquement les résultats des exécutions de nos réseaux de neurones. Cet outil est installé par défaut au moment de l'installation de TensorFlow ; pour l'utiliser, il n'est donc pas nécessaire d'installer de nouveaux paquets.

Sans entrer dans les détails de l'utilisation de l'outil TensorBoard, le code suivant permet de montrer comment l'activer.

```
logdir = os.path.join(os.curdir, "../MyLogs")
def get_logdir():
    import time
    run_id = time.strftime("folder_%Y_%m_%d-%H_%M_%S")
    return os.path.join(logdir, run_id)

current_logdir = get_logdir()
```

Ce code permet tout simplement de créer un dossier *MyLogs* si celui-ci n'existe pas et de créer un nouveau dossier dans *MyLogs* dans lequel seront stockés les résultats de la phase d'entraînement de notre réseau de neurones. À chaque exécution de notre code, un nouveau dossier sera donc créé est nommé avec le préfixe *folder_* suivi de la date et l'heure en cours. Cela permettra d'avoir un dossier différent à chaque exécution.

Une fois que le nouveau dossier *current_logdir* est créé, le code suivant permet de créer une fonction callback de type `keras.callbacks.TensorBoard` en lui transmettant le nom de ce nouveau dossier. Cette fonction callback sera utilisée plus tard lors du lancement de la phase d'entraînement au même titre que la fonction callback précédemment créée.

```
tensorboard_callback =
            keras.callbacks.TensorBoard(current_logdir)
```

Le code suivant permet justement de lancer la phase d'entraînement de notre réseau de neurones `model`. La nouveauté de ce code réside dans l'utilisation de l'argument `callbacks` de la méthode `fit` qui permet d'indiquer les deux fonctions callbacks `early_stopping`, qui permet de stopper la phase d'entraînement lorsqu'un point de convergence sera détecté, et la fonction `tensorboard_callback`, qui permet de stocker les résultats de l'évaluation de chaque itération sur les données de validation.

© Editions ENI - All rights reserved

L'argument `shuffle`, lorsqu'il est égal à `True`, permet d'indiquer notre souhait de changer l'ordre de passage des images dans le réseau de neurones d'une itération à une autre, cela peut influencer positivement les performances d'un réseau de neurones.

```
result = model.fit(x_train, y_train,\
                   batch_size=32,\
                   epochs=500,\
                   validation_data=[x_test, y_test],\
                   shuffle=True,\
                   callbacks=[early_stopping,
                              tensorboard_callback])
```

Les résultats de notre réseau de neurones sont les suivants au bout de la 26e et dernière itération :

```
Epoch 26/500
50000/50000 [==============================] - 14s 271us/sample -
loss: 1.2307 - accuracy: 0.5578 - val_loss: 1.4593 -
val_accuracy: 0.4956
```

En effet, la phase d'apprentissage a pris fin au bout de la 26e itération au lieu des 500 itérations prévues avec l'argument `epochs=500` de la méthode `fit`, car les itérations entre 16 et 26 n'ont pas réussi à améliorer la métrique `accuracy`. Cet arrêt prématuré de la phase d'apprentissage est une conséquence de l'utilisation de la fonction callback `early_stopping` de type `keras.callbacks.EarlyStopping`, qui est, pour rappel, exécutée à chaque fin d'une itération pour évaluer le réseau de neurones sur les données de validation.

Les résultats de l'évaluation sur les données de test sont de l'ordre de 49.56 % pour la métrique `accuracy`.

Remarque

On constate donc que notre réseau de neurones n'est pas suffisamment performant ! Nous allons comparer ces résultats avec un réseau de neurones plus sophistiqué que nous allons développer plus loin dans ce chapitre lorsque nous réaliserons un réseau de neurones avec des couches à convolution.

À la fin de la phase d'entraînement de notre réseau de neurones, vous allez trouver le dossier *MyLog* créé dans le dossier Code de ce chapitre. Dans ce dossier, vous constaterez également qu'un sous-dossier est aussi créé dont le nom commence par *folder_* et suivi de la date et heure correspondant à la dernière exécution. Si vous exécutez de nouveau la méthode `fit` précédente, vous allez constater qu'un deuxième sous-dossier sera créé dans le dossier *MyLogs*.

- Pour pouvoir utiliser TensorBoard et visualiser les résultats stockés dans le dossier *MyLogs*, lancez la commande suivante dans une console Anaconda (voir la figure 14-17 qui illustre comment lancer la console Anaconda) :

```
activate TensorFlow_Learning
```

Cette commande permet d'activer l'environnement virtuel dans lequel nous avons installé, entre autres, la bibliothèque TensorFlow.

- Dans la console Anaconda, positionnez-vous dans le dossier où vous avez créé votre notebook pour cet exemple, ou positionnez-vous dans le dossier Code de ce chapitre si vous avez utilisé le notebook *Cifar10.ipynb*.
- Exécutez la commande suivante afin de lancer le serveur TensorBoard.

```
tensorboard --logdir=./MyLogs --port=6006
```

Cette commande devrait afficher le message suivant :

```
Serving TensorBoard on localhost; to expose to the network, use a
proxy or pass --bind_all
TensorBoard 2.4.0 at http://localhost:6006/ (Press CTRL+C to quit)
```

© Editions ENI - All rights reserved

- Utilisez l'URL http://localhost:6006/ dans votre navigateur web afin de lancer la page d'accueil de TensorBoard. Vous devriez obtenir un résultat similaire à celui de la figure suivante :

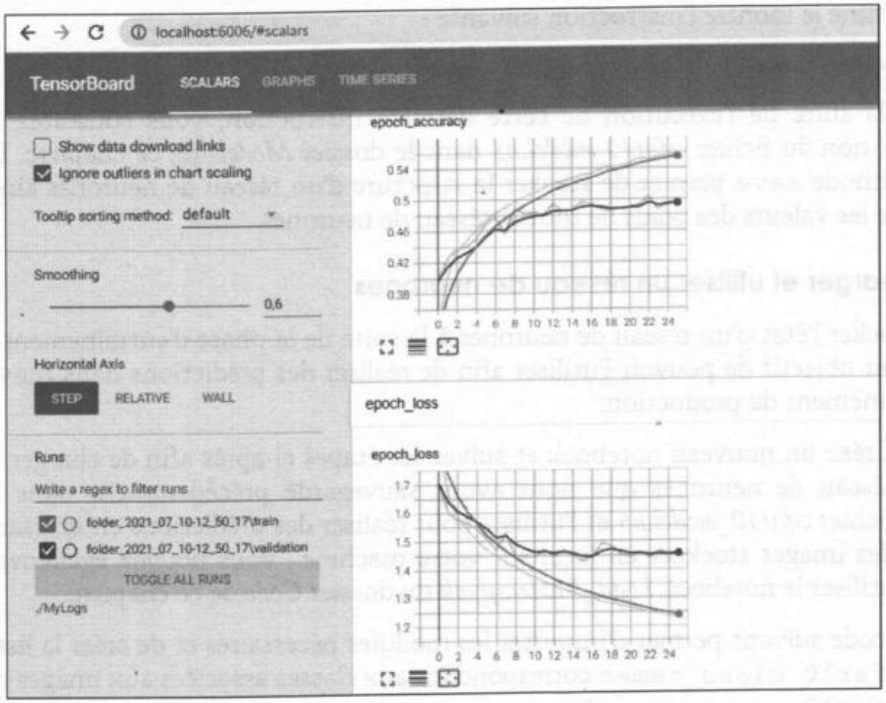


Figure 14-35 : visualisation des résultats de la phase d'entraînement avec TensorBoard

Vous pouvez commencer à explorer les informations affichées dans TensorBoard !

Par exemple, sur la page d'accueil de TensorBoard, on voit s'afficher les courbes d'évolution de la fonction `loss` et de la métrique de performance `accuracy` sur les données d'entraînement et sur les données de validation au fil des itérations. En cliquant sur le lien **Graphs**, vous pouvez visualiser la structure de notre réseau de neurones.

6.1.3 Sauvegarder un réseau de neurones

Pour sauvegarder dans un fichier l'état de notre réseau de neurones, une fois que son entraînement est terminé, nous pouvons utiliser la méthode `save` comme le montre l'instruction suivante :

```
model.save("../Models/cifar10_model.h5")
```

À la suite de l'exécution de cette dernière instruction, vous constatez la création du fichier *cifar10_model.h5* dans le dossier *Models* de ce chapitre. La méthode `save` permet de stocker la structure d'un réseau de neurones ainsi que les valeurs des poids de tout le réseau de neurones.

6.1.4 Charger et utiliser un réseau de neurones

Stocker l'état d'un réseau de neurones à la suite de la phase d'entraînement a pour objectif de pouvoir l'utiliser afin de réaliser des prédictions dans l'environnement de production.

- Créez un nouveau notebook et suivez les étapes ci-après afin de charger le réseau de neurones que nous avons sauvegardé précédemment dans le fichier *cifar10_model.h5* et l'utiliser pour réaliser des prédictions en utilisant des images stockées en local sur votre machine ; vous pouvez également utiliser le notebook *Load_Model.ipynb* du dossier *Code* de ce chapitre.

Le code suivant permet d'importer les modules nécessaires et de créer la liste `cifar10_class_names` correspondant aux classes associées aux images de `cifar10`.

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
import numpy as np


os.environ['TF_CPP_MIN_LOG_LEVEL']='2'


cifar10_class_names = {
   0:"Plane",
   1:"Car",
```

© Editions ENI - All rights reserved

```
    2:"Bird",
    3:"Cat",
    4:"Deer",
    5:"Dog",
    6:"Frog",
    7:"Horse",
    8:"Boat",
    9:"Truck",
}
```

Pour charger un réseau de neurones stocké au préalable dans un fichier, nous pouvons utiliser la méthode `load_model` comme le montre l'instruction suivante :

```
model = keras.models.load_model("../Models/cifar10_model.h5")
```

À l'exécution de cette instruction, nous aurons chargé le réseau de neurones que nous avions construit précédemment dans la variable `model`.

Le code suivant permet de charger les deux images *dog.jpg* et *cat.jpg* en utilisant la méthode `load_img`. Ces deux images sont formatées afin qu'elles soient dans le format attendu par la couche d'entrée de notre réseau de neurones. Une fois formatées, les deux images ont été stockées dans le tableau `my_images`. En utilisant ce tableau d'images, nous allons pouvoir réaliser des prédictions sur un ensemble d'images et non pas des prédictions image par image.

```
my_images = np.zeros((2, 32, 32, 3))
img1= keras.preprocessing.image.load_img("../Data/dog.jpg",
target_size=(32,32))
img2= keras.preprocessing.image.load_img("../Data/cat.jpg",
target_size=(32,32))
img1 = keras.preprocessing.image.img_to_array(img1) / 255
img2 = keras.preprocessing.image.img_to_array(img2) / 255


my_images[0, :] = img1
my_images[1, :] = img2
```

La méthode `predict` ci-après permet de réaliser des prédictions sur les images de `my_images`.

Les résultats de ces prédictions qui seront stockés dans la variable `results` seront affichés en indiquant la probabilité associée.

```
results = model.predict(my_images)


for r in results:
   predected_class_index = int(np.argmax(r))
   predected_prob = r[predected_class_index]
   print(f"{cifar10_class_names[predected_class_index]} - prob =
         {round(float(predected_prob),2)} ")
```

Le résultat est le suivant :

```
Dog - prob = 0.58
Cat - prob = 0.54
```

Donc, bien qu'il ne soit pas très performant sur les données de test, notre réseau de neurones parvient à classer correctement nos deux images !

6.2 Réseaux de neurones de convolution

Dans cette section, nous allons développer un réseau de neurones de classification des images du dataset `cifar10` comme nous l'avons fait précédemment, sauf que cette fois nous allons utiliser, entre autres, des couches de convolution.

- Créez un nouveau notebook et suivez les étapes ci-après pour réaliser ce nouvel exemple ; vous pouvez également utiliser directement le notebook *CNN.ipynb* du dossier *Code* de ce chapitre.

La première partie de ce nouvel exemple de code est similaire à la première partie du code de l'exemple de la section Monitorer un réseau de neurones précédente. Ce code permet entre autres de charger et formater les données du dataset `cifar10`.

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
import numpy as np
```

© Editions ENI - All rights reserved

```
os.environ['TF_CPP_MIN_LOG_LEVEL']='2'

cifar10_class_names = {
    0:"Plane",
    1:"Car",
    2:"Bird",
    3:"Cat",
    4:"Deer",
    5:"Dog",
    6:"Frog",
    7:"Horse",
    8:"Boat",
    9:"Truck",
}

cifar10 = keras.datasets.cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()


x_train = x_train.astype("float32")
x_test = x_test.astype("float32")

x_train= x_train/255
x_test = x_test /255


y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)
```

Le code suivant permet de développer un réseau de neurones avec des couches à convolution.

Plus précisément, ce nouveau réseau de neurones est composé des douze couches suivantes :

- Une couche d'entrée est elle-même une couche à convolution de type `Conv2D` qui applique 32 filtres. Chacun de ces filtres est représenté avec une matrice de 3*3.
- Une couche de convolution de type `Conv2D` de 32 filtres de 3*3 chacun.

- Une couche est de type `MaxPool2D`. Ce type de couche permet de diminuer la taille des données issues de la couche précédente afin de ne garder que les données jugées comme étant les plus pertinentes et sans redondance.
- Une couche de type `Dropout` avec un taux de poids à ignorer de 25 %.
- Une couche à convolution de type `Conv2D` avec 64 filtres de 3*3 chacun.
- Une couche à convolution de type `Conv2D` avec 64 filtres de 3*3 chacun.
- Une couche de type `MaxPool2D`.
- Une couche de type `Dropout` avec un taux de poids à ignorer de 25 %.
- Une couche de type `Flatten`. Cette couche va permettre de transformer les matrices reçues en entrée vers des tableaux à une dimension.
- Une couche de type `Dense` avec 512 neurones.
- Une couche de type `Dropout` avec un taux de poids à ignorer de 50 %.
- Une couche de sortie de type `Dense` avec 10 neurones.

```
model = keras.Sequential()

model.add(keras.layers.Conv2D(32,(3,3), padding="same",
        activation ="relu", input_shape=(32,32,3)))

model.add(keras.layers.Conv2D(32,(3,3), activation ="relu"))

model.add(keras.layers.MaxPool2D(pool_size=(2,2)))

model.add(keras.layers.Dropout(0.25))

model.add(keras.layers.Conv2D(64,(3,3), padding="same",
        activation ="relu"))

model.add(keras.layers.Conv2D(64,(3,3), activation ="relu"))

model.add(keras.layers.MaxPool2D(pool_size=(2,2)))

model.add(keras.layers.Dropout(0.25))

model.add(keras.layers.Flatten())

model.add(keras.layers.Dense(512, activation='relu'))
```

© Editions ENI - All rights reserved

```
model.add(keras.layers.Dropout(0.5))

model.add(keras.layers.Dense(10, activation="softmax"))


model.compile(loss="categorical_crossentropy",
              optimizer="adam",
              metrics=["accuracy"].)

early_stopping = keras.callbacks.EarlyStopping(patience=10,
                 restore_best_weights=True)

result = model.fit(x_train, y_train,\
                   epochs=1,\
                   validation_data=[x_test, y_test],\
                   callbacks=[early_stopping])
```

À la fin de l'exécution de ce code, les résultats suivants ont été obtenus :

```
Epoch 33/500
50000/50000 [==============================] - 111s 2ms/sample -
loss: 0.4777 - accuracy: 0.8332 - val_loss: 0.6576 -
val_accuracy: 0.7918
```

On voit que notre réseau de neurones à convolution donne des résultats nettement meilleurs que ceux que nous avons obtenus précédemment avec un réseau de neurones sans l'utilisation des couches de convolution. Pour rappel, ces résultats étaient les suivants :

```
Epoch 26/500
50000/50000 [==============================] - 14s 271us/sample -
loss: 1.2307 - accuracy: 0.5578 - val_loss: 1.4593 -
val_accuracy: 0.4956
```

En résumé, avec l'utilisation des couches de convolution, nous sommes passés de 49.56 % à 79.18 % de prédictions correctes.

Un autre point intéressant à observer dans ces résultats est que notre dernier réseau de neurones a mis plus d'itérations que le réseau de neurones précédent avant de converger : 33 itérations pour le dernier modèle contre 26 itérations pour le modèle précédent. C'est peut-être une conséquence de l'utilisation des couches de type Dropout dans notre dernier réseau de neurones.

6.3 Réutiliser un réseau de neurones

Réutiliser un réseau de neurones revient tout simplement à charger ce réseau de neurones en mémoire et l'utiliser tout comme nous aurions utilisé un de nos réseaux de neurones !

En réalité, nous avons déjà eu l'occasion de réutiliser un réseau de neurones lorsque nous avons chargé le réseau de neurones qui était stocké dans le fichier *cifar10_model.h5*. Cependant, cette section montre comment réutiliser des réseaux de neurones développés par des équipes tierces et mises à disposition lors de l'installation de TensorFlow !

Ainsi, nous allons utiliser un des réseaux de neurones disponibles dans `tensorflow.keras.applications`. Nous allons charger le réseau de neurones `vgg16` qui est composé de 16 couches cachées et qui permet de détecter la présence dans une image de 1 000 objets différents ! Les objets reconnus avec `vgg16` sont tous listés dans le fichier *imagenet_1000_classes.txt* inclus dans le dossier Code de ce chapitre, il s'agit des objets de la base *imagenet*.

Ce type de réseau de neurones est intéressant car, si nous souhaitons développer un nouveau réseau de neurones pour détecter des objets dans des images et que ces objets font partie des objets couverts avec un réseau de neurones déjà existant, alors nous n'avons pas besoin d'entraîner notre propre réseau de neurones puisqu'il suffit d'utiliser l'un de ces réseaux de neurones qui sont déjà entraînés pour la reconnaissance des mêmes objets que ceux que nous souhaitons inclure dans notre projet !

Pour mieux comprendre, nous allons réaliser un exemple dans lequel le réseau de neurones `vgg16` sera utilisé afin de reconnaître des chiens en utilisant nos propres images.

© Editions ENI - All rights reserved

- Créez un nouveau notebook ou utilisez le notebook *Reuse_vgg16.ipynb* du dossier *Code* de ce chapitre.

Le code suivant permet de charger avec l'instruction `model = vgg16.VGG16()` le réseau de neurones `vgg16` qui est, pour rappel, entraîné pour reconnaître les 1 000 objets d'*imagenet*. Le reste du code permet de charger l'image *border-collie-2648945_1920.jpg*, puis d'utiliser cette image pour voir la réponse de notre réseau de neurones.

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications import vgg16

os.environ['TF_CPP_MIN_LOG_LEVEL']='2'

model = vgg16.VGG16()

img_src =
image.load_img("../Data/Dogs/train/male/border-colli-
            2648945_1920.jpg", target_size=(224,224))

img_test = image.img_to_array(img_src)

img_test_lst = np.expand_dims(img_test, axis=0)

img_test_lst = vgg16.preprocess_input(img_test_lst)


predictions = model.predict(img_test_lst)


predected_classes = vgg16.decode_predictions(predictions, top=5)

print(predected_classes)
```

Le résultat de ce code est le suivant :

```
[[('n02106166', 'Border_collie', 0.60584766), ('n02106030',
'collie', 0.27727306), ('n02090622', 'borzoi', 0.054326974),
('n02092002', 'Scottish_deerhound', 0.022940766), ('n02107683',
'Bernese_mountain_dog', 0.012941362)]]
```

On voit que ce réseau de neurones reconnaît bien que notre image *border-collie-2648945_1920.jpg* est une image où figure un chien et cela avec une probabilité de 60,58 !

Ce réseau de neurones parvient non seulement à reconnaître que cette image correspond à un chien mais en plus, ce modèle reconnaît aussi la race de ce chien, Border Collie.

Cela n'est bien sûr pas gênant dans la mesure où notre objectif est la reconnaissance de chiens dans des images, quelle que soit leur origine.

Le code de cet exemple nous a montré comment réutiliser un réseau de neurones déjà existant afin de nous éviter de réaliser par nous-mêmes les phases d'entraînement. Cela est d'autant plus important que généralement nous ne disposons pas de suffisamment d'images pour entraîner efficacement nos réseaux de neurones !

6.4 Le Transfer Learning

Dans cette section, nous allons mettre en œuvre via un exemple concret toutes les étapes de Transfer Learning évoquées au début de ce chapitre à la section précédente Le Transfer Learning. Pour illustrer la technique du Transfer Learning, nous allons nous fixer un objectif qui consiste à développer un réseau de neurones capable de reconnaître le sexe d'un chien à partir d'une image donnée. En d'autres termes, nous disposons de deux ensembles d'images de chiens. Un premier ensemble d'images de chiens dans le dossier *Data/Dogs/train* que nous souhaitons utiliser afin d'entraîner un réseau de neurones pour apprendre à reconnaître le sexe des chiens en fonction de leurs caractéristiques détectées à partir de ces images. Nous disposons également d'un deuxième ensemble d'images de chiens dans le dossier *Data/Dogs/test* que nous allons utiliser pour la phase d'évaluation.

© Editions ENI - All rights reserved

Chacun des deux ensembles d'images est divisé en deux sous-dossiers d'images : un dossier *males* contenant des images de chiens mâles et un deuxième sous-dossier *female* contenant des images de chiens femelles. Au total, il y a 41 images dans le dossier *train*.

Comme vous pouvez le constater, nous n'avons pas suffisamment d'images dans le dossier *train* pour nous permettre d'entraîner efficacement un réseau de neurones. En effet, avant qu'un réseau de neurones soit capable de reconnaître le sexe d'un chien à partir d'une image, il serait intéressant que ce réseau de neurones puisse d'abord reconnaître la présence d'un chien dans une image, et ce quel que soit son sexe, puis ensuite chercher à reconnaître son sexe !

Comme nous ne disposons pas de suffisamment d'images pour entraîner efficacement un réseau de neurones pour la reconnaissance de chiens, nous allons utiliser le concept du Transfer Learning en nous appuyant sur le réseau de neurones `VGG16`. Nous avons choisi d'utiliser `VGG16` puisque, comme nous l'avons vu précédemment, il est capable de reconnaître les chiens dans des images.

Ainsi, nous allons réaliser les étapes suivantes :

1. Charger le réseau de neurones `VGG16`.
2. Supprimer la couche de sortie du réseau de neurones `VGG16` pour ne garder que sa couche d'entrée et ses couches cachées.
3. Utiliser les couches cachées de `VGG16` pour extraire les caractéristiques des images du dossier *train*. Ces caractéristiques devraient permettre de reconnaître les 1 000 objets d'*imagenet* dont les chiens !
4. Stocker les caractéristiques extraites précédemment dans un fichier afin de les utiliser en tant que données d'entrée d'un nouveau réseau de neurones que nous allons créer.
5. Créer un nouveau réseau de neurones et l'entraîner avec les caractéristiques extraites précédemment afin que ce nouveau réseau de neurones apprenne à reconnaître le sexe des chiens.

- Créez un nouveau notebook et suivez les étapes de code ci-après afin d'implémenter un Transfer Learning et entraîner un réseau de neurones pour la reconnaissance du sexe des chiens ; vous pouvez utiliser directement le notebook *Transfer_Learning-1.ipynb* du dossier *Code*.

Le code suivant permet d'importer les bibliothèques nécessaires et de définir le chemin vers les données du dossier *train* :

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
import numpy as np
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications import vgg16

from pathlib import Path
import joblib


os.environ['TF_CPP_MIN_LOG_LEVEL']='2'


male_path = Path("../Data/Dogs/train/male")
female_path = Path("../Data/Dogs/train/female")
```

6.4.1 Chargement des données locales

Le code suivant permet de charger dans `train_images` les images du sous-dossier *male* du dossier *train*. Pour chacune de ces images correspondant aux chiens mâles, nous lui avons associé la valeur `0` dans la liste `train_target`. En d'autres termes, les chiens mâles sont représentés avec la classe `0`.

```
train_images = []
train_target = []

height = 128
width = 128

for current_file in male_path.glob("*.JPG"):
    current_img = image.load_img(current_file,
                                 target_size=(height,width))
    current_img = image.img_to_array(current_img)
```

© Editions ENI - All rights reserved

```
    train_images.append(current_img)
    train_target.append(0)
```

Le code suivant permet de charger dans `train_images` les images du sous-dossier *female* du dossier *train*. Pour chacune de ces images correspondant aux chiens femelles, nous lui avons associé la valeur 1 dans la liste `train_target`. En d'autres termes, les chiens femelles sont représentés avec la classe 1.

```
for current_file in female_path.glob("*.JPG"):
    current_img = image.load_img(current_file,
                                 target_size=(height,width))
    current_img = image.img_to_array(current_img)
    train_images.append(current_img)
    train_target.append(1)
```

Le code suivant permet de créer les deux tableaux NumPy `x_train` et `y_train` à partir respectivement des listes `train_images` et `train_target`, puis `x_train` est formaté dans le format attendu par la couche d'entrée de `VGG16` grâce à la méthode `preprocess_input`.

```
x_train = np.array(train_images)
y_train = np.array(train_target)
x_train = vgg16.preprocess_input(x_train)
```

6.4.2 Chargement du modèle VGG16

Le code suivant permet de charger le réseau de neurones `VGG16`, mais sans sa couche de sortie puisque nous avons indiqué `include_top=False`.

```
pretrained_model= vgg16.VGG16(weights="imagenet",
                              include_top=False,
                              input_shape=(height,width,3))
```

6.4.3 Extraction des features

La méthode `predict` du réseau de neurones précédent, qui représente la couche d'entrée et les 16 couches cachées de `VGG16`, permet d'obtenir les caractéristiques ou les features des 41 images de `x_train`.

```
features = pretrained_model.predict(x_train)
```

À ce stade, nous aurions donc stocké dans `features` la sortie de la dernière couche cachée de VGG16 pour chacune de nos 41 images. Ces caractéristiques devraient permettre de reconnaître la présence d'un chien dans une image !

6.4.4 Étendre un modèle

Le code suivant permet de créer un réseau de neurones simple composé des couches suivantes :

1. Une couche d'entrée dimensionnée pour recevoir les informations extraites de nos 41 images et qui sont stockées dans `features`.
2. Une couche cachée composée de 256 neurones afin d'apprendre à reconnaître le sexe des chiens.
3. Une couche de sortie composée d'un seul neurone afin de donner le verdict binaire par rapport au sexe détecté pour un chien donné.

```
model = keras.Sequential()

model.add(keras.layers.Flatten(input_shape=features.shape[1:]))

model.add(keras.layers.Dense(256, activation='relu'))

model.add(keras.layers.Dense(1, activation='sigmoid'))
```

Le code suivant permet de compiler notre réseau de neurones et de lancer la méthode `fit` comme nous l'avons fait précédemment.

Remarquez que nous avons utilisé la fonction callback `EarlyStopping` afin d'arrêter la procédure d'entraînement si un point de convergence est atteint avant la fin des 200 itérations prévues avec l'argument `epochs`.

```
model.compile(loss="binary_crossentropy",
              optimizer="adam",
              metrics=["accuracy"])

early_stopping = keras.callbacks.EarlyStopping(patience=10,
                                restore_best_weights=True)

result = model.fit(features, y_train,\
                   epochs=200,\
                   validation_data=[features, y_train],\
                   callbacks=[early_stopping])
```

© Editions ENI - All rights reserved

Le résultat suivant montre que la phase d'entraînement est arrêtée à l'itération 18 avec un taux d'accuracy de 100 % !

```
Epoch 18/200
41/41 [==============================] - 0s 854us/sample - loss:
5.5351e-06 - accuracy: 1.0000 - val_loss: 1.3136e-05 -
val_accuracy: 1.0000
```

À la fin de la phase d'entraînement, nous avons sauvegardé le réseau de neurones model dans le fichier *vgg_model.h5* avec l'instruction suivante :

```
model.save("../Models/vgg_model.h5")
```

6.4.5 Chargement des données de test pour le Transfer Learning

Pour tester le réseau de neurones que nous avons développé précédemment avec le concept du Transfer Learning, nous allons le charger et l'évaluer sur les images du dossier test.

- Créez un nouveau notebook et suivez les étapes de code ci-après pour évaluer notre réseau de neurones ; vous pouvez utiliser directement le notebook *Transfer_Learning-2.ipynb* du dossier *Code* de ce chapitre.

Sans entrer dans les détails, le code suivant permet de charger les images du sous-dossier *male* et celles du sous-dossier *female* du dossier *test*. Comme précédemment, la liste test_target contient des 0 pour les chiens mâles et des 1 pour les chiens femelles.

```
import tensorflow as tf
from tensorflow import keras
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications import vgg16

from pathlib import Path
import joblib


os.environ['TF_CPP_MIN_LOG_LEVEL']='2'
```

```
male_path = Path("../Data/Dogs/test/male")
female_path = Path("../Data/dogs/test/female")


test_images = []
test_target = []

height = 128
width = 128


for current_file in male_path.glob("*.JPG"):
   current_img = image.load_img(current_file,
target_size=(height,width))
   current_img = image.img_to_array(current_img)
   test_images.append(current_img)
   test_target.append(0)

for current_file in female_path.glob("*.JPG"):
   current_img = image.load_img(current_file,
target_size=(height,width))
   current_img = image.img_to_array(current_img)
   test_images.append(current_img)
   test_target.append(1)


x_test = np.array(test_images)
y_test = np.array(test_target
x_test = vgg16.preprocess_input(x_test)
```

Le code suivant permet de charger le réseau de neurones `model` à partir des informations enregistrées précédemment dans le fichier *vgg_model.h5* :

```
model = keras.models.load_model("../Models/vgg_model.h5")
```

Nous avons également chargé, comme précédemment, un réseau de neurones constitué de la couche d'entrée et des 16 couches cachées de `VGG16` dans la variable `pretrained_model`.

© Editions ENI - All rights reserved

```
pretrained_model= vgg16.VGG16(weights="imagenet",
        include_top=False, input_shape=(height,width,3))
```

Le code suivant extrait les caractéristiques des images de `x_test` en utilisant `VGG16` :

```
features_extracted =  pretrained_model.predict(x_test)
```

Puis notre réseau de neurones est utilisé avec l'instruction suivante pour prédire le sexe des chiens en se basant sur les caractéristiques extraites précédemment à partir des images de test :

```
results = model.predict_classes(features_extracted)
```

L'instruction suivante permet de comparer les résultats des prédictions réalisées avec notre réseau de neurones avec les vraies valeurs connues des images `x_test` :

```
df =
pd.DataFrame({"Prédictions":results[:,0],"Observation":y_test})
print(df)
```

Le résultat est le suivant :

	Prédictions	Observation
0	0	0
1	0	0
2	0	0
3	1	0
4	1	0
5	1	0
6	0	0
7	1	0
8	0	1
9	0	1
10	0	1
11	0	1
12	0	1
13	1	1
14	0	1
15	0	1

Figure 14-36 : résultats des prédictions sur le sexe des chiens versus les vraies valeurs

Ces résultats montrent que notre réseau de neurones parvient à reconnaître mieux les chiens mâles que les chiens femelles. En effet, on voit que sur les huit chiens mâles, quatre ont été reconnus en tant que tels, et sur les huit chiens femelles, il n'y en a qu'un qui a été reconnu en tant que femelle.

Plusieurs raisons peuvent justifier ces résultats pas assez concluants. Parmi ces résultats, nous pouvons citer :

1. La taille des données d'entraînement. En effet, pour entraîner notre réseau de neurones à distinguer les chiens mâles des chiens femelles, nous n'avons eu que 41 images. Cela n'est généralement pas assez pour aboutir à des modèles performants.
2. Le réseau de neurones que nous avons utilisé pour compléter le réseau de neurones `VGG16` est constitué d'une seule couche cachée. Peut-être qu'un réseau de neurones plus sophistiqué aurait pu être plus performant que le réseau de neurones que nous avons utilisé.

Bien sûr, l'objectif visé avec cet exeple n'est pas la construction du modèle le plus performant possible, mais plutôt de montrer une des techniques qui peret d'implémenter le Transfer Learning

7. Aller plus loin avec le Deep Learning et TensorFlow

Évidemment, il y a encore tellement de choses à dire sur le Deep Learning et sur TensorFlow ! Cependant, comme ce livre n'est dédié à aucun de ces deux sujets, nous n'allons pas aller plus loin dans l'exploration de ce fabuleux et prometteur domaine qu'est le Deep Learning.

Cependant, pour aller plus loin dans l'exploration des concepts de Deep Learning et l'exploitation de la bibliothèque TensorFlow, le lecteur est encouragé à creuser les pistes suivantes :

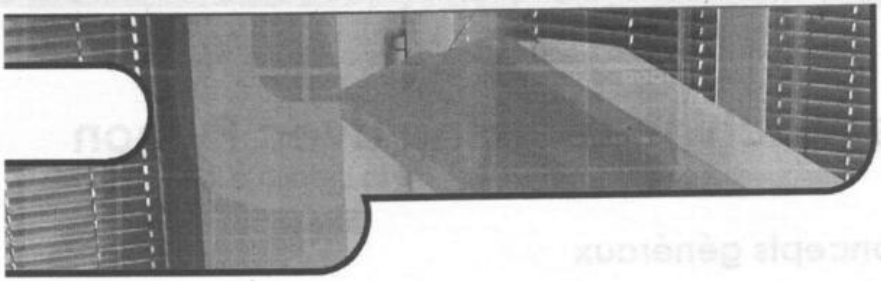
- Augmenter artificiellement la taille des données d'entraînement lorsque celles-ci ne sont pas suffisantes, comme dans notre dernier exemple.
- La documentation officielle de TensorFlow que vous pouvez consulter sur le site suivant : https://www.tensorflow.org/.

© Editions ENI - All rights reserved

- Un site plein de tutoriels et de projets réalisés avec TensorFlow : https://github.com/jtoy/awesome-tensorflow
- La page des différents datasets proposés avec TensorFlow : https://www.tensorflow.org/resources/models-datasets
- Le site officiel de TensorBoard pour le monitoring des résultats de vos réseaux de neurones : https://www.tensorflow.org/tensorboard/get_started

8. Conclusion

Ce chapitre nous a permis d'aborder les concepts fondamentaux de Deep Learning en complément des concepts de base abordés lors du chapitre précédent sur les réseaux de neurones. Ainsi, entre autres, nous avons eu l'occasion de mettre en pratique certains des concepts de Deep Learning en nous appuyant sur la bibliothèque TensorFlow. En résumé, après avoir abordé les opérations élémentaires possibles avec TensorFlow, nous avons développé des exemples de réseaux de neurones divers et variés. Nous avons réalisé un réseau de neurones de classification, un réseau de neurones pour la régression linéaire, nous avons vu comment réutiliser des réseaux de neurones existants tels que le réseau de neurones `VGG16`. Nous avons mis en pratique un réseau de neurones avec des couches de convolution et un réseau de neurones utilisant le concept du Transfer Learning. Avec les concepts abordés dans ce chapitre, la lecture de la documentation officielle de TensorFlow devrait normalement être abordable pour aller plus loin dans l'aventure Deep Learning !



Chapitre 5-2
Le traitement automatique du langage

1. Objectif du chapitre

Ce chapitre aborde le sujet du traitement automatique du langage, appelé en anglais *Natural Language Processing* (NLP). Après avoir passé en revue quelques concepts techniques clés du NLP, nous allons réaliser un modèle concret sur un vrai jeu de données qui sera capable de reconnaître et distinguer les spams des courriels ordinaires. Ce chapitre devrait être considéré comme une initiation au NLP où nous allons évoquer certaines notions élémentaires, mais qui sont suffisantes pour résoudre un large éventail de problèmes qui relèvent du domaine du traitement automatique du langage.

À la fin de ce chapitre, le lecteur aura abordé :

- les techniques de nettoyage de documents,
- les concepts de stopwords, de Stemming et de Lemmatization,
- les techniques de vectorisation des données textuelles,
- la vectorisation par comptage d'occurrences de mots,
- la vectorisation avec la méthode TF-IDF,
- la vectorisation avec N-Grams,
- le développement de modèle de classification de documents.

2. NLP : concepts généraux

Le traitement automatique du langage, également appelé le traitement automatique du langage naturel, est un domaine de l'intelligence artificielle.

Ces technologies du traitement automatique du langage naturel sont toutes des technologies dont le but est d'humaniser l'interface homme-machine, voire de doter la machine d'un caractère plus ou moins semblable à celui d'un humain.

Lors de l'apparition des moteurs de recherche, dont Google principalement, les utilisateurs ont dû adapter leur manière de s'exprimer pour retrouver les informations qui les intéressent sur Internet. À l'époque, les utilisateurs saisissaient des requêtes telles que "*comment changer une roue de voiture Peugeot 306*". Aujourd'hui, la même personne précisera juste "*changer roue 306*" et Google fera le reste ! Avec les technologies NLP, et grâce aux évolutions technologiques fulgurantes de ces dernières années, l'utilisateur est invité à utiliser son propre langage naturel pour interagir avec la machine, c'est-à-dire que c'est la machine qui va s'adapter à l'homme et non pas l'inverse.

Les cas d'utilisation du NLP sont divers et variés, nous pouvons citer notamment :

- la classification de documents : par exemple, une application qui analyse un courriel et décide si celui-ci est un spam ou pas,
- l'analyse des sentiments : par exemple, une application qui analyse les courriels envoyés par des clients afin de détecter ceux qui expriment un sentiment de mécontentement ; cela permet de prioriser le traitement de ces demandes,
- la reconnaissance d'entités nommées : par exemple, une application qui doit qualifier un ticket entrant dans une base de gestion d'incidents afin de lui attribuer la bonne priorité et orienter sa résolution vers les ressources humaines adéquates par rapport aux compétences du personnel du service. Grâce à la détection des entités nommées, une application peut détecter et comprendre le sens des phrases d'un document,
- la génération automatique de texte : une application qui échange avec les clients d'une entreprise via un chat,

© Editions ENI - All rights reserved

- le Part-Of-Speech (POS) tagging : par exemple, une application qui doit associer des métadonnées aux mots présents dans un document, comme la détection des nombres, des pronoms, des verbes, etc.,
- le développement des chatbots : par exemple, un robot ou un chatbot qui répond aux demandes des clients du service après-vente. Grâce au NLP, ces chatbots peuvent simuler des conversations humaines. D'ailleurs, on parle de plus en plus de chatbots dotés d'une "persona", ce qui est l'équivalent de la personnalité chez l'humain. Certains éditeurs de chatbots recrutent des acteurs pour substituer leurs voix avec celles des bots avec l'illusion d'injecter une âme à ces derniers.

Le NLP peut sans problème être considéré comme une application classique du Machine Learning. En effet, la différence entre une application dite de NLP d'une application classique en Machine Learning réside dans la nature des données utilisées !

Dans les chapitres précédents, nous avons vu que les algorithmes du Machine Learning traitent des données numériques, c'est-à-dire des nombres. Avec le NLP, les données traitées sont des données textuelles. Alors, comment peut-on considérer le NLP comme une application du Machine Learning classique alors que les données dont il est question avec le NLP sont des données textuelles et que les algorithmes du Machine Learning manipulent uniquement des données purement numériques ?

La réponse à cette question est simple. En NLP, nous pouvons utiliser tous les algorithmes classiques du Machine Learning, mais avant d'appliquer ces algorithmes, nous devons d'abord transformer les données textuelles en données numériques !

En effet, les algorithmes du Machine Learning ne savent pas traiter les données textuelles ! Pour pouvoir les utiliser et construire des modèles tels que les modèles de classification de documents, il est nécessaire de représenter ces données textuelles sous forme de nombres !

Justement, les deux sous-sections suivantes expliquent comment préparer des données textuelles et comment les transformer vers des données numériques afin de les préparer pour l'application des algorithmes du Machine Learning.

En effet, une fois que les textes contenus dans des documents sont transformés en données numériques, c'est-à-dire des chiffres, nous pouvons leur appliquer tout type d'algorithmes comme ceux que nous avons abordés dans les chapitres précédents.

2.1 Le nettoyage des données textuelles

Cette section aborde quelques méthodes qui permettent de réduire la taille des données textuelles, puis la section suivante aborde quelques méthodes afin de transformer les données sous forme de texte vers une représentation numérique.

2.1.1 Suppression des stopwords

Quel que soit le but visé lorsque nous analysons des données textuelles, souvent ces données contiennent une part non négligeable d'informations inutiles à notre analyse. Pour mener à bien nos analyses, il convient donc de supprimer ces informations pour ne garder que celles que nous jugeons utiles. Le fait de réduire la taille des documents traités a un impact positif sur les performances des algorithmes qui seront appliqués sur ces documents.

Justement, les stopwords sont tous les mots présents dans un document mais qui n'apportent aucune valeur ajoutée à ce document en matière de sémantique. En d'autres termes, ce sont des mots dont la suppression ne change en rien le sens d'un document.

Par exemple, supposons qu'un texte correspondant à un courriel contienne la phrase suivante :

```
Salut! Fais-moi signe quand tu seras arrivé… je serai déjà sur
place!
```

Pour appliquer le concept de suppression des stopwords sur ce message, nous allons procéder tout simplement à la suppression des mots qui sont considérés comme des mots sans valeur ajoutée dans la langue française. En effet, les mots considérés en tant que stopwords dépendent directement de la langue utilisée.

© Editions ENI - All rights reserved

Le résultat de la suppression des stopwords du précédent message est le suivant :

```
Salut! Fais signe quand arrivé... déjà place!
```

Vous remarquez que nous n'avons pas supprimé les signes de ponctuation, car ils ne sont pas considérés en tant que stopwords !

En revanche, les mots `moi`, `tu`, `seras`, `je`, `serai` et `sur` sont généralement considérés comme des stopwords.

Comme vous l'avez sans doute remarqué avec cet exemple, l'un des problèmes liés à la suppression des stopwords est que, sans eux, une phrase peut perdre une bonne partie du sens qu'elle véhicule ! En effet, le sens de la précédente phrase sans les stopwords peut facilement être interprété comme si à l'origine c'était la phrase :

```
Salut! Fais-moi signe quand tu seras arrivé déjà sur place !
```

Bien sûr, hormis qu'elle n'est pas correctement formée du point de vue syntaxique de la langue française, cette dernière phrase n'a pas exactement le même sens que la phrase d'origine, et c'est bien là le problème !

En plus de supprimer tous les stopwords d'un document, nous pouvons encore réduire la taille de ce document en procédant à la suppression de tous les signes de ponctuation. Évidemment, la suppression des signes de ponctuation n'est pertinente que dans la mesure où ils n'apportent aucune valeur ajoutée par rapport à ce que nous cherchons à déduire à partir de nos données textuelles. En effet, dans certains cas, les signes de ponctuation peuvent être d'une grande aide pour atteindre l'objectif. Par exemple, dans l'analyse des sentiments, les points d'exclamation, et même les points d'interrogation, peuvent jouer un rôle important pour capturer un sentiment d'étonnement.

Les signes de ponctuation considérés lorsque nous utilisons la classe `string` du langage Python sont les suivants :

```
!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~
```

Nous allons utiliser ces signes de ponctuation dans un exemple concret à la fin de ce chapitre.

2.1.2 Appliquer le Stemming sur un texte

La technique de Stemming permet de réduire la taille d'un document en contractant les mots qu'il contient.

Par exemple, avec la méthode de Stemming, le mot `racine` devient `racin`. De même, le mot `racines` sera contracté en `racin`, exactement comme racine au singulier.

La méthode de Stemming permet de réduire les mots à leur racine ! Ainsi, le mot `racin` est considéré comme la racine des deux mots `racine` et `racines`.

Cette technique permet évidemment de réduire la taille des documents. Cependant, les mots engendrés avec cette méthode peuvent ne pas faire partie des mots du dictionnaire ! Cela n'est pas vraiment un problème si le sens des mots n'est pas important.

2.1.3 Appliquer la Lemmatization sur un texte

La Lemmatization, tout comme le Stemming, permet elle aussi de réduire la taille des documents en contractant les mots dans leur forme racine. La différence majeure entre la Lemmatization et le Stemming est qu'avec la Lemmatization les mots engendrés font toujours partie des mots du dictionnaire. En effet, la Lemmatization tente de contracter un mot vers sa racine à condition que le mot résultant soit un mot à part entière et qui a un sens, contrairement au Stemming.

2.1.4 Stemming versus Lemmatization

Les deux sections précédentes ont présenté les méthodes de Stemming et de Lemmatization, qui ont toutes deux un même objectif : réduire la taille des documents traités.

Alors quelle est celle à choisir parmi ces deux méthodes lorsque nous manipulons des données textuelles ?

En réalité, le choix d'une méthode plutôt qu'une autre dépend de l'objectif du projet en cours de réalisation et, parfois, cela vaut la peine de tester les deux approches et comparer leurs résultats.

© Editions ENI - All rights reserved

Ces méthodes présentent les deux différences majeures suivantes :

- La méthode de Stemming, contrairement à la méthode de Lemmatization, peut engendrer des mots en dehors du dictionnaire de la langue utilisée. Ce point est évidemment en la faveur de la Lemmatization.
- La méthode de Stemming est beaucoup plus rapide à l'exécution alors que la méthode de Lemmatization peut demander des temps de calcul importants en fonction de la taille des documents traités.

Un autre avantage de la méthode de Lemmatization par rapport à la méthode de Stemming, qui n'est pas des moindres, est qu'avec cette dernière on peut perdre une partie du sens des documents traités. Par exemple, si nous nous positionnons dans la langue anglaise, alors la méthode de Stemming contractera les mots `meanness` et `meaning` vers le mot `mean`. Sachant que le mot `meanness` veut dire « méchanceté » et le mot `meaning` veut dire « sens » et le mot `mean` veut dire « moyenne », nous en concluons que la méthode de Stemming fait perdre complètement le sens des phrases utilisant ces deux mots `meanness` et `meaning` !

A contrario, la méthode de Lemmatization va contracter le mot `meanness` vers le mot `meanness` et le mot `meaning` vers le mot `meaning` ! Oui, cette méthode ne va pas contracter ces deux mots, elle va les garder dans leur forme d'origine. Dans ce dernier cas, la méthode de Lemmatization ne réduit pas la taille des mots `meanness` et `meaning`, mais elle ne va pas nous faire perdre le sens de nos documents !

2.2 Vectorisation des données textuelles

Cette section expose quelques méthodes qui permettent de transformer des données textuelles vers des données numériques. Pour rappel, une fois que les données textuelles sont transformées en données numériques, nous pouvons leur appliquer les algorithmes du Machine Learning tels que Random Forest, SVM ou les réseaux de neurones afin de construire des modèles de classification ou tout autre type de modèle.

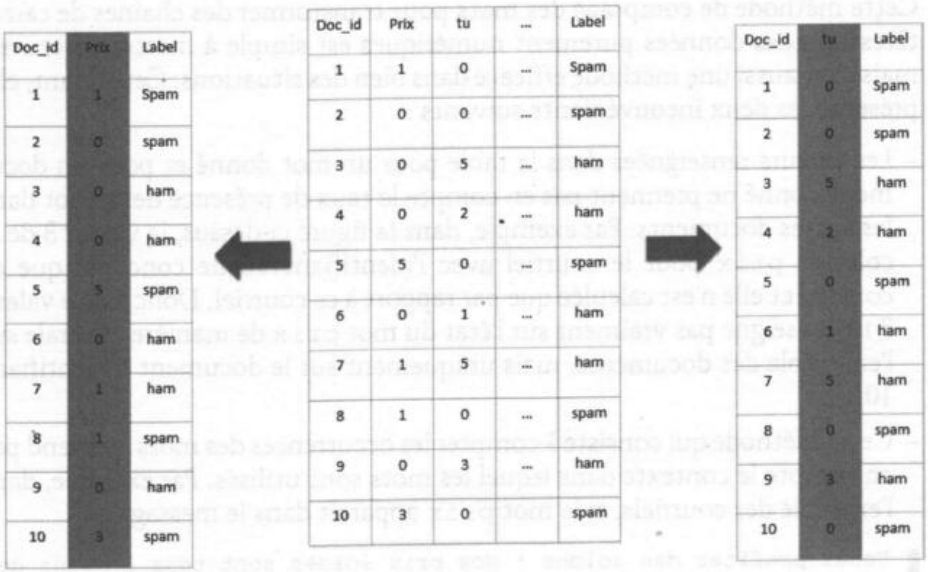
2.2.1 La vectorisation par comptage d'occurrences des mots

Une des méthodes de transformation des données textuelles en données numériques les plus simples, mais aussi les plus efficaces, est la méthode qui consiste à compter le nombre d'occurrences de chaque mot dans la base de données.

Supposons que nous avons une base de données composée de plusieurs courriels. Chaque courriel est identifié par l'identifiant `Doc_id`, le texte qui compose le courriel, et la catégorie du courriel associée à un champ `Label` qui peut être égal soit à la valeur `spam` pour indiquer qu'un courriel est un courriel indésirable ou égal à `ham` pour indiquer qu'un courriel est normal.

Pour vectoriser cette base de données avec la méthode de comptage des occurrences des mots et la transformer ainsi en une base de données dont les données seront des nombres et non pas des chaînes de caractères, nous pouvons construire une table dans laquelle chacun des mots présents dans les textes associés à l'ensemble des courriels sera une colonne. Puis, dans cette table, pour un mot donné, c'est-à-dire pour une colonne donnée, nous allons renseigner le nombre d'occurrences de ce mot dans chaque courriel. Par exemple, la figure suivante montre que le mot `prix` est présent avec zéro occurrence dans le courriel avec l'identifiant 3 et avec cinq occurrences dans le courriel avec l'identifiant 5. De son côté, le mot `tu` est présent avec zéro occurrence dans le courriel avec l'identifiant 1 et avec cinq occurrences dans le courriel avec l'identifiant 7.

© Editions ENI - All rights reserved



Doc_id	Prix	Label
1	1	Spam
2	0	spam
3	0	ham
4	0	ham
5	5	spam
6	0	ham
7	1	ham
8	1	spam
9	0	ham
10	3	spam

Doc_id	Prix	tu	...	Label
1	1	0	...	Spam
2	0	0	...	spam
3	0	5	...	ham
4	0	2	...	ham
5	5	0	...	spam
6	0	1	...	ham
7	1	5	...	ham
8	1	0	...	spam
9	0	3	...	ham
10	3	0		spam

Doc_id	tu	Label
1	0	Spam
2	0	spam
3	5	ham
4	2	ham
5	0	spam
6	1	ham
7	5	ham
8	0	spam
9	3	ham
10	0	spam

Figure 15-01 : exemple de vectorisation par comptage des mots

Cette représentation permet de construire une base de données textuelle avec des nombres. Dans l'exemple de la figure ci-dessus, vous remarquez que le mot `prix` a tendance à avoir plus d'occurrences dans les courriels de type `spam` que dans les courriels normaux de type `ham` alors que le mot `tu` a tendance à avoir plus d'occurrences dans les courriels normaux que dans les courriels de type spam. Ces informations sont évidemment intéressantes puisqu'elles peuvent servir pour discriminer les courriels de type spam !

Cette méthode de comptage des mots pour transformer des chaînes de caractères vers des données purement numériques est simple à mettre en œuvre, mais c'est aussi une méthode efficace dans bien des situations. Cependant, elle présente les deux inconvénients suivants :

- Les valeurs renseignées dans la table pour un mot donné et pour un document donné ne prennent pas en compte le taux de présence de ce mot dans les autres documents. Par exemple, dans la figure ci-dessus, la valeur 3 de la colonne `prix` pour le courriel avec l'identifiant 10 ne concerne que ce courriel et elle n'est calculée que par rapport à ce courriel. Donc, cette valeur 3 ne renseigne pas vraiment sur l'état du mot `prix` de manière générale sur l'ensemble des documents, mais uniquement sur le document d'identifiant 10.
- Cette méthode qui consiste à compter les occurrences des mots ne prend pas en compte le contexte dans lequel les mots sont utilisés. Par exemple, dans l'exemple des courriels, si le mot `prix` apparaît dans le message :

```
Venez profiter des soldes ! Nos prix soldés sont tous à moins de
50%
```

la méthode de comptage des mots va associer la valeur 1 au mot `prix` pour ce courriel alors qu'elle va associer la valeur 2 pour ce même mot dans le courriel suivant :

```
Coucou Madjid, le prix dont tu m'as parlé hier ne correspond pas
au prix que j'ai trouvé sur leur site !
```

Le problème est que le premier courriel a tout d'un spam et le deuxième courriel a tout d'un message provenant d'une connaissance !

2.2.2 La vectorisation avec TF-IDF

Dans la section précédente, nous avons vu la méthode de vectorisation qui consiste à compter les occurrences des mots dans un document et nous avons vu que cette méthode, bien qu'efficace, présente l'inconvénient de ne pas prendre en compte ni l'ensemble des documents lors du comptage des occurrences d'un mot dans un document précis ni le contexte dans lequel un mot a été utilisé.

© Editions ENI - All rights reserved

Pour un document donné, la méthode de vectorisation TF-IDF (*Term Frequency-Inverse Document Frequency*) donne une note à un terme donné en fonction du nombre d'occurrences de ce mot dans ce document, mais le calcul de cette note prend aussi en compte l'appartenance globale de ce terme aux autres documents.

Ainsi, la note $W_{i,j}$ correspondant à un mot i et un document j se calcule avec la formule suivante $W_{i,j} = tf_{i,j} * \log(\frac{N}{df_i})$

Avec :

$tf_{i,j}$ est le taux d'apparition du mot i dans le document j.

N est le nombre de documents total dans la base de données.

df_i est le nombre de documents contenant le mot i.

Le résultat de la méthode TF-IDF peut être schématisé comme dans la figure suivante :

Doc_id	mot1	mot2	...	Label
1	0.002	0.0000	...	Spam
2	0.0000	0.2500	...	spam
3	0.0000	0.0010	...	ham
4	0.0000	0.0000	...	ham
5	0.2300	0.0000	...	spam
6	0.0000	0.2300	...	ham
7	0.2249	0.0000	...	ham

Figure 15-02 : vectorisation avec TF-IDF

Dans cette figure, la valeur 0.23 attribuée au mot `mot1` du document avec l'identifiant 5 signifie que ce mot apparaît de façon beaucoup plus importante dans ce document que dans tous les autres documents. C'est une différence importante avec la méthode de vectorisation précédente, puisque cette fois la note attribuée à un mot pour un document donné tient compte de l'apparition de ce mot dans les autres documents.

Avec la méthode TF-IDF, plus les valeurs sont importantes, plus elles sont discriminantes par rapport au document en question. En effet, si la note i est élevée, cela veut dire que le mot i apparaît de façon plus importante dans le document j que dans tous les autres documents. D'ailleurs, dans le domaine du NLP, la méthode TF-IDF est très couramment utilisée dans le développement des modèles de classification de documents, puisque les notes attribuées avec cette méthode permettent de donner l'importance que peut avoir un mot pour une catégorie de documents par rapport aux autres catégories.

Remarque

Dans la suite de ce chapitre, nous aurons l'occasion d'utiliser cette méthode dans un exemple concret.

2.2.3 La vectorisation avec N-Gram

Nous avons vu précédemment que la méthode de comptage des occurrences d'un mot dans un document présente l'inconvénient de ne pas prendre en compte ni les apparitions de ce mot dans les autres documents ni le contexte dans lequel ce mot apparaît dans un document. Nous avons également abordé la méthode TF-IDF qui, elle, permet de noter un mot pour un document donné en prenant en compte l'apparition de ce mot dans l'ensemble des documents. La méthode N-Gram, quant à elle, permet de compter un regroupement de mots et non pas les mots un par un séparément.

Ainsi, avec la méthode N-Gram, chaque colonne de la table qui sera construite va correspondre à plusieurs mots en fonction du degré souhaité.

Si nous utilisons N-Gram avec des constructions de deux mots, alors chaque colonne de la table construite correspond à deux mots retrouvés l'un à la suite de l'autre dans une phrase.

© Editions ENI - All rights reserved

Concrètement, si nous considérons N-Gram avec des assemblages de deux mots, alors à partir de la phrase suivante :

```
Venez profitez des soldes ! Nos prix soldés sont tous à moins de
50%
```

Nous allons avoir une colonne avec l'assemblage des deux mots `prix` et `soldés` qui contiendra le nombre d'occurrences du mot `prix` suivi par le mot `soldés`. Ainsi, nous allons capturer le contexte dans lequel le mot `prix` a été utilisé et nous allons donc distinguer les messages où le mot `prix` est utilisé avec le mot `soldés` des messages où le mot `prix` est utilisé en combinaison avec d'autres mots ! C'est en ce sens que la méthode N-Gram prend en compte le contexte dans lequel les mots sont rencontrés et c'est l'avantage principal de cette méthode par rapport aux autres méthodes vues précédemment. Elle est cependant beaucoup plus gourmande en matière de temps de calcul.

2.2.4 Feature Engineering sur des documents

Une fois que les données textuelles sont transformées en données numériques, nous pouvons leur appliquer les algorithmes classiques du Machine Learning. Mais avant d'utiliser ces algorithmes pour développer des modèles, il serait intéressant de procéder à la fameuse étape du Feature Engineering. En effet, nous pouvons opérer deux types de traitements sur les données afin de les rendre plus riches en matière d'informations : nous pouvons soit calculer de nouvelles variables à partir de celles déjà existantes et/ou nous pouvons transformer les variables déjà existantes afin, par exemple, d'amplifier les différences entre certaines valeurs.

Ainsi, sur des données textuelles comme des courriels, nous pouvons opérer les traitements suivants :

- Ajouter de nouvelles variables comme les variables ci-dessous :
  - Taux de caractères de ponctuation. En effet, cette information peut être exploitée dans le cadre de la détection des spams, puisque nous savons que les spams sont souvent écrits dans des buts commerciaux et donc rédigés avec un style plus officiel que les courriels ordinaires.
  - Taille des messages. Également, la taille des courriels peut nous être utile pour discriminer les spams ! En effet, les spams sont souvent plus longs que les e-mails ordinaires.

- Taux de lettres en majuscules. Tout comme les signes de ponctuation, les lettres majuscules sont évidemment plus utilisées dans les courriels officiels et professionnels que dans les courriels ordinaires, donc cela peut également servir pour détecter les spams.
- Présence ou non de certains mots tels que « lol ». En effet, ce type de mots serait plus présent dans des courriels ordinaires que dans les courriels de publicité.

- Transformer les variables existantes. La transformation des variables existantes peut s'avérer utile si les valeurs des variables d'origine cachent des informations non exploitables en l'état. Par exemple, si nous transformons une variable en calculant une nouvelle variable qui serait égale au carré de la variable d'origine, alors les différences entre les valeurs de la variable d'origine seront évidemment amplifiées. Par exemple, si une variable présente les deux valeurs 3 et 9, alors la nouvelle variable qui serait égale au carré de cette variable d'origine aura les valeurs 9 et 81 à la place des valeurs initiales 3 et 9 ! De toute évidence, la différence entre 9 et 81 est beaucoup plus importante que la différence entre 3 et 9.

3. Exemple complet pour la détection des spams

Dans ce dernier exemple du présent ouvrage, nous allons développer un modèle de classification de courriels afin de reconnaître ceux qui sont des courriels indésirables ou des spams, ou encore des pourriels, pour ainsi les distinguer des courriels ordinaires.

Pour la réalisation de notre exemple, nous allons utiliser une base de données composée de 5 572 courriels stockés dans le fichier *SMSSpamCollection.txt* du dossier *Data* de ce chapitre.

Les références de cette base de données sont les suivantes :

- [1] Almeida, T.A., Gómez Hidalgo, J.M., Yamakami, A. Contributions to the Study of SMS Spam Filtering: New Collection and Results. Proceedings of the 2011 ACM Symposium on Document Engineering (DOCENG'11), Mountain View, CA, USA, 2011.

© Editions ENI - All rights reserved

- [2] Gómez Hidalgo, J.M., Almeida, T.A., Yamakami, A. On the Validity of a New SMS Spam Collection. Proceedings of the 11th IEEE International Conference on Machine Learning and Applications (ICMLA'12), Boca Raton, FL, USA, 2012.
- [3] Almeida, T.A., Gómez Hidalgo, J.M., Silva, T.P. Towards SMS Spam Filtering : Results under a New Dataset. International Journal of Information Security Science (IJISS), 2(1), 1-18, 2013.

Le fichier *SMSSpamCollection.txt* est une simple table avec deux colonnes. La première colonne renseigne la catégorie d'un courriel parmi les deux catégories `spam` pour les pourriels et `ham` pour les courriels normaux. La deuxième colonne concerne le contenu des courriels. En résumé, chaque ligne de ces données correspond au texte d'un courriel et sa catégorie `spam` ou `ham`.

La version du fichier *SMSSpamCollection.txt* utilisée dans cet exemple est une version légèrement modifiée de la base de données que vous pouvez retrouver sur le site Kaggle avec le lien suivant :
https://www.kaggle.com/uciml/sms-spam-collection-dataset

L'objectif de cet exemple est de montrer comment appliquer les algorithmes du Machine Learning sur des données textuelles, comme celles de notre fichier et *SMSSpamCollection.txt*. Ainsi, nous allons développer un modèle qui sera capable d'analyser le contenu d'un courriel, c'est-à-dire le texte associé à un courriel, et décider si ce courriel est un spam ou pas.

3.1 Installation de la NLTK

Avant de développer notre exemple, nous allons commencer par l'installation de la bibliothèque NLTK (*Natural Language Toolkit*). Cette bibliothèque, qui est l'une des bibliothèques les plus utilisées dans le domaine du NLP, propose des fonctionnalités qui permettent de simplifier l'analyse et le traitement des données textuelles.

- Choisissez dans Anaconda Navigator un environnement virtuel, puis, pour y installer NLTK, suivez les mêmes étapes que lorsque nous avons installé les bibliothèques telles que NumPy, Pandas et TensorFlow dans le chapitre précédent (voir la section Installer des bibliothèques dans un environnement virtuel avec Anaconda du chapitre Le Deep Learning avec TensorFlow).

3.2 Modèle de détection de spams

- Créez un nouveau notebook Jupyter et suivez les étapes de code ci-après afin de réaliser cet exemple ; vous pouvez utiliser directement le notebook *SVM_TF-IDF_final.ipynb* du dossier *Code* de ce chapitre.

Le code suivant permet tout simplement d'importer les bibliothèques nécessaires à notre exemple :

```
import nltk
import pandas as pd
import re
from sklearn.feature_extraction.text import TfidfVectorizer
import string
from sklearn.metrics import precision_recall_fscore_support as
score
from sklearn.model_selection import train_test_split
from sklearn  import svm
```

Nous avons rencontré la plupart de ces modules importés. Les modules `nltk` et `sklearn.feature_extraction.text` sont spécifiques pour la réalisation de tâches dans le domaine du NLP.

Le code suivant lit les données du fichier SMSSpamCollection.txt dans le DataFrame `data` et nomme les deux colonnes de cette base de données avec `label` pour la classe d'un courriel et `Content` pour le texte qui lui est associé :

```
data = pd.read_csv("../Data/SMSSpamCollection.txt", sep='\t',
header=None)
data.columns = ['label', 'Content']
print(data)
```

© Editions ENI - All rights reserved

La figure suivante montre les cinq premières et les cinq dernières lignes du DataFrame `data` :

```
     label                                            Content
0      ham  Go until jurong point, crazy.. Available only ...
1      ham                      Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
...    ...                                              ...
5567  spam  This is the 2nd time we have tried 2 contact u...
5568   ham              Will ü b going to esplanade fr home?
5569   ham  Pity, * was in mood for that. So...any other s...
5570   ham  The guy did some bitching but I acted like i'd...
5571   ham                        Rofl. Its true to its name

[5572 rows x 2 columns]
```

Figure 15-03 : les données initiales de SMSSpamCollection.txt

Le code suivant permet d'ajouter la nouvelle colonne `Content_len` au DataFrame `data` en utilisant la méthode `apply` que nous avons abordée au chapitre La bibliothèque Pandas. Cette nouvelle colonne enregistre la taille des courriels. En effet, la taille d'un courriel peut nous servir pour la détection des spams.

```
data['Content_len'] = data['Content'].apply(lambda x: len(x) -
x.count(" "))

print(data)
```

La figure suivante montre le résultat après l'ajout de la nouvelle colonne `Content_len` :

```
     label                                            Content  Content_len
0      ham  Go until jurong point, crazy.. Available only ...           92
1      ham                      Ok lar... Joking wif u oni...           24
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...          128
3      ham  U dun say so early hor... U c already then say...           39
4      ham  Nah I don't think he goes to usf, he lives aro...           49
...    ...                                              ...          ...
5567  spam  This is the 2nd time we have tried 2 contact u...          131
5568   ham              Will ü b going to esplanade fr home?           29
5569   ham  Pity, * was in mood for that. So...any other s...           48
5570   ham  The guy did some bitching but I acted like i'd...          100
5571   ham                        Rofl. Its true to its name           21

[5572 rows x 3 columns]
```

Figure 15-04 : ajout de la nouvelle colonne `Content_len`

Le code suivant définit la fonction `count_punctuation`. Celle-ci calcule le taux de caractères de ponctuation présents dans un courriel reçu en argument. L'attribut `string.punctuation` correspond à la liste des caractères de ponctuation. La liste `binary_array` est définie en utilisant le concept de liste en compréhension présenté au chapitre Le langage Python lorsque nous avons étudié les bases de ce langage.

Donc la liste `binary_array` contiendra autant de valeurs 1 qu'il y a de caractères de ponctuation dans la chaîne de caractères `text`. Donc, pour calculer le taux de caractères de ponctuation dans le texte associé à un courriel, il suffit de diviser la somme de tous les éléments de la liste `binary_array` sur le nombre de caractères, hormis l'espace blanc, présents dans la chaîne de caractères `text`.

```
def count_punctuation(text):
    binary_array = [1 for ch in text if ch in string.punctuation]
    nb_ponctuation = sum(binary_array)
    total = len(text) - text.count(" ")
    return round(nb_ponctuation/(total), 4)*100
```

Le code suivant permet d'ajouter la nouvelle colonne `punctuation_rate` au DataFrame `data` en appliquant la fonction `count_punctuation` sur ce DataFrame via la méthode `apply`.

```
data['punctuation_rate'] = data['Content'].apply(lambda x:
count_punctuation(x))

print(data)
```

© Editions ENI - All rights reserved

Après avoir ajouté la nouvelle colonne `punctuation_rate`, le DataFrame `data` devient comme sur la figure suivante :

```
     label                                            Content  Content_len  \
0      ham  Go until jurong point, crazy.. Available only ...          92
1      ham                      Ok lar... Joking wif u oni...          24
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...         128
3      ham  U dun say so early hor... U c already then say...          39
4      ham  Nah I don't think he goes to usf, he lives aro...          49
...    ...                                                ...         ...
5567  spam  This is the 2nd time we have tried 2 contact u...         131
5568   ham              Will ü b going to esplanade fr home?          29
5569   ham  Pity, * was in mood for that. So...any other s...          48
5570   ham  The guy did some bitching but I acted like i'd...         100
5571   ham                        Rofl. Its true to its name          21

      punctuation_rate
0                 9.78
1                25.00
2                 4.69
3                15.38
4                 4.08
...                ...
5567              6.11
5568              3.45
5569             14.58
5570              1.00
5571              4.76

[5572 rows x 4 columns]
```

Figure 15-05 : ajout de la nouvelle colonne `punctuation_rate`

Le code suivant définit la fonction `clean_email`. Le rôle de cette fonction est de créer une liste de tokens pour chaque courriel. Ainsi, pour un courriel donné, nous aurions tous les mots qui le composent, hormis les signes de ponctuation et les espaces blancs, stockés dans une liste. Également, cette fonction applique la technique de Stemming sur tous les mots de la liste créée précédemment.

En d'autres termes, cette fonction permet de fragmenter une chaîne de caractères afin de stocker les mots qui la composent dans une liste, puis cette fonction applique le principe de Stemming sur les mots de cette liste qui ne font pas partie des stopwords de la langue anglaise.

```
en_stopwords = nltk.corpus.stopwords.words('english')
ps = nltk.PorterStemmer()

def clean_email(email):
    result = "".join([word for word in email if word not in
                                                string.punctuation])
    tokens = re.split('\W+', result)
    text = [ps.stem(word) for word in tokens if word not in
                                                    en_stopwords]
    return text
```

La fonction `clean_email` définie précédemment est utilisée pour la vectorisation de chacun des courriels du DataFrame `data` avec le code suivant :

```
vectorisation_full = TfidfVectorizer(analyzer=clean_email)
vect_final = vectorisation_full.fit_transform(data['Content'])
```

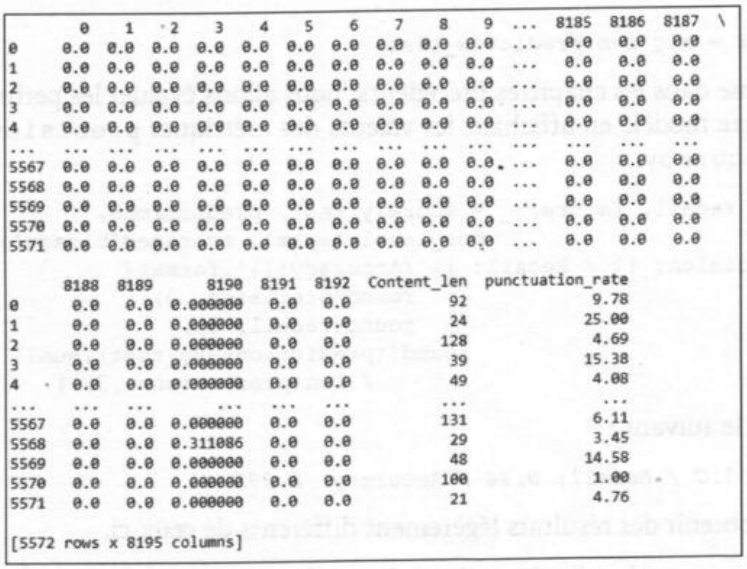
Ce dernier code utilise un objet de type `TfidfVectorizer` afin de réaliser la vectorisation avec la méthode TF-IDF abordée plus haut dans ce chapitre. Pour cet objet `vectorisation_full`, nous avons indiqué la fonction `clean_email` qui sera appliquée sur chaque valeur de la colonne `Content`. La vectorisation est réalisée grâce à la méthode `fit_transform` qui est appliquée sur la colonne `Content`.

Le code suivant permet de construire le nouveau DataFrame `all_data` en combinant le résultat de la vectorisation précédente qui est stocké dans `vect_final` avec les deux nouvelles colonnes `Content_len` et `punctuation_rate` que nous avons créées précédemment.

```
all_data = pd.concat([pd.DataFrame(vect_final.toarray()),
data['Content_len'], data['punctuation_rate']], axis=1)
print(all_data)
```

© Editions ENI - All rights reserved

La figure suivante affiche le contenu du nouveau DataFrame `all_data` :



	0	1	2	3	4	5	6	7	8	9	...	8185	8186	8187	\
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
5567	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
5568	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
5569	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
5570	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	
5571	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	

	8188	8189	8190	8191	8192	Content_len	punctuation_rate
0	0.0	0.0	0.000000	0.0	0.0	92	9.78
1	0.0	0.0	0.000000	0.0	0.0	24	25.00
2	0.0	0.0	0.000000	0.0	0.0	128	4.69
3	0.0	0.0	0.000000	0.0	0.0	39	15.38
4	0.0	0.0	0.000000	0.0	0.0	49	4.08
...	...	...	...	...	...	...	...
5567	0.0	0.0	0.000000	0.0	0.0	131	6.11
5568	0.0	0.0	0.311086	0.0	0.0	29	3.45
5569	0.0	0.0	0.000000	0.0	0.0	48	14.58
5570	0.0	0.0	0.000000	0.0	0.0	100	1.00
5571	0.0	0.0	0.000000	0.0	0.0	21	4.76

[5572 rows x 8195 columns]

Figure 15-06 : résultat du Feature Engineering et de la vectorisation

À ce stade, nous avons transformé les données textuelles du fichier *SMSSpamCollection.txt* en données numériques stockées dans le DataFrame `all_data` et les classes associées à chaque ligne de ce DataFrame sont stockées dans la colonne `label` du DataFrame `data`. Nous pouvons donc suivre les mêmes étapes que dans les chapitres précédents afin d'entraîner un modèle.

Le code suivant permet de créer deux jeux de données : un jeu de données d'entraînement (`x_train, y_train`) et un jeu de données de test (`x_test, y_test`). Dans cet exemple, nous avons mis de côté 20 % des courriels pour la phase d'évaluation.

```
x_train, x_test, y_train, y_test = train_test_split(all_data,
data['label'], test_size=0.2)
```

Puis le code suivant permet de créer une instance de l'algorithme SVM et de l'utiliser pour construire notre modèle de classification :

```
alg_svm= svm.SVC(kernel = 'linear')
alg_svm.fit(x_train, y_train)
```

L'instruction suivante permet de tester le modèle `alg_svm` sur les données `x_test` :

```
predictions = alg_svm.predict(x_test)
```

Et enfin, comme dans les chapitres précédents, nous allons évaluer les performances de notre modèle en affichant les valeurs des métriques `precision`, `recall` et `accuracy`.

```
precision, recall, fscore, _ = score(y_test, predictions,
                                     pos_label='spam', average='binary')
print('Precision: {} / Recall: {} /Accuracy:{}'.format(
                                        round(precision, 3),
                                        round(recall, 3),
                                    round((predictions==y_test).sum()
                                           / len(predictions),3)))
```

Le résultat est le suivant :

```
Precision: 1.0 / Recall: 0.94 / Accuracy: 0.992
```

Vous pouvez obtenir des résultats légèrement différents de ceux-ci.

On voit que sans trop chercher à améliorer les performances de notre modèle, le taux de réponses correctes est de l'ordre de 99.2 % !

De plus, ces résultats montrent que tous les courriels prédits en tant que spams sont vraiment des spams, puisque la métrique `precision` est égale à 100 % et que 94 % des spams présents dans le jeu de données de test ont été capturés par notre modèle !

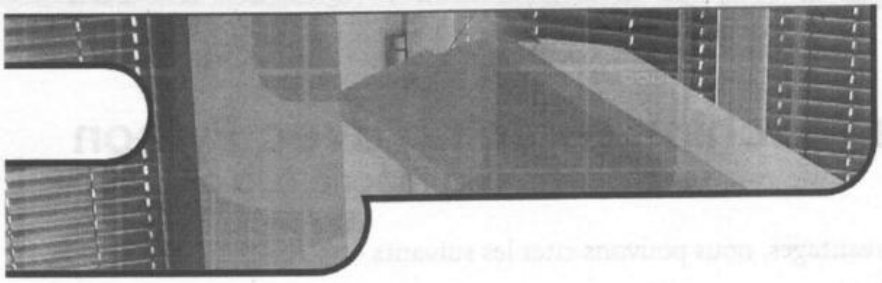
Bien sûr, comme pour tous les exemples de ce livre, le but de ce dernier exemple n'est pas du tout de trouver le modèle le plus efficace, mais d'illustrer comment appliquer les algorithmes du Machine Learning sur des données textuelles.

Pour entraîner ce dernier modèle, nous avons fait le choix arbitraire d'utiliser l'algorithme SVM qui a par ailleurs donné d'excellents résultats ; vous pouvez cependant utiliser n'importe quel autre algorithme pour entraîner un modèle sur les données `x_train` et `y_train`, à partir du moment où cet algorithme est applicable.

© Editions ENI - All rights reserved

4. Conclusion

Ce chapitre, qui conclut ce livre, nous a permis d'aborder le domaine du traitement automatique du langage. Nous avons utilisé un exemple concret pour illustrer les opérations de manipulation des données textuelles afin de les transformer en données purement numériques pour ainsi leur appliquer des algorithmes du Machine Learning.



Annexe
La programmation orientée objet avec Python

1. Programmation orientée objet avec Python

L'application des notions orientées objet lorsque nous programmons avec Python n'est pas obligatoire, surtout lorsqu'on débute avec ce langage. Cependant, souvent, ceux qui essayent la programmation orientée objet ont du mal à s'en passer après ! Cette section propose d'aborder les principes fondamentaux de la programmation orientée objet avec Python.

1.1 Pourquoi la programmation orientée objet?

La programmation orientée objet est avant tout une manière de concevoir les programmes informatiques. La principale différence entre la programmation orientée objet et la programmation procédurale réside essentiellement dans l'organisation du code source. Cette différence organisationnelle procure plusieurs avantages à la programmation orientée objet.

Parmi ces avantages, nous pouvons citer les suivants :

- L'extensibilité : une application développée avec les concepts objet peut être étendue plus facilement. En effet, l'ajout de nouvelles fonctionnalités à une application est généralement basé sur des objets déjà existants pour lesquels il suffirait d'ajouter de nouvelles compétences. Avec le concept de l'héritage, qui est inhérent à la programmation orientée objet, nous pouvons facilement créer de nouveaux types d'objets en nous basant sur les classes déjà existantes.
- La réutilisabilité : une fois qu'une classe est définie, elle peut être adaptée pour créer une tout autre nouvelle classe, ce qui permet dans certains cas un gain de temps important.
- L'abstraction : la conception orientée objet permet un niveau d'abstraction élevé tout en restant le plus fidèle possible à la réalité. Il est d'usage de concevoir des classes abstraites et des interfaces afin de définir non pas des objets concrets dotés d'un comportement définitif, mais plutôt afin de définir le comportement que devrait avoir une certaine catégorie d'objets.
- Facilité de la maintenance : un programme orienté objet est plus facile à maintenir puisqu'il est plus facile à comprendre. Chaque classe a son périmètre de responsabilité bien défini et chaque objet encapsule ses propres données. Ces concepts d'isolation du périmètre de responsabilité et d'encapsulation de données permettent une lecture plus facile du code source. Bien sûr, un code source plus facile à lire est un code source plus facile à maintenir !
- Plus simple à concevoir : il est certainement plus intuitif de réaliser un projet informatique de grande taille en s'appuyant sur plusieurs équipes dont chacune prend en charge le développement d'une partie bien distincte du projet. Par exemple, une première équipe va se charger de l'architecture applicative, une deuxième équipe prendra en charge la partie frontend, au sens IHM du projet, une troisième équipe va s'occuper de la partie backend du projet, une quatrième équipe assure la conception et l'implémentation de la base de données, etc. De la même manière, la conception d'un programme orienté objet est grandement simplifiée par la décomposition de l'application en un ensemble de types d'objets dont chacun prend en charge une partie fonctionnelle du projet.

© Editions ENI - All rights reserved

1.2 Classes et objets

Une classe permet de regrouper un ensemble de variables et un ensemble de fonctions dans un même type. Dans le paradigme orienté objet, les variables manipulées dans une classe sont appelées des attributs et les fonctions définies dans une classe sont appelées des méthodes.

Le fait de regrouper un ensemble de variables ou d'attributs dans une même classe permet d'avoir une vision plus au moins centralisée sur ces variables et faciliter leur gestion.

Dans une classe donnée, on regroupe un ensemble d'attributs cohérents du point de vue de l'application en cours de développement et les méthodes définies dans cette classe représentent les traitements que nous souhaitons appliquer sur ces attributs.

Une fois qu'une classe est définie avec le mot-clé `class`, le nom associé à cette nouvelle classe représente un type de données au même titre que les types de données définies nativement dans le langage Python tels que les types `list` et `int`.

Le nom d'une classe peut être utilisé pour la création de nouveaux objets. Chacun des objets d'une classe peut avoir ses propres attributs et ses propres méthodes, mais également les objets d'une même classe peuvent partager un certain nombre d'attributs et de méthodes, comme nous allons le voir tout au long de cette section via des exemples.

1.2.1 Définir une classe

Hormis les classes déjà prédéfinies dans le langage Python, nous pouvons définir une nouvelle classe comme dans l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *class_.py* situé dans le dossier *Code* de cette annexe.

```
class Book:
    pass

print(Book)
```

Ce code définit une nouvelle classe nommée `Book`. Pour définir une nouvelle classe, on utilise toujours le mot-clé `class` suivi du nom de cette nouvelle classe. Les deux points après le nom de la nouvelle classe signalent le bloc de code associé à la définition du contenu de la nouvelle classe.

Dans cet exemple, le contenu de la classe `Book` est vide. Le mot-clé `pass` sert à signaler à l'interpréteur Python qu'un bloc de code est vide, c'est-à-dire qu'il ne contient pas d'instructions.

L'instruction `print` à la fin de ce programme affiche le nom du nouveau type qui vient d'être créé comme suit :

```
<class '__main__.Book'>
```

Pour l'instant, le nouveau type `Book` ne définit ni d'attributs ni de méthodes. Dans la suite de cette annexe, nous allons voir comment enrichir la classe `Book` avec de nouveaux attributs et de nouvelles méthodes.

1.2.2 La fonction __init__

Une classe peut être utilisée de deux manières différentes : soit elle est utilisée directement en tant que classe afin d'accéder aux attributs et aux méthodes de classes, soit elle est utilisée pour la création de nouveaux objets.

Les attributs et les méthodes de classes seront traités plus tard dans cette section. Voyons comment définir quelques attributs d'objets et comment créer un objet.

Le code suivant permet d'enrichir la classe `Book` avec quelques attributs d'objets.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *méthode__ini__.py* situé dans le dossier *Code* de cette annexe.

```
class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language
```

© Editions ENI - All rights reserved

```
print(Book)
```

Ce programme introduit deux concepts importants : la méthode `__init__` et le mot-clé `self`.

Dans le bloc de code de la classe `Book`, nous avons défini une fonction avec le mot-clé `def` comme nous l'avons fait précédemment dans le chapitre Le langage Python lorsque nous avons étudié les fonctions. Cette fonction `__init__` est une méthode particulière de la classe `Book`. En effet, comme nous allons le voir à la section suivante, cette méthode sera exécutée de manière automatique à chaque fois qu'un objet de type `Book` sera créé.

Nous avons défini la méthode `__init__` avec les six paramètres `self`, `title`, `author`, `year`, `nbPages` et `language`.

Pour un livre donné, les paramètres `title`, `author`, `year`, `nbPages` et `language` seront associés respectivement au titre, au nom de l'auteur, à l'année de publication, au nombre de pages qui le composent et à la langue utilisée pour sa rédaction. Dans la méthode `__init__`, nous pouvons mentionner autant de paramètres que nécessaire et qui définissent un livre.

En revanche, le paramètre `self` n'est pas associé à la définition d'un attribut quelconque. En réalité, ce paramètre `self` sera utilisé afin de référencer un livre en particulier. En d'autres termes, le paramètre `self` sera utilisé pour pointer vers un objet donné de type `Book`. En effet, lorsque nous allons créer un objet de type `Book`, c'est-à-dire un livre donné, les instructions `self.title`, `self.author`, `self.year`, `self.nbPages` et `self.language` vont permettre d'associer les valeurs des arguments `title`, `author`, `year`, `nbPages` et `language` à ce livre. L'exemple de la section suivante explique plus en détail cette association entre les arguments et un objet.

L'instruction `print(Book)` de ce programme affiche exactement le même résultat que le programme précédent.

1.2.3 Instanciation d'un objet

Pour comprendre les notions d'objets et d'attributs, nous allons reprendre le code de l'exemple précédent auquel nous allons ajouter des instructions afin de créer de nouveaux objets de type `Book`.

▶ Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Instancier_un_objet.py* situé dans le dossier *Code* de cette annexe.

```
class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language


book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
2018, 346, "Français")
book2 = Book("The Old Man and the Sea", "Ernest Hemingway", 1952,
109, "Anglais")

print("book1:", book1)
print("book2:", book2)
```

Ce programme définit la même méthode `__init__` que celle de l'exemple précédent.

L'instruction `book1 = Book("Data Science avec Microsoft Azure", "M.Khichane", 2018, 346, "Français")` permet de créer un nouveau livre qui sera référencé avec l'objet `book1`. Cet objet définit donc un livre avec les champs suivants :

- `title` : "Data Science avec Microsoft Azure"
- `author` : "M.Khichane"
- `year` : 2018
- `nbPages` : 346
- `language` : "Français"

© Editions ENI - All rights reserved

Remarquez que lors de la création de l'objet book1, nous n'avons fait appel explicitement à aucune méthode. Cependant, la méthode __init__ sera exécutée automatiquement. C'est justement l'exécution de cette méthode qui va associer les arguments transmis entre parenthèses à la classe Book avec les attributs de l'objet book1.

Les deux instructions print à la fin de ce programme vont afficher les informations suivantes :

```
book1: <__main__.Book object at 0x00000242029B78E0>
book2: <__main__.Book object at 0x0000024202A4D190>
```

On voit que les deux objets book1 et book2 sont de type Book. Les deux valeurs en hexadécimales affichées à droite du type Book indiquent les adresses mémoire attribuées par l'interpréteur Python à nos deux objets book1 et book2.

Remarquez également que dans l'instruction book1 = Book("Data Science avec Microsoft Azure", "M.Khichane", 2018, 346, "Français") nous avons mentionné uniquement cinq arguments entre les deux parenthèses qui ont suivi le nom de la classe Book, alors que la méthode __init__ qui est exécutée automatiquement pour assurer la création de ce nouvel objet book1 attend six arguments ! Cela est tout à fait normal, puisque l'argument self est utilisé par l'interpréteur Python pour pointer vers l'objet book1 et il n'est donc pas considéré comme un attribut.

Remarque

Si vous n'avez pas encore compris les nuances entre l'argument self et les autres paramètres de la méthode __init__, les choses vont devenir plus claires au fur et à mesure des exemples des sections suivantes.

1.2.4 Les attributs d'un objet

Un attribut d'objet est une variable associée à un objet. Un objet peut avoir plusieurs attributs. L'ensemble des attributs d'un objet définit l'état de cet objet.

L'exemple de code suivant reprend le code précédent sauf que cette fois les fonctions print affichent les valeurs des attributs des deux objets book1 et book2.

Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Attributs_Objet.py* situé dans le dossier *Code* de cette annexe.

```
class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language


book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
2018, 346, "Français")

book2 = Book("The Old Man and the Sea", "Ernest Hemingway", 1952,
109, "Anglais")

print(f"Voici les informations du livre : {book1.title}")
print(f"Auteur : {book1.author}")
print(f"Année de sortie : {book1.year}")
print(f"Nombre de pages : {book1.nbPages}")
print(f"Langue : {book1.language}\n\n")

print(f"Voici les informations du livre : {book2.title}")
print(f"Auteur : {book2.author}")
print(f"Année de sortie : {book2.year}")
print(f"Nombre de pages : {book2.nbPages}")
print(f"Langue : {book2.language}")
```

Ce programme affiche le résultat suivant :

```
Voici les informations du livre : Data Science avec Microsoft
Azure
Auteur : M.Khichane
Année de sortie : 2018
Nombre de pages : 346
Langue : Français


Voici les informations du livre : The Old Man and the Sea
Auteur : Ernest Hemingway
Année de sortie : 1952
```

© Editions ENI - All rights reserved

```
Nombre de pages : 109
Langue : Anglais
```

Le point le plus important à remarquer dans ces résultats est que les attributs de l'objet `book1` ont des valeurs différentes des valeurs des attributs de l'objet `book2`.

Nous distinguons les attributs d'objets des attributs de classes par le fait que les attributs d'objets définissent l'état en mémoire des objets alors que les attributs de classes ne sont pas associés à des objets. Nous allons aborder les attributs de classes à travers des exemples dans la suite de cette annexe.

1.2.5 Les méthodes d'objet

Les méthodes d'un objet permettent de définir son comportement. En effet, ce sont les méthodes d'un objet qui permettent de réaliser des opérations sur les attributs de cet objet. Ces méthodes peuvent affecter l'état d'un objet en modifiant les valeurs de ses attributs.

Le code suivant est un exemple où nous avons défini la méthode d'objet get_infos.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Attributs_Objet.py* situé dans le dossier *Code* de cette annexe.

```
class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language

    def get_infos(self):
        print(f"Voici les informations du livre : {self.title}")
        print(f"Auteur : {self.author}")
        print(f"Année de sortie : {self.year}")
        print(f"Nombre de pages : {self.nbPages}")
        print(f"Langue : {self.language}\n\n")


book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
```

```
2018, 346, "Français")

book2 = Book("The Old Man and the Sea", "Ernest Hemingway", 1952,
109, "Anglais")

book1.get_infos()
book2.get_infos()
```

La différence entre ce programme et le programme précédent est que dans le programme précédent nous avons utilisé plusieurs appels à la fonction `print` afin d'afficher les différents attributs des deux objets `book1` et `book2`, alors que dans ce nouveau programme nous avons défini la méthode d'objet `get_infos` qui permet d'afficher les attributs d'un objet donné.

Avec ce nouveau programme, pour afficher les attributs d'un objet il suffit d'appeler la méthode `get_infos` et de l'appliquer sur un l'objet pour lequel nous souhaitons afficher les attributs.

Par exemple, dans notre code, nous avons appliqué la méthode `get_infos` sur l'objet `book1` grâce à l'instruction `book1.get_infos()` et, de la même manière, nous avons appliqué cette méthode `get_infos` sur l'objet `book2` grâce à l'instruction `book2.get_infos()`.

Le résultat de la méthode `get_infos` sera différent en fonction de l'objet sur lequel elle est appliquée. C'est en ce sens que cette méthode est une méthode d'objet et c'est exactement ce qui la distingue d'une méthode de classe. Dans la suite de cette annexe, nous allons voir un exemple de code avec une méthode de classe.

L'interpréteur Python va remplacer l'instruction `book1.get_infos()` dans le programme précédent par l'instruction `get_infos(book1)`. En effet, dans cette nouvelle instruction, le paramètre `book1` sera transmis comme étant la valeur de l'argument `self` de la fonction `get_infos` définie dans la classe `Book`. C'est la raison pour laquelle nous avons indiqué précédemment que le paramètre `self` permet de référencer un objet précis au moment de l'exécution d'une méthode.

En résumé, les méthodes d'objets permettent d'accéder, de réaliser des calculs et de modifier les attributs d'objets.

© Editions ENI - All rights reserved

1.2.6 Les attributs de classe

Contrairement à un attribut d'objet qui, lui, est associé à un objet précis, un attribut de classe n'est pas défini au niveau d'un objet mais plutôt au niveau d'une classe. Examinons l'exemple de code suivant (voir le code du fichier *Attributs_Classe.py*) :

```
class Book:
    nb_book = 0

    def __init__(self):
        type(self).nb_book = type(self).nb_book + 1

book1 = Book()
book2 = Book()
book3 = Book()

print(f"Le nombre de livres avec Book.nb_book  : {Book.nb_book}")
print(f"Le nombre de livres est : {book1.nb_book}")
print(f"Le nombre de livres est : {book2.nb_book}")
print(f"Le nombre de livres est : {book3.nb_book}")
```

Ce programme définit l'attribut `nb_book` en dehors de toute définition de méthodes ! En effet, vous remarquez que cet attribut, qui est initialisé à `0`, n'est pas défini dans la méthode `__init__` comme nous l'avons fait dans nos précédents exemples lorsque nous avions défini des attributs d'objets.

Dans la méthode `__init__`, vous remarquez que pour accéder à l'attribut `nb_book`, nous avons utilisé la notation `type(self).nb_book`. Le fait de précéder le nom de cet attribut par `type(self)` permet de signaler à l'interpréteur Python notre souhait d'accéder à un élément de la classe `Book` et non pas un élément d'un objet quelconque car, comme nous l'avons vu précédemment, `self` est associé à un objet précis et donc `type(self)` va nous retourner le type de l'objet `self`, qui est dans notre cas le type `Book`. Dans cette méthode `__init__`, nous avons tout simplement incrémenté la valeur de l'attribut `nb_book` d'une unité. Donc, à chaque fois qu'un objet de type `Book` sera créé, l'attribut `nb_book` sera incrémenté d'une unité.

L'attribut nb_book ne sera associé à aucun objet. En réalité, cet attribut est commun à tous les objets qui seront créés de type Book ! D'ailleurs, l'exécution de ce programme affiche le résultat suivant :

```
Le nombre de livres avec Book.nb_book  : 3
Le nombre de livres est : 3
Le nombre de livres est : 3
Le nombre de livres est : 3
```

On voit que la valeur de l'attribut nb_book est la même, quelle que soit la variable utilisée pour y accéder.

1.2.7 Les méthodes de classe

Contrairement aux méthodes d'objets, les méthodes de classes n'ont pas accès aux attributs d'objets. En revanche, elles ont accès aux attributs de classe. Pour une classe donnée, les méthodes de classes peuvent être vues comme des services offerts par cette classe et qui permettent de réaliser des traitements qui ne dépendent pas de l'état de l'objet sur lequel ces méthodes sont appliquées.

Si nous reprenons l'exemple de code précédent, nous pouvons ajouter la définition de la méthode de classe get_Book_infos comme suit (voir le code du fichier *Methode_Classe.py*) :

```
class Book:
    nb_book = 0

    def __init__(self):
        type(self).nb_book = type(self).nb_book + 1

    @classmethod
    def get_Book_infos(cls):
        if nb_bool == 0 :
            print("Pour l'instant aucun livre n'est crée !")
        else :
            print(f"Le nombre de livres est :{cls.nb_book}")

Book.get_Book_infos()

book1= Book()
book2= Book()
```

© Editions ENI - All rights reserved

```
Book.get_Book_infos()
```

Dans ce programme, la méthode de classe `get_Book_infos` affiche un message en fonction de la valeur de l'attribut de classe `nb_book`.

Remarquez l'utilisation du décorateur `@classmethod` avant la définition de la méthode `get_Book_infos`. Ce décorateur signale à l'interpréteur Python que la méthode qui suit est une méthode de classe. Dans la signature d'une méthode de classe, le premier paramètre correspond au type de la classe concernée. Ainsi, dans notre exemple, le paramètre `cls` correspond au type de la classe `Book`. En réalité, au moment de l'exécution de l'instruction `Book.get_Book_infos()`, l'interpréteur Python va remplacer cette instruction par l'instruction `get_Book_infos(Book)` de telle sorte que le paramètre `cls` va être associé à la valeur `Book`.

Il faut bien noter que dans l'instruction `Book.get_Book_infos()`, la méthode de classe `get_Book_infos()` est appliquée sur la classe `Book` est non pas sur un objet !

Ce programme affiche le résultat suivant :

```
Pour l'instant aucun livre n'est créé !
Le nombre de livres est :2
```

Remarque

Dans la pratique, les méthodes de classes sont souvent utilisées pour la création de nouvelles instances d'objets à partir de données différentes des données utilisées par le constructeur `__init__`.

Remarque

Dans cet exemple, le paramètre `cls` de la méthode `get_Book_infos` représente le nom de la classe `Book`. Vous pouvez nommer ce paramètre comme bon vous semble, le plus important est de savoir que le premier paramètre de cette méthode correspond au nom du type de la classe concernée par cette méthode. La même remarque est valable pour le paramètre `self` que vous pouvez également nommer comme vous le souhaitez, l'essentiel est de savoir que le premier paramètre d'une méthode d'objet fait toujours référence à l'objet auquel la méthode est appliquée.

1.2.8 Les méthodes statiques

Nous avons vu que les méthodes d'objets permettent de manipuler les états des objets, c'est-à-dire les attributs d'objets, et les méthodes de classes permettent de réaliser des traitements au niveau des classes comme manipuler des attributs de classes. Les méthodes statiques, quant à elles, peuvent être vues comme des services offerts par une classe et qui manipulent des concepts généraux relatifs aux concepts représentés par une classe. Ces méthodes statiques ne peuvent accéder ni aux attributs d'objets ni aux attributs de classes.

L'exemple de code suivant permet de montrer comment définir une méthode statique (voir le code du fichier *Methode_Statique.py*) :

```
class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language

    @staticmethod
    def is_french_book(book):
        return (book.language == "Français")


book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
"2018", "346", "Français")

book2 = Book("The Old Man and the Sea", "Ernest Hemingway", "1952",
"109", "Anglais")

print("book1 : ", Book.is_french_book(book1))
print("book2 : ", Book.is_french_book(book2))
```

Ce code crée deux livres de la classe `Book`, `book1` et `book2`, puis utilise la méthode statique `is_french_book` afin de vérifier si la langue dans laquelle sont écrits ces deux livres est le français.

Remarquez l'utilisation du décorateur `@staticmethod` avant la définition de la méthode `is_french_book`. Ce décorateur signale à l'interpréteur Python que la méthode qui suit est une méthode statique.

© Editions ENI - All rights reserved

Remarquez également que la signature de cette méthode `is_french_book` ne contient aucun des paramètres `self` ou `cls` comme pour les méthodes d'objets ou les méthodes de classes. C'est tout à fait normal puisqu'une méthode statique n'accède ni aux attributs d'objets ni aux attributs de classes.

Tout comme les méthodes de classes, les méthodes statiques s'appliquent directement sur leur classe et non pas sur un objet. Par exemple, dans notre code, la méthode statique `is_french_book` a été appliquée à deux reprises avec la notation `Book.is_french_book(...)`.

Ce programme affiche les résultats suivants :

```
book1 :  True
book2 :  False
```

1.2.9 Sécuriser les attributs

La sécurisation d'un attribut d'objet revient à décider de la portée que devrait avoir cet attribut. Par exemple, dans certains cas, une fois qu'un attribut est défini dans un objet, on aimerait que cet attribut ne soit utilisé que par les méthodes de cet objet. En d'autres termes, on aimerait que certains attributs ne soient pas visibles de l'extérieur des objets dans lesquels ils sont définis.

Pour donner un exemple concret, dans les deux derniers programmes, nous avons eu accès à l'attribut `title` de l'objet `book1` de deux manières différentes. Dans notre tout dernier programme, nous avons accédé à l'attribut `title` via la méthode `get_infos()` qui est définie à l'intérieur même de la classe `Book`. En revanche, dans notre avant-dernier programme, nous avons accédé à ce même attribut `title` à partir de l'extérieur de la classe `Book` avec l'écriture `book1.title`.

Dans plusieurs langages de programmation tels que C++, C# ou Java, dans la définition d'une classe, nous pouvons spécifier au compilateur notre souhait de cacher certains attributs afin de les rendre inaccessibles de l'extérieur de cette classe, voire même de les rendre inaccessibles à partir des classes dérivées de cette classe. Nous allons aborder la notion de classe dérivée dans la suite de cette annexe.

Au risque de décevoir le lecteur aguerri dans les langages orientés objet classiques, le langage Python n'a aucun moyen de protection d'attributs. Les attributs d'un objet sont accessibles de partout !

En revanche, il existe une convention d'écriture du code qui permet d'indiquer aux autres développeurs la visibilité d'un attribut. Ainsi, les attributs dont le nom commence par le caractère _ ne doivent jamais être utilisés à l'extérieur de la classe où ils sont définis. Bien sûr, c'est juste une convention d'écriture du code et Python ne va pas empêcher un développeur d'accéder à un attribut dont le nom commence par le caractère _ de l'extérieur de la classe où cet attribut est défini. Cette convention a été établie dans un esprit où les développeurs sont responsables et donc, lorsqu'ils rencontrent de tels attributs, ils savent que leur utilisation en dehors des classes où ils sont définis peut altérer le bon fonctionnement des programmes et rendre leur maintenance beaucoup plus complexe.

Il existe une astuce intéressante permettant d'assurer un niveau de sécurité minimal d'un attribut et qui consiste à nommer cet attribut en commençant par deux caractères _. Voyons l'exemple du code ci-dessous :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *Sécuriser_les_attributs-01.py* situé dans le dossier *Code* de cette annexe.

```
class PrivateClass:
    def __init__(self, valeur):
        self.__private = valeur

    def get_private_value(self):
        return self.__private

obj = PrivateClass(10)
print("La valeur de la variable private :",obj.__private)
```

L'exécution de ce programme nous donne le résultat ci-dessous :

```
Traceback (most recent call last):
  File "Sécuriser_les_attributs-01.py", line 9, in <module>
    print("La valeur de la variable private :",obj.__private)
AttributeError: 'PrivateClass' object has no attribute '__private'
```

© Editions ENI - All rights reserved

On voit bien que l'interpréteur Python n'aime pas l'instruction où l'attribut est utilisé en dehors de la classe et il considère que les objets de la classe `PrivateClass` n'ont pas d'attributs portant le nom `__private` bien que cet attribut existe. Pour l'interpréteur Python, cet attribut, qui est nommé avec deux caractères _ au début, est accessible uniquement via les méthodes de la classe `PrivateClass`.

Si un développeur souhaite rendre accessible la valeur d'un attribut déclaré avec un nom commençant par deux caractères _ sans pour autant référencer cet attribut de manière explicite à l'extérieur de sa classe, alors il faut définir une méthode d'objet permettant tout simplement de retourner la valeur de cet attribut. Pour l'exemple précédent, nous pouvons écrire le code suivant (voir le code du fichier *Sécuriser_les_attributs-02.py*) :

```
class PrivateClass:
    def __init__(self, valeur):
        self.__private = valeur

    def get_private_value(self):
        return self.__private

obj = PrivateClass(10)
print("La valeur de la variable private :",obj.get_private_va-
lue())
```

L'exécution de ce code donne le résultat suivant :

```
La valeur de la variable private : 10
```

Ainsi, la valeur de l'attribut `__private` peut être connue à l'extérieur de la classe `PrivateClass` sans pour autant que cet attribut soit référencé explicitement en dehors de cette classe. Cela peut être intéressant lorsque par exemple nous voulons exprimer notre souhait qu'un attribut soit utilisé en lecture à l'extérieur de sa classe, mais pas en écriture.

Bien sûr, comme nous l'avons mentionné précédemment, l'implémentation native de Python n'a aucun moyen de protection des attributs pour les rendre invisibles de l'extérieur d'une classe, et cela même lorsque nous nommons ces attributs avec deux caractères _ au début.

En effet, hélas, même si le nom d'un attribut commence avec les deux caractères _, nous pouvons en réalité accéder à cet attribut à partir de l'extérieur de sa classe en utilisant le nom de la classe même ! Considérons l'exemple suivant (voir le code du fichier *Sécuriser_les_attributs-03.py*) :

```
class PrivateClass:
    def __init__(self, valeur):
        self.__private = valeur


obj = PrivateClass(10)
print("La valeur de la variable private :",
obj._PrivateClass__private)
```

Exécutez ce programme pour constater qu'il affiche bien le résultat suivant :

```
La valeur de la variable private : 10
```

Dans ce dernier programme, nous avons utilisé l'attribut `_PrivateClass__private` pour accéder à l'attribut `__private` !

En réalité, Python, lorsqu'il a rencontré l'attribut `__private`, a compris que cet attribut ne doit pas être surdéfini dans les classes dérivées de la classe `PrivateClass` (pour rappel, nous allons aborder les classes dérivées dans la suite de cette annexe). Pour éviter que les classes dérivées ne puissent pas surdéfinir cet attribut, Python renomme `__private` en lui donnant le nouveau nom `_PrivateClass__private`. Python procède au renommage de tous les attributs dont le nom commence par les deux caractères _, en les préfixant avec un caractère _ suivi du nom de la classe en question. C'est précisément pour cette raison que dans l'un des exemples précédents, lorsque nous avons essayé d'accéder à l'attribut `__private` avec `obj.__private`, Python nous a affiché le message suivant :

```
AttributeError: 'PrivateClass' object has no attribute
'__private'
```

En effet, le nom de l'attribut `__private` est remplacé par `_PrivateClass__private`.

En résumé, avec le langage Python, la sécurité des attributs est basée sur des conventions de nommage que tout développeur Python doit connaître et respecter.

© Editions ENI - All rights reserved

1.3 L'héritage

Souvent, lorsque nous définissons les classes qui composent un projet, nous constatons que certaines de ces classes ont plusieurs éléments en commun, c'est-à-dire un sous-ensemble de classes qui ont en commun plusieurs attributs et/ou plusieurs méthodes. Dans ce cas-là, il serait intéressant de factoriser ou de regrouper ces attributs et ces méthodes communes dans une seule et même classe. Justement, l'héritage permet de réaliser ce type d'opérations de factorisation.

En effet, l'héritage est un concept qui nous permet de dériver de nouvelles classes, qu'on appelle communément des classes enfants, à partir de classes déjà existantes, qu'on appelle dans ce cas les classes parents.

Les attributs et les méthodes communes seront définis dans les classes parents et les classes enfants vont définir les attributs et les méthodes spécifiques.

L'héritage permet de nous éviter de dupliquer le code associé aux attributs et aux méthodes communs qui sont définis une seule fois dans les classes parents.

Une classe enfant peut hériter des attributs et des méthodes d'une classe parent ou de plusieurs classes parents. Ainsi, nous distinguons le concept de l'héritage simple du concept de l'héritage multiple.

1.3.1 L'héritage simple

On parle de l'héritage simple lorsqu'une nouvelle classe hérite des attributs et des méthodes d'une seule classe parent. L'exemple de code ci-après montre comment implémenter le concept de l'héritage simple (voir le code du fichier *Héritage_Simple.py*).

Dans cet exemple de code, nous avons défini les trois classes `Book`, `ITBook` et `NovelBook`. La classe `Book` contient toutes les informations concernant le concept du livre de manière générale, alors que la classe `ITBook` est associée au concept du livre traitant de la technologie et la classe `NovelBook` contient des informations relatives aux livres de type roman.

En d'autres termes, les classes `ITBook` et `NovelBook` représentent toutes les deux des livres, mais `ITBook` est spécialisée pour représenter des livres IT et `NovelBook` est spécialisée pour représenter des romans.

`ITBook` et `NovelBook` vont donc avoir des membres ou des éléments spécifiques, mais ces deux classes auront des éléments en commun.

La première classe est la classe `Book` qui est définie avec les cinq attributs d'objets `title`, `author`, `year`, `nbPages` et `language` et la méthode d'objet `get_infos`. Tous les membres de cette classe `Book` sont communs entre les deux classes `ITBook` et `NovelBook`.

```
class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language

    def get_infos(self):
        print(f"Voici les informations du livre : {self.title}")
        print(f"Auteur : {self.author}")
        print(f"Année de sortie : {self.year}")
        print(f"Nombre de pages : {self.nbPages}")
        print(f"Langue : {self.language}")
```

La deuxième classe `ITBook` va hériter de la classe `Book` (voir le code ci-dessous). Cela signifie que tout objet de type `ITBook` sera défini avec les attributs et les méthodes de la classe `ITBook`, mais cet objet sera également défini avec les attributs et les méthodes de la classe `Book`.

```
class ITBook(Book):
    def __init__(self, title, author, year, nbPages, language,
domain, technologies):
        super().__init__(title, author, year, nbPages, language)
        self.domain = domain
        self.technologies = technologies

    def get_infos(self):
        super().get_infos()
        print(f"Domaine d'application : {self.domain}")
        print(f"Technologies utilisées : {self.technologies}\n\n")
```

© Editions ENI - All rights reserved

On dérive la classe enfant `ITBook` à partir de la classe parent `Book` grâce à la notation `class ITBook(Book)`. Le fait de mentionner ainsi la classe `Book` entre deux parenthèses juste devant le nom de la classe `ITBook` suffit pour que la classe `ITBook` hérite des attributs et des méthodes de la classe `Book`.

La méthode `__init__` de la classe `ITBook` reçoit les paramètres `self`, `title`, `author`, `year`, `nbPages`, `language`, `domain`, `technologies` qui seront utilisés pour la création des objets de type `ITBook`.

Remarquez que dans cette méthode `__init__` de la classe enfant `ITBook` nous avons appelé la méthode `__init__` de la classe parent `Book` avec l'instruction :

```
super().__init__(title, author, year, nbPages, language)
```

Cette dernière instruction permet de réutiliser le code de la classe parent pour l'affectation des attributs hérités et c'est grâce à l'appel de la méthode `super(` que nous avons pu appeler la méthode `__init__` de la classe parent.

Remarquez également que lorsque nous avons appelé la méthode `__init__` de la classe parent, nous ne lui avons transmis que les cinq paramètres `title`, `author`, `year`, `nbPages` et `language` qui sont communs entre les classes `ITBook` et la classe `NovelBook`. Les deux autres paramètres `domain` et technologies qui, eux, sont spécifiques à la classe `ITBook`, ont été utilisés au sein de la méthode `__init__` de cette classe enfant pour compléter la création d'objets de type `ITBook`.

De la même manière, nous avons créé la classe `NovelBook` par héritage de la classe `Book`. Cette nouvelle classe `NovelBook` est définie avec les deux attributs spécifiques `category` et `movie_adaptation` en plus des cinq attributs communs avec la classe `ITBook`.

```
class NovelBook(Book):
    def __init__(self, title, author, year, nbPages, language,
category, movie_adaptation = False):
        super().__init__(title, author, year, nbPages, language)
        self.category = category
        self.movie_adaptation = movie_adaptation

    def get_infos(self):
        super().get_infos()
```

```
        print(f"Catégorie : {self.category}")
        print(f"Adaptation cinématographique :
{self.movie_adaptation}\n\n")
```

Remarquez que les deux classes `ITBook` et `NovelBook` définissent chacune sa propre méthode d'objet `get_infos` afin de prendre en compte l'affichage des attributs spécifiques en plus de l'affichage des attributs hérités de la classe `Book`. Dans ces deux méthodes `get_infos`, l'affichage des attributs hérités est réalisé en appelant la méthode `get_infos` de la classe parent en utilisant la méthode `super`.

En résumé, la classe `ITBook` hérite de la classe `Book` et le type de cet héritage est dit simple car `ITBook` a comme unique parent la classe `Book`. Également, la classe `NovelBook` hérite de la classe `Book` par un héritage simple.

La suite de ce programme consiste en la création de deux objets `book1` de type `ITBook` et `book2` de type `NovelBook`. Les informations de l'objet `book1` seront affichées avec la méthode `get_infos` de la classe `ITBook` qui elle-même fera appel à la méthode `get_infos` de la classe `Book` et les informations de l'objet `book` seront affichées avec la méthode `get_infos` de la classe `NovelBook` qui à son tour fera appel à la méthode `get_infos` de la classe `Book`.

```
book1 = ITBook("Data Science avec Microsoft Azure", "M.Khichane",
2018, 346, "Français", "Data Science", ["Azure", "Azure ML",
"PowerBI", "R"])

book2 = NovelBook("The Old Man and the Sea", "Ernest Hemingway",
1952, 109, "Anglais", "Roman", True)

book1.get_infos()
book2.get_infos()
```

Ce programme affiche le résultat suivant :

```
Voici les informations du livre : Data Science avec Microsoft
Azure
Auteur : M.Khichane
Année de sortie : 2018
Nombre de pages : 346
Langue : Français
Domaine d'application : Data Science
Technologies utilisées : ['Azure', 'Azure ML', 'PowerBI', 'R']
```

© Editions ENI - All rights reserved

```
Voici les informations du livre : The Old Man and the Sea
Auteur : Ernest Hemingway
Année de sortie : 1952
Nombre de pages : 109
Langue : Anglais
Catégorie : Roman
Adaptation cinématographique : True
```

1.3.2 L'héritage multiple

L'héritage multiple est une forme généralisée de l'héritage simple. Avec l'héritage simple, une classe enfant hérite d'une seule classe parent. Avec l'héritage multiple, une classe enfant hérite de plusieurs classes parents. Afin de comprendre l'héritage multiple, nous allons considérer l'exemple de code suivant (voir le code du fichier *Héritage_multiple.py*) :

Dans ce nouveau code, nous avons réutilisé exactement la même classe `Book` que celle définie dans l'exemple précédent. Nous avons ajouté la nouvelle classe `Product_Line` comme suit :

```
class Product_Line :

    def __init__(self, product_line_name, price):
        self.product_line_name =  product_line_name
        self.price = price

    def get_infos(self):
        print(f"Gamme : {self.product_line_name}")
        print(f"Prix : {self.price}")
```

La classe `Product_Line` permet de définir des objets avec les deux attributs `product_line_name`, `price`.

Puis nous avons modifié le code de la classe `ITBook` précédente comme suit :

```
class ITBook(Book,Product_Line):
    def __init__(self, title, author, year, nbPages, language,
                 domain, technologies, product_line_name, price):
        Book.__init__(self,title, author, year, nbPages,
                      language)
        Product_Line.__init__(self, product_line_name, price)
        self.domain = domain
```

```
        self.technologies = technologies

    def get_infos(self):
        Book.get_infos(self)
        Product_Line.get_infos(self)
        print(f"Domaine d'application : {self.domain}")
        print(f"Technologies utilisées :
                {self.technologies}\n\n")
```

Remarquez que dans ce nouveau code, nous avons dérivé la classe `ITBook` à partir des deux classes `Book` et `Product_Line` en mentionnant ces deux classes parents entre parenthèses au tout début de la définition de la classe `ITBook`.

Les méthodes `__init__` des deux classes parents ont été appelées en utilisant le nom de ces classes à l'intérieur de la méthode `__init__` de la classe `ITBook`.

Également, la méthode `get_infos` de la classe `ITBook` évoque les deux méthodes `get_infos` des deux classes parents.

Les deux instructions suivantes permettent de créer un objet de type `ITBook` et d'afficher les valeurs de ses attributs.

```
book1 = ITBook("Data Science avec Microsoft Azure", "M.Khichane",
2018, 346, "Français", "Data Science", ["Azure", "Azure ML",
"PowerBI", "R"], "Epsilon", 54)
book1.get_infos()
```

Ce programme affiche le résultat suivant :

```
Voici les informations du livre : Data Science avec Microsoft
Azure
Auteur : M.Khichane
Année de sortie : 2018
Nombre de pages : 346
Langue : Français
Gamme : Epsilon
Prix : 54
Domaine d'application : Data Science
Technologies utilisées : ['Azure', 'Azure ML', 'PowerBI', 'R']
```

© Editions ENI - All rights reserved

Les cinq premières lignes de ce résultat sont affichées par la méthode `get_infos` de la classe `Book`, puis les deux lignes suivantes sont affichées par la méthode `get_infos` de la classe `Product_Line` et les deux dernières lignes sont affichées par la méthode `get_infos` de la classe `ITBook`.

Remarque

Même si l'héritage multiple est un concept intéressant, il est très peu pratiqué sur le terrain. En effet, d'une part l'utilisation de l'héritage multiple peut facilement aboutir à des conceptions et architectures d'applications complexes, donc difficile à maintenir, et d'autre part l'héritage multiple peut être à l'origine de comportements inattendus et indésirables d'un programme.

1.4 Les classes abstraites

Dans les exemples d'héritage simple et multiple précédents, nous avons défini des classes parents, comme les classes `Book` et `Product_Line`, et nous avons défini des classes enfants qui ont hérité de ces classes parents. Puis nous avons instancié les classes enfants pour montrer l'effet de l'héritage sur des objets créés à partir des classes enfants. En réalité, nous aurions pu instancier des objets directement à partir des classes parents.

Per exemple, dans l'exemple précédent, nous aurions pu créer un objet de type `Book` ou de type `Product_Line`.

Souvent, nous avons besoin de définir une classe et de bloquer la possibilité de l'instancier afin de la rendre uniquement utilisable en tant que classe parent. En d'autres termes, nous avons besoin de définir des classes qui n'ont de sens que si elles sont utilisées en tant que classes parents via le mécanisme de l'héritage.

Pour empêcher l'instanciation d'une classe, il suffit de la déclarer en tant que classe abstraite !

En réalité, le besoin de rendre une classe non instanciable, c'est-à-dire abstraite, vient du besoin de forcer toute classe enfant à implémenter certaines méthodes. Le programme ci-après donne un exemple concret d'utilisation de classe abstraite, (voir le code du fichier *Classe_Abstrait.py*).

Avec Python, pour implémenter la notion de classe abstraite il est nécessaire d'utiliser des éléments définis dans le module `abc`.

Cet exemple de code commence par importer la classe `ABC` et la fonction `abstractmethod` afin de pouvoir implémenter respectivement le concept de classe abstraite et de méthode abstraite. Toute classe qui possède une méthode abstraite est elle-même une classe abstraite.

Dans la classe `Geom`, la méthode `get_area()` est une méthode abstraite, puisqu'elle est précédée du décorateur `@abstractmethod`.

```
from abc import ABC, abstractmethod

class Geom(ABC):
    def __init__(self, color):
        self.color = color

    @abstractmethod
    def get_area():
        pass
```

L'unique instruction de la méthode `get_area` est l'instruction `pass` qui signifie que cette méthode ne réalise aucun traitement dans la classe `Geom`. En revanche, comme cette méthode `get_area` est abstraite, cela signifie deux choses : la première est que la classe `Geom` est elle-même une classe abstraite et la deuxième est que toute classe qui hérite de la classe `Geom` doit absolument définir cette méthode `get_area`. Nous pouvons donc dire que les méthodes abstraites représentent en quelque sorte un contrat qui doit être respecté par les classes enfants.

La classe `Geom` définit un seul attribut qui est l'attribut `color`.

La suite de ce programme définit les deux classes `Circle` et `Square` qui héritent toutes les deux de la classe abstraite `Geom`. De ce fait, ces deux nouvelles classes doivent absolument définir en leur sein la méthode `get_area`.

```
class Circle(Geom):
    def __init__(self, color, radius):
        super().__init__(color)
        self.radius = radius

    def get_area(self):
        area = 3.14 * (self.radius**2)
```

© Editions ENI - All rights reserved

```
        return area

class Square(Geom):
    def __init__(self, color, side):
        super().__init__(color)
        self.side = side

    def get_area(self):
        area = self.side**2
        return area
```

La méthode `get_area` de la classe `Circle` est bien sûr différente de la méthode `get_area` de la classe `Square`. La suite de ce programme définit un objet `circle` de type `Circle` et un objet `square` de type `Square`, puis le résultat de l'exécution de leur méthode `get_area` est affiché.

```
circle = Circle("red", 5)
square = Square("green", 5)

print(f"Cercle de diamètre {circle.radius} a une surface de
{circle.get_area()}")

print(f"Carré de côté {square.side}cm a une surface de
{square.get_area()}")
```

Ce programme affiche le résultat suivant :

```
Cercle de diamètre 5 a une surface de 78.5
Carré de côté 5cm a une surface de 25
```

1.5 Les interfaces

Une interface représente un contrat de comportement que doit respecter toute classe implémentant cette interface. Définir une nouvelle interface revient à définir un nouveau type dans lequel nous allons déclarer un certain nombre de méthodes. Ces méthodes seront uniquement déclarées mais pas définies, c'est-à-dire que nous allons juste définir leur signature mais pas leur corps. Puis nous obligerons toute classe qui implémente cette nouvelle interface à définir concrètement toutes les méthodes déclarées dans cette interface.

Avec le langage Python, les interfaces ne sont pas supportées en tant que telles. Néanmoins, nous pouvons obtenir un comportement similaire aux interfaces en nous appuyant sur les classes et les méthodes abstraites. Le code ci-après montre un exemple où une classe implémente la pseudo-interface `JSONify` (voir le code du fichier *Interface.py*).

Remarque

Cette interface `JSONify` est qualifiée ici de pseudo-interface car elle n'en est pas une à proprement parler. En effet, les langages de programmation classiques tels que C++, Java ou C# permettent d'implémenter les interfaces par un mécanisme dédié, alors qu'en Python nous ne pouvons que simuler le fonctionnement des interfaces.

Dans cet exemple, nous avons commencé par la définition de la classe abstraite `JSONify` où sont déclarées les deux méthodes abstraites `to_json_string` et `save_to_file`.

```
from abc import ABC, abstractmethod
import json

class JSONify(ABC):
    @abstractmethod
    def to_json_string(self):
        pass

    @abstractmethod
    def save_to_file(self):
        pass
```

Dans ce nouveau programme, nous avons réutilisé la classe `Book` qui, pour rappel, est définie comme suit :

```
class Book:
    def __init__(self, title, author, year, nb_pages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nb_pages = nb_pages
        self.language = language

    def get_infos(self):
        print(f"Voici les informations du livre : {self.title}")
```

© Editions ENI - All rights reserved

```
        print(f"Auteur : {self.author}")
        print(f"Année de sortie : {self.year}")
        print(f"Nombre de pages : {self.nb_pages}")
        print(f"Langue : {self.language}")
```

La classe `ITBook` est dérivée de la classe `Book` et de la classe abstraite `JSONify`. Cela signifie que la classe `ITBook` hérite de tous les attributs et des méthodes de la classe `Book`, mais aussi que la classe `ITBook` doit définir les méthodes abstraites de la classe abstraite `JSONify`. Comme la classe `ITBook` doit définir les méthodes abstraites `to_json_string` et `save_to_file`, nous pouvons dire que finalement `JSONify` est une interface !

```
class ITBook(Book, JSONify):

    def __init__(self, title, author, year, nb_pages, language,
                 domain, technologies):
        super().__init__(title, author, year, nb_pages, language)
        self.domain = domain
        self.technologies = technologies

    def get_infos(self):
        super().get_infos()
        print(f"Domaine d'application : {self.domain}")
        print(f"Technologies utilisées :
              {self.technologies}\n\n")

    def to_json_string(self):
        technologies = '['
        max_size = len(self.technologies)
        for index in range(0, max_size-1) :
            technologies+=f'"{self.technologies[index]}",'
        technologies+= f'"{self.technologies[max_size-1]}"]'
        json_form= f'{{"Titre":"{self.title}",'\
                   + f'"Author":"{self.author}",'\
                   + f'"Year":"{self.year}",'\
                   + f'"nbPages":"{self.nb_pages}",'\
                   + f'"Language":"{self.language}",'\
                   + f'"Field":"{self.domain}",'\
                   + f'"Technologies":{technologies}}}'
        return json_form


    def save_to_file(self, filename):
```

```
        json_str = json.loads(self.to_json_string())
        with open(filename, "w") as json_file :
            json.dump(json_str, json_file, indent=4,
                      ensure_ascii=False)
```

La méthode `to_json_string` permet de transformer les valeurs des attributs d'un objet de type `ITBook` en une chaîne de caractères formatée en JSON. La méthode `save_to_file` permet d'enregistrer la chaîne de caractères retournée par la méthode `to_json_string` dans un fichier dont le nom est reçu en argument.

La suite de ce programme permet de créer l'objet `book1` de type `ITBook`, puis d'enregistrer cet objet dans le fichier *book1.json*.

```
book1 = ITBook("Data Science avec Microsoft Azure", "M.Khichane",
2018, 346, "Français", "Data Science", ["Azure", "Azure ML",
"PowerBI", "R"])

book1.save_to_file("../Data_Files/book1.json")
```

En exécutant ce programme, vous devriez trouver le fichier *book1.json* créé dans le dossier *Data_Files* avec le contenu JSON suivant :

```
{
    "Titre": "Data Science avec Microsoft Azure",
    "Author": "M.Khichane",
    "Year": "2018",
    "nbPages": "346",
    "Language": "Français",
    "Field": "Data Science",
    "Technologies": [
        "Azure",
        "Azure ML",
        "PowerBI",
        "R"
    ]
}
```

En synthèse de cet exemple, le fait que la classe `ITBook` a hérité de la classe abstraite `JSONify` nous a permis de simuler le comportement d'une interface.

© Editions ENI - All rights reserved

1.6 Les méthodes spéciales

Les méthodes spéciales, appelées en anglais les *magic methods* ou les *dunder methods*, sont des méthodes qui ne sont habituellement pas appelées explicitement par le programmeur, mais qui sont plutôt appelées de manière implicite pour réaliser des tâches bien déterminées au moment de la survenue de certains événements.

Le nom d'une méthode spéciale est toujours préfixé par deux caractères _ et également suffixé par deux caractères _. La suite de cette section donne plusieurs exemples de ces méthodes spéciales.

1.6.1 Afficher un objet avec la fonction print()

En programmation orientée objet, nous sommes souvent amenés à transformer les objets de leur représentation en mémoire vive vers une représentation et en chaînes de caractères. C'est le cas lorsque nous souhaitons afficher à l'écran les valeurs prises par les différents attributs d'un objet.

L'affichage à l'écran est habituellement réalisé avec la fonction `print`. Le problème est que cette fonction `print` fonctionne très bien lorsque nous lui demandons d'afficher une variable de type `int` ou `str` ; en revanche, si nous demandons à cette fonction `print` d'afficher un objet, le résultat obtenu est l'adresse mémoire dans laquelle cet objet est stocké, comme dans l'exemple de la section Instanciation d'un objet de cette annexe où nous avions affiché à l'aide de la fonction `print` les deux objets `book1` et `book2`. Pour rappel, le résultat de l'exécution des deux instructions :

```
print("book1:", book1)
print("book2:", book2)
```

était le suivant :

```
book1: <__main__.Book object at 0x00000242029B78E0>
book2: <__main__.Book object at 0x0000024202A4D190>
```

Bien sûr, ce résultat n'est généralement pas le résultat souhaité ! Lorsque nous souhaitons afficher un objet, ce sont les valeurs prises par ses différents attributs à un instant donné qui nous intéressent généralement.

Justement, afin de pouvoir continuer à utiliser la fonction `print` pour l'affichage des résultats à l'écran tout en affichant des résultats autres que les adresses mémoire des objets lorsque cette fonction est appliquée sur des objets, nous allons utiliser une méthode spéciale. Il s'agit de la méthode `__str__`.

En réalité, lorsque nous appelons la fonction `print` sur un objet, cette fonction `print` va faire appel à son tour à la méthode `__str__` de l'objet sur lequel elle est appliquée. Cette méthode `__str__` est par défaut définie pour toutes les classes.

Donc, si nous souhaitons changer le comportement de la fonction `print` lorsqu'elle est appliquée sur un objet d'une de nos classes, il suffit de redéfinir le comportement de la méthode `__str__`, qui par défaut retourne l'adresse d'un objet. Pour mieux comprendre, examinons l'exemple de code suivant (voir le code du fichier *Magic_method_print.py*) :

```
class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language

    def __str__(self):
        book_to_string = f"Voici les informations du livre :
                         {self.title}\n"\
                         f"Auteur : {self.author}\n"\
                         f"Année de sortie : {self.year}\n"\
                         f"Nombre de pages : {self.nbPages}\n"\
                         f"Langue : {self.language}\n\n"
        return book_to_string
```

© Editions ENI - All rights reserved

La classe Book a les mêmes attributs que la classe Book des exemples précédents. La méthode __str__ a été redéfinie afin d'afficher les valeurs des attributs de tout objet de type Book. La suite de ce code permet de créer les deux objets book1 et book2 de type Book, puis la fonction print a été appelée pour afficher les deux objets book1 et book2.

```
book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
            2018, 346, "Français")

book2 = Book("The Old Man and the Sea", "Ernest Hemingway", 1952,
            109, "Anglais")

print(book1)
print(book2)
```

Cette fois, contrairement à l'exemple de la section Instanciation d'un objet de cette annexe où la fonction print affichait les adresses mémoire des objets book1 et book2, le résultat des deux appels à la fonction print est le suivant :

```
Voici les informations du livre : Data Science avec Microsoft
Azure
Auteur : M.Khichane
Année de sortie : 2018
Nombre de pages: 346
Langue : Français


Voici les informations du livre : The Old Man and the Sea
Auteur : Ernest Hemingway
Année de sortie : 1952
Nombre de pages : 109
Langue : Anglais
```

Ce résultat montre que les appels à la fonction print se sont comportés comme si nous avions appelé directement la fonction __str__.

1.6.2 Personnaliser les accès aux attributs d'une classe

Souvent, lorsque nous définissons les classes d'une application, nous aimerions personnaliser les traitements réalisés au moment de l'affectation de certains attributs de certaines classes. C'est ce qu'on appelle habituellement en programmation orientée objet les accesseurs et les mutateurs, ou en anglais respectivement les *getters* et les *setters*.

Avec le langage Python, on redéfinit la méthode spéciale `__getattribute__` pour implémenter la logique des getters et on redéfinit la méthode spéciale `__setattr__` pour implémenter la logique des setters comme dans l'exemple suivant (voir le code du fichier *Magic_method_set_get.py*) :

```
class Book:

    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price


    def __getattribute__(self, attr_name):
        if attr_name == "price":
            return (super().__getattribute__(attr_name)  * 0.80)
        return super().__getattribute__(attr_name)


    def __setattr__(self, attr_name, attr_value):
        if attr_name == "price":
            if type(attr_value) is not float:
                print("Le prix doit être de type float!")
                raise ValueError
        return super().__setattr__(attr_name, attr_value)


book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
             100.00)

print(f"Le prix de {book1.title} = { book1.price}")
```

© Editions ENI - All rights reserved

La méthode `__getattribute__`, lorsque l'argument `attr_name` est égal à `price`, retourne la valeur de l'attribut `price` diminuée de 20 %. Sinon, si l'argument `attr_name` concerne un attribut autre que l'attribut `price`, alors la valeur de cet attribut sera tout simplement retournée.

La méthode `__setattr__` permet de personnaliser les associations des valeurs avec l'attribut `price`. En effet, cette méthode lève une exception de type `ValueError` si nous essayons d'affecter une valeur autre qu'une valeur réelle à l'attribut `price`.

L'objet `book1` a été créé avec la valeur de l'attribut `price` égale à 100. Le résultat de ce programme est le suivant :

```
Le prix de Data Science avec Microsoft Azure = 80.0
```

Remarquez l'effet de la méthode `__getattribute__` qui nous a retourné la valeur de 80.0 au lieu de la valeur 100.

Si vous essayez d'exécuter ce programme en modifiant la valeur de l'attribut `price` au moment de la création de l'objet `book1` en la positionnant par exemple à 100, ce qui correspond à une valeur entière et non pas une valeur réelle, alors vous constaterez que la méthode `__setattr__` vous répondra par une exception de type `ValueError`.

1.6.3 Vérifier la validité d'un attribut

Contrairement aux langages tels que les langages C++ et Java, avec le langage Python les attributs des objets sont entièrement définis dans la méthode `__init__`. De ce fait, certains objets, même s'ils sont de la même classe, peuvent ne pas avoir les mêmes attributs ! Il est donc important de vérifier si le nom d'un attribut auquel nous essayons d'accéder existe ou pas.

La méthode spéciale `__getattr__` est appelée automatiquement si un programme tente d'accéder à un attribut d'un objet qui n'existe pas. Pour donner un exemple, considérons le code suivant (voir le code du fichier *Magic_method_getattr.py*) :

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        if price < 60 :
```

```
            self.price = price

    def __getattr__(self, attr_name):
        return "l'attribut "+ attr_name + " n'est pas défini pour
                cet objet"


book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
            54)
book2 = Book("Le Machine Learning avec Python", "M.Khichane",
             1000)

print(f"Le titre =  {book1.title}: prix = {book1.price}")
print(f"Le titre =  {book2.title}: prix = {book2.price}")
```

Ce programme redéfinit la méthode spéciale `__getattr__` qui sera automatiquement appelée par l'interpréteur Python si ce programme tente d'accéder à un attribut qui n'existe pas.

On voit que l'attribut `price` ne sera créé que pour les livres dont le prix est inférieur à 60 ; c'est le cas du livre `book1`. Si nous appelons la méthode `__init__` avec la valeur du dernier argument supérieure ou égale à 60, alors ce programme ne va pas créer l'attribut `price` pour cet objet, ce qui est le cas du livre `book2`.

Le résultat suivant montre ce qui se passe lorsque la dernière fonction `print` tente d'afficher `book2.price`.

```
Le titre =  Data Science avec Microsoft Azure: prix = 54
Le titre =  Le Machine Learning avec Python: prix = l'attribut
price n'est pas défini pour cet objet
```

On voit qu'à la place prévue pour la valeur de l'attribut `price` au niveau de la dernière instruction, c'est le message retourné par la méthode spéciale `__getattr__` qui est affiché, puisque l'attribut `price` n'existe pas pour l'objet `book2`.

Remarque

À partir de la version 3.7 du langage Python, nous pouvons obtenir des représentations d'objets plus ou moins similaires aux langages orientés objet classiques en utilisant le module `dataclasses`. Ce module permet de définir des classes de manière descriptive.

© Editions ENI - All rights reserved

1.6.4 Comparer deux objets

Habituellement, les opérateurs binaires tels que +, -, *, /, > et < sont utilisés pour réaliser des traitements entre des valeurs numériques. Grâce aux méthodes spéciales, nous pouvons réutiliser ces opérateurs et les appliquer sur nos propres objets !

Par exemple, supposons que nous souhaitons comparer deux livres par rapport à leur année de sortie : si l'année de sortie du livre `book1` est supérieure à l'année de sortie du livre `book2`, alors nous aimerions que l'opération `book1>book2` nous retourne la valeur `True`. L'exemple du code suivant montre comment implémenter l'opérateur > entre deux objets de type `Book` (voir le code du fichier *Magic_method_gt.py*) :

```
class Book:

    def __init__(self, title, author, year, nb_pages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nb_pages = nb_pages
        self.language = language

    def __gt__(self, book):
        result = self.year > book.year
        return result

book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
             2018, 346, "Français")

book2 = Book("The Old Man and the Sea", "Ernest Hemingway", 1952,
             109, "Anglais")

my_bool =  book2 >  book1

print("book1>book2 ?: ", my_bool)
```

Ce programme implémente l'opérateur > entre des objets de type `Book` en définissant la méthode spéciale `__gt__`. Cette méthode compare l'objet référencé avec l'argument `self` avec l'objet référencé par l'argument `book` en se basant sur leur attribut `year`.

Si `self.year` est supérieur à `book.year`, alors cette méthode retourne la valeur `True`, sinon elle retourne la valeur `False`.

Comme le livre `book1` est sorti en 2018 et `book2` est sorti en 1952, ce programme affiche le résultat `False` comme suit :

```
book1>book2 ?:  False
```

Dans cet exemple, nous avons montré comment implémenter la logique de l'opérateur en redéfinissant la méthode `__gt__`. De la même manière, vous pouvez implémenter la logique des opérateurs =, ≠, <, ≤, ≥ en redéfinissant respectivement les méthodes `__eq__`, `__nq__`, `__lt__`, `__le__` et `__ge__`.

1.6.5 Rendre les objets callable

Grâce aux méthodes spéciales, nous pouvons mentionner à l'interpréteur Python ce qu'il doit réaliser comme traitement si nous utilisons un objet en tant que fonction (c'est-à-dire utiliser un objet comme si c'était une fonction) ! Cela est souvent utile lorsque le nombre de fois où une méthode est utilisée devient important. Pour mieux comprendre, examinons l'exemple de code suivant (voir le code du fichier *Magic_method_call.py*) :

```
class Book:

    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def __call__(self):
        self.price = 10

book1 = Book("Data Science avec Microsoft Azure", "M.Khichane",
             100)
print(f"Le prix de {book1.title} = { book1.price}")

book1()

print(f"Le prix de {book1.title} = { book1.price}")
```

© Editions ENI - All rights reserved

Ce programme définit la classe `Book` avec les trois attributs `title`, `author` et `price` et la méthode spéciale `__call__`. Lorsque cette méthode est appelée, elle affecte la valeur 10 à l'attribut `price`.

L'objet `book1` a été créé avec la valeur 100 pour l'attribut `price`. Le premier appel à la fonction `print` affiche la valeur de l'attribut `price` juste après la création de l'objet `book1`.

Remarquez l'avant-dernière instruction de ce programme où l'objet `book1` a été utilisé comme si c'était une fonction ! Il s'agit de l'instruction `book1()`.

Le fait d'avoir utilisé un objet de type `book` comme étant une fonction implique que l'interpréteur Python sait qu'à ce moment-là il doit appeler la méthode `__call__`. Donc, après l'exécution de cette instruction `book1()`, la valeur de `price` sera fixée à 10 et c'est ce qu'affiche la dernière instruction `print`. Le résultat de ce programme est le suivant :

```
Le prix de Data Science avec Microsoft Azure = 100
Le prix de Data Science avec Microsoft Azure = 10
```

2. Les modules

Les modules sont tout simplement des fichiers contenant du code Python et qui nous permettent de mieux structurer nos programmes. En effet, les modules permettent une meilleure organisation du code en répartissant les fonctions et les classes et toute autre partie de nos codes sources de manière cohérente dans différents fichiers.

Il existe principalement les deux catégories de modules suivantes :

1. Les modules standards, qui ne font pas partie directement du langage Python mais qui sont automatiquement installés et intégrés au moment de l'installation de l'interpréteur Python comme, par exemple, les modules `random` et `re` que nous avons eu l'occasion d'utiliser dans ce livre.

2. Les modules externes, qui, eux, sont développés par d'autres développeurs, dont nous-mêmes, et que nous devons importer dans nos programmes afin de les utiliser. Nous pouvons distinguer les deux types de modules externes suivants :
   - Les modules externes développés par un éditeur tiers que nous devons installer dans nos environnements de travail avant de les utiliser, comme par exemple certains des modules que nous avons utilisés dans ce livre tels que Pandas, NumPy, sklearn ou TensorFlow.
   - Les modules externes que nous développons nous-mêmes afin de pouvoir les utiliser dans nos différents projets.

2.1 Importer des modules

Dans ce livre, nous avons déjà eu l'occasion de voir comment utiliser des modules standards et des modules externes développés par un tiers. L'exemple de code suivant montre comment créer et utiliser son propre module (voir le code des deux fichiers *Book_Module.py* et *main_1.py* du dossier *Module* de cette annexe).

Dans le fichier *Book_Module.py*, nous avons commencé par l'affichage du message `Import du module Book_Module`, puis nous avons défini la classe `Book` et nous avons également défini la fonction `just_function`. Remarquez que cette fonction n'est pas définie dans la classe Book !

```
print("\nImpor du module Book_Module\n")

class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language

    def get_infos(self):
        print(f"Voici les informations du livre : {self.title}")
        print(f"Auteur : {self.author}")
        print(f"Année de sortie : {self.year}")
```

© Editions ENI - All rights reserved

```
        print(f"Nombre de pages : {self.nbPages}")
        print(f"Langue : {self.language}\n\n")


def just_function():
    print("Je suis une fonction du module Book_Module !")
```

Le fait que ce code soit enregistré dans un fichier avec l'extension *.py* signifie que nous pouvons utiliser le code de ce fichier en tant que module !

Justement, dans le fichier main.py, nous avons importé ce module `Book_Module` grâce à l'instruction import `Book_Module`.

```
import Book_Module

book1 = Book_Module.Book("The Old Man and the Sea", "Ernest
                         Hemingway", 1952, 109, "Anglais")


book1.get_infos()

Book_Module.just_function()
```

Une fois que le module `Book_Module` a été importé, nous pouvons utiliser la classe `Book` ainsi que la fonction `just_function` en les préfixant avec le nom du module `Book_Module`. Ainsi, nous avons créé l'objet `book1`, puis nous avons appelé sa méthode `get_infos` et nous avons également appelé la fonction `just_function`. Le résultat de l'exécution de la commande python *main_1.py* est le suivant :

```
Import du module Book_Module

Voici les informations du livre : The Old Man and the Sea
Auteur : Ernest Hemingway
Année de sortie : 1952
Nombre de pages : 109
Langue : Anglais


Je suis une fonction du module Book_Module !
```

Remarquez que la fonction `print` qui figure au tout début du module `Book_Module` a été exécutée dès l'import de ce module ! Cela signifie que toute instruction, en dehors de la définition de nouveaux types et de nouvelles fonctions, sera exécutée au moment de l'import d'un module.

Dans le fichier *main_1.py*, nous avons importé tous les éléments définis dans le module `Book_Module`. Souvent, lorsque nous travaillons avec un module, nous n'avons pas besoin d'importer tout son contenu, mais uniquement une partie de ses éléments. Le code du fichier *main_2.py* montre comment importer la classe `Book` du module `Book_Module` sans importer la fonction `just_function` comme dans l'exemple précédent :

```
from Book_Module import Book

book1 = Book("The Old Man and the Sea", "Ernest Hemingway", 1952,
109, "Anglais")

book1.get_infos()
```

Dans ce nouveau code, pour importer la classe `Book` du module `Book_Module`, nous avons utilisé l'instruction `from Book_Module import Book`. Cela implique que la classe `Book` devient visible à partir du module `main_2`. D'ailleurs, vous remarquez que cette fois nous n'avons pas eu besoin de préfixer la classe `Book` par `Book_Module` comme dans l'exemple précédent. L'exécution de ce programme avec la commande python *main_2.py* affiche les informations du livre `book1`, mais aussi la fonction `print` au tout début du module `Book_Module` sera exécutée.

Dans ce fichier *main_2.py*, si vous tentez d'appeler la fonction `just_function` du module `Book_Module`, l'interpréteur Python vous affichera un message d'erreur car cette fonction n'a pas été importée ! Dans le fichier *main_2.py*, si vous tentez d'appeler cette fonction `just_function` en la préfixant avec `Book_Module`, cela ne fonctionnera pas non plus car même le module `Book_Module` n'est pas importé, il y a uniquement la classe `Book` qui est importée de ce module.

© Editions ENI - All rights reserved

2.2 Le module principal

Avec le langage Python, théoriquement, tout fichier portant l'extension *.py* peut être exécuté et considéré comme étant le programme principal de votre application, c'est-à-dire que tout fichier *.py* peut être considéré comme le point d'entrée de votre application.

En effet, contrairement aux langages tels que Java ou C# où il est nécessaire d'indiquer au moment de l'exécution d'un programme le fichier contenant la méthode `main`, avec l'interpréteur Python, tout code bien formé rencontré dans le fichier qui a été indiqué après la commande `python` sera exécuté.

C'est-à-dire que lorsque nous développons une application, tous les fichiers portant l'extension *.py* peuvent être lancés avec la commande `python`. Cependant, pour conserver la cohérence de l'exécution des différentes parties de code constituant notre application, il est important de distinguer le fichier qui représente le point d'entrée de notre application.

Si nous reprenons le module `Book_Module` précédent et si vous exécutez la commande `python Book_Module.py`, vous allez constater l'affichage du message `Import du module Book_Module` qui est affiché par l'appel de la fonction `print` située au tout début de ce module. Donc, nous constatons que le fichier *Book_Module.py* s'exécute au même titre que les fichiers *main_1.py* et *main_2.py* des deux derniers exemples.

Dans un script Python, nous pouvons utiliser la variable ou l'attribut `__name__` afin de détecter si ce script est celui lancé avec la commande `python` ou pas. Pour mieux comprendre, considérons l'exemple de code du fichier *Book_Module_bis.py* suivant :

```
import sys
print(f"Je suis le programme {__name__}")
if __name__ == "__main__":
    sys.exit(0)


print("\nImport du module Book_Module\n")

class Book:
    def __init__(self, title, author, year, nbPages, language):
        self.title = title
```

```
        self.author = author
        self.year = year
        self.nbPages = nbPages
        self.language = language

    def get_infos(self):
        print(f"Voici les informations du livre : {self.title}")
        print(f"Auteur : {self.author}")
        print(f"Année de sortie : {self.year}")
        print(f"Nombre de pages : {self.nbPages}")
        print(f"Langue : {self.language}\n\n")


def just_function():
    print("Je suis une fonction du module Book_Module !")
```

Dans ce code, nous avons repris à l'identique la définition du module `Book_Module` précédent, mais nous avons ajouté les quatre premières lignes tout en haut. Ainsi, nous avons importé le module standard `sys`, puis nous avons affiché un message qui indique la valeur de la variable `__name__`. Cette variable correspond au nom du module associé au fichier qui a été indiqué devant la commande `python`.

La troisième ligne de ce programme compare la valeur de la variable `__name__` avec la valeur `__main__`. Si `__name__` est égale à `__main__`, cela veut dire que c'est bien le fichier *Book_Module_bis.py* qui a été placé après la commande `python` au moment du lancement de ce programme ; sinon, si `__name__` est différent de `__main__`, alors cela signifie que la commande `python` a été lancée avec un autre fichier que le fichier *Book_Module_bis.py*.

Dans cet exemple, si la commande `python Book_Module_bis.py` est lancée, le résultat attendu est l'affichage du nom du module principal, c'est-à-dire `__main__`, suivi du message indiquant la fin du programme. Cela veut dire que la classe `Book` et la fonction `just_function` ne seront pas définies, puisque le programme prend fin avant que l'interpréteur Python ait le temps de rencontrer leur définition.

© Editions ENI - All rights reserved

L'exécution de la commande `python Book_Module_bis.py` affiche le résultat suivant :

```
Je suis le programme __main__
Fin de ce programme !
```

Dans cet exemple, nous avons rendu le module `Book_Module_bis` utilisable uniquement en tant que module externe importé dans un autre module, mais pas utilisable directement en tant que module principal.

Le code suivant (voir le fichier *main_3.py*) montre comment importer le module `Book_Module_bis` :

```
from Book_Module_bis import Book

book1 = Book("The Old Man and the Sea", "Ernest Hemingway", 1952,
109, "Anglais")

book1.get_infos()
```

L'exécution de la commande `python main_3.py` affiche le résultat suivant :

```
Je suis le programme Book_Module_bis

Import du module Book_Module

Voici les informations du livre : The Old Man and the Sea
Auteur : Ernest Hemingway
Année de sortie : 1952
Nombre de pages : 109
Langue : Anglais
```

Ce résultat montre que la variable `__name__` a été associée cette fois-ci à la valeur `Book_Module_bis` et non pas à la valeur `__main__` comme dans l'exécution précédente.

En synthèse, pour distinguer dans nos scripts Python le module principal, qui est celui utilisé avec la commande `python`, des autres modules, il suffit de comparer dans chacun des scripts la variable `__name__` avec la valeur `__main__`. Cela permettra d'éviter de lancer notre application de manière inappropriée.

3. Pour aller plus loin avec Python

Il y a encore tellement de choses à dire sur le langage Python ! Cependant, les concepts vus dans les chapitres Le langage Python, La bibliothèque NumPy et La bibliothèque Pandas et ceux vus dans cette annexe devraient être largement suffisants pour être opérationnel avec ce langage. De manière générale, les notions étudiées dans ce livre par rapport au langage Python devraient permettre de comprendre plus ou moins facilement les autres notions qui n'ont pas été abordées.

Ainsi, pour aller plus loin avec le langage Python, nous vous encourageons vivement à vous documenter sur les quelques points suivants :

- Les Design Patterns avec Python.
- Les bonnes pratiques de programmation Python.
- Les conventions et styles d'écriture des codes Python.
- Comment documenter ses programmes.

Les points évoqués ci-dessus permettent d'avoir une meilleure compréhension du langage Python, mais vont aussi vous permettre d'aborder plus sereinement les frameworks Python dédiés pour le développement de projets spécifiques comme le développement de projets web avec Django ou le développement d'applications asynchrones ou encore la programmation parallèle avec Python.

© Editions ENI - All rights reserved

!

@staticmethod, 724
@classmethod, 722
__getattribute__, 744
__init__, 714

A

ABC, 736
abstractmethod, 736
Accès aux données, 191, 229
    cellule spécifique, 239
    critères textuels, 233
    filtrer les lignes avec plusieurs conditions, 232
    filtrer les lignes avec une condition, 232
    lire plusieurs lignes, 230
    lire une ligne, 230
    parcourir les lignes, 231
    réinitialiser les index, 234
    str.contains, 237
    valeurs d'une colonne, 238
    variables d'un Dataframe, 237
ACP, 538
Algèbre linéaire, 199
Algorithmes, 50
Analyse en composantes principales, 537
Arbre de décision, 428
    profondeur, 428
Attributs
    d'un objet, 717
    de classe, 721

B

Bases de la programmation Python, 77
Boucles, 114
    for, 114
    while, 125
break, 126

C

__call__, 748
callable, 748
Cercle des corrélations, 547
Chargement des données, 226
CLARA, 491
class, 713
Classes, 713
    abstraites, 735
CNN, 611
Coefficient de détermination R2, 396
continue, 127
Contribution
    des individus, 546
    des variables, 549

© Editions ENI - All rights reserved

D

DataFrame, 222, 239
    ajouter une variable, 240
    méthode malt, 245
    modifier les valeurs, 239
    modifier la structure, 240
    réordonner les variables, 243
    supprimer une variable, 244
Deep Learning, 604
def, 129
Descente de gradient, 363
Dictionnaire, 105
Distribution des proportions, 322
Documents, 272
    charger, 276
    créer un dossier, 273
    déplacer un dossier, 275
    navigation, 278
    notebook, 279
    renommer un dossier, 274
    supprimer des éléments, 277
Données
    d'entraînement, 59
    modifier, 196
    nettoyage des données textuelles, 690
    non linéairement séparables, 503
    normalisation des données, 368
    préparation des données, 44
    récolter les données, 36
    sauvegarder, 254
    séparer les données, 37
    tests, 39
    types de données, 42
Dropout, 613

E

Échantillonnage, 312
Élbow, 472
Ensemble, 105
Entraînement du modèle, 40
Environnement Anaconda, 264
Environnement virtuel, 620
Estimation ponctuelle, 328
Expérimentation, 34
Exceptions, 170
    clause finaly, 175
    levée des exceptions, 170
    raise, 178
    try-except, 172
    try-except-finaly-else, 176
    ValueError, 171
    ZeroDivisionError, 171
Expressions régulières, 152
    findall, 160
    groupage des résultats, 163
    re, 156
    re.compile, 160
    re.Match, 166
    re.search, 156
    re.split, 169
    re.sub, 169

© Editions ENI - All rights reserved

F

Facteurs pertinents, 554
Feature engineering, 37
Fichiers
    lecture, 259
Filtrer avec les listes en compréhension, 152
Fonction, 128
    avec paramètres, 131
    avec une liste en compréhension, 150
    d'activation à seuil, 579
    input, 81
    print, 77
    radiale, 579
    récursives, 143
    sans paramètre, 129
    sigmoïde, 411, 579
    zip, 119
Fonction noyau, 507
    gaussienne, 507
    polynomiale, 507
    sigmoïde, 507
Functional API, 654

G

GridSearchCV, 531

H

Héritage, 729
    multiple, 733
Hyperparamètres, 53

I

if-else
    avec les listes en compréhension, 151
Individu
    représentation, 560
    nouveaux, 571
    score, 546
Instanciation, 716
Instructions conditionnelles, 109
    structure if, 110
    structure if-elif-else, 112
    structure if-else, 111
Interfaces, 737
Intervalle de confiance, 332

J

Jupyter, 270

© Editions ENI - All rights reserved

K

Kernel Trick, 505
k-means, 448
    limites, 485
k-medoids, 491
k-modes, 491

L

Lasso, 404
Lecture des fichiers, 259
Lemmatization, 692
Lignes
    ajouter, 250
Liste, 105
Listes en compréhension, 148
load_digits, 595
Loi
    normale, 308
    probabilité, 306

M

Machine Learning, 30
Masques booléens, 208
Matrice des corrélations, 419
Mercer, 507

Méthode
apply, 247
Bootstrap, 58
des moindres carrés, 358
Hold Out, 56
k-fold, 57
Leave One, 58
d'objet, 719
de classe, 722
spéciales, 741
statiques, 724
Métriques de performance, 60
Métriques pour la classification, 65
courbe ROC, 70
matrice de confusion binaire, 65
matrice de confusion générale, 67
pour le clustering, 71
Métriques pour les problèmes de régression, 62
carré des erreurs relatives RSE, 65
coefficient de détermination R2, 65
erreur absolue moyenne, 63
erreur relative absolue RAE, 64
racine carrée de la moyenne du carré des erreurs RMSE, 64
MLPClassifier, 594
Modèle
choisir, 40
entraînement, 40
Modélisation, 31
Modification avec conditions, 249
Modules, 749

© Editions ENI - All rights reserved

N

Neurone
artificiel, 578
biologique, 577
N-Gram, 698
NLP
Voir Traitement automatique du langage
NLTK, 701
Normalisation des données, 368
Notebook, 280
cellules Jupyter, 281
code Python, 281
renommer, 280
Nouveaux individus, 571
NumPy, 182
lire les données à partir d'un fichier, 207

O

Objets, 713
One-vs-All, 426

P

Pandas, 220
installation, 221
Paradoxe de Simpson, 352
Paramètre, 53
C, 525
Gamma, 529
valeurs par défaut, 133

Paramètres de dispersion, 304
écart interquartile, 306
écart-type, 305
formule de Koening, 305
variance, 305
Paramètres de position, 300
médiane, 301
mode, 301
moyenne, 300
quartiles, 304
Passage d'arguments, 140
Portée des variables, 137
Post-élagage, 438
Pré-élagage, 437
Préparation des données, 44
Programmation orientée objet, 711
abstraction, 712
extensibilité, 712
maintenance, 712
réutilisabilité, 712
Projet Data Science, 47
Propagation
arrière, 586
avant, 585
Python, 74
installer l'interpréteur, 74
listes, 213

© Editions ENI - All rights reserved

R

Random Forest, 439
    RandomForestClassifier, 444
Récolter les données, 36
Régression
    linéaire multiple, 361
    linéaire simple, 356
    logistique, 408
    polynomiale, 388
Représentation des individus, 560
Réseau de neurones, 580
Rétropropagation, 583
return, 137
Ridge, 405

S

Scikit-learn, 365
    LinearRegression(), 372
    sklearn.linear_model, 366
    sklearn.metrics, 366
    sklearn.model_selection, 366
    train_test_split, 371
Score des individus, 546
Sélection des variables, 418
Sequential API, 635
__setattr__, 744
Seuil de décision, 422
Silhouette, 479
Statistiques
    descriptives, 205, 300
    inférentielles, 327

Statistiques sur un DataFrame, 255
agrégation avec plusieurs paramètres, 258
résumé direct, 255
résumé par agrégation, 256
Stemming, 692
stopwords, 690
__str__, 743
Structurer un code Python, 106
commentaires, 109
instruction sur plusieurs lignes, 107
les blocs de code, 106
plusieurs instructions sur une ligne, 109
Structures de données, 81
chaînes de caractères, 87
dictionnaires, 99
ensembles, 101
listes, 95
nombres entiers, 82
nombres flottants, 84
tuples, 98
type booléen, 89
variables numériques, 82
Surapprentissage, 52
SVM, 494

T

Tableaux NumPy, 182
//.ndim, 185
//.shape, 185
copier, 197
dtype, 187
initialisation de tableaux NumPy, 189
itemsize, 187

© Editions ENI - All rights reserved

ndim, 185
ones, 189
restructurer, 202
shape, 185
size, 187
tableau à une dimension, 183
tableau à plusieurs dimensions, 183
type, 187
zeros, 189
TensorBoard, 664
TensorFlow, 619
Tensors, 628
Test d'hypothèse, 340
hypothèses H0 et H1, 343
seuil de risque α, 344
tests non paramétriques, 341
tests paramétriques, 341
TF-IDF, 696
Théorème central limite, 326
Traitement automatique du langage, 688
Transfer Learning, 615
Transformation de variables, 416
Tri, 251
avec plusieurs critères, 253
avec un seul critère, 251
Tuple, 105
Types de données, 42
Types de tests d'hypothèse, 345
test d'adéquation, 346
test de conformité, 345
tests d'homogénéité, 347
tests d'indépendance, 348

V

Validation croisée, 55
Variable
catégorielles, 420
de segmentation, 433
portée, 137
sélection, 418
transformation, 416
Vectorisation, 693

W

Widgets Jupyter, 291
DatePicker, 296
Floatslider, 291
image, 296
interact, 294
widgets.Play, 294

© Editions ENI - All rights reserved

INFORMATIQUE
TECHNIQUE

Le Machine Learning avec Python

De la théorie à la pratique

*Préface de **Patrick Albert***
Cofondateur d'ILOG et du HUB France IA

Ce livre sur le **Machine Learning** avec le **langage Python** permet de disposer des **connaissances théoriques** nécessaires pour une compréhension approfondie du Machine Learning et d'appréhender les **outils techniques** utiles pour mettre en pratique les concepts étudiés. L'auteur y expose des **exemples concrets** sur les concepts de l'**apprentissage automatique**. Les lecteurs avertis trouveront dans ce livre une occasion d'aller plus loin dans leur compréhension des algorithmes du Machine Learning.

L'auteur commence par expliquer les **enjeux de la Data Science** ainsi que les **notions fondamentales** du Machine Learning avant de présenter la **démarche théorique d'une expérimentation en Data Science** avec les notions de **modélisation d'un problème** et de **métriques de mesure de performances** d'un modèle.

Le lecteur peut ensuite **passer à la pratique** en manipulant les bibliothèques Python **Numpy** et **Pandas** ainsi que l'environnement **Jupyter**. Il peut ainsi aborder sereinement les chapitres à venir qui lui feront découvrir les **concepts mathématiques**, et la pratique sous-jacente, relatifs aux **algorithmes du Machine Learning** tels que les statistiques pour la Data Science, les **régressions** linéaire, polynomiale ou logistique, les **arbres de décision** et **Random Forest**, l'algorithme **K-means**, les machines à vecteurs de support (**Support Vector Machine**), l'analyse en composantes principales, les **réseaux de neurones** ou encore le **Deep Learning** avec **TensorFlow** et le traitement automatique du langage (**Natural Language Processing**).

Des éléments complémentaires sont en téléchargement sur le site www.editions-eni.fr.

Sur **www.editions-eni.fr** :
➜ Le code source des exemples du livre

Madjid KHICHANE

Après un diplôme d'ingénieur en Informatique obtenu à l'université Mouloud Mammeri à Tizi-Ouzou en Algérie puis un Master en Intelligence Artificielle - Systèmes multi-agents obtenu à l'université Paris 5 (René Descartes), **Madjid KHICHANE** a soutenu son PhD en Intelligence Artificielle en collaboration entre l'Université Claude Bernard Lyon 1 et IBM. Cette thèse doctorale a donné naissance à des innovations algorithmiques dans le domaine de l'apprentissage par renforcement qui sont aujourd'hui publiées dans des conférences internationales de premier niveau. Dans ce livre, c'est toute son expertise sur le Machine Learning avec le langage Python qu'il met au bénéfice du lecteur. Madjid KHICHANE est également l'auteur d'autres parutions aux Editions ENI telles que les ouvrages *Microsoft Bot Framework - Maîtrisez le développement de chatbots avec les services cognitifs d'Azure* et *Data Science avec Microsoft Azure - Maîtrisez le Machine Learning sur Cortana Intelligence Suite* ou la vidéo *Azure Machine Learning Studio - Développez vos modèles de Machine Learning dans Azure*.

Pour plus d'informations :

39 €

ISBN : 978-2-409-03181-6

9 782409 031816

eni

www.editions-eni.fr